사물인터넷（Smart_IoT）
설계시스템

저작권

Copyright © 2015 Huins Corp. All Rights Reserved.

이 문서는 ㈜휴인스 Smart IoT 설계 시스템에 대한 정보 제공의 목적으로만 제공됩니다. ㈜휴인스는 문서에 수록된 정보의 완전성과 정확성을 검증하기 위해 노력하였으나, 발생할 수 있는 내용상의 오류나 누락에 대해서는 신속히 보완하여 알려드리겠습니다.

관련 URL 정보를 포함하여 이 문서에서 언급한 특정 소프트웨어 상품이나 제품은 해당 소유자가 속한 현지 및 국내외 관련법을 따르며, 해당 법률을 준수하지 않음으로 인해 발생하는 결과에 대한 책임은 사용자에게 있습니다.

㈜휴인스에서 문서의 내용을 보완이 필요할 경우 예고 없이 변경할 수 있습니다. 자세한 내용은 아래 사이트를 참조하시기 바랍니다.

㈜휴인스 : http://www.huins.com

문서 정보

문서 개요

이 문서는 Smart IoT 설계 시스템을 사용하는 학습자에게 제공되는 문서입니다. 이 문서는 IoT 설계/실습 시스템을 사용하여 라즈베리파이를 배우고, IoT 제품을 설계 개발할 수 있는 능력을 키웁니다.

독자

이 문서는 IoT 설계/실습 시스템을 사용하는 학습자를 대상으로 합니다. 이 문서에서 IoT 에 관한 상세한 설명을 포함하고 있으며, 센서로부터 정보수집, 액츄에이터 제어, 안드로이드 앱 프로그램 등을 배울 수 있습니다. 프로그래밍 언어인 C, C++, Java, Python 에 대해서는 관련 서적을 참조하시기 바랍니다.

문의처

이 문서의 내용에 오류가 있거나 내용과 관련한 의문 사항이 있으면 아래의 연락처로 문의합니다.

연락처: 김미선 kms@huins.com 031-719-8200

문서 버전 및 이력

버전	일자	이력사항
1.0	2015.04.06	1.0 작성/배포
1.1	2015.07.07	1.1 작성/배포
1.2	2015.07.29	개발툴 Android Studio 추가
1.3	2016.01.08	wiringPi API 설명 추가

표기 규칙

참고 표기



참고

독자가 참고해야 할 내용을 기술합니다.

주의 표기



주의

독자가 반드시 알아야 할 사항, 시스템 에러를 유발할 수 있는 사항, 수행하지 않았을 때 재산상의 피해를 줄 수 있는 사항을 기술합니다.

실습 시스템 이름, 구성품, 센서 이름, 액츄에이터 이름, 사용자 입력 값의 표기에 대해

이 문서에서 실습시스템, 구성품, 센서 이름, 액츄에이터 이름, 사용자 입력 값 등은 아래와 같이 표기합니다.

- 시스템 이름 : Smart IoT 실습시스템

• 구성품 이름 : Gateway, 센서, 액츄에이터, IoT IO Adapter, 케이블, SD 카드, 2Pin 연결선, 3Pin 연결선
• 센서, 액츄에이터 : 각각의 센서와 액츄에이터 이름.
• 사용자 입력 값: 실습 시에 사용자가 입력해야 하는 값을 말함.

소스 코드 표기

이 문서에서 소스 코드는 회색 바탕에 검정색 글씨로 표기합니다.

```
if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
{
        fprintf(stdout, "wiringPiSPISetup Failed :%s₩n", strerror(errno));
        return 1;
}

pinMode(CS_MCP3208, OUTPUT);
CODE Code = “We are Here, Our Planet”;
```

목차

Chapter 01. IoT 개요 16

1.1 IoT 란? 16

1.2 IoT 발전과정 19

1.3 IoT 기술동향 20

(1) IoT(사물인터넷) 통신 프로토콜 20

(2) 국내 주요 서비스 사례 22

(3) 스마트폰과 하드웨어의 연결 사례 25

1.4 Reference 29

Chapter 02. IoT 기반기술 30

2.1 IoT 기반기술 개요 30

2.2 센서 31

(1) 센서시대의 출현 31

(2) 인간의 오감과 센서 33

(3) 센서 분류 34

2.3 영상처리(카메라) 35

2.4 네트워크 인프라 39

(1) 사물 인터넷 시대의 네트워크 인프라 39

(2) 네트워크 인프라 경쟁의 배경 40

(3) 주요 네트워크 인프라 기술 42

2.5 IoT 통신기술 45
2.6 서비스 인터페이스 기술 50
2.7 OS 와 소프트웨어 50
2.8 IoT 보안 51
(1) IoT 보안 개요 51
(2) 사물 인터넷이 보안에 있어 취약한 이유 52
(3) 사물 인터넷 보안취약성 사례 53
(4) IoT 보안을 침해하는 방법들 55
(5) IoT 보안 방안 57
(6)Secure Coding 57
(7)암호화 알고리즘의 종류 58
2.9 Reference 59

Chapter 03. IoT 센서와 액츄에이터 60
3.1 IoT 스마트센서 개념 60
3.2 센서의 기초 62
3.3 센서의 기본적인 특성과 용어 설명 63
3.4 융복합 센서 개발 64
(1) 융복합센서 개요 64
(2) 복합센서 발전방향 65
3.5 Actuator 68

Chapter 04. Raspberry Pi 70
4.1 Raspberry Pi 란 70
4.2 Raspberry Pi Quick Start 74
(1) SD 카드에 운영체제 이미지 쓰기 74
(2) SD 카드 이미지 백업 77
(3) 주변기기연결 78
(4) 부팅 및 Raspberry Pi 설정 80
4.3 Raspberry Pi GPIO 87

(1) wiringPi 설치 87
(2) wiringPi API 종류와 사용법 90
4.4 라즈베리 파이 유/무선 랜 설정 106
4.5 SSH 접속(원격접속) 108
4.6 삼바접속 111

Chapter 05 IoT 카메라 연동 117
5.1 라즈베리 파이 카메라 모듈 117
(1) Raspberry pi camera 설명 117
(2) 카메라 모듈 Software 118
5.2 카메라 영상 제어(C, Java) 118
5.3 동영상 스트리밍 재생 122
5.4 타임랩스 사진 촬영법 1 129
5.4 타임랩스 사진 촬영법 2 131

Chapter 06 스마트폰 프로그램 135
6.1 스마트폰 통신기술 135
6.2 개발툴설치 142
6.2.1 자바 설치 (JRE) 142
6.2.2. Android Studio 설치 155
6.2.3안드로이드 스튜디오 앱 개발 설정과 처음 앱 구현 164

Chapter 07 Apache, PHP, MySQL 서버구축 175

Chapter 08 데이터베이스 실습 183
8.1 데이터베이스 생성 184
8.2 데이터 베이스 실습 188

Chapter 09. Smart IoT 설계 시스템 소개 197

9.1 Smart IoT 설계 시스템의 구성 197
(1) Smart IoT 설계 시스템 개요 197
(2) Smart IoT 설계 시스템의 주요특징 199
(3) 요소 기술 200
(4) Smart IoT 5종 제품 구성 203
9.2 Smart IoT 설계시스템 소프트웨어 207
9.3 Smart IoT 하드웨어 209
(1) 라즈베리파이(model B+) 209
(2) IO Adapter 보드 212
(3) 센서(Sensor) 216
(4) 액츄에이터(Actuator) 218

Chapter 10 IoT 스마트 농장 실습 221

10.1 Iot 스마트 농장관리 시스템 개요 221
(1) 스마트 농장관리 시스템 하드웨어 224
(2) 스마트 농장관리 시스템 소프트웨어 225
10.2 스마트 농장관리 시스템의 연결 구성 227
10.3 스마트 농장관리 시스템 센서 실습 230
(1) 온도 습도센서 테스트 231
(2) 조도센서 테스트 243
(3) 다른 센서 테스트 256
10.4 스마트 농장관리 시스템 액츄에이터 실습 257
(1) 워터펌프 261
(2) Fan 265
(3) DC Motor 269
(4) RGB Led 274
(5) 스마트 농장 통합 액츄에이터 실습 283
(6) 다른 액츄에이터 테스트 288
10.5 스마트 농장관리 시스템 시나리오 실습 289
10.7 웹(브라우저)에서 데이터베이스 모니터링 306
10.7.1 모바일 웹과 모바일 앱의 구분 306

10.7.2. 모바일 웹과 앱 차이점 306
10.7.3. 모바일 웹과 앱의 장단점 308
10.7.4 실습 309
10.8 웹(브라우저)에서 스마트 농장 액츄에이터 제어 325
10.9 스마트 폰으로 농장 액츄에이터 제어하기 330
(1) 하드웨어 연결도 331
(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍 331
(3) 라즈베리 파이TCP/IP 서버 프로그래밍 342
10.10 스마트농장 시스템 서버 프로그램 357
(1) 데이터 베이스 설계 358
(2) 서버 실행과 카메라 영상 360
10.11 스마트 농장 안드로이드 앱 프로그램 362

Chapter 11 IoT 무인경비 시스템 실습 375
11.1 무인경비 시스템 시스템 개요 375
(1) 무인경비 시스템 하드웨어 376
(2) 무인경비 시스템 소프트웨어 377
11.2 무인경비 시스템 연결 구성 379
11.3 무인경비 시스템 센서 실습 381
(1) 사운드 센서(음향센서) 382
(2) 인체 감지센서 394
(3) 초음파 센서 400
11.4 무인경비 시스템 액츄에이터 실습 405
(1) LED Bar 실습 409
(2) DC Motor 실습 416
(3) Buzzer(부저) 실습 419
(4) RGB Led 실습 424
(5) 액츄에이터 전체 실습 424
11.5 무인경비 시스템 시나리오 실습 427
11.6 스마트 폰으로 액츄에이터 제어하기 428
(1) 하드웨어 연결도 428

(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍 428
(3) 라즈베리 파이TCP/IP 서버 프로그래밍 440
11.7 무인경비 시스템 서버 프로그램 454
(1) 데이터베이스 설계 455
(2) 서버실행과 카메라 영상 458
11.8 무인경비 시스템 안드로이드 앱 프로그램 461

Chapter 12 IoT 화재경보 시스템 실습 471
12.1 화재경보 시스템 개요 471
(1) 화재경보 시스템 하드웨어 472
(2) 화재경보 시스템 소프트웨어 473
12.2 화재경보 시스템 연결 474
12.3 화재경보 시스템 센서 실습 475
(1) 불꽃 센서(Flame sensor) 475
(2) 가스센서 481
(3) 온습도 센서 487
12.4 화재경보 시스템 액츄에이터 실습 488
(1) 워터펌프 실습 489
(2) DC Motor 실습 489
(3) 부저 실습 489
(4) RGB Led실습 490
(5) 액츄에이터 일괄 실습 490
12.5 화재경보 시스템 시나리오 실습 495
12.6 화재경보 시스템 서버 프로그램과 카메라 구동 497
(1) 데이터 베이스 설계 497
(2) 서버실행과 카메라 영상 498
12.7 화재경보 시스템 안드로이드 앱 프로그램 503

Chapter 13 IoT 애완동물 도우미 실습 513
13.1 애완동물 도우미 실습시스템 개요 513

(1) 애완동물 도우미 시스템 하드웨어 514
(2) 애완농물 도우미 시스템의 소프트웨어 515
13.2 애완동물 도우미 시스템 연결 구성 517
13.3 애완농물 도우미 센서 실습 520
(1) 인체감지 센서 521
(2) 사운드센서 523
13.4 애완동물 도우미 액츄에이터 실습 526
(1) 서보모터 실습 527
(2) DC 모터 실습 532
(3) LED Bar 실습 532
(4) RGB LED 실습 535
(5) 통합 액츄에이터 실습 535
13.5 애완동물 도우미 시나리오 실습 540
13.6 스마트폰으로 액츄에이터 제어하기 542
(1) 하드웨어 연결도 542
(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍 542
(3) 라즈베리 파이TCP/IP 서버 프로그래밍 554
13.7 안드로이드 앱 프로그램 568

Chapter 14 IoT 인터넷제어 토이 실습 581

14.1 인터넷제어 Toy 개요 581
(1) 인터넷 제어 토이 하드웨어 581
(2) 인터넷 제어토이 시스템 소프트웨어 583
14.2 인터넷 제어토이의 연결 구성 584
14.3 인터넷 제어토이 센서 실습 588
(1) 인체 감지센서 588
(2) 사운드 센서 593
(3) 초음파 센서 593
(4) 근접센서 599
14.4 제어토이 액츄에이터 실습 603
(1) 워터펌프 604

(2) Fan ____ 604
(3) DC Motor ____ 604
(4) RGB Led ____ 605
(5) 인터넷 제어토이 시나리오 실습 ____ 605
14.6 스마트 폰으로 인터넷 토이 액츄에이터 제어하기 ____ 621
(1) 하드웨어 연결도 ____ 621
(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍 ____ 621
(3) 라즈베리 파이TCP/IP 서버 프로그래밍 ____ 633
14.7 인터넷 토이 서버 프로그램 ____ 647
(1) 데이터 베이스 설계 ____ 648
(2) 서버 실행과 카메라 영상 ____ 649
14.8 인터넷 토이 안드로이드 앱 프로그램 ____ 651

Chapter 15 참고자료 ____ 661

Chapter 01. IoT 개요

이 장에서는 IoT(Internet of Things, 사물 인터넷)의 개념을 이해하고, 발전과정을 살펴본다. 또한 사물 인터넷이 적용된 사례들을 살펴보고 향후 IoT 의 미래를 예상해 본다.

1.1 IoT 란?

최근 사물 인터넷(Internet of Things)에 대한 관심이 증가하고 있다. 사물 인터넷이란 주위의 사물 하나하나가 인터넷과 연결된다는 의미다. 다양한 물건이나 제품뿐만 아니라 동식물의 위치까지도 인터넷에 연결된다. 아주 간단하지만 이 변화가 인간과 기업 그리고 세상에 던지는 영향력은 간단치 않다.

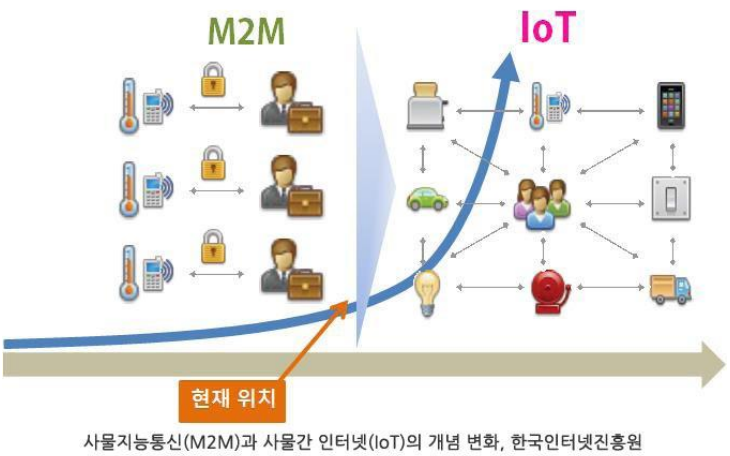


그림 1 IoT 서비스 개념

모든 사물이 인터넷에 연결되어 정보를 공유하는 사물 인터넷(IoT, Internet of Things)에 대하여 글로벌 표준기구(ITU, 3GPP, IEEE, ETSI)들은 다양하게 정의하고 있다. 특히 ITU(International Telecommunication Union, 국제 전기통신연합)는 “IoT 를 기기 및 사물에 통신 모듈이 탑재되어, 유무선 네트워크로 연결됨으로써 사람과 사물 간, 사물과 사물 간에 정보 교환 및 상호 소통할 수 있는 지능적 환경”으로 해석하고 있다.

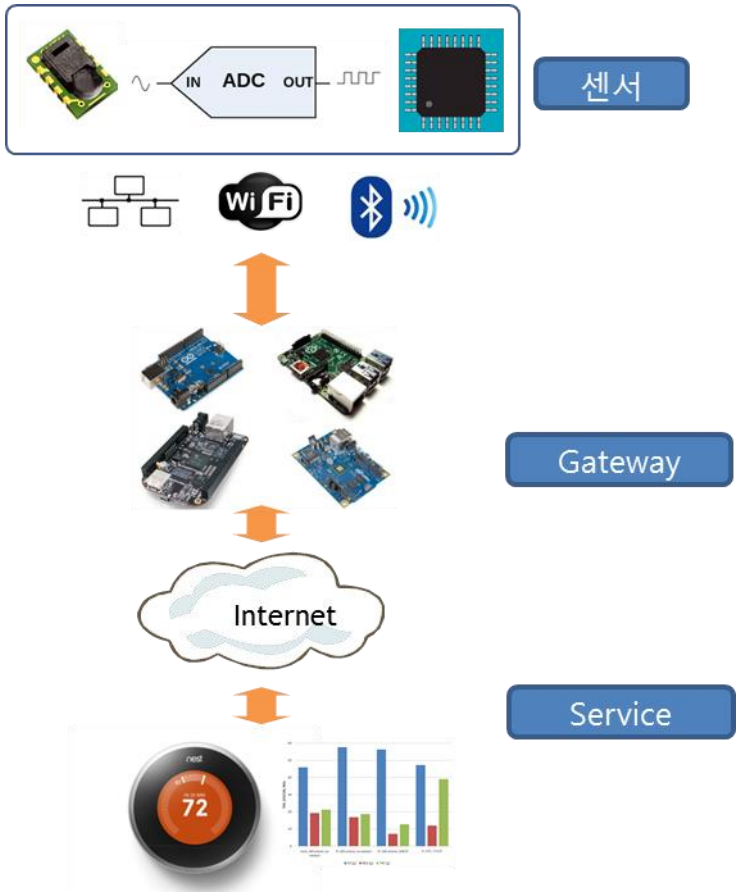


그림 2 IoT(사물인터넷) 개념도

또한 다양한 ICT(Information & Communication Technology) 기반 기술의 발달로 사물이 소형화 및 스마트화 되면서 사물 인터넷 시대가 도래할 것으로 전망하고 있다. 최근 IoT 의 활용을 통해 즉각적인 범죄감시가 가능한 DAS(Domain Awareness System) 시스템, 상황을 감지하여 스스로 주행하는 구글카(Google Car), 실내의 온도를 어디서나 조절하여 에너지를 절감할 수 있는 온도조절기(Thermostat) 등 사용자에게 새로운 가치를 제공하거나, 기존에 제공하던 서비스가 확장되고 있다.

사물인터넷을 자세히 표현하면 발전된 IT 를 기반으로 세상의 모든 사물을 인터넷으로 연결해 사람과 사물, 사물과 사물 간 정보를 교환하고 상호 소통하는 인프라를 사물인터넷이라 부른다. 사물은 인간, 차량, 교량, 각종 전자기기,

자전거, 안경, 시계, 의류, 문화재, 동식물 등 자연 환경을 이루는 모든 물리적 객체를 의미한다. 모든 사물의 정보를 수집하고 상호 교류하는 환경인 사물 인터넷은 1 세대 유선 인터넷, 2 세대모바일 인터넷을 넘어 디지털 발전의 3 단계인 3 세대 인터넷으로 불리고 있다. 그 동안 1, 2 세대 인터넷의 경우 정보를 쓰고 읽고 활용하는 주체가 사람이었다면, 3 세대 인터넷인 사물 인터넷은 정보를 생성하고 읽고, 활용하는 주체로 사물이 등장했다는 점에서 커다란 의미를 갖는다. IoT(사물 인터넷)의 개념을 간략하게 나타낸 [그림 2]를 보면 주위 환경을 센싱하고 유선이나 무선을 사용해 게이트웨이에 전송하여 사용자에게 서비스를 제공하는 것이 바로 사물인터넷의 개념이다.

글로벌 기업들이 생각하는 IoT 개념을 알아보자. (출처 : Bob Violino | Network World)

▪ Cisco : 사물 인터넷이란 스마트 디바이스를 지능적으로 연결하는 것이다. 이는 효율성, 비즈니스 성장, 삶의 질을 높이는데 크게 기여할 것으로 기대된다. 사물이 서로를 인지해 커뮤니케이션을 하면 의사결정 방법, 장소, 주체가 바뀐다.

▪ HP : 사물 인터넷은 사물을 물리적 매개체나 무선 매개체를 통해 인터넷에 연결하면, 스스로와 다른 디바이스, 사람, 그 능력을 식별할 수 있다는 ICT 개념이다.

▪ IBM : 사물 인터넷을 통한 '스마터 플래닛(더 똑똑한 지구)'이라는 비전을 갖고 있다. 우리는 세상이 점차 장치화(Instrumented), 상호연결화 (Interconnected), 지능화(Intelligent) 되면서 발생하는 가장 큰 도전(문제점) 중 일부를 극복할 수 있게 될 것이다.

▪ **인텔** : 사물 인터넷은 단순한 보수계(보행 횟수를 측정하는 기기)에서 복잡한 CT 스캐너에 이르기까지 인터넷으로 연결된 다양한 디바이스로 구성되어 있다. 이들 지능형 디바이스가 서로 커뮤니케이션을 하고, 클라우드와 더불어 데이터를 공유하면서 비즈니스에 변화를 가져올 정보와 실제적인 통찰력을 발굴할 때, 사물 인터넷의 진짜 가치가 실현된다.

▪ **마이크로소프트** : 마이크로소프트의 관점은 단순하다. 사물 인터넷은 미래적인 기술 트렌드가 아니라는 것이다. 현대의 강력한 디바이스, 센서, 클라우드, 인프라스트럭처, 기타 비즈니스 지능 툴에 존재하는 기술이다. 진정한 가치는 데이터에 있다. 사물이 연결되어 기업이 활용할 수 있는 많은 정보를 생산하기 때문이다. 우리는 이를 IoYT(Internet of Your Things)로 부르고 있다.

1.2 IoT 발전과정

초기에 Internet of Objects 라고도 불렸던 IoT(Internet of Things)는 우리를 포함한 모든 것을 바꿀 것이다. 인터넷이 교육, 커뮤니케이션, 비즈니스, 과학, 정부 및 인류 전체에 미친 영향을 생각해 보면 그리 지나친 말은 아니다. 확실히 인터넷은 인류 역사를 통틀어서 가장 중요한 발명품중의 하나다.

수 많은 IoT 프로젝트가 이미 진행되고 있지만 IPv6 로의 전환, 국제 표준화의 설정, 수 백 만개에서 수 십 억 개에 달하는 센서를 위한 에너지 문제 등은 남겨진 많은 도전과제들이다. 사물 인터넷은 최근에 갑자기 등장한 개념이 아니라 아주 오래 전부터 존재해 왔다. 그 동안 많은 이름으로 설명되었기 때문에 우리는 낯설게 받아들일 뿐이다. 센서 네트워크, M2M(Machine-to-Machine), 사물지능통신, 유비쿼터스(Ubiquitous), RFID/USN(Radio Frequency IDentification/Ubiquitous Sensor Network) 등 이 바로 그 대표적인 이름들이다. 이 가운데서도 M2M 은 사물 인터넷과 가장 유사한 개념으로 이해할 수 있다.

M2M 은 B2B(Business To Business), B2C(Business To Consumer)라는 용어들이 등장했던 십여 년 전부터 유럽을 중심으로 확산되고 있었다. M2M 은 사람이 직접 제어하지 않는 상태에서 장비나 사물 또는 지능화된 기기들이 사람을 대신해 통신의 양쪽 모두를 맡고 있는 기술을 의미한다. 또한 센서 등을 통해 데이터의 전달, 수집, 가공된 위치, 시각, 날씨 등의 데이터를 다른 장비나 기기 등에 전달하기 위한 통신을 포함하고 있다.

M2M 과 유사한 개념으로는 RFID/USN 이 있다. RFID/USN 은 사물간 통신을 한다는 점에서는 공통된 점이 있어 경쟁 기술인 듯 보이지만 주파수나 시장 측면에서 다른 영역을 확보, 성장했다. M2M 은 일반적으로 사람이 접근하기 힘든 지역의 원격 제어나 위험품목의 상시 검시 등의 영역에서 적용된 반면, RFID 는 홈 네트워킹이나 물류, 유통 분야에 적용되다가 NFC(Near Field Communication)로 진화해 모바일 결제 부문으로 영역을 확장했다.

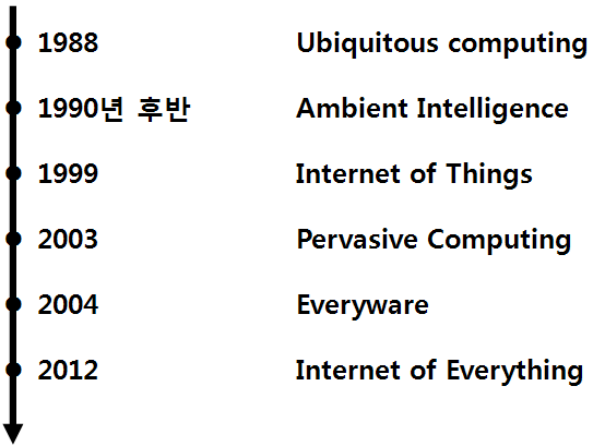


그림 3 유비쿼터스의 발전(출처 https://iot24.wordpress.com/)

M2M 과 사물 인터넷은 사물간 통신을 한다는 점에서 같아 많이 혼용해서 쓰이기도 한다. 하지만 M2M 이 통신 주체인 사물을 중심으로 한 개념인데 비해 사물 인터넷은 인간을 둘러싼 환경에 초점을 맞췄다는 점에서 차이를 보인다. 사물 인터넷이 인간을 중심으로 바라본다는 점에서는 유비쿼터스와 유사하다.

유비쿼터스는 사용자가 네트워크나 컴퓨터를 의식하지 않고 장소, 시간에 상관없이 자유롭게 네트워크에 접속할 수 있는 환경을 의미하는 것으로, 사물 인터넷이 정의하는 세상보다 좀더 먼 미래를 보는, 사물 인터넷 너머에 있는 만물 인터넷(Internet of Everything) 시대와 유사하다. 그리고 사물지능통신은 M2M 을 한글로 전환하는 과정에서 이름을 공모해서 탄생한 국내에서만 사용되는 용어다.

1.3 IoT 기술동향

(1) IoT(사물인터넷) 통신 프로토콜

사물 인터넷(IoT: Internet of Things)은 지능화된 사물들이 인터넷에 연결되어 네트워크를 통해 사람과 사물 (물리 또는 가상), 사물과 사물간에 상호 소통하고 상황인식 기반의 지식이 결합되어 지능적인 서비스를 제공하는 글로벌 인프라이며, 스마트 디바이스, 클라우드, 빅데이터 기술 등과 융합하여 개방과

공유를 지향하는 초연결사회의 핵심이 될 전망이다. 2020 년에 인터넷에 연결되는 사물의 수는 약 260 억 개까지 증가할 것으로 전망되며(PC, 태블릿, 스마트폰 제외), 약 3,000 억 달러의 시장(서비스, 제품) 창출과 1.9 조 달러의 경제적 파급효과가 기대되며 제조와 헬스케어 분야가 가장 크다.

IoT 기술은 디바이스, 네트워크, 플랫폼, 분석/소셜 비즈니스 및 응용산업 서비스로 구분되며, 사물에 부착 또는 내장되는 IoT 디바이스는 IoT 구현을 위해 서비스 환경과 사물의 형태에 따라 매우 다양한 요구 성능은 물론 향후 10 년간 가격 하락, 전원공급 문제의 해결, 더 넓은 연결 등 기술의 성숙이 요구된다.

최근에는 사물 인터넷 서비스를 보다 편리하게 구현하고 관리할 수 있는 기술들이 개발되고 있으며, 사물인터넷 정보 전송, 메시지 처리, 통신 프로토콜 등에 대한 기술의 중요성이 부각되고 있다. 몇 가지 메시지 전송 프로토콜을 살펴보기로 한다.

▪ REST(REPRESENTATIONAL STATE TRANSFER)

REST(Representational State Transfer)는 사물인터넷을 구성하는 기기들의 상태 전송이란 뜻을 가지고 있는데, 인터넷의 정보를 조직하고 전송하는 규칙의 조합을 의미한다. REST 는 리소스라는 이름으로 인터넷상의 문서, 이미지, 서비스와 같은 정보를 지칭하고 클라이언트 서버의 네트워크 환경에서 리소스의 CRUD(Create, Read, Update, Delete) 처리를 지원한다.

MQTT(MESSAGE QUEUING TELEMETRY TRANSPORT)

MQTT(Message Queuing Telemetry Transport)는 제한된 컴퓨팅 성능과 빈약한 네트워크 연결 환경에서의 동작을 고려하여 설계된 대용량 메시지 전달 프로토콜로서 IBM 에 의해 개발되고 OASIS(Organization for the Advancement of Structured Information Standards)에 의해 사물인터넷 표준 프로토콜로서 선정된 기술이다. 현재 Facebook 에서 사용되는 기술이다. 특히 페이스북이나 구글의 서비스가 MQTT 프로토콜을 이용하여 구현되었고 카카오톡이나 밴드와 같은 에어 푸시 서비스에도 이용된다. MQTT 는 발행과 구독(publish/subscribe) 방식의 메시지 큐(message queue)로, 원격 검침 (telemetry)영역에 사용하기 위해 개발되었다. 원격 검침기는 대부분 소형이며 통신 대역폭과 전원이 한정적인 환경에서 동작한다. 이는 배터리 용량이 제한적이고 통신 품질을 일정 수준으로 유지하기 어렵다는 점에서 스마트폰 환경과 매우 유사하다.

MQTT 개발 시 고려한 원격 검침 환경에서는 온도, 경보 등의 검침 정보가 주기적으로 발생된다. 예를 들어 10 분 간격으로 온도를 검침하여 발행하는 검침 환경을 가정해 보자. 오류로 인해 10 분 전 정보와 현재 시점의 검침 정보가 동시에 브로커에게 전달된다면 일반적인 검침 환경에서 최신 정보가 우선한다는 가정 하에 양쪽 정보를 모두 구독자에게 전달하는 것보다는 현재 정보만을 전달하는 것이 효율적일 것이다.

XMPP(EXTENSIBLE MESSAGING AND PRESENCE PROTOCOL)

XMPP(eXtensible Messaging and Presence Protocol)은 인스턴트 메신저를 위한 IETF 에서 제정한 국제 표준 프로토콜로서 다수의 클라이언트 간에 Publish/Subscribe 구조를 바탕으로 확장성 있는 XML 기반 실시간 메시지 교환이 가능한 프로토콜이다. XMPP 는 클라이언트와 서버, 두 서버간의 통신을 지원하는 프로토콜을 정의한다. 현재 Google, MSN, Yahoo 메신저에서 사용되는 XML 스트리밍 기반 확장형 실시간 메시징 기술이다.

XMPP 는 공개형 프로토콜인 JABBER(http://www.jabber.org) 인스턴트 메신저에 근간을 두고 있다. Jabber 프로토콜은 확장성을 고려하여 초기부터 XML 이라는 구조적 기술언어를 이용하여 프로토콜이 설계되어 있다. XML 의 특성상 풍부한 확장성을 제공하기에 Jabber 응용프로그램을 개발하는 개발자는 자신이 새로 설계한 구조적인 정보를 Jabber 네트워크를 통하여 쉽게 전송할 수 있다. 결과적으로 XML 에 의해 어떠한 구조적인 정보를 정의하느냐가 Jabber 어플리케이션 개발의 핵심이 되는 것이다.

COAP(CONSTRAINED ENVIRONMENTS APPLICATION PROTOCOL)

CoAP(Constrained Environments Application Protocol)은 인터넷에서 센서 노드와 같이 제한된 컴퓨팅 성능을 갖는 디바이스들의 통신을 실현하기 위해 IETF 의 CoRE(Constrained RESTful Environments) 워킹 그룹에서 만들고 있는 응용계층 표준 프로토콜이다. 웹 서비스를 구현함에 있어서 제약이 많은 환경에서 TCP, HTTP 와 같은 무거운 통신 프로토콜을 사용할 수 없어 웹 서비스를 할 수 있는 가벼운 프로토콜을 목적으로 설계된 기술이다.

(2) 국내 주요 서비스 사례

SKT 의 스마트 팜 서비스

SK 텔레콤은 제주도 서귀포와 경북 성주지역에 비닐하우스 내부의 온도와 습도, 급수와 배수, 사료공급 등까지 원격 제어 지능형 비닐하우스 관리 시스템인 스마트 팜 서비스를 제공하고 있다.

SK 텔레콤이 개발한 지능형 비닐하우스 관리 시스템인 스마트 팜을 이용해 온도와 습도 등을 체크하고 있다. 스마트 팜에서는 스마트폰을 이용해 비닐하우스 내부의 온도와 습도, 급수와 배수, 사료공급 등까지 원격 제어할 수 있다.

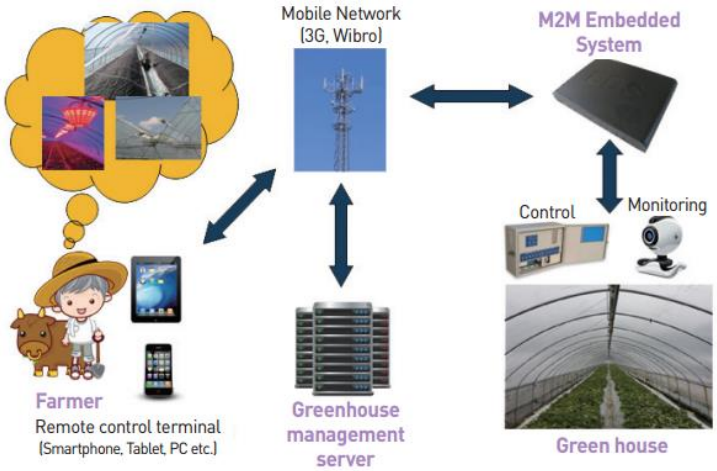


그림 4 스마트 팜 서비스 구성(KCA 한국방송통신전파진흥원 정책연구본부)

KT 의 스마트 홈 서비스

KT 는 스마트 폰을 활용한 댁내 방범, 전력제어, 검침 등의 다양한 사물인터넷 서비스를 제공하고 있다. 원격지에서 거주자가 스마트 폰으로 KT 의 사물인터넷 플랫폼을 통해 실시간으로 댁내 환경을 모니터링 할 수 있으며, 간단한 스마트 폰 작동을 통해 전등, 출입문 등을 제어할 수 있다. 또한, 실시간 침입 및 화재 경보를 수신할 수 있으므로 스마트 원격 관제 서비스가 가능하다.

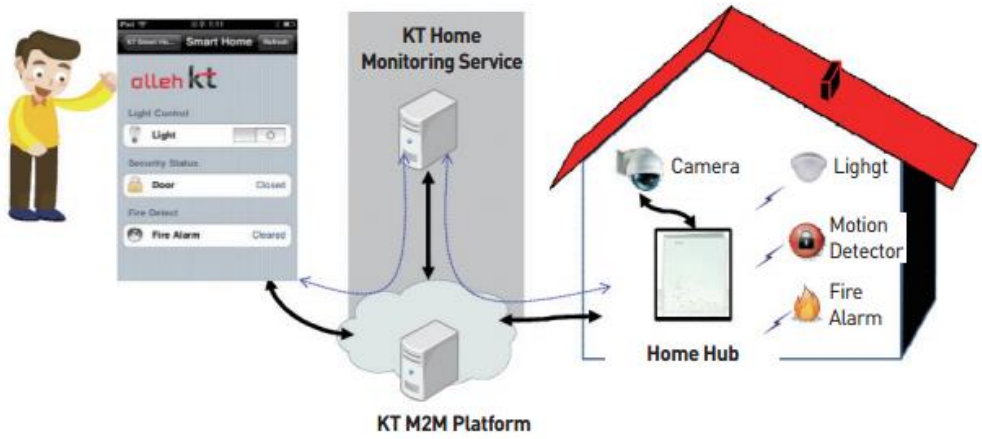


그림 5 스마트 홈 서비스(KCA 한국방송통신전파진흥원 정책연구본부)

LG U+의 지능형 차량 관제 서비스

LG U+는 DTG(Digital Tacho Graph)와 사물 인터넷 플랫폼과의 연동을 통하여 실시간 차량 관제 서비스를 화물차량, 버스, 택시 등을 대상으로 제공하고 있다. 또한, 2012 여수 세계박람회 기간 동안 LTE 기반의 사물인터넷 솔루션을 적용한 차량관제 시스템을 운영하여 승무원, 승객관리, 운행상태와 속도, 이동거리 등의 차량 정보를 실시간으로 교통관제 센터에 전송하는 서비스를 제공하였다.

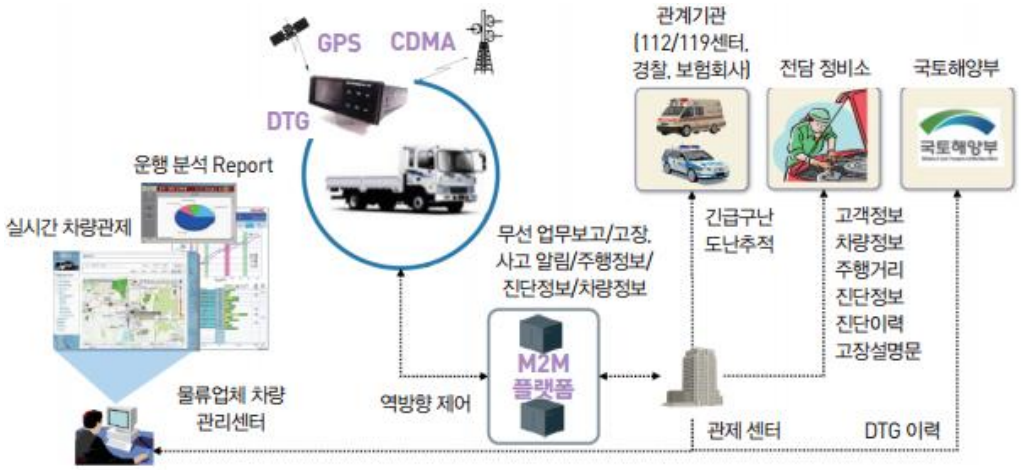


그림 6 지능형 차량 관제 서비스(KCA 한국방송통신전파진흥원 정책연구본부)

NURITELCOM 의 스마트 그리드 서비스

국내 사물 인터넷 중소기업인 누리텔레콤은 근거리 무선통신 기술인 ZigBee 기반의 Mesh 무선통신망을 이용하여 스웨덴 예테보리시에 26 만 5 천 가구를 대상으로 스마트 시티를 구축하였으며, 원격검침 서비스를 제공하고 있다. 또한, 2015 년까지 가나의 가정 등 10 만 저압 수용가에 AMI(Advanced Metering Infrastructure)를 공급한다. 1 차로 '13 년 7 월까지 가나 프람프람와 아킴오다시 지역 주택 1 만호에 AMI 를 구축한다.

Mesh 네트워크와 GPRS(General Packet Radio Service) 무선 통신 기반원격검침모뎀이 탑재된 스마트 계량기, 데이터 수집 장치(DCU, Data Concentration Unit), 플랫폼 등 누리텔레콤의 토털 AMI 솔루션이 공급된다.

(3) 스마트폰과 하드웨어의 연결 사례

최근 스마트폰은 디바이스 성능뿐만 아니라 다양한 하드웨어와의 연결에 초점을 맞춰 발전하고 있다. 실제로 우리 주변에서 스마트폰과 연결된 장난감, 온도계, 저울, 현미경 등에서 이런 현상을 볼 수 있다. 장치들이 어떤 방식으로 스마트폰과 연결되고, 장치들과 아이폰 또는 안드로이드폰을 연결하는 프로그램은 어떻게 작동하는지 설명한다.

이를 위해 스마트폰과 연결된 장치의 기술적인 접근과 방법들을 살펴보고 아이폰과 연결된 장치의 하드웨어 및 관련 프로그램을 살펴보자. 연결 방식은 블루투스와 와이파이를 이용하는 것을 위주로 설명한다. 물론 안드로이드폰과 연결된 장치의 하드웨어 및 관련 프로그램도 살펴본다. 특히 BLE(Bluetooth Low Energy) 및 BT 연결 방법을 알아본다. 그리고 스마트폰을 이용한 IoT(Internet Of Things) 구현 및 사용에 대해 설명한다. 자세한 프로그래밍 방법은 '안드로이드 프로그램' 장에서 배우기로 한다.

스마트폰과 연결되는 장치들에는 무선 조종 장난감, 운동량 측정 장치, 원격 조정 장치 등 다양하다. 이들을 통칭 앱세서리(Appcessory)라고 부른다. 앱을 통해 조정하는 장치 혹은 부가 장치라는 의미를 가지고 있다. 이들 대부분은 스마트폰에 내장된 블루투스나 와이파이 등의 통신방법을 사용해 연결하지만, 마이크 또는 스마트폰과 PC 와의 동기화 과정에서 사용하는 커넥터를 통한

연결방법도 사용한다. 지금부터 스마트폰과 연결된 대표적인 장치들의 사례를 알아보자.

(출처 : 마이크로소프트웨어 2013 년 9 월)

▪ 로버

와우위의 로버(Rover)는 스마트폰으로 원격 제어해 로봇에서 촬영된 영상을 보고 제어할 수 있는 장난감이다. 와이파이를 사용해 실시간으로 영상과 음성을 전달할 수 있다.



그림 7 로버 http://www.brookstone.com/

▪ 스페로 로봇



그림 8 스페로 로봇 http://www.gosphero.com/

공 모양의 스페로 로봇(Sphero Robot)은 블루투스를 사용해 움직이는 장난감이다. 특히 SDK 를 배포하고 있어 스패로 로봇을 활용한 다양한 앱을 개발할 수 있다. 그런 점에서 앱에 연결하는 장치와 앱 개발 툴이 제공되는 스패로 로봇은 진정 한 의미의 스마트폰용 연결 장치라고 할 수 있다. 그럼에도 불구하고 아직까지는 개인용 로봇 형태라는 한계 때문에 할 수 있는 게 그렇게 많지 않다는 게 아쉽다.

▪ 락키트론

락키트론(Lockitron)은 문에 부착하는 인터넷 자물쇠다. 블루투스, 충격감지 센서, 와이파이 등을 내장하고 있어 스마트폰 앱을 이용해 문을 여닫거나 또는 스마트폰을 문 앞에 가져다 대면 자동으로 열리게 할 수도 있다. 그리고 방문자가 노크하면 락키트론이 이를 인지해 그 사실을 스마트폰으로 알려주는 기능까지 제공한다.



그림 9 락키트론 https://lockitron.com/

▪ 위모

컴퓨터 액세서리 전문 기업 벨킨이 출시한 위모(WeMo)는 가전기기 전원을 스마트폰 앱을 이용해 쉽게 제어할 수 있게 해준다. 위모 스위치를 전기 콘센트에 설치한 후 가전기기의 전원을 꽂으면 앱을 통해 가전기기의 전원을 켜고 끌 수 있다. 특히 위모 모션은 인터넷과 연결돼 있어 집 밖에서 스마트폰 앱으로 집 안 가전기기의 전원 상태를 확인 가능하다.



그림 10 위모 http://www.belkin.com/

▪ 트와인

작은 센서를 이용해 다양한 방법으로 자유롭게 사물 인터넷을 구성할 수 있는 제품도 있다. 직사각형 모양으로 생긴 트와인(Twine)에는 온도 센서와 가속도 센서 등 5 개의 센서가 내장돼 있다. 물론 외장 센서를 추가로 장착할 수 있다. 트와인에는 와이파이 모뎀이 탑재돼 있어 센서에서 감지한 데이터를 PC 나 스마트폰 등에서 확인 가능하다. 특히 다양한 방식으로 구성할 수 있는 스크립트 프로그램을 제공한다. 이 프로그램을 이용하면 센서에서 특정값이 입력되면 SMS, 이메일, 트위터 등의 미리 지정해둔 컴퓨팅 작업이 이뤄지게 설계할 수 있다. 예를 들어 현관문에 트와인을 장착하고 문이 열리면 그 내용을 SMS 로 발송하라는 식으로 작업을 설정할 수 있다. 트와인을 특별히 언급한 것은 와이파이를 이용한 스마트폰 통신이 트와인과 동일한 구조로 구성되기 때문이다.



그림 11 트와인 https://www.kickstarter.com/

[그림 12]는 트와인의 구조를 나타낸 블럭도이다. 트와인의 구조는 마이크로프로세서, 센서, 와이파이 모듈이 핵심 구성요소이고, 그 외 전원을 공급하는 부분으로 구성돼 있다. 특히 와이파이 모듈에는 TCP/IP 스택이 내장돼 있어 웹서비스를 쉽게 마이크로 프로세서에서 구현할 수 있다. 이렇게 간단한 와이파이 구조를 통해 스마트폰과의 연동장치를 구성할 수 있다.

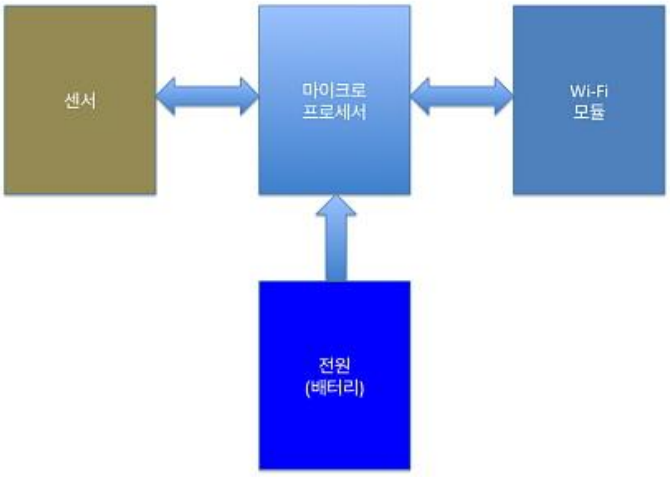


그림 12 트와인의 하드웨어 구조

1.4 Reference

[1] ITU-T Internet Report “The Internet of Things – Executive Summary”, Nov. 2005.

[2] 민경식, Net Term, 인터넷 & 시큐리티 이슈, “사물 인터넷 (Internet of Things)”, 2012.

[3] 전자부품연구원(KETI), “IoT(Internet of Things) Presentation Design”, Nov. 2012.

[4] 이준섭, TTA 표준화 교육, 스마트 통신시장의 핵심열쇠 “M2M”, “M2M/IoT Standardization in ITU-T”, Oct. 23, 2012.

[5] ITU-T, ITU-T SG13, “Y.WoT, Framework of Web of Things”, Oct. 10, 2011.

[6] ITU-T, ITU-T SG13, “Y.2060, Overview of the Internet of Things”, July 04, 2012.

[7] IETF, Draft-lee-iot-problem-statement-05.txt

[8] JP Vasseur, IETF 77, Internet Area Meeting, “The Internet of Things”, March 2010.

[9] IoT Protocol http://helloworld.naver.com/helloworld/textyl

Chapter 02. IoT 기반기술

이 장에서는 IoT 를 구현하는 대표적 4 대 기반 기술인 센싱기술, 카메라 기술, 통신기술, 서비스 인터페이스 기술 등에 대해 알아보자.

2.1 IoT 기반기술 개요

사물 인터넷의 주요 기술은 센싱 기술, 유무선 통신 및 네트워크 인프라 기술, 서비스 인터페이스 기술이다. 센싱 기술은 센서로부터 정보를 수집·처리·관리하고 정보가 서비스로 구현되기 위한 인터페이스 구현을 지원한다. 네트워크 종단간(end-to-end)에 사물 인터넷 서비스를 지원하기 위해서는 근거리 통신기술(WPAN, WLAN 등), 이동 통신기술(2G, 3G 등)과 유선 통신기술(Ethernet, BcN 등) 등의 유무선 통신 및 네트워크 인프라 기술이 필요하다.

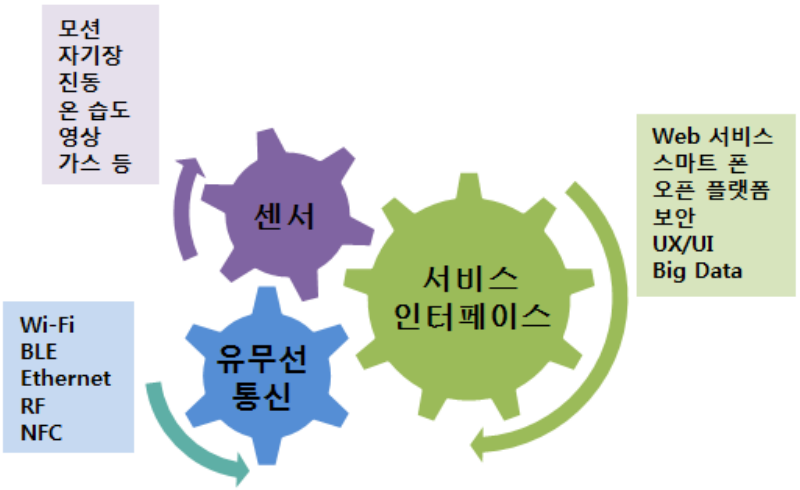


그림 13 사물인터넷 3 대 기반기술

사용자에게 사물 인터넷 서비스를 제공하기 위해서는 정보를 센싱, 가공/추출/처리, 저장, 판단, 상황 인식, 인지, 보안/프라이버시 보호, 인증/인가, 디스커버리, 객체 정형화, 오픈 API, 오픈 플랫폼 기술 등을 포함하는 서비스 인터페이스 기술이 필요하다.

최근에는 사물 인터넷 서비스를 보다 편리하게 구현하고 관리할 수 있는 기술들이 개발되고 있으며, 사물 인터넷 정보 전송, 메시지 처리, 통신 프로토콜 등에 대한 기술의 중요성이 부각되고 있다.

2.2 센서

아이폰을 시작으로 휴대폰에서의 센서 사용이 급증했다. 최근 출시되고 있는 스마트폰이 탑재하고 있는 센서의 개수는 약 20 개에 이른다. 각종 안전 관련 신기능들이 구현되고 있는 자동차의 경우 2015 년경 약 200 개 이상의 센서가 사용될 것으로 예상된다. 또한 밴드, 시계 형태의 웨어러블 기기의 출시로 다양한 분야에서 센서 활용도가 증가하고 있다.

센서 활용 본격화로 디바이스의 영역은 스마트폰, 태블릿과 같은 IT 산업 중심에서 건강, 교육, 의료 등 다양한 영역으로 확대되고 있다. 이에 따라 IoT 시대가 현실화 되고 있다.

(1) 센서시대의 출현

센서 분야에서 200 편 이상의 논문을 발표하며 '센서의 아버지'라고 불리는 Janusz Bryzek 교수는 향후 10 년 이내에 지구상에 사용되는 센서의 개수가 1 조 개를 넘게 될 것이라고 예상했다(2013 년 10 월 스탠포드 대학 'Trillion Sensors Summit').

센서 제조사 중심의 낙관론처럼 비춰 질 수 있겠지만, 인텔, HP, 보쉬, 퀄컴 등 다양한 분야의 IT 기업들 또한 5~10 년 이내 Trillion 센서 시대를 전망하고 있다. 이는 단순히 센서 제조 기업뿐만 아니라 모바일 프로세서, 디바이스, 서비스 등 IT 내 다양한 산업 군의 기업들이 공통적으로 전망하고 있다는 점에서 더 큰 의미를 갖는다.

가속도계, 자이로스코프 등의 센서를 예를 들더라도 이미 80 년대 이전부터 기술이 개발되어 2000 년대부터 상용화가 시작되었다. 이들 센서는 스마트폰에 본격적으로 사용되며 시장이 급속히 확대되었고 시장 확대에 따른 가격 하락으로 인해 다양한 기기에 범용적으로 사용이 가능해졌다. 실제로 이들 센서들은 초기 스마트폰의 가로/세로 화면 전환, 게임 컨트롤 등과 같은 일부 기능에 사용되었으나 현재는 밴드, 시계 등 웨어러블 (Wearable) 기기를 통해 피트니스(Fitness) 서비스, 헬스케어 (Health care) 분야 등 그 적용 범위가 IT

산업을 넘어 다양하게 활용되고 있다. 대표적인 센서 제조 기업인 STMicroelectronics 에 따르면 이와 같은 모션 센서 시장 규모는 지속적으로 증가하여 2017 년에는 1 조 7 천 억원에 이를 것이라고 전망하고 있다.

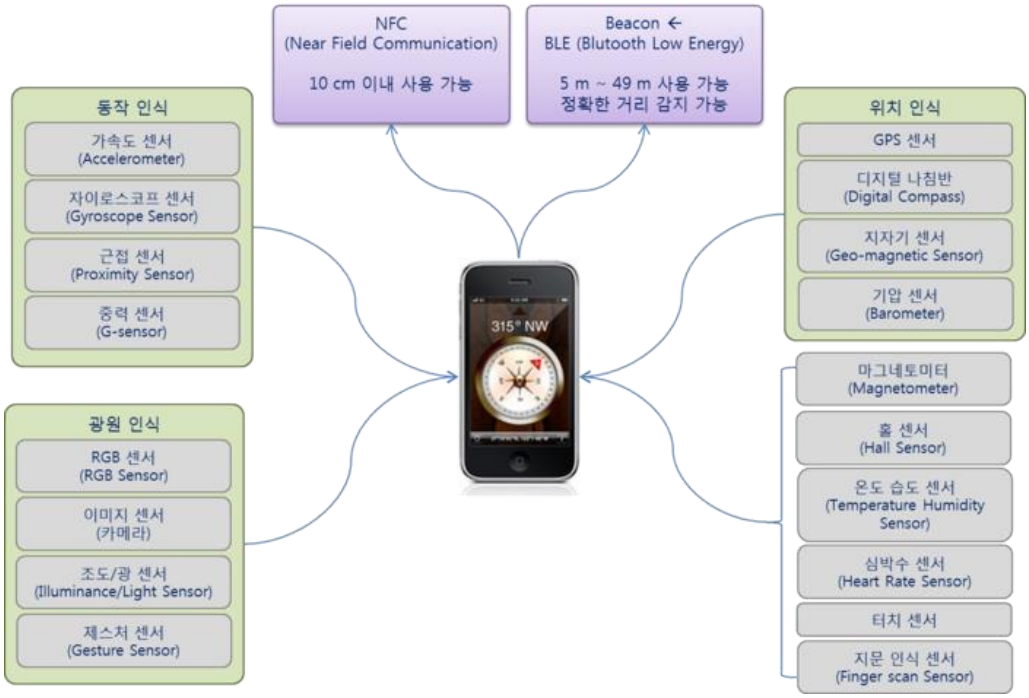


그림 14 스마트 폰에 적용된 센서들 (출처 : :www.jopenbusiness.com)

한국센서연구조합에 따르면 현재 기술적으로 구현 가능한 센서의 종류는 약 350 여 가지에 이른다고 한다. 학계, 산업의 센서를 연구하는 주요 연구 방향은 아래와 같다.

- 이전에는 센싱(Sensing)할 수 없었던 새로운 영역에 대한 센서 연구
- 제조 공정, 작동 원리 등의 혁신 (예: 3D 프린팅 등의 기술을 활용한 Flexible 형태 센서)
- 전력 소비 향상을 통한 가용성 확대 (예: 운동, 열 에너지의 변환을 통한 자가 발전 센서)
- 기존 센싱 가능영역에 대한 센싱의 정확도와 정밀도를 향상 시키는 연구 및 개발 (예: 미세 동작 감지 센서)

하지만 이러한 센서 사용의 확대가 단순히 센서 기술의 발달 자체에 의해서만 가능해 진 것은 아니다. 스마트폰을 통해 발달해 온 모바일 컴퓨팅의 진화, 네트워크 인프라 확대, 클라우드 컴퓨팅의 보편화는 이미 개발은 되었으나 제대로 활용되지 못했던 센서를 활용한 시스템 및 서비스를 가능하게 만들고 있다.

모바일 프로세서의 발전으로 센서 모듈과 같은 작은 디바이스에서도 저전력으로 고성능의 정보 처리가 가능하게 되었고, WiFi/3G/LTE 등의 기술을 활용한 통신 인프라의 확대는 다양한 기기들의 연결성(기기-기기, 기기-인터넷) 확대를 가지고 왔다. 클라우드 컴퓨팅은 센서에서 수집된 정보를 인터넷 공간에 저장, 처리할 수 있게 하였고, 이를 통해 인터넷 기반의 Big Data 및 다양한 서비스와 결합을 가능하게 하고 있다. 따라서 이러한 발전된 컴퓨팅 기술을 활용해 다양한 기기 및 서비스 구현이 가능해 지고 있고, 그에 따라 그 응용 영역도 빠르게 넓어지고 있다.

(2) 인간의 오감과 센서

Sensor 의 의미를 알아보자.

- 라틴어로 지각, 느낌 등의 의미
- 인간의 5감 기능(시각·청각·후각·미각·촉각)을 센스(SENSE)라 함.
- 리드SW. 패턴 인식을 하는 시스템 레벨의 센서.
- 최근에는 인텔리전트 센서로CPU 내장형. 복잡 다양성을 띠고 있다.

센서란 주변의 물리 화학 생물학적 정보를 감지하여 전기적 신호로 변환하는 모든 장치(Device)를 의미한다.

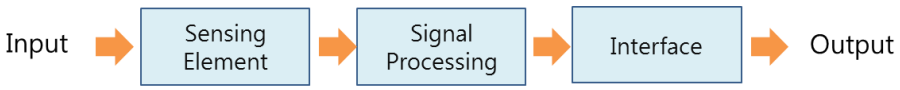


그림 15 센서의 처리

인간의 오감과 Sensor

- 눈-시각: 광센서{광도전소자, 카메라 CCD, 영상(이미지)센서, 포토다이오드}
- 귀-청각: 음향센서(마이크로폰, 압력소자)
- 피부-촉각: 촉각센서
- 혀-미각: 미각센서(백금, 산화물, 반도체가스센서, 입자센서, 산화계열 선형소결반도체)
- 코-후각: 후각센서(가스, 메탄, 바이오케미칼센서, 실리콘산티탄산염)

(3) 센서 분류

센서를 크게 분류하면 다음과 같이 분류할 수 있다. 이러한 분류 기준에 의해 분류된 센서는 다시 전기적, 화학적, 물리량, 오감등의 기준으로 세부적으로 나누어진다. 센서가 분류되는 방법을 알아보자.

- 구성에 의한 분류 : 기본센서, 조립센서, 응용센서
- 기구에 의한 분류 : 기구형(또는구조형), 물성형, 기구, 물성혼합형
- 검출신호에 의한 분류 : 아날로그센서, 디지털센서, 주파수형센서, 2 진형센서
- 감지기능에 의한 분류 : 공간량, 역학량, 열역학량, 전자기학량 공학량, 화학량, 시각, 촉각
- 변환방법에 의한 분류 : 역학적, 열역학적, 전기적, 자기적, 전자기적, 광학적, 전기화학적, 촉매화학적, 미생물학적, 효소화학적
- 재료별 : 세라믹, 반도체, 금속, 고분자, 효소, 미생물
- 용도별 : 계측용, 감시용, 검사용, 제어용
- 특징별 : 다차원센서, 다기능센서
- 분야별 : 산업용, 민수용, 의료용, 화학 실험용, 우주용, 군사용

아래는 역학량, 물리량, 화학량등에 따른 센서 분류를 나타낸다. 센서의 구분에 따른 센싱 대상과 어떤 효과를 이용하는가에 따라 분류하는 방법이다.

표 1 센서의 역학량에 따른 분류

감지대상	센 서

변위 길이	차동트랜스, 스트레인게이지, 이미지센서, 콘덴서(정전형) 변위계
속도 가속도	회전형 속도계, 동전형 가속도계
회전수 진동	로터리 엔코더, 스코프, 압전형 검출기
압 력	다이어프램, 로드셀, 수정압력계
힘(토크)	저울, 천칭, 토션바(비틀림 바)

표 2 물리량과 화학량에 따른 센서의 분류

구분	감지대상	센서	주요효과
물리센서	온도	열전대, 서미스터, 온도계	열저항, 열복사
	빛 색	광전도, 광결합형, 이미지센서, 포토다이오드	광전도, 패러데이 필터
	자기	홀(hall)소자, 자기저항소자	Hall,Josephoson
	전류	분류기, 변류기	전류의 크기 변화
	자외선, 방사선	조도계, 광량계, GM계수계	
화학센서	습도	세라믹센서, 결로센서, 고분자막센서	저항, 유전율의 변화
	가스	매연센서, 반도체 가스 센서, 동성가스센서	저항 변화, 기전력 변화
	이온	PH전극센서, 이온선택	기전력, 저항

2.3 영상처리(카메라)

이미지 센서(image sensor)는 반도체소자의 제조기술을 이용하여 집적회로화된 광전변환 소자이다. 팩시밀리나 복사기에 사용되는 라인 이미지 센서 (일차원 이미지 센서)와 텔레비전 카메라나 디지털 카메라에 사용되는 어레이 이미지 센서 (이차원 이미지 센서)가 있다.



그림 16 라즈베리파이용 카메라 모듈

라인 이미지 센서는 빛을 검출해서 전하를 발생시키는 광다이오드를 일렬로 배치한 것이며, 면적을 가지는 영역을 촬상하려면 대상물 (문서나 사진)을 센서라인과 직각방향으로 주사 (스캔)해서 정지화면을 취득하는데 사용한다. 어레이 이미지 센서는 광다이오드를 이차원에 배열한 것이며, 한번에 화면전체 이미지를 획득할 수 있어 비디오같은 동영상용으로 사용된다. 광다이오드가 발생한 전하를 출력하여 전송기능으로 전하 결합 소자 (CCD) 및 시모스 소자가 있으며, 이 차이에 의하여 CCD 이미지 센서, 시모스 이미지 센서로 불린다.

▪ CCD

CCD 는 Charge-Coupled Device 의 약자로 빛을 전하로 변환시켜 화상(畵像)을 얻어내는 센서이다. 전하결합소자 (電荷結合素子)라고도 부르며, 1969 년 조지 E. 스미스와 윌러드 보일이 공동개발하였다. CCD 는 여러 개의 축전기(Condenser)가 쌍으로 상호 연결되어 있는 회로로 구성되어 있고, 회로 내의 각 축전기는 자신 주변의 축전지로 충적된 전하를 전달한다. CCD 는 디지털 스틸 카메라, 광학 스캐너, 디지털 비디오 카메라와 같은 장치의 주요 부품으로 사용된다.

CCD 칩은 많은 광다이오드 들이 모여있는 칩이다. 각각의 광다이오드에 빛이 빛추어지면 빛의 알갱이 즉 광자의 양에 따라 전자가 생기고 해당 광다이오드의

전자량이 각각 빛의 밝기를 뜻하게 되어 이 정보를 재구성 함으로써 화면을 이루는 이미지 정보가 만들어진다.

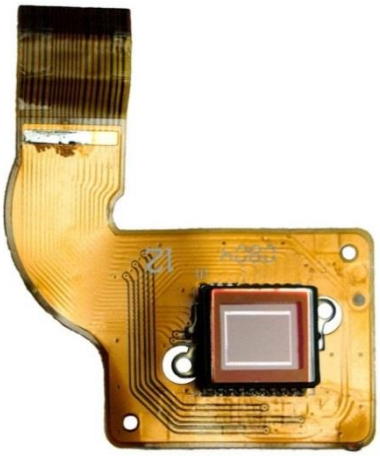


그림 17 CCD 이미지 센서(위키백과)

각각의 광다이오드에 생성된 전자의 양을 전달하기 위해서 여러가지 방법이 쓰이는데 크게 Interline Transfer, Frame Transfer, Frame-Interline Transfer, Fullframe 등의 방법이 있다.

Interline Transfer 방식은 광다이오드의 약 30% 가량의 크기를 가진, 전자들을 전달할 수 있는 레지스터가 각각의 광다이오드 바로 옆이나 혹은 각각의 광다이오드 바로 위에 붙어있는 형식으로 이 레지스터는 빛으로부터 가려지도록 되어 있다.

▪ CMOS 이미지 센서

CMOS 이미지 센서의 원리가 고안된 것은 1960 년대 후반이지만 실용화된 것은 미세 가공 기술이 첨단화된 1990 년대 이후이다. 2000 년대 후반부터 Image Sensor 기술의 고해상도화가 진행됨에 따라 BSI(back side illumination) 공정 기술과 3D Stacking Sensor 제작 공정기술이 대두되고 있다.

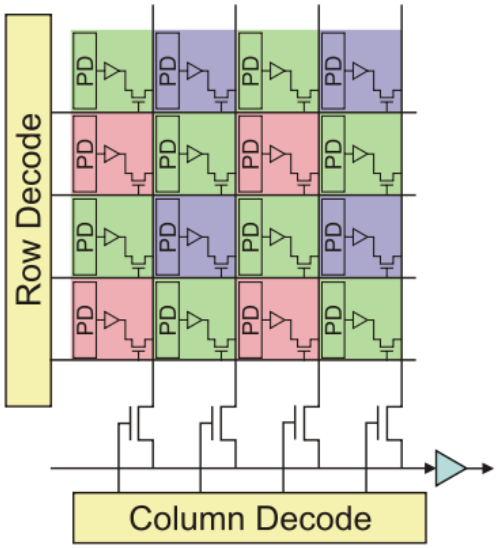


그림 18 CMOS image sensor 제작

Wafer Stacking 을 통한 3D CIS(CMOS image sensor) 제작 공정은 한 Wafer 에다 Pixel 의광 집적 부분 과 구동 회로 부분을 동시에 공정 진행하는 것과는 달리, 광 집적 부분과 구동 회로 부분을 각 각 다른 Wafer 에 공정 진행하여, 일정 공정 후 두 Wafer 를 각각 서로 Bonding 한 후 후속 일정 공정을 진행하는 방식으로 이루어진다. 즉, 한 Wafer 에는 광 집적 부분만을 효율적으로 공정 진행하고 또 다른 Wafer 에는 광 집적 부분의 신호를 받아 이를 처리하여 출력 신호로 내보내는 구동 회로 부분만 역시 효율적으로 공정을 진행한다. 이 때 필수적으로 두 Wafer 를 Bonding 하기 위한 Bonding Pad 를 각 Wafer 에 형성하게 되고 이 Bonding Pad 를 상,하 서로 포개어 접촉 시키게 된다.

CMOS 이미지 센서는 CCD 이미지 센서보다 범용 반도체 제조 장치를 이용하여 제조가 가능해서 CCD 이미지 센서보다 가격이 저렴하다. 그렇기 때문에 저가 디지털 카메라나 디지털 비디오 카메라, 느린 프레임의 텔레비전 카메라에서 활발하게 사용되고 있다. 특히 비디오 채팅에 사용되는 웹 카메라의 대부분이 CMOS 를 탑재하고 있다. 그리고 야노 경제 연구소(일본어: 矢野経済研究所)에 의하면 카메라 기능을 가진 휴대 전화가 보급된 결과

2004 년에는 CMOS 이미지 센서의 출하량이 CCD 이미지 센서의 출하량을 뛰어 넘었다고 예상했다.

과거에는 CCD 보다 화질이 많이 떨어져서 저가 제품에만 사용되었다. 하지만 최근에는 큰 크기로 설계가 가능한 점, 낮은 제조 비용, 대폭 향상된 화질로 인해 DSLR 카메라과 같이 고화질을 요구하는 고가 제품에도 많이 사용되고 있다. 캐논은 자체 개발, 제조한 CMOS 이미지 센서를 2002 년부터 DSLR 에 채용하고 있다. 니콘의 최고급형 DSLR 인 D2X 에서도 소니사의 CMOS 를 채용하고 있다. 이러한 DSLR 에 사용되는 CMOS 센서의 경우 비슷한 시기에 개발된 CCD 와 비교했을 때, 비슷하거나 오히려 더 높은 화질을 보여준다. 그리고 소니나 캐논은 보급형 HD 비디오 카메라에서도 CMOS 이미지 센서를 사용하고 있다.

2.4 네트워크 인프라

(1) 사물 인터넷 시대의 네트워크 인프라

이전에는 생소하게 느껴졌던 사물 인터넷은 이미 다양한 산업에 걸쳐 빠르게 적용되고 있다. 기술 수준이 발전하고 제반 환경이 구축되면서 사물 인터넷이 제조업과 서비스업 등 거의 모든 분야에서 막대한 가치를 창출하게 될 것이라는 전망이 제기되고 있다. 시장조사기관 IDC 는 사물 인터넷에 대한 수요 및 인프라 구축이 확대됨에 따라 2020 년에는 관련 시장 규모가 8 조 9 천 억 달러에 이르게 될 것으로 전망하기도 하였다.

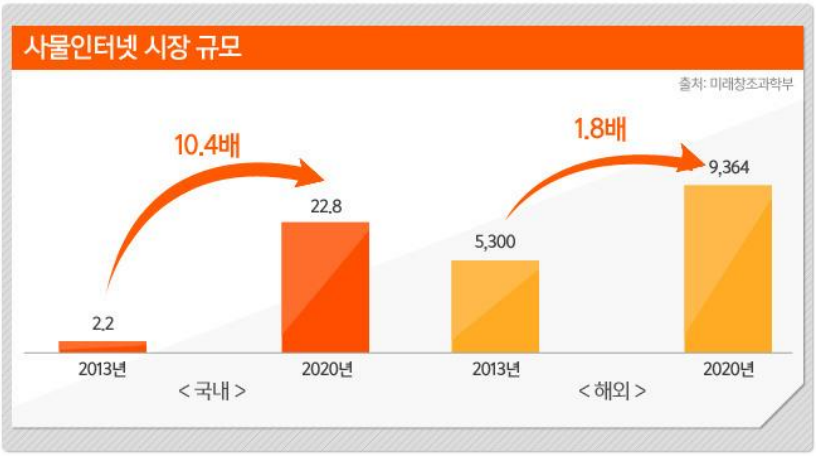


그림 19 미래 창조 과학부 사물인터넷 시장전망

광범위한 지역에 걸쳐 사람과 각종 사물을 인터넷과 연결하고 필요한 정보를 주고 받는 사물 인터넷이 구현되기 위해서는 언제(Anytime) 어디에서나(Anywhere) 어떤 사물(Anything)과도 연결되어 데이터를 주고 받을 수 있는 초연결(Hyper connection)의 실현이 필수적이다. 따라서 이를 위해서는 다양한 종류의 유무선 네트워크 기술을 이용하여 효과적으로 데이터를 각지로 송수신하고 원활하게 처리할 수 있는 네트워크 인프라가 구축되어야 한다.

원래 네트워크 인프라는 데이터를 유무선으로 송수신하는 라우터(Router)와 게이트웨이 Gateway), 기지국 Base station) 등의 장비를 주로 지칭하였다. 그러나 기술의 융합 및 장비 간 역할의 경계가 흐려지면서 현재는 방송 제작 및 송수신을 위한 방송 장비 및 데이터의 수집과 처리 및 저장을 위한 데이터 센터까지 포괄하는 광의의 개념으로 확장되고 있으며 그 중요성 또한 한층 강조되고 있다.

(2) 네트워크 인프라 경쟁의 배경

▪ 기존 네트워크 인프라의 한계 봉착

현재 구축된 네트워크 인프라만으로는 미래에 새롭게 등장하게 될 각종 단말과 서비스를 충분히 수용하기 어렵다는 주장은 사물 인터넷이 중요한 이슈로 떠오르기 이전부터 지속적으로 제기되어 왔다. 따라서 글로벌 IT 기업 및 여러 표준화 단체를 중심으로 새로운 네트워크 인프라에 대한 구상이

지속적으로 논의되었다. 그러나 최근 사물 인터넷이 글로벌 경제를 견인할 새로운 기술 패러다임으로 떠오르게 되면서, 차세대 네트워크 인프라 역시 사물 인터넷의 개념과 비전을 효과적으로 구현하기 위한 방향으로 발전하게 될 것이다.

▪ 새로운 가치 창출을 위한 플랫폼의 역할

또한 사물 인터넷 시대의 네트워크 인프라는 단순히 데이터가 오고 가는 경로의 역할(Dumb pipe)에 머물던 것에서 벗어나 자체적으로 필요한 데이터를 분류하고 재생산하여 새로운 가치를 창출하는 적극적인 역할을 수행하게 될 것으로 예상된다.

각 기업 및 개인을 위하여 인프라가 다양한 기능과 형태를 갖춘 서비스로 제공되는 IaaS(Infrastructure as a Service)의 개념이 도입되면서 네트워크 인프라는 사용자의 수요와 편의에 맞게 다양하게 활용될 것으로 보인다. 따라서 이러한 추세에 따라 네트워크 인프라는 새로운 아이디어 및 비즈니스의 탄생을 견인하게 될 사물 인터넷 시대의 핵심 플랫폼으로 자리잡게 될 것으로 전망되고 있다.

이미 여러 분야에서 사물 간 통신(M2M Communication)을 위한 네트워크 인프라가 자리잡고 있기 때문에 사물 인터넷의 기본적인 기능 구현이 어려운 것은 아니다. 그러나 데이터의 전달이 주된 역할인 오늘날의 네트워크 인프라로는 사람과 사물간의 자유로운 연결 지원, 안전한 데이터 전송 및 활용과 서비스의 품질 (Quality of Service) 보장 등 사물 인터넷이 제시하고 있는 비전을 충분히 달성하기 쉽지 않을 것이라는 주장이 많다. 특히 각 산업에 걸쳐 사물 인터넷 의 활용이 한층 광범위하게 이루어진다면 각종 센서 및 사용자 단말 등 사물 자체의 고도화만으로는 한계가 있을 것이라는 예상이다.

따라서 미래 사물 인터넷의 다양한 서비스를 효과적으로 구현하기 위해서는 네트워크 인프라가 거대한 플랫폼으로서 데이터의 축적과 분석 및 각종 서비스 지원 등 여러 핵심적인 기능을 자체적으로 수행할 수 있게 될 것으로 보인다. 이러한 관점에서 현재 학계 및 기업을 중심으로 데이터의 종류 및 특징과 전송 환경 등을 고려하여 사용자간 데이터 송수신 및 활용을 지원할 수 있는 네트워크 인프라 기술이 집중적으로 연구되고 있다.

(3) 주요 네트워크 인프라 기술

• 네트워크의 가상화

차세대 네트워크 인프라 구축을 위한 가장 중요한 기술로 논의되고 있는 것이 바로 물리적인 네트워크 인프라를 마치 여러 개의 네트워크처럼 자유롭게 활용할 수 있도록 구현하는 가상화(Virtualization) 기술이다. 가상화 기술은 사실 오래전부터 PC 와 데이터 센터 등 여러 분야에서 연구되어 왔는데, 최근 네트워크 인프라에 대한 투자를 최소한으로 유지하면서 데이터 전송 용량과 컴퓨팅 능력 등 네트워크 인프라의 자원 활용을 극대화하는 것이 통신 서비스 기업의 화두로 떠오르게 되면서 네트워크 분야에서도 많은 주목을 받고 있다.

네트워크의 가상화 구현을 위한 대표적인 기술이 바로 소프트웨어 정의 네트워크(SDN: Software Defined Network)이다. SDN 이란 데이터의 전달과 제어 기능을 담당하고 있는 라우터 및 게이트웨이 등 네트워크 장비에서 제어 기능을 분리하여 중앙 센터로 이전함으로써 전체 네트워크 인프라를 중앙에서 관리하는 동시에 상황에 따라 네트워크의 구성 및 데이터의 흐름을 자유롭게 제어하는 기술을 말한다. SDN 을 통하여 원활한 데이터 송수신 지원은 물론이고 각각의 목적에 맞는 가상 네트워크 구축을 지원함으로써 네트워크 자원의 활용률을 크게 끌어 올릴 수 있을 것으로 기대되고 있다.



SDN(Software Defined Networking) 이란 ?

SDNd이란 "Software Defined Networking"을 의미하며 우리말로 흔히 소프트웨어 정의 네트워킹이라 부른다. SDN은 OpenFlow라는 기술 혹은 소프트웨어를 통하여 널리 알려졌다. OpenFlow와 SDN은 뗄레야 뗄 수 없는 관계이다. SDN이 물론 더 큰 개념으로 네트워크 구조 혹은 새로운 패러다임이며, OpenFlow는 SDN을 위한 “인터페이스 표준 기술”로 정의된다.

SDN은 이른바 소프트웨어를 통해서 현재의 인터넷이 가지는 구조적 문제를 근본적으로 해결하고 혁신할 수 있도록 대두된 새로운 네트워크 구조 혹은 패러다임으로써, 어플리케이션, 네트워크 OS, 하드웨어 추상화, 표준화된 인터페이스 및 하드웨어를 모두 아우르는 개념이다.

SDN 은 미국 스탠포드 대학의 주도로 시작된 오픈 플로우 (Open Flow) 기술 연구에서 비롯되었다.

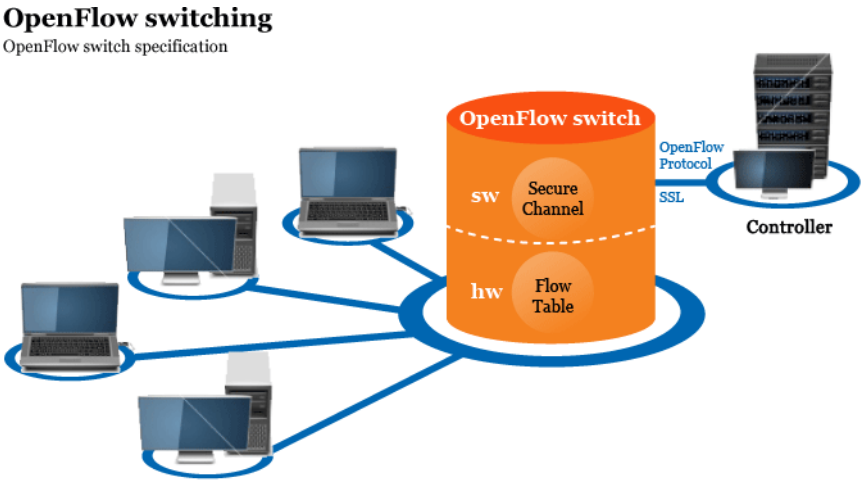


그림 27 오픈 플로우(Open Flow) 기술 (출처 : http://www.xinguard.com/)

오픈 플로우는 수많은 네트워크 기반의 연구 프로젝트를 동시에 수행할 수 있도록 테스트베드(Testbed)을 구축하기 위한 시도에서 출발하였는데, 이를 통하여 다양한 SDN 기반의 네트워크 장비가 새롭게 개발될 수 있었다. 특히 2012 년에 오픈 네트워킹 파운데이션(ONF: Open Networking Foundation)이 설립된 이후 많은 통신 서비스 기업들이 SDN 을 활용하여 네트워크 인프라를 새롭게 구축하려는 시도가 더욱 활발하게 이루어지고 있다. 특히 구글은 전세계에 흩어져 있는 광범위한 자사의 데이터 센터를 연결하고 있는 네트워크 인프라에 SDN 을 도입하는 G-Scale 네트워크 프로젝트를 통하여 30~40%대에 머물던 네트워크 인프라 자원의 활용률을 100%에 가깝게 끌어 올렸다고 주장하기도 하였다.

한편으로 네트워크 기능 가상화(NFV: Network Functions Virtualization) 역시 최근 통신 서비스 기업을 중심으로 새로운 가상화 기술로 많은 관심을 받고 있다. NFV 는 유럽전기통신표준협회 (European Telecommunication Standards Institute)가 2012 년 처음 소개된 이래 단시간에 큰 주목을 받게 되었는데, 각종 네트워크 장비에서 소프트웨어 기능을 분리하고 이를 범용적인 서버 시스템에 통합적으로 설치하여 중앙집중식으로 운영할 수 있도록 지원하는 기술이다.

따라서 NFV 를 통하여 기존처럼 각 장비의 기능을 개별적으로 설치 및 유지하지 않고 한 곳에서 관리함으로써 인프라의 효율성을 증가시키고 유지 비용의 부담도 덜 수 있게 된다. NFV 는 아직 개념이 도입된 지 얼마 지나지 않은 탓에 여전히 기술적 난제가 적지 않지만, SDN 과 더불어 네트워크 가상화의 핵심으로 더욱 활발하게 연구될 것으로 예상된다.

• 네트워크의 지능화

네트워크 인프라 자체가 데이터 처리 및 활용을 능동적으로 담당하게 될 것이라는 주장은 사물 인터넷의 등장과 함께 더욱 강조되고 있다. 기존 네트워크 인프라가 데이터 센터와 단말 사이에서 데이터를 송수신하기 위한 경로로만 활용되었다면, 향후에는 다양한 사물들이 만들어 내는 데이터의 생산부터 활용까지 전 과정을 능동적으로 판단하고 데이터 센터의 서비스 지원 등 주요 기능까지 자체적으로 수행할 수 있게 된다는 것이다.

특히 지능화된 네트워크 인프라는 개인과 사물의 위치 및 특성과 같은 인지적 상황 정보(Contextual Information)를 적극적으로 활용하여 각종 사물 인터넷 서비스를 효과적으로 수행하는 데에 큰 도움을 줄 수 있을 것으로 예상되고 있다.

사실 네트워크의 지능화라는 개념은 현실성이 떨어지고 이를 실현하기 위한 기술의 상용화 역시 쉽지 않을 것이라는 의견도 적지 않았다. 그러나 네트워크 인프라의 자율적인 상황 판단과 제어를 통하여 시스템 유지의 비용과 병목 현상을 낮출 수 있으며, 네트워크 인프라의 자원 활용률 제고 및 전력 소모 감소 등 새로운 장점이 부각되면서 여러 방면에 걸쳐 활발히 연구되고 있다. 현재 학계를 중심으로 데이터가 포함하는 정보를 활용하여 실시간으로 송수신 방법을 제어할 수 있는 정보 중심 네트워크(ICN: Information Centric Network) 기술이 꾸준히 연구되어 온 가운데, 필요에 따라 네트워크 구성과 자원 할당을 자율적으로 변화하여 효과적인 데이터 전송을 지원할 수 있는 자가 구성 네트워크(SON: Self Organizing Network)에 대한 기술 연구 및 표준화 작업도 이루어지고 있다. 그러므로 이런 흐름 아래 많은 기업들 역시 다양한 방식의 지능형 네트워크 기술 연구에 나서고 있다.

• 네트워크의 융합화

사물 인터넷의 부상에 따라 한편으로는 사물 간 데이터 송수신을 위한 전용 네트워크 인프라를 구축하려는 움직임도 있다. 프랑스의 무선 통신 기업 시그폭스(SigFox)는 스마트폰에 비하여 데이터 전송량이 적은 매연 감지기와 경보기 등 실생활의 각종 단말 간 통신을 목적으로 프랑스와 네덜란드, 스페인 등에 전용 네트워크 인프라를 구축하고 있다. 이는 오늘날 사물 인터넷 기반 서비스가 LTE 등 현재의 네트워크 인프라를 효과적으로 활용하기 어렵다는 문제를 감안하여 별도의 인프라를 구축함으로써 기존 시스템과의 공존을 모색하려는 시도로 볼 수 있다.

최근 많은 기업 및 연구 기관들을 중심으로 네트워크의 융합화를 위한 여러 방안들이 고려되고 있는데, 현재 활발하게 상용화되고 있는 HetNet(Heterogonous Network) 등이 이를 실현시킬 수 있는 유망한 기술로 예상되고 있다. 원래 HetNet 은 사람들이 몰리는 곳이나 기지국의 전파가 약한 곳에 펨토셀(Femto cell)과 원격무선장비(Radio Remove Hend) 등 작은 기지국을 추가적으로 설치하여 통신 품질을 강화하기 위한 방안이다. 그러나 이러한 HetNet 의 개념이 확장되면서 현재는 Wi-Fi 등 여러 네트워크 기술을 유기적으로 통합하고 상황에 따라 특정 기술의 네트워크 접속을 선택적으로 제공하는 방안 등이 논의되고 있다. 따라서 이와 같은 시도들이 한층 진전된다면 궁극적으로는 다양한 네트워크 기술을 유연하게 수용할 수 있는 새로운 개념의 네트워크 인프라가 구축될 수 있을 것으로 전망된다.

2.5 IoT 통신기술

무선 센서 네트워크(Wireless Sensor Network)는 21 세기에 중요한 기술 중 하나로 자리매김하고 있다. 특히 무선 기술과 MEMS(Micro-Electro-Mechanical Systems)의 발전으로 작고 값싼 스마트 센서들이 다양한 분야에서 응용되고 있으며, 그 응용분야는 환경 모니터링, 헬스 케어, 보안 및 감시, 스마트 홈, 스마트 그리드 등 매우 다양하다. 지난 10 년 동안 학계와 산업계에서는 이러한 센서 네트워크 분야에서 다양한 연구개발 성과를 이루었으며, 이러한 기술을 기반으로 센서들과 다양한 디바이스들이 모두 연결되는 IoT(Internet of Things) 기술로의 진화를 시작하였다. IoT 기술은

다양한 네트워크 기술을 바탕으로 다양한 장치들 간의 인터넷 연결성을 제공하여 이들을 활용한 지능적 서비스를 창출할 수 있도록 하는 기술적 개념이며, 무선 센서 네트워크 기반으로 하는 IoT 기술 이슈와 동향에 대해 살펴보고자 한다.

무선 센서 네트워크 기술은 컴퓨팅 능력과 무선 통신 능력을 갖춘 센서노드를 응용 환경에 배치하여 자율적인 네트워크를 형성하고 센서 노드로부터 획득한 정보들을 무선으로 수집하여 감시/제어 등의 용도로 활용하는 기술이다. 무선 센서 네트워크 기술의 궁극적인 목표는 모든 사물에 컴퓨팅 능력 및 무선 통신 능력을 부여하여 언제, 어디서나 사물들끼리의 통신이 가능한 유비쿼터스 환경을 구현하는 것이며, 현재 다양한 기술이 융합되면서 IoT, M2M2) 기술로 진화하고 있다.

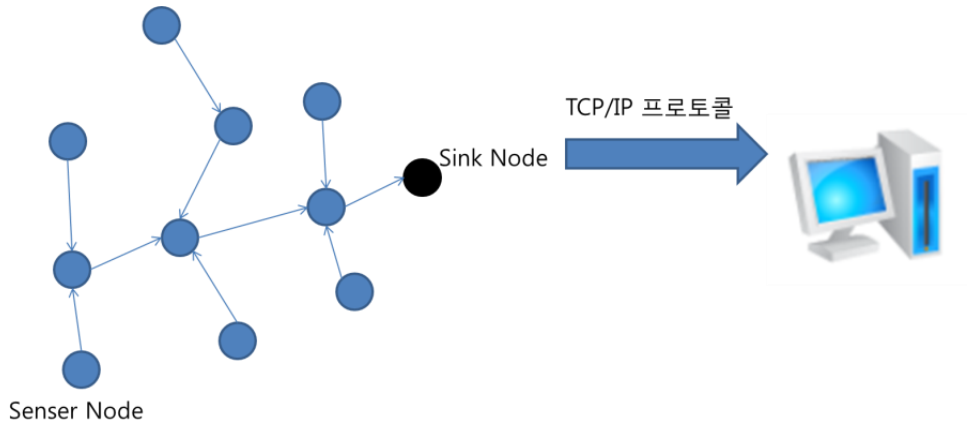


그림 20 센서노드 개념 (Devsign-Develop & Design)

이와 같이 무선 센서 네트워크 기술은 IoT 기술 전개에 있어 근간이 되며 특히, 인간의 개입없이, 혹은 최소한의 개입으로 사물간 협력하여 센싱, 정보처리 및 교환을 위한 상호 지능적 자율적인 네트워크 형성에 있어 무선 센서 라우팅 기술은 중요하다. 또한 다양한 기기와의 정보 교환과 연동을 위해 센서 네트워크의 물리계층 및 네트워크 기술도 중요한 기술 요소로 자리 잡고 있다.

무선 센서 네트워크는 일반적으로 저비용/저전력/다기능의 센서노드들로 이루어져 있으며, 이러한 센서들은 라디오 트랜시버와 데이터 처리를 실현하는 마이크로 프로세서가 장착되어 있다. 이 센서들은 전통적인 RF 기술을 이용한

단거리 통신을 시작으로 3G, LTE 등의 기술을 이용한 장거리 통신을 실현하고 있으며 RFID, NFC, WiFi, Bluetooth, ZigBee, GSM, 3G, LTE 등 다양한 네트워크 기술에 적용되고 있다.

▪ 비 IP 네트워크 표준 기술

유무선 통신 및 네트워크를 위한 기술로는 근거리 통신, WiFi, 3G/4G/LTE 등이 대표적이다. 한편, 디바이스가 인터넷 연결을 기반으로 정보를 주고받는 IP 방식의 프로토콜과는 달리, IP 를 사용하지 않는 경우의 디바이스간 통신은 블루투스, ZigBee, RFID, ZWave 등을 이용한다. 이러한 비 IP 네트워크 기술을 구현하고 있는 전통적인 센서 노드 구조는 싱크노드(Sink Node)를 통해 인터넷과 연결되어 정보를 공유할 수가 있으며, 이를 위해 센서 노드는 6LoWPAN 과 같은 기술을 적용할 필요가 있다. 6LoWPAN(IPv6 over Low power WPAN)은 IEEE 802.15.4 를 PHY/MAC 으로 하는 저전력 WPAN 상에 IPv6 를 탑재하여 기존 IP 네트워크에 연결하는 기술이며, 6LoWPAN 을 포함한 관련된 여러 기술들에 대해 기술해 보기로 한다.

▪ ZigBee

ZigBee 는 저속 전송 속도와 근거리 통신을 위하여 ZigBee Alliance 에서 개발한 무선 네트워크 기술인데, 작은 크기로 전력 소모량이 적고 값이 싸 홈네트워크 등 유비쿼터스 구축 솔루션으로 각광받고 있으며 지능형 홈네트워크, 빌딩 등의 근거리 통신 시장과 산업용기기 자동화, 물류, 휴먼 인터페이스, 텔레매틱스, 환경 모니터링, 군사 등에 활용된다. ZigBee 프로토콜은 물리 계층, 미디어 액세스 제어(MAC) 계층, 네트워크 계층, 그리고 어플리케이션 계층으로 이루어져 있다.

ZigBee 의 물리 계층과 MAC 계층은 IEEE 802.15.4 표준에 정의되어 있으며, 그 이외의 프로토콜 스택은 ZigBee 사양에 정의되어 있다. IEEE 802.15.4 초기 버전에서 유럽은 868 MHz 대역에 서 20kbps 를, 미국은 915 MHz 에서 40kbps 를 지원하였다. ZigBee 네트워크 계층은 트리 구조와 메쉬 구조를 위한 라우팅과 어드레싱를 지원하고 있으며, 어플리케이션 프로파일러는 ZigBee Home Automation Public Profile 과 ZigBee Smart Energy Profile 이 대표적으로 사용된다. 또, 새로운 ZigBee 사양인 RF4CE 는 가전의 원격 제어를 위한 솔루션과 스타 토폴로지를 위한 간단한 네트워크 스택을 정의하고 있는데,

RF4CE 는 2.4GHz 의 주파수 대역을 사용하고 AES-128 을 이용한 보안을 제공한다

▪ Z-Wave

Z-Wave 는 ZenSys 가 주축이 된 Z-Wave Alliance 에서 제정한 홈오토메이션 무선 전송 방식이며, Z-Wave 의 주 목적은 무선 네트워크에서 하나 이상의 노드들과 제어유니트 사이에서 신뢰성 있는 통신을 제공하는 것이다. Z-Wave 는 물리 계층, 미디어 액세스 제어(MAC) 계층, 전송 계층, 라우팅 계층, 그리고 어플리케이션 계층으로 구성되어 있으며, 900MHz 대역(유럽: 869MHz, 미국: 908MHz)과 2.4GHz 대역을 사용하면서 9.6kbps, 40kbps, 그리고 200kbps 의 속도를 제공한다. Z-Wave 기술은 장치간의 통신을 위해 콘트롤러와 슬레이브의 두가지 장치를 정의하는데, 콘트롤러는 슬레이브에서 명령을 전송하며, 슬레이브는 명령에 대한 응답을 하거나 명령을 수행하는 기능을 한다. Z-Wave 의 라우팅 계층은 소스 라우팅 기반의 라우팅을 지원한다

▪ INSTEON

SmartLabs 에서 개발한 INSTEON 은 무선 기술을 활용하여 조명 스위치를 연결하는 기술로서, RF 링크와 AC-전원 링크간에 메쉬 네트워크 토폴로지를 구성하여 작은 구역에서 장치간의 통신을 실현한다. INSTEON 기술은 904 MHz 대역에서 동작하며, 38.4 kbps 데이터 전송 속도를 제공한다. INSTEON 의 디바이스들은 전송자의 역할, 수신자의 역할, 혹은 중계자의 역할을 수행할 수 있으며, 동일한 구간에 위치하지 않은 디바이스들 간의 통신은 시간 구간 통기화 방법을 이용한 멀티-홉 라우팅을 사용하여 가능하게 한다

▪ WAVENIS

Wavenis 무선 기술은 WSN 응용과 M2M 응용들을 지원하기 위해 2-way 무선 플랫 폼 기술을 제공하며, Coronis Systems 에서 모니터링 응용과 제어를 위한 프로토콜 스택을 구현하여 빌딩/홈 자동화에 적용하였다. Wavenis 저전력 및 장거리 무선 기술은 간단한 포인트-투-포인트 연결과 여러 리피터 고급 트리 구조, 메쉬 네트워크의 다양한 구성을 지원한다

▪ 6LoWPAN

비 IP 네트워크 기술을 구현하고 있는 전통적인 센서 노드 구조는 싱크노드(Sink Node)를 통해 인터넷과 연결되어 정보를 공유할 수가 있다. 이를 위해, 센서 노드는 6LoWPAN 과 같은 기술을 적용할 필요가 있다. 6LoWPAN(IPv6 over Low power WPAN)은 IEEE 802.15.43)를 PHY/MAC 으로 하는 저전력 WPAN 상에 IPv6 를 탑재하여 기존 IP 네트워크에 연결하는 기술이며, IPv64) 기술로 인해 다양한 스마트 디바이스와 센서 디바이스들이 IP 기반의 네트워크에 연결이 게이트웨이와 같은 중간 장치가 없이 가능하게 되었다. 하지만, 무선 센서 네트워크에서는 제한된 패킷 사이즈와 제한된 전력 사용으로 인해, 헤더 압축과 어드레스 자동 설정 기능 등이 필요하다. 이를 위해 6LoWPAN IETF 워킹 그룹이 만들어져 IPv6 와 IEEE 802.15.4 와의 관련 연구가 수행 중이다.

표 3 무선통신 종류

Technology	Radio
ZigBee1	IEEE802.15.4 @ 868MHz (EU), 915MHz (US), 2.4GHz (global)
ISA100.11a	IEEE802.15.4 @ 2.4GHz
WirelessHART	IEEE802.15.4 @ 2.4GHz
Bluetooth2	2.4GHz
Wifi3	IEEE802.11v@ 2.4GHz/5GHz/60GHz
NFC	13.56MHz
ANT+	2.4GHz
Nike+	2.4GHz
Z-Wave	ITU-T G.9959: 868.42MHz(EU), 908.42MHz(US)
EnOcean	315MHz/868.3MHz/902 MHz
Insteon	869.85MHz(EU) and 915MHz(US)
WirelessHD	60GHz

2.6 서비스 인터페이스 기술

IoT 서비스 인터페이스는 IoT 의 주요 구성요소인 인간과 사물과 서비스에 대해 특정 기능을 수행하는 응용서비스와 연동하는 역할을 한다. 따라서 IoT 서비스 인터페이스는 네트워크 인터페이스의 개념이 아니라, 정보를 센싱, 가공/추출/처리, 저장, 판단, 상황 인식, 인지, 보안/프라이버시 보호, 인증/인가, 디스커버리, 객체 정형화, 온톨러지 기반의 시맨틱, 오픈 센서 API, 가상화, 위치확인, 프로세스 관리, 오픈 플랫폼 기술, 미들웨어 기술, 데이터 마이닝 기술, 웹 서비스 기술, 소셜네트워크 등, 서비스 제공을 위해 인터페이스(저장, 처리, 변환 등)역할을 수행한다.

예를 들어 데이터마이닝의 기술은 가공되지 않은 방대한 양의 데이터를 기반으로 사용자의 수요에 따라 재조합 및 재구성하여 그에 부합하는 의미 있는 정보를 새롭게 취할 수 있도록 한다. 즉, 서로 연관성이 없어 보이는 무질서한 기존 비가공 데이터를 분류 및 가공, 처리함으로써 상황에 맞는 의미 있는 정보를 추출할 수 있다.

2.7 OS 와 소프트웨어

칩셋 영역은 디바이스 내부에 존재하는 것이다. 기존 임베디드 전용 칩셋을 생각하면 된다 현재로선 ARM 아키텍처의 코어텍스 M 시리즈가 각광받고 있다. 인텔이 쿼크를 통해 IoT 시장에 x86 아키텍처를 밀어넣기 위해 막 출사표를 던진 상태다. 이밖에 브로드컴, 프리스케일 같은 칩셋 제조업체들도 IoT 용 칩셋을 공급한다.

모듈은 일종의 메인보드 같은 것으로 보면 된다. 기본적으로 IoT 기기는 그 크기가 작기 때문에 보드도 소형이다. 최근엔 아두이노나 라즈베리파이 같은 메인보드격 제품에 센서나, 이더넷, 카메라, 블루투스 같은 기능을 추가하기 위한 여러 모듈이 나오고 있다.

칩셋과 모듈을 이용해 하나의 기기로 만들면 IoT 디바이스가 된다. 일상생활에서 가장 쉽게 접할 수 있는 스마트폰 역시 일종의 IoT 디바이스로 볼 수 있다. 그 디바이스에 어떤 소프트웨어를 심느냐에 따라 용도와 성격이 달라진다. 디바이스의 종류는 무궁무진하다. 웨어러블 컴퓨팅으로 거론되는 구글글래스나 나이키 퓨얼밴드, 제조업 생산자동화기기, 자동차 같은 기기부터 인류 주위의 모든 물건이 통신만 가능하다면 IoT 디바이스가 된다.

이 디바이스를 작동하게 해주는 소프트웨어가 당연히 존재한다. 임베디드 소프트웨어가 IoT 디바이스 구동에 주로 사용된다. 운영체제(OS)로 보면 안드로이드와 iOS, 윈도모바일, 리눅스가 해당된다. 윈드리버, MDS 테크놀로지 같은 회사의 RTOS 도 포함된다. 여기에 오라클 자바 ME-E 같은 미들웨어가 필요하다.

IoT 디바이스를 위한 SW 는 무엇보다 경량이란 점이 중요하다. 전력소모를 최소화하면서도, 저렴한 하드웨어로 고성능의 처리를 수행하도록 하는데 초점을 맞춘다. 가령, 오라클 자바 ME-E 의 경우 RTX OS 와 결합돼 128MHz 수준의 낮은 클럭수 칩셋과 2MB RAM 으로 고성능을 낼 수 있다, 이렇게 만들어진 기기가 스마트워치 같은 UI 를 제공할 수 있다.

2.8 IoT 보안

(1) IoT 보안 개요

수많은 지능형 사물이 인터넷에 연결돼 서로 통신하는 환경이 점차 현실화되면서 '사물인터넷(IoT)' 보안 위협에 대한 경고가 나오기 시작했다. 인터넷에 연결된 기기와 서비스가 다양해지고, 여러 분야로 확장되면서 보안 취약성과 보안 위험성이 커질 것이란 우려가 커지고 있다.

광범위한 IoT 기기들과 플랫폼, 연결된 네트워크에서 보안신뢰성을 제대로 확보하지 못한다면, 우리 주변의 모든 것이 공격 대상이자 도구가 될 수 있기 때문이다. 사람들이 사용하는 기기나 자동적으로 서로 통신하는 다양한 사물, 네트워크로 연결된 시스템에 불법 침입해 개인정보를 유출하고 도청하고 이상동작을 일으키거나 금전을 빼돌리는 피해가 발생할 수 있다.



그림 21 IoT 환경에서 보안위협 예(http://www.dt.co.kr/)

모든 것이 연결된 환경에서는 피해 대상과 범위가 상상 이상으로 광범위해질 수 있다. 사이버공격이 개인이나 기업을 넘어 사회나 국가 전체를 위협할 수 있다는 것은 그동안 충분히 경험했다. 보안업체인 시만텍은 올해 주목해야 할 주요 보안 위협요소 가운데 하나로 'IoT'를 지목했다. 시만텍은 인터넷에 연결된 모든 기기가 보안에 취약할 수 있다는 점을 인지해야 하며, IoT 의 확산으로 등장할 새로운 보안위협에 경각심을 가져야 한다고 주문하고 있다. 이같은 IoT 보안위협에는 각종 첨단기기나 애플리케이션의 보안취약점을 노린 공격뿐만 아니라 시스템상의 오류로 인한 개인정보나 사생활 침해도 포함된다.

(2) 사물 인터넷이 보안에 있어 취약한 이유

사물 인터넷은 여러 가지 요소기술이 통합되어 특정 서비스를 구성하기 때문에 각 요소 기술 자체의 보안 취약성과 연동 시 새로운 보안 취약성 발생할 가능성이 매우 높다. 즉 사물 인터넷은 정보를 센싱하기 위한 센서 기술과 센싱된 정보에 대한 원활한 통신/네트워킹을 위한 기술, 사물인터넷 디바이스 자체를 위한 칩 기술, 기능 구현을 위한 OS 기술/임베디드 시스템 기술, 디바이스의 자율 동작과 지능적 동작을 위한 플랫폼 기술, 대량의 데이터를 처리하는 빅데이터 기술, 유용한 정보 추출을 위한 데이터 마이닝 기술, 사용자 중심의 사물 인터넷 서비스를 위한 웹 서비스/응용 서비스/WoT(Web of Things) 기술 등 다양한 형태의 기술이 사용되어 사물과 사람, 서비스가 통합된 사물인터넷 서비스가 실현된다.

이를 구성하는 각 요소 기술의 보안 기술은 디바이스나 OS 기술, 통신/네트워킹, 웹 서비스/응용 서비스 등에서는 기밀성, 무결성, 인증, 접근제어, 해킹 방지, Anomaly 탐지/대응 등 다양한 보안 기술이 많이 개발되어 사용되고 있다. 또한, 데이터마이닝이나 빅데이터 기술 분야에서는 프라이버시 보호중심으로 다양한 기술이 개발되어 사용되고 있다.

한편, 사물 인터넷은 언급한 것처럼 다양한 기술의 복합적 특성 외에, 사물 인터넷 서비스가 기존의 서비스와 다른 특성을 가진다. 즉, 기존의 응용 서비스는 단일 사업자가 주도하는 vertical application market 이었지만, 사물 인터넷 서비스는 다양한 주체가 공존하는 horizontal market 특성을 가진다.

이때문에 여러 주체 간 프라이버시 보호 책임과 권한 선정의 어려움이 생긴다. 즉, 사물 인터넷 환경은 센서 디바이스 공급자와 통신/네트워크 공급자, 서비스 개발자, API 개발자, 플랫폼 공급자, 데이터 소유자 등 다양한 주체가 존재하기 때문에 보안/프라이버시 보장이 어렵게 되는 것이다. 물론 프라이버시를 보장하는 기술은 프라이버시 보호를 위한 법이나 체계에서 프라이버시 보호를 위한 범위 및 대상을 명확히 정의해야 하는데, 현재 개인정보보호법(한국)과 Consumer Protection/Data Protectin Act(미국)에서는 사물 인터넷 환경에 적용 가능한 구체적인 프라이버시 보호 체계가 정의되지 않았다는 문제가 존재한다.

(3) 사물 인터넷 보안취약성 사례

본 절에서는 시중에 나와 있는 사물 인터넷 제품(서비스)에 대한 보안/프라이버시 침해 상황을 분석함으로서 사물 인터넷 보안 취약성을 이해하고자 한다.

아래 [그림 22]처럼 체중계가 인터넷에 연결되고 각종 분석 기능과 다양한 서비스와 연동이 가능한 사물인터넷 제품/서비스 사례를 보여주고 있다. 사용자가 측정한 체중 정보는 서비스 제공업체의 클라우드에 전송되며 해당 정보를 사용자는 스마트폰과 같은 단말에서 앱을 통해 확인할 수 있다.

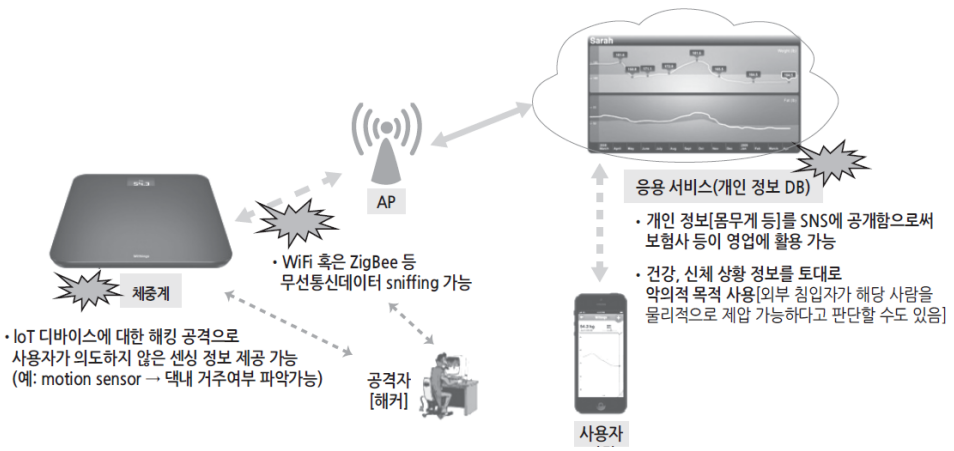


그림 22 사용자 데이터의 탈취

언급한 체중 정보 흐름은 단순해 보이며 서비스도 단순해 보이지만 개인의 체중 정보는 매우 다양한 형태의 서비스에 활용될 수 있다. 예를 들어, 어떤 사물인터넷용 체중계는 측정한 개인의 체중 정보를 개인의 선택에 따라 트위터와 같은 SNS 에 업로드할 수 있다.

이는 공개된 정보이므로 보험회사에서 해당 정보를 활용하여 사용자에게 맞춤형 보험 상품 판매 영업을 할 수 있다. 또한, 이미 보험 가입자라면 체중의 변화에 따라 건강 위험도의 변화로 인식하여 자사의 보험 손실을 우려하여 해당 사용자에 영향력을 미치고자 할 수도 있을 것이다. 아울러, 서비스업체에 저장된 개인의 체중 정보가 악의적인 목적으로 사용될 수도 있으며, 서비스 업체가 자사의 이익을 위해서 분석/가공을 통해 회사 입장에서는 부가가치가 높은 다른 정보로 변환할 수도 있을 것이다.

어떤 경우에든 원래의 데이터 소유 주체의 의사에 반하는 형태로 데이터가 활용되는 것이므로 프라이버시 침해가 발생하게 되는 것이다.

그런데 만약 이런 상황에서 체중계 사용자(체중 정보 소유자)가 자신의 정보에 대한 통제권을 확보하고자 하는 경우, 현재의 사물 인터넷 서비스 환경에서는 쉽게 자기정보 통제권을 얻을 수 있을지 의문이다. 예를 들어, 만약 업체에서 자사가 보유하고 있는 정보를 가공하여 제 3 의 업체에 판매한 경우, 판매 이후에는 정보를 제공한 업체에서는 해당 사용자의 정보를 통제할 수 없을 것이며, 가공된 정보는 원래의 정보라고 볼 수 없기 때문에 최초의 체중 정보 소유자가 이 경우에 해당 정보에 대한 통제권을 가진다고 볼 수도 없게 된다. 지금까지 단일 업체에 의한 정보 처리 및 관리가 이뤄졌고 사물인터넷 환경처럼 여러 주체가 관계되는 경우는 없었기 때문에 기존의 프라이버시 관련 법과 체계는 새로운 사물인터넷 환경에 맞게 정비될 필요가 있다.

언급한 프라이버시 침해 가능성뿐만 아니라 체중계에서 센싱되어 무선 통신채널로 전송되는 정보를 악의적인 공격자가 가로챈다면(sniffing), 공격자는 해당 정보를 활용하여 사용자의 신체적 특성을 유추하여 이를 악의적인 목적에 사용할 수 있을 것이다. 많은 사물인터넷 디바이스와 플랫폼에는 접근 제어, 인증/인가 기술에 있어서 높은 등급의 기술이 구현되지 않기 때문에 상대적으로 공격에 취약하다. 만약 체중계에 motion sensor 가 있다면 공격자는 취약한 보안 체계를 뚫어서 해당 센서에 대한 제어권을 확보하여, 개인의 체중 정보뿐만

아니라, 댁내 거주 여부도 쉽게 알 수 있게 된다. 이처럼 사물 인터넷 제품과 서비스에서는 각별히 보안과 프라이버시 침해 문제를 살펴볼 필요가 있다.

(4) IoT 보안을 침해하는 방법들

▪ DoS 공격

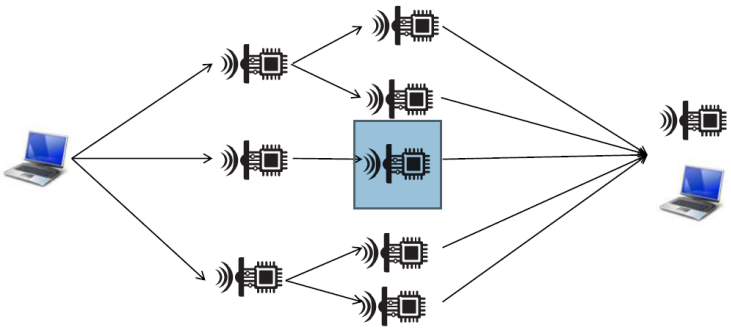


그림 23 Dos 공격 모형

공격대상이 수용할 수 있는 능력 이상의 정보나 사용자 또는 네트워크의 용량을 초과 시켜 정상적으로 작동하지 못하게 하는 공격, 자동화 툴을 이용하여 짧은 시간에 여러 곳에서 일어나게 하는 공격하며, 공격자의 위치와 구체적인 발원지 파악하는 것이 어렵다.

▪ 스푸핑 공격

IP 주소, 호스트 이름, MAC 주소 등 여러 가지를 인위적으로 만들어 낼 수 있으며, ARP 스푸핑, IP 스푸핑, DNS 스푸핑 등 공격이 있다. 무선 네트워크 상에 존재하는 모든 연결에 스푸핑이 가능하며, 정보를 얻어내는 것으로 시스템을 마비시키는 것도 가능하다. IP 충돌 문제 역시 고의가 아닌 IP 스푸핑이라고 추측할 수 있다.

▪ 하이제킹 공격

이미 인증을 받아 세션을 생성하여 유지하고 있는 연결을 여러 가지 방법으로 빼앗는 공격으로 마스터와 슬래이브 간의 통신을 모니터링한다. 인증에 대한 문제점을 해결하기 위해 도입한 One Time Password, 토큰 기반 인증(Token Based Authentication: Kerberos)을 이용한 세션의 가로채기도 가능하다.

대책으로는 데이터 전송의 암호화, 지속적인 인증 필요(일정시간이 되면 세션을 무효화 하고 재 인증을 받아 유효한 사용자인지 확인하는 방법이 있다.

▪ 스니핑

고의적으로 자신이 원하는 정보를 데이터 속에서 찾는 것으로 막는 것도 어렵고, 탐지 역시 쉽지 않다. A 와 B 는 데이터를 주고 받는데, 암호화되어 있지 않다면 A,B 간의 송수신되는 데이터를 스니핑한 후에 데이터의 내용을 분석하여 악용하는 방법이다.



그림 24 스니핑

표 4 보안을 위협하는 여러가지 공격유형

보안취약성 및 공격 유형	IoT 상에서의 대상 분야
Worm과 virus	IoT 통신/네트워크, 디바이스, 게이트웨이, 플랫폼, 응용 서비스
DoS 및 분산 DoS	IoT 통신/네트워크
비인가된 접근	IoT 디바이스, 게이트웨이, 플랫폼, 응용 서비스
패치되지 않는 시스템 OS, OS 보안 취약성	IoT 디바이스, 게이트웨이, 플랫폼, 응용 서비스
Antivirus 소프트웨어의 부적절한 사용	IoT 플랫폼, 응용 서비스
방화벽의 부적절한 사용	IoT 통신/네트워크
비인가된 서비스 접근	IoT 응용 서비스
프로토콜 보안 취약성	IoT 통신/네트워크
비인가된 사용자의 접근	IoT 응용 서비스
복제 공격	IoT 디바이스, 게이트웨이
비인가된 I/O 접근	IoT 디바이스, 게이트웨이, 플랫폼, 응용 서비스

부적적한 시스템 로그 기록	IoT 플랫폼, 응용 서비스
설정 오류 및 실수	IoT 디바이스, 게이트웨이, 플랫폼, 응용 서비스
기밀성/무결성 공격	IoT 통신/네트워크, 디바이스, 게이트웨이, 플랫폼, 응용 서비스
안전하지 않은 패스워드	IoT 응용 서비스
보호되지 않은 펌웨어	IoT 디바이스, 게이트웨이
프라이버시 침해	IoT 플랫폼, 응용 서비스

(5) IoT 보안 방안

▪ 방화벽

방화벽의 기본 역할은 신뢰 수준이 다른 네트워크 구간들 사이에 놓여서 신뢰 수준이 낮은 네트워크로부터 오는 해로운 트래픽이 신뢰 수준이 높은 네트워크로 오지 못하게 막는 것이다. 흔히 네트워크 관리자의 입장에서 높은 신뢰도를 갖는 구간은 내부 네트워크 구간이라 하고, 낮은 신뢰도를 갖는 구간을 인터넷 구간 또는 외부 네트워크 구간이라고 한다. 이 밖에도 외부에 서비스를 제공하는 서버들을 위한 DMZ 구간이 있으며 인터넷으로부터 내부 네트워크로의 침입을 막는 동시에 내부 네트워크에서 인터넷과 자유롭게 통신할 수 있도록 도와 준다. 대부분의 방화벽은 정책 기반의 방화벽이며 다양한 수준의 정책으로 네트워크 간의 트래픽을 제어한다.

- 접근제어(Access Control)
- 로깅과 검사(Logging & Auditing)
- 사용자 인증 (Authentication)
- 데이터 암호화 (Data Encryption & Decryption)

이러한 네트워크 보안을 해결하기 위한 솔루션은 Secure Coding, Cryptography Algorithm 이 있으며, 간단하게 살펴보고 이장을 마친다.

(6)Secure Coding

Secure Coding 이란 SW 개발 과정 중 소스코드 구현(개발) 단계에서 해킹 등의 공격을 유발할 가능성이 있는 잠재적인 보안 취약점을 사전에 제거하여, 외부 공격으로부터 안전한 소프트웨어를 개발하는 기법을 말한다. SW 의

문제점들, 버그나 로직의 결점들은 일반적으로 개발 단계에서 SW 취약점을 가진 채로 개발되기 때문이라는 점을 기록들을 통해 알 수 있다. 수 천 개의 취약점 보고서 분석을 통해 SW 안정성 분야 전문가들은 대부분의 취약점들이 적은 수의 평범함 SW 프로그램 에러에 기인하고 있다는 점을 발견했다. 이런 문제점들을 발생시키는 좋지 못한 코딩 사례를 발견해내는 작업과 대안이 되는 좋은 사례들로 개발자들을 교육하는 작업을 통해, 조직은 SW 가 취약점을 가진 채로 배포되기 젂에 그 취약점들을 눈에 띄게 줄이거나 제거할 수 있게 도움을 주도록 하는 것이다. - 위키피아 번역

CERT(Computer Emergency Response Team) 란 1980 년 후반 DARPA 에 의해 구성되었고, 컴퓨터 해킹사고가 빈발함에 따라 컴퓨터 통싞망을 해커로부터 보호하기 위해 결성된 통신보안 전문가 그룹이다. 우리나라에서는 1995 년 CERT-korea 설립(인터넷침해대응센터)으로 시작되었다.

(7)암호화 알고리즘의 종류

암호화 알고리즘의 종류는 다양하나 IoT 와 임베디드 시스템에는 AES, SHA, RSA 등이 많이 사용되고 있다. 종류를 간략히 살펴보자.

1) AES(Advanced Encryption Standard)

대칭형 암호 방식이며 AES 암호 알고리즘은 평문을 128bit 단위로 나누어 암호화, 복호화를 수행하며, 각각 의 128bit 를 4X4 행렬로 표현하여 연산을 수행한다.

2) SHA(Secure Hash Algorithm)

2004 년 2 월에 삼중 DES 의 키 길이에 맞춰 해시값 길이를 조정한 SHA-224 가 표준에 추가되었다. SHA-256 과 SHA-512 는 각각 32 비트 및 64 비트 워드를 사용하는 해시 함수이며, 몇 몇 상수들이 다르긴 하지만 그 구조는 라운드의 수를 빼고는 완전히 같게 구성되었다. SHA-224 와 SHA-384 는 서로 다른 초기값을 가지고 계산한 SHA-256 과 SHA-512 해시값을 최종 해시값 길이에 맞춰 잘라낸다.

3) RSA(Ron Rivest, Adi Shamir, Leonard Adleman)

비대칭형 암호 방식으로 A 가 B 에게 암호화된 메시지를 전달하는 상황을 가정할 때, 비대칭형 암호 방식에서는 A 가 B 의 공개키를 이용하여 암호화 메시지를 전달하고 B 는 자신의 비밀키를 이용하여 원래 메시지로 복호화 하게

된다. 암호화된 메시지를 전달하고 싶은 모든 사람에게 공개되고 비밀키는 복호화를 하는 자신만 가지고 있는 키 비 대칭형 암호 방식의 대부분의 단점은 시간이 대칭형 암호 방식에 비해 많이 소요된다.

2.9 Reference

[1] 사물인터넷을 이용한 스마트홈 시스템 - 사회문화조사실 과학 방송통신팀 (2014. 2)

[2] Internet of Things : Smart Device as a Service - 김학용 사물인터넷 전략연구소

[3] 사물인터넷(IoT)분야의 주요 임베디드 SW 기술 - MDS 테크놀로지㈜ 장성균

[4] 스마트 디바이스와 사물인터넷 (IoT) 융합 기술 동향 - ETRI 2013.

[5] 인터넷 환경에서의 보안/프라이버시 이슈 - 김호원 부산대학교 정보컴퓨터공학부 교수, TTA Journal Vol.153

Chapter 03. IoT 센서와 액츄에이터

이 장에서는 IoT 의 3 대 기술중의 하나인 센서에 대해 알아본다. 센서의 기능에 대해 알아보고 활용도와 현재 발전모습을 알아보고 센서 실습까지 하도록 한다.

3.1 IoT 스마트센서 개념

스마트센서의 대량생산을 사물인터넷에 대한 관심이 관심이 높아짐에 높아짐에 따라 센서에 수요증가가 기대되면서 기대되면서 센서에 대한 관심 또한 높아지고 높아지고 있다. 전통적인 온도/습도/열/가스/조도/초음파 센서 등에서부터 원격 감지, SAR, 레이더, 위치, 모션, 영상 센서 등 유형 사물과 주위 환경으로부터 정보를 얻을 수 있는 물리적 센서를 포함하여 여러 센서들이 한데 결합한 복합 센서까지 다양해져 가고 있다.

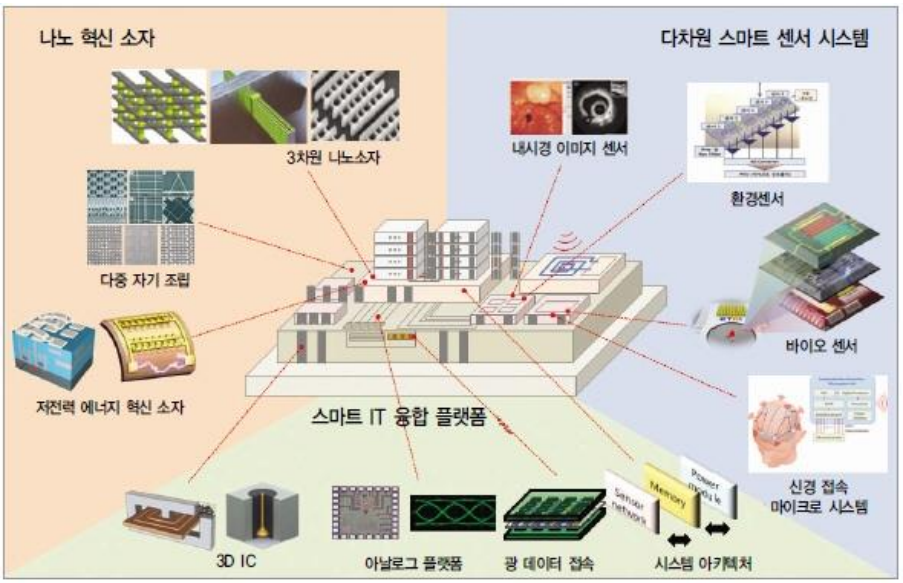


그림 25 스마트센서의 전망
(https://www.hellot.net/magazine/magazine_print.php?idx=676)

또한 물리적인 센서는 응용 특성을 좋게 하기 위해 표준화된 인터페이스와 정보 처리 능력을 내장한 스마트 센서로 발전하고 있으며, 또한 이미 센싱한 데이터로부터 특정 정보를 추출하는 가상 센싱 기능도 포함되기도 한다. 가상

센싱 기술은 실제 IoT 서비스 인터페이스에 구현되어 기존의 독립적이고 개별적인 센서보다 한 차원 높은 다중(다분야) 센서기술을 사용하기 때문에 한층 더 지능적이고 고차원적인 정보를 추출할 수 있다.

표 5 산업별 활용도가 높은 주요 센서

소비 /가전	온도/습도 센서 진동/소음 센서 접촉/터치/압력 센서 마이크로폰, 카메라 근접/조도/색감 센서 가속도/지자기계/자이로스코프 IR 동작/체온 센서 등	**자동차**	흡/배기 가스 센서 유량 센서 공기압 센서 온도/습도 센서 마이크로폰, 카메라 레인센서, 레이다 등
의료 /건강	체온계, 혈압계 심전도, 심박 센서 뇌파 센서 당뇨 센서 지문, 홍체 인식 센서 등	**가정 /빌딩**	CO2, 미세 먼지, 황사 감지 가스/연기/화재 감지 카메라 적외선, 음향, 진동 감지 동작, 압력 센서 등
항공	항법 장치(GPS 등) 기온/습도/풍속/풍향 센서 RF/Laser 기반 고도센서 등	**도시 /환경**	기압/온도/습도 일조량/적외선/자외선 황사/먼지/오존 감지 센서 농약, 독성 검출 등

스마트센서는 통신기술, 플랫폼과 함께 사물인터넷을 구성하는 3 대 기술요소의 하나이며, 센서의 종류 는 감지대상(힘, 온도, 가스, 혈당 등)이나 구현방식 (반도체 , MEMS-Micro Electro Mechanical Systems, 나노 , 융.복합 등), 적용분야 (자동차, 모바일, 가전, 국방 등)에 따라 다양한 방식으로 구분될 수 있다. 스마트센서는 주로 MEMS 방식으로 구현되어 단순한 단순한 감지를 넘어 판단 및 제어의 기능이 포함된다. 여기에 블루투스 등의 통신기술이 융합된 다수의 스마트센서가 전송하는 경우를 유비쿼터스 센서네트워크라고 불리기도 한다.

스마트센서의 대량생산을 위해서는 플랫폼화가 필요하며, 핵심요소는 수명과 가격이다. 플랫폼화는 스마트센서가 설치되는 위치와 감지대상의 특징(예를 들어 전염병, 방사능, 오염가스의 계측은 등은 모두 공기환경을 감지하는 센서를 필요로 함) 등의 공통요소를 반영해 하나의 반도체 회로와 연결하여 생산할 수 있도록 하는 것을 의미한다.

스마트센서의 수명은 교량이나 건물의 안전진단에 적용되는 스마트센서의 경우 센서자체의 수명이 100 년은 되어야 하며, 자동차에 적용되는 경우 자동차 수명인 15 년은 되어야 한다. 낮은 가격을 위해서는 스마트센서에 요구되는 여러 기능들이 하나의 작은 모듈 안에 구성되어야 하며, 이를 위해 3 차원 반도체 기술이 요구된다.

3.2 센서의 기초

센서(Sensor) 란 외부자극을 받아 이것을 전기신호로 변환하는 소자를 말한다. 여기서 외부자극이란 검출 또는 측정하고자 하는 양, 특성, 상태, 변화를 의미한다. 전기신호로 변환하는 기능은 트랜스듀서 (Transducer)라고 하는데 한 에너지 형태(신호)를 다른 에너지 형태(신호)로 변환하는 소자를 말한다. 특히 센서데이터를 전기신호로 변환하는 이유는 입력단으로의 피드백, 잡음이나 간섭을 제거하기 위한 필터링, 극소 시간으로 나누는 미분, 데이터로 저장, 원거리 정보전송 등의 많은 중간 단계를 이루는 신호처리를 쉽게 하기 위함이다.

표 6 센서 동작에 관한 에너지 형태와 인간의 오감의 유사성

에너지 형태	센서에 이용되는 특성	인간의 오감
기계 에너지	위치, 속도, 가속도, 힘, 토크, 압력, 응력, 변형, 유량, 질량, 밀도, 모멘트, 변위, 형상, 방위, 점도	청각, 촉각
복사 에너지(빛)	복사강도, 에너지, 파장, 진폭, 위상, 투과율, 편광	시각, 촉각
자기 에너지	자계세기, 자기 모멘트, 투자율, 자속밀도	
열 에너지	열, 온도(수치), 열속(흐름)	촉각
화학 에너지	농도, 반응율, 산화환원전위, 생물학적 특성	후각, 미각
전기 에너지	전압, 전류, 저항, 정전용량, 주파수	

3.3 센서의 기본적인 특성과 용어 설명

센서의 성능이 입력과 출력이 정확한 관계를 가지며 일정하게 유지되는것을 목표로 하지만 현실적으로 불가능하다. 따라서 센서의 정특성(static characteristics)과 동특성(dynamic characteristics)을 확인하여 적당한 수준의 데이터를 수집하는 방법으로 테스트 한다. 잠시 센서에서 자주 쓰이는 용어에 대해 간단하게 살펴본다.

▪ 감도

입력량에 대한 출력량의 비율을 말하며 보통 그래프로 나타낸다.

▪ 동작범위와 풀 스케일 출력

동작 범위란 센서의 입력량의 범위에 따른 출력값의 제한 정도를 말한다. 보통 센서는 특성상 무한대의 입력에 대해 무한대의 출력이 나오지 않으므로 동작법위는 반드시 지정되어 있다. 풀 스케일 출력이란 최대 입력시 출력과 최소 입력시 출력의 차를 말한다.

▪ 분해능

검출 가능한 최소 입력량 (최소입력증분)을 말하며 센서의 검출 정도, 즉 해상도를 의미한다. 센서 자체이서 흡수하거나 잡음으로 상쇄되는 비율을 정의한다. 분해능은 작을 수 록 센서의 성능은 좋아지는 경향이 있다.

▪ 감도오차와 오프셋

감도오차(감도변동)이란 센서의 입출력 특성의 기울기가 이상(정상)적인 직선의 기울기로부터 벗어나는 것을 말한다.

▪ 직선성

센서의 출력은 직선성을 유지해야 바람직하지만 현실적으로 불가능하며 대부분의 센서 출력은 입력이 상한을 초과하면 직선성을 잃고 비 직선으로 된다.

▪ 히스테리시스

입력값 x 를 증가시켜 가면서 출력을 측정할 때와 감소시켜가면서 출력을 측정하였을 때 동일한 입력에서 출력이 같지 않는 현상을 말하며, 센서 재료, 혹은 센싱하는 물체의 특성으로 인해 나타난다.

▪ 확도와 정도

- 확도 : 참 값에 얼마나 가까운가. (근접도)
- 정도 : (계측)분포의 정확성

▪ 반복성

동일한 양을 동일조건하에서 동일방법으로 단기간에 연속 측정할 때 측정값들이 서로 얼마나 일치하는가 나타낸다.

▪ 재현성

동일한 양을 같은 방법으로 장기간에 걸쳐 측정하거나 다른 사람에 의해서 측정되거나 또는 다른 실험실에서 측정될 때 측정값 사이에 일치한 정도를 나타내는 것

▪ 선택성

센서는 원하는 물리 현상만을 검출하고, 다른 외부 요인의 현상의 영향을 받지 않는 것이 바람직하다.

▪ 잡음

원하지 않는 불규칙한 신호에 대해 센서는 반응을 최소화 해야 한다. 신호대 잡음비를 향상 시킴으로서 검출 하한치를 낮추고 필터를 사용함으로 개선한다.

▪ 출력 임피던스

전류출력 센서에서 출력 임피던스의 크기는 커야하고 입력임피던스의 크기는 작아야 한다. 전압출력 센서에서 출력 임피던스의 크기는 작아야 하고 입력 임피던스의 크기는 커야 한다.

▪ 환경특성

환경 파라미터 : 센서의 성능에 영향을 미치는 외부 변수 ; 온도, 습도 등과 같이 센서의 성능에 영향을 미치는 외부 변수들을 환경 파라미터(environmental parameter)라고 부른다.

3.4 융복합 센서 개발

(1) 융복합센서 개요

센서를 활용한 서비스 특화 디바이스들의 출시로 인해 IT 기술을 활용하는 산업 영역이 피트니스, 의료, 교육 등으로 빠르게 확산되고 있다.

센서 분야의 전문가들은 향후 10 년 내에 전세계에서 사용되는 센서의 개수가 1 조개에 다다르는 Trillion 센서 시대를 전망한다. 사용되는 센서의 개수가

이렇게 급증하지만 실제 시장에서 센서를 활용한 경쟁은 새로운 센서를 혁신적으로 개발하는 측면 보다는 제품, 공정 혁신을 통해 가격 경쟁력이 확보된 센서들을 복합적으로 조합해 새로운 가치를 만들어 내는 측면이 크다.

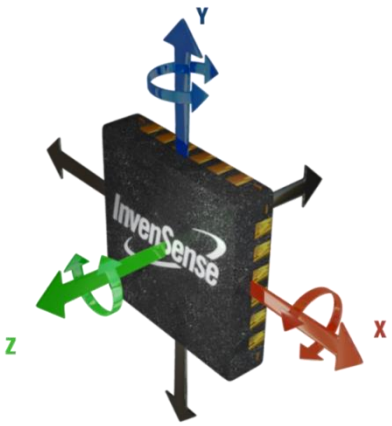


그림 26 9 축 지자기 센서

복합 센서를 통한 차별화된 기능 및 서비스의 구현이 산업의 핵심 경쟁력으로 부상할 것으로 예상 됨에 따라 센서 제조사, 디바이스, 클라우드, 서비스 제공자에 이르기 까지 다양한 기업들은 Value-Chain 상에서 각자의 핵심 경쟁력을 기반으로 서로 유기적으로 경쟁하며 협력하고 있다. 특히 디바이스 제조사들은 지문 인식 센서, 환경 센서 등을 빠르게 적용하는 동시에 다양한 하드웨어 Formfactor 를 통해 센서를 활용한 기기의 차별성을 높이고 있다. 또한 서비스 제공 기업들은 자신들의 서비스를 최적의 상태로 제공하기 위해 특화 센서를 활용하거나 다양한 센서가 탑재된 디바이스를 직접시장에 출시하고 있다. 기업들 간의 센서 활용을 통한 경쟁은 이미 시작되었다. 다양한 디바이스와 서비스의 출시로 IoT 시대가 열리고 있다.

(2) 복합센서 발전방향

센서를 활용한 IoT 디바이스, 서비스 구현이 급속도로 확산되고 있지만, 이러한 센서 활용의 확산은 기존에 존재하지 않았던 혁신적인 센서의 개발이 아닌 이미 기술적으로 구현 가능한 약 50 개 미만의 센서들이 복합적으로

사용되면서 다양한 역할로 구현되고 있는 측면이 크다. 따라서 의료 분야와 같이 센싱의 정확도, 정밀도가 매우 중요한 일부 영역을 제외하면, 센서를 둘러싼 IoT 시대의 경쟁의 핵심은 가용한 센서를 복합적으로 활용한 센서 퓨젼(Sensor Fusion)을 통해 새로운 기능 및 서비스를 구현해 내는데 있다. 물론 사용되는 센서의 개수가 많으면 더욱 다양한 서비스를 구현해 낼 수 있을 것 같지만 복합 센서를 구동하는데 필요한 비용(배터리 소모, 컴퓨팅 오버헤드, 가격 상승 등)을 고려해야 한다.

즉 하나의 IoT 디바이스, 서비스에서 사용 가능한 센서들은 물리적으로 제한되어 있기 때문에 성능과 서비스 완성도를 위한 HW/SW 적 센서 퓨전 역량이 향후 경쟁의 핵심이 될 전망이다. 이미 기업들은 복합 센서의 효과적인 활용을 위한 연구 및 기술 구현을 시작하고 있다. 모바일 프로세서 시장의 주요 기업인 퀄컴은 다양한 센서를 저전력으로 제어하는 동시에 복합 센서 정보를 분석해 상황 정보(Context Information)로 처리하는 'Gimbal Project'을 진행 중이다. 예를 들면 스마트폰에 탑재된 GPS, 통신 모듈(WiFi, 3G/LTE), 모션센서, 기압계 등의 정보를 종합하면 사용자가 실내에 있는지, 몇 층에 있는지, 어느 방향으로 움직이고 있는지 등의 정보를 유추할 수 있다. 디바이스 업체들 또한 이러한 상황 정보 분석을 위한 기술 구현에 노력하고 있다. 특히 이들은 하나의 디바이스에 탑재되는 센서의 개수가 증가함에 효율적으로 센서를 'On/Off' 하기 위한 'Sensor Hub' 기술 개발에도 노력 중이다. 모든 센서가 항상 'On' 상태로 동작한다면 주변 상황 변화를 즉각적으로 감지하여 반응할 수 있겠지만, 스마트폰/Wearable 과 같은 모바일 디바이스는 배터리의 제약으로 인해 적절히 센서들의 구동 주기(Duty Cycle)를 조절해야 하기 때문이다.



그림 27 복합 센서를 활용한 서비스 영역 확대 사례: 위치 정보 서비스 고도화

애플은 디바이스 내에 복합 센서를 활용하는 동시에 디바이스 외부에 센서 인프라를 구축 하고 있다. 도로, 건축물 등에 애플 디바이스에 정보를 전달 할 수 있는 작은 모듈(iBeacon)을 설치해 사용자의 위치 변화에 따라 상황에 맞는 정보, 서비스를 제공하려는 시도이다. 이미 애플은 작년부터 미국 메이저리그 구장에 'iBeacon'을 설치, 테스트를 하였고 샌프란시스코 구장을 시작으로 본격적으로 서비스를 제공하고 있다. 예를 들면, 아이폰 사용자가 'iBeacon'이 설치 된 구장에 들어가게 되면 아이폰은 자동적으로 야구 경기와 관련된 정보, 앱을 제공하게 된다. 즉 그동안 사용자가 원하는 정보를 얻기 위해 겪었던 검색의 번거로움과 시행착오를 'iBeacon'과 같은 센서 인프라를 통해 덜어 주는 것이다. 이는 과거 개발자들에게 앱 개발 환경을 구축해 주며 앱 스토어의 성공을 이끌었던 것과 마찬가지로, 물리적인 콘텐츠/서비스 제공환경을 구축하는 움직임이며 향후 IoT 시대에 다양한 영역의 서비스와 접목될 가능성이 높기 때문에 그 확장성과 잠재력이 무한하다.

이와 같이 센서를 통한 IoT 시대는 IT 산업뿐만 아니라 비 IT 영역에서의 센서 기반의 IT 기술 활용이 급속히 확산되며 가속화 되고 있다. 센서는 인간의 인지 능력 및 입력의 한계를 뛰어 넘어 광범위한 영역의 정보를 정확하게 수집하고 있으며 모바일 컴퓨팅과 클라우드 환경과 같은 관련 기술을 응용하여 과거에 시도되었던 M2M/IoT 와는 달리 실체화된 모습으로 제공되고 있다. 기업들 간의 센서를 활용을 통한 경쟁은 이미 시작되었고 다양한 서비스,

디바이스들로 그러한 경쟁의 결과물 들이 사용자에게 다가오면서 IoT 시대가 본격화 되고 있다.

3.5 Actuator

액츄에이터(actuator)란 신호처리기에 의하여 다듬어져 나온 신호에 대응하여 동작을 하는 장치 또는 수단이다. 입력된 신호에 대응하여 작동을 수행하는 작동기(作動器), 또는 명령신호에 따라서 작동하는 집행기(執行器)이다.

즉 에너지 전달매체로부터 에너지를 전달받아 기계적 운동으로 바꾸어주는 장치를 통칭하여 액츄에이터라고 한다. 모터, 로봇의 동작기관, 솔레노이드 밸브, 실린더등이 가장 대표적인 액츄에이터라고 할 수 있다.

전기, 유압, 압축공기등을 사용하는 원동기의 총칭으로서, 보통은 유체에너지를 이용하여 기계적일을 하는 기기를 말한다. 액츄에이터를 분류하는 방법에는 작동형태별 분류 (왕복형, 회전형), 동력원별 분류 (공압, 유압, 전기)가 있다.

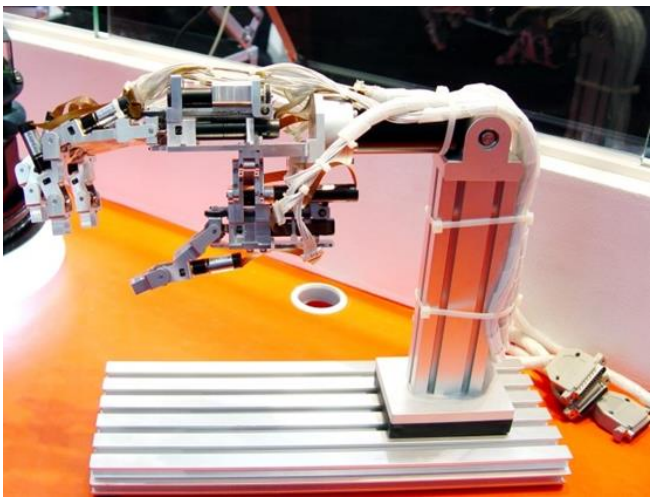


그림 28 대표적인 로봇 액츄에이터

엑츄에이터의 요구사항으로는 1)안전성 및 운전성, 2)속응성, 3)용량, 4)소형경량, 5) 유지보수 용이, 6)수명과 가격과 같은 고려요소가 적당한 수준에서 선정해야 한다. 최근 액츄에이터의 기술 동향은 소형화, 경량화, 대용량화, 신소재 응용 등으로 많은 발전이 이루어지는 추세이다. 유압 액츄에이터는 유압실린더, 유압 모터, 유압요동 모터로 세분화 된다. 공압

액츄에이터는 공압실린더, 유압 모터로 세분화 되며, 전기 액츄에이터는 회전형(D.C 서보모터, A.C 서보모터), 직선형(LINEAR 모터)으로 분류하고 있다. 동력 사용에 따라 장-단점이 다르므로 선정시 주의가 필요하다.

Chapter 04. Raspberry Pi

이 장에서는 스마트 홈 시스템의 Gateway 역할을 하는 라즈베리 파이에 대해 설명한다.

4.1 Raspberry Pi 란

라즈베리 파이(Raspberry Pi)는 영국의 라즈베리파이 재단(Raspberry Pi Foundation)에서 어린이들의 컴퓨터 및 과학 교육을 증진시키기 위해 만든 싱글 보드 컴퓨터 (Single-Board Computer)이다. 라즈베리파이에 대해 설명하기전에 우선적으로 싱글보드 컴퓨터에 대해 알아보자.

싱글보드 컴퓨터(Single-Board Bomputer)는 줄여서 SBC 라고 부르듯이 하나의 보드위에 CPU 및 메모리, 입출력 장치를 장착한 하나의 컴퓨터이다. 이는 흔히, 집이나 회사에서 많이 사용하는 데스크톱이 메인보드를 중심으로 각종 슬롯을 끼우는 형태와 구별되며, 노트북도 여러장의 보드가 결합된 형태를 띠고 있어서 이 또한 구별된다고 볼 수 있다. 한장의 보드로 구성되어 있기 때문에 관리하기 쉽고 성능대비 비교적 저렴하다는 것이 특징 중에 하나이다.



그림 29 Dyna-micro



그림 30 BBC Micro

그 역사도 상당히 오래되었다. 대략, 역사를 거슬러 올라가면 1976 년대부터 나왔다고 알려져 있다. 그 시초는 지금의 x86 보다 훨씬 오래된 인텔의 C8080A 및 eeprom 인 C8080A C1702A 가 장착된 "dyna-micro"라고 알려져 있다. 그 뒤로 가장 유명한 것은 라즈베리파이와 같은 고향 선배라고 볼 수 있는 영국의 "BBC Micro" 이다. 이 싱글보드 컴퓨터는 MOS Technology 의 2 MHz 6502/6512 8 비트 CPU, 16kb 의 메모리를 탑재하였다. 주 목적은 학교에서

컴퓨터 교육을 위해서 만들어졌으며, 영국 BBC 의 후원으로 영국의 컴퓨터 교육에 큰 역사의 획을 그었다고 알려져 있다. 그 명맥을 이어가고 있는 것이 바로 2012 년도의 라즈베리파이이다.

일단, 싱글 보드 컴퓨터는 크게 두 가지로 구분된다. ARM 코어, x86 계열 중 어떤 CPU 를 사용하고 있는가로 나눌 수 있다. 왜냐하면 수십년전 싱글보드 컴퓨터는 CPU 에 따라서 전용의 매우 간단한 펌웨어를 돌리기 위해서 많이 사용되었지만, 최근의 싱글 보드 컴퓨터는 어떤 OS 를 올릴 수 있는지가 매우 중요하기에 사용되는 CPU 명령어를 기준으로 ARM 코어, x86 계열으로 나뉘게 된다. 그 중 최근에는 오픈 소스 정책을 펼치고 있는 리눅스 계열 및 안드로이드를 지원하는 ARM 코어를 탑재한 싱글 보드 컴퓨터들이 속속 개발/판매되고 있다.

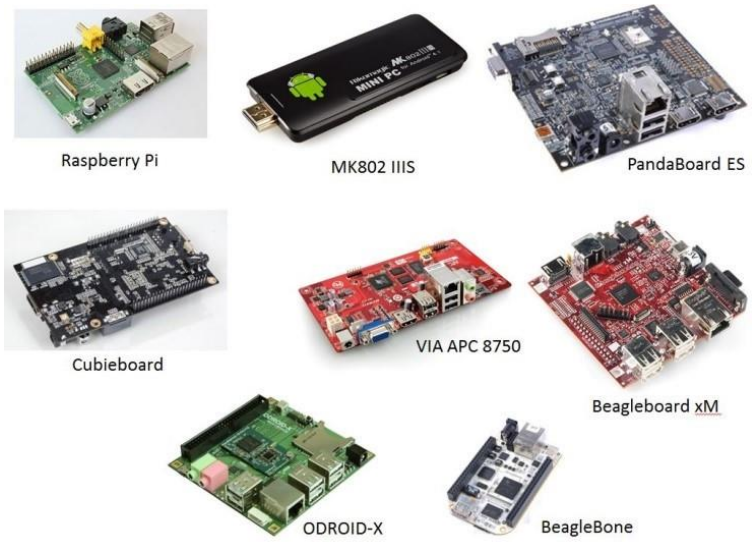


그림 31 싱글보드 컴퓨터

그리고, 아래의 Arduino Due 및 mbed 도 나오는데 이것 또한 싱글 보드 마이크로 컨트롤러 (Single-Board Microcomputer)기 때문이다. 싱글보드 컴퓨터와는 성능이 좀 떨어지나 목적이 제어용으로 나온것이라 싱글 보드 마이크로컨트롤러라 부른다. 하지만, 이것도 어느정도 작은 사이즈의 리얼타임계열의 OS 가 돌아가기에 SBC 와의 구분이 모호해진 상태이다.

마찬가지로 SBC 또한 범용 목적의 입출력 포트인 GPIO 를 지원하고 있기에 SBC 와 SBM 은 그 차이가 없어진 것 같다.

표 7 다양한 마이크로 컴퓨터 비교

제품명	가격($)	CPU	RAM	사이즈(cm)
Mbed LPC1768	59$	ARM Cortex-M3 (96MHz)	64KB	5.4*2.6
Arduino Due	59$	ARM Cortex-M3 (84MHz)	96KB	10.1*5.3
MK802IIIS	60$	DualCore 1.6GHz, ARM Cortex-A9	1GB	9.0*4.0
ODROID-X2	135$	QuadCore 1.7GHz, ARM Cortex-A9	2GB	9.0*9.4
BeagleBone Black	45$	1GHz, ARM Cortex-A8	512MB	8.6*5.3
Cubieboard	49$	1GHz, ARM Cortex-A8	1GB	10.0*6.0
Pandaboard ES	182$	DualCore, 1.2GHz, ARM Cortex-A9	1GB	11.4*10.1
Rasberry Pi B-type	35$	700MHz, ARM11	512MB	8.5*5.3

표 8 라즈베리파이 모델의 사양

제품사양(Raspberry Pi Model B+)	
Chip Broadcom	BCM2835 SoC
Core architecture	ARM11
CPU	700 MHz Low Power ARM1176JZFS
GPU	Dual Core VideoCore IV® Multimedia Co-Processor Provides Open GL ES 2.0, hardware-accelerated OpenVG, and 1080p30 H.264 high-profile decode Capable of 1Gpixel/s, 1.5Gtexel/s or 24GFLOPs with texture filtering and DMA infrastructure
Memory	512MB SDRAM
Operating System	Boots from Micro SD card, Linux operating system
Dimensions	85 x 56 x 17mm

Power	Micro USB socket 5V, 2A
Ethernet	10/100 BaseT Ethernet socket
Video Output	HDMI, Composite RCA (PAL and NTSC)
Audio Output	3.5mm jack, HDMI
USB	4 x USB 2.0 Connector
GPIO Connector	40-pin header: 27 GPIO pins
Camera Connector	15-pin MIPI Camera Serial Interface (CSI-2)
Display Connector	Display Serial Interface (DSI) 15 way flat flex cable
Memory Card Slot	SDIO

다음은 라즈베리파이 B+ 의 상세 보드 이미지이다.

표 9 라즈베리파이 Model B+ 상세사진(http://www.leocom.kr/RaspberryPi/)





라즈베리 파이의 응용분야는 센서, 모터, 광원 등의 하드웨어를 직접 제어하는 분야에 사용할 수 있으며, 어린이를 대상으로 하는 로봇들, 쿼드콥터, 출입감지 문 센서 등과 같은 다양한 물리적 컴퓨팅 프로젝트가 진행 중이다. 미디어센터로 활용하거나 게임 개발 플랫폼으로 이용이 가능하다. 강력한 멀티미디어 및 3D 그래픽 성능이 요구되는 환경에 적하다. 라즈베리파이의 매력은 매우 작은 범용 컴퓨터라는 점이다. 라즈베리파이를 활용한 많은 사례들을 살펴보자

- 다목적 컴퓨터로 이용이 가능
- 비디오 및 음악감상 전용 서버로 사용 가능

- 웹 서버로 사용
- 미니 게임기, 로봇, 장난감 제어기로 활용
- 카메라 모듈 연결시 방범용 CCTV, 사진 저장 가능
- SNS 에 글을 올리거나, 인터넷에 연결된 p2p 서버로 활용

4.2 Raspberry Pi Quick Start

(1) SD 카드에 운영체제 이미지 쓰기

공식 홈페이지(http://www.raspberrypi.org/downloads) 에 접속하여 Raspbian "wheezy"의 최신 버전인 2014-09-09-wheezy-raspbian.zip 를 다운로드 하도록 하자. 사이트를 방문하면 가장 최근에 공개된 운영체제 이미지 파일을 다운 받아 사용하면 된다.

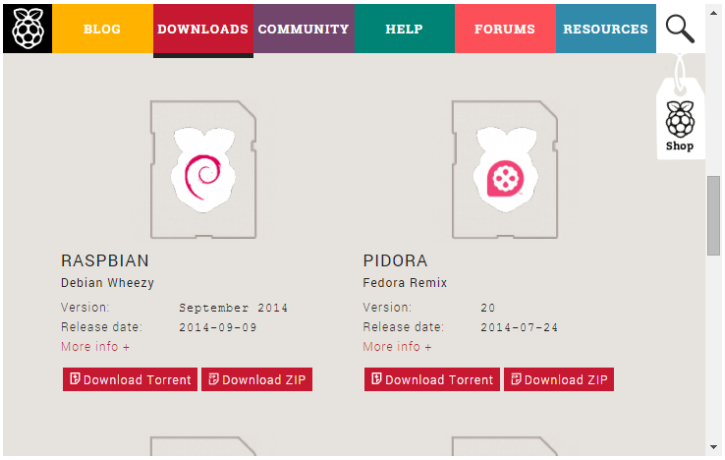


그림 32 라즈베리파이 공식 홈페이지

이를 다운로드 한 후, "2015-05-05-wheezy-raspbian.zip"를 압축 해제하도록 하자. 그러면 대략 1.8GB 의 "2015-05-05-wheezy-raspbian.img"파일이 생성된다.

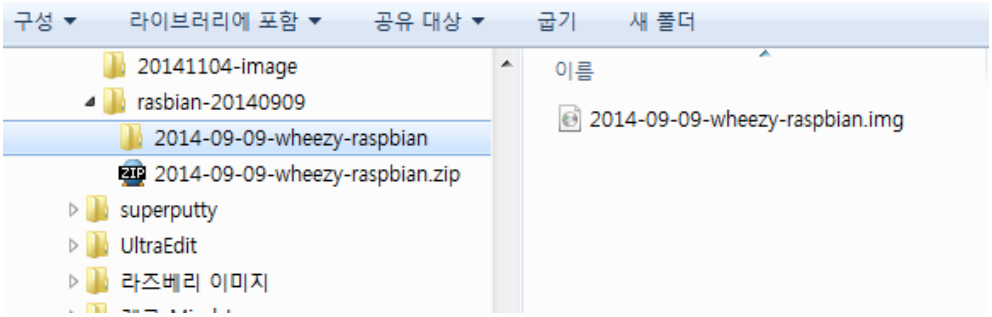


그림 33 라즈베리파이 다운 폴더

win32diskimager 의 실행파일을 다운받기 위해 다음 홈페이지에 접속한다. 하여 다운로드 한다.



그림 34 win32diskimager 다운 페이지

일단 http://sourceforge.net/projects/win32diskimager/ 사이트에 접속하여 "win32diskimager " 를 다운로드 받아 압축 해제한다. 그러면 설치없이 실행가능한 프로그램을 담은 하나의 폴더가 생성되고, 그 안의 "Win32DiskImager.exe" 실행 파일을 확인할 수 있을 것이다.

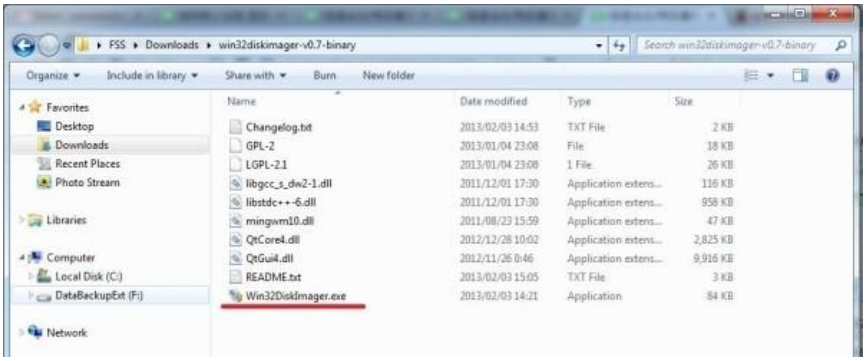
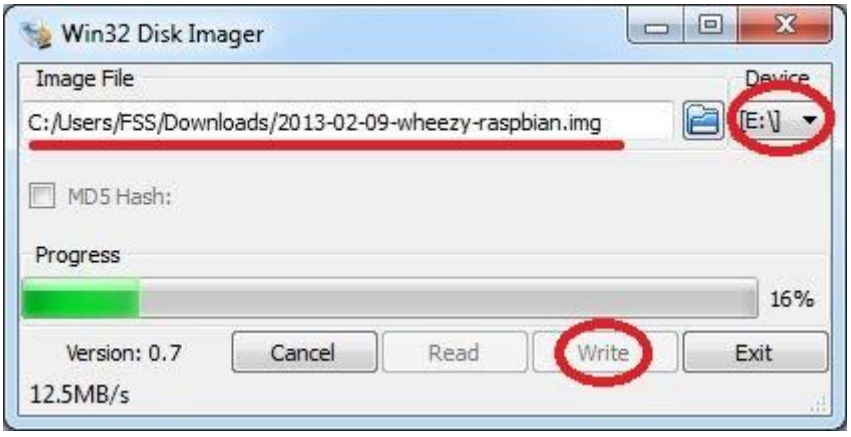


그림 35 win32diskimager 실행화일

PC 에 SD 메모리 카드를 삽입 후, 전단계에서 다운로드한 "Win32DiskImager.exe" 를 실행한다. 프로그램의 Image file 의 디스크 모양을 클릭하여 방금 전 다운로드 한 "2013-02-09-wheezy-raspbian.img" 파일을 선택한다. 그리고, 그 오른쪽의 Device 선택은 좀 전에 삽입한 SD 메모리 카드의 드라이브로 설정한 후, 프로그램 하단부의 "Write" 을 선택하여 SD 메모리에 이미지 파일을 저장한다.

(SD 메모리카드는 16GB, 클래스 10 추천, SD 카드에 호한 여부 테스트는 수많은 유저들의 자발적 차원으로 이루어지고 있다. 별도의 SD 메모리 카드를 구입하기전에 다음의 http://elinux.org/RPi_SD_cards 링크를 참고하도록 한다.)



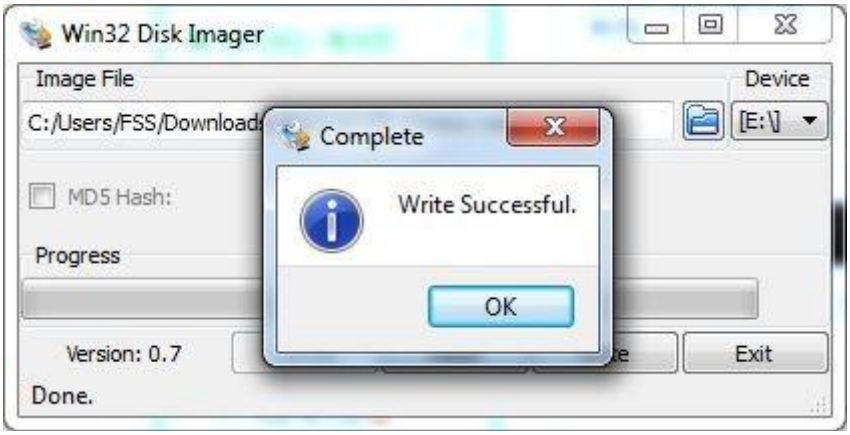
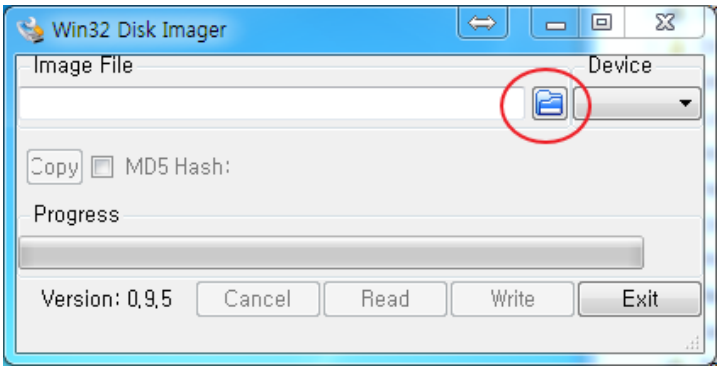


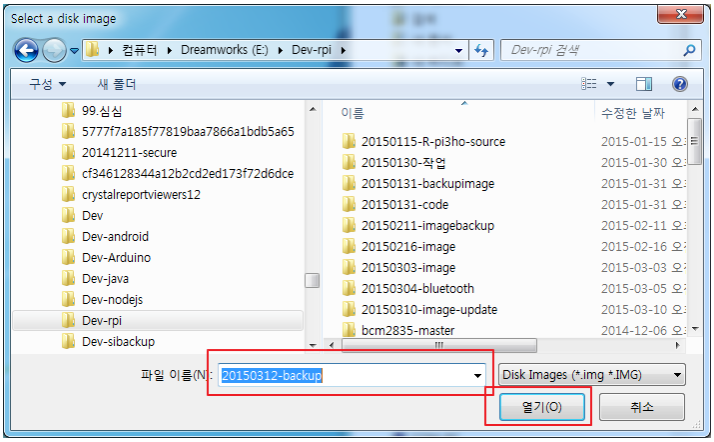
그림 36 win32diskimager 실행화면

(2) SD 카드 이미지 백업

실습을 진행하는중이거나 SD 카드의 내용을 전부 백업 받아야 하는 경우가 생길 경우 SD 카드 이미지를 백업하는 방법을 설명한다. 이미지를 SD 카드 리더기를 이용해 국는 경우와 유사한데 일단 Win32DiskImager.exe 를 실행한다.



실행화면에서 동그라미 표시가 된 디스크 이미지를 클리하면 윈도우 탐색기 이미지가 뜨는데 여기에서 저장 될 폴더와 파일이름을 지정하고 “열기”를 선택한다.



이후 처음 화면으로 되돌아오면 이때는 “Raed”를 클리하면 이미지가 백업된다. 백업된 이미지는 언제든 처음 이미지를 구울때와 마찬가지로 사용할 수 있게 된다.

(3) 주변기기연결

라즈베리파이를 기동하기 위해 주변기기를 연결해야 한다.

1. SD 메모리 카드 삽입
2. HDMI-HDMI 케이블이나 DVI-HDMI 케이블을 모니터와 연결시켜준다.

참고

모니터는 HDMI 혹은 DVI 출력 단자가 있는 모니터를 연결해야 한다. 일부 구형 모니터는 라즈베리 파이와 연결시 화면을 지원하지 못하는 경우가 발생하기도 한다.

3. 유선 키보드 및 유선 마우스를 접속시켜준다.
4. 랜 케이블을 연결한다.
5. microUSB 케이블을 이용하여 전원을 공급해준다.(PC 또는 어댑터에 연결)

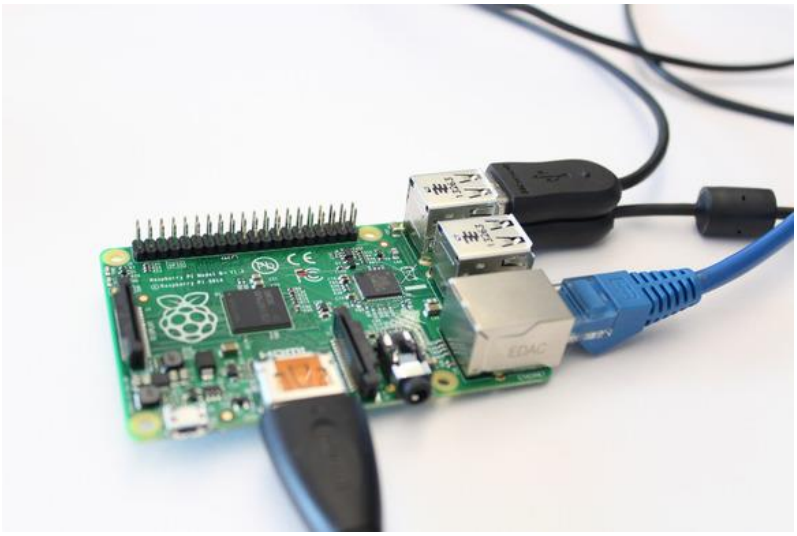


그림 37 라즈베리파이와 주변장치 연결도

6. 라즈베리 뒷면에 마이크로 SD 카드를 꼽아준다.

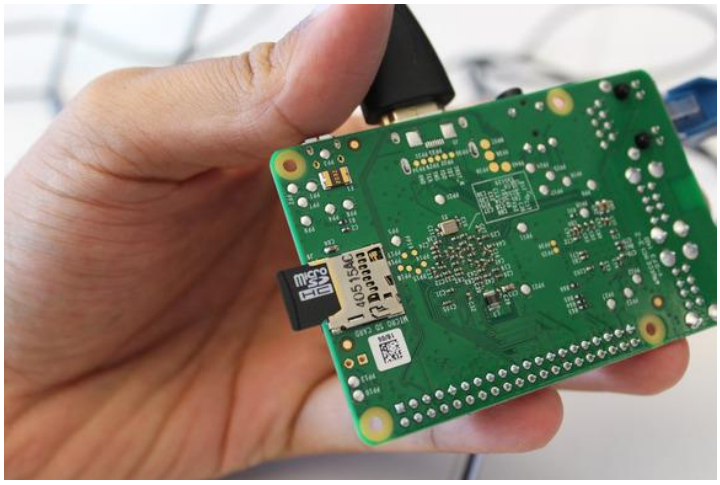


그림 38 라즈베리파이에 마이크로 SD 카드 삽입사진

(이미지 출처 :http://www.itworld.co.kr/news/89213)

(4) 부팅 및 Raspberry Pi 설정

SD 메모리 카드를 처음 넣은 후의 부팅은 부팅 도중에 "Raspi-config" 화면이 표시되어 몇 가지 설정을 하게된다.

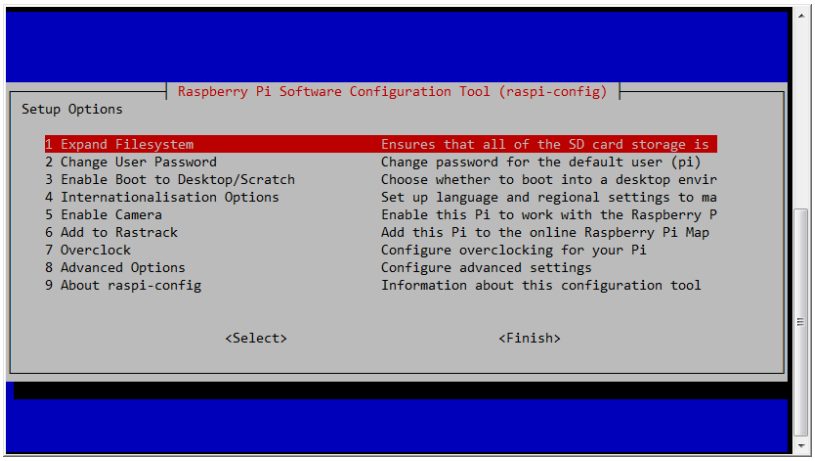


그림 39 라즈베리파이 설정화면

가장 먼저 해야 할 설정은 1. Expand Filesystem 이다. 가장 중요한 설정으로 라즈베리파이 OS 를 설치할 시 꼭 이를 선택해 주자. 이는 공식 OS 인 raspbian “wheezy” 를 SD 메모리 카드에 담은 후 이미지 파일을 풀때, 사용되는 SD 메모리가 2 기가 바이트 이상이더라도 2 기가 바이트에 맞도록 설정된다. 이를 확장해주기 위해서는 이 설정을 꼭 해주어야지만 사용자가 사용하는 SD 메모리의 용량에 맞도록 확장, 재설정이 되어 사용할 수 있게 된다. 설정에는 별다른 사항은 없고 클릭해주면 셋팅을 스스로 한 후, 재부팅이후에 설정된 내용이 적용된다.

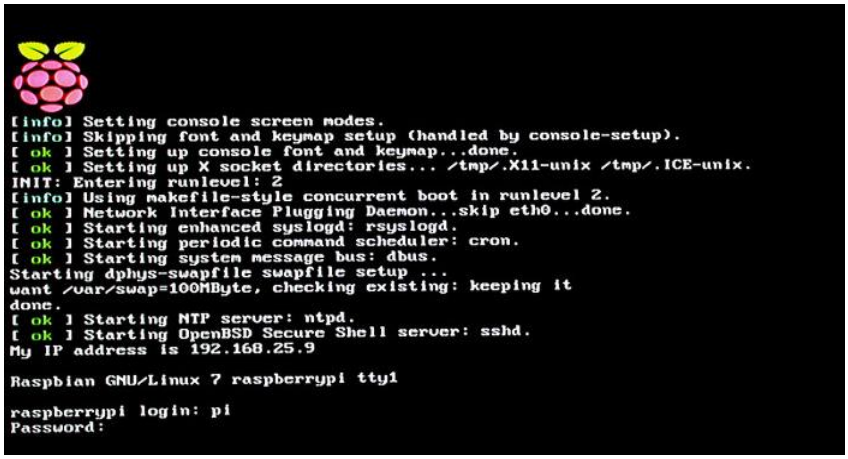


그림 40 라즈비안 OS 로그인 화면

설정 후 재부팅을 하게되며, 재부팅이 완료되면 아래의 그림과 같이 아이디와 비밀번호를 묻는 로그인 화면이 나온다. 로그인 아이디는 "pi", 로그인 비밀번호는 "raspberry" 이다.

```
raspberrypi login: pi
Password : raspberry
```

로그인이 문제없이 진행되었으면, startx 을 터미널창에 입력하여 X 윈도우가 실행하도록 하자.

$startx 명령은 GUI 화면을 보여준다.

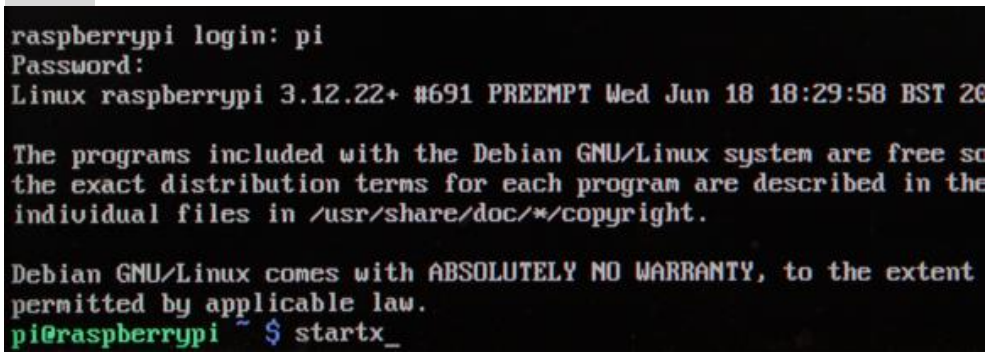


그림 41 라즈베리파이 Startx 명령 실행화면

이제 GUI 의 라즈비안 화면을 볼 수 있다.

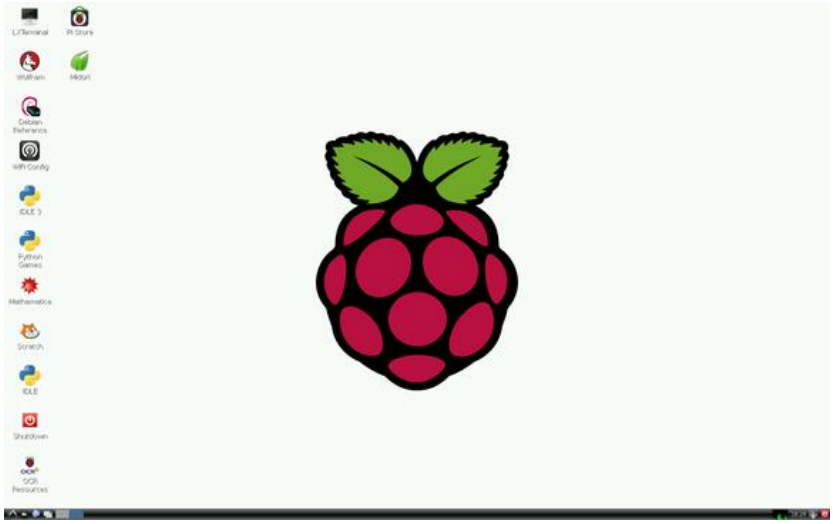


그림 42 라즈비안 초기 화면

여기까지 하면 본격적으로 라즈베리파이를 사용할 준비는 끝난것이다. 라즈베리파이 설정 명령어는 $sudo raspi-config 이며 라즈베리파이 설정화면으로 안내한다.

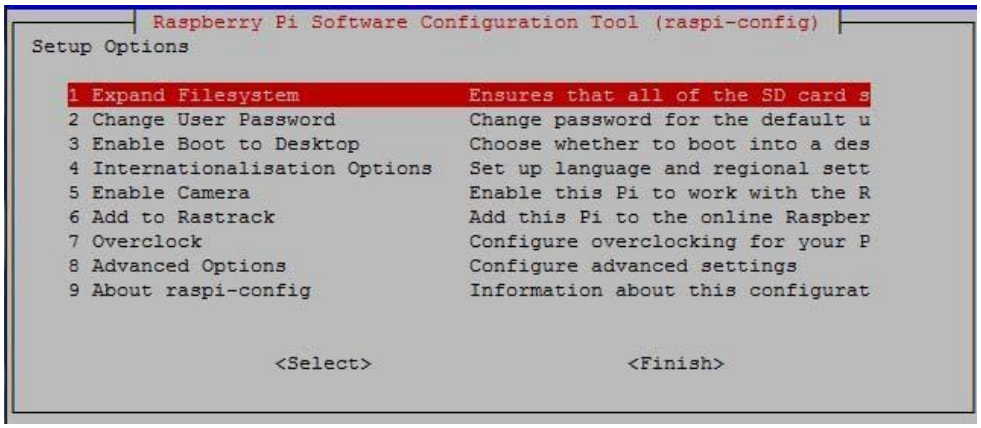


그림 43 라즈베리파이 설정 초기 화면

1. Expand Filesystem

가장 필수적인 설정으로 사용자의카드의 모든 영역을 사용하기 위한 설정입니다. 변경 내용 없이 클릭 한번으로 자동 설정이 되며 부팅 후에 적용된다.

2. Change User Password

라즈베리 파이의 기본 설정된 암호는 raspberry 로 설정되어 있다. 사용자의 편의에 따라 바꿔서 사용이 가능하며 보안을 위해서라도 바꿔서 사용하길 권장한다.

3. Enable Boot to Desktop

부팅 시에 시작을 커멘드 라인으로 할지 윈도우로 할지를 결정하는 부분으로 “Y”를 선택하면 부팅시에 자동으로 윈도우로 시작된다.

4. Internationalization options

국제화 설정 부분으로 다음과 같은 3 가지 설정이 볼 수 있다.

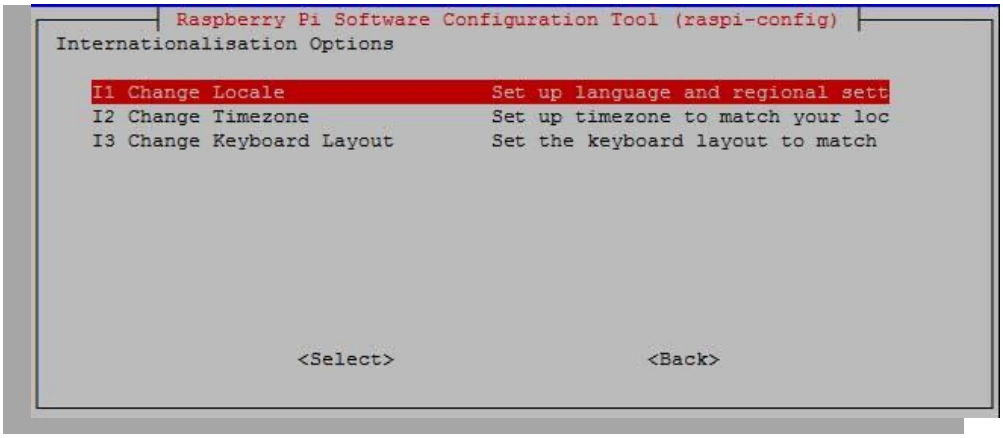


그림 44 국제화 설정 화면

A. I1 Change Local

지역 설정 부분으로 영국이 기본 설정으로 되어 있으며 목록 중에서 <ko_KR.UTF-8 UTF-8>을 선택하면 한국어로 변경이 가능하며, 윈도우에서 한글이 표시가 되지만 화면이 깨지거나 번역이 어색한 부분도 있어서 사용자가 선택해서 사용하고, 한글 키보드와 언어설정에 대한 부분은 http://www.rasplay.org/?p=3786 사이트를 참고하길 바란다.

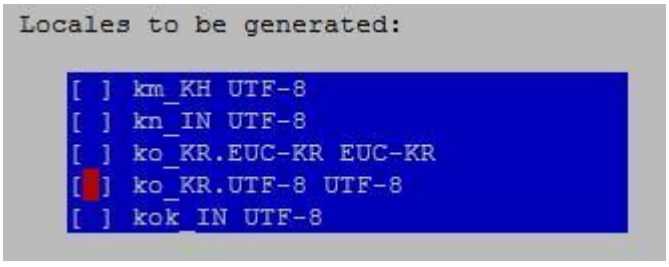


그림 45 언어설정 화면

B. I2 Change Timezone

사용 시간대를 설정하는 부분이며 국가 수도를 기준으로 설정하기 때문에 한국은 Asia->Seoul 을 찾아서 선택해 주면 된다.

C. I3 Change Keyboard Layout

키보드 레이아웃을 설정하는 부분으로 기본 설정하면 된다.

5. Enable Camera

만약 카메라 모듈이 있다면 카메라를 사용하기 위해서 <Enable>을 해 주어야 한다. 라즈베리 파이 카메라를 사용하기 위한 설정이다.

6. Add to Rastrack

Rastrack 사이트(http://rastrack.co.uk)를 통해서 전세계 라즈베리파이 사용자의 분포를 볼 수 있는 설정으로 사용자의 이름이나 이메일 정보를 입력하여 등록이 되면 자신의 지역에 라즈베리파이가 등록된다. 사이트를 방문하여 전세계 사용자 분포를 한번 확인해 볼 수 있다.

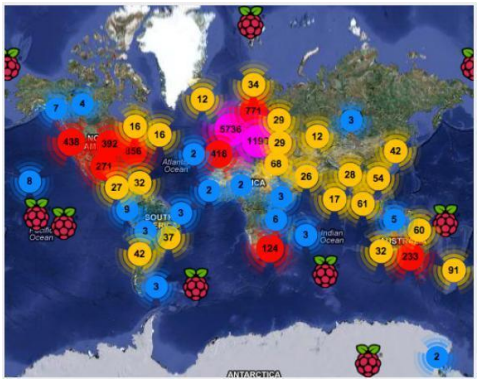


그림 46 라즈베리파이 사용 분포 화면

7. Overclock

말 그대로 오버 클럭을 설정하는 부분으로 기본 700MHz 클럭 이외에 다음과 같이 설정이 가능하지만 기본 설정으로 사용할 것을 권장한다.

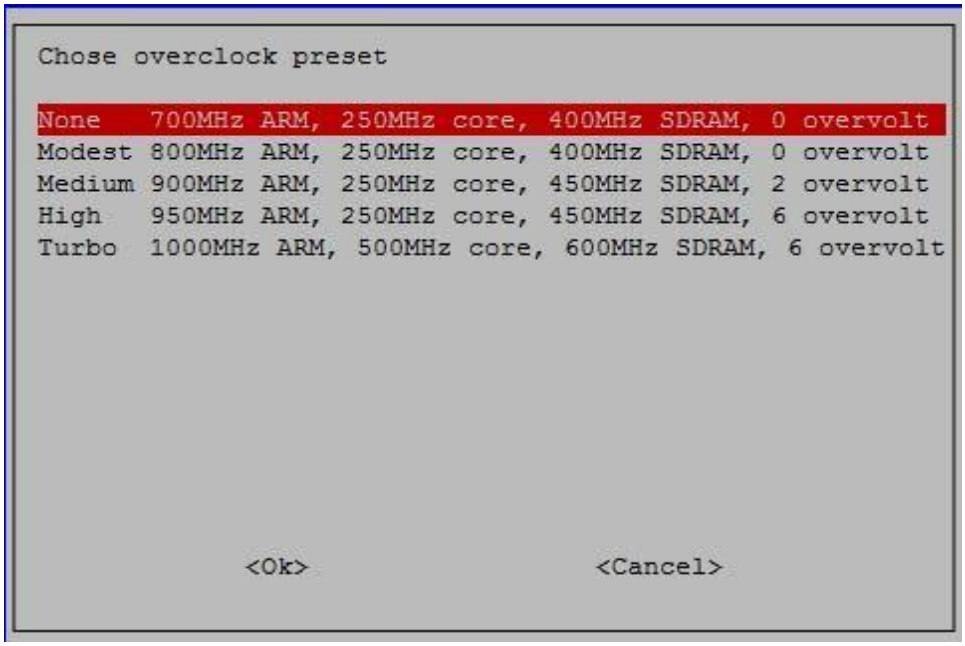


그림 47 오버클럭 설정 화면

8. Advance Option

고급 설정으로 추가적인 기능의 설정이 가능한 메뉴이다.

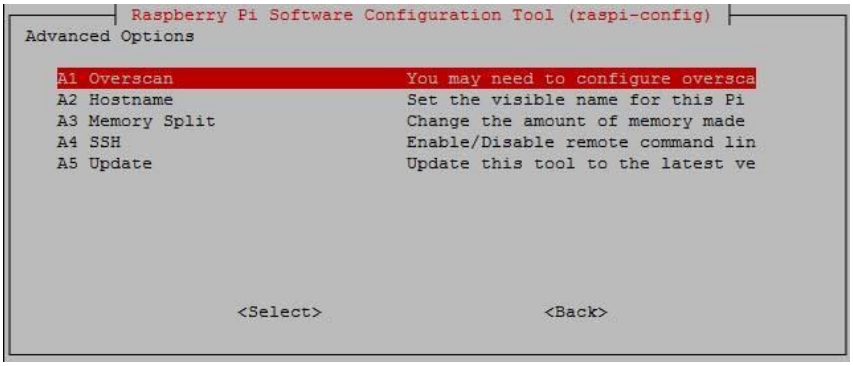


그림 48 고급설정 화면

A. A1 Overscan - 구형 디스플레이 장치에 맞도록 화면을 설정.

B. A2 Hostname - 호스트네임을 변경하는 부분으로 기본 raspberrypi 로 기본 설정되어 있으며 사용자가 임의로 정해서 사용 가능

C. A3 Memory Split - 메모리를 설정할 수 있는 부분으로 사용자의 환경에 따라서 변경해 주면 되며 윈도우를 사용하지 않는다면 가능한 작게 설정하여

CPU 가 메모리를 여유롭게 사용할 수 있도록 할 수 있고, 16/32/64/128/256MB 로 설정도 가능.

D. A4 SSH - Secure-Shell (SSH)를 사용에 관한 설정으로 기본으로는 사용하도록 되어 있고, 사용자의 컴퓨터에서 원격으로 라즈베리 파이에 접속해서 사용할 수 있는 방법을 제공.

E. A5 Update - raspi-config 툴을 최신 버전으로 업데이트.

9. About raspi-config

설정과는 상관없는 부분으로 툴에 대한 설명을 제공한다. 설정 변경 후에는 <Finish>를 선택하고 재부팅을 하면 설정 상태가 적용된다.

Raspbian 종료

Raspbian 을 종료하기위해서는 바탕화면의 LXTerminal 을 실행시킨후 아래와 같이 입력해주면 된다.

```
$sudo shutdown -h now
```

or

```
$sudo halt
```

만약 종료가 아닌 재부팅을 원한다면 아래의 명령어와 같다.

```
$sudo shutdown -r now
```

or

```
$sudo reboot
```

마지막으로, 처음으로 라즈베리파이를 구동하였다면 기본적인 업데이트를 실행해주자. 명령어는 아래와 같이 두개로 구성되어 있으며 터미널창에서 실행하도록 하자.

1. 업데이트 된 라이브러리 및 프로그램을 업데이트한다. 다운로드 개념이라고 보면된다.

```
$sudo apt-get update
```

2. 업데이트된 라이브러리 및 프로그램을 설정 및 설치하게 된다.

```
$sudo apt-get upgrade
```

4.3 Raspberry Pi GPIO

라즈베리파이의 GPIO 활용법을 알아보자. 사용 언어는 C 언어이다. 개발환경은 하드웨어적으로는 라즈베리파이 모델 B 버전 2 512RAM 환경에, 소프트웨어적으로는 라즈베리파이의 공식 OS 라고도 할 수 있는 데비안 리눅스계열의 Raspbian "wheezy"의 최신버전을 사용한다. 또한, 사용되는 언어로는 C 언어를 이용할 생각이다. 컴파일러는 Raspbian 운영체제에 기본적으로 설치되어 있는 gcc 컴파일러를 사용한다.

실습을 하기에 앞서 필수적인 GPIO 제어프로그램에 대해 알아보자. 가장 "Gordons Projects" 라는 블로그를 운영하는 곳에서 C 언어를 지원하는 "wiringPi" 라이브러리를 이용할 예정이다. 자세한 내용과 사용법이 필요하다면 다음의 사이트를 참조하기 바란다. http://wiringpi.com/

(1) wiringPi 설치

Raspberry Pi 에 사용되는 BCM2835 를 위한 C 로 작성된 GPIO access library 이다. wiringPi 라이브러리는 GNU LGPLv3 license 에 따라서 사용하거나 변경이 가능하다. C 와 C++로 사용 가능하지만, 적절한 wrapper 를 사용하여 다른 언어에서도 사용 가능하도록 개발되었다. 아두이노 시스템에 익숙한 사용자를 위해 설계되어 사용법도 비슷하다. Raspberry Pi GPIO 특징을 아래에 나타낸다.

- 40 핀 범용 (General Purpose Input/Output , GPIO) 커넥터(P1)를 가짐
- 신호와 버스로 구성
- 8 개의 범용 디지털 I/O 핀을 가짐 : 입력 혹은 출력 모드로 프로그래밍 설정 가능 (→ 이러한 핀 중 1 개(GPIO_1)은 PWM 출력으로 설정 가능)

- 추가적으로 2-wire I2C interface 와 4-wire SPI interface(2 개의 선택 신호, 총 5-핀 사용), 그리고 직렬 UART(serial UART)용 2 개의 핀을 갖고 있음.
- Raspberry Pi (Revision 2)는 별도의 커넥터(P5)에 4 개의 추가의 GPIO 라인을 갖고 있음 (단, P5 connector 는 납땜을 필요로 함).
- I2C, SPI 와 UART 인터페이스는 자신의 버스 모드로 사용하지 않을 경우, 범용 I/O 핀으로 사용 가능 (→ P1 connector 의 경우 최대 17 개(8 + 2 + 5 + 2) I/O pins 가능하고, Revision 2 Pi 보드의 경우 P5 connector 에 추가의 4 개의 핀 존재

wirringPi 라이브러리를 사용하기 위해 설치를 해야 하는데 설치 방법은 아래와 같다.

1) 일단, 라즈베리파이의 업데이트 및 업그레이드를 실시한다.

```
$sudo apt-get update
$sudo apt-get upgrade
```

2) 소스관리툴 git 를 다운로드한다.
(오픈소스 https://github.com/WiringPi/WiringPi 의 소스를 체크아웃하기 위함)

```
$sudo apt-get install git-core
```

3) "wiringPi" 프로젝트를 통째로 받아온다.

```
$git clone git://git.drogon.net/wiringPi
```

4) 빌드 및 설치를 진행한다.

```
$cd wiringPi
$sudo ./build
```

5) 설치가 잘 되었는지 아래의 명령어로 확인해 본다.

```
$gpio -v
$gpio readall
```

위 두명령어를 사용한 화면을 아래에 나타낸다.

```
pi@raspberrypi ~ $ gpio -v
gpio version: 2.26
Copyright (c) 2012-2015 Gordon Henderson
This is free software with ABSOLUTELY NO WARRANTY.
For details type: gpio -warranty

Raspberry Pi Details:
  Type: Model B+ , Revision: 1.2, Memory: 512MB, Maker: Sony
```

설치가 제대로 되었는지 확인해본다. pi@raspberrypi ~ $ gpio readall 명령을 실행하면

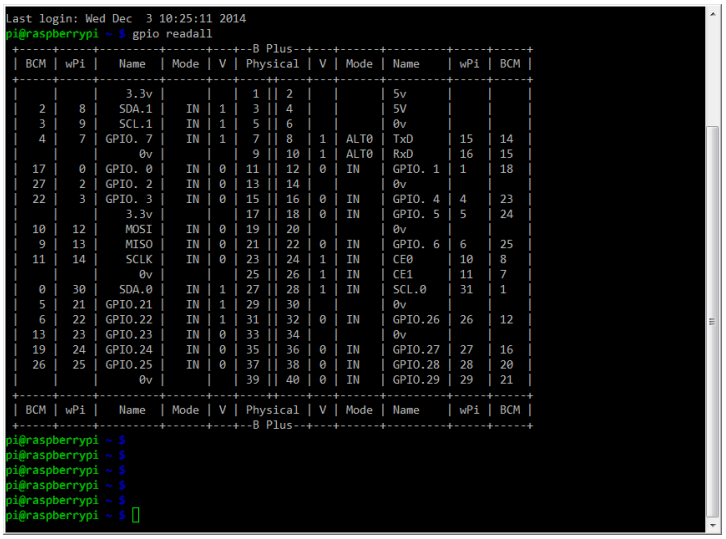


그림 49 GPIO 구성 화면

위와 같은 화면이 나온다면 제대로 설치가 된것이다. 아래는 라즈베리 파이의 GPIO 핀 번호를 나타낸 그림이다.

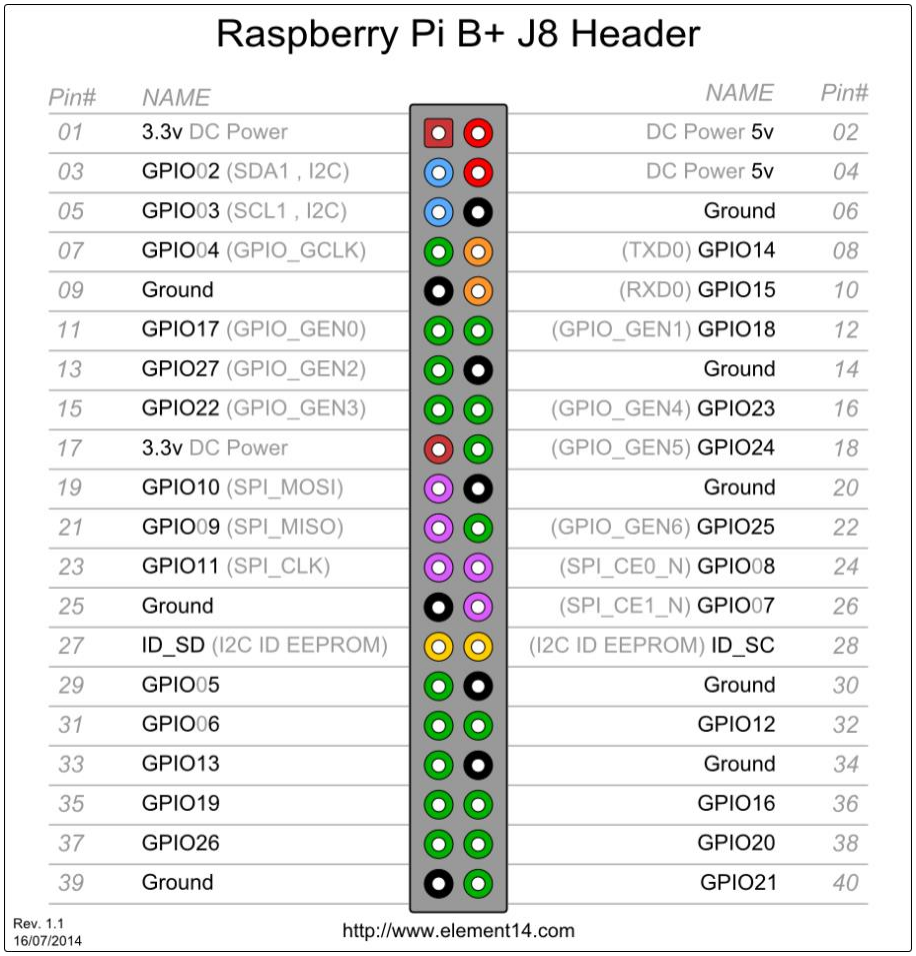


그림 50 GPIO 배치 화면

(2) wiringPi API 종류와 사용법

wiringPi 의 주요 API 함수를 나래에 설명한다. 요약해서 정리한 자료이고 버전이 업데이트 되면서 변경되느 부분도 있을 것이다. 정확한 내용은 홈페이지(http://wiringpi.com 을 참조하기 바란다.

WiringPi 함수 (API) wiringPi 라이브러리의 주요 함수
- 아두이노 wiring system 과 유사하게 설계

아두이노 시스템과 주된 차이

- program 의 주된 loop 가 제공되지 않음 (→ 사용자는 필요 시 선택적으로 구현)
- 이러한 특징은 Linux 시스템에서 command-line 인자를 접근하는 경우에 유용
- WiringPi 라이브러리 사용시 주의 사항 프로그램에서 header 파일 포함 #include <wiringPi.h> build 시 wiringPi 를 link 시킴 -lwiringPi

WiringPi 함수 (API) : setup 함수

wiringPi 를 초기화하는 4 가지 방법

- 초기화 함수 중 1 개가 프로그램 초기에 호출되어야 함 - wiringPi 의 버전 1 은 오류 코드로 (-1)을 반환했지만, 버전 2 는 0 을 반환함

```
int wiringPiSetup (void) ;
int wiringPiSetupGpio (void) ;
int wiringPiSetupPhys (void) ;
int wiringPiSetupSys (void) ;
```

wiringPiSetup (void) 함수

- 이 함수는 호출 프로그램이 wiringPi 핀 번호 방식을 사용하는 경우를 가정하여 wiringPi 를 초기화함
- wiringPi 핀 번호 방식은 단순화된 핀 번호 방식이며, 가상적인 핀 번호 0~16 을 실제 Broadcom GPIO 핀 번호로 매핑을 제공하는 방식.
- 이 함수는 루트 권한으로 호출 필요 wiringPiSetupGpio (void) 함수
- 호출 프로그램이 어떠한 re-mapping 을 하지 않고, 직접 Broadcom GPIO 핀 번호를 사용하며, 기능은 위와 동일 - 이 함수는 루트 권한으로 호출 필요 wiringPiSetupPhys (void) 함수
- 위와 기능이 동일하지만, 호출 프로그램이 실제 핀 번호를 사용하는 경우에 사용 (P1 connector 만 가능)
- 이 함수는 루트 권한으로 호출 필요 wiringPiSetupSys (void) 함수
- 이 함수를 wiringPi 를 초기화하는 기능은 동일하지만, 하드웨어를 직접 접근하지 않고, /sys/class/gpio 인터페이스를 사용.

- GPIO 핀이 gpio 프로그램으로 사전에 export 되면, 비-루트 권한으로 호출 가능

- 이러한 모드의 핀 번호는 기존 Broadcom GPIO 핀 번호 방식 사용 - 이러한 모드는 /sys/class/gpio 인터페이스를 통해 핀을 export 하기 위해, 사용자 프로그램에서 gpio 프로그램을 호출하는 system() 함수를 사용하거나, 별도의 shell-script 를 수행할 필요.

WiringPi 함수 (API) : Core 함수

void pinMode (int pin, int mode) 함수 - 핀의 모드를 INPUT, OUTPUT, 혹은 PWM_OUTPUT 으로 설정 가능

- wiringPi pin 1 (BCM_GPIO :18 , P1 connector 12) 만 PWM 출력을 지원 가능 void digitalWrite (int pin, int value) 함수

- 이전에 출력 모드로 설정된 핀에 HIGH 또는 LOW (1 또는 0)을 기록 void pwmWrite (int pin, int value) 함수

- 주어진 핀에 대한 PWM 레지스터에 PWM 제어 값을 기록 - 값의 범위 : 0 ~ 1023

- value/RANGE 의 비로 PWM 제어 (기본 RANGE = 1024) int digitalRead (int pin) 함수 - 주어진 핀의 값을 읽어서 반환 (값: HIGH 또는 LOW (1 또는 0))

WiringPi 함수 (API) : core 함수

void pullUpDnControl (int pin, int pud) 함수

- 이전에 입력 모드로 설정된 핀에 pull-up 또는 pull-down resistor mode 를 설정함

- 아두이노와 달리 BCM2835 는 pull-up 과 pull-down 내부 저항을 지원

- pud 의 3 가지 값

pud= PUD_OFF : no pull up/down
pud =PUD_DOWN: pull to ground
pud =PUD_UP : pull to 3.3v

- 사용자가 pull-up/pull-down 을 활성화하려면, 사용자는 프로그램을 시작하기 전에 script 내 gpio 프로그램으로 활성화해야 함 입력 VDD 20kΩ 내부 Pull-up 저항 Input data 입력 Gnd 20kΩ 내부 Pull-down 저항 Input data

analogRead (int pin) 함수

- 제공된 아날로그 입력 핀에서 읽은 값을 반환
- Gertboard, quick2Wire 아날로그 보드와 같은 디바이스에 대해 이러한 함수를 활성화하려면 추가의 아날로그 모듈을 등록해야 함

analogWrite (int pin, int value) 함수

- 제공된 핀에 주어진 값을 기록 - Gertboard 보드와 같은 디바이스에 대해 이러한 함수를 활성화하려면 추가의 아날로그 모듈을 등록해야

WiringPi 함수 (API) : timing 함수

unsigned int millis (void) 함수 :

- 프로그램이 setup 함수(예: wiringPiSetup)를 호출한 이후 경과한 ms 의 시간을 반환
- 최대 49 일 까지 지원 unsigned int micros (void) 함수
- 프로그램이 setup 함수를 호출한 이후 경과한 us 의 시간을 반환
- 최대 71 분 지원 void delay (unsigned int howLong) 함수
- 프로그램 수행을 howLong ms 만큼 정지시킴 - linux 의 multi-tasking 특성으로, 정해진 시간보다 좀 더 길게 됨 - 최대 지연 : 49 일

void delayMicroseconds (unsigned int howLong) 함수

- 프로그램 수행을 howLong us 만큼 정지시킴
- linux 의 multi-tasking 특성으로, 정해진 시간보다 좀 더 길게 됨 - 최대 지연 : 71 분

WiringPi 함수 (API) : PWM 함수

pwmSetMode (int mode) 함수

- PWM 발생기는 2 가지 모드 (“balanced” 와 “mark:space”)로 동작
- “mark:space” 모드가 기존 방식이지만, Pi 의 기본 설정 모드(default mode)는 “balanced” 모드임
- 모드 변경 설정 mode = PWM_MODE_BAL 혹은 PWM_MODE_MS

pwmSetRange (unsigned int range) 함수

- PWM 발생기내 범위 레지스터(range register)를 설정함 - 기본 값 = 1024
pwmSetClock (int divisor) 함수
- PWM 클럭에 대한 divisor 를 설정함

PWM 모드
- Range 와 Data 레지스터 비로 제어
Balanced mode (0) - RANGE 개의 펄스 기간 동안 , DATA 개의 펄스를 균형있게 발생시킴
Mark-Space mode (1) - 먼저, DATA clock 펄스 기간 동안 “HIGH”를 출력하고, 이어서 “RANGE-DATA” 클럭 펄스 기간 동안 “LOW”를 출력함

PWM clock
- PWM 펄스 폭을 제어하도록 설정 가능
- 19.2 Mhz 클록에서 유도됨 - 사용자는 divider 를 설정하여, PWM clock 을 제어 가능
(예) DC motor 를 약 1KHz 단계로 PWM 방식으로 제어
- divisor = 16, Range= 1024 설정
- 19.2 MHz /16 = 1.2 MHz (PWM clock)
- stop 상태 : 1.2MHz * (0/1024) - full-on 상태 : 1.2 MHz * (1024/1024)
- 증분 : 1/1024 = 1.2MHz/1024 = 1kHz

WiringPi 함수 (API) : PWM 함수 mark-space 모드

WiringPi 함수 (API) : Priority, Interrupt, Thread 함수
- 사용자 프로그램(혹은 thread)를 관리하고, 사용자 프로그램에서 새로운 Thread 를 개시하도록(launch) 하는 도움 함수(helper functions)를 제공함
- Thread 는 사용자 프로그램과 병행적으로 수행되며, 다양한 목적으로 사용 가능 int piHiPri (int priority) 함수
- 이 함수는 사용자 프로그램 (혹은 multi-threaded program 에서 thread)를 높은 우선 순위로 조정하며, 실시간 스케줄링(real-time scheduling)을 가능하도록 함
- priority 인자는 0(default) ~99 (최대) 범위여야 함

- 이 함수는 사용자 프로그램을 빠르게 수행하도록 하는 것이 아니라, 다른 프로그램이 수행될 때 더 큰 시간 할당을 갖도록 함 - priority 인자는 다른 thread 와 상대적인 값으로 동작함

- 반환 값은 성공 시 0, 오류 시 -1 을 반환 (→ 오류가 발생하면, 프로그램은 errno 전역 변수 확인)

- root 로 수행중인 프로그램만 priority 를 변경 가능

WiringPi 함수 (API) : Priority, Interrupt, Thread 함수 Interrupts - GPIO interrupt handling code 를 patch 한 새로운 커널을 사용할 경우, 사용자 프로그램에서 interrupt 를 처리할 수 있음.

- GPIO 는 입력 신호의 상승(rising), 하강(falling), 상하강(rising, falling)에 대한 인터럽트를 처리할 수 있도록 설정 가능. int wiringPiISR (int pin, int edgeType, void (*function)(void)) 함수

- 명시된 핀에 수신된 인터럽트를 처리할 함수(ISR)을 등록함 - edgeType 인자는 INT_EDGE_FALLING, INT_EDGE_RISING, INT_EDGE_BOTH 혹은 INT_EDGE_SETUP 이다.

- edgeType 인자가 INT_EDGE_SETUP 이면, 핀의 초기화가 발생되지 않음. (외부에서 gpio 명령으로 이미 setup 이 된 상태 가정 : 예: gpio edge 0 falling)

- edgeType 인자가 INT_EDGE_SETUP 가 아닌 다른 3 가지의 경우, 해당 핀이 외부로 공지(exported)되며, 명시한 형태로 초기화됨. (→ 내부적으로 gpio 호출을 통해 구현됨)

- 핀 번호는 모드(native wiringPi, BCM_GPIO, Sys modes)에 따라 정해짐 - 이 함수는 임의의 모드에서 수행가능하며, root 권한이 필요하지 않음

- 인자 function 는 인터럽트가 trigger 되는 경우 호출되는 ISR(interrupt service routine)이며, ISR 은 높은 우선 순위로, 주 프로그램과 병행적으로 동작하며, 모든 전역 변수에 대한 접근이 가능하며, 파일 핸들을 개방할 수 있음.

Concurrent Processing(multi-threading)

- wiringPi 는 Posix threads 에 대한 Linux 구현에 대한 단순한 인터페이스와 mutex's (Mutual exclusions)를 접근하는 단순화된 메커니즘을 갖고 있음

- 이러한 함수를 사용하여, 사용자 프로그램에서 주 프로그램과 병행적(concurrent)하게 수행하는 새로운 process 를 생성할 수 있고, mutex

메커니즘을 사용하여, 프로세스 간에 변수를 안전하게 전달할 수 있음 int piThreadCreate (name) 함수

- 이 함수는 PI_THREAD 선언을 통해 이전에 선언된 다른 함수를 사용자 프로그램에 thread 로 생성함
- 이 함수는 사용자 주 프로그램과 병행적으로 수행됨
- 이러한 예로는 사용자 프로그램이 다른 작업을 수행하고 있는 동안, 이러한 다른 thread 가 인터럽트를 대기하도록 할 수 있다.
- thread 는 주 프로그램 또는 다른 thread 와 통신을 하기 위해, event 또는 전역 변수를 사용한 동작으로 지시할 수 있다.

Thread 함수 선언 예 (PI_THREAD 선언을 사용) PI_THREAD (myThread) { .. 주 프로그램과 병행적으로 수행하는 코드를 둠 }

WiringPi 함수 (API) : Priority, Interrupt, Thread 함수

주 프로그램에서 Thread 를 수행하는 형태 x = piThreadCreate (myThread) ; if (x != 0) printf ("it didn't startn") Thread 함수 선언 예 (PI_THREAD 선언을 사용) PI_THREAD (myThread) { .. 주 프로그램과 병행적으로 수행하는 코드를 둠 }

WiringPi 함수 (API) : Priority, Interrupt, Thread 함수

piLock (int keyNum) 함수 piUnlock (int keyNum) 함수

- 주 프로그램에서 사용자 프로그램에서 수행 중인 모든 Threads 로 변수 갱신을 동기화하도록 하는 함수
- keyNum 인자는 0 ~ 3 사이의 값이며, 하나의 “key”를 나타낸다. - 다른 process 가 동일한 key 를 lock 하려고 하면, 이전 process 가 해당 key 를 unlock 할 때까지 대기하도록 함
- 사용자는 이러한 함수를 사용하여, 주 프로그램과 Thread 간에 데이터를 교환할 때 올바른 데이터를 얻는 것을 보장한다.

WiringPi 함수 (API) : Serial 함수

Serial library 를 사용하기 위해 헤더 파일 추가

#include <wiringSerial.h>

int serialOpen (char *device, int baud) 함수

- 직렬 디바이스를 개방하고 초기화하며, 보율을 설정함 - 포트를 “raw” 모드(한번에 한 개의 문자, 변환 없음)로 설정하고, 읽기 타임 아웃을 10 초로 설정 - 반환 값은 파일 기술자이며, 오류 시 “-1” 반환

void serialClose (int fd) 함수

- 주어진 파일 기술자로 식별되는 디바이스를 닫음 void serialPutchar (int fd, unsigned char c) 함수

- 주어진 파일 기술자로 지시하는 직렬 디바이스에게 1 개의 바이트를 전송함 void serialPuts (int fd, char *s) 함수

- 주어진 파일 기술자로 지시하는 직렬 디바이스에게 1 개의 문자열(null-terminated string)을 전송함

WiringPi 함수 (API) : Serial 함수

void serialPrintf (int fd, char *message, …) 함수

- 직렬 디바이스에게 printf 함수를 에뮬레이션 함 int serialDataAvail (int fd) 함수

- 읽을 수 있도록 대기하고 있는 문자 수를 반환 - 오류 조건인 경우 “-1”을 반환 int serialGetchar (int fd) 함수

- 직렬 디바이스에서 이용 가능한 다음 문자를 반환 - 데이터가 없는 경우 10 초 동안 대기한 후 “-1” 반환 void serialFlush (int fd) 함수

- 주어진 디바이스에 수신된 모든 데이터 혹은 전송을 대기하고 있는 모든 데이터를 제거함

WiringPi 함수 (API) : SPI 함수

SPI 인터페이스 사용 전에 준비 작업 커널에 SPI 드라이버를 적재(load)
$ gpio load spi

4KB 보다 큰 버퍼가 필요한 경우, 명령 행 라인에 이를 명시할 필요 (크기 : KB 단위) (예) 100kB 의 버퍼 사용 경우

$ gpio load spi 100 프로그램에 SPI 라이브러리 사용을 지시하는 헤더 파일 포함 #include <wiringPiSPI.h> 프로그램의 컴파일 시 link 작업 - -lwiringPi 로 link 필요

WiringPi 함수 (API) : SPI 함수

int wiringPiSPISetup (int channel, int speed) 함수

- 하나의 채널을 초기화 함 (Pi 는 2 개의 채널, 0 와 1 을 가짐) - “speed” 파라미터는 500,000 ~ 32,000,000 사이의 정수로, Hz 단위의 SPI clock 속도를 나타냄 - 반환 값은 linux 파일 기술자이며, 오류 시 “-1”을 반환

int wiringPiSPIDataRW (int channel, unsigned char *data, int len) 함수

- 선택된 SPI 버스로 병행의(simultaneous) 쓰기/읽기 동작(write/read transaction) 수행

- 사용자 버퍼에 있던 데이터는 SPI 에서 읽혀진 데이터로 over-write 됨

- 표준 read()와 write() 시스템 호출(system call)을 사용하여 SPI 버스에 대한 읽기/쓰기가 가능

- write() 함수는 일련의 shift 레지스터 혹은 LED 문자열 등에 데이터를 전송할 때 적합하고, A/D 와 D/A 변환기는 병행 write/read 동작이 필요하다. .

WiringPi 함수 (API) : I2C 함수

I2C 인터페이스 사용 전에 준비 작업 커널에 SPI 드라이버를 적재(load) $ gpio load i2c

100kbps 가 아닌 다른 baud rate 을 사용할 경우 명령 행 라인에 이를 명시할 필요 (크기 : Kbps 단위) (예) 1000 kbps 를 사용하는 경우 $ gpio load i2c 1000

프로그램에 I2C 라이브러리 사용을 지시하는 헤더 파일 포함 #include <wiringPiI2C.h>

프로그램의 컴파일 시 link 작업 - -lwiringPi 로 link 필요

WiringPi 함수 (API) : I2C 함수 I2C 디바이스를 검사하기 위해 command line 명령 $ i2cdetect -y 0 # Rev 1 $ i2cdetect -y 1 # Rev 2 i2cdetect 를 수행하기 위해 gpio 를 수행하는 경우 $ gpio i2cdetect

WiringPi 함수 (API) : I2C 함수

int wiringPiI2CSetup (int devId) 함수 - 주어진 디바이스 ID 로 I2C 시스템을 초기화함

- ID 는 i2cdetect 프로그램을 통해 얻은 디바이스의 I2C 번호임

- 이 함수는 Raspberry Pi 의 버전을 체크하고, /dev 폴더에 적절한 디바이스를 개방함

int wiringPiI2CRead (int fd) 함수

- 단순한 read 동작 수행 - 일부 디바이스는 레지스터를 지정할 필요 없이 데이터를 읽어서 제공 int wiringPiI2CWrite (int fd, int data) 함수

- 단순한 write 동작 - 일부 디바이스는 레지스터를 지정할 필요 없이 데이터를 쓰기(수용) 기능 int wiringPiI2CWriteReg8 (int fd, int reg, int data) 함수 int wiringPiI2CWriteReg16 (int fd, int reg, int data) 함수

- 디바이스 내 지정한 레지스터에 8 혹은 16-비트 데이터를 씀

int wiringPiI2CReadReg8 (int fd, int reg) 함수

int wiringPiI2CReadReg16 (int fd, int reg) 함수

- 디바이스내 지정한 레지스터에서 8 혹은 16-비트 데이터를 읽음

WiringPi 함수 (API) : software PWM 함수

Software PWM library

- 임의의 라즈베리 파이 GPIO 핀에 PWM 신호를 출력하는 기능 제약 사항 - CPU 사용을 적게 하기 위해 최소 펄스 폭은 100μS 임

- 기본 range 인 100 과 결합할 경우 100 Hz 의 PWM 주파수를 얻을 수 있음

1). range 를 낮출 경우 높은 주파수를 얻을 수 있음

2). range 를 증가시킬 경우, 낮은 주파수를 얻을 수 있음 - 이러한 제약 범위 내에서 LED 와 Motor 를 제어 가능

software PWM 라이브러리 사용을 위해 추가해야 할 헤더 파일 #include <wiringPi.h> #include <softPwm.h> 컴파일 시 2 개의 라이브러리를 link 해야 함 (pthread library, wiringPi library)

```
$ cc -o myprog myprog.c -lwiringPi -lpthread
```

WiringPi 함수 (API) : software PWM 함수 제공하는 함수

int softPwmCreate (int pin, int initialValue, int pwmRange) 함수

- 소프트웨어로 제어되는 PWM 핀을 생성함 - 임의의 GPIO 핀을 사용 가능

- 핀 번호는 setup 함수(예: wiringPiSetup() function)에서 지정하는 방식

- pwmRange 에 100 을 사용하면, value 는 0(off) ~100 (fully on)이 됨

- 성공 시 0 을 반환 void softPwmWrite (int pin, int value) 함수 - 주어진 핀의 PWM 값을 갱신함

WiringPi 함수 (API) : software Tone 함수 Software Tone library - 임의의 라즈베리 파이 GPIO 핀에 square/tone wave signal 을 출력하여 소리 생성 하는 기능 제약 사항 - CPU 사용을 적게 하기 위해 최소 펄스 폭은 100µS 임 - 최대 주파수 = 1/0.002 = 5000 Hz - 높고 실시간 우선순위의 루틴이 수행되고 있는 동안에는 Linux 는 발생되는 tone 의 정밀도에 영향을 줌 - 압전 스피커나 높은 저항의 스피커로 소리 연주 가능 software Tone 라이브러리 사용을 위해 추가해야 할 헤더 파일 #include <wiringPi.h> #include <softTone.h> 컴파일 시 2 개의 라이브러리를 link 해야 함 (pthread library, wiringPi library) $ cc -o myprog myprog.c -lwiringPi -lpthread

WiringPi 함수 (API) : software Tone 함수 제공하는 함수 int softToneCreate (int pin) 함수 - 소프트웨어로 제어되는 Tone 핀을 생성함 - 임의의 GPIO 핀을 사용 가능 - 핀 번호는 setup 함수(예: wiringPiSetup() function)에서 지정하는 방식 - 성공 시 0 을 반환 void softToneWrite (int pin, int freq) 함수 - 주어진 핀의 tone 주파수 값을 갱신함 - tone 은 주파수가 0 으로 설정될 때까지 연주함

WiringPi 함수 (API) : shift 함수 shift library - Pi 를 통해 shift-register(예 : 74×595) 로 8-비트 데이터를 shift-out 하거나, shift-register 를 통해 8-비트 데이터를 shift-in 할 수 있도록 함) shift 라이브러리 사용을 위해 추가해야 할 헤더 파일 #include <wiringPi.h> #include <wiringShift.h > 제공하는 함수 uint8_t shiftIn (uint8_t dPin, uint8_t cPin, uint8_t order) 함수 - dPin 에 나타나는 8-비트 데이터를 cPin 의 클록으로 shift 하여 받음 - 순서 : LSBFIRST 또는 MSBFIRST. - 데이터는 cPin 이 High 될 때 표본화 - 8-비트 값을 반환 void shiftOut (uint8_t dPin, uint8_t cPin, uint8_t order, uint8_t val) 함수 - 8-bit val 값을 dPin 으로 통해 cPin 의 클록으로 shift-out 함 - 순서 : LSBFIRST 또는 MSBFIRST.

WiringPi LCD 라이브러리 (dev-Lib) LCD Library (HD44780U) - wiringPi LCD devLib 는 Hitachi HD44780U 혹은 호환 제어기에 기반을 둔 일반적인 1, 2 와 4-line LCD displays 를 구동할 수 있는 함수 제공 - 사용자는 하나의 라즈베리 파이에 다수 개의 display 를 연결할 수 있도록 함 - LCD 디스플레이는 Pi 의 on-board GPIO 를 통해 직접 연결하거나, wiringPI 를 지원하는 많은 GPIO expander chips(예: MCP32017 I2C GPIO expander)을 통해 연결 가능 16×2 표준 LCD display 가 라즈베리 파이에 연결된 경우

Text LCD 모듈의 회로도 핀 신호 이름 기 능 1 GND GND 2 VCC 전원 + 5V (혹은 3.3V) 3 Vo Contrast adjustment (VO) 4 RS Register Select (0=Inst, 1 = data) 5 R/W' Read/Write (0 : Write, PC →LCD) (1: Read, LCD → PC) 6 E Enable Signal for read/write LCD 7 DB0(LSB) 8-비트 데이터 버스 8 DB1 9 DB2 10 DB3 11 DB4 4-비트 데이터 버스 12 DB5 13 DB6 14 DB7 (MSB) 15 LED+ Back light 에 대한 전원 16 LED- Back light 에 대한 전원 WiringPi LCD 라이브러리 (dev-Lib) (주) 뒤의 LCD 와 핀 배열이 다름 Vo
Page 48
WiringPi LCD 라이브러리 (dev-Lib) 4-bit 모드로 연결된 경우 8-bit 모드로 연결된 경우

WiringPi LCD 라이브러리 (dev-Lib) LCD library 사용 시 주의 사항 - 1 개 이상의 LCD 디스플레이를 연결 가능 8-비트 모드인 경우 - 첫 번째 LCD 디스플레이는 10 개의 GPIO 핀이 필요 - 각각의 추가되는 디스플레이는 하나의 추가의 GPIO 핀이 필요 (→ 8 개의 LCD 디스플레이를 사용할 경우, 최대 17 개의 GPIO 핀 이 필요) 4-bit interface 를 사용하는 경우(코드도 단순) - 최대 12 개의 LCD 를 연결 가능 LCD display 의 동작 전원 - LCD display 는 5V display 혹은 3.3v display 로 사용 가능함 5V display 로 사용하는 경우 주의 사항 - 디스플레이 값이 라즈베리 파이로 전달되어 쓰지 않도록 보장 필요 (→ 그렇지 않으면, GPIO 상에 5V 가 입력으로 나타날 경우 보드 핀 드라이브 손상 가능) - 5V display 방식으로 사용할 경우, LCD 의 R/W* 핀을 항상 Ground 로 하여, LCD 가 Host 에 대해 항상 Read 모드로 동작하도록 함 (주 : 라즈베리 파이의 경우 항상 Write 동작)

WiringPi LCD 라이브러리 (dev-Lib) LCD 초기화 관련 함수 - LCD library 를 사용하려면, 사용자 프로그램의 앞 부분에 다음 선언 필요 #include <wiringPi.h> #include <lcd.h> int lcdInit (int rows, int cols, int bits, int rs, int strb, int d0, int d1, int d2, int d3, int d4, int d5, int d6, int d7) 함수 - LCD 의 핀을 지정하여 초기화하는 함수로 다른 함수보다 먼저 호출 필요 - 반환 값은 뒤따르는 LCD 를 처리하는 데 사용할 'handle' 값이며, "-1"은 오류를 나타낸다. 여기서, - Rows 와 cols : 디스플레이의 row 와 column 지시 (예: 2, 16 혹은 4, 20). - Bits : 인터페이스의 비트 수 (4 혹은 8). - rs : RS 신호의 핀 번호 - strb : Strobe (E) 신호의 핀 번호 - d0 ~ d7 : Pi 에서 LCD display 로 연결된 대응하는 핀 번호 (→ 4-비트 모드인 경우 처음 4 개의 핀 정보만 사용)

wiringPi LCD 라이브러리 (dev-Lib) (예 1) 4-비트 인터페이스인 경우 fd = lcdInit (2, 16, 4, 11,10 , 0,1,2,3,0,0,0,0) ; d4~d7 를 사용 +5V R/W"=0 RS E Vo (가변저항 사용) WiringPi 핀 번호 방식 d4 밝기 조정

WiringPi LCD 라이브러리 (dev-Lib) (예 2) 8-비트 인터페이스인 경우 fd = lcdInit (2, 16, 8, 11,10 , 0,1,2,3,4,5,6,7) ; d0~d7 를 사용

WiringPi LCD 라이브러리 (dev-Lib) LCD 관련 주요 함수 lcdHome (int handle) - 커서를 홈으로 이동 lcdClear (int handle) - 화면을 지움 lcdDisplay (int fd, int state) - 디스플레이를 On 혹은 Off 함 - state 인자 값 : True 혹은 False - 초기 값 : display on. lcdCursor (int fd, int state) - 커서를 On 혹은 Off 함 - state 인자 값 : True 혹은 False - 초기 값 : cursor off. lcdCursorBlink (int fd, int state) - 커서를 blink-on , Blink-off 할 지 결정 - state 인자 값 : True 혹은 False - 초기 값 : cursor blink off.

WiringPi LCD 라이브러리 (dev-Lib) lcdPosition (int handle, int x, int y) 함수 - 다음 글자 입력에 대한 커서 위치를 설정 여기서, x : column 의 위치 (0 : 맨 왼쪽) y : row 의 위치 (0: 맨 위 라인) lcdCharDef (int handle, int index, unsigned char data [8]) 함수 - LCD 디스플레이에 8 개의 사용자 정의 가능 문자 중 1 개를 문자를 재 정의할 수 있도록 함 여기서, data array: 맨 위 라인에서 맨 아래 라인까지 문자를 나타내는 8 bytes (5×8 font 크기로 된

1 개의 글자 폰트를 정의하는 문자 배열) (→ 글자(문자)는 항상 5×8 font 크기를 가지므로, 하위 5-비트만 사용) index: 0 ~ 7 (→ lcdPutchar() 함수로 정의된 문자 출력 가능)

WiringPi LCD 라이브러리 (dev-Lib) lcdPutchar (int handle, unsigned char data) 함수 - 하나의 ASCII character 를 출력. lcdPuts (int handle, const char *string) 함수 - 하나의 문자열을 출력 lcdPrintf (int handle, const char *message, …) ; - printf 포맷 명령을 사용하여, 포맷된 문자열을 출력 아직, 스크롤 등 나머지 기능은 구현되지 않은 상태

WiringPi LCD 라이브러리 (dev-Lib) 라즈베리 파이에 LCD 를 배선하는 방법 데이터 버스에 연결할 4 개 혹은 8 개의 GPIO 핀을 선택하고, 추가적으로 제어에 사용할 2 개의 GPIO 핀을 선택 - 2 개의 제어 핀 : RS 와 E. (E 핀은 strobe pin 으로 불리기도 함) 5V 전원을 사용할 경우, LCD 의 R/W" 핀은 항상 Ground 에 연결 - 라즈베리 파이의 손상 방지 여러 개의 LCD display 를 연결할 경우 - LCD display 를 병렬로 배선함 - E 핀만 각 LCD 에 별도로 사용하고, 나머지 핀을 공유해서 사용 - 각 디스플레이는 독립적인 E 를 사용 (다른 GPIO 핀 사용)

WiringPi 라이브러리 : GPIO utility GPIO utility - WiringPi 라이브러리는 on-board GPIO 인터페이스와 다른 모듈(예: PiFace, Gertboard, 범용 GPIO expander type device)를 관리하도록 하는 별도의 프로그램(gpio utility) 존재 - gpio 프로그램은 GPIO pins 을 조작(예: 입력, 혹은 출력으로 설정)하기 위해, script 로 사용 가능하며, 심지어 전체 프로그램을 단지 shell-script 내 gpio command 로 구성 가능 - C/C++ 혹은 다른 프로그램 언어에서 gpio 명령을 호출하는 방법은 system() 함수를 사용하는 방법이 가능 - gpio 명령은 setuid 프로그램으로 설치되도록 설계되어 있으며, sudo 명령이나 root 로 login 할 필요 없이 일반 사용자에 의해 호출 가능 GPIO utility 이 지원하는 기능 GPIO 핀을 제어하고, 읽고, 쓰는 기능 /sys/class/gpio interface 를 통해 핀을 Export 하거나 Unexport 함 - 그 경우 사용자 프로그램에서 이용 가능 /sys/class/gpio interface 를 통해 핀을 edge-triggered interrupt 를

enable 하도록 Export 가능 SPI 와 I2C 모듈(modules)을 커널에 적재(load)하고, 사용자가 gpio 프로그램을 수행하도록 함에 의해서, read/write 가 가능하도록 /dev/ 허가를 설정 가능 보드에 맞는 적적한 flag 설정으로 i2cdetect 프로그램 수행 가능 module 이 적재된 경우 SPI buffer size 와 I2C baud rate 을 설정 가능 라즈베리 파이 하드웨어 revision 을 결정 가능

WiringPi 라이브러리 : GPIO utility GPIO utility 사용법 - Linux command line 에서 수행 gpio -v - 버전을 출력 gpio -g … - 선택 옵션 -g flag 는 핀 번호를 표준 wiringPi pin 번호 매기기가 아닌 BCM_GPIO 핀 번호 방식으로 해석하도록 지시 gpio -1 … - 선택 옵션 -1 flag 는 핀 번호를 헤더의 hardware pin numbers 로 해석하도록 지시 gpio -p … - 선택 옵션 -p flag 는 라즈베리 파이 보드에 PiFace board 가 장착되어 있는 것으로 가정하도록 하며, 뒤에 이어지는 명령은 PiFace 상의 핀에 대한 것으로 해석하도록 지시 - PiFace 상의 핀은 읽기와 쓰기에 200 ~ 207 을 사용하며, 208 ~ 215 사이의 핀은 출력 래치 레지스터의 상태를 읽는 용도로 사용 (즉, 출력 핀의 상태를 읽을 수 있음) gpio -x … - 선택 옵션 -x flag 는 추가의 확장 모듈을 초기화하는데 사용 - 확장 모듈은 이름(예: mcp23s17)과 콜론으로 구별된 파라미터로 정의됨 (→ 첫번 째 파라미터는 항상 확장 모듈의 베이스 핀 번호(base pin number)임.

WiringPi 라이브러리 : GPIO utility 표준 입출력 관련 GPIO utility gpio mode <pin> in/out/pwm/clock/up/down/tri - 핀의 모드를 input, output, pwm 혹은 clock mode 로 설정하며, 추가적으로 내부 pull-up/pull-down 저항 설정 gpio write <pin> 0/1 - 출력 핀을 high (1) 또는 low (0)로 설정 gpio pwm <pin> <value> - 핀에 대한 PWM 값 (0-1023 가능)을 설정 gpio read <pin> - 주어진 핀의 논리 값을 읽어 출력(값은 0 (low) 또는 1 (high)) 임 gpio awrite <pin> <value> - 주어진 핀의 아날로그 값(<value>)을 출력 - 기본적으로 on-board analog hardware 가 없으므로, 사용자는 -x flag 로 외부 모듈을 명시할 필요 gpio aread <pin> - 주어진 핀의 아날로그 값을 읽음 - 기본적으로 on-board analog hardware 가 없으므로, 사용자는 -x flag 로 외부

모듈을 명시할 필요 gpio readall - 일반적으로 접근하는 모든 핀을 읽어, 표의 형태로 출력 gpio wfi <pin> rising/falling/both - 상태가 특정 상태로 변할 때 까지, 하나의 GPIO 핀을 non-busy wait 상태로 만듦 (wfi => wait-for-interrupt)

WiringPi 라이브러리 : GPIO utility Kernel module 적재(load) 관련 GPIO 명령 gpio load spi [buffer size in KB] - SPI kernel modules 모듈을 적재하고, 선택 사항으로 내부 버퍼 크기를 kB 단위로 설정 (기본 버퍼 크기 = 4kB) gpio load i2c [baud rate in Kb/sec] - I2C kernel modules 을 적재하고, 선택 사항으로 baud rate(보율)을 주어진 Kb/sec 로 설정 (기본 보율 100Kb/sec) I2C detection 관련 GPIO 명령 gpio i2cdetect - I2C 버스에 대한 올바른 인자로 i2cdetect 명령을 수행함 (예: i2cdetect -y 0 : Rev. 1 Pi, i2cdetect -y 1 : a Rev. 2 Pi)

WiringPi 라이브러리 : GPIO utility /sys/class/gpio mode 관련 GPIO 명령 gpio export <pin> in/out - 주어진 핀 (BCM-GPIO 핀 번호)를 입출력으로 export 함 gpio unexport <pin> - 주어진 핀의 export 를 해제함 gpio unexportall - 모든 /sys/class/gpio exports 를 제거함 gpio exports - /sys/class/gpio interface 를 통해 export 된 모든 gpio 핀의 목록과 모드를 출력 gpio edge <pin> rising/falling/both/none - 주어진 핀에 대해 인터럽트 triggering edge 를 설정함 (rising, falling 혹은 양쪽 edges) - 아무런 edge 를 적지 않을 경우, disable 시킴

(예 1) wiringPi 핀 번호 방식 gpio mode 0 out gpio write 0 1 (예 2) BCM_GPIO 핀 번호 방식 gpio -g mode 17 out gpio -g write 17 1 (예 3). Physical P1 핀 번호 방식 gpio -1 mode 11 out gpio -1 write 11 1 WiringPi 라이브러리 : GPIO utility

WiringPi 라이브러리 : GPIO utility 내부 pull-up 과 pull-down 저항 관련 GPIO 명령 - 핀이 입력 모드일 때, 소프트웨어에 의해 제어되는 내부 pull up 혹은 pull-down 저항을 설정함 (예) gpio mode 0 up gpio mode 0 down gpio mode 0 tri Pull-up 저항을 없도록 함

4.4 라즈베리 파이 유/무선 랜 설정

라즈베리파이의 가장 큰 장점은 신용카드 크기의 초소형이며, 소비전력은 최대 700mA 인 저전력의 미니 컴퓨터라는 것이다. 이 장점을 살리기 위해서는 라즈베리 파이의 액세서리인 모니터, HDMI 케이블, 마우스, 키보드, 유선랜 케이블을 제거하고 별도의 PC 에서 SSH 에 접속하거나 RDP(Remote Desktop Protocol)을 이용하여 원격 접속하는 방법을 권장한다. 무선 환경 구축을 위한 무선랜 설정에 대해 알아보자.

1. 라즈비안은 다음의 명령어로 유/무선 인터넷의 환경설정이 가능하다.

```
$sudo nano /etc/network/interfaces
auto lo

iface lo inet loopback
iface eth0 inet dhcp

allow-hotplug wlan0
iface wlan0 inet manual
wpa-roam /etc/wpa_supplicant/wpa_supplicant.conf
iface default inet dhcp
```

를 입력하고 Ctrl-X 를 입력하면 저장할까요 ? 메시지에서 Y 를 누르고 파일명은 같은 것으로 하므로 Enter 를 치고 빠져나온다.

유/무선 인터넷의 환경설정 파일인 interfaces 에는 기본적인 설정되어 있다. 유무선 인터넷 모두 자동으로 할당 받는 DHCP(Dynamic Host Configuration Protocol)로 설정되어 있다. 또한 다음의 파일을 수정하는 것으로 라즈비안의 무선 인터넷 WPA(Wi-Fi Protocol Access) 설정이 가능하다.

```
$sudo nano /etc/wpa_supplicant/wpa_supplicant.conf
ctrl_interface=DIR=/var/run/wpa_supplicant GROUP=netdev
update_config=1
```

현재 등록된 무선랜 WPA 정보가 없다.

라즈베리파이에는 무선랜 어댑터가 포함되어 있지 않지만, 최근에는 저가이면서 초소형 USB 무선랜 어댑터 제품이 많이 출시되고 있다. 라즈베리파이에 맞는 USB 랜 어댑터는 다음의 사이트에서 호환성을 미리 검토하고 구매해야 한다. 라즈베리파이 호환 동글 목록은 다음을 참조한다.

http://elinux.org/RPi_VerifiedPeripherals#USB_Wi-Fi_Adapters

국내에는 ipTime 사의 NCUBIC, N100mini, N150UA 등의 제품이 소형이면서 호환성도 문제가 없다.

2. 무선인터넷 수동설정 - USB WiFi 동글을 꼽지 않은 상태에서 부팅한다. USB 동글이 꼽혀 있으면 설정할 때 에러가 날 수 있다. 아래와 같은 명령어를 실행한다. 기존 인터페이스 설정파일을 카피해 둔다.

```
sudo cp /etc/network/interfaces /etc/network/interfaces.original


sudo nano /etc/network/interfaces
```

nano 편집기가 뜰 것이다. 그러면 다음과 같이 바꿔준다. 원본은 .original 로 백업했으므로 파일을 지우고 수정한다.

```
$sudo nano /etc/network/interfaces
auto lo

iface lo inet loopback
iface eth0 inet dhcp

allow-hotplug wlan0
auto wlan0

iface wlan0 inet dhcp
wpa-ssid huins
wpa-psk abcd0000
```

wpa-ssid huins: 사용자의 SSID(Service Set Identifier 서비스 세트 식별자)입력

wpa-psk abcd0000 : 사용자의 PSK(Pre Shared Key 사전 공유키) 입력

pi@raspberrypi ~ $ ifconfig 로 확인하면

```
wlan0     Link encap:Ethernet  HWaddr 64:e5:99:f7:98:69
          inet addr:192.168.1.222  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:1549 errors:0 dropped:9 overruns:0 frame:0
          TX packets:284 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:206185 (201.3 KiB)  TX bytes:46179 (45.0 KiB)
```

무선랜 IP 주소가 잘 잡혀있는 것을 확인할 수 있다.

4.5 SSH 접속(원격접속)

SSH(Secure SHell)는 네트워크를 통해 원격으로 다른 호스트에 액세스하는 과정을 보호하기 위한 의도로 설계되었다. SSH 는 더욱 우수한 인증 기능뿐만 아니라 SCP(Secure Copy), SFTP(Secure File Transfer Protocol), X 세션 전달 및 포트 전달과 같은 기능을 제공하여 네트워크에서 교환되는 데이터를 암호화함으로써 기타 비보안 프로토콜의 보안성을 강화한다. 사용 가능한 암호화 유형은 512 비트 암호화에서 Blowfish, Triple DES, CAST-128, AES(Advanced Encryption Scheme) 및 ARCFOUR 와 같은 암호를 포함한 32768 비트 암호화에 이르기까지 다양하다.

1. 라즈베리 파이 설정

라즈베리 파이 설정 명령인 $raspi-config 에서 Advanced Option 의 A4 SSH 항목을 Enable 로 설정해야 사용할 수 가 있는데 초기에 기본설정으로 Enable 되어 있다.

2. PuTTy 다운로드(윈도우용 SSH 프로그램)

윈도우에서는 기본적으로 SSH 기능을 수행하는 프로그램이 없으므로 SSH 와 관련된 오픈 소스 프로그램인 PuTTY 를 다운로드 받아 설치한다.

다운로드 사이트 :

http://www.chiark.greenend.org.uk/~sgtatham/putty/download.html 에서 For Windows on Intel x86 항목의 Putty.exe 를 다운받아 실행하면 된다.

3. 다운로드 받은 putty 프로그램을 실행하여 Host Name 영역에 라즈베리 파이의 네트워크 IP 주소를 입력하자. 라즈베리 파이의 IP 주소는 부팅시에도 나오지만 콘솔에서 $ifconfig 명령으로 확인할 수 있다.

```
pi@raspberrypi ~ $ ifconfig
eth0      Link encap:Ethernet  HWaddr b8:27:eb:f0:fd:96
          inet addr:192.168.1.238  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
```

위와 같이 확인한 IP 주소를 입력하고 하단의 [Open] 버튼을 클릭하면 바로 SSH 접속이 시작된다. 이렇게 접속을 하게 되어 윈도우 상에서 작업을 하게 되면 마우스와 키보드 연결은 필요가 없게 된다. 그러나 라즈베리 파이의 IP 주소를 확인할 방법이 없기 때문에 UART 통신을 사용해 접속하는 방법을 사용해야 한다.

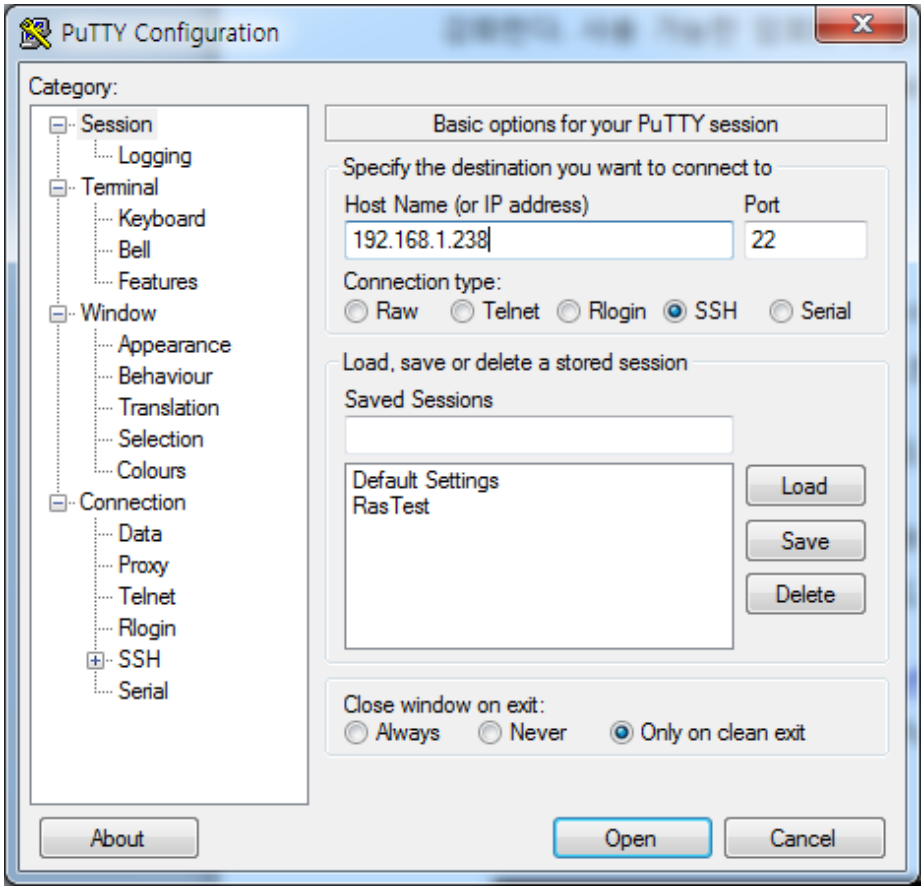


그림 51 PuTTY 설정 화면

다음은 SSH 를 접속한 후 로그인 과정을 거쳐 접속한 화면이다.

```
login as: pi
pi@192.168.1.238's password: Raspberry (암호는 화면에 보이지 않는다)
```

```
pi@raspberrypi ~ $
login as: pi
pi@192.168.1.247's password:
Linux raspberrypi 3.18.3+ #738 PREEMPT Sun Jan 18 17:37:14 GMT 2015
armv6l

The programs included with the Debian GNU/Linux system are free software;
```

```
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Apr  3 11:57:05 2015
pi@raspberrypi ~ $
```

4.6 삼바접속

삼바(samba)는 SMB(Server Message Block) 또는 CIFS (Common Internet File System)로 알려져 있으며 리눅스와 윈도우간에 파일 및 프린터를 공유할 수 있게 해주는 프로그램이다. 삼바를 통해 리눅스 서버를 타운영체제와 파일을 공유할 수 있는 파일서버로도 사용할 수 있다. 이와같이 삼바가 공유할 수있는 이유는 SMB(Server Message Block)/CIFS(Common Internet File System)라는 프로토콜을 이용함으로써 가능해졌다.



SMB 란 ?

SMB(Server Message Block)는 마이크로소프트사와 인텔이 윈도우 시스템이 다른 시스템의 디스크나 프린터와 같은 자원을 공유할 있도록하기 위해 개발된 프로토콜이다. TCP/IP 기반하의 NetBIOS 프로토콜을 이용하기 때문에 이 프로토콜은 NFS, NIS, lpd 와 같은 유닉스의 분산인증구조와 유사하다. 따라서 윈도우 중심의 네트워크 환경에서는 리눅스를 이용한 공유시스템에서는 SMB가 필수적이다..



CIFS 란?

CIFS(Common Internet File System) 는 네트워크를 위한 SMB 파일 공유 프로토콜의 확장된 버전이며, 윈도우와 유닉스 환경을 동시에 지원하는 인터넷의 표준 파일 규약의 프로토콜이다. CIFS는 이전의 폐쇄적인 SMB

프로토콜과는 달리 CIFS 규약 정의는 여러 유닉스 업체의 참여하여
결정된 내용이기 때문에 삼바도 버전이 올라감에 따라(삼바 버전 2.2 이상)
CIFS 규약을 잘 준수하여 안정성이 상당히 향상 되었다.

즉, 삼바를 이용하면 라즈베리파이의 파일을 윈도우 환경에서 수정하거나 자유롭게 생성하할 수 있다. 즉 윈도우 탐색기에서 하는 기능을 그대로 라즈베리파이 환경에서 구동된다. 이를 이용하면 영상이나 음성 파일을 자유롭게 공유할 수 있고 라즈베리파이를 이용한 개발에도 매우 편리한 환경을 갖게 된다.

1. 삼바 설치

우선 삼바 프로토콜 및 삼바서버, 클라이언트와 관련된 의존성 패키지를 다음의 명령어를 이용하여 설치한다.

```
$sudo apt-get install samba samba-common-bin
```

2. 필요한 메모리를 허용

설치도중에 필요한 공간 할당을 물어오는데 y 를 입력하면 설치가 계속된다.

```
pi@raspberrypi ~ $ sudo apt-get install samba samba-common-bin
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  tdb-tools
Suggested packages:
  openbsd-inetd inet-superserver smbldap-tools ldb-tools ctdb
The following NEW packages will be installed:
  samba samba-common-bin tdb-tools
0 upgraded, 3 newly installed, 0 to remove and 0 not upgraded.
Need to get 6,119 kB of archives.
After this operation, 36.1 MB of additional disk space will be used.
Do you want to continue [Y/n]? y
```

3. 삼바 유저 추가 및 패스워드 설정

다음의 명령어로 pi 라는 삼바유저를 추가한다. 그러면 패스워드를 물어볼 것이다. 자신이 원하는 패스워드를 입력해준다.

```
$sudo smbpasswd -a pi
```

아래와 같이 실행되어 패스워드를 두번 설정한다. 패스워드는 raspberry 로 설정하였다.

```
pi@raspberrypi ~ $ sudo smbpasswd -a pi
New SMB password:
Retype new SMB password:
Added user pi.
pi@raspberrypi ~ $
```

4. 삼바설정

다음의 명령어로 삼바 설정 파일을 수정해주어야 한다. 다른 설정을 변경할 필요없이 파일의 제일 하단에 새로운 유저 pi 에 대한 설정을 추가한다.

```
$sudo nano /etc/samba/smb.conf
```

명령을 사용하면 터미널에서 nano 편집기가 실행된다. 화살표키를 이용하여 제일 아래로 내려가서 다음 명령줄들을 입력한다. 입력이 끝나면 ctrl-x 를 누르면 저장할것이냐고 물어오면 y 를 누르고 같은 파일 이름이므로 enter 를 누르고 빠져나오면 된다.

```
[pi]
comment = rpi samba server by girin
path = /home/pi
valid user = pi
writable = yes
browseable = yes
```

comment : 유저 서버 설정에 대한 코멘트로 마음대로 적는다.

path : 공유할 폴더를 지정한다. 작업 폴더인 /home/pi 를 지정한다.

valid user : 사용 가능한 유저를 지정

writable : 쓰기를 가능하게 할 것인지에 대한 옵션

browseable : 공유 폴더의 목록을 보여주는 옵션

5. 삼바 재실행

설정된 내용이 반영될 수 있도록 삼바를 재실행시켜준다.

```
$sudo service samba restart
```

6. 공유폴더접속

윈도우의 탐색기 혹은 네트워크를 열어서 폴더 주소 칸에 ₩₩192.168.1.238₩ 과 같이 라즈베리파이의 IP 주소를 적어주면 아래와 같이 공유를 설정해둔 라즈베리 파이 폴더에 접속할 수 있다. 여기에서 새로운 디렉토리를 만들고, 파일의 수정, 삭제, 복사, 이동이 가능하게 된다.

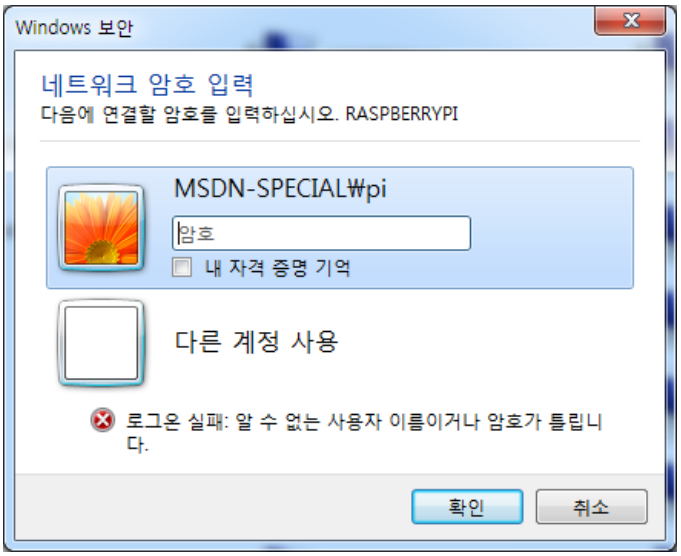


그림 52 **삼바를 이용한 파일공유**

참고자료

[1] "라즈베리파이 공식 사이트", http://www.raspberrypi.org/
[2] "element14 의 레즈베리파이 커뮤니티", http://www.element14.com/community/groups/raspberry-pi
[3] "RASPBERRY PI INFORMATION AND VIDEOS", http://raspi.tv/
[4] "Raspberry Pi Spy", http://www.raspberrypi-spy.co.uk/
[5] "RaspiHub", http://www.raspihub.com/
[6] "Adafruit Blog", http://www.adafruit.com/blog/category/raspberry-pi/
[7] "Raspberry Pi Pod", http://www.recantha.co.uk/blog/
[8] "bcm2835", http://www.open.com.au/mikem/bcm2835/

[9] “Gordons Projects”, https://projects.drogon.net/raspberry-pi/wiringpi/

Chapter 05 IoT 카메라 연동

5.1 라즈베리 파이 카메라 모듈

(1) Raspberry pi camera 설명

라즈베리 파이 카메라 모듈은 라즈베리 파이를위한 부가 설계된 확장형 모듈이다. 보드의 상단 표면에 두 개의 작은 소켓 중 F-PCB 컨텍터를 통하여 라즈베리 파이에 연결된다..



그림 53 라즈베리 파이 카메라 모듈

이 인터페이스는 특히 카메라의 인터페이스를 위해 설계 되었고, 전용 CSI 인터페이스를 사용하고, CSI 버스는 매우 높은 데이터 속도로 동영상을 실시간으로 보는데도 지장이 없다. 보드 자체는 약 25mm X 20mm X 9mm 이며 , 무게는 3g 이다. 카메라가 CSI 버스, 프로세서 카메라에서 픽셀 데이터를 전송하는 높은 대역폭 링크를 통해 파이의 BCM2835 프로세서에 연결되어 있다.

센서 자체는 5 메가 픽셀의 해상도를 가지고 있으며, 고정 초점 렌즈가 내장되어있다. 스틸 이미지의 측면에서, 카메라는 2592 X 1944 픽셀의 정적인 이미지까지 촬영이 가능하다.

표 10 라즈베리파이용 카메라 모듈 사양

센서	OmniVision OV5647(8.5 x 8.5 x 5mm)
PCB 크기	25 x 20 x 9 mm

무게	3 g
해상도	5 Megapixels
지원 비디오 모드	1080p30, 720p60 and 640x480p60/90
영상/스틸 지원	정지영상 캡처 2592×1944 초당 30프레임의 1080p 지원 초당 60프레임의 720p 지원 초당 90프레임의 640x480p 지원

(2) 카메라 모듈 Software

표 11 라즈베리 카메라의 소프트웨어적 특성

Picture formats	JPEG (accelerated) , JPEG + RAW , GIF , BMP , PNG , YUV420 , RGB888
Video formats	raw h.264 (accelerated)
Effects	negative , solarise , posterize , whiteboard , blackboard , sketch , denoise , emboss , oilpaint , hatch , gpen , pastel , watercolour, film , blur , saturation
Exposure modes	auto , night , nightpreview , backlight , spotlight , sports , snow , beach , verylong , fixedfps , antishake , fireworks
Metering modes	average, spot, backlit, matrix
Automatic White Balance modes	off, auto , sun , cloud, shade, tungsten, fluorescent , incandescent , flash, horizon
Triggers	Keypress , UNIX signal , timeout
Extra modes	demo , timelapse , circular buffer , video with motion vectors , segmented video , live preview on 3D models

5.2 카메라 영상 제어(C, Java)

카메라 모듈을 사용하기 위해 하시려면 라즈비안부터 업그레이드를 해야 한다. 터미널에서 다음 명령을 입력하여 업그레이드를 수행한다.

```
$sudo apt-get update
```

```
$sudo apt-get upgrade
```

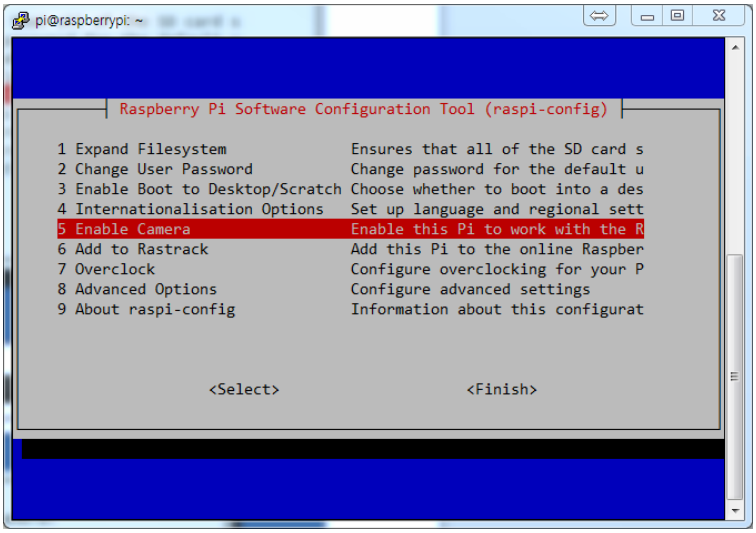
위 명령을 한 번에 실행하려면 아래와 같이 입력한다.

```
$sudo apt-get update && upgrade
```

이제 라즈비안의 환경 설정 유틸리티를 열고 카메라를 사용하겠다고 해주면 모든 준비는 끝난것이다.

카메라의 설정값은 disable 이 디폴트이다. 이를 enable 로 바꾸면 다음 부팅때부터는 카메라가 사용가능해진다. Finish 를 선택하면 라즈베리파이가 자동으로 재부팅된다. 환경 설정 유틸리티를 실행하는 명령은 다음과 같다.

```
$sudo raspi-config
```



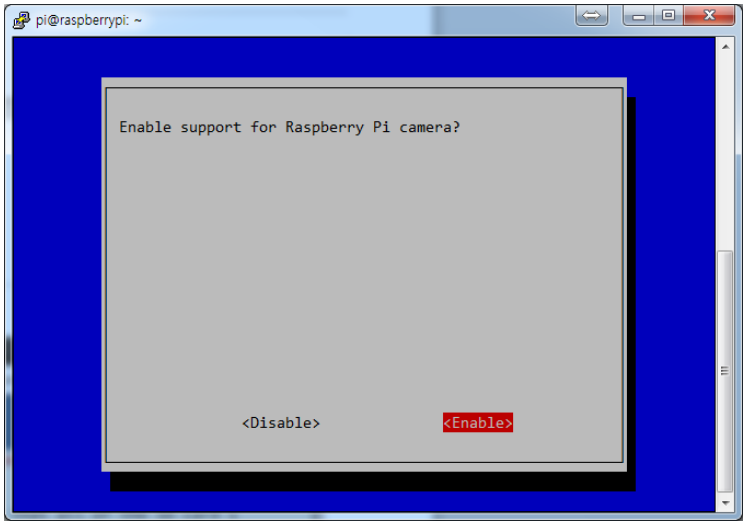


그림 54 라즈베리파이에서 카메라 사용 설정화면

라즈베리파이 설정에서 카메라를 활성화(Enable) 하고 재부팅을 한다. 라즈베리파이 카메라를 사용하기 위하여 부가적인 패키지를 설치해주어야 한다.

$sudo apt-get install mplayer netcat ffmpeg gpac

```
$sudo apt-get install mplayer netcat ffmpeg gpac
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  gpac-modules-base liba52-0.7.4 libav-tools libavdevice53 libavfilter2
libavfilter3 libavformat53 libavresample1
  libgpac2 liblircclient0 liblzo2-2 libmpeg2-4 libopencv-core2.3 libopencv-
imgproc2.3 libpostproc52 libxvmc1
Suggested packages:
  frei0r-plugins lirc mplayer-doc netselect fping
The following NEW packages will be installed:
  ffmpeg gpac gpac-modules-base liba52-0.7.4 libav-tools libavdevice53
libavfilter2 libavfilter3 libavformat53
```

```
    libavresample1 libgpac2 liblircclient0 liblzo2-2 libmpeg2-4 libopencv-core2.3
libopencv-imgproc2.3
    libpostproc52 libxvmc1 mplayer netcat
  0 upgraded, 20 newly installed, 0 to remove and 0 not upgraded.
  Need to get 10.3 MB of archives.
  After this operation, 24.1 MB of additional disk space will be used.
  Do you want to continue [Y/n]?
```

명령을 실행하고 24M 의 공간이 필요하다는 메시지가 나오면 'Y' 입력하여 설치를 진행한다. 이제 카메라를 테스트할 수 있는 몇 가지 명령을 알아보자. 우선, jpg 포맷으로 이미지를 캡처하려면 다음과 같이 입력한다.

```
$raspistill -o image.jpg
```

화면에 카메라 영상이 보이면서 5 초 후에 사진을 찍는 것을 확인할 수 있다.
$ls 명령어로 image.jpg 파일이 생성된 것을 볼 수 있다.
이번에는

ifconfig
h.264 포맷으로 5 초 동안 동영상을 캡처해 보는 명령이다.

```
$raspivid -o video.h264 -fps 30 -t 10000
```

-o : 출력파일 이름
-t : 촬영시간 밀리세컨드(10000 = 10 초)
-fps : 초당프레임
$ls 명령어로 확인하면 video.h264 파일이 생성되었다.
10 초로 늘리려면 다음과 같이 입력한다.

```
$raspivid -o video.h264 -t 10000
```

라즈베리 파이 카메라 프로그램은 H264 포맷을 사용하고 있기에 별도의 프로그램으로 보거나 일반적인 동영상 파일로 변환하려면 별도로 파일을 변환시켜줘야 한다. 다음의 명령어는 ffmpeg 을 이용하여 H264 포맷을 mkv 포맷으로 변환시키는 예제다.

```
$ffmpeg -r 30 -i video.h264 -vcodec copy video.mkv
```

확인해 보면 video.mkv 파일이 생성되었다.

다음의 명령어로도 가능하다.

```
$MP4Box -add video.h264 video.mp4
```

확인해보면 video.mp4 로 변환되었음을 확인해 볼 수 있다. 위에서 실행한 두 명령어의 다양한 옵션을 아래 명령어로 확인해 보자.

```
$raspivid | less


$raspistill | less
```

만일, CSI 에 연결되는 카메라 모듈이 아니라 개인적으로 사용하는 USB 에 연결하는 웹캠을 사용하기 위한 간단한 애플리케이션이 있다. luvcview 라는 애플리케이션인데, 설치 방법은 다음과 같다.

```
sudo apt-get install luvcview
```

웹캠이 올바로 동작하는지 간단한 테스트를 해보려면.

```
luvcview -s 320x240
```

이렇게 하면 화면에 3220x200 크기로 웹캠이 보이면 설치가 제대로 된것이다. 위에 설명한 옵션과 다양한 명령으로 원하는 카메라 제어를 할 수 있다.

5.3 동영상 스트리밍 재생

Smart IoT 설계 시스템은 카메라 모듈이 장착되어 있는 시스템이다. 카메라 모듈을 이용하여 Mjpg-Stream 을 구현한다. 속도가 많이 느린 편이지만 현장 상황을 웹을 이용하여 모니터링 하기에는 무리가 없다. 웹스트리밍 또는 기타 외부 기기에서 라즈베리 파이 카메라 모듈을 이용한 웹스트리밍 서비스를 하기 위해선 Mjpg-Stream 이 가장 최적화 되어 있다.

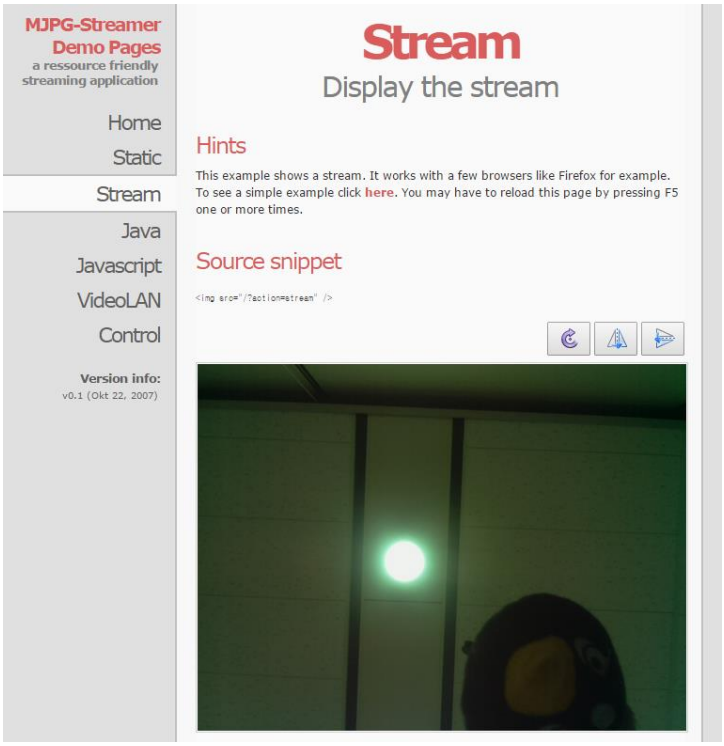


그림 55 mjpg-streamer 구동 화면

Mjpg-stream 컴파일을 위해 아래 영상관련 라이브러리 및 cmake 패키지를 설치 해 주도록 한다.

```
$sudo apt-get install git cmake libjpeg8-dev imagemagick -y
```

mjpg-streamer 를 컴파일 진행시에는 요구하는 videodev.h 헤더파일을 변경된 videodev2.h 파일로 링크 하자.

```
$sudo ln -s /usr/include/linux/videodev2.h /usr/include/linux/videodev.h
```

mjpg-streamer 다운로드 및 컴파일 하기

```
$git clone https://github.com/liamfraser/mjpg-streamer

Cloning into 'mjpg-streamer'...
remote: Counting objects: 1997, done.
```

```
remote: Total 1997 (delta 0), reused 0 (delta 0), pack-reused 1997
Receiving objects: 100% (1997/1997), 3.33 MiB | 813 KiB/s, done.
Resolving deltas: 100% (1150/1150), done.
```

Git 툴을 사용하여 다운로드 하였다. 다운 받은 폴더로 이동하자

```
$cd ~/mjpg-streamer/mjpg-streamer-experimental
pi@raspberrypi ~/mjpg-streamer/mjpg-streamer-experimental
$ make clean all
```

명령를 사용하면 컴파일이 실행된다. 앞의 과정들을 통해 mjpg-streamer 를 실행 할 수 있는 환경이 갖추어 졌으니, 마지막으로 파이 캠에서 mjpg-streamer 를 이용 할 수 있도록 자동 스크립트를 작성하고 명령어를 입력하여 웹 스트리밍을 구현 해 보자.

아래에 편집하는 파일은 root 의 mjpg.sh 파일이며 쉘 파일이므로 실행시에는 $sh mjpg.sh 명령으로 실행한다. Nano 편집기에 대한 상세한 설명은 실습문서 초반에 나와있으니 참고하기 바란다.

```
$ pi@raspberrypi ~ $cd ~
$ pi@raspberrypi ~ $sudo nano mjpg.sh

export STREAMER_PATH=$HOME/mjpg-streamer/mjpg-streamer-experimental
export LD_LIBRARY_PATH=$STREAMER_PATH
$STREAMER_PATH/mjpg_streamer -i "input_raspicam.so -d 200" -o "output_http.so -w $STREAMER_PATH/www"
```

편집이 끝나고 Ctrl-x 를 누르면 저장하려면 y 를 누르고, 저장하지 않으려면 n 를 입력하라는 메시지가 뜬다. Y 를 눌러 저장하고 나면 파일이름을 물어오는데 같은 이름이므로 enter 를 치고 빠져나온다. 아래 명령어로 실행한다.

```
$ pi@raspberrypi ~ $sh mjpg.sh
```

아래는 실행화면이다.

```
pi@raspberrypi ~ $ sh mjpg.sh
MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
 i: camera parameters.....:

Sharpness 0, Contrast 0, Brightness 50
Saturation 0, ISO 400, Video Stabilisation No, Exposure compensation 0
Exposure Mode 'auto', AWB Mode 'auto', Image Effect 'none'
Metering Mode 'average', Colour Effect Enabled No with U = 128, V = 128
Rotation 0, hflip No, vflip No
 o: www-folder-path...: /home/pi/mjpg-streamer/mjpg-streamer-experimental/www/
 o: HTTP TCP port.....: 8080
 o: username:password.: disabled
 o: commands.....: enabled
 i: Starting Camera
```

```
    DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c,    worker_thread(),    531):
Host init, starting mmal stuff
    DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c,    worker_thread(),    636):
Camera enabled, creating encoder
   Encoder Buffer Size 81920
    DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c,    worker_thread(),    719):
Encoder enabled, creating pool and connecting ports
    DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c,    worker_thread(),    777):
Starting output
   100 frames captured in 29.515364 seconds (3.388066 fps)
```

mjpg-streamer 웹 스트리밍 영상확인 mjpg-streamer 를 간단히 비유하자면 라즈베리파이 카메라를 통해 jpeg 파일로 생성된 이미지를 학창시절에 교과서 귀퉁이에 손으로 자그마하게 그림을 그리고 빠른 속도로 책장을 넘겨가면서 정지된 이미지를 파노라마처럼 스틸 샷으로 만들어 마치 영상물처럼 보던 원리와 같은 이치로 구현된다.

이러한 스틸 샷 영상은 " http://라즈베리파이아이피 :8080/?action=stream " 파일로 저장이 되는데 mjpg-streamer 라는 툴을 통해 이러한 여러 장의 스틸샷을 웹으로 전송하여 사용자들이 영상처럼 느끼고 볼 수 있도록 도움을 주는 툴이라 이야기 할 수 있고, 웹 이외에도 VLC Player 와 같은 툴을 이용해 스트리밍 영상을 확인 할 수가 있다.

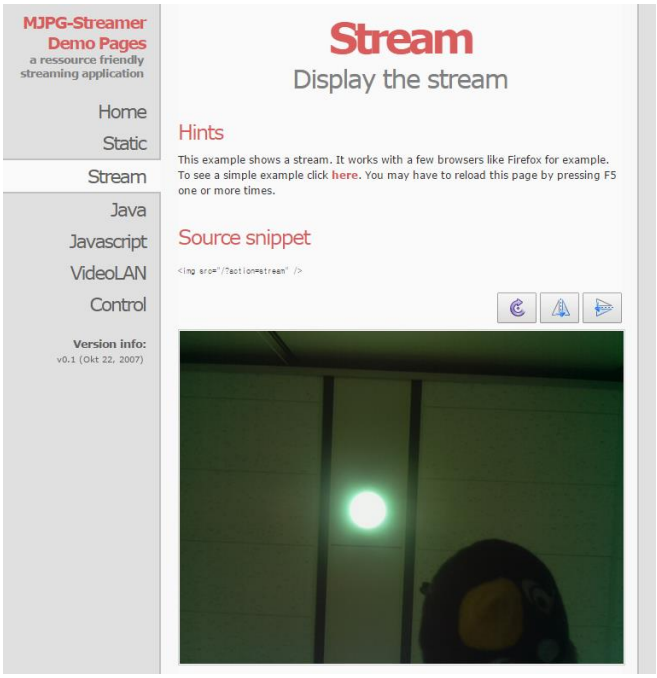


그림 56 http://라즈베리파이아이피:8080 접속 후, 스트리밍 영상 확인 화면

또한 VLC Media Player 로 확인하기 위해 우선 다운로드 한다. http://vlc-media-player.ko.softonic.com/download 에서 프로그램을 다운로드 하여 설치한다. 실행 후 미디어 메뉴에서 네트워크 스트림 열기 를 선택하여 라즈베리 파이의 주소를 넣어준다.

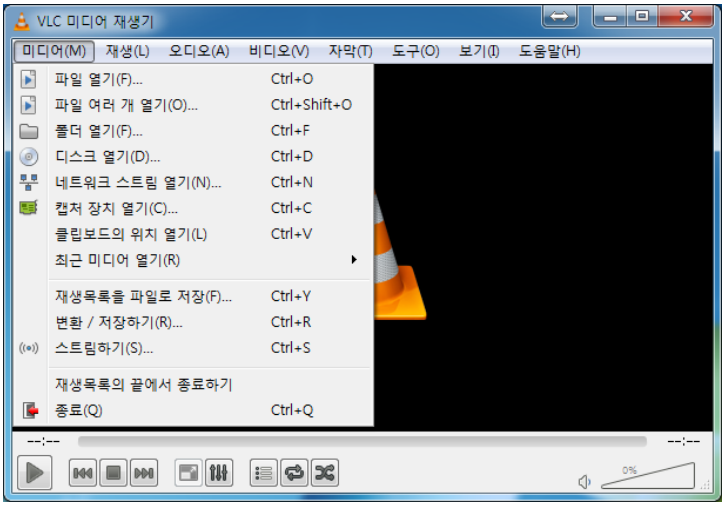


그림 57 VLC Media Player 화면

주소입력은 “http://라즈베리파이 IP 주소/?action=stream” 하면 된다. 예를 들면 http://192.168.1.247:8080/?action=stream 처럼 입력하면 화면이 나온다.

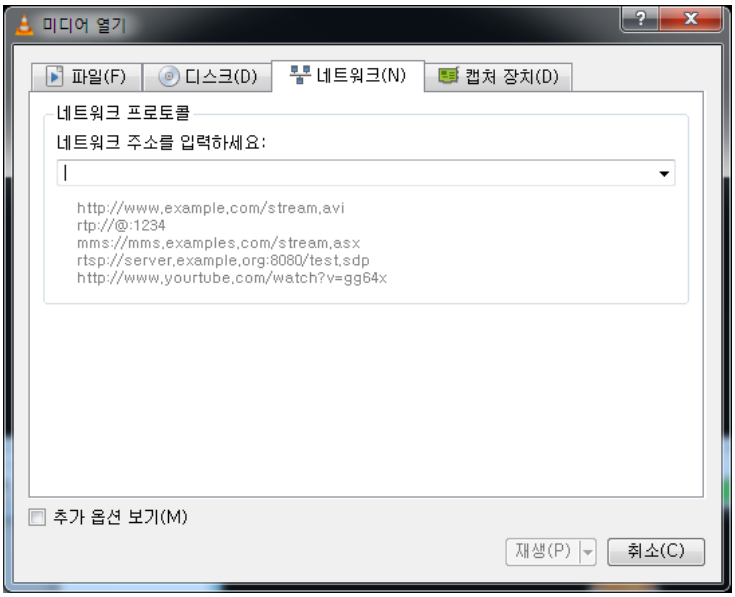


그림 58 라즈베리 파이 주소 입력

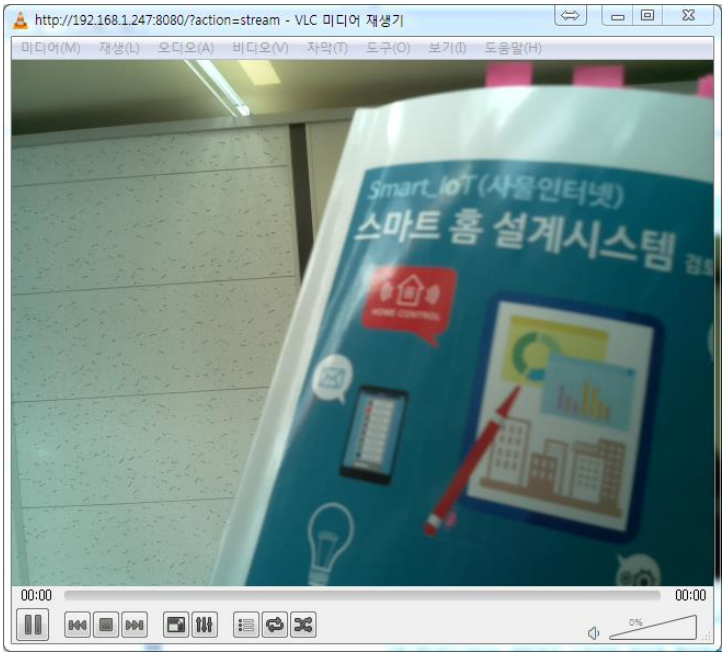


그림 59 VLC player 네트워크 스트리밍 영상을 확인하는 화면

5.4 타임랩스 사진 촬영법 1

앞에서 설명한 대로 이미지 촬영이나 동영상 캡쳐가 제대로 되었다면 이제는 타임랩스 사진을 찍어보는 실습을 진행한다. 타임랩스(time-lapse)를 사전에서 찾아보면 '사진촬영에서 저속 촬영의'라고 나와있다. 정상 속도(real time)보다 빨리 돌려서 보여주는 특수영상기법을 말하는 것이다.

(예: https://www.youtube.com/watch?v=Qs17Pra0ePw)

방법은 두 가지가 있는데 하나는 쉘 프로그램을 사용해서 리눅스에세 제공하는 명령어를 사용해서 최소 1 분 마다 사진을 찍게 하는 방법이고, 두 번째 방법은 파이썬 프로그램을 실행시켜 원하는 시간(초) 단위로 사진을 찍게 만드는 방법이다. 두 번째 방법은 다음장에서 설명할 것이다.

라즈베리 파이 케메라 모듈을 연결하고 enable 상태로 되어야 한다.

```
$raspistill -o cam.jpg
```

명령으로 사진 한 장을 찌고 확인한다

```
$ls
```

우선 쉘 프로그램을 작성한다.(운영체제의 커널과 사용자 사이를 이어주는 역할을 한다. 커널이란 운영체제의 일부로서 컴퓨터의 메모리에 항상 떠있는 하나의 프로그램이다. shell 은 상용자의 명령어를 해석하고 운영체제가 알아들을 수 있게 지시해준다. 다시 운영체제는 shell 에게서 받은 지시를 해석하여 하드웨어를 위한 지시어로 바꾸어준다.) **camera.sh 파일작성**

```
$nano camera.sh

#!/bin/bash

DATE=$(date +"%Y-%m-%d_%H%M")

raspistill -vf -hf -o /home/pi/camera/$DATE.jpg
```

이미지가 저장될 카메라 폴더를 생성한다.

```
$mkdir camera
```

쉘 프로그램을 실행할 권한을 바꿔준다.

```
$chmod +x camera.sh
```

아래 명령으로 실행하고 1~2 분후에 카메라 폴더를 확인한다.

```
$ sudo ./camera.sh
$ cd camera
$ls
```

주기적으로 실행해야 하는 프로그램으로 cron 을 사용한다. Crontab –e 명령을 사용하여 맨 아랫줄에 다음처럼 추가한다.

```
$ sudo crontab -e
```

```
.
.
.

# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
```

```
* * * * * /home/pi/camera.sh 2>&1
```

이렇게 하면 1 분 마다 주기적으로 사진이 camera 폴더에 저장될 것이다.

사용하지 않을 경우에는 $sudo crontab -e 파일을 열어서 추가한 코드를 지워주면 된다.

5.4 타임랩스 사진 촬영법 2

두 번째 방법은 파이썬 프로그램으로 약 3 초 간격부터 원하는 시간 타임으로 사진을 촬영하도록 코딩을 하고, 라즈베리 파이가 켜지면 언제든지 실행하도록 rc.local. 파일을 수정하는 방법이다.

우선 파이썬으로 아래 소스를 프로그램 한다. capture1sec.py

```
$nano capture1sec.py

#!/usr/bin/env python

import time as t
from datetime import datetime
import subprocess

FRAMES = 5000      #number of frame
TIMEBETWEEN = 6 #second

#time = datetime.now()
filepath = "/home/pi/camera"
fileNamePrefix = "image"

framecount = 0
#while True:               #if infinitely
while framecount < FRAMES:
   time = datetime.now()
   filename = filepath + "/" + fileNamePrefix + "-%04d%02d%02d-
%02d%02d%02d.jpg" % (time.year, time.month, time.day, time.hour,
time.minute, time.second)
   command = "raspistill -t 1 -o %s -n " % (filename)

   subprocess.call(command, shell = True)
   print "Captured %s" % filename

   t.sleep(1)
```

위 파이썬 프로그램을 실행시킨다.

$python capture1sec.py

```
pi@raspberrypi ~ $ python capture1sec.py
Captured /home/pi/camera/image-20150710-141720.jpg
Captured /home/pi/camera/image-20150710-141722.jpg
Captured /home/pi/camera/image-20150710-141724.jpg
Captured /home/pi/camera/image-20150710-141726.jpg
Captured /home/pi/camera/image-20150710-141728.jpg
```

종료는 ctrl-c 를 눌러 종료한다. 실행이 잘되면 우리는 위 프로그램이 라즈베리 파이가 부팅 될 때마다 항상 실행하록 하기 위하여 rc.local 파일을 수정해야 한다. 아래와 같이 파일을 열고 편집한다.

$sudo nano /etc/rc.local

```
#!/bin/sh -e
#
# rc.local
#
# This script is executed at the end of each multiuser runlevel.
# Make sure that the script will "exit 0" on success or any other
# value on error.
#
# In order to enable or disable this script just change the execution
# bits.
#
# By default this script does nothing.

# Print the IP address

/home/pi/capture1sec.py &

_IP=$(hostname -I) || true
if [ "$_IP" ]; then
  printf "My IP address is %s\n" "$_IP"
#  /home/pi/capture1sec.py &
fi

exit 0
```

간단하게 파일의 if ~ fi 를 제외한 곳에 백그라운드로 실행하도록 & 를 추가하여 코드를 넣은 것이다.

이렇게 하면 매번 부팅 될 때마다 2 초마다 촬영하는 프로그램이 실행될 것이다. 사진은 camera 폴더에 저장된다. 여기에는 5 천장까지 저장하게 되어 있는데 메모리 여분, 사진 매수에 따라 프로그래 실행을 제어하는 것이 필요하다. 사용자의 몫으로 남겨둔다.

이제 이렇게 하면 매번 부팅해서 터미널 화면을 보면 프로그램이 실행될 텐데 정지시키는 방법을 설명한다. 백그라운드로 실행되기 때문에 부팅 후 로그인 화면이 뜨면 아이디 pi, 비밀번호 raspberry 를 넣고 로그인 한다. 실행되고 있는 프로세스를 $ps -ax 명령어로 확인해서 PID 를 알아낸다.

```
pi@raspberrypi ~ $ ps -ax
warning: bad ps syntax, perhaps a bogus '-'?
See http://gitorious.org/procps/procps/blobs/master/Documentation/FAQ
  PID TTY      STAT   TIME COMMAND
    1 ?        Ss     0:01 init [2]
    2 ?        S      0:00 [kthreadd]
.
.
.
2800 ?        S      0:00 python /home/pi/capture1sec.py
.
.
.
```

사진 촬영 프로세스는 8200 번임을 보여진다. 여기서 아래 명령어를 사용하여 프로세스를 중지시킨다.

$sudo kill -9 2800

이렇게 하면 촬영이 중지되고 파일을 이동, 편집이 가능할것이다.

/etc/rc.local 파일은 부팅시 언제나 실행되는 파일을 넣기에 적당한 곳이다. 스마트 팜이든, 사진 촬영이든, 서버 파일도 마찬가지이다.

여기까지 라즈베리 파이 카메라를 이용하여 타임랩스 사진을 만드는 방법을 설명하였다. 유용한 기능을 아주 쉽게 사용하는 방식이다. 많은 활용을 기대한다.

Chapter 06 스마트폰 프로그램

6.1 스마트폰 통신기술

지금까지 등장한 스마트폰과 연결 장치 간 통신방법은 매우 다양하다. 그리고 스마트폰에는 통신을 위한 대부분의 장치들이 내장돼 있다. 따라서 어떤 장치를 사용해 스마트폰과 연결할 것인지가 핵심이다. 실제로 블루투스 통신기술은 비교적 저렴하게 연결 장치에 내장할 수 있지만, 와이파이 통신기술은 연결 장치에 내장하기에 가격이 매우 비싸다. 지금부터 스마트폰과 연결 장치 간 통신방법을 좀더 자세히 알아보자.

● 블루투스

앞서 설명한 대로 가장 저렴한 방식이다. 일반적인 스마트폰이라면 어떤 것이든 와이파이 및 블루투스 장치를 모두 내장하고 있으므로 다양한 스마트폰에서 사용할 수 있다. 그래서 이번 연재에도 블루투스를 사용한 통신방법을 주로 다룬다. 과거에 아이폰에서의 블루투스 통신은 애플에서 MFI(Made For IPhone/IPad) 인증을 받아야 장치를 만들 수 있었다. 하지만 블루투스 4.0 버전부터는 MFI 인증 없이도 아이폰과 연동되는 장치를 개발할 수 있다.

● Wi-Fi

스마트폰에서는 문제가 없지만 와이파이 모듈을 탑재한 연결 장치를 만들기 위해서는 하드웨어 사양이 좋거나 TCP/IP 와 같은 프로토콜이 내장된 와이파이 칩을 사용해야 한다. 이로 인해 제품 단가가 상승한다. 결국 고급 사양의 제품이거나 고기능을 요하는 제품에서 주로 사용되는 방식이다. 앞에서 설명한 트와인과 같은 장치들이 이 방법을 사용하고 있다.

● 헤드폰잭

스퀘어는 스마트폰상에서 간편하게 신용카드 결제 기능을 제공하는 혁신적인 서비스로 호평받았다. 그 스퀘어가 사용한 방식이 헤드셋을 장착할 수 있는 헤드폰잭을 사용한 것이다. 이 방식의 장점은 저렴한 부품으로 스마트폰과 연동되는 장치를 만들 수 있다는 것이다. 이러한 장점 때문에 스퀘어에서도 10 달러 대의 저렴한 카드리더를 판매해 서비스를 제공할 수 있다.



그림 60 스퀘어의 카드리더

● 근거리 무선통신

근거리 무선통신(Near Field Communication, NFC)은 13.56MHz 의 대역을 가진 매우 근거리의 무선통신을 위한 기술이다. 현재 지원되는 데이터 통신 속도는 초당 424kbit 다. 교통, 티켓, 지불 등 여러 서비스에서 사용할 수 있으며 스마트폰에서는 아래 그림과 같은 NFC 태그를 사용해 간단한 통신을 하거나 정보를 전달하는 용도로 쓰인다.



그림 61 NFC 태그

● 블루투스 통신기술

블루투스는 다른 어떤 통신 방식보다 통신 기능이 내장되지 않은 디바이스들 간의 무선 네트워크를 쉽고 값싸게 구현할 수 있게 돕는다. 그래서 블루투스는

휴대폰, 스마트 디바이스, 노트북, 태블릿 PC, PC, 심지어 TV 나 냉장고와 같은 백색가전에서의 네트워크 시대를 열어줄 기술로 인정받고 있으며 실질적인 성장은 최근의 스마트폰에서 많이 이루어졌다.

블루투스는 1994 년 이동통신 그룹인 에릭슨(Ericsson)이 휴대폰과 주변기기 사이에서의 낮은 소비전력과 가격을 보장하는 무선통신 인터페이스를 연구하면서 시작됐다. 그랬던 것이 1996 년 2 월에는 에릭슨, 노키아, IBM, 도시바, 인텔이 블루투스 SIG (Special Interest Group, www.bluetooth.com)를 결성했다. 현재는 마이크로소프트, 루센트(Lucent), 쓰리콤, 모토로라가 참여해 총 9 개 기업이 활약 중이며, 관련기업만 해도 2,000 여 곳에 이른다.

블루투스는 칩 당 가격이 5 달러 이하이며 100mW 라는 적은 전력으로 10m 또는 100m 내에서 2.4Ghz 의 ISM(Industrial, Scientific, and Medical) 밴드 주파수 대역을 사용한다. 이 때 10m 에서는 0dBm, 100m 에서는 +20dBm 출력이 가능한 전력증폭기를 사용한다. 블루투스 네트워크에서는 1 대 1 통신과 1 대 n 통신이 모두 가능하다.

● 지그비, 무선 랜(Wi-Fi), 블루투스의 비교

스마트폰 통신에 사용되는 대표적인 기술에는 블루투스를 비롯해 IEEE 802.11, 즉 무선 랜과 홈 RF 그리고 IrDA 등이 있지만, 가장 주목받는 것은 블루투스다. 왜냐하면 블루투스는 비교적 통신방식이 간단하고, 세계적인 칩과 스마트폰 제조사들 대부분이 블루투스를 채용하고 있으며, 저렴한 가격으로 공급되고 있기 때문이다. 또한 블루투스에서 사용하는 주파수 대역은 ISM 밴드로 거의 모든 국가에서 공통으로 사용하고 있어 별도의 인증과정 없이도 쓸 수 있다.

표 12 스마트폰에서의 근거리 통신방식 비교

	지그비	Wi-Fi	블루투스
통신 거리	10-100 m	50-100 m	10 - 100 m
네트워크토폴로지	Ad-hoc, peer to peer, star, 또는 mesh	Point to hub	Ad-hoc, 아주 작은 네트워크
동작 주파수	868 MHz (Europe) 900-928 MHz	2.4 and 5 GHz	2.4 GHz

	(NA) 2.4 GHz (worldwide)		
복잡성 (Device and application impact)	적음	높음	높음
전력 소비(배터리 옵션과 수명)	매우 적음	많음	중간 정도
Security	128 AES 와 application layer 보안	인터넷 보안	64 와 128 bit 암호화
적용 분야	산업 제어 및 모니터링, 센서 네트워크 빌딩 자동화, 홈 오토메이션 및 제어, 장난감, 게임	무선 LAN 연결, 광대역 인터넷 연결	기기간 무선 연결 ex> 전화기, PDA, 노트북, 헤드셋

● 블루투스 프로토콜 내부 소프트웨어 구성

블루투스 내부 소프트웨어는 크게 세 가지로 구분된다.

첫 번째로 블루투스 하드웨어와 밀접한 관계를 가지고 있으며 펌웨어(Firmware) 형태로 제공되는 연결 관리자(Link Manager)와 호스트 컨트롤러 인터페이스(Host Controller Interface, HCI)다. 이 둘은 베이스밴드(Baseband) 소프트웨어라고 불리며, 보통 칩 제조업체에서 제공한다.

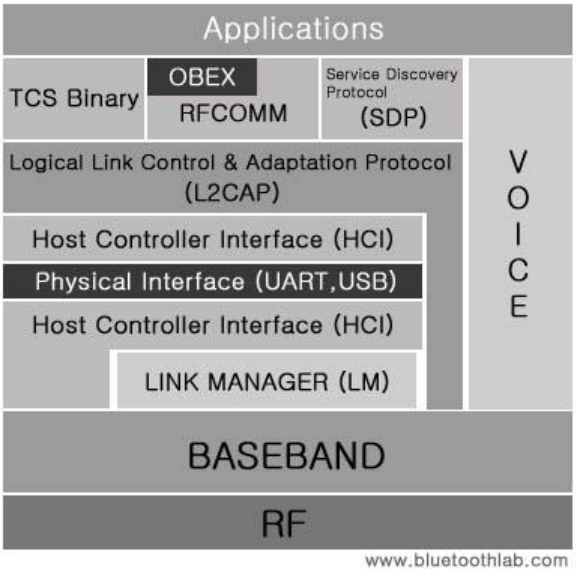


그림 62 블루투스 프로토콜 스택 구조

보통 HCI(Host Controller Interface)를 기준으로 호스트 컨트롤러(Host Controller) 프로토콜과 호스트(Host) 프로토콜로 나뉘게 된다. [그림 67]을 보면 HCI 가 두 개의 계층에 위치하는데 바로 아래쪽 HCI 가 호스트 컨트롤러에 포함되는 것이고, 위쪽의 HCI 가 호스트에 포함되는 것이다. 여기서 호스트 컨트롤러에 포함되는 HCI 를 `HCI Bottom', 호스트에 포함되는 HCI 를 `HCI Top'이라고 말하기도 하며, 두 개의 HCI 사이는 물리 링크인 UART, USB, PCMCIA 등의 인터페이스로 연결된다.

호스트 컨트롤러란 바로 블루투스 모듈에 해당한다. 그리고 호스트 컨트롤러 프로토콜은 보통 베이스밴드(Baseband), 링크 매니저(LM), HCI Bottom 정도가 포함된다. 대부분 이 세 개의 프로토콜이 펌웨어(Firmware) 형태로 모듈 내부에 포함된다. 호스트는 호스트 컨트롤러인 블루투스 모듈과 연결되어 블루투스 모듈을 제어하고 어플리케이션을 수행하는 곳으로 그 종류는 시스템에 따라 달라질 수 있다. 보통 PC, PDA, 핸드폰 등이 모두 호스트가 될 수 있고, 임베디드 시스템의 경우 마이크로 프로세서가 호스트가 된다. 또 호스트에 포함되는 프로토콜은 HCI Top 부터 그 상위 계층 프로토콜(L2CAP, RFCOMM, SDP, TCS, OBEX) 모두에 해당된다. 그러나 항상 상위 계층 프로토콜이 모두

포함되는 것은 아니고, 어플리케이션의 종류나 프로파일(Profile)에 따라 포함되는 프로토콜이 달라진다.

[블루투스 버전에 따른 분류]

블루투스의 다양한 버전별 내용을 간단하게 살펴보자.

▪ Bluetooth v2.0 + EDR

기존 최고 전송속도였던 721kbit/s 를 3Mbit/s 로 끌어 올렸다. 실제 전송 가능한 속도는 2.1Mbit/s 다.

▪ Bluetooth v2.1 + EDR

이전 버전과 가장 눈에 띄게 달라진 점은 손쉬운 페어링(Paring)이 가능하도록 SSP(Secure Simple Pairing) 기능을 추가한 것이다. 그 외에 연결 시 필터링이 쉽도록 EIR(Extended inquiry response)을 강화하고, 저전력 모드에서도 소비 전류를 줄일 수 있는 기능을 추가했다. 대부분의 안드로이드폰이나 초기 아이폰에서 사용된 블루투스 통신모드다.

▪ Bluetooth v3.0 + HS

여기서부터 비교적 최근에 나온 갤럭시 S2 나 동급 스마트폰에서 제대로 지원하기 시작했다. Bluetooth 3.0 + HS 는 이론적으로 24Mbit/s 이라는 엄청난 속도를 제공한다.

▪ 블루투스 4.0

소비전력을 줄인 저에너지 기술에 고속 무선통신을 지원하는 것이 특징으로 평균 전력과 대기전력을 줄여 동전만한 크기의 배터리로 수 년 동안 작동할 수 있다. 기존 3Mbps 전송속도를 24Mbps 로 확장해 실시간 비디오 전송이나 대용량 데이터 전송을 보다 쉽고 빠르게 구현할 수 있다. 블루투스 핵심 규격인 4.0 버전에서 가장 주요한 기능이 될 것으로 평가되는 저에너지 블루투스 기술의 특징은 다음과 같다.

- 극히 적은 피크전력, 평균전력 및 대기전력
- 동전 모양의 배터리로 수년간 작동할 수 있음
- 다수 업체 간 상호호환성
- 길어진 송수신 거리

또 다른 블루투스 4.0 핵심 규격은 싱글 모드와 듀얼 모드 두 형태로 구현할 수 있다는 점이다.

- 싱글모드 칩들은 통합성이 고도로 높고 크기가 매우 작은 디바이스들에 적합하다.
- 이 칩들은 극히 적은 대기모드 전력과 간단한 디바이스 감지 그리고 최소의 비용으로 고도의 절전 기능과 보안성이 높은 암호화된 연결을 통해 믿을만한 1:1 연결이나 1:N 연결 데이터 전송 기능

[스마트폰에서의 블루투스]

스마트폰에서 사용하는 주요 블루투스의 기능 및 프로파일은 다음과 같다.

- HFP(Hands-Free Profile)

일반적으로 핸즈프리를 지원하는 기기에서 사용되는 프로파일이다. 오직 음성만을 지원하며, 주로 자동차에 사용되는 블루투스 기기에 사용된다. 현재 최신 버전은 1.5 이며, iOS4 에서 지원한다.

- PBAP(Phone Book Access Profile)

전화번호부에 접근하기 위한 프로파일이다. 아이폰에서 홈 버튼을 계속 누르고 있으면 시작되는 음성 명령 애플리케이션 등이 PBAP 를 지원하는 프로그램이다

- A2DP(Advanced Audio Distribution Profile)

헤드셋이나 스피커로 음악을 전송할 때 사용하는 프로파일이다. 아이폰 3G 와 iOS3 버전부터 이 방식이 지원되기 시작했다. 아이폰으로 음악을 들을 경우 직접 화면을 보고 조절할 수 있다.

- AVRCP(Audio/Video Remote Control Profile)

오디오뿐만 아니라 비디오까지 컨트롤할 수 있다. 아이폰 3G, iOS4.1 버전부터 지원됐다.

- PAN(Personal Area Network Profile)

피코넷(piconet)이라고 불리기도 하는 이 프로파일은 최대 8 개의 장치와 마스터-슬레이브로 연결될 수 있는 연결방식으로, IEEE 802.15 에 정의돼 있다. 일반적인 지원영역은 10m 정도이며, 기기끼리 연결해 최대 100m 까지 사용할 수 있다. 데이터 교환을 위해서는 반드시 필요하며, 초기 아이폰을 제외하고는 모두 지원한다.

- HID(Human Interface Device Profile)

각종 키보드, 마우스, 게이밍 디바이스, 태블릿 등을 지원하는 프로파일이다. 아이폰 3GS 이상, iOS4 이상부터 지원되기 시작했다. 이 프로파일을 이용하면 블루투스 키보드를 아이폰이나 아이패드와 연결할 수 있다

지금까지 스마트폰과 연결 장치 간 프로그래밍을 위한 기초 정보를 살펴봤다. 다소 기술이고 용어를 설명하는 것에 중점을 뒀는데, 그 이유는 프로그래밍하기 위한 기초 지식이 필요했기 때문이다.

6.2 개발툴설치

안드로이드 프로그래밍을 학습하려면 우선 개발툴을 설치해야 한다. 개발에 필요한 모든 도구는 무료로 배포되므로, 인터넷 환경만 갖추어져 있으면 바로 설치 할 수 있다. 여기서는 기존 개발환경에 대해 간단하게 알아보고, 2013 년 중반에 패포한 새로운 안드로이드 통합 개발 환경인 Android Studio 설치방법을 자세하게 알아볼 것이다. 우선 기존 이클립스 개발환경 설치에 대해 알아본다. 설치 순서는 지켜서 설치하는 것이 바람직하다.

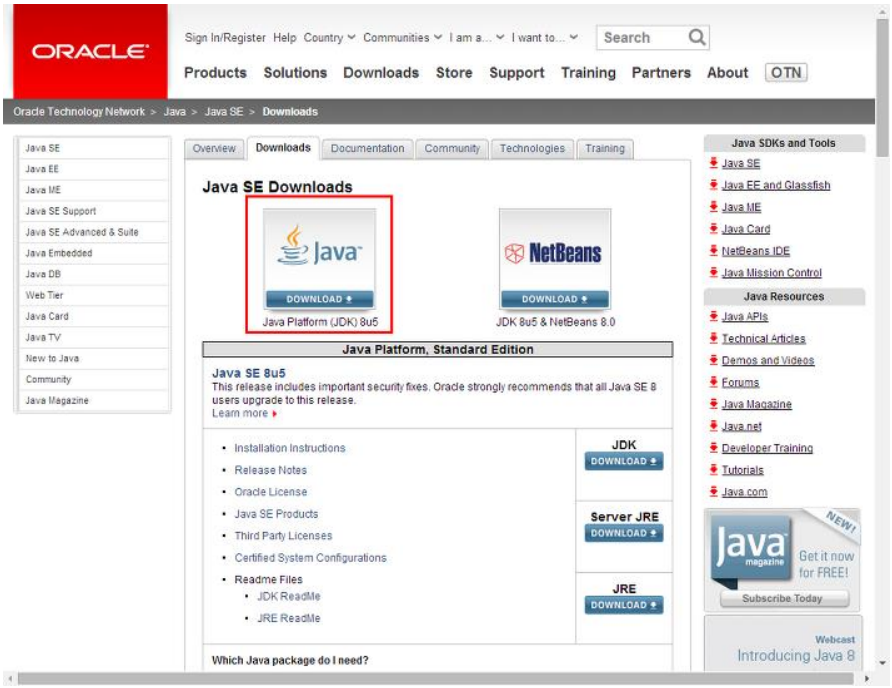
1. JDK
2. 이클립스
3. ADT 플러그인
4. 안드로이드 SDK
5. AVD 생성 및 옵션조정

위와 같은 순서로 설치하게 되면 기존의 이클립스 개발환경이 만들어 진다. 기존 방법의 자세한 설치방법은 인터넷 문서들을 참고한다. 여기서는 자바 개발환경인 JRE 설치와 안드로이드 스튜디오 설치만을 설명한다.

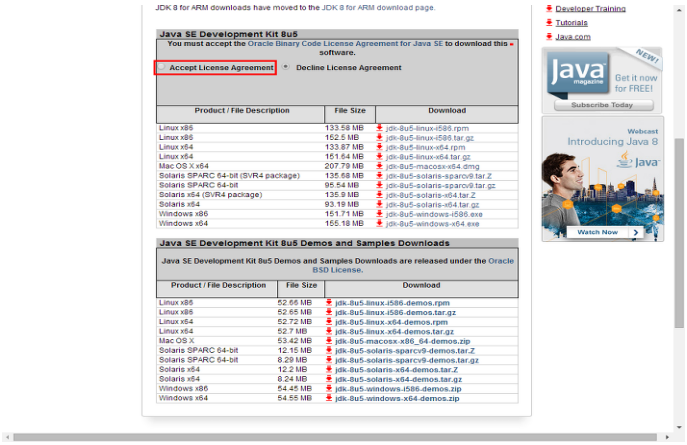
6.2.1 자바 설치 (JRE)

아래의 설치과정은 http://prolite.tistory.com/175 사이트의 자료를 참고하였다. 자바를 설치하면 자바 언어가 동작하고 안드로이드 역시 개발환경이 일부 셋팅된다.

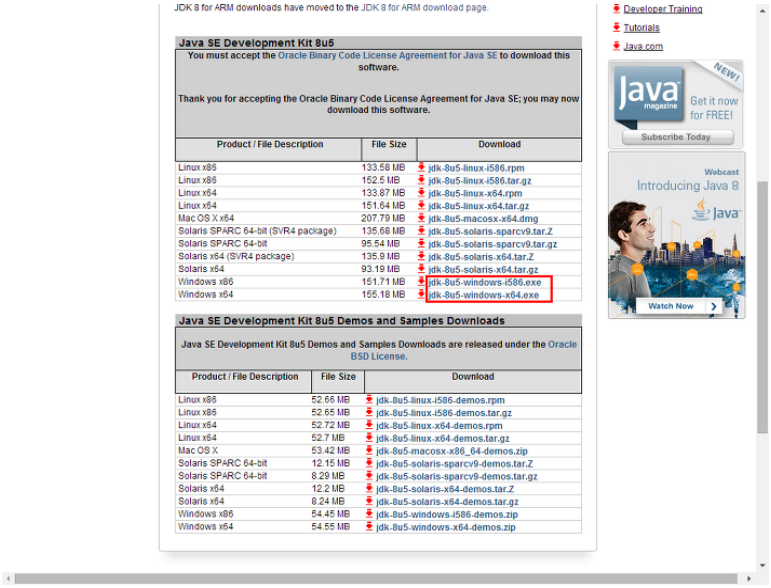
1) 아래 링크로 들어가서 Java Platform (JDK) 이것을 받습니다.
http://www.oracle.com/technetwork/java/javase/downloads/index.html?ssSourceSiteId=ocomen



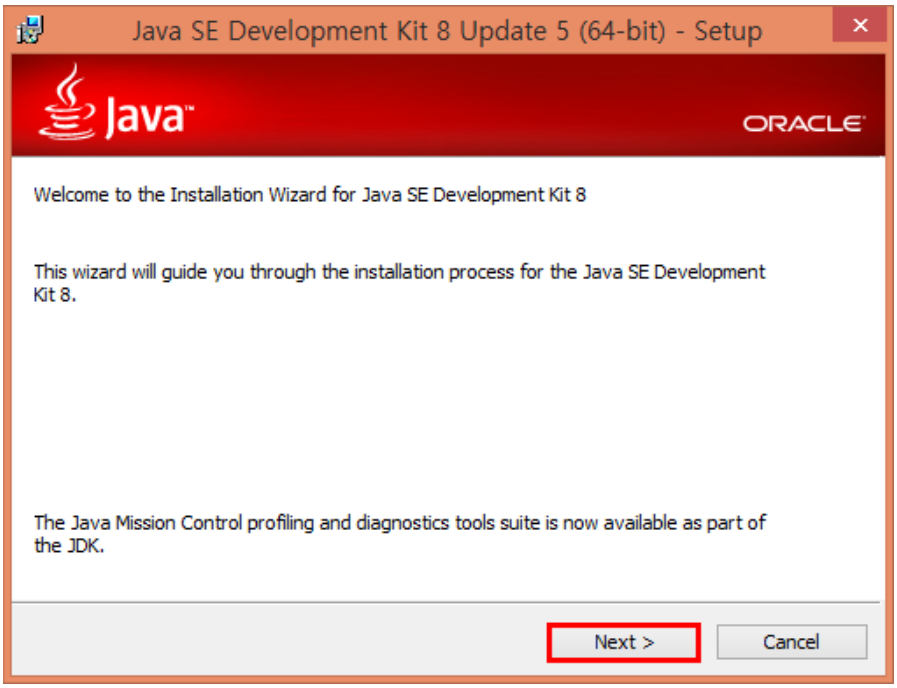
2) Accept License Agreement 옵션을 누릅니다.



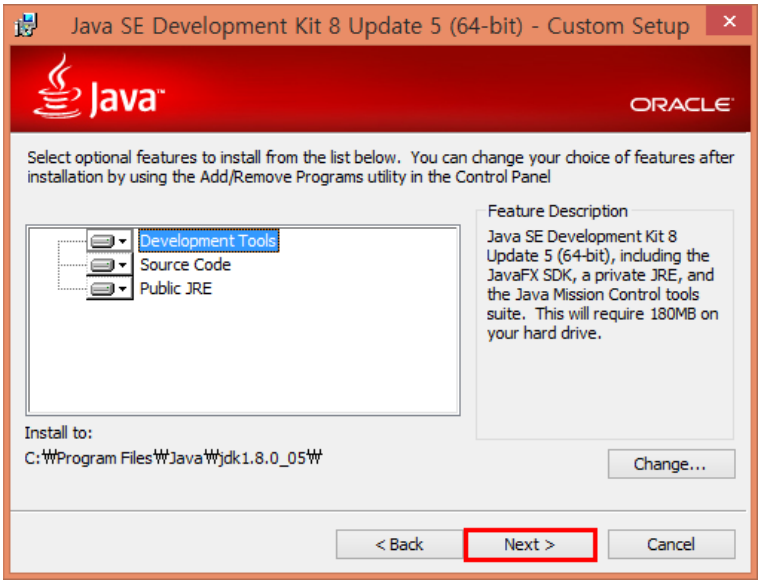
3) 각자 윈도우 별 비트에 맞게 다운로드 합니다.



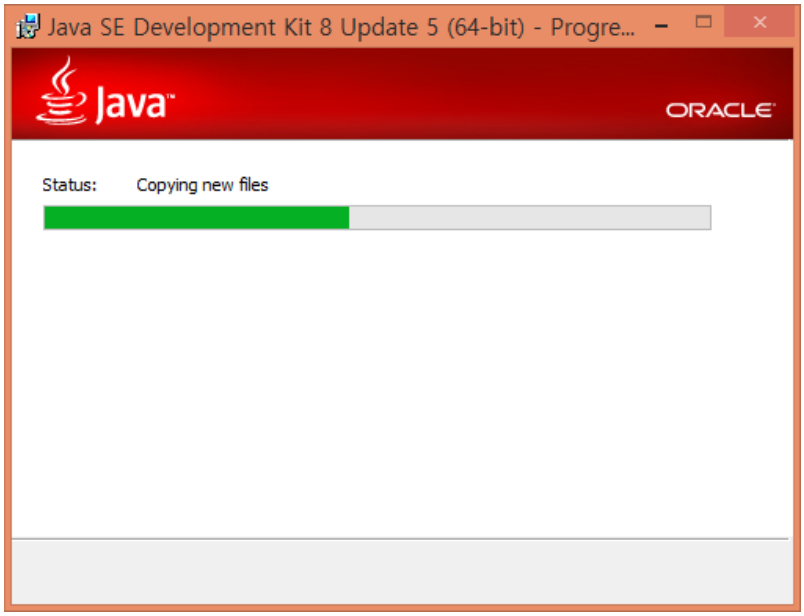
4) Next > 를 누릅니다.



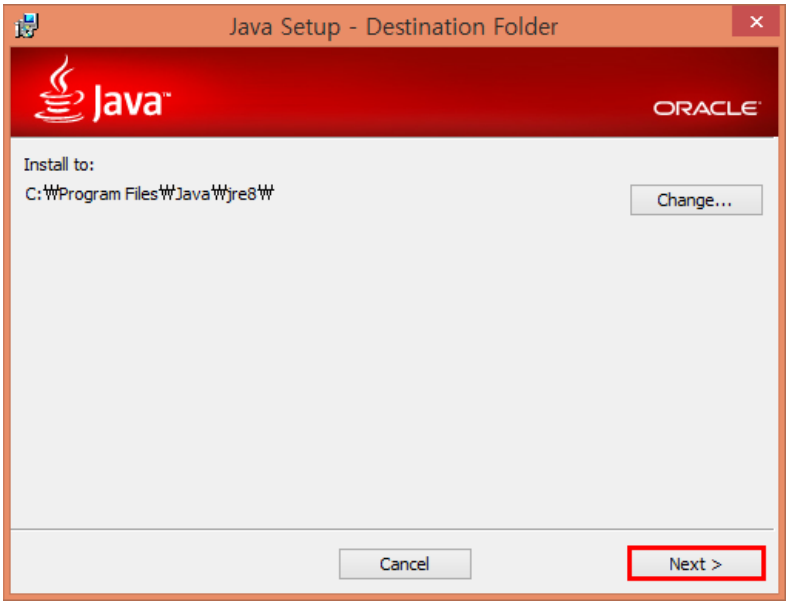
5) 설치 옵션인데 그냥 Next > 를 누릅니다.



6) 설치 합니다.



7) 옵션에 따라 설치창이 또 뜹니다. ext > 를 누릅니다.



8) 설치 합니다. 얼마 안걸립니다.

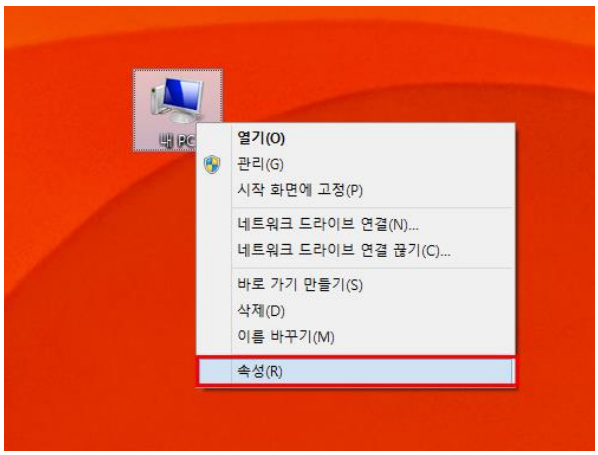


9) 설치 마무리가 되었습니다. Close 를 누릅니다.



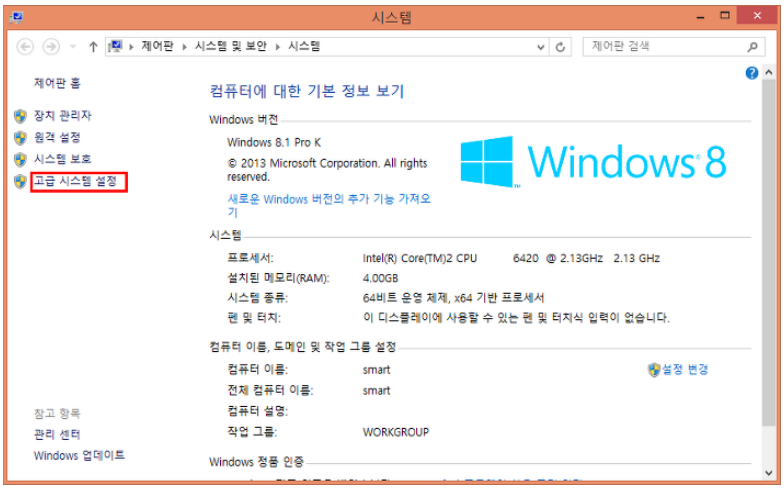
10) 설치는 완료 되었습니다. 본격적으로 자바를 사용하기 위해 환경 변수를 설정합니다. 설치 컴퓨터가 윈도우 8.1 로 업그레이드 된상태라 조금 달리 표시될 수도 있습니다.

내 컴퓨터" 혹은 "내 PC" 아이콘에 마우스 오른쪽 버튼을 누릅니다. 맨 아래 속성 버튼을 누릅니다.

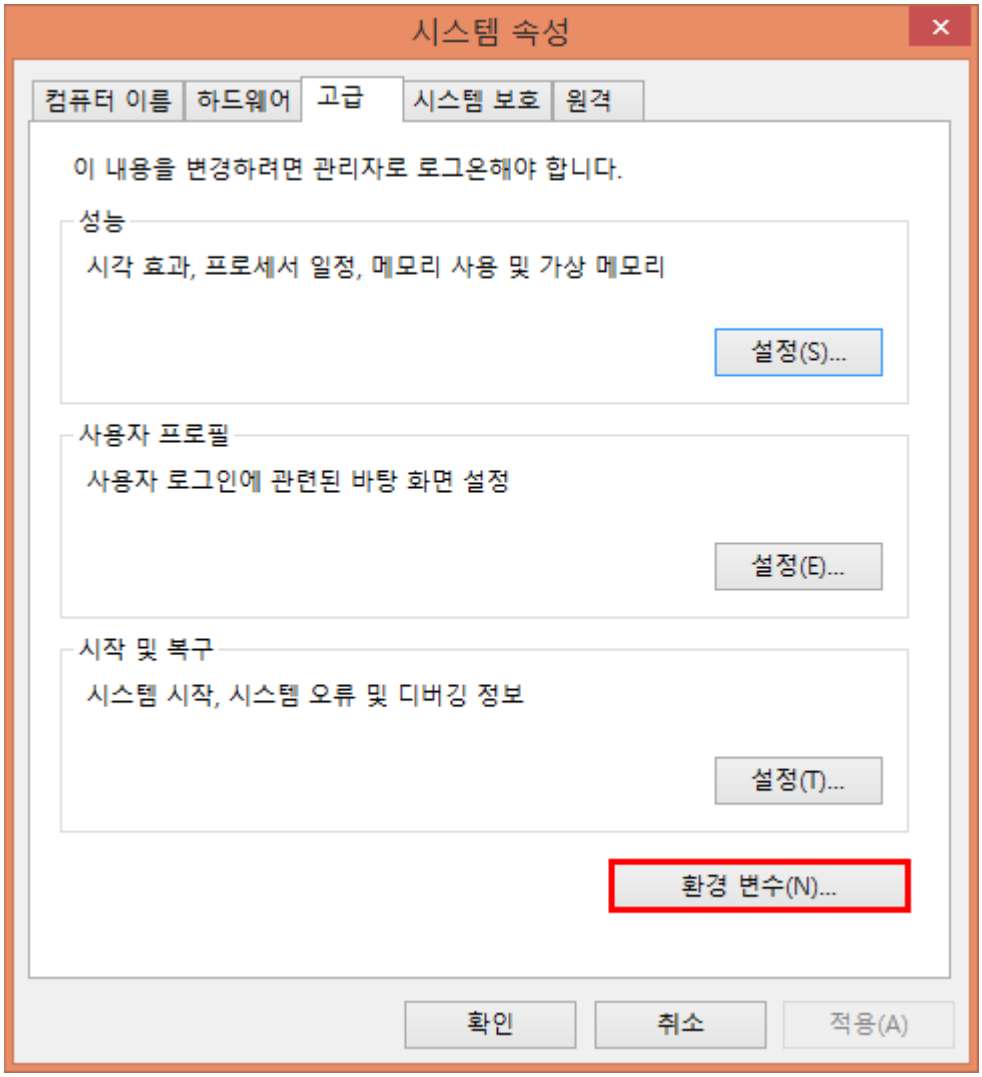


11) 고급 시스템 설정으로 들어갑니다.

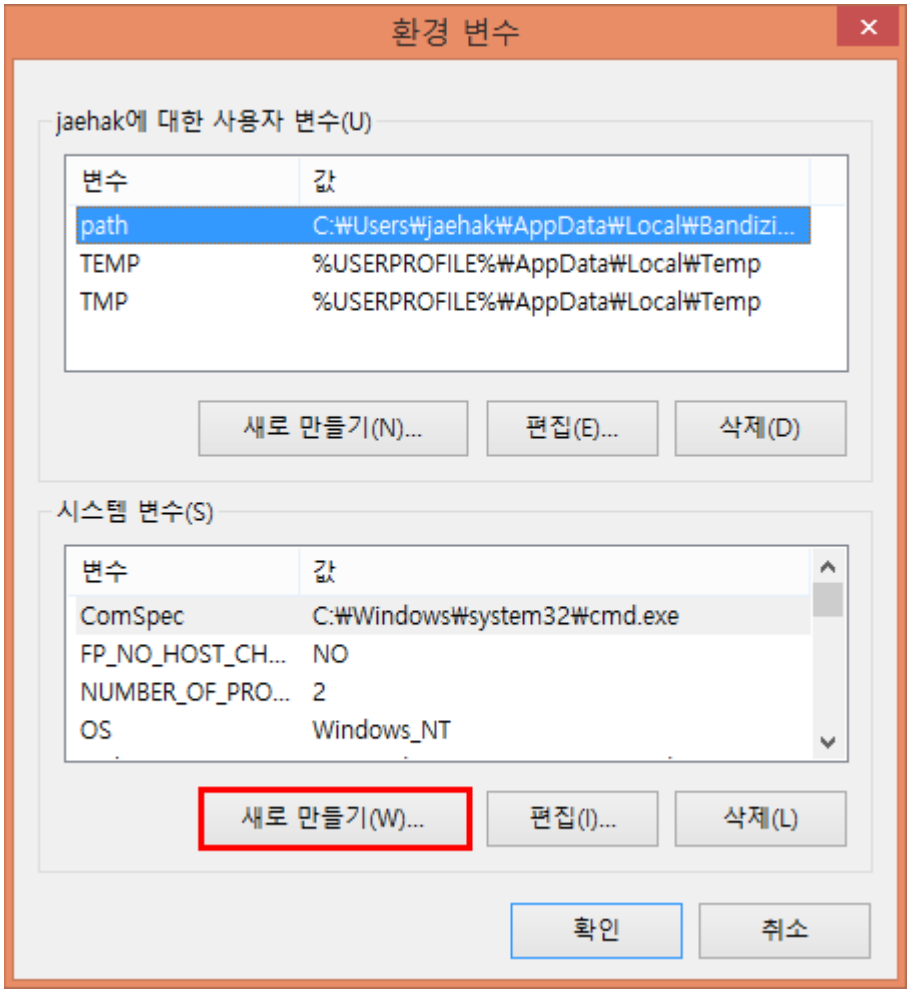
이것 역시 윈도우 별로 메뉴 위치나 이름이 조금 달리 보일 수 있습니다.



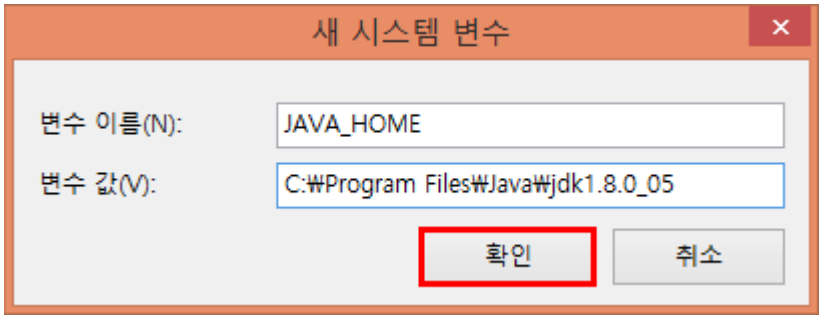
12) 환경 변수를 클릭합니다.



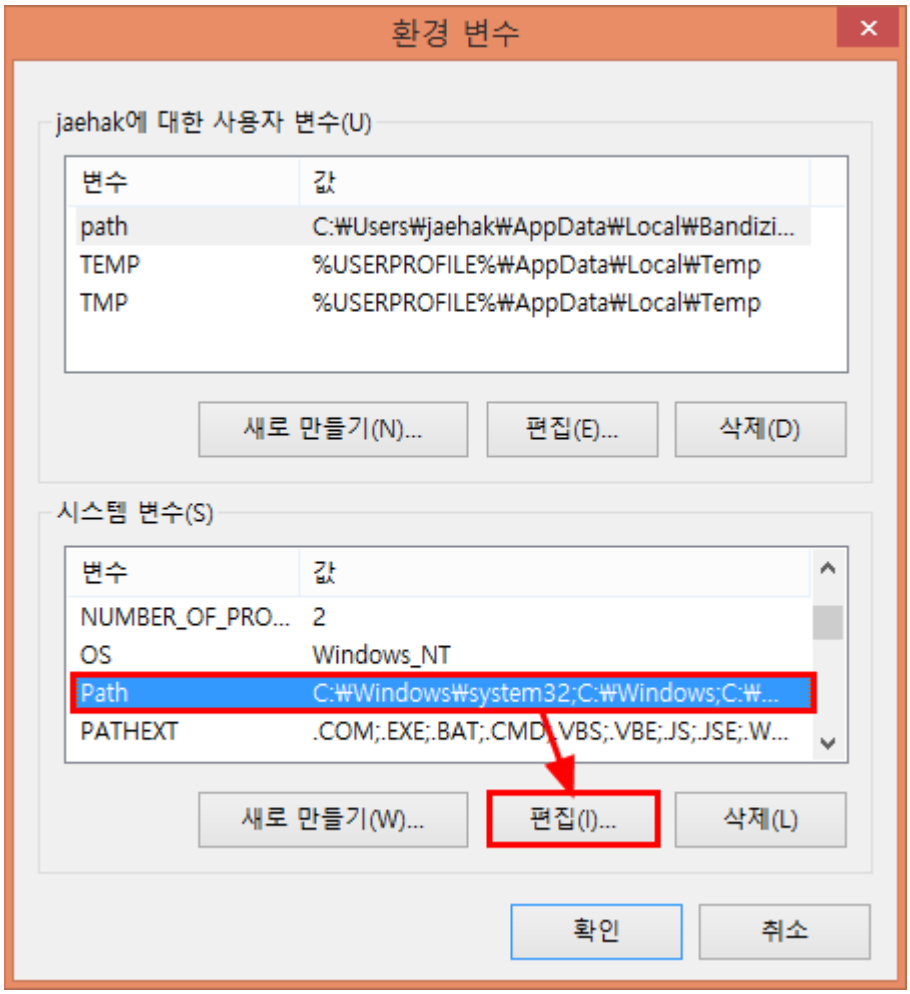
13) 새로 만들기를 클릭합니다.



14) 변수 이름 "JAVA_HOME" / 변수 값 "자바가 설치된 경로"
확인 버튼을 누릅니다.



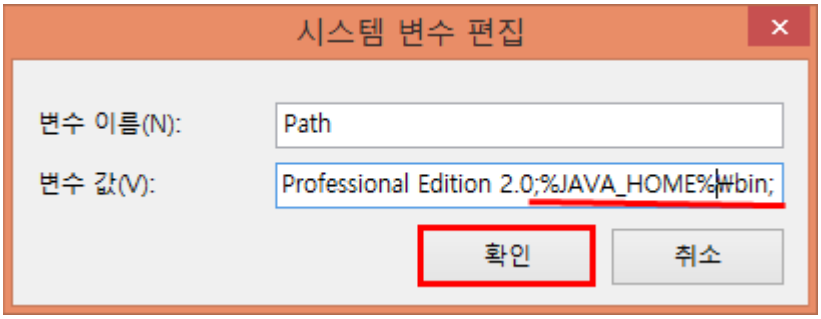
15) 시스템 변수 목록에 Path 를 찾아 선택 후 편집을 누릅니다.



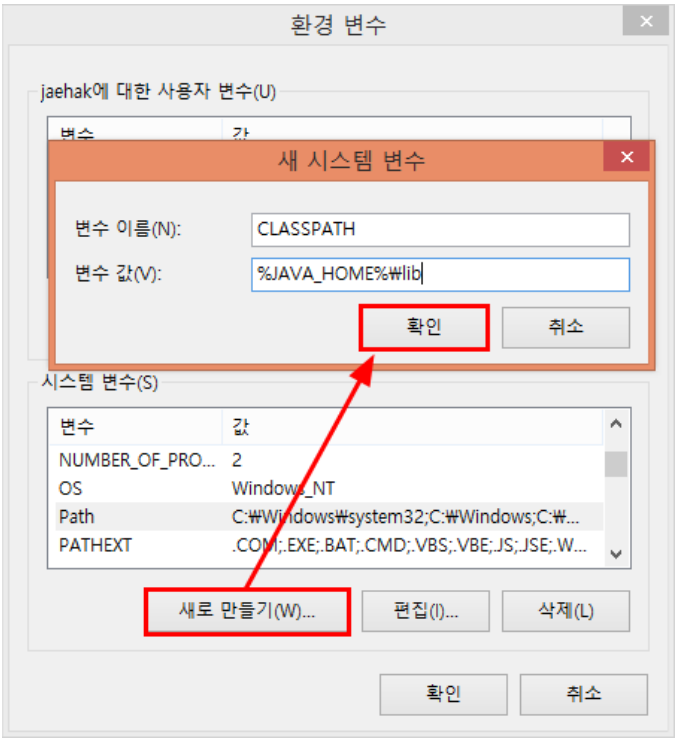
16) 변수 값 ";%JAVA_HOME%₩bin;"

여기서 변수 값은 맨 끝쪽에 씁니다.

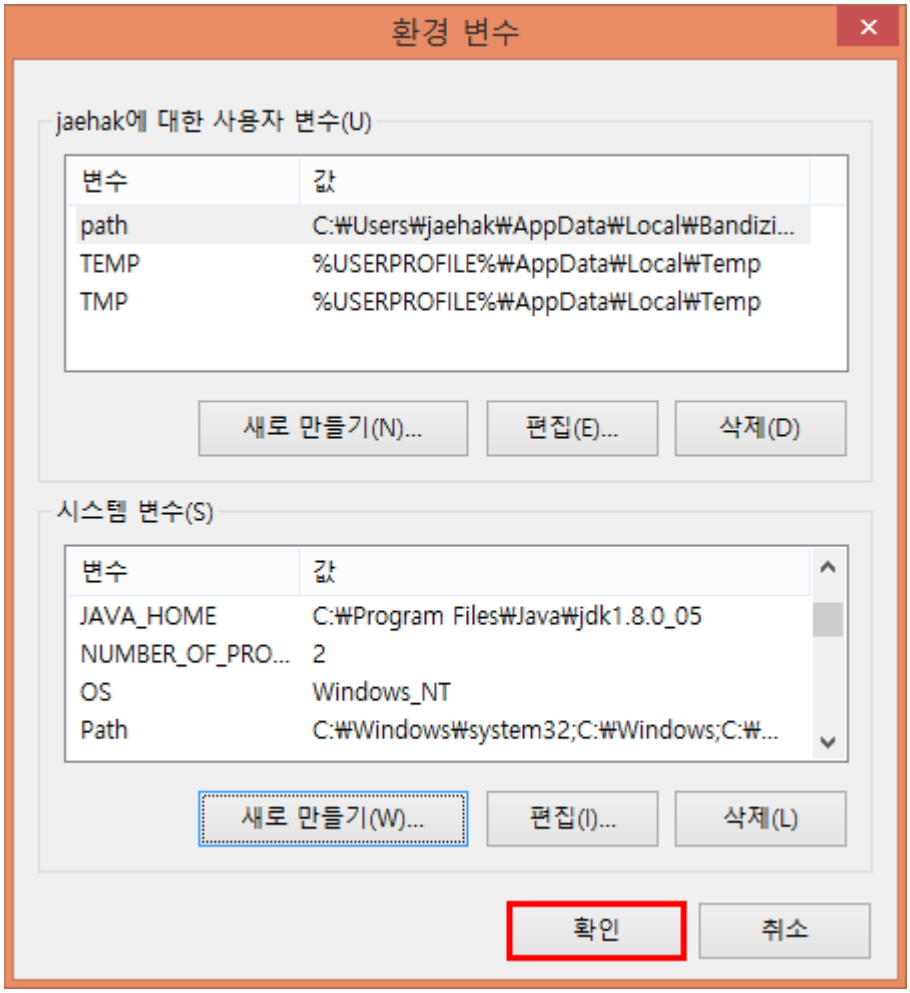
반드시 ; 표시가 양옆에 있어야 합니다. 확인을 누릅니다.



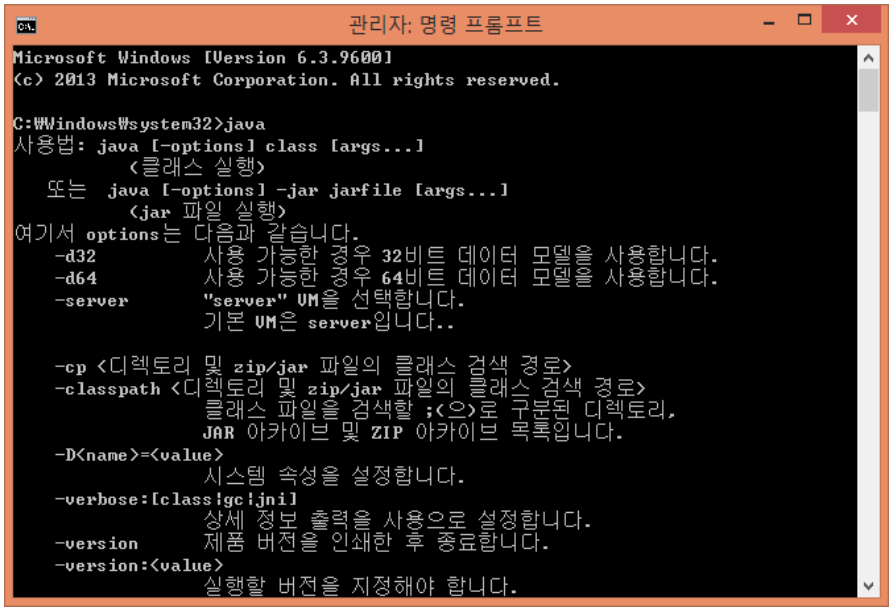
17) 마지막으로 새로 만들기를 눌러 아래와 같이 입력합니다. 변수 이름 "CLASSPATH" / 변수 값 "%JAVA_HOME%₩lib" 확인을 누릅니다.



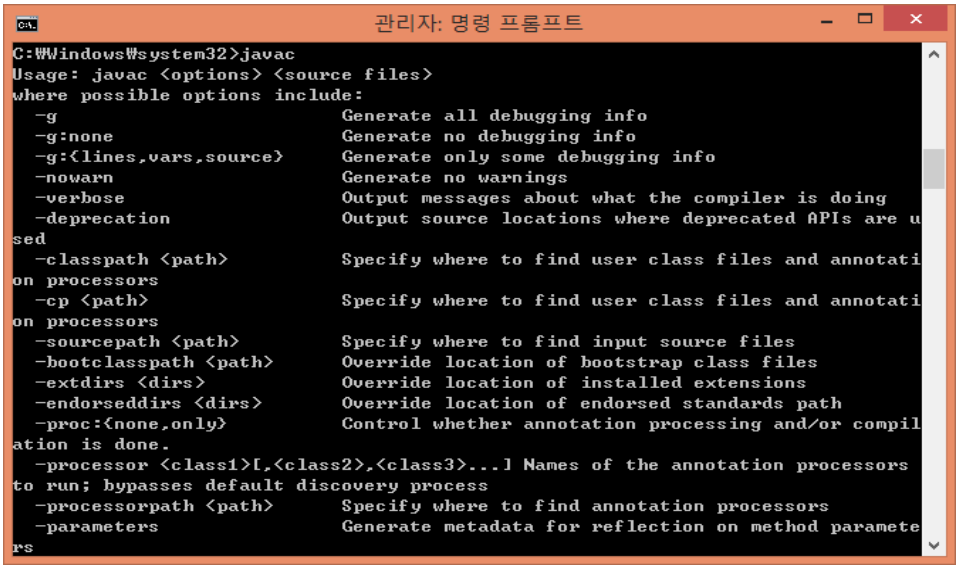
18) 최종적으로 확인을 누릅니다.



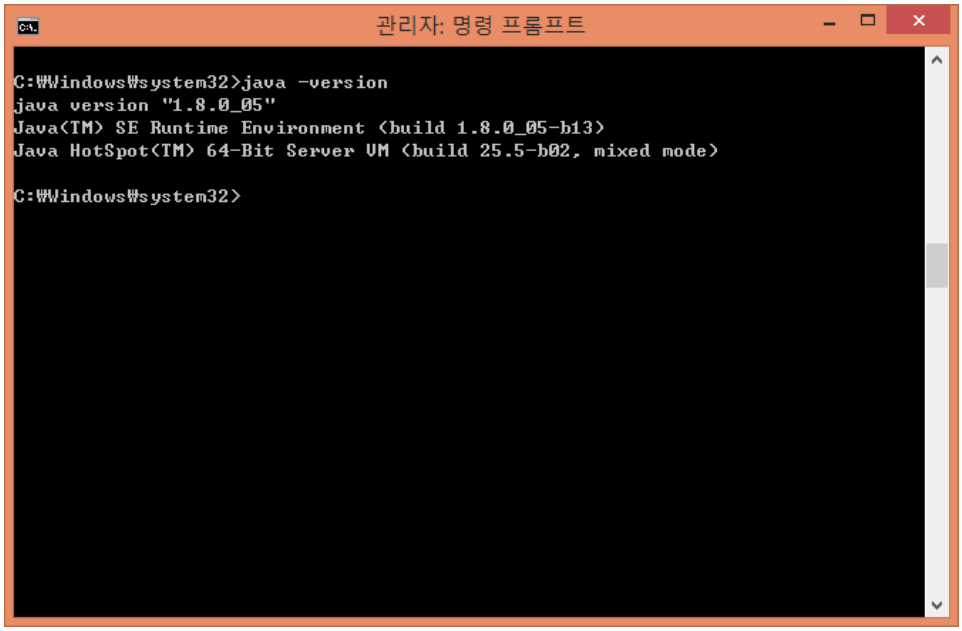
19) 명령프롬프트를 열어 java 를 입력하여 아래와 같이 뜨는지 테스트 합니다. 만약 안되시면 오타가 있는지 빠진 부분이 있는지 확인합니다. (환경변수)



20) javac 컴파일 명령어 입니다.



21) 자바 버전을 나타냅니다.



이로써 자바 설치 혹은 환경 변수 설정이 완료되었다. 블로그 마다 환경 변수 설정이 약간 차이가 날 수 있으니 여러곳을 참고하여 설치하기 바란다. 이제 안드로이드 스튜디오 설치방법을 알아보자.

6.2.2. Android Studio 설치

아래에 설명하는 안드로이드 스튜디오설치 자료는 http://prolite.tistory.com/456 를 참고하였다. Android 앱을 개발 할 때는 eclipse 로 개발하는 것이 당연하듯 공식화 되어 왔다. 안드로이드 개발자 사이트에서 eclipse ADT(Android Development Tools)를 포함해서 배포하고 무료로 사용할 수 있는 안드로이드 개발툴 중에서 단연 eclipse 가 가장 훌륭했기 때문이다. 하지만 이젠 안드로이드 개발자 사이트에서 공식적으로 android Studio 를 정식으로 다운받아서 안드로이드를 개발할 수 있다.

Android Studio 라는 툴은 IntelliJ 기반으로 만들어졌다. IntelliJ 는 자바 개발자라면 대부분 알고 있을 것이다. IntelliJ 는 eclipse 와 같이 통합개발툴인데 안정성과 속도면에서 eclipse 보다 월등히 좋기 때문이다. 우리 연구소에서도

eclipse 를 개발 툴로 사용하다가 IntelliJ 로 변경을 하였다. 원래 IntelliJ 는 상용 툴이다. 그래서 소규모 회사나 개인 개발자들은 IntelliJ 가 좋은 도구임에도 불구하고 라이센스 가격 때문에 무료로 사용할 수 있는 eclipse 를 많이 사용했는데 지금은 IntelliJ 에서 커뮤니티 버전을 무료로 다운 받아서 사용할 수 있다. 오픈소스 개발 또한 무료버전으로 사용할 수 있는데 아마 안드로이드 개발 진영에서 안드로이드 SDK 를 오픈소스로 하기 때문에 IntelliJ 오픈소스 버전을 사용해서 Android Studio 를 만든건 아닌가 생각된다. Android Studio 는 IntelliJ 기반으로 만들어졌기 때문에 IntelliJ 가 가지고 있는 IDE 기능을 대부분 사용할 수 있다.



JAVA JDK 설치 그리고 환경변수 설정...

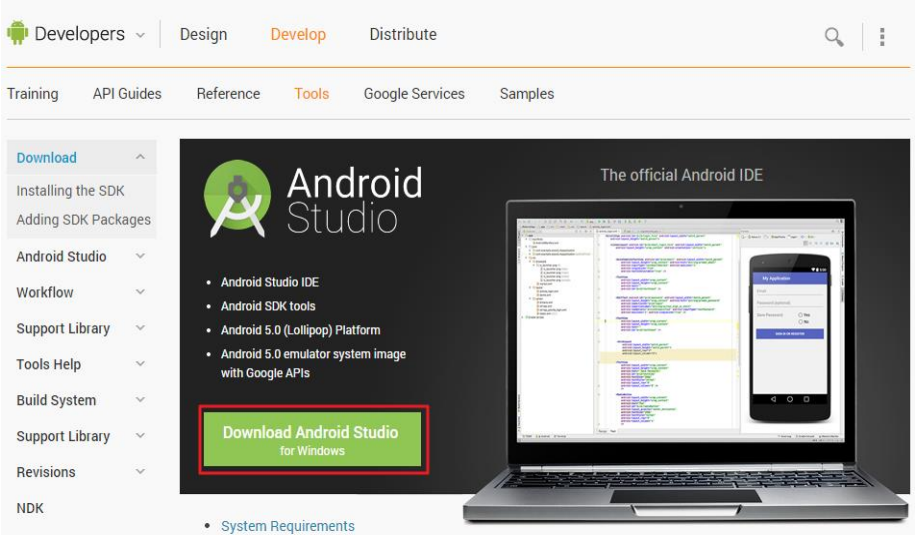
우선 **자바는** 정말 기본중의 기본으로 **설치는 반드시** 하셔야 할 부분으로 새로 다루기에는 뭐 하고**기존 포스팅을 참고**하셔서 **JAVA 설치를 먼저 진행** 하시기 바랍니다.

http://prolite.tistory.com/175
(JAVA 버전이 조금 다를 수 있으니 최신 버전에 맞게 설치하시면 됩니다.)

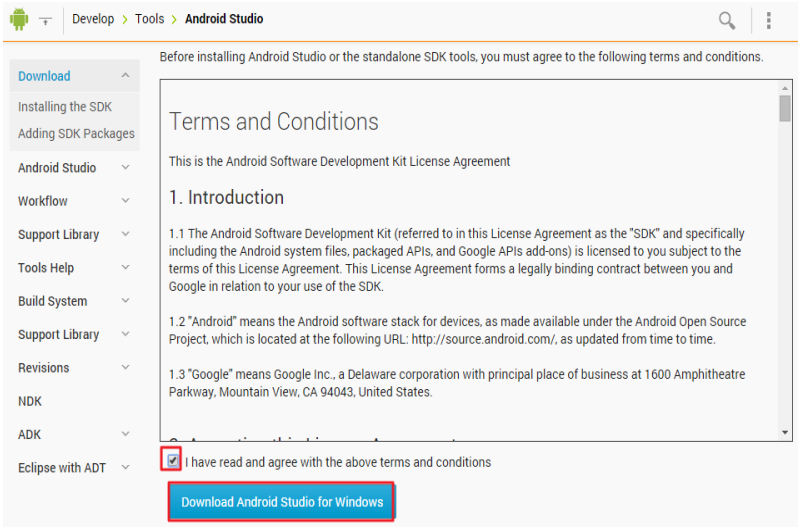
안드로이드 스튜디오 다운로드

1) 아래 Developers 사이트로 가서 Android Studio 의 Download Android Studio for Windows **녹색 버튼을 선택** 합니다.

[안드로이드 스튜디오 사이트 링크]



2) 그럼 영어로 **라이센스 약관**이 나오는데 맨 아래로 가면 **약관 동의를 체크**를 하고 Download Android Studio for Windows **하늘색 버튼**을 눌러 안드로이드 스튜디오를 다운로드 합니다.

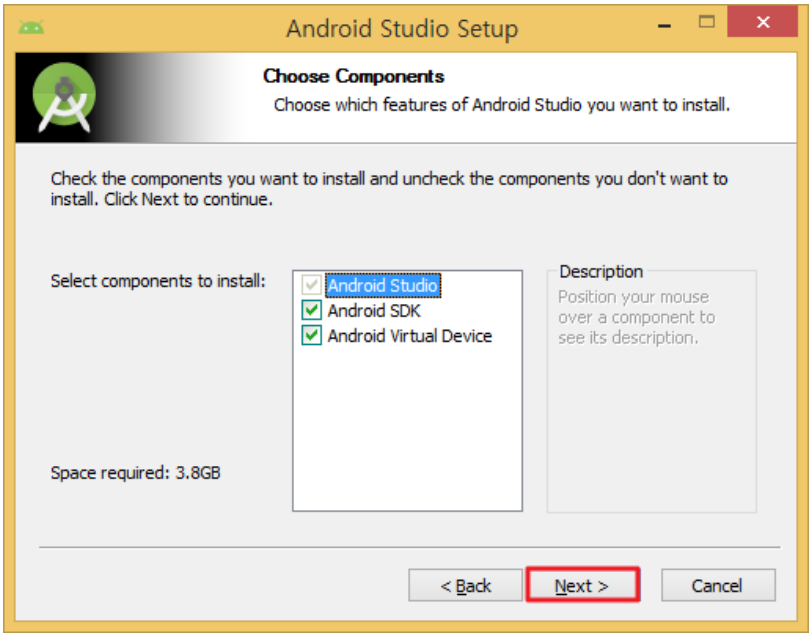


안드로이드 스튜디오 설치 시작

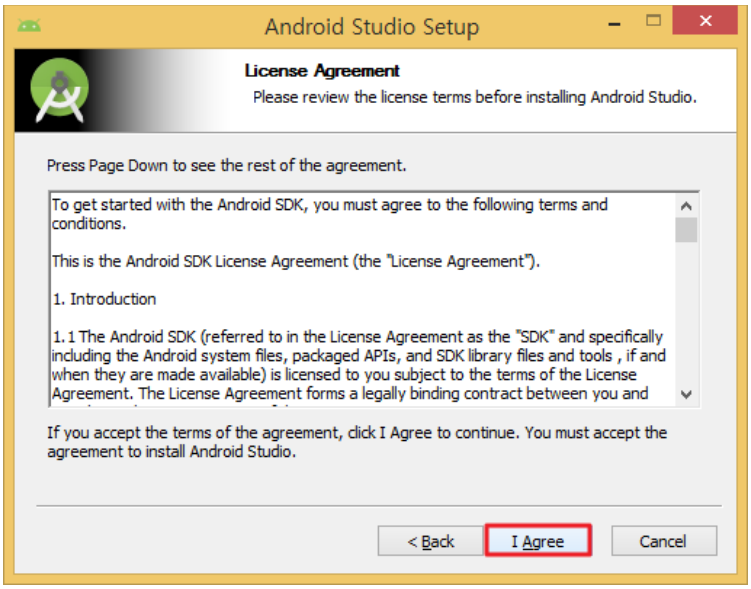
3) 다운로드를 받았으면 설치파일을 실행해 Next > 버튼을 눌러 설치 다음 단계로 넘어갑니다.



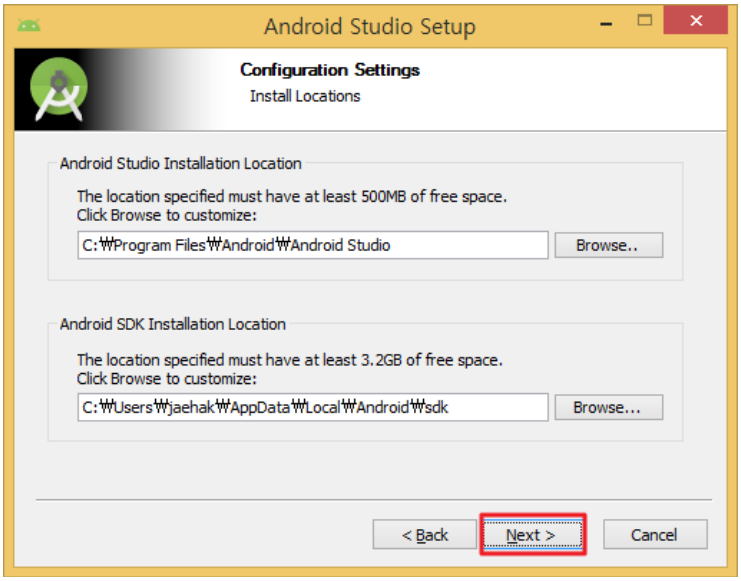
4) 앱 개발을 위해 필수 인 SDK 와 **가상**(Virtual) **드라이브가 기본으로 체크** 된것을 확인할 수 있고 Next > 버튼을 누릅니다.



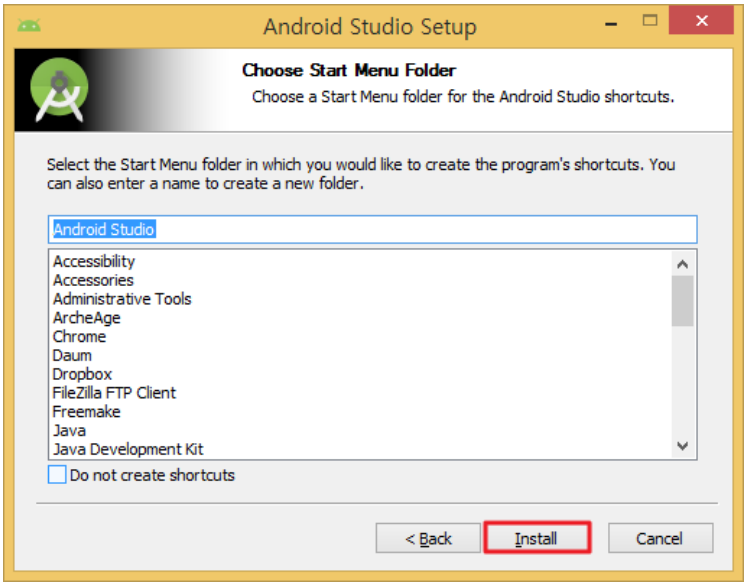
5) 이용 약관에 I Agree 버튼을 누릅니다.



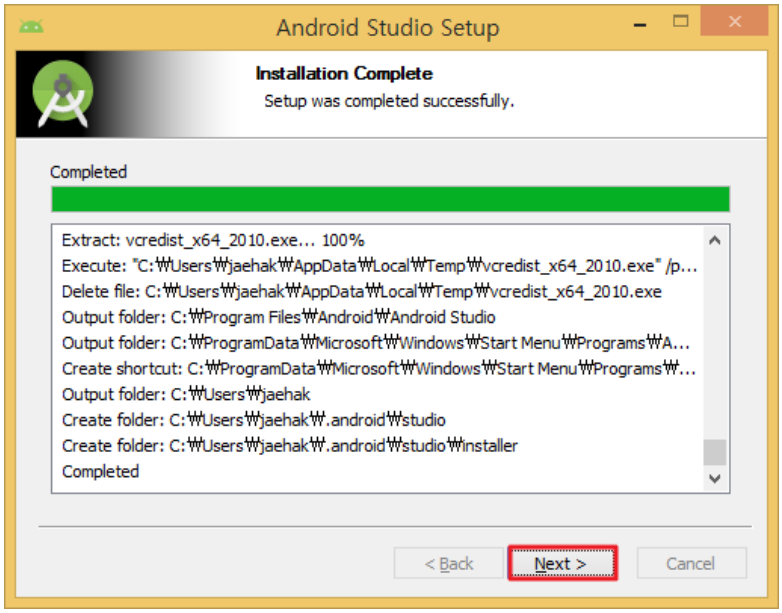
6) 그럼 **안드로이드 스튜디오 와 SDK 설치 경로**가 나오는데 그냥 기본 경로로 두고 Next > 버튼을 누릅니다.



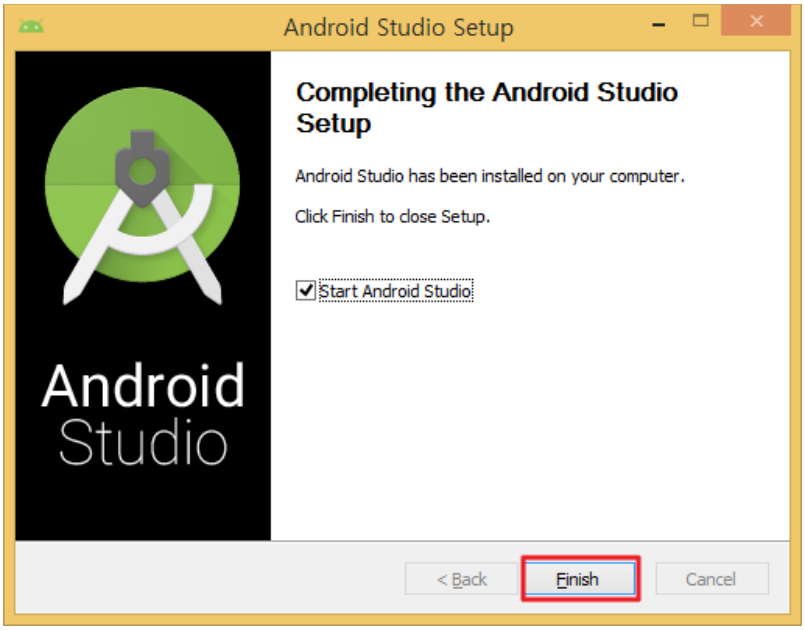
7) 시작메뉴에 추가할 필요없이 Install **버튼을 눌러 설치 시작**합니다.



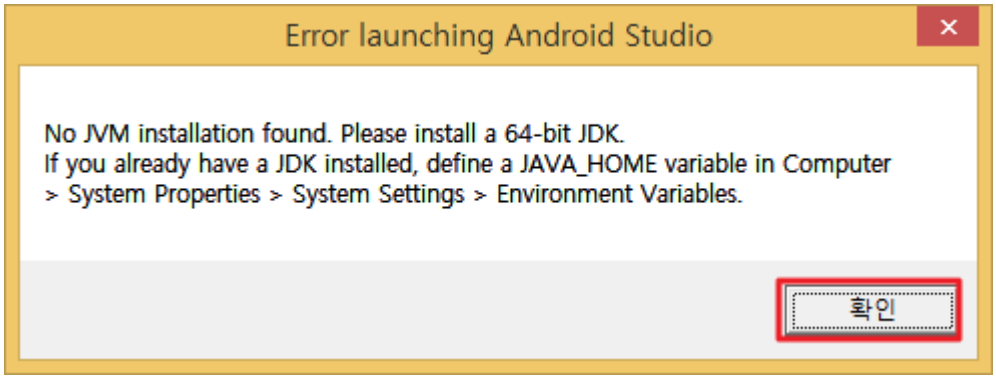
8) 설치과정중에 시간이 좀 걸릴것이며 Next > 버튼을 누릅니다.



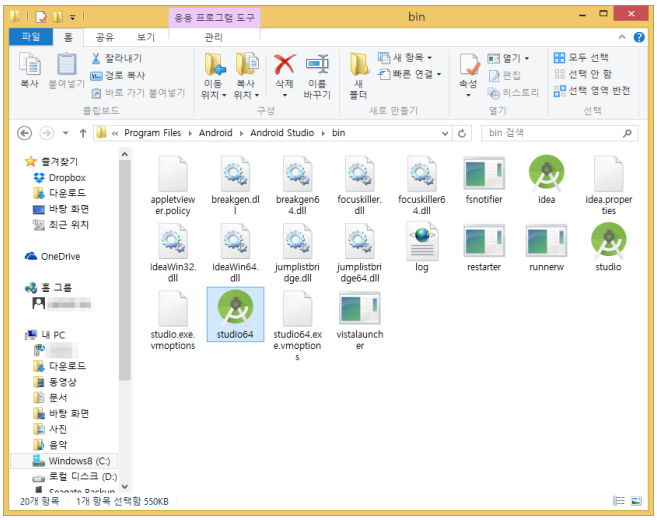
9) **기본 실행 옵션**으로 두고 Finsh 버튼을 눌러 설치를 최종적으로 마칩니다.



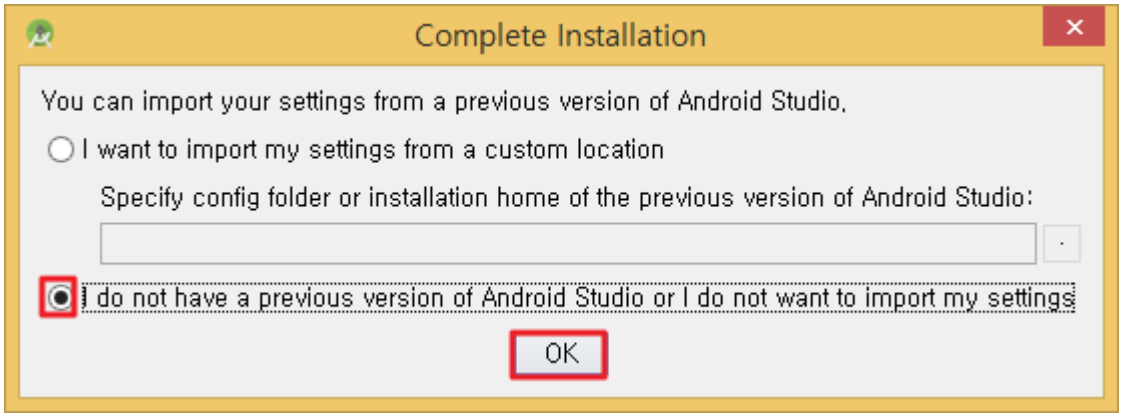
10) 만약에 이런메시지가 뜰 경우 **64 비트 컴퓨터** 에서 **32 비트 파일 실행시 오류가 나는 것으로** 제컴퓨터는 64 비트 이기에 자바를 각 OS 환경 비트 별로 설치하셨다면 상관없습니다.



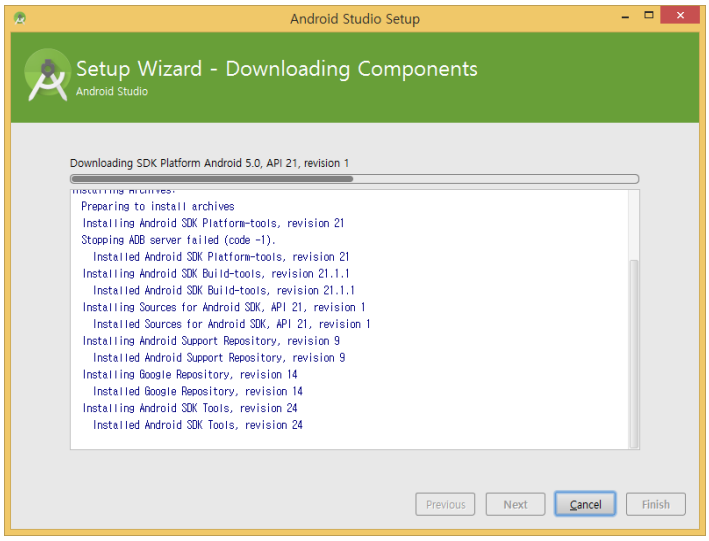
11) 그럼 위와 같은 오류가 안뜨면 상관없지만 안드로이드 스튜디오 프로그램이 종료가 될텐데 바탕화면에 없는 경우 C:₩Program Files₩Android₩Android Studio₩bin₩ 경로로 들어가 Studio64 **파일을 실행**합니다.(32 **비트** : Studio)



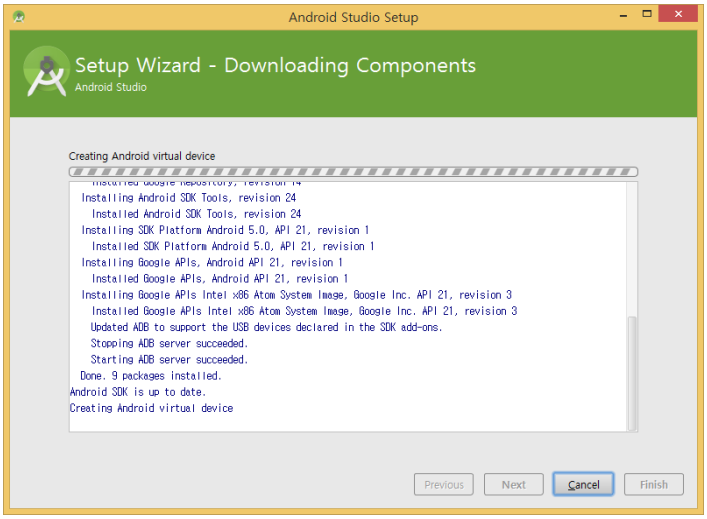
12) **기존 사용자 라면 첫번째 옵션**으로 **기본 환경 설정을 되돌릴** 수 있지만 우리는 **처음이므로 기본 두번째 옵션을 선택후** OK 버튼을 누릅니다.



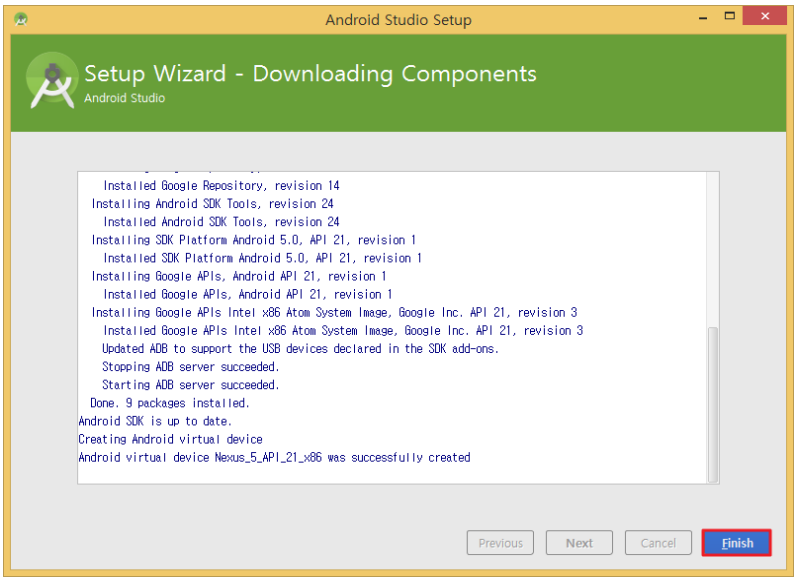
13) 그럼 본격적으로 SDK **설치가 자동적으로 진행**이 되면서 시스템 사양에 따라 약간의 시간이 필요합니다.



14) **가상 머신 설치**가 진행됩니다.

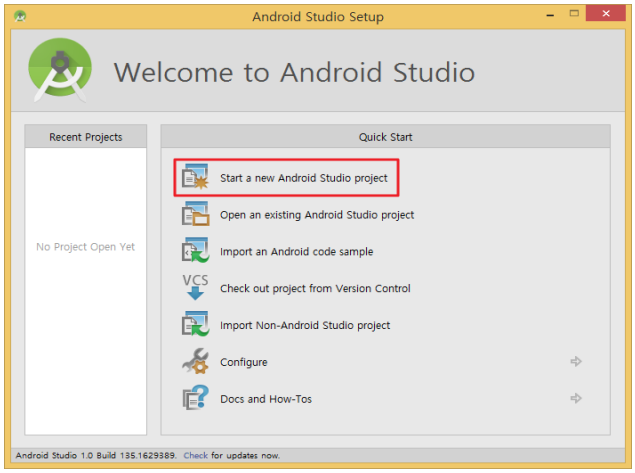


15) SDK 와 가상머신이 설치되었다면 **Finsh** 버튼을 눌러 마칩니다.

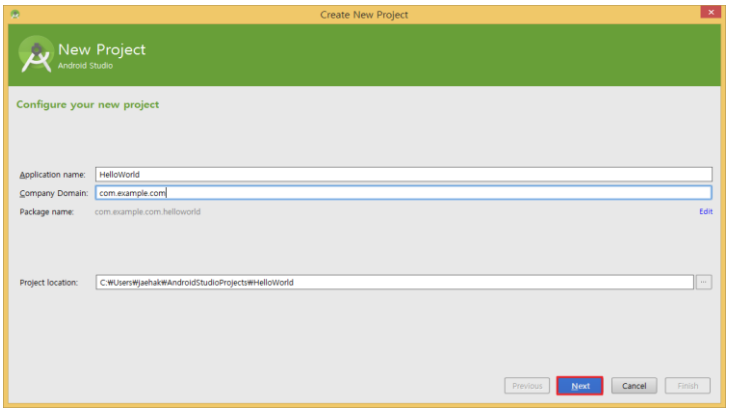


6.2.3 안드로이드 스튜디오 앱 개발 설정과 처음 앱 구현

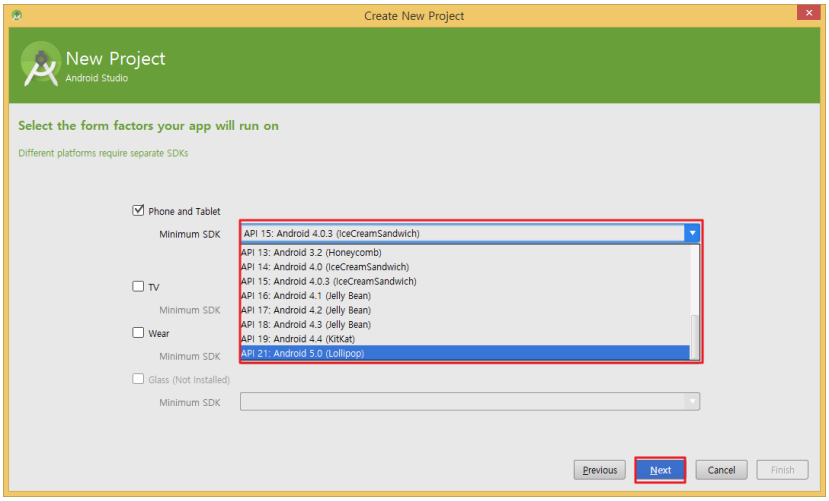
16) 설치가 완료 되었으면 첫번째 메뉴의 **Start a new Android Studio project 를 선택**합니다.



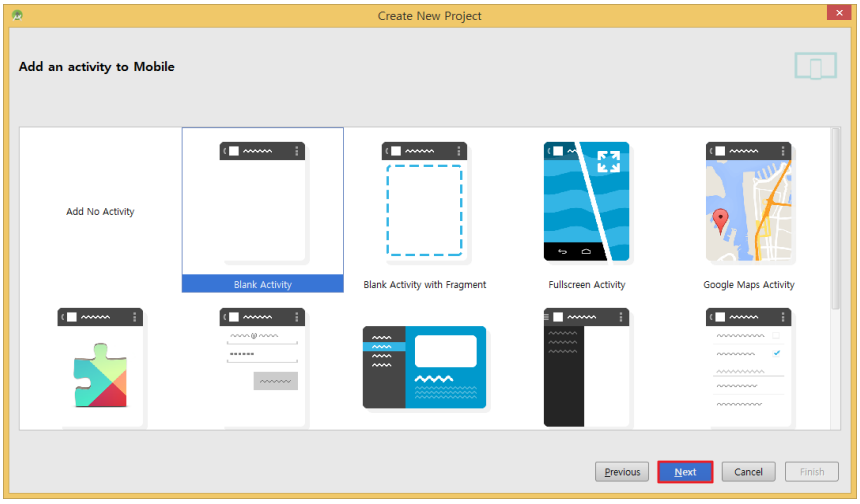
17) 여기서 현재 **어플리케이션 프로젝트 이름**을 정한 후 Next 버튼을 누릅니다.



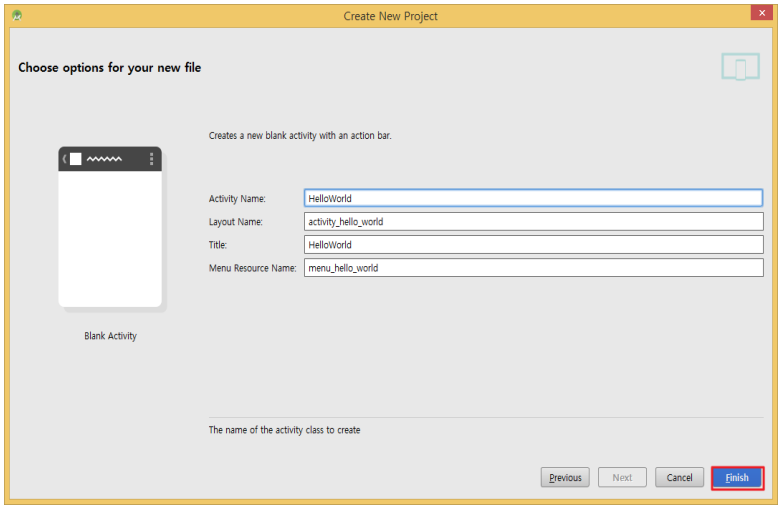
18) 그럼 **버전별로** SDK **목록**이 쫘악 뜨는데 **원하는 버전으로 선택** 후 Next 버튼을 클릭합니다.



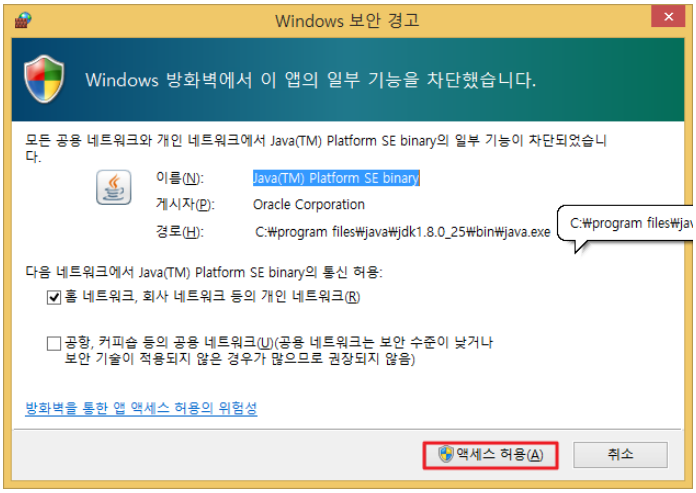
19) 앱 개발시 **기본 테마를 선택 후** Next 버튼을 누릅니다.



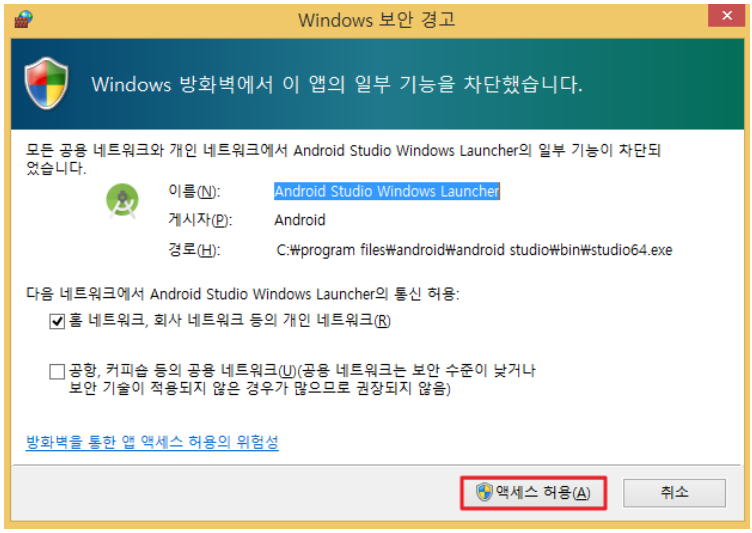
20) 최종적으로 **프로젝트 이름**과 **레이아웃 이름** 혹은 **타이틀** 그리고 **메뉴 리소스 이름**을 설정했으면 Finish 버튼을 누릅니다.



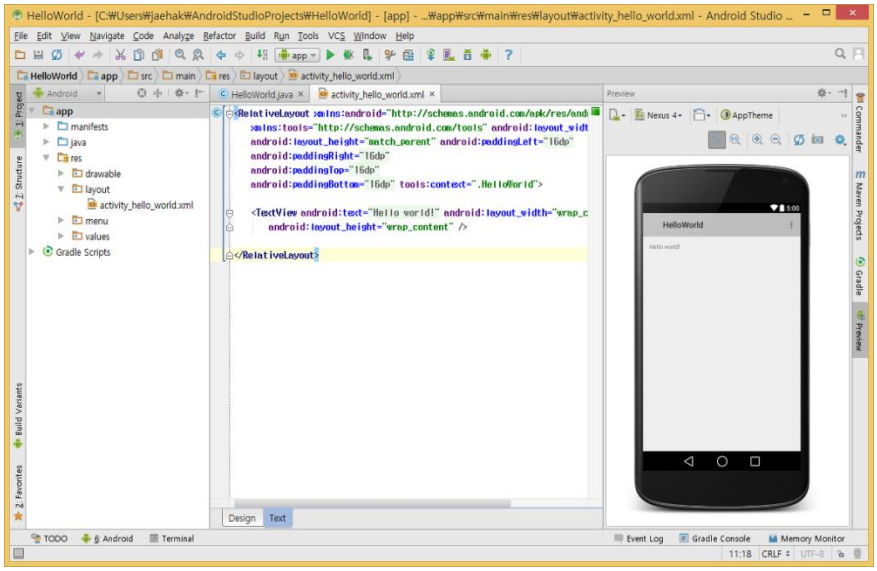
21) Windows 보안 경고 가 뜨면서 Java 가 방화벽을 통해 액세스를 허용할것인지 묻는데 **액세스 허용** 버튼을 누릅니다.



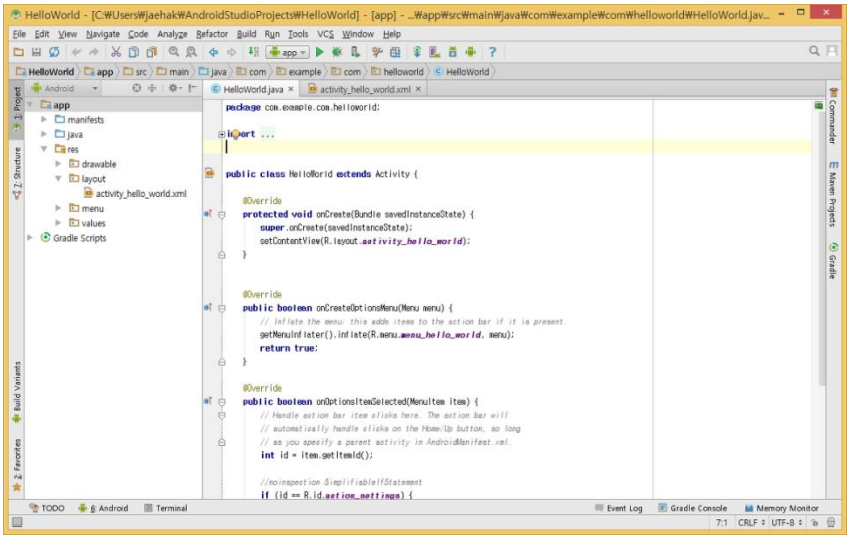
22) 이것도 또 다시 Android Studio Windows Launcher 방화벽 **액세스 허용**을 눌러줍니다.



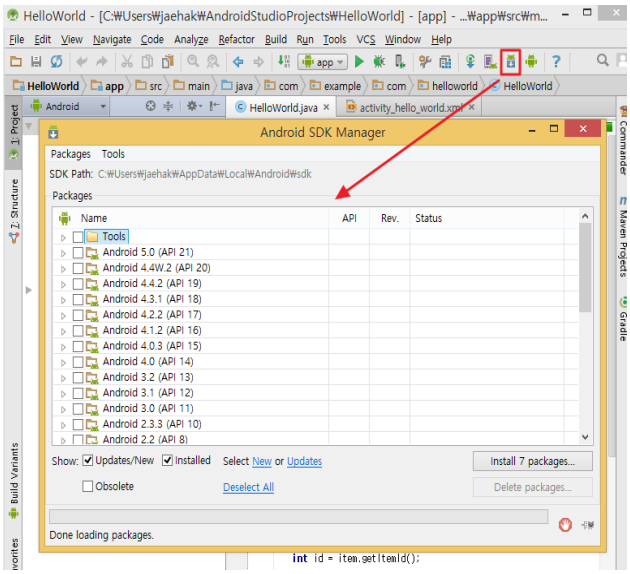
23) 그럼 이와 같이 개발 셋팅이 완료 되었는데 특히 **xml 을 작성하면서 오른쪽에 실시간으로 모바일 화면**을 볼 수 있습니다.



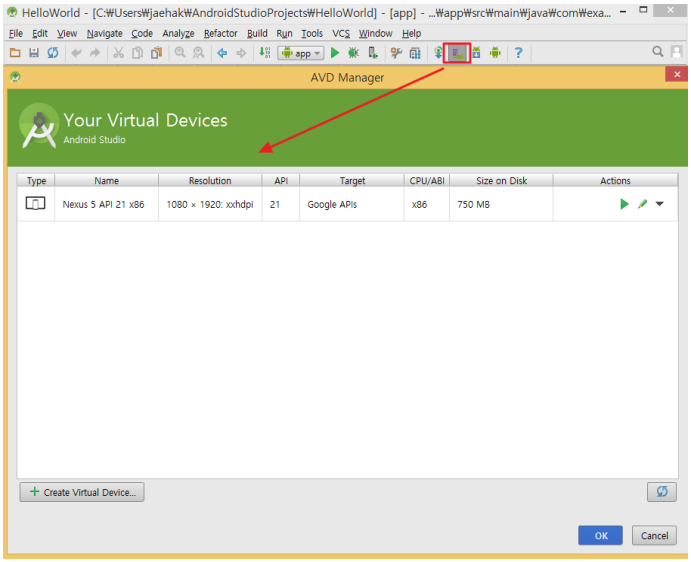
24) 마찬가지로 **이클립스와 매우 흡사한 구조**로 어렵지 않으며 Java **코딩도 이젠 탭을 이용해 개발**을 좀 더 쉽게 효율적으로 할 수 있습니다.



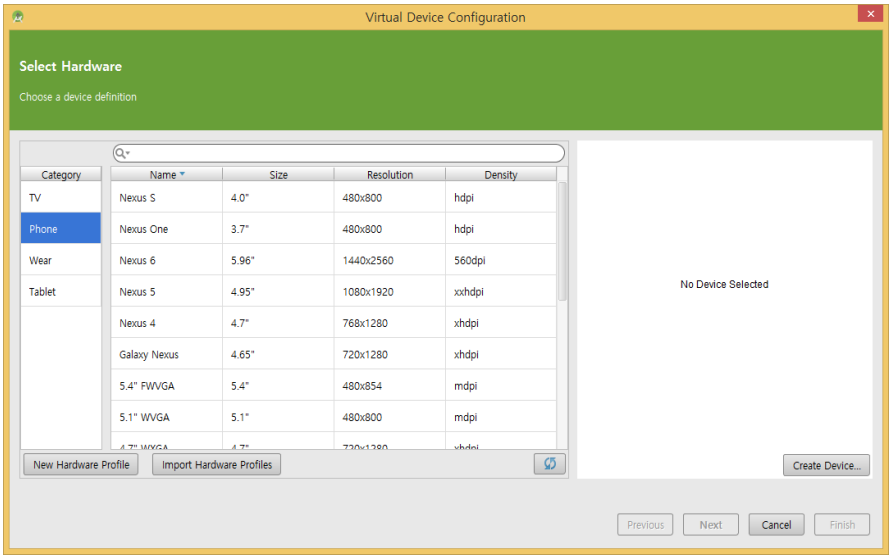
25) 현재는 **아직 SDK 가 완전히 설치가 안됐으므로** 아직 현재 추천하는 버전은 **혹은 킷캣**(4.4.4) 과 **롤리팝**(5.0) 이 되겠으며 아직 **롤리팝은** PC **사양을 너무 많이** 잡아먹기에 우선 **킷캣 부터 권장**하며 이젠 Android **버전 과** API **를 선택해 설치**하면 되겠습니다.



26) AVD Manager 이라고 해서 이미 앱 개발을 해보신 분들은 잘 아시겠지만 **컴파일시 어떤 안드로이드 API 에뮬레이터 환경에서 할것인지 선택**이 가능합니다. **해상도와 그리고 API 버전 용량 인치등 자유롭게 선택이 가능**하며 **너무 최신기종에 맞게 설정하면 컴퓨터가 버벅**거릴 수 있습니다.

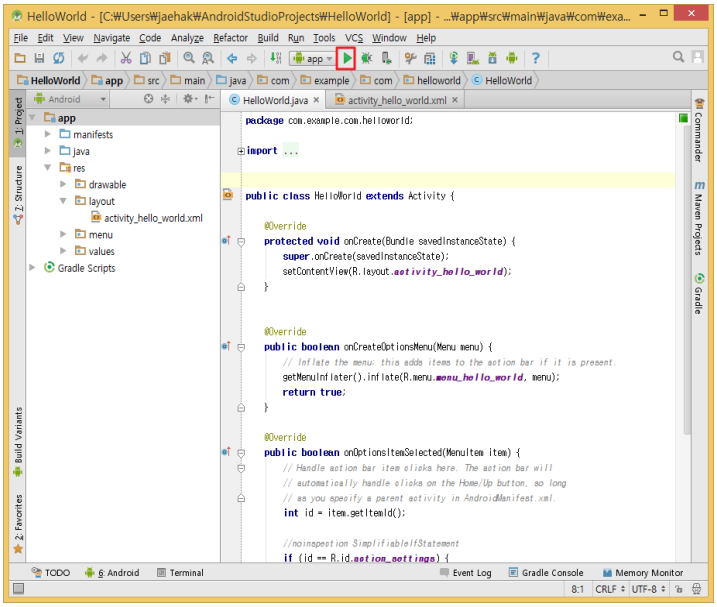


27) **만약 위에서** Create Virtual Device **버튼**을 눌렀을 경우 **또 다른 에뮬 환경에서 구동이 가능**합니다.

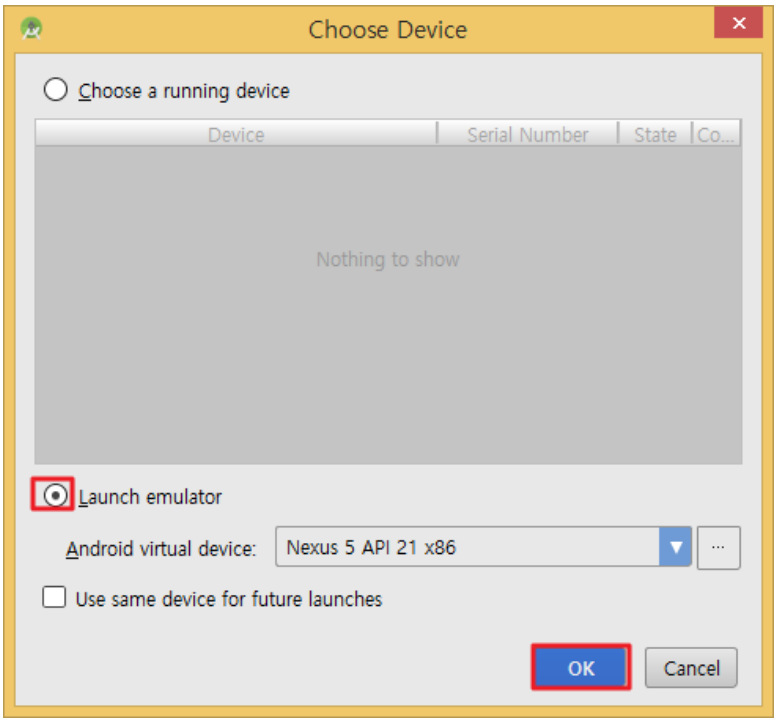


안드로이드 스튜디오 앱 컴파일

28) 상단 메뉴에 보시면 **초록색 재생 버튼**이 있는데 이 버튼을 클릭하면 AVD **에뮬레이터가 바로 구동**이 되면서 **앱 컴파일**이 시작 됩니다.

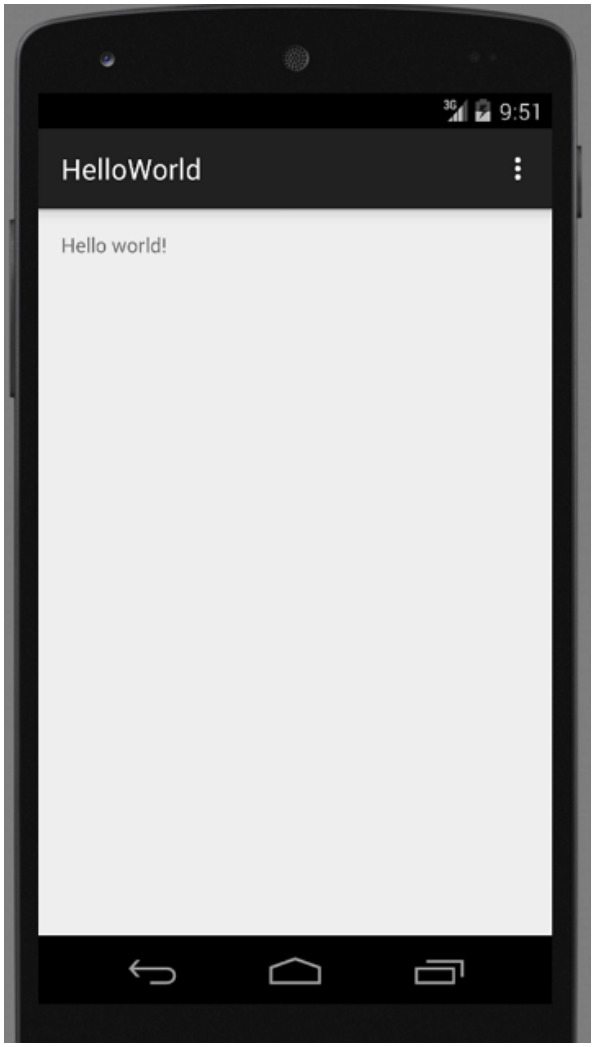


29) **가상 머신이나 실제 연결된 디바이스를 선택**하는 곳으로 OK 버튼을 눌러줍니다.



30) 그럼 이렇게 시스템 사양도 조금 타긴하지만 부팅 속도는 느려도 아래와 같이 **컴파일이 되면서** HelloWorld 를 보여주고 환영해주고 있는 모습을 볼 수 있습니다.

만약 이화면이 안뜬다면 다시 끄지 말고 아까 메뉴 상단의 녹색 재생 버튼만 눌러주면 됩니다.



이상으로 Sndroid Studio 설치와 처음으로 앱을 개발하는 과정을 알아보았다. 부족한 부분은 인터넷 사이트를 찾아 보충하길 바란다.

Chapter 07 Apache, PHP, MySQL 서버구축

이 장에서는 데이터베이스를 이용한 웹 서버의 표준 구성이라고 할 수 있는 APM(Apache + PHP + MySQL)을 설치해본다. 웹을 이용하여 MySQL 을 이용할 수 있는 툴인 phpMyAdmin 도 설치하도록 한다. 여기서 무엇보다 중요한 것은 설치 순서다. Apache, MySQL, PHP, phpMyAdmin 순서대로 설치한다.

한 가지 중요한 것은 서버 설치나 DB 설치중에 입력하는 암호는 반드시 기억해 두어야 한다. 그래야만 서버 프로그램을 실행하거나 데이터 베이스에 접근할 때 제대로 실행할 수 있다. 아래 설치시에는 모든 패스워드를 root 로 하였다.

먼저 apt-get 을 업데이트 해준다. (미러사이트를 최신버전으로 업데이트를 하기 위함이다.)

```
$sudo apt-get update
$sudo apt-get upgrade
```

업데이트와 업그레이드가 완료되었다.

1) Apache 웹 서버 설치

apt-get 을 이용해 Apache 를 간단히 설치할 수 있다.

```
$sudo apt-get install apache2
```

디스크 스페이스를 사용하겠다는 물음에 'Y' 를 선택하고 Enter 키를 누른다. 설치가 계속 진행된다.

```
pi@raspberrypi ~ $ sudo apt-get install apache2
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  apache2-mpm-worker apache2-utils apache2.2-bin apache2.2-common
libapr1 libaprutil1 libaprutil1-dbd-sqlite3
```

```
  libaprutil1-ldap ssl-cert
Suggested packages:
  apache2-doc apache2-suexec apache2-suexec-custom openssl-blacklist
The following NEW packages will be installed:
  apache2 apache2-mpm-worker apache2-utils apache2.2-bin apache2.2-
common libapr1 libaprutil1
  libaprutil1-dbd-sqlite3 libaprutil1-ldap ssl-cert
0 upgraded, 10 newly installed, 0 to remove and 0 not upgraded.
Need to get 1,360 kB of archives.
After this operation, 4,936 kB of additional disk space will be used.
Do you want to continue [Y/n]?
```

pi 유저로 원할한 작업을 위해 “pi” 유저가 웹 서버의 문서 폴더인 “/var/www” 폴더 및 파일 등의 소유 권한을 갖도록 소유권 설정을 한다.

```
$sudo chown -R pi /var/www (- 문자에 주의)
```

라즈베리 파이가 다음 기동할 때부터 apache2 를 자동적으로 기동하도록 설정한다.

```
$sudo update-rc.d apache2 defaults
```

2) MyAQL 설치

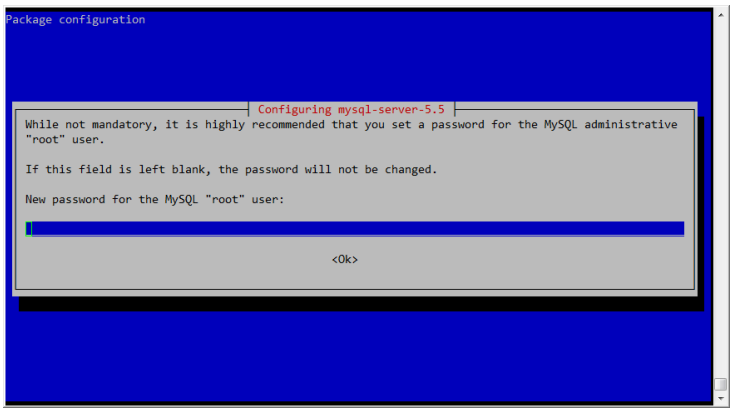
```
$ㅛ
```

설치를 시작하면 99.3M 의 디스크가 필요하다고 하는데 ‘Y’를 누르고 계속 진행하자.

```
$ sudo apt-get install mysql-server mysql-client libmysqlclient-dev
Reading package lists... Done개ㅐㅅ
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
```

```
    heirloom-mailx libaio1 libdbd-mysql-perl libdbi-perl libhtml-template-perl
libmysqlclient16 libmysqlclient18
    mysql-client-5.5 mysql-common mysql-server-5.5 mysql-server-core-5.5
  Suggested packages:
    exim4 mail-transport-agent libipc-sharedcache-perl libterm-readkey-perl
tinyca
  Recommended packages:
    mailx
  The following NEW packages will be installed:
    heirloom-mailx libaio1 libdbd-mysql-perl libdbi-perl libhtml-template-perl
libmysqlclient-dev libmysqlclient16
    libmysqlclient18 mysql-client mysql-client-5.5 mysql-common mysql-server
mysql-server-5.5 mysql-server-core-5.5
  0 upgraded, 14 newly installed, 0 to remove and 0 not upgraded.
  Need to get 11.6 MB of archives.
  After this operation, 99.4 MB of additional disk space will be used.
  Do you want to continue [Y/n]?
```

진행이 되면 Package configuration 화면이 나온다. 패스워드를 입력하는 화면에서 패스워드를 입력하고 잘 기억한다. Root 유저에 대한 암호를 여기서는 root 로 암호를 입력한다.



재차 암호를 묻는 화면에서 암호를 입력하면 설치가 된다. 설치 시간이 약간 오래 걸리므로 기다린다.

외부에서도 MySQL 에 접속할 수 있도록 다음의 명령어를 이용하여 “my.conf” 파일을 열고, 다음의 항목과 같이 코멘트 아웃(앞에 #을 입력한다)하여 비활성화 시킨다.

```
$sudo nano /etc/mysql/my.cnf 문서를 편집기로 연다
약 60 라인 근처의
#
# Instead of skip-networking the default is now to listen only on
# localhost which is more compatible and is not less secure.
bind-address            = 127.0.0.1
#
위 문장에서 bind-address 문장의 앞에 #을 추가한다.
#bind-address - 127.0.0.1
```

Ctrl-x (컨트롤 키 ctrl 키와 x 키를 동시에 누르는 기능)를 눌러 저장하겠냐는 물음에 y 를 누르고, 파일 이름은 동일하니 enter 키를 누르고 빠져나온다.

3)PHP 설치

```
$sudo apt-get install php5
```

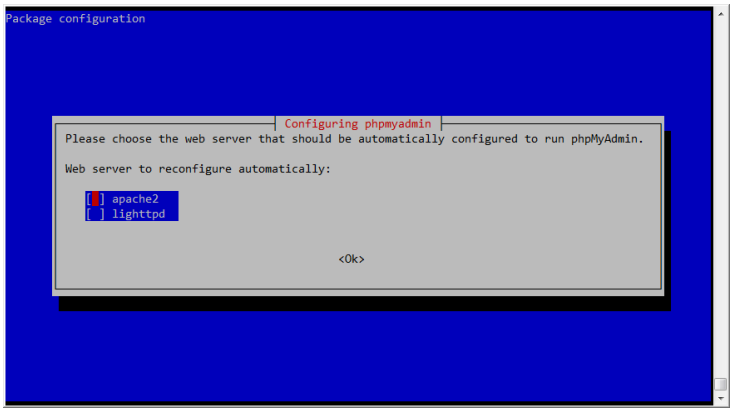
설치시 메모리 공간이 필요하다고 메시지가 나오면 y 를 누르고 설치를 계속한다.

```
$ sudo apt-get install php5
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  apache2-mpm-prefork libapache2-mod-php5 libonig2 libqdbm14 lsof php5-
cli php5-common
Suggested packages:
  php-pear
The following packages will be REMOVED:
  apache2-mpm-worker
```

```
The following NEW packages will be installed:
  apache2-mpm-prefork libapache2-mod-php5 libonig2 libqdbm14 lsof php5
php5-cli php5-common
0 upgraded, 8 newly installed, 1 to remove and 0 not upgraded.
Need to get 6,136 kB of archives.
After this operation, 17.5 MB of additional disk space will be used.
Do you want to continue [Y/n]?
```

4)phpMyAdmin 설치

설치할 때 아래와 같은 화면에서 디폴트로 apache2 를 선택하도록 한다. 선택 후 Enter 를 입력하여 다음 단계로 넘어간다..



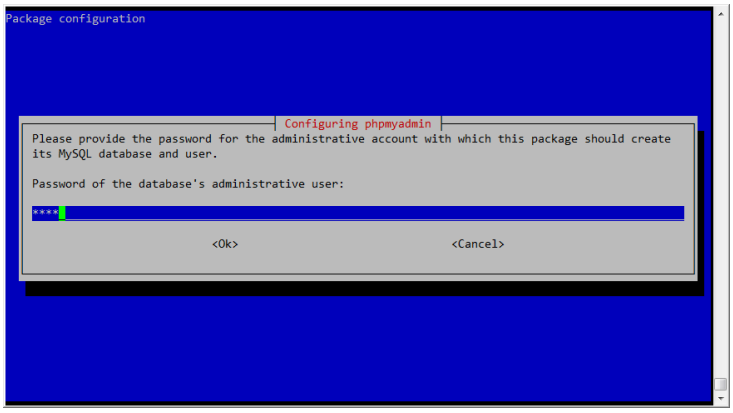

```
$sudo apt-get install phpmyadmin
```

실행을 하게 되면 중간에 메모리 허용을 물어오게 된다. Y 를 누르고 설치를 진행한다.

```
$ sudo apt-get install phpmyadmin
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following extra packages will be installed:
  apache2-mpm-prefork dbconfig-common libapache2-mod-php5 libmcrypt4
libonig2 libqdbm14 lsof php5-cli php5-common
  php5-gd php5-mcrypt php5-mysql
```

```
  Suggested packages:
    php-pear libmcrypt-dev mcrypt
  The following packages will be REMOVED:
    apache2-mpm-worker
  The following NEW packages will be installed:
    apache2-mpm-prefork dbconfig-common libapache2-mod-php5 libmcrypt4
libonig2 libqdbm14 lsof php5-cli php5-common
    php5-gd php5-mcrypt php5-mysql phpmyadmin
  0 upgraded, 13 newly installed, 1 to remove and 0 not upgraded.
  Need to get 11.7 MB/12.3 MB of archives.
  After this operation, 34.9 MB of additional disk space will be used.
  Do you want to continue [Y/n]?
```

설치하는 중간에 패스워드와 설정을 물어오는데 디폴트로 설정하면 되고 비밀번호도 동일하게 입력해주면 된다. 여기서는 root 로 설정한다.



여기까지 수행하면 기본적으로 모두 설치가 된것이다. DB 를 만들고 데이터를 저장하는 것은 뒷장의 실습에서 수행할 것이다. 이 상태에서 웹브라우져에서 라즈베리의 주소를 아래와 같이 입력하 du 화면을 확인하고, 안뜰경우 아래에 해결방법을 설명한다.

참고로 라즈베리파이 APM 설치후 phpmyadmin 안뜰때 해결 방법을 알려주면 다른 교재나 책에 나와 있는대로 잘 설치를 하였는데 http://192.168.1.242/phpmyadmin/ 해도 화면이 안나오는 경우가 있다. 이럴경우에는 아래 명령어로 아파치 설정 파일을 열어서 편집하도록 한다.

```
$sudo nano /etc/apache2/apache2.conf
```

그래서 적당한 곳에 아래 설정을 추가한다. (맨 아래 넣는다.)

```
# Enable PhpMyAdmin
Include /etc/phpmyadmin/apache.conf
```

이렇게 하고 나면 당연히 아파치를 재시작해야 한다.

```
$sudo /etc/init.d/apache2 restart
```

이후 접속이 잘 될것이다.

(참고 주소 : http://sonhc.tistory.com/435 , http://mytory.net/archives/1711)

Chapter 08 데이터베이스 실습

데이터베이스란 여러 사람들이 공유하고 사용할 목적으로 통합 관리되는 정보의 집합이다. 논리적으로 연관된 하나 이상의 자료의 모음으로 그 내용을 고도로 구조화함으로써 검색과 갱신의 효율화를 꾀한 것이다. 즉, 몇 개의 자료 파일을 조직적으로 통합하여 자료 항목의 중복을 없애고 자료를 구조화하여 기억시켜 놓은 자료의 집합체라고 할 수 있다. 공동 자료로서 각 사용자는 같은 데이터라 할지라도 각자의 응용 목적에 따라 다르게 사용할 수 있다.

관계 데이터모델 (relational datamodel)은 데이터 모델 중에서 가장 개념이 간단한 모델이다. IBM 연구소에서 근무하던 코드(E.F.Codd)가 1970 년에 제안했다. 이 모델은 상대 수학적인 이론을 기반으로 한다. 코드가 수학자였기 때문에 수학 분야, 특히 집합론과 논리분야의 개념을 사용했다. 데이터 모델을 개발하기 위해서 테이블 관계로 묘사하는 이론적 모델 과정이 발생하는데 이를 개체관계모델(entity relational model)이라고 한다.

이 관계 데이터베이스를 위한 설계과정은 이론적으로 관계수학에 기초한 실지 구현이라 보면 된다. 현실 세계는 객체관계 그림(diagram)으로 표현되며, 개체와 그 관계는 각기 사각과 선으로 SQL 개체 관계형 데이터베이스를 지원하기 위해 1974 년 IBM 연구소에서 만든 SQL(Structured Query Language) (원어(SEQUEL): Structured English Query Language)가 창안되었으며, 이 언어는 수학적 관계 대수와 관계 논리(relational calculus)에 기반을 두고 있다. 데이터 모델은 데이터를 조작하기 위한 연산집합을 가져야 한다. 왜냐하면 그것은 데이터베이스 구조와 제약 조건을 정의하기 때문이다. 다시 말해, 관계 데이터모델 연산집합(a set of operations)은 관계대수로 표현되고, 그 연산은 사용자에게 여러 질의를 가능하게 한다.

데이터베이스 설계 후 데이터베이스 관리 시스템을 사용해야 한다. 여러가지 데이터베이스 관리 시스템(DBMS)이 존재한다. 데이터베이스 관리 시스템으로 현재에는 많이 사용하지 않으나 IBM 메인프레임 환경 하에서 운영되는 'IMS'가 있다. IMS 는 정보 관리 시스템(Information Management System)의 약자로 DBMS 와 DC 기능을 수행한다. IMS 가 관리하는 DB 종류에는 'DEDB'와 'MSDB'가 있다. DEDB 는 Data Entry DB 로 전통적인 계층형 DB 이며 MSDB 는 주기억 DB 로 시스템 가동시 주 메모리에 상주하는 DB 로 빠른 접근이 필요한

업무에 사용된다. 다만, 접근 속도가 매우 빠른 반면 메모리에 상주하기 때문에 용량 및 구조에 제한이 많다.

데이터베이스는 웹프로그램, 서버 프로그램을 포함하여 스마트 폰 앱 프로그래밍에서도 중요하고 약간은 어려운 부분이지만 반드시 숙지하고 넘어가야 하는 중요한 분야이다. 다른 서적을 통하여 중요한 명령어와 처리방법을 반드시 익힌다면 오랫동안 행복하게 되는 그러한 분야인 것이다. 여기서는 기초적인 명령어와 실제 시스템에서 사용되는 실습을 위주로 진행하고 세부적인 것은 참고도서를 통해 배우길 바란다.

8.1 데이터베이스 생성

일단 데이터베이스에 대해 알아보자. 앞서 APM(Apache, PHP, MySQL)을 설치하였다. 그것을 바탕으로 데이터베이스를 활요해보자. 여기서 사용하는 데이터베이스는 스마트 홈의 대표적인 센서들의 데이터를 주기적으로 데이터베이스에 저장한다. 몇 개월, 몇 년간의 농장자료를 저장하는 것은 큰 의미가 있다. 농장이 있는 지역의 기후와 농장의 계절 특성을 파악하는 데에 많은 도움이 될것이다. 데이터베이스의 기초적인 부분들을 살펴보고 자세한 것은 Mysql 관련도서를 참조하자.

데이터베이스 기동은 '$mysql -u root -p' 이고, 패스워드는 'root' 이다. 설치할때 사용자가 설정한 패스워드를 입력하면 된다.

```
$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or ₩g.
Your MySQL connection id is 48
Server version: 5.5.40-0+ wheezy1 (Debian)

Copyright (c) 2000, 2014, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
```

```
Type 'help;' or '₩h' for help. Type '₩c' to clear the current input statement.

mysql>
```

현재 시스템에 만들어진 데이터베이스 목록을 출력한다.

```
mysqlshow
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| phpmyadmin         |
+--------------------+
4 rows in set (0.00 sec)
```

데이터베이스를 생성한다.

```
mysql> create database iotfarm;
Query OK, 1 row affected (0.00 sec)
```

데이터베이스 보기- 생성된 DB 확인

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| iotfarm            |
| mysql              |
| performance_schema |
| phpmyadmin         |
```

```
+--------------------+
5 rows in set (0.00 sec)
```

사용할 데이터베이스를 지정해준다.

```
mysql> use iotfarm;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
```

테이블 만들기

```
mysql> create table envdata(
    -> time char(19) primary key,
    -> optical int,
    -> temperature int,
    -> humidity int);
```

테이블 보기

```
mysql> show tables;
+--------------------+
| Tables_in_iotfarm |
+--------------------+
| envdata                |
+--------------------+
1 row in set (0.01 sec)
```

테이블 구성 보기

```
mysql> explain envdata;
+-------------+----------+------+-----+---------+-------+
| Field       | Type     | Null | Key | Default | Extra |
+-------------+----------+------+-----+---------+-------+
| time        | char(19) | NO   | PRI | NULL    |       |
| optical     | int(11)  | YES  |     | NULL    |       |
```

```
| temperature | int(11)   | YES  |     | NULL    |       |
| humidity    | int(11)   | YES  |     | NULL    |       |
+-------------+-----------+------+-----+---------+-------+
4 rows in set (0.00 sec)
```

데이터 수동으로 입력

```
mysql> insert into envdata values(now(), 1,2,3);
Query OK, 1 row affected (0.01 sec)
```

데이블 데이터 확인

```
mysql> select * from envdata;
+---------------------+---------+-------------+----------+
| time                | optical | temperature | humidity |
+---------------------+---------+-------------+----------+
| 2014-12-22 20:12:30 |       1 |           2 |        3 |
+---------------------+---------+-------------+----------+
1 row in set (0.00 sec)
```

데이터 삭제

```
mysql> delete from envdata where time < now();
Query OK, 1 row affected (0.02 sec)
```

데이터 보기

```
mysql>select   * from envdata;
Empty set (0.01 sec)
```

데이터 권한 설정

모든 데이터베이스와 테이블을 'root' 라는 id 의 사용자가 "******"의 비밀번호(사용자가 설정한 비밀번호)로 접근할 수 있도록 설정하고, 관련된 설정을 반영하라는 명령어를 입력하여 설정을 갱신하였다.

```
mysql> grant all privileges on *.* to root@localhost identified by 'root';
Query OK, 0 rows affected (0.01 sec)
```

```
mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)

mysql>
```

mysql 데이터베이스를 빠져나올때는 'quit' 나 ctrl-d 로 나온다.

8.2 데이터 베이스 실습

센서 데이터를 데이터베이스에 저장하는 프로그램을 살펴보자. 데이터베이스를 구동하여 데이터베이스를 만들고 테이블을 만든다.

`$ mysql -u root -p`

명령을 사용하여 데이터베이스에 접속한다.

```
pi@raspberrypi ~/smarthome_server $ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or ₩g.
Your MySQL connection id is 47
Server version: 5.5.40-0+ wheezy1 (Debian)

Copyright (c) 2000, 2014, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '₩h' for help. Type '₩c' to clear the current input statement.

이미 iotfarm 이라는 데이터 베이스를 생성하였다
mysql> show databases;
+--------------------+
```

```
| Database           |
+--------------------+
| information_schema |
| iotfarm            |
| mysql              |
| performance_schema |
| phpmyadmin         |
+--------------------+
5 rows in set (0.01 sec)
```

데이터베이스로 iotfarm 을 사용한다.

```
mysql> use iotfarm
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> show tables;
+--------------------+
| Tables_in_iotfarm |
+--------------------+
| envdata            |
+--------------------+
1 row in set (0.00 sec)

mysql>
```

앞에서 env 테이블을 만들고 시간, 빛, 온도, 습도 항목을 만들었다.

```
mysql> explain envdata;
+-------------+----------+------+-----+---------+-------+
| Field       | Type     | Null | Key | Default | Extra |
+-------------+----------+------+-----+---------+-------+
| time        | char(19) | NO   | PRI | NULL    |       |
```

```
| optical     | int(11)  | YES  |     | NULL    |       |
| temperature | int(11)  | YES  |     | NULL    |       |
| humidity    | int(11)  | YES  |     | NULL    |       |
+-------------+----------+------+-----+---------+-------+
4 rows in set (0.01 sec)

mysql>
```

다음의 소스는 데이터베이스에 접속한 후 각각의 센서데이터를 저장하는 소스이다. 지면관계상 메인 루틴 만을 보여준다. 함수 호출 소스는 farmstrory.c 와 같다.

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <errno.h>
#include <sys/types.h>
#include <unistd.h>
#include <wiringPi.h>
#include <mysql/mysql.h>

#define PUMP        21      // BCM_GPIO 5
#define FAN         22              // BCM_GPIO 6
#define DCMOTOR 23          // BCM_GPIO 13
#define RGBLEDPOWER   24 //BCM_GPIO 19

#define RED27               //GPIO 16
#define GREEN       28      //GPIO 20
#define BLUE        29      //GPIO 21

#define LIGHTSEN_OUT 2   //gpio27 - J13 connect
```

```
#define MAXTIMINGS 85

#define DBHOST "192.168.1.238" // 라즈베리 파이 IP 주소
#define DBUSER "root"
#define DBPASS "root"
#define DBNAME "iotfarm"

MYSQL *connector;
MYSQL_RES *result;
MYSQL_ROW row;

void Bpluspinmodeset(void);
void act_waterpump_on();
void act_waterpump_off();
void act_fan_on();
void act_fan_off();
void act_rgbled_on();
void act_rgbled_off();

int ret_humid, ret_temp;

static int DHTPIN = 11;
static int dht22_dat[5] = {0,0,0,0,0};

int ret_humid, ret_temp;

int main (void)
{
    if(wiringPicheck()) printf("Fail");

    Bpluspinmodeset();
```

```
	int curtemp = 0, curhumid = 0;

	//MySQM Connection
	connector = mysql_init(NULL);
	if (!mysql_real_connect(connector, DBHOST, DBUSER, DBPASS, DBNAME,
3306, NULL, 0))
	{
		fprintf(stderr, "%s₩n", mysql_error(connector));
		return 0;
	}

	printf("MySQL(rpidb) opened.₩n");


	while(1)
	{
		char query[1024];

		if(get_light() == 0)
		{
			act_rgbled_off();
			printf("light ! ₩n");
		}
		else
		{
			act_rgbled_on();
			printf("dark ! ₩n");
		}

		curtemp = get_temperature_sensor();
```

```
                printf("%d ₩n", curtemp);

                if(curtemp > 26)digitalWrite (PUMP, 1) ; // On
                {

                        act_waterpump_on();
                        delay(2000);
                        act_waterpump_off();
                }

                curhumid = get_humidity_sensor();
                if(curhumid < 50)
                {

                        act_fan_on();
                        delay(2000);
                        act_fan_off();
                }

                //Write iotfarmDB
                sprintf(query,"insert into envdata values (now(),%d,%d,%d)",
get_light(),curtemp,curhumid);

                if(mysql_query(connector, query))
                {
                        fprintf(stderr, "%s₩n", mysql_error(connector));
                        printf("Write DB error₩n");
                }

                delay(5000);
        }
```

```
    return 0 ;
}
```

컴파일 후 실행을 시키면

```
pi@raspberrypi ~/smartfarm $ gcc -o farmstorydb farmstorydb.c -lwiringPi -lmysqlclient
pi@raspberrypi ~/smartfarm $ sudo ./farmstorydb
Host '192.168.1.238' is not allowed to connect to this MySQL server
```

이런 메시지가 나오면 DB 접속 권한이 없어서 나오는 에러이니 다음과 같이 해준다.

다시한번 DB 에 접속하여

```
pi@raspberrypi ~/smartfarm $ mysql -u root -p
mysql> grant all privileges on *.* to root@192.168.1.238 identified by'root';
Query OK, 0 rows affected (0.00 sec)
```

모든 권한을 부여한다.

```
mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)

mysql> ^CCtrl-C -- exit!
```

실행을 하면 아래와 같은 메시지를 출력하며 5 초마다 database 조도 온도 습도 데이터를 저장할 것이다.

```
pi@raspberrypi ~/smartfarm $ sudo ./farmstorydb
MySQL(rpidb) opened.
light !
Data not good, skip
Humidity = 17.50 % Temperature = 24.50 *C
Humidity = 17 Temperature = 24
Temperature = 24
```

```
24
Humidity = 17.40 % Temperature = 24.40 *C
Humidity = 17 Temperature = 24
Humidity = 17
^Cpi@raspberrypi ~/smartfarm $
```

데이터베이스에 접속해 데이터를 확인해보자.

```
pi@raspberrypi ~/smartfarm $ mysql -u root -p
mysql> use iotfarm
mysql> select * from envdata;
+---------------------+---------+-------------+----------+
| time                | optical | temperature | humidity |
+---------------------+---------+-------------+----------+
| 2014-12-28 18:03:55 |       0 |          24 |       17 |
| 2014-12-28 18:18:45 |       1 |          24 |       17 |
| 2014-12-28 18:18:54 |       0 |          24 |       17 |
| 2014-12-28 18:19:04 |       0 |          24 |       17 |
| 2014-12-28 18:19:13 |       0 |          24 |       17 |
| 2014-12-28 18:19:23 |       0 |          24 |       17 |
| 2014-12-28 18:19:33 |       0 |          24 |       17 |
| 2014-12-28 18:19:42 |       0 |          24 |       17 |
| 2014-12-28 18:19:51 |       0 |          24 |       17 |
| 2014-12-28 18:20:02 |       1 |          24 |       17 |
+---------------------+---------+-------------+----------+
10 rows in set (0.00 sec)

mysql>
```

여기서 조도센서는 기준 조도(쓰레쉬홀드) 이상일 경우(밝은 경우) 0 을 출력하고, 어두울 경우 1 을 출력하게 된다.

Chapter 09. Smart IoT 설계 시스템 소개

IoT(사물 인터넷) 설계/실습시스템은 라즈베리파이 기반으로 구성되었다. 이 장에서는 시스템을 이루는 HW 와 SW 에 대해 전반적으로 설명하고, 센서, 액츄에이터에 대해 알아본다. 또한 IoT 설계 시스템을 이용하는 방법, 5 종 실습과제에 대해서도 설명한다.

9.1 Smart IoT 설계 시스템의 구성

(1) Smart IoT 설계 시스템 개요

Smart IoT 설계 시스템은 Gateway(Raspberry Pi)와 8 종의 센서, 7 종의 액츄에이터, 다양한 주변장비로 구성된다. 5 종의 설계 /실습과제를 통합 시스템에서 직접 설계 및 실습할 수 있는 환경을 제공하고, 사용자가 설계한 시스템을 인터넷이나 안드로이드 앱을 통해 정보를 확인하고, 직접 제어할 수 있는 교육 시스템이다. 제품에 포함된 동영상, CD, 홈페이지와 교재를 보고 따라하는 사이에 IoT 에 대한 개념을 배우고, 창의적인 아이디어를 바로 적용 제작할 수 있는 시스템이다. **(스마트 홈 별도)**

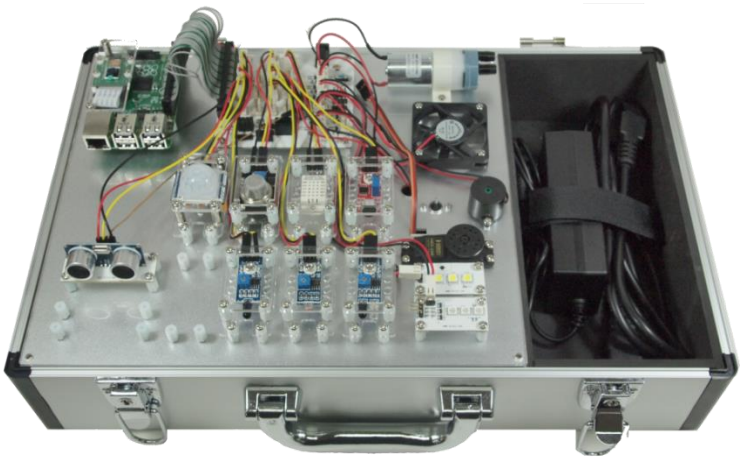


그림 63 SMART IoT 설계 시스템

Smart IoT 설계 시스템은 통합 설계 시스템과 과제별 실습 패키지로 제공된다. 통합 패키지의 경우 5 종의 설계 실습과제를 설계하고 프로그램까지 할 수 있는 과정으로 개인이나 학교의 과정으로 적합하다.

표 13 실습과제 구성

설계/실습 과제	카메라	센서	액츄에이터	Software
IoT 스마트농장 관리 시스템	○	5종	4종	Firmware/Web/Android
IoT 무인경비 시스템	○	4종	5종	Firmware/Web/Android
IoT 화재감시 시스템	○	4종	5종	Firmware/Web/Android
IoT 반려동물 도우미	○	2종	2종	Firmware/Web/Android
IoT 인터넷 제어 Toy	○	3종	3종	Firmware/Web/Android
IoT 스마트 홈(별도)	○	6종	6종	Firmware/Web/Android

과제별 실습 패키지는 5 종의 실습 과제중 한가지를 직접 구성하여 시험하고 응용할 수 있는 제품이다. 실습 패키지는 전자, 컴퓨터를 전공하지 않은 초보자나, 프로그래밍과 하드웨어에 대한 경험이 부족한 초보자도 쉽게 배우고, 응용 할 수 있는 개발환경을 제공한다.

Smart IoT 설계 시스템 전체 구성도

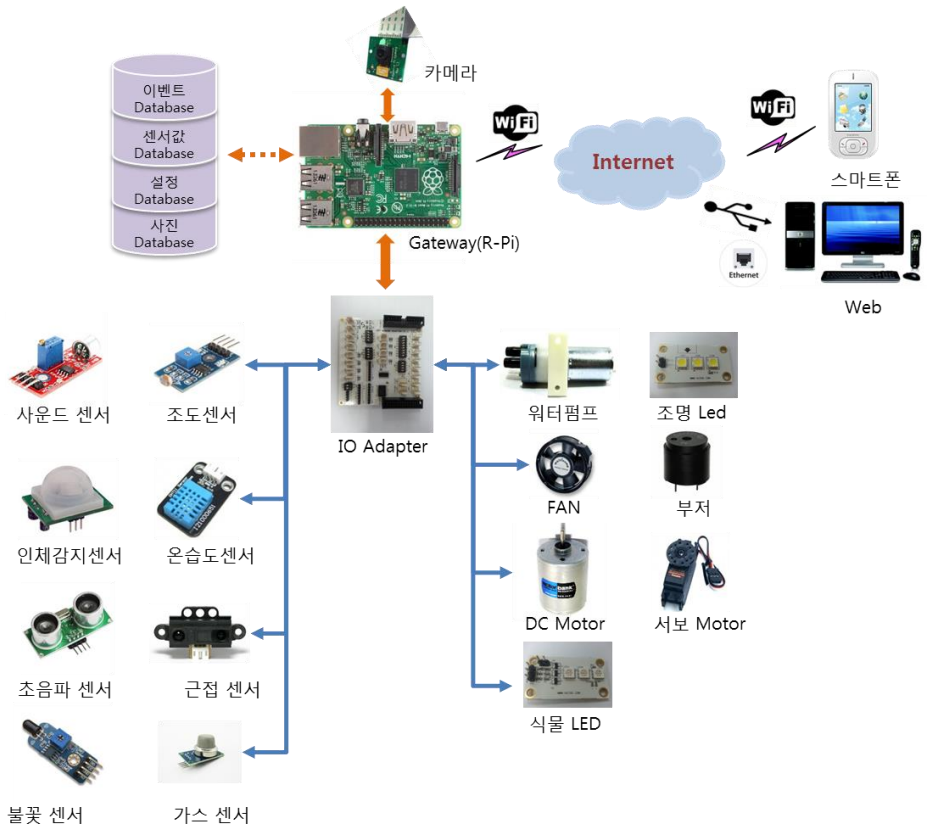


그림 64 스마트 IoT 시스템 전체 구성도

(2) Smart IoT 설계 시스템의 주요특징

Smart IoT 설계 시스템은 IoT 에 대해 처음 배우거나, IoT 설계 원리, 구현방법을 배우려는 학생과 일반인을 위한 실습장비로 다음과 같은 특징을 가지고 있다.

- 농장, 공장/학교/사무실/빌딩, 하우스/아파트 등에서 실시간으로 영상정보와 센서 수집데이터를 스마트폰으로 전송한다.
- 아이디어를 현실화 할 수 있는 쉽고 재미있는 개발 환경을 제공합니다. 프로그래밍과 하드웨어에 대한 경험이 부족한 사용자도 쉽게 배우고 응용할 수 있는 개발환경을 제공한다.

- 각 응용 예제는 Gateway(라즈베리파이) S/W, 센서 S/W, 엑추에이터 S/W, 스마트폰 응용 소프트웨어 등으로 구성되어 있다.
- Gateway 는 다양한 센서(온도, 습도, 조도, 움직임, 소리, 불꽃, 연기, 가스, 토양 온/습도, 카메라 등)와 연동하고 다양한 액츄에이터(Relay, LED, Buzzer, Sound, 경광등, Piezo Siren, Pump, Motor 등)를 제어한다.
- 스마트폰 앱을 통해 영상과 센서 데이터에 따른 다양한 시나리오를 구현할 수 있다.
- 다양한 응용 제품 설계/실습에 꼭 필요한 Firmware, Web program, 스마트폰(Android 용) 소프트웨어 예제를 제공한다.

(3) 요소 기술

Smart IoT 설계 시스템에서 실습하고 배울 수 있는 요소기술들을 설명한다. 리눅스는 전혀 몰라도 된다는 점이 라즈베리파이를 가지고 실습하기에 장점이 된다. 가장 기본적인 명령어만을 사용하면서 실습을 진행할 수 있다. 아래 테이블은 스마트 IoT 시스템의 주요 기술이 되는 것들을 테이블로 나타낸것이다.

표 14 Smart IoT 설계 시스템 주요 기술

Raspberry Pi®	게이트웨이(Raspberry Pi) 라즈베리파이는 영국 비영리조직 '라즈베리파이 재단'에서 학습용으로 만든 초소형 컴퓨터로 전산(Computing)을 가르치는 학교의 학습도구로 개발되었다. 센서 소프트웨어와 액츄에이터 소프트웨어를 실행하며 인터넷 연결기능을 수행한다. Embeded H/W 및 Linux 기반의 Firmware 실습할 수 있다.
	카메라 영상정보를 실시간으로 사용자에게 전송하는 장치로 Gateway 에 연결되어 현장의 상황을 볼 수 있으며, 영상 저장, 정지화면 저장등의 기능을 수행한다. 움직임을 감지하는 센서로서의 기능을

	수행하는 센서로도 활용 가능한 기술을 직접 구현할 수 있다.
	센싱 모든 사물에 부착되어 사용되는 센서 기술은 사물인터넷에서 중요한 분야이다. IoT 사물통신 설계/실습 시스템은 다양한 센서(온도, 습도, 조도, 움직임, 소리, 불꽃, 연기, 가스, 진동)를 제공하여 사용자가 원하는 정보를 수집하여 응용할 수 있도록 한다. 실습 시스템의 센서 소프트웨어가 제공되어 각 센서의 특징에 따라 I2C 통신, ADC Analog, GPIO 등을 통한 데이터 수집/분석기능을 이해할 수 있다.
	액츄에이터 다양한 액츄에이터(Relay, LED, Buzzer, Sound, 경광등, Piezo Siren, Water Pump, Motor 등)는 Gateway 와 유선으로 연동하여 원격지 스마트폰의 제어 신호에 따라 적절한 Action 을 수행하는 것으로 모터제어, LED 제어등 다양한 Actuator 운용 실습을 할 수 있다.
	USN RF Communication Raspberry Pi Gateway 와 Sensor board 간에 무선 통신을 위한 2.4GHz 기반의 IEEE802.15.4 통신 연결을 제공하며, IEEE802.15.4 기반 센서네트워크를 이해하고 제어하는 기능을 실습할 수 있다. (옵션 사항)
	Bluetooth 4.0 BLE Communication Raspberry Pi Gateway 와 Sensor board 간에 무선 통신을 위한 2.4GHz 기반의 Bluetooth 4.0 BLE 통신 연결을 제공하며, 저전력기반 블루투스 통신을 통해 센서데이터의 무선 송수신 기능을 경험할 수 있다. (옵션 사항)

	Web 프로그래밍 여러 브라우저에서 직접 IoT 환경을 구현하는 S/W 기술로 데이터 베이스, 사용자 인터페이스, 서버 구축등을 쉽게 구현할 수 있는 방법을 제공한다.
	Smartphone Application 안드로이드 기반의 스마트폰 앱을 통해 원격지의 상황을 실시간으로 수집 및 제어하는 기능을 수행한다. 사용자는 안드로이드 프로그래밍을 통해 다양한 모바일 기기에 응용할 수 있다.

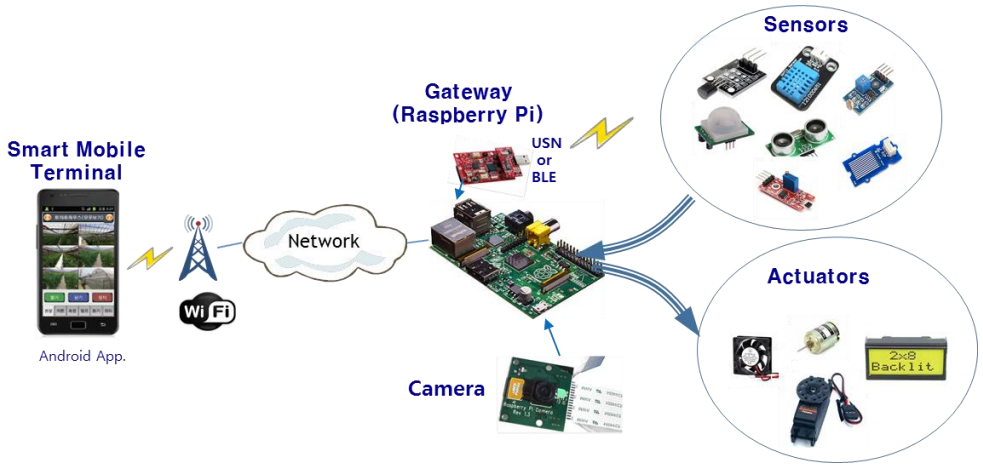
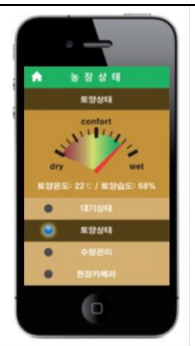
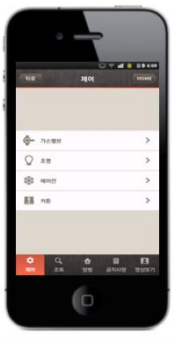


그림 65 스마트 IoT 시스템간 구성도

▪ IoT 안드로이드 앱 UI

		
스마트농장 관리시스템	무인경비 시스템	화재경보 시스템
		
스마트 홈 시스템	반려동물 도우미	인터넷 제어 Toy

(4) Smart IoT 5 종 제품 구성

• IoT 스마트 농장관리 시스템

IoT 스마트 농장관리 시스템은 Raspberry Pi Gateway 기반의 스마트 식물농장 환경을 구현한 IoT 실습용 시스템으로 식물농장 환경의 다양한 센서정보와 영상정보를 실시간으로 수집하여 스마트폰으로 전송한다. 또한 스마트폰으로 현장의 시설물(Actuator)을 원격제어 및 관리할 수 있는 ARM11 기반의 IoT 실습용 Starter Kit 이다.

주요기능

- 스마트폰을 이용한 스마트 식물농장시스템 Actuator 의 원격제어
- 토양의 온도와 습도를 감지하여 솔레노이드 밸브 동작을 통한 물주기

- 온도와 습도를 감지하여 환기팬 동작 및 외부 창문제어(모터)
- 조도 상태를 통한 식물용 LED 작동제어
- 원격영상을 통한 실시간 모니터링 및 Actuator 의 원격제어

구분	센서	액츄에이터	Software	스마트폰 앱
IoT 스마트농장 관리 시스템	카메라 온도센서 습도센서 조도센서	Solenoid valve Fan LED Bar DC Motor	○	○

• IoT 무인경비 시스템

IoT 무인경비 시스템(Smart Guard)은 모니터링이 필요한 학교, 빌딩, 가정, 공장, 상가 등에서 사용할 수 있는 시스템으로 외부 침입자가 발생시 실시간으로 센서 정보 및 영상 정보를 수집하여 스마트폰으로 전송한다. Smart mobile 단말기를 통해 현장의 시설물(Actuator)을 원격제어 및 관리할 수 있는 ARM11 Raspberry 기반의 IoT 실습용 Starter Kit 이다.

주요기능

- 스마트폰을 이용한 스마트 안전경비시스템 Actuator 의 원격제어
- 음향센서와 인체감지센서를 통한 침입자 센싱 및 경광등(RGB LED), 싸이렌 경보 알림
- 스마트폰에 비상 알림 및 카메라 화상 전송 Display
- 원격영상을 통한 실시간 모니터링 및 Actuator 의 원격제어

구분	센서	액츄에이터	Software	스마트폰 앱
IoT 무인 경비시스템	카메라 음향센서 인체감지센서 초음파 센서	LED Bar Piezo Siren 스피커 DC Motor	○	○

• IoT 화재경보 시스템

IoT 화재경보 시스템(Smart Fire)은 화재 모니터링이 필요한 학교/빌딩/가정/공장/상가 등에서 사용할 수 있는 시스템으로 화재 발생시 실시간으로 센서 정보 및 영상정보를 수집하여 스마트폰으로 전송한다. Smart mobile 단말기를 통해 현장을 확인하거나 시설물(Actuator)을 원격제어 및 관리할 수 있는 ARM11 Raspberry 기반의 IoT 실습용 Starter Kit 이다.

주요기능

- 스마트폰을 이용한 스마트 화재경보시스템 Actuator 의 원격제어
- 불꽃센서, CO 가스센서, 온.습도 센서에 의한 화재발생 감지 시 가청(싸이렌) 경보 알림
- 불꽃센서, CO 가스센서, 온.습도 센서에 의한 화재발생 감지 시 물펌프 동작
- 불꽃센서, CO 가스센서, 온.습도 센서에 의한 화재발생 감지 시 방화벽(모터) 동작
- 온도, 불꽃, 가스 정보를 이용한 화재 위험도의 표시기능
- 원격영상을 통한 실시간 모니터링 및 Actuator 의 원격제어

구분	센서	액츄에이터	Software	스마트폰 앱
IoT 화재경보 시스템	카메라 가스센서 온도센서 습도센서 불꽃 감지 센서	LED Array Piezo Siren Stepping Motor Water Pump	○	○

• IoT 애완동물 도우미

IoT 반려동물 도우미는 집에 반려동물을 키우는 사용자에게 유용한 시스템이다. Wi-Fi 연 연결된 도우미는 반려동물을 촬영하여 사용자에게 실시간 영상을 제공하고, 시간에 맞추어 음식을 제공하는 장치이다. 반려동물 도우미 실습 시스템은 반려동물과 대화를 나누고, 반려동물의 영상을 확인하여 주인과 일체감을 느끼게 하는 장치이다. 사용자는 카메라 제어와 영상 전송, 오디오 전송을 실습하게 되고 마이크와 스피커를 제어하는 방법을 실습한다.

주요기능

- 스마트폰을 이용한 반려동물 도우미 원격 제어
- 반려동물에게 음식을 직접 줄 수 있는 모터를 동작시킴
- 마이크와 스피커를 이용해 반려동물과 대화가 가능.
- 반려동물의 상태를 영상을 이용해 직접 모니터링 가능

구분	센서	액츄에이터	Software	스마트폰 앱
IoT 반려동물 도우미	카메라 모션감지 센서	마이크 스피커 Servo 모터	○	○

• IoT 인터넷 제어 Toy

IoT 인터넷 제어 Toy 는 Wi-Fi 에 연결된 장난감 개발 실습 시스템으로 각종 로봇, 자동차, 청소기 로봇을 설계 개발할 수 있는 시스템이다. Gateway 는 Ethernet 을 통해 인터넷과 연결되고 사용자의 스마트폰으로 로봇 제어가 가능한 환경을 제공한다.

주요기능

- 스마트폰을 이용한 장난감 제어
- 자동차에 카메라를 설치하여 이동 영상을 실시간으로 스마트폰에 전송
- 자동차를 원격제어하여 보고 싶은 장소의 영상을 확인
- 모션감지, 음향 센서를 사용하여 가정의 상황을 원격 모니터링 기능
- 인터넷에 접속하여 원격 제어가 가능한 로봇과 장난감의 개발을 설계 실습할 수 있는 시스템

구분	센서	액츄에이터	Software	스마트폰 앱
IoT 인터넷 제어 Toy	카메라 마이크 스피커 모션감지 센서	모터 LCD 스위치 릴레이 모듈	○	○

	음향 센서			

- **IoT 스마트홈(별도 제품임)**

가정내에 있는 가스, 현관도어락, 조명, 난방 벨브 등을 원격으로 모니터링하거나 제어할 수 있는 시스템이다. 가정내의 센서 정보 및 영상정보를 수집하여 스마트폰으로 전송한다. Smart mobile 단말기를 통해 가정의 기기 등을 원격제어 및 관리할 수 있는 ARM11 Raspberry 기반의 IoT 실습용 Starter Kit 이다. 스마트폰 단말기앱을 통해 다양한 시나리오에 따른 실습을 할 수 있는 시스템이다.

주요기능

- Smartphone 을 이용한 스마트홈 시스템 Actuator 의 원격제어
- RFID 출입카드 인증 시 출입문 동작
- 조도센서를 통한 실내 밝기 감지 후 LED 등 ON/FF
- CO 가스 감지 시 가스밸브 차단
- 원격영상을 통한 실시간 모니터링

구분	센서	액츄에이터	Software	스마트폰앱
IoT 스마트홈 시스템	카메라 사운드센서 조도센서 온습도센서 모션감지 센서 불꽃 감지센서 가스센서	도어락 가스밸브 온수밸브 RF ID 조명등 Led 무드등 Led	○	○

9.2 Smart IoT 설계시스템 소프트웨어

Smart IoT 설계 시스템의 소프트웨어는 라즈베리파이 펌웨어와 안드로이드 스마트폰 소프트웨어로 구성되어 있다.

라즈베리파이의 펌웨어는 라즈비안 운영체제에서 실행되는 소프트웨어로 센서로부터 데이터를 획득하고, 액츄에이터를 제어하며, 카메라 영상 자료를 전송하고, 웹 서버와 데이터베이스가 설치되어 동작하는 서버 기능을 수행한다.

안드로이드 스마트폰 소프트웨어는 원격지의 사용자가 모바일 기기를 이용하여 실시간으로 현장의 상황을 카메라를 통해 모니터링할 수 이하고, 센서와 이벤트 데이터를 확인하고, 액츄에이터를 직접 제어하는 프로그램이다.

라즈베리 파이의 소프트웨어 개발 환경은 데비안 GNU/Linux 기반의 Linux raspberrypi 3.18.3 를 사용한다. C 컴파일러로 gcc 버전 4.6.3 을 사용한다. 안드로이드 폰 개발환경은 PC 상에서 수행되는데 안드로이드 4.2 젤리빈 환경에서 모든 소스코드를 작성하고 테스트 하였다.

표 15 Smart IoT 시스템 구성요소

구 분	기능블럭	규 격
구동 Software	OS	RASPBIAN - Debian Wheezy. Release date:2014-09-09 Android Jelly Bean 4.3
	Compiler	GCC Compiler – 리눅스 커널 기반
	센서 프로그램	C, C++, Python, Java
	Actuator 프로그램	C, C++, Python, Java
Gateway CPU Board		ARM/Linux 기반 싱글보드컴퓨터로 센서어댑터, 액츄에이터 어댑터와 연동하고 인터넷에 연결되어 Gateway 역할을 수행. 센서로부터 수집한 정보를 저장하고 원격지의 사용자에게 전송하거나 제어 신호에 따라 해당 액츄에이터를 On/Off 기능
Camera Module		Gateway 에 CSI 인터페이스 및 커넥터로 연결되는 카메라 모듈로 주요기능은 설치 장소의 환경과 액츄에이터가 동작하는 상황을 사진이나 동영상으로 촬영하여 사용자에게 제공
SD Card		Gateway 의 부팅과 Linux 기반의 운영 체제가

	저장된 저장장치이며 Gateway 의 실행 application Software 탑재
센서 S/W	센서에서 다양한 데이터를 수집하고, 분석하는 소프트웨어로
액츄에이터 S/W	센서 입력 정보 및 사용자의 요구에 따라 모터, 팬, 펌프와 같은 동작 장치를 제어하는 소프트웨어
Web Software	Ethernet 을 통해 웹에 접속할 수 있습니다. 웹 서버와 데이터 베이스를 이용하여 Gateway 가 수집한 센서데이터를 웹 브라우저에서 조회하고, 현장 환경을 영상을 통해 확인하고 설치된 시설을 직접 제어
Smart Phone Application	Android 기반으로 하는 Mobile Application Software 를 통하여 사용자는 스마트폰을 이용하여 Gateway 의 센서에서 수집된 데이터 및 영상정보를 수신하여 적절한 액츄에이터를 구동 제어. 사용자는 스마트폰을 통해 이미지나 동영상 Play 기능, Gateway 에 저장된 이벤트와 log 기록 확인 가능

9.3 Smart IoT 하드웨어

Smart IoT 설계 시스템의 H/W 는 리눅스 운영체제 기반의 라즈베리파이와 IO Adpter 보드, 그리고 센서와 액츄에이터로 이루어져 있다. 현재 5 종의 IoT 실습과제를 각 상황별로 구성하여 설계 실습할 수 있다.

(1) 라즈베리파이(model B+)

라즈베리파이(Raspberry Pi)는 영국의 라즈베리파이 재단(Raspberry Pi Foundation)에서 어린이들의 컴퓨터 및 과학 교육을 증진시키기 위해 만든 싱글 보드 컴퓨터(Single-Board Computer)이다. 실습 시스템은 가장 최근 출시된 라즈베리파이 Model B+를 사용하고 운영체제는 라즈비안 (Rasbian) 을 사용한다. 라즈비안은 라즈베리파이와 데비안을 합친 말이다. 데비안은 리눅스의

한 종류이니, 라즈비안은 쉽게 말해 리눅스인 셈이며, 실제로 라즈베리파이 재단에서는 라즈비안을 비롯하여 아치 리눅스의 다운로드를 제공하고 있다.(http://www.raspberrypi.org/downloads/ 참고)

라즈베리파이 자체로 미니 컴퓨터가 되고, 부팅은 SD 카드에 이미지를 저장하는것으로 구동된다. 라즈베리파이에 필요한 주변기기들을 연결한 모습을 나타내었다.

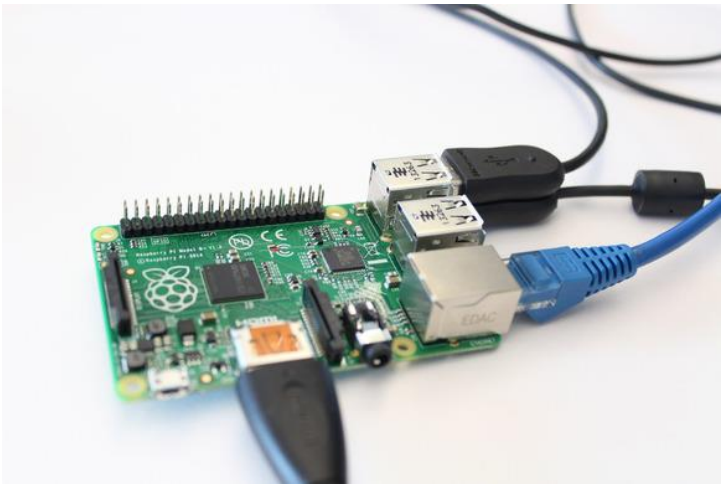


그림 66 라즈베리파이에 HDMI 모니터, 이더넷, 키보드, 마우스를 연결한 모습

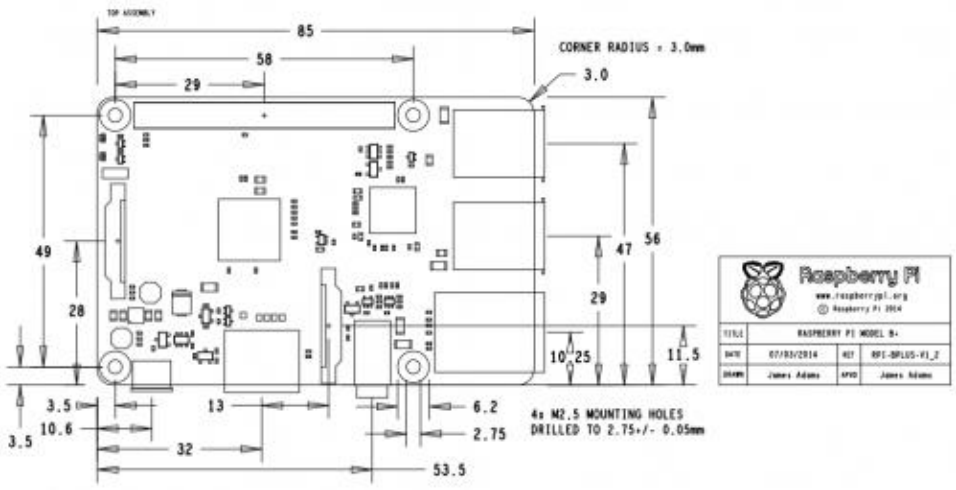


그림 67 라즈베리파이 디멘젼

Smart IoT 설계 시스템을 이루는 하드웨어 구성품을 아래 테이블에 나타내었다.

표 16 Smart IoT 설계 시스템 하드웨어 구성품

구분	제품	규격
Gateway (Raspberry Pi B+)		Raspberry Pi Model B+ Broadcom BCM2835 SoC 700 MHz ARM1176JZF-S CPU
Camera Module		5M Pixel Omnivision 5647 Camera Module Video : Supports 1080p @ 30fps, 720p @ 60fps
Micro SD Card		Gateway 의 부팅과 Linux 기반의 운영체제가 저장된 저장장치이며 Gateway 의 실행 application Software 탑재. Class 10
센서 (Sensor)		Raspberry Standard GPIO Headers(40pin) 연동 온.습도센서, 광센서, 초음파센서, 인체감지센서, 근접센서, 불꽃센서, 가스센서 작동 전압 : DC 3.3V, DC5V

액츄에이터 (Actuator)		Raspberry Pi 와 독립적으로 연동(40pin) Character LCD, Fan, LED Bar, DC Motor, Servo Motor, Water Pump, Piezo Siren, 경광등 LED 작동 전압 : DC 5V, DC12V
전원 어댑터		DC 5V 전원 어댑터 DC 12V 전원 어댑터
소모품		센서 연결 케이블, 저항, LED(Red, Blue, Green, 버튼 스위치, 1 핀 점퍼, 3pin cable, 4pin cable

(2) IO Adapter 보드

센서로부터 센서값을 수집하거나 액츄에이터 제어하는 기능은 게이트 웨이에 연결된 IO Adapter 보드를 통해서 수행한다. IO Adapter 보드는 센서 입력 전원(DC 3.3V, DC 5V)과 출력(ananlog, digital)을 갖는 여러가지 센서들을 연결할 수 있도록 만든 회로 기판이다. 또한 IO Adapter 보드는 구동 전원(DC 5V, DC 12V)이 다른 여러가지 액츄에이터들을 연결하여 구동할 수 있도록 설계하였다.

모든 실습 시스템에 들어가는 IO Adapter 보드는 여기에 소개된 센서나 액츄에이터 뿐만 아니라 사용자가 개발하려고 하는 IoT 제품에 필요한 다른 센서와 액츄에이터를 확장할 수 있도록 도와준다. 설계 시스템 시험은 IoT

Adapter 보드에 필요한 센서를 연결하고, 필요한 Actuator 를 구동하는 실습을 진행한다.

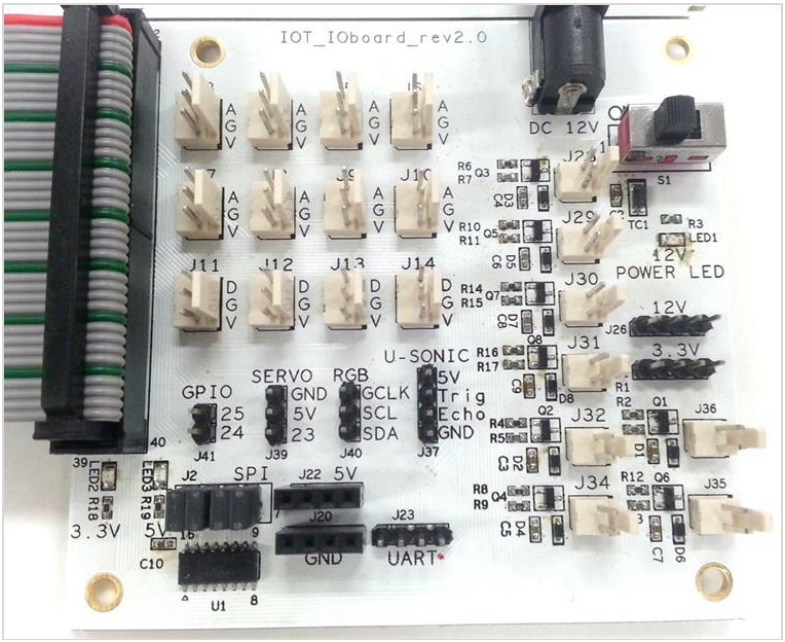


그림 68 IO Adaptor 보드

IO Adapter 보드가 가지고 있는 커넥터들과 사양을 입출력을 구분하여 테이블로 나타내었다. 기본적으로 라즈베리 파이는 전원을 3.3V 사용한다. 만약 5V 를 라즈베리 파이에 공급하거나 GPIO 핀으로 입력이 될 경우 라즈베리 파손의 원인이 된다. 다양한 센서와 핀 맵에 적용하기 위해 설계된 IO 어댑터 보드는 실습에 아주 유용한 도구이다.

표 17 IO Adapter 보드 입출력 포트 구성

IO Adapter 보드의 입출력 사양		
항목	사양	줄력/입력
인터페이스	40pin GPIO(라즈베리 파이)	GPIO
전원 공급	DC 3.3V, DC 5V	GPIO
6 채널 AD 컨버터	MCP3208, SPI 인터페이스	입력
DC 3.3V 디지털 센서	6 포트	입력

커넥터		
DC 12V 전원 출력	6 포트	출력
DC 5V 전원 출력커넥터	2 포트	출력
DC 3.3V UART 커넥터	1 포트	출력
	1 포트	출력
GPIO 출력	9 pin Header	출력

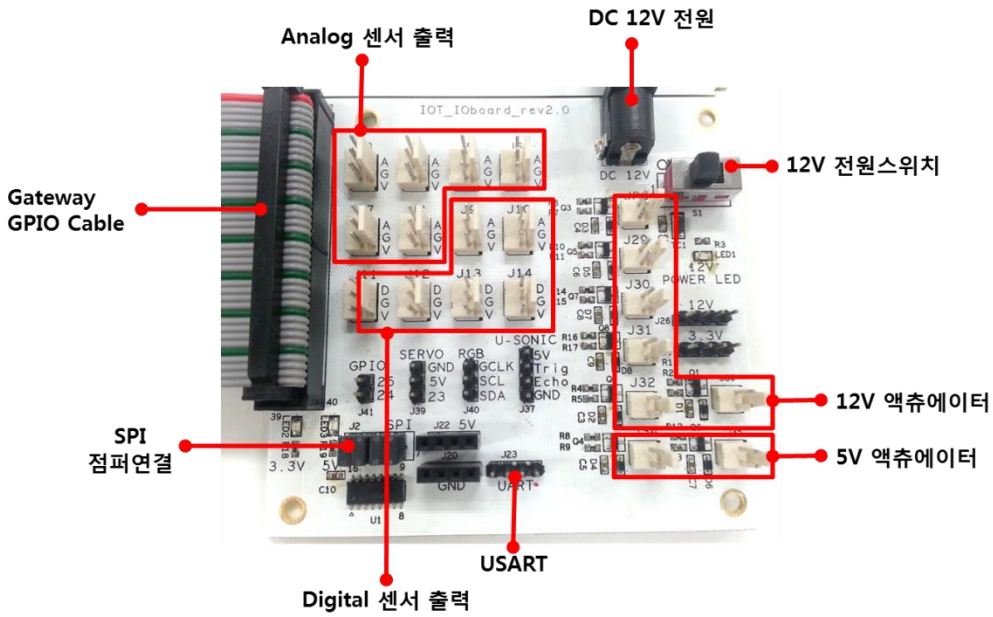


그림 69 IO 어댑터 보드

각 실습과제에 따라 IO Adapter 보드에 센서와 액츄에이터를 연결하고, 라즈베리파이 프로그래밍을 한다. 센서에 필요한 전원과 출력 형태에 따라 적당한 커넥터에 연결을 해야한다. DC 12V 전원 입력은 대부분의 액츄에이터가 12V 전원을 쓰기 때문이다. 그러나, DC 5V 전원이 공급되는 액츄에이터를 위해 별로로 4 채널의 DC 5V 출력 커넥터가 존재한다. 아래 사진은 케이블로 연결한 IO Adapter 그림이다.

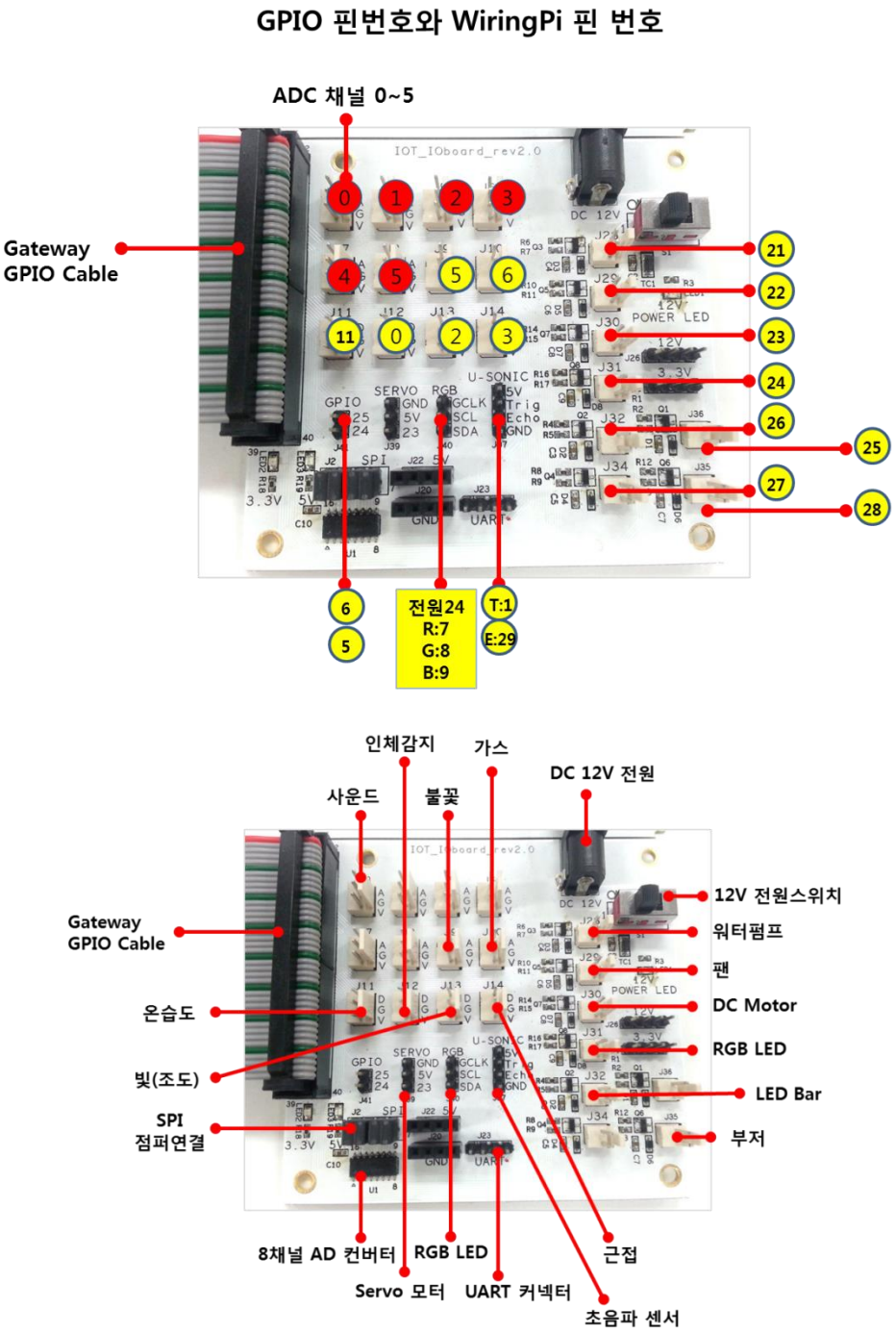


그림 70 센서와 액츄에이터를 연결한 IO Adapter

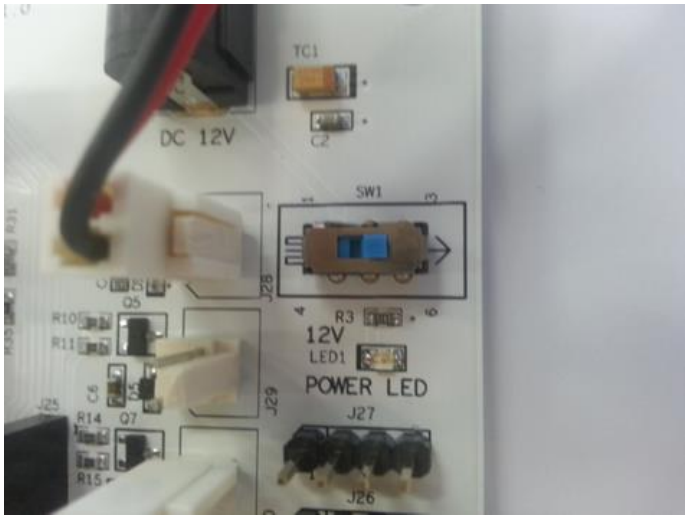


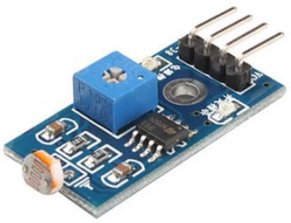
그림 71 DC12V 입력전원 스위치와 LED

(3) 센서(Sensor)

Smart IoT 시스템의 센서는 모두 8 종을 지원한다. 센서들을 모두 같은 전원에 출력도 하나로 통일된 것들을 구매하기는 어렵다. 또한 보다 정확한 데이터를 수집해야 하는 상황에서라면 아날로그 출력의 센서값을 변환해서 사용하는 것이 정확성 측면에서 유리할 것이다. 여기서는 시스템에 사용되는 센서의 종류와 역할에 대해 간단하게 알아보고 자세한 명세서나 시험방법은 실습시 알아보기로 한다.

표 18 다양한 종류의 센서

여러가지 센서의 기능과 외관		
센서	설명	Image
온.습도센서	온도센서와 습도센서를 하나로 합친 센서로 온도 계산식과 습도 계산식이 있기도 함.	

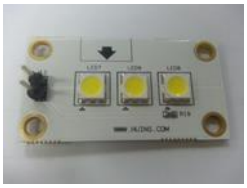
사운드 센서	주변의 소리를 감지하여 아날로그 신호나 디지털 신호로 출력하는 센서.	
광센서 (조도센서)	빛 자체 또는 빛에 포함되는 정보를 전기신호로 변환하여 검지하는 소자. 센서 특징은 비접촉, 비파괴, 고속도, 주변에 잡음의 영향을 주지 않고 할 수 있음.	
인체감지 센서	인간의 몸이나 동물 에서 나오는 적외선은 약 10 ㎛ 의 파장을 갖고 있다. 이 파장을 감지하였을 때 센서가 반응을 할 수 있게 되는 소자	
초음파센서	초음파 센서는 일정한 간격으로 높은 주파수 펄스를 통하여 짧게 소리를 발사합니다. 이러한 것들은 소리의 속도를 통하여 공기 중에 전파되며 개체에 충돌하여 에코 신호로 반사되어 오는 것을 감지하는 센서	
근접 센서	대략 10~80cm 거리를 측정할 수 있는데, 전원을 넣어주면 거리에 따라 출력 전압이 변화하는 센서.	

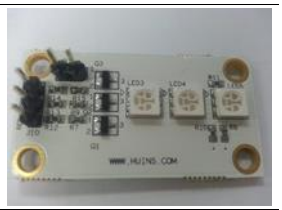
불꽃센서	약 1m 내외의 거리에서 불꽃의 파장(주파수) 760nm ~ 1100nm 대역을 감지하는 센서	
가스센서	내부에 포함된 히터와 센서로 맘모니아, 생활가스, 오염가스등을 검출해낼 수 있는 모듈이다. 내부에 포함된 히터가 적정온도로 올라가서 안정적인 결과를 얻기 위해서는 대기시간(burn-in time)이 필요	

(4) 액츄에이터(Actuator)

Smart IoT 설계 시스템에 사용되는 엑츄에이터를 설명한다.

표 19 액츄에이터 모듈

종 류	규 격	이미지
Fan	Brushless DC FAN 정격 전압: DC 5V Speed : 2.4m/s 30 x 30 x 10mm 팬 속도: 7000-14000 RPM	
LED Bar	LED 3EA DC 12V with 4pcs 1W Xeon emitter Superior ESD protection	
DC Motor	Rated voltage : 12V Rated current : 70mA No load speed : 8700rpm	

Servo Motor	서보종류 : 브러쉬리스 디지털서보 동작전압 : 4.8V,4.8V/6.0V,6V 토크(Kg.cm) : 10Kg↓ 스피드 : 중속 크기(g) : 50g↓ 기어재질 : 프라스틱 특징 : 듀얼볼베어링,브러쉬리스 회전각 : 180도	
Water Pump	정격전압 : 12V 최대압력 : >40PSI 유량 : >500mL/min	
Piezo Siren	102dB Piezo Siren Operates on 6-14 VDC 150mA Current at 12V Buzzer Tone rated at 2,000-4,500Hz	
경광등 LED	RGB 3-chip luminosity Chip LED 5.7 x 5.0 x 1.8 mm (L x W x H) Wide Viewing angle Long operating life (70,000 hours)	

Chapter 10 IoT 스마트 농장 실습

10.1 Iot 스마트 농장관리 시스템 개요

IoT 스마트 농장관리 시스템은 Raspberry Pi(Gateway) 기반의 스마트 식물농장 환경을 실제처럼 구현한 IoT 실습용 시스템으로 라즈베리 파이는 Wi-Fi 를 통해 원격지의 사용자와 연결하는 Gateway 기능을 수행한다. Gateway 는 식물농장 환경의 다양한 센서로부터 수집된 데이터와 카메라 영상정보를 즉시 사용자의 모바일 기기로 전송한다. 또한 스마트폰으로 현장의 시설물(Actuator)을 원격제어 및 관리할 수 있는 ARM11 기반의 IoT 실습 시스템이다.

첨부한 그림은 Smart IoT 설계 시스템의 통합 버전을 나타낸다. 이러한 통합버전에서 센서는 온습도 센서와 조도센서를 사용하고, 액츄에이터로는 워터펌프와 팬, DC Motor, RGB Led 를 사용하여 스마트 농장을 구현한다

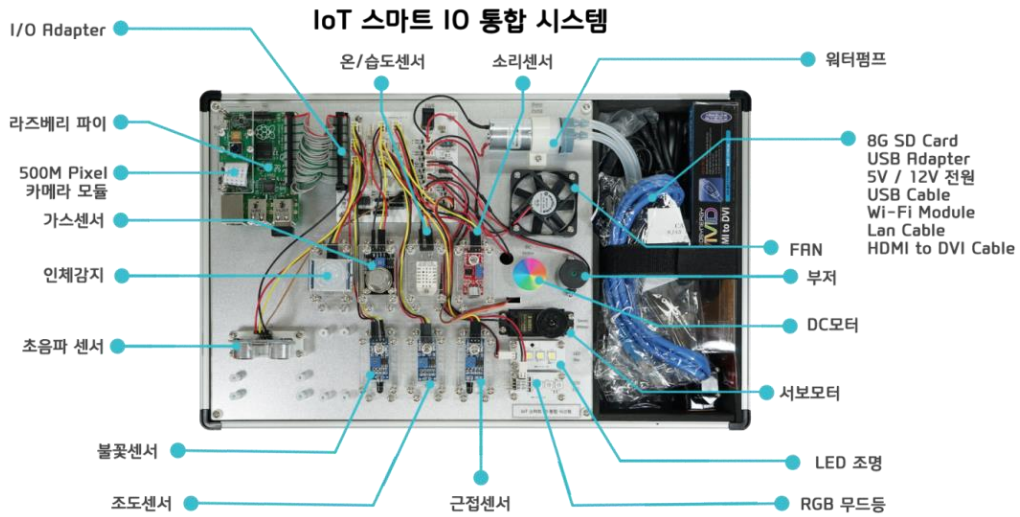


그림 72 Smart IoT 통합시스템

스마트 농장 관리 시스템의 전체 구성을 보면 DB 와 카메라, 이더넷(Wi-Fi)이 지원되는 게이트웨이, 여러가지 센서와 액츄에이터를 제어하는 중간 연결부 인 IO Adapter 보드, 사용자 서버를 나타낸다.

IoT 스마트 농장관리 시스템의 주요기능은 아래와 같다.

- 스마트폰을 이용한 스마트 식물농장시스템 Actuator 의 원격제어
- 대기 온 습도를 센싱하여 DC Motor 와 Fan 제어
- 대기 온 습도를 센싱하여 워터펌프를 통한 물주기 구현
- 온도와 습도를 감지하여 환기팬 동작 및 외부 창문제어(모터)
- 조도 상태를 통한 식물용 LED 작동제어
- 원격영상을 통한 실시간 모니터링

IoT 스마트 농장관리 시스템은 Gateway 시스템과 모바일 프로그램으로 구성된다. 농장관리 Gateway 는 온습도 센서와 조도 센서에서 수집한 데이터를 Database 에 저장하고, 센서 데이터를 판단하여 다양한 액츄에이터(워터펌프, 팬, DC 모터, 식물 LED)를 구동한다. 또한 안드로이드 폰과의 연동을 통해 원격지의 사용자 요청이 발생할 경우 Ethernet 이나 Wi-Fi 를 통해 센서데이터나 영상정보를 송신한다.

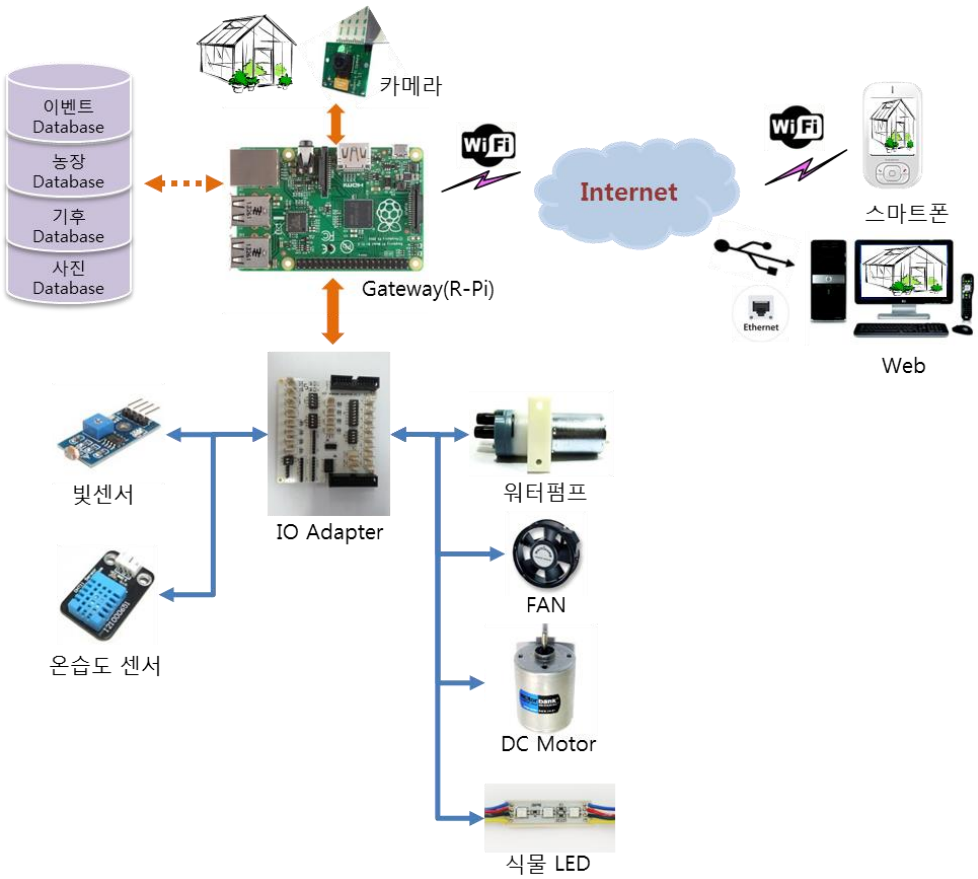


그림 73 스마트 농장 관리시스템 구성도

사용자는 주변의 농장이나 가정의 화분에 적용할 수 있도록 실제 환경과 거의 유사한 작동조건을 구현할 수 있다. Gateways 에는 라즈베리파이 기반으로 리눅스 운영체제를 최적화한 라즈비안이 설치되어 있다. 특히 다양한 액츄에이터의 하나인 워터 펌프는 실제 화분이나 농작물에 물을 줄 수 있도록 실리콘 관으로 연결되어 있다. 시험은 가능한 한 실제 현장에서 발생하는 상황과 유사하게 설계하는 것이 학습에 효과적이라 판단된다.

(1) 스마트 농장관리 시스템 하드웨어

IoT 스마트 농장관리시스템은 Gateway 기능을 수행하는 하는 라즈베리 파이, 카메라, IO Adapter, 센서 2 종(온습도센서, 조도센서), 액츄에이터 4 종(워터펌프, 팬, DC 모터, 식물 LED)으로 구성되어 있다. 센서는 다른 센서 데이터가 필요할 경우 얼마든지 확장될 수 있고, 액츄에이터 또한 사용자의 설계 의도에 맞게 추가하여 구성할 수 있다.

표 20 IoT 스마트 농장관리시스템 H/W

구성품	사양	기능
라즈베리 파이	Raspberry Pi Model B+ Broadcom BCM2835 SoC 700 MHz ARM1176JZF-S CPU	센서 소프트웨어와 액츄에이터 소프트웨어를 실행하며 인터넷 연결기능을 수행합니다. Embeded H/W 및 Linux 기반의 Firmware 실습
IO Adapter	Huins IoT IO Board	Raspberry Standard GPIO Headers(40pin) 연동
카메라	5MP Omnivision 5647 Camera Module Video : Supports 1080p @ 30fps, 720p @ 60fps	영상정보를 실시간으로 사용자에게 전송하는 장치로 Gateway에 연결. 영상 촬영, 정지 화면 저장등의 기능을 수행. 움직임을 감지하는 센서로서의 기능을 수행하는 센서로도 활용 가능.
센서	온습도센서	대기 환경의 온도와 습도 측정
	빛 센서(조도센서)	농장의 빛의 양을 측정
Actuator	워터펌프	식물에 물을 공급한다
	Fan	식물 환경의 공기 순환
	Dc Motor	차양이나, 문을 가동하기 위한 모터

	RGB LED	적색과 청색 LED(식물 LED)
전원 어댑터	DC 12V, 3A 전원 어댑터 DC5V, 1A 전원 어댑터	스테이션 전원공급 +5V 액츄에이터 전원공급 +12V

스마트 농장관리시스템을 구성하는 IO Adapter 보드는 여러 종류의 센서와 다양한 액츄에이터를 라즈베리 파이기반의 Gateway 와 연결해주는 역할을 한다. 무엇보다 IO Adapter 보드의 구조와 연결 커넥터에 대해 숙지해야 한다.

(2) 스마트 농장관리 시스템 소프트웨어

IoT 스마트 농장관리 시스템의 Software 는 크게 게이트웨이(Gateway) 소프트웨어와 안드로이드 앱의 두 부분으로 구성된다. 라즈베리 파이에서 수행되는 게이트웨이 소프트웨어는 센서로부터 데이터를 획득하고 저장하며 원격지 사용자 요청에 따라 액츄에이터를 동작시키며 카메라 영상을 전송하는 기능을 한다.

스마트 폰에서 동작하는 안드로이드 앱은 원격지의 영상을 실시간으로 확인하고, 농장의 센서데이터를 조회하며, 환경에 따라 액츄에이터를 직접 제어하는 기능을 한다. 카메라 구동 드라이버는 이미 게이트웨이에 포함되어 있으며 안드로이드 폰과 연동시 게이트 웨이의 서버 프로그램이 수행된다. 게이트웨이 역할을 하는 라즈베리 파이에는 리눅스 운영체제를 최적화한 라즈비안 OS 가 설치되어 있다. 센서로부터 수집된 데이터를 날짜 시간 데이터와 함께 데이터 베이스에 저장하고, 원격지의 사용자 요청에 따라 카메라 영상을 송신하고 설치된 액츄에이터를 제어하는 기능을 한다. 따라서 게이트웨이와 스마트폰 앱에서 수행되는 Software 의 전체 구조를 살펴보면 아래와 같다.

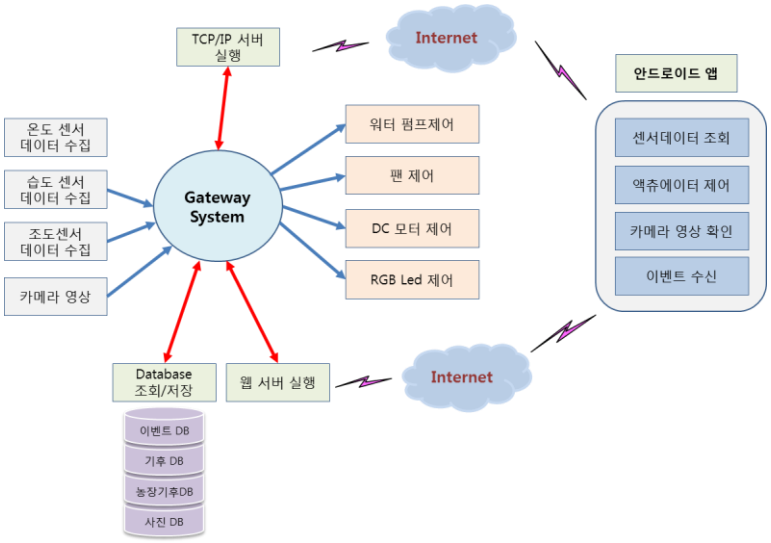


그림 74 스마트 농장관리 시스템의 소프트웨어 구성도

IoT 스마트농장 관리시스템의 소프트웨어 구성은 다음 표와 같다.

표 1 스마트 농장 관리시스템의 소프트웨어 구성

구 분	기능블록	규 격
구동 Software	OS	RASPBIAN - Debian Wheezy. Release date:2014-09-09 Android Jelly Bean 4.3
Compiler		GCC Compiler - 리눅스 커널 기반
센서 프로그램		C, C++, Python, Java
Actuator 프로그램		C, C++, Python, Java
Gateway 소프트웨어		ARM/Linux 기반 싱글보드컴퓨터로 센서어댑터, 액츄에이터 어댑터와 연동하고 인터넷에 연결되어 Gateway 역할을 수행. 센서로부터 수집한 정보를 저장하고 원격지의 사용자에게 전송하거나 제어 신호에 따라 해당 액츄에이터를 On/Off 기능
Camera Module 소프트웨어		Gateway에 CSI 인터페이스 및 커넥터로 연결되는 카메라 모듈로 주요기능은 설치 장소의 환경과 액츄에이터가 동작하는 상황을 사진이나 동영상으로 촬영하여 사용자에게 제공
센서 S/W		센서에서 다양한 데이터를 수집하고, 분석하는 소프트웨어
액츄에이터 S/W		센서 입력 정보 및 사용자의 요구에 따라 모터, 팬, 펌프와 같은 동작 장치를 제어하는 소프트웨어

Web Software	Ethernet을 통해 웹에 접속하고, 웹 서버와 데이터 베이스를 이용하여 Gateway가 수집한 센서데이터를 웹 브라우저에서 조회하고, 현장 환경을 영상을 통해 확인하고 설치된 시설을 직접 제어
Smart Phone Application	Android 기반으로 하는 Mobile Application Software를 통하여 사용자는 스마트폰으로 Gateway의 센서에서 수집된 데이터 및 영상정보를 수신하여 적절한 액츄에이터를 구동 제어. 사용자는 스마트폰을 통해 이미지나 동영상 Play기능, Gateway에 저장된 이벤트와 log기록 확인 가능.

10.2 스마트 농장관리 시스템의 연결 구성

스마트 농장을 설계하기 위해 농장 상황에 대한 많은 분석이 필요하지만 여기서는 몇 가지 요구되는 기능 구현을 위해 단순화 할 필요가 있다. 즉 토양의 온도 습도는 대기중의 온도 습도 센서로 획득하고, 차양이나 실내 농장의 문을 작동하는 액츄에이터로 DC 모터를 사용하는등의 상황을 가정해야 한다. 스마트 농장 시스템은 작은 화분이나 실내 농장에 적용이 가능하게 끔 만들어진 실습 시스템으로 위에 언급한 제약사항을 가정하고 있더라도 충분히 설계 실습할 수 있는 환경을 제공한다.

게이트웨이 기능을 수행하는 라즈베리 파이 연결 커넥터는 40 핀 GPIO 케이블로 연결된다. 라즈베리 파이의 GPIO 포트는 사용자의 프로그램에서 입력으로 설정하거나 출력으로 설정을 한다. 또한 할당된 GPIO 핀 입력을 인터럽트로 설정하여 이벤트 리스너를 함수로 등록하여 사용하는것은 훌륭한 외부 인터럽트를 처리하는 방법이다.

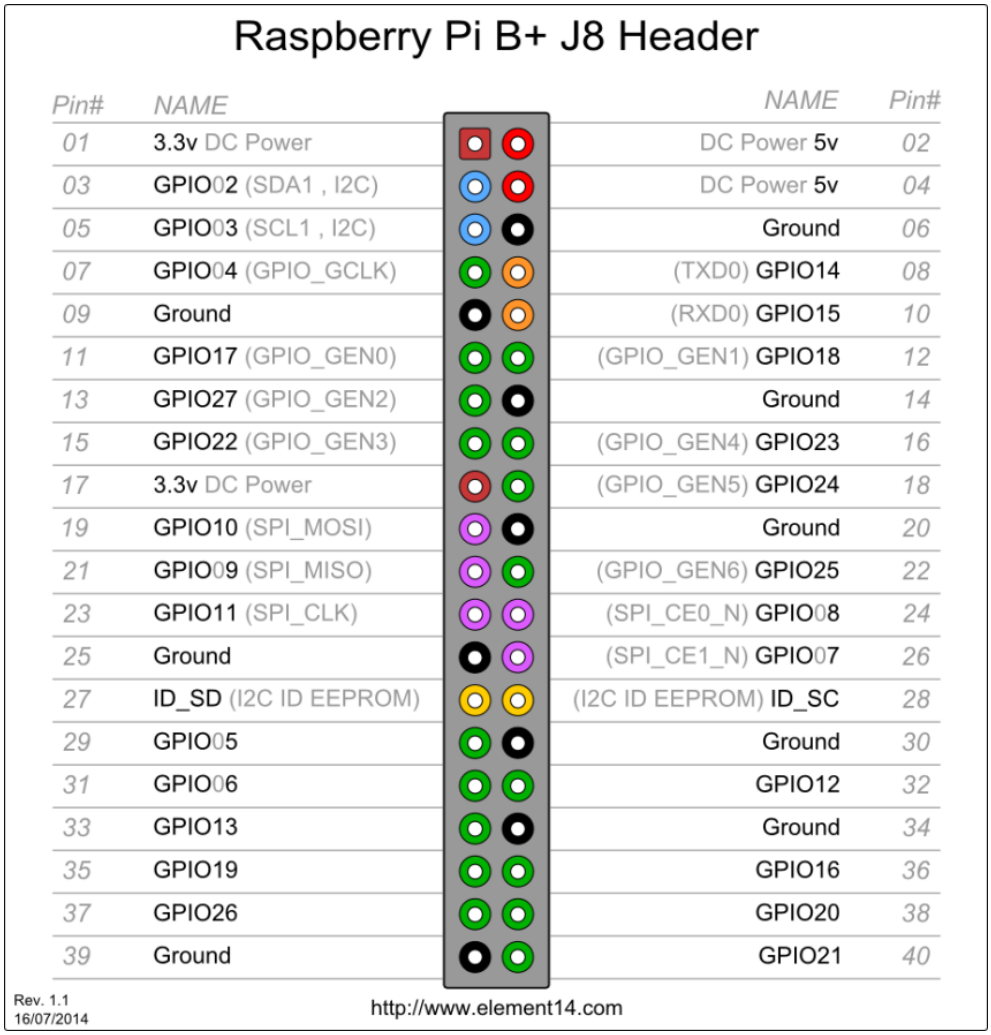


그림 75 라즈베리 파이 GPIO 핀 구성

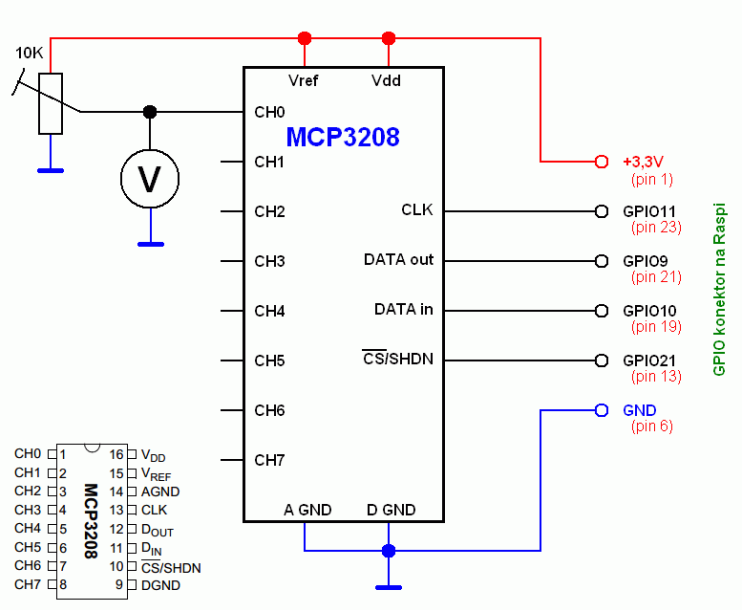


그림 76 라즈베리 파이 ADC 연결도

Gateway 기능의 라즈베리 파이에는 카메라와 Wi-Fi USB 무선 랜카드가 연결된다. IO Adapter 보드에 온습도 센서, 조도센서를 연결한다. IO Adapter 보드에 연결되는 액츄에이터는 워터펌프, FAN, DC Motor, RGB Led 이다.

연결의 정확성

센서와 액츄에이터 보드의 연결도는 실습 후에 스마트 폰으로 입력을 확인하고 액츄에이터를 제어할 때까지 동일한 연결과 테스트를 수행해야 한다. 개별 센서와 액츄에이터 실습시에는 센서 데이터와 액츄에이터 제어가 올라보 되는지를 확인하고 실제 상황을 스마트 폰과 연동 실습할때에는 모든 연결이 정상적으로 정확하게 연결되어야 실습이 진행된다.

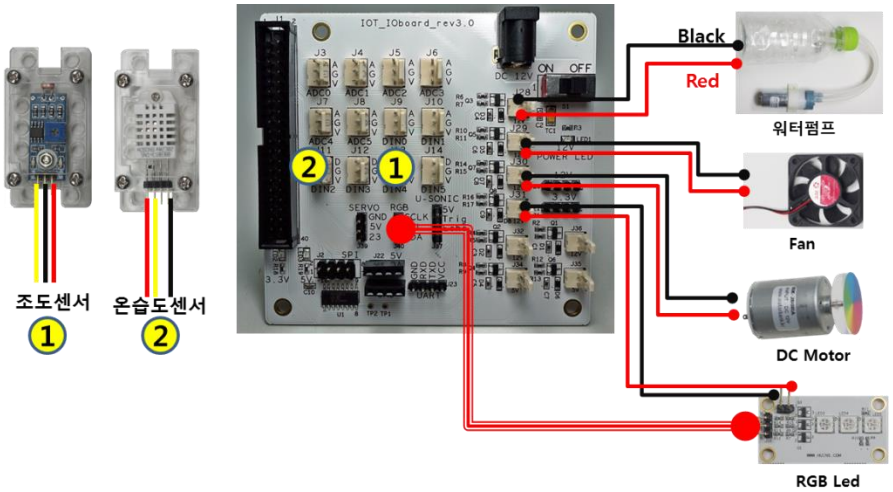


그림 77 센서와 액츄에이터 연결도

IO Adapter 보드는 센서와 액츄에이터를 게이트웨이에 연결하는 역할을 한다. IO Adapter 보드에는 액츄에이터 구동을 위해 DC 12V 전원이 공급되므로 각종 센서나 액츄에이터를 연결할 때 반드시 전원을 끈 상태에서 주의를 기울여 연결한다.

주의

워터펌프, DC motor, RGB Led 등의 액츄에이터의 구동을 위해 IO Adapter 보드에는 DC 12V독전원이 연결되므로 센서나 액츄에이터를 연결할 때는 반드시 전원을 Off 하고 연결해야 하며, 쇼트, 단락에 주의해야 한다.

10.3 스마트 농장관리 시스템 센서 실습

IoT 스마트 농장관리 시스템에 사용되는 센서를 알아보자. 실제 화분이나 식물농장의 상태에는 토양의 온습도를 측정하는 센서가 필수이지만 실습을 위해서 대기 온습도 센서를 사용하기로 한다. 또한 하루 조도량을 정해진 시간별로 측정하기 위해 조도센서를 준비한다.

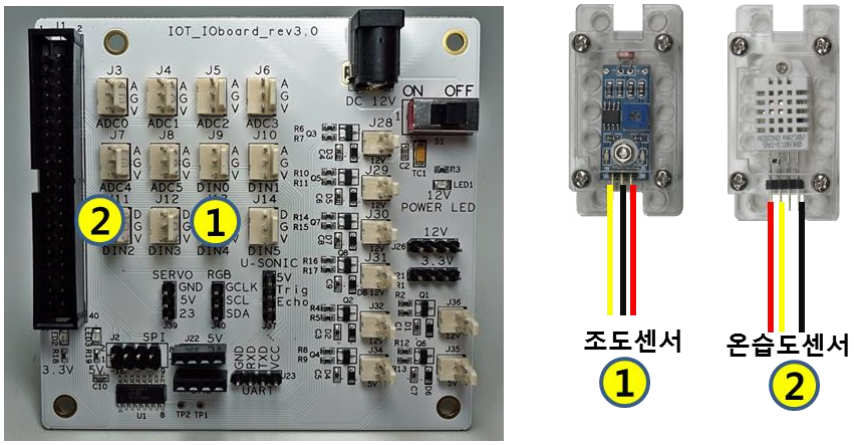


그림 78 스마트 농장관리시스템의 센서구성과 연결 위치

(1) 온도 습도센서 테스트

실습에서 사용되는 온습도 센서는 고정밀 디지털 온습도 센서 모듈 DHT22/AM2302 를 사용하기로 한다. 온도와 습도를 측정하고 이를 이용하여 제어하는 기술이 많은 분야에서 이용되고 있다. 아날로그 출력의 온습도 센서를 사용하여 마이크로컨트롤러에서 응용하기 위해서는 부가적인 회로나 AD 컨버터 회로가 별도로 필요한데 DHT22/AM2302 온습도 센서는 4.7KΩ~10KΩ 의 풀업 저항을 달아 1wire 통신을 이용해 센서 데이터를 획득할 수 있다.

요즘에는 신호 증폭, ADC, 보정, 통신 등의 회로를 내장하고 온도, 습도를 동시에 측정할 수 있는 센서들이 나오고 있다. DHT22/AM2302 은 DHT11 과 사용법이 비슷하나 DHT11 에 비해 정밀도가 높고 응답시간이 빠른 장점이 있다.

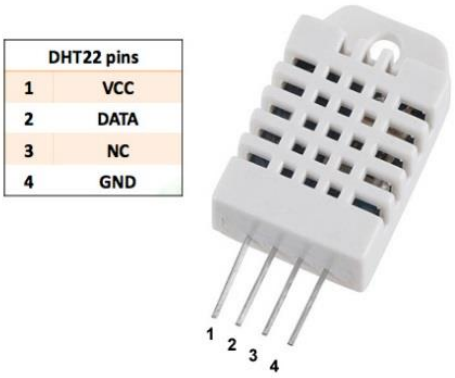


그림 79 DHT22/AM2302 온습도 센서

DHT22/AM2302 센서의 주요 특징을 아래 표로 나타내었다.

표 2 DHT22/AM2302 온습도 센서의 특징

특징	전기적 특성
1. 모델명 : AM2302 2. 정밀도 및 해상도 : 0.1 3. 습도 측정 범위 : 0-100%RH 4. 온도 측정 범위 : -40~80℃ 5. 습도 측정 오차 : ±2%RH 6. 온도 측정 오차 : ±0.5℃ 7. 저전력소모 8. 부가회로가 불필요	- 사용 전압 : 3.3-6V DC - 싱글핀 디지털 데이터 출력 - 측정 응답 시간 : min 2초 - 측정 조건 VDD = 5V , T = 25 ℃ - Pin Connection Pin 1 - 전원 VCC 3.5~5.5V DC Pin 2 - DATA (시리얼 양방향) Pin 3 - 사용안함(No Connection) Pin 4 - GND

DHT22/AM2302 온습도 센서의 통신방법, 프로토콜에 대한 상세한 내용은 데이터 북을 참조한다. (http://www.adafruit.com/datasheets/DHT22.pdf)

아래와 같이 연결한다.

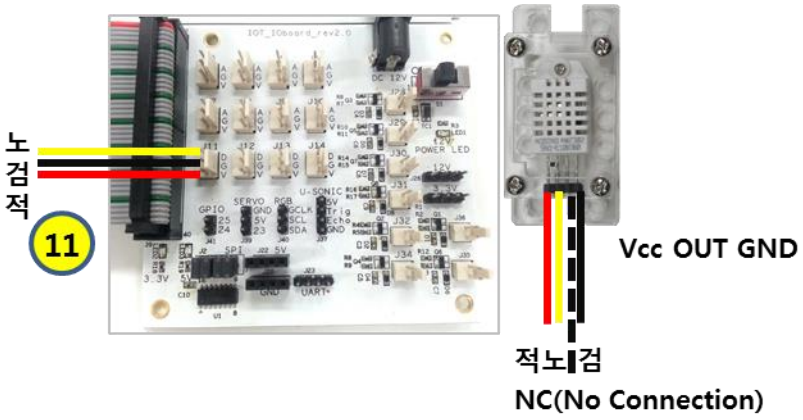


그림 80 IO 어댑터 보드와 습도센서의 연결

A. 라즈베리파이와 센서 연결 회로도

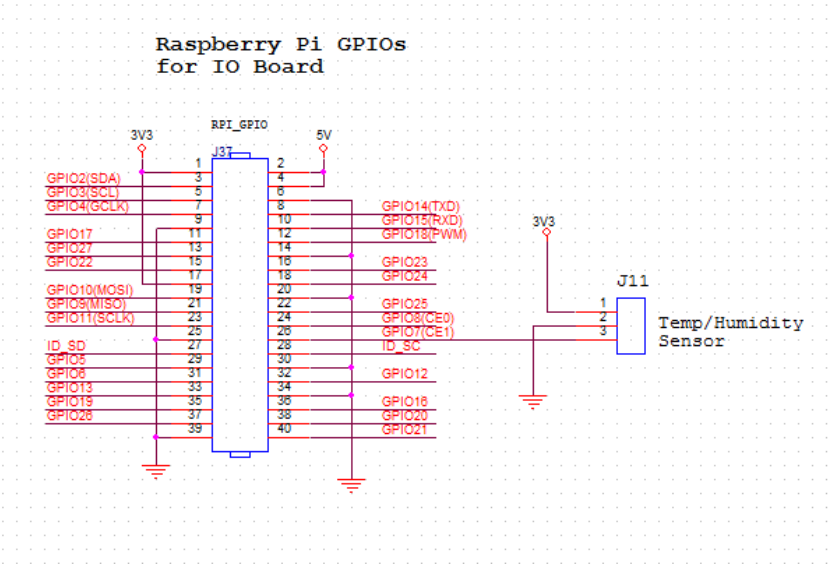


그림 81 라즈베리 파이와 습도센서 연결 회로도

B. wiringPi 설치

라즈베리 파이의 GPIO 를 제어할 수 있는 언어는 C, C++, C#, Python, Java 등 매우 다양하게 있다. 그 중에서 제일 많은 사용자들이 있고, 초기부터 최근까지 꾸준이 업데이트가 유지된 wiringPi 를 이용하기로 한다. (wiringPi -

Gordon 이 개발, GNU LGPLv3 라이선스, 홈페이지 http://wiringpi.com)를 이용한다.

GPIO 란?

GPIO(General Purpose Input Output)란 하나의 연결 단자를 입력이나 출력으로 고정시키지 않고 선택적으로 사용할 수 있게 융통성을 높인 범용 입출력 단자를 말한다. GPIO 단자를 입력으로 프로그래밍하면 인터럽트 소스로 사용 가능하며, 마찬가지로 일부 GPIO 단자는 프로세서를 휴면 상태에서 깨우는 사건(event)를 생성하기도 한다.

1. 새로운 패키지를 실행하기 전에 꼭 해주어야 할것이 하나 있다. 바로 $sudo apt-get update 이다. 이 명령은 패키지 관리 서버로부터 이용 가능한 프로그램 패키지의 목록을 최신버전으로 업데이트 하는 명령이다.

```
$sudo apt-get update
```

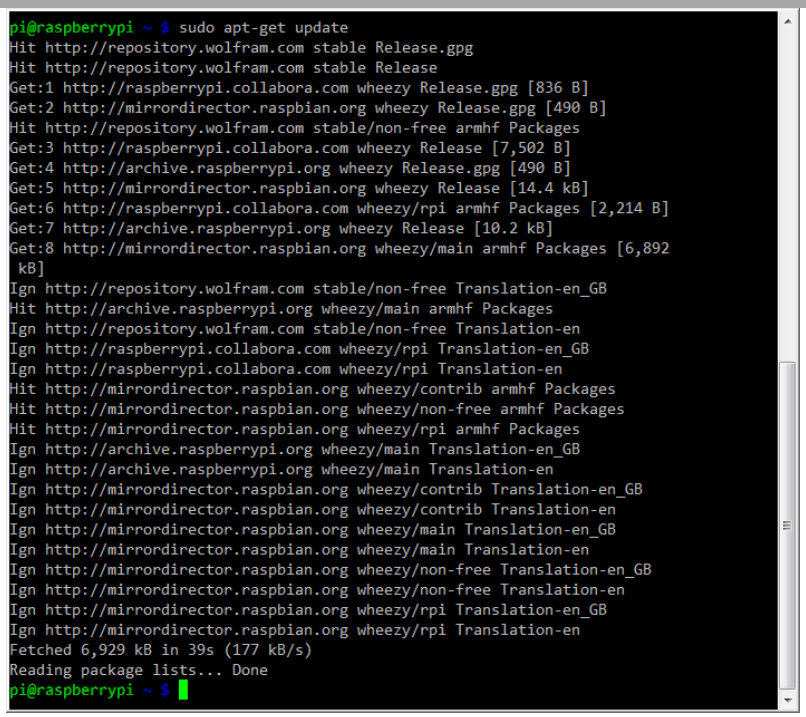


그림 82 sudo apt-get update 실행화면

2. 다음으로 업데이트 명령어로 갱신된 패키지 목록 중 컴퓨터에 설치된 프로그램이 업데이트 되었으면 새로운 버전으로 설치하게 해주는 작업이다. 사용하는 OS 배포버전에 따라 많은 업데이트가 이루어진 경우도 있으며 이러한 경우에는 시간이 오래 걸릴 수 도 있다.

```
$sudo apt-get upgrade
```

3. 소스관리 툴인 git 를 다운로드 한다.

```
$ sudo apt-get install git-core
```

4. git 를 이용하여 "wiringPi" 라이브러리를 다운로드하고 빌드한다.

```
$ git clone git://git.drogon.net/wiringPi
$ cd wiringPi
$ ./build
```

설치가 잘 되었는지 다음의 명령어로 확인한다.

```
$ gpio -v
```

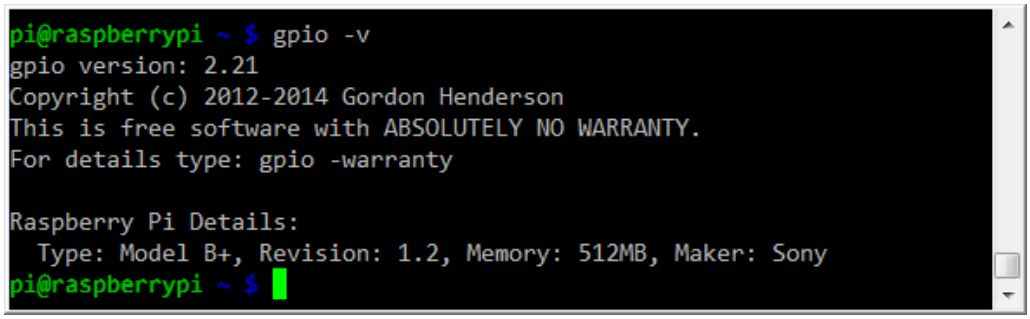
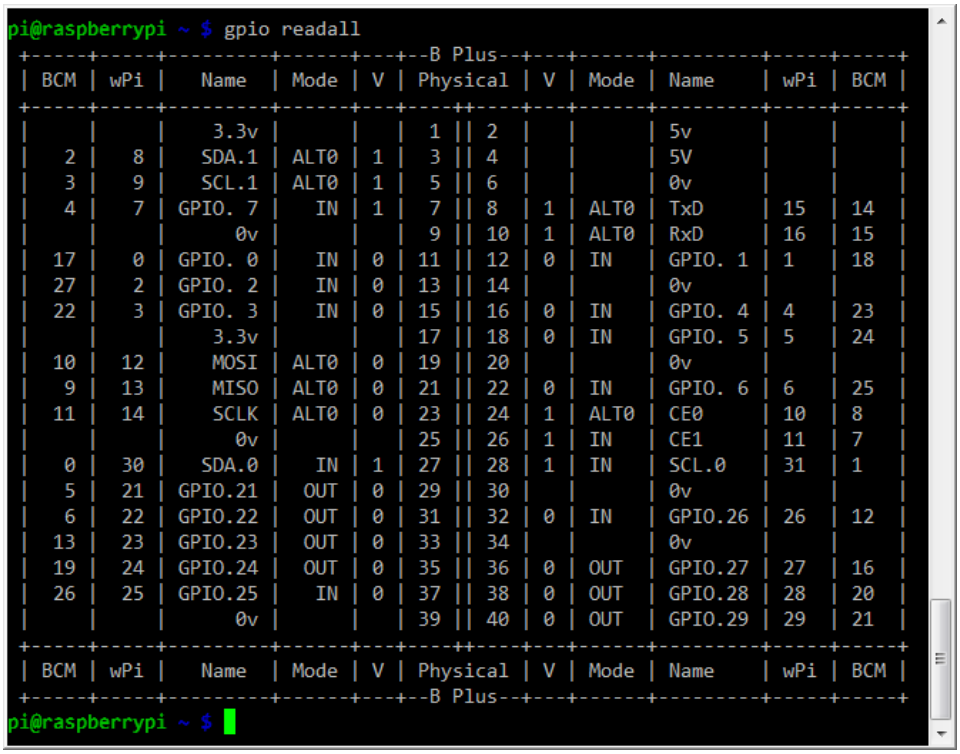


그림 83 wiringPi 설치확인 명령 실행화면

```
$ gpio readall
```

명령어는 현재 GPIO 핀 상태를 모두 보여준다. 테이블 상단의 BCM 열의 번호가 회로도에 나왔는 GPIO 핀 번호를 나타내고, 옆의 wPi 열의 핀 번호가 wiringPi 라이브러리를 사용할 때의 실제 번호이다. 프로그램으로 구현시 우리는 wPi 의 핀 번호를 사용하게 된다. 아래 GPIO 핀 번호는 프로그램 개발시 항상 참고하므로 명령어를 잘 알아두어야 한다.



```
pi@raspberrypi ~ $ gpio readall
 +-----+-----+---------+------+---+--B Plus--+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 |     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
 |   2 |   8 |   SDA.1 | ALT0 | 1 |  3 || 4  |   |      | 5V      |     |     |
 |   3 |   9 |   SCL.1 | ALT0 | 1 |  5 || 6  |   |      | 0v      |     |     |
 |   4 |   7 | GPIO. 7 |   IN | 1 |  7 || 8  | 1 | ALT0 | TxD     | 15  | 14  |
 |     |     |      0v |      |   |  9 || 10 | 1 | ALT0 | RxD     | 16  | 15  |
 |  17 |   0 | GPIO. 0 |   IN | 0 | 11 || 12 | 0 | IN   | GPIO. 1 | 1   | 18  |
 |  27 |   2 | GPIO. 2 |   IN | 0 | 13 || 14 |   |      | 0v      |     |     |
 |  22 |   3 | GPIO. 3 |   IN | 0 | 15 || 16 | 0 | IN   | GPIO. 4 | 4   | 23  |
 |     |     |    3.3v |      |   | 17 || 18 | 0 | IN   | GPIO. 5 | 5   | 24  |
 |  10 |  12 |    MOSI | ALT0 | 0 | 19 || 20 |   |      | 0v      |     |     |
 |   9 |  13 |    MISO | ALT0 | 0 | 21 || 22 | 0 | IN   | GPIO. 6 | 6   | 25  |
 |  11 |  14 |    SCLK | ALT0 | 0 | 23 || 24 | 1 | ALT0 | CE0     | 10  | 8   |
 |     |     |      0v |      |   | 25 || 26 | 1 | IN   | CE1     | 11  | 7   |
 |   0 |  30 |   SDA.0 |   IN | 1 | 27 || 28 | 1 | IN   | SCL.0   | 31  | 1   |
 |   5 |  21 | GPIO.21 |  OUT | 0 | 29 || 30 |   |      | 0v      |     |     |
 |   6 |  22 | GPIO.22 |  OUT | 0 | 31 || 32 | 0 | IN   | GPIO.26 | 26  | 12  |
 |  13 |  23 | GPIO.23 |  OUT | 0 | 33 || 34 |   |      | 0v      |     |     |
 |  19 |  24 | GPIO.24 |  OUT | 0 | 35 || 36 | 0 | OUT  | GPIO.27 | 27  | 16  |
 |  26 |  25 | GPIO.25 |   IN | 0 | 37 || 38 | 0 | OUT  | GPIO.28 | 28  | 20  |
 |     |     |      0v |      |   | 39 || 40 | 0 | OUT  | GPIO.29 | 29  | 21  |
 +-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
 | BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
 +-----+-----+---------+------+---+--B Plus--+---+------+---------+-----+-----+
pi@raspberrypi ~ $
```

그림 84 GPIO 핀 상태확인 실행화면

dht22 온습도 센서는 1 wire 통신으로 온도데이터와 습도 데이터를 송신한다. 실습의 편의를 위해 우선 온도 데이터를 수집하고 다음으로 습도 데이터를 얻어오는 실습을 진행한다. 프로그램은 온도와 습도를 다 받게 되어 있으니 온습도 데이터를 모두 보고 싶은 사용자는 소스 코드를 수정하여 사용하면 된다.

다음은 온도 데이터를 읽어오는 소스이다. 소스코드를 살펴보자. **(get_temperature.c 파일)**

```
/*
/*
 *     dht22.c:
 * Simple test program to test the wiringPi functions
 * Based on the existing dht11.c
 * Amended by technion@lolware.net
 */

#include <wiringPi.h>

#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

```
  #include <sys/types.h>
  #include <unistd.h>

  //#include "locking.h"

  #define MAXTIMINGS 85

  int ret_humid, ret_temp;

  static int DHTPIN = 11;    //

  static int dht22_dat[5] = {0,0,0,0,0};

  static uint8_t sizecvt(const int read)
  {
    /* digitalRead() and friends from wiringpi are defined as returning
a value
    < 256. However, they are returned as int() types. This is a safety
function */

    if (read > 255 || read < 0)
    {
      printf("Invalid data from wiringPi library\n");
      exit(EXIT_FAILURE);
    }
    return (uint8_t)read;
  }

  int read_dht22_dat()
  {
    uint8_t laststate = HIGH;
    uint8_t counter = 0;
    uint8_t j = 0, i;

    dht22_dat[0]  =  dht22_dat[1]  =  dht22_dat[2]  =  dht22_dat[3]  =
dht22_dat[4] = 0;

    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepare to read the pin
    pinMode(DHTPIN, INPUT);

    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++) {
```

```
    counter = 0;
    while (sizecvt(digitalRead(DHTPIN)) == laststate) {
      counter++;
      delayMicroseconds(1);
      if (counter == 255) {
        break;
      }
    }
    laststate = sizecvt(digitalRead(DHTPIN));

    if (counter == 255) break;

    // ignore first 3 transitions
    if ((i >= 4) && (i%2 == 0)) {
      // shove each bit into the storage bytes
      dht22_dat[j/8] <<= 1;
      if (counter > 16)
        dht22_dat[j/8] |= 1;
      j++;
    }
  }

  // check we read 40 bits (8bit x 5 ) + verify checksum in the last
byte
  // print it out if data is good
  if ((j >= 40) &&
      (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
        float t, h;

        h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
        h /= 10;
        t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
        t /= 10.0;
        if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

            ret_humid = (int)h;
            ret_temp = (int)t;
            //printf("Humidity = %.2f %% Temperature = %.2f *C \n", h,
t );
            //printf("Humidity  =  %d  Temperature  =  %d\n",  ret_humid,
ret_temp);

    return ret_temp;
  }
  else
  {
    printf("Data not good, skip\n");
    return 0;
  }
 }
```

```
int main (void)
{
   int received_temp;

   DHTPIN = 11;

   if (wiringPiSetup() == -1)
           exit(EXIT_FAILURE) ;

   if (setuid(getuid()) < 0)
   {
           perror("Dropping privileges failed\n");
           exit(EXIT_FAILURE);
   }
   while (read_dht22_dat() == 0)
   {
           delay(500); // wait 1sec to refresh
   }
   received_temp = ret_temp ;
   printf("Temperature = %d\n", received_temp);

   return 0;
}
```

편집을 완료하였다면 ctrl-x 를 눌르고, 같은 파일 이름이므로 y 를 누르고 엔터를 눌러 빠져나온다.

이제, gcc 명령을 이용하여 위 소스를 컴파일을 하고, 실행해보면 온도 값이 화면에 출력됨을 볼 수 있다.

static int DHTPIN = 11; 은 온 습도 센서 데이터의 출력 핀 번호를 지정한다.

1. 아래와 같이 컴파일을 수행한다.

```
$gcc -o get_temperature get_temperature.c -l wirlingPi
```

2. 아래 명령어로 온도 데이터를 확인하기 위해 실행한다.

```
pi@raspberrypi ~/smartfarm $ sudo ./get_temperature
Temperature = 26
```

아래의 소스는 습도 데이터를 읽어오는 소스코드이다. 온도와 동일한 구조와 방법으로 프로그램 되었다. (get_humidity.c **파일**)

```
  /*
   *     dht22.c:
   * Simple test program to test the wiringPi functions
   * Based on the existing dht11.c
   * Amended by technion@lolware.net
   */

  #include <wiringPi.h>

  #include <stdio.h>
  #include <stdlib.h>
  #include <stdint.h>
  #include <sys/types.h>
  #include <unistd.h>

  //#include "locking.h"

  #define MAXTIMINGS 85

  int ret_humid, ret_temp;

  static int DHTPIN = 11;    //gpio 7

  static int dht22_dat[5] = {0,0,0,0,0};

  static uint8_t sizecvt(const int read)
  {
    /* digitalRead() and friends from wiringpi are defined as returning
a value
    < 256. However, they are returned as int() types. This is a safety
function */

    if (read > 255 || read < 0)
    {
      printf("Invalid data from wiringPi library\n");
      exit(EXIT_FAILURE);
    }
    return (uint8_t)read;
  }

  int read_dht22_dat()
  {
    uint8_t laststate = HIGH;
    uint8_t counter = 0;
    uint8_t j = 0, i;

    dht22_dat[0]  =  dht22_dat[1]  =  dht22_dat[2]  =  dht22_dat[3]  =
dht22_dat[4] = 0;

    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
```

```
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepare to read the pin
    pinMode(DHTPIN, INPUT);

    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++) {
      counter = 0;
      while (sizecvt(digitalRead(DHTPIN)) == laststate) {
        counter++;
        delayMicroseconds(1);
        if (counter == 255) {
          break;
        }
      }
      laststate = sizecvt(digitalRead(DHTPIN));

      if (counter == 255) break;

      // ignore first 3 transitions
      if ((i >= 4) && (i%2 == 0)) {
        // shove each bit into the storage bytes
        dht22_dat[j/8] <<= 1;
        if (counter > 16)
          dht22_dat[j/8] |= 1;
        j++;
      }
    }

    // check we read 40 bits (8bit x 5 ) + verify checksum in the last
byte
    // print it out if data is good
    if ((j >= 40) &&
        (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
          float t, h;

          h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
          h /= 10;
          t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
          t /= 10.0;
          if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

              ret_humid = (int)h;
              ret_temp = (int)t;
```

```
            //printf("Humidity = %.2f %% Temperature = %.2f *C \n", h,
t );
            //printf("Humidity  =  %d  Temperature  =  %d\n",  ret_humid,
ret_temp);

     return ret_humid;
    }
    else
    {
     printf("Data not good, skip\n");
     return 0;
    }
  }

  int main (void)
  {
    int received_humid;

    DHTPIN = 11;

    if (wiringPiSetup() == -1)
            exit(EXIT_FAILURE) ;

    if (setuid(getuid()) < 0)
    {
            perror("Dropping privileges failed\n");
            exit(EXIT_FAILURE);
    }
    while (read_dht22_dat() == 0)
    {
            delay(500); // wait 1sec to refresh
    }
    received_humid = ret_humid;
    printf("Humidity = %d\n", received_humid);

    return 0;
  }
```

편집을 완료하였다면 ctrl-x 를 눌르고, 같은 파일 이름이므로 y 를 누르고 엔터를 눌러 빠져나온다.

이제, gcc 명령을 이용하여 위 소스를 컴파일을 하고, 실행해보면 습도 값이 화면에 출력됨을 볼 수 있다.

static int DHTPIN = 11; 은 온습도 센서 데이터의 출력 핀 번호를 지정한다. 컴파일을 수행한다.

```
$gcc -o get_humidity get_humidity.c -l wirlingPi
```

아래 명령어로 온도 데이터를 확인하기 위해 실행한다.

```
pi@raspberrypi ~/smartfarm $ sudo ./ get_humidity
Humidity = 26
```

(2) 조도센서 테스트

조도센서는 광량(빛의 세기)를 센싱하여 전기적 신호로 출력하는 센서이다. 센서의 핀이 3 개로 구성되어 단순히 빛이 있다 없다의 출력을 내주는 디지털 조도센서가 있고, 아날로그 디지털 두 신호가 출력되는 4 핀의 조도센서가 있다. 여기서는 식물농장 시스템에 설치된 조도 센서가 디지털 아날로그 두가지 출력을 내는 4pin 조도센서를 싫습한다. 그리고 디지털 출력을 내는 3pin 조도센서를 테스트 한다.

4 핀 조도센서 – Analog 출력 실습

본 실습에서 사용되는 조도센서는 모델 6495 조도센서(4-핀)이다. 조도센서의 출력은 두가지를 이용할 수 있는데 일정 밝기 이상에서 하이가 출력되는 디지털 출력과 빛의 양에 따라 값이 아날로그로 변환되어 출력하는 아날로그 출력을 동시에 가지고 있다. 여기에서는 아날로그 출력을 이용한다. 아날로그 출력값은 8 채널 12bit 아날로그 디지털 컨버터인 MCP3208 을 이용하여 디지털 값을 출력하여 보기로 한다.

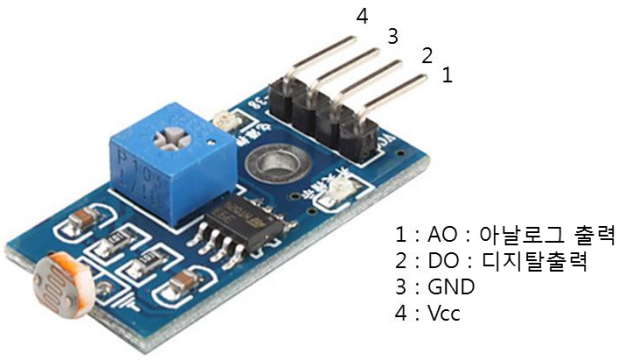


그림 85 4 핀 6495 조도센서

표 21 조도센서의 특징

특징	전기적 특성
1. 모델명 : GL5537(5Pie Photo Resistor) 2. 종류 : Cds Photo CELL 2. 감도 조절 (파란색 가변저항 조정)밝기와 주변 빛의 세기를 감지 3. 출력 : 아날로그 전압 출력, 디지털 스위치 출력 (0, 1) 4. Power dissipation : 100mW 5. Max Voltage : 150Vdc or AC 6. Temp.Range: : -30~+70℃	- 작동 전압 : 3.3V ~ 5V DC - 출력 : 아날로그 전압 출력, 디지털 스위치 출력 (0, 1) - Light Resistance at 10Lux(at 25): min 20, max 50Kohm- 측정 조건 VDD = 5V , T = 25 ℃ - Gamma Value at 10~100Lux: 0.7typ - Dark Resistance at 0 Lux(10sec after shut off 10Lux): min 2Mohm - Peak Spectrol Response: min 550, max 650nm

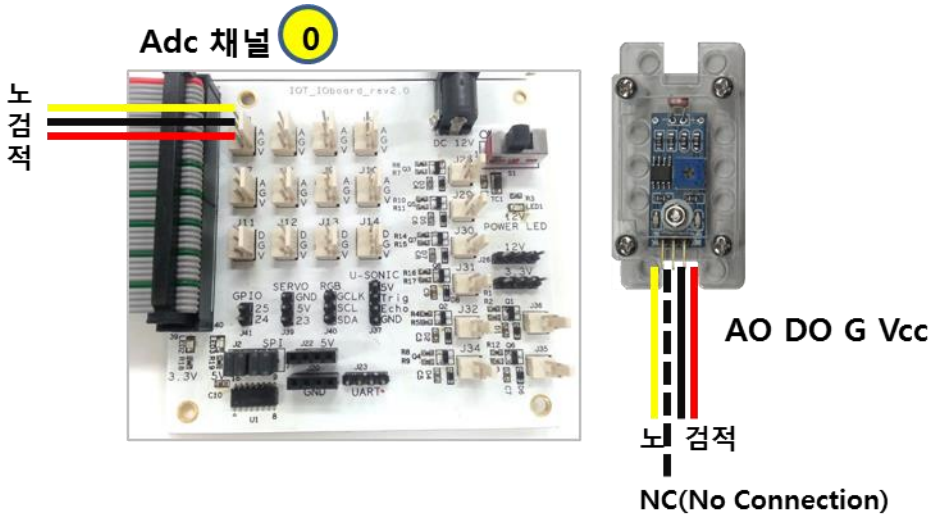


그림 86 4 핀 조도센서와 IO Adapter 보드 연결

ADC 컨버터 MCP3208 설명

MCP3208 은 8-Channel 12-Bit A/D Converters 이다. 즉, 센서의 출력값인 아날로그 값을 12 비트의 디지털 값으로 변환하는 ADC 칩이다. 예를들어 0v ~ 3.3v 의 센서 출력 값을 0 ~ 4095 (12bit) 의 디지털 값으로 변환해주는 기능을 해준다. 라즈베리파이는 범용적인 목적으로 입/출력을 담당하는 GPIO (General Purpose Input / Output) 를 가지고 있으며, SPI 통신, I2C 통신, UART 통신을 가지고 있지만, 아쉽게도 ADC 기능이 없어서 센서를 활용못하는 단점이 있다. 그래서, ADC 칩인 MCP3208 을 이용하여 외부 ADC 기능을 가지도록 한다. 이를 위해서는 라즈베리파이와 MCP3208 간에 SPI 통신을 이용하여 ADC 데이타를 받아와야 한다.

MCP3208 는 아래와 같은 핀 구조를 가지고 있다. 이를 라즈베리파이의 SPI 와 연결하기위해서는 아래의 회로도와 같이 연결해줘야 한다. MCP3208 은 총 8 개의 ADC 포트(CH0~CH7)를 가지고 있는데 이번 강좌에서는 CH0 에 가변저항을 달아 돌려가면서 ADC 값을 변경해보며 테스트를 해보도록 하겠다. 자신이 이용하고자하는 센서를 CH0 ~ CH7 에 연결하여 사용하여도 무방하다.

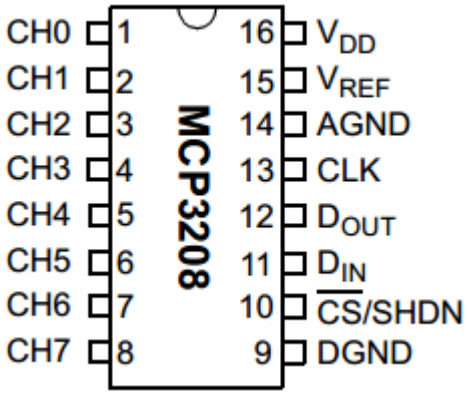


그림 87 MCP3208 Pin Map

Name	Function
V_{DD}	+2.7V to 5.5V Power Supply
DGND	Digital Ground
AGND	Analog Ground
CH0-CH7	Analog Inputs
CLK	Serial Clock
D_{IN}	Serial Data In
D_{OUT}	Serial Data Out
$\overline{CS}$/SHDN	Chip Select/Shutdown Input
V_{REF}	Reference Voltage Input

그림 88 3 MCP3208 Pin Description

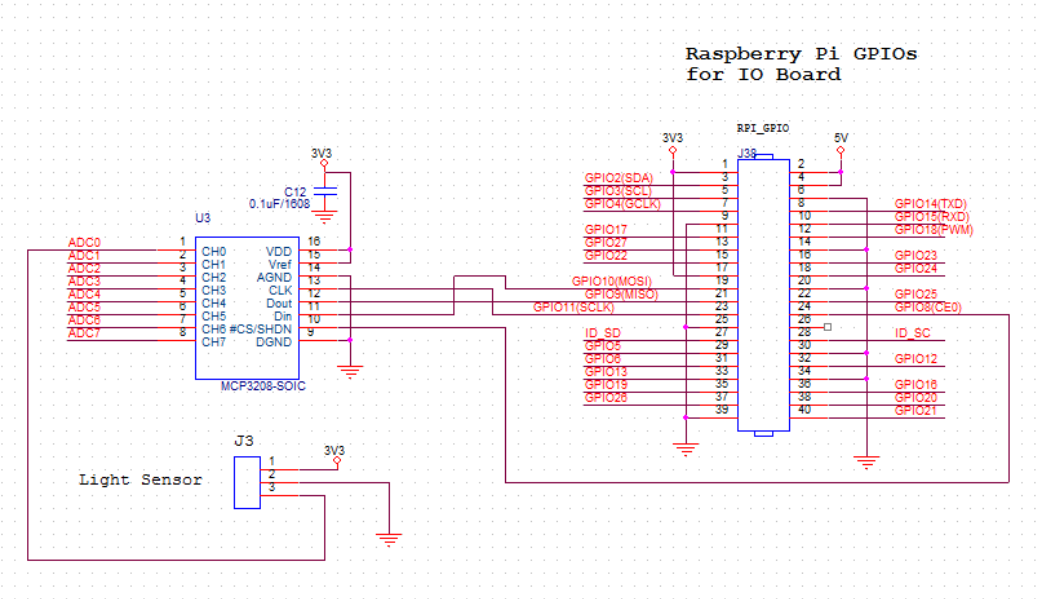


그림 89 라즈베리 파이와 ADC 간의 핀 연결도

이번 SPI 테스트는 C 언어로 작성해 보고자 한다. 매우 간단하니 천천히 따라해 보자. 우선, 위와 같은 회로를 구성한 후 전원을 넣고, 라즈베리파이를 구동하자. 아래의 순서대로 소스코드를 작성하도록 하자.

1. **(SPI 및 I2C 활성화)** 라즈베리파이는 기본적으로 SPI 및 I2C 통신이 활성화 되어 있지 않다. 아래의 설정 파일을 수정하여 비활성화 설정된 두 기능 모두를 활성화 한다. 먼저 raspi-blacklist.conf 파일을 오픈한다, 파일에서는 #을 앞에 넣어주어 블랙리스트를 비활성화 시킨다.

```
$sudo nano /etc/modprobe.d/raspi-blacklist.conf
```

```
# blacklist spi and i2c by default (many users don't need them)

#blacklist spi-bcm2708
#blacklist i2c-bcm2708
blacklist snd-soc-pcm512x
blacklist snd-soc-wm8804
```

2. **(SPI 로드)** SPI 기능을 사용하기 위해서는 아래와 같이 커널로 부터 spi 드라이브를 로드해야한다.

```
gpio load spi
```

3. **(작업폴더 생성)** SPI 에 관한 설정이 끝났다. 다음은 자신의 작업 폴더 생성 및 이동하여 아래와 같이 작업 폴더를 설정해 주자.

```
mkdir smartfarm
cd smartfarm
```

4. **(소스코드 작성)**프로그램 작성 (편집기는 vi, nano, Leafpad 등 원하는 것을 사용한다.)

프로그램은 "Gordons Projects" 의 "wiringPi" 의 라이브러리를 이용할 것이다. 특히, 이번에 사용할 라이브러리는 wiringPiSPI 이다. 자세한 라이브러리를 확인하고자 하는 경우에는 wiringPiSPI 홈페이지를 참고한다.

위와 같이 내용을 전부 입력해주고, "spi-test.c" 라는 이름으로 저장해주자.

※ SPI 통신 프로그램은 마스터와 슬레이브로 나뉜다. 본 실습에서는 라즈베리파이가 마스터이고, MCP3208 이 슬레이브이다. 즉, 마스터인 라즈베리파이가 정해진 명령어를 슬레이브인 MCP3208 에게 정해진 명령을 날려주면 그에 해당되는 값을 전송해주는 시스템이다. 우리는 센서 값을 ADC 하여 라즈베리파이가 받을 필요가 있으므로 아래와 같은 규칙(MCP3208 데이타시트, 첨부파일 참조)으로 데이타를 전송해야 한다. 라즈베리파이는 총 3 개의 바이트를 보내주는데, 그 내용은 아래와 같다.

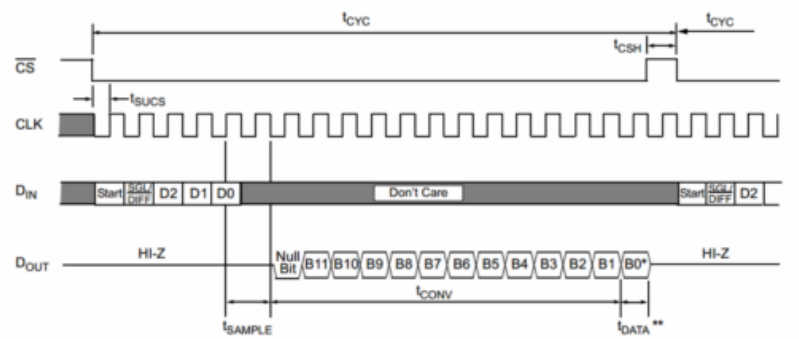
1 첫째 바이트 0 0 0 0 0 Start SGL D2

2 번째 바이트 D1 D0 x x x x x x

3 번째 바이트 x x x x x x x x

(D2,1,0 은 ADC 채널을 의미하고, x 는 어떠한 값도 상관없는 Don`t Care 를 의미한다.)

이와 같이 라즈베리파이는 3 바이트의 데이타를 보냄과 동시에 MCP3208 로부터 3 바이트의 데이타를 받게 되는데 그 데이타 중에서 밑으로부터 12 비트가 ADC 결과값이다. 이를 위의 소스와 같이 비트연산으로 하나의 데이타 값으로 변환하면 된다.



* After completing the data transfer, if further clocks are applied with $\overline{CS}$ low, the A/D converter will output LSB first data, followed by zeros indefinitely (see Figure 5-2 below).

** t_{DATA}: during this time, the bias current and the comparator power down while the reference input becomes a high impedance node, leaving the CLK running to clock out the LSB-first data or zeros.

그림 90 MCP3208 데이터 수신과정

Control Bit Selections				Input Configuration	Channel Selection
Single /Diff	D2	D1	D0		
1	0	0	0	single-ended	CH0
1	0	0	1	single-ended	CH1
1	0	1	0	single-ended	CH2
1	0	1	1	single-ended	CH3
1	1	0	0	single-ended	CH4
1	1	0	1	single-ended	CH5
1	1	1	0	single-ended	CH6
1	1	1	1	single-ended	CH7
0	0	0	0	differential	CH0 = IN+ CH1 = IN-
0	0	0	1	differential	CH0 = IN- CH1 = IN+
0	0	1	0	differential	CH2 = IN+ CH3 = IN-
0	0	1	1	differential	CH2 = IN- CH3 = IN+
0	1	0	0	differential	CH4 = IN+ CH5 = IN-
0	1	0	1	differential	CH4 = IN- CH5 = IN+
0	1	1	0	differential	CH6 = IN+ CH7 = IN-
0	1	1	1	differential	CH6 = IN- CH7 = IN+

그림 91 MCP3208 의 채널정보

다음은 조도 센서에서 데이터를 읽어오는 소스코드이다. 먼저 센서 데이터를 읽어오는 소스를 살펴보자. (getlightsensor .c **파일**)

편집은 nano 편집기를 이용한다. 편집 명령은 $nano getlightsensor.c 명령으로 시작하고 편집이 완료되면 ctrl-x 를 누르고 파일네임 변경이 없을 경우 y 를 눌러 enter 로 빠져 나온다.

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
```

```
#include <wiringPi.h>
#include <wiringPiSPI.h>

#define CS_MCP3208 8 //GPIO 8
#define SPI_CHANNEL 0
#define SPI_SPEED 1000000 //1Mhz

// spi communication with Rpi and get sensor data

int read_mcp3208_adc(unsigned char adcChannel)
{
   unsigned char buff[3];
   int adcValue = 0;

   buff[0] = 0x06 | ((adcChannel & 0x07) >> 2);
   buff[1] = ((adcChannel & 0x07) << 6);
   buff[2] = 0x00;

   digitalWrite(CS_MCP3208, 0);
   wiringPiSPIDataRW(SPI_CHANNEL, buff, 3);

   buff[1] = 0x0f & buff[1];
   adcValue = (buff[1] << 8 ) | buff[2];

   digitalWrite(CS_MCP3208, 1);

   return adcValue;
}

int main(void) {

   unsigned char adcChannel_light = 0;

   int adcValue_light = 0;// 여기서 읽어올 아날로그 채널 설정

   float vout_light;
   float vout_oftemp;
   float percentrh = 0;
   float supsiondo = 0;

   printf("start");


   if(wiringPiSetupGpio() == -1)
   {
           fprintf(stdout,   "Unable   to   start   wiringPi   :%s\n",
strerror(errno));
           return 1;
   }

   if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
```

```
    {
            fprintf(stdout,     "wiringPiSPISetup     Failed     :%s\n",
strerror(errno));
            return 1;
    }

    pinMode(CS_MCP3208, OUTPUT);

    while(1)
    {
            adcValue_light = read_mcp3208_adc(adcChannel_light);

            //printf("Humiity = %u temparature = %u\n", adcValue_humi,
adcValue_temp);
            printf("light sensor = %u\n", adcValue_light);

            delay(100);
    }
    return 0;
 }
```

편집이 완료되면 ctrl-x 를 누르고 파일네임 변경이 없을 경우 y 를 눌러 enter 로 빠져 나온다.

5. **(컴파일)** gcc 를 이용하여 소스 코드를 다음과 같이 컴파일 한다.

```
$gcc -o getlightsensor getlightsensor.c -lwiringPi
```

: 위의 컴파일 명령을 실행해 주면 'spi-test' 라는 실행 파일이 생성된다.

6. **(실행)** 프로그램을 실행한다

라즈베리파이의 GPIO 관련 프로그램 실행은 항상 루트 권한으로 실행해줘야 한다. 작성한 프로그램을 실행해 보자.

```
$sudo ./ getlightsensor
```

실행하게 되면 아래와 같이 터미널을 통해 ADC 된 조도센서의 데이터 값을 확인해 볼 수 있다. 이것은 한 채널의 ADC 값을 테스트해 본 것이고, 소스를 수정하여 각 채널의 ADC 를 확인해 볼 수도 있다.

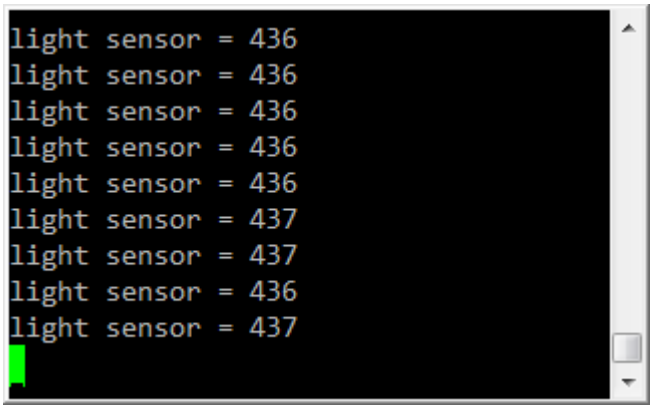


그림 92 아주 밝은 빛일때의 조도센서 출력 값

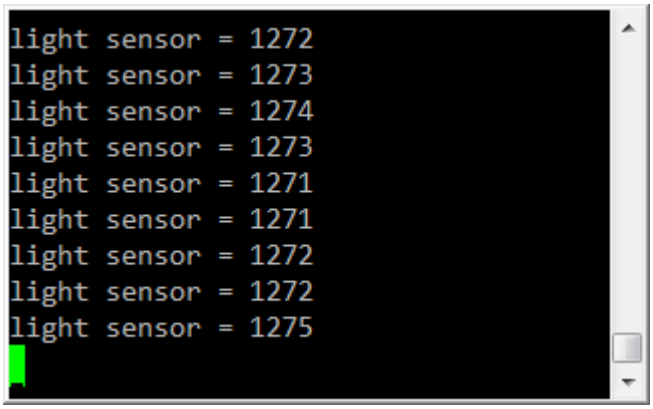


그림 93 보통 밝기에서의 조도센서 출력 값

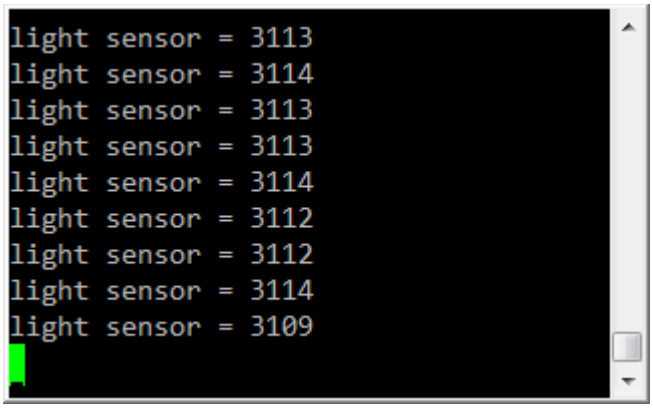


그림 94 어둡게 하였을 경우 조도센서 출력 값

3pin 조도센서 – 디지털 출력 실습

만약에 조도센서의 출력이 3 핀이라면 출력은 디지털 신호로 high(1), Low(0)을 출력하는 센서이다. 이러한 경우에는 앞에서 설명한 아날로그 값을 읽어서 판단하지 않고 가변저항을 돌려 정한 기준값과 비교하여 단순히 밝다, 어둡다 를 표시해 주어야 한다. 그 프로그램 예제를 살펴볼 것이다. 연결은 J13 에 한다.

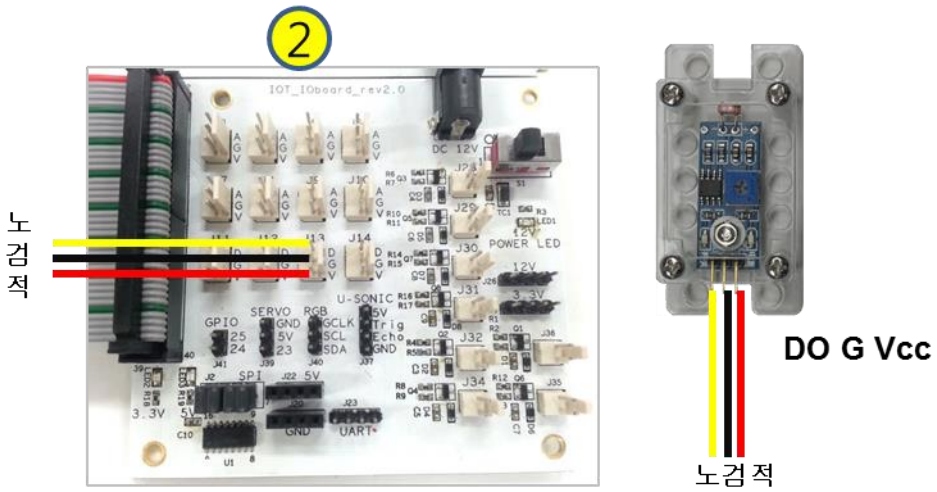


그림 95 3 핀 조도센서와 IO adapter 보드 연결

다음은 3-핀 조도센서에서 데이터를 읽어오는 소스코드이다. **(lightdetect.c 파일)**

$nano lightdetect.c 명령어로 소스를 입력한다.

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>

#define LIGHTSEN_OUT 2  //gpio27 - J13 connect

// ------------------------------------------------------------------------
------
// main
```

```
  int main(void)
  {
     // sets up the wiringPi library
     if (wiringPiSetup () < 0)
     {
             fprintf (stderr, "Unable to setup wiringPi: %s\n", strerror
(errno));
             return 1;
     }

     pinMode(LIGHTSEN_OUT, INPUT);

     // set Pin 17/0 generate an interrupt on high-to-low transitions
     // and attach myInterrupt() to the interrupt
     /*if ( wiringPiISR (LIGHTSEN_OUT, INT_EDGE_RISING, &myInterrupt) <
0 )
     {
             fprintf  (stderr,  "Unable  to  setup  ISR:  %s\n",  strerror
(errno));
             return 1;
     }*/

     // display counter value every second.
     while ( 1 )
     {
             //printf( "%d\n", eventCounter );
             //eventCounter = 0;

             if(digitalRead(LIGHTSEN_OUT) == 0)
                     printf("light full ! \n");
             if(digitalRead(LIGHTSEN_OUT) == 1)
                     printf("dark \n");

             delay( 200 ); // wait 1 second
     }

     return 0;
  }
```

아래 명령으로 컴파일을 실행한다.

```
  $gcc -o lightdetect lightdetect.c -lwiringPi
```

아래 명령으로 실행한다.

```
  $sudo ./lightdetect
```

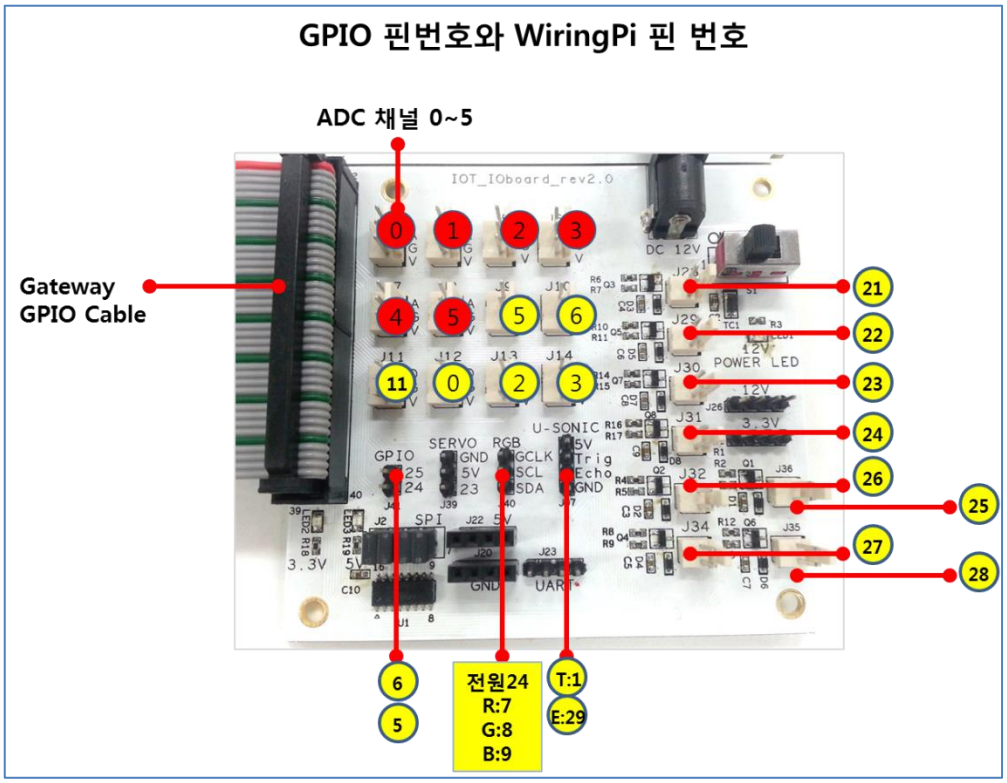
아래는 터미널에서 실행화면을 나타낸다.

```
pi@raspberrypi ~/smartfarmtest $ sudo ./lightdetect
light full !
light full !
light full !
light full !
dark
dark
dark
dark
light full !
light full !
```

(3) 다른 센서 테스트

다른 센서들 – 사운드, 인체감지, 불꽃, 가스, 초음파, 근접센서 – 도 위에서 설명한 조도센서와 온습도 센서 실습방법과 동일하게 테스트를 수행한다. 우리는 실습 편의성과 하드웨어 연결에 시간을 들이지 않기 위해 IO 어댑터 보드를 사용하였다. 중요한것은 센서의 출력핀이 디지털인지 아날로그인지와 해당 GPIO 핀 넘버가 몇 번인지 파악하는 것이다. 그런 상태에서라면 IO 어댑터 보드도 사용하지 않아도 되고, 프로그램도 거의 동일한 코드를 사용하여 시험할 수 있게 된다.

IO adapter 보드와 대응되는 GPIO 핀을 표시한 그림을 참고한다.



우선은 첨부 CD 의 사용자 매뉴얼 폴더에 smart iot-sensor-actu-all.pdf 파일을 참고한다. Smart IoT 시스템의 모든 센서와 액츄에이터를 테스트 할 수 있도록 설명한 문서이다.

10.4 스마트 농장관리 시스템 액츄에이터 실습

스마트 식물농장의 액츄에이터는 워터펌프, 팬, DC Motor, RGB Led 와 같이 4 가지를 사용한다. 토양이 말라 있을 때 물을 뿌려주고, 실내 농장의 온도에 따라 환기 팬을 작동하고, 야간이나 구름이 낀 날은 청색과 적색의 식물 Led 를 켜준다. 또한 차양을 올린다거나 문을 열기 위해 DC 모터를 사용하는것으로 가정한다.

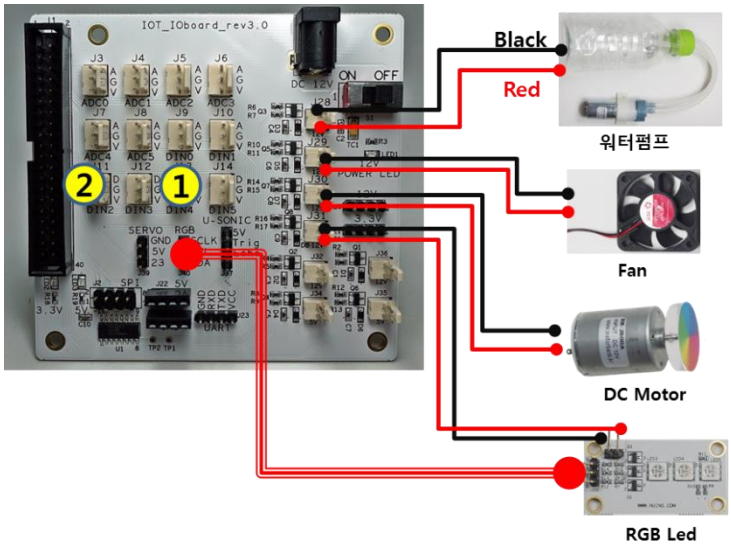


그림 96 스마트 농장관리시스템의 액츄에이터 구성과 연결위치

4 종의 액츄에이터에 대해 외관, 사이즈와 제품특성에 대해 알아보고 전체 액츄에이터에 대한 테스트는 일괄적으로 수행하도록 프로그램을 제작하고 실행해 본다.

스마트 농장에 사용되는 액츄에이터를 제어하는 방법은 GPIO 핀을 통한 On/Off 제어가 전부이다. 각각의 액츄에이터는 DC 12V 와 DC 5V 전원을 사용하며 TR 과 저항등 부가적인 회로는 IO Adapter 보드에 실장되어 있다. 액츄에이터 부분의 상세한 회로는 자료를 참고한다. (IO Adapter 보드 회로도 참조)

프로세스 종료시 주의할 점

액츄에이터 제어 실습시 보통 실험을 종료할 경우 [ctrl-c] 를 눌러 종료하게 되는데, 이 때 GPIO 신호는 그대로 실습때의 상태를 유지하게 된다. 즉, 전원이 켠 상태나 전등이 ON 상태로 프로그램을 종료해도 전원이 공급되고, 전등은 ON 상태로 남아있게 되는데 이렇게 종료되는 것을 방지하는 방법을 설명한다.

컴퓨터 시스템 상에서 발생할수 있는 상황도 여러가지가 있다. 이는 OS 에 따라 다르게 설정된다. OS 에서 어떠한 시그널을 지원하는 지는 kill 명령을 이용해서 알아볼수 있다. 아래는 라즈베리 파이에서 지원하는 시그널을 캡쳐한 화면이다.

```
pi@raspberrypi ~/safeguard $ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5)
SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL      10)
SIGUSR1
 11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15)
SIGTERM
 16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20)
SIGTSTP
 21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25)
SIGXFSZ
 26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30)
SIGPWR
 31) SIGSYS      34) SIGRTMIN    35) SIGRTMIN+1  36) SIGRTMIN+2  37)
SIGRTMIN+3
 38) SIGRTMIN+4  39) SIGRTMIN+5  40) SIGRTMIN+6  41) SIGRTMIN+7
42) SIGRTMIN+8
 43) SIGRTMIN+9  44) SIGRTMIN+10 45) SIGRTMIN+11 46) SIGRTMIN+12
47) SIGRTMIN+13
 48) SIGRTMIN+14 49) SIGRTMIN+15 50) SIGRTMAX-14 51) SIGRTMAX-13
52) SIGRTMAX-12
 53) SIGRTMAX-11 54) SIGRTMAX-10 55) SIGRTMAX-9  56) SIGRTMAX-8
57) SIGRTMAX-7
 58) SIGRTMAX-6  59) SIGRTMAX-5  60) SIGRTMAX-4  61) SIGRTMAX-3
62) SIGRTMAX-2
 63) SIGRTMAX-1  64) SIGRTMAX
 pi@raspberrypi ~/safeguard $ ^C
```

보통 32 번까지의 시그널은 대부분의 UNIX 계열에서 비슷한 값을 보여주며, 33 번부터 64 번까지는 아마도 OS 마다 약간식의 차이를 보여줄것이다. 33 번부터 62 번까지는 리얼타임 시스널을 위해서 예약된 시그널들이다. 각각의

시그널은 int 형의 숫자로 표시되는데, 시그널 번호에 따라서 다른 상황을 나타낸다. 각각의 시그널은 int 형의 숫자로 표시되는데, 시그널 번호에 따라서 다른 상황을 나타낸다. 모든 시그널에 대해서 설명하기엔 양이 좀 많으므로 그중 중요한 시그널만을 정리하도록 하겠다.

표 21 프로세스 시그널

시그널 이름	설명
SIGHUP	터미널을 읽어버렸을때 발생한다.
SIGABRT	프로그램의 비정상종료시 발생한다.
SIGINT	Control-C 나 DELETE 키를 입력했을때 발생한다.
SIGIO	비동기적인 입출력이 발생했을때
SIGKILL	프로세스를 죽이기 위해서
SIGPIPE	단절된 파이프에 write 할경우 발생
SIGSEGV	잘못된 메모리 참조(주로 포인터를 잘못 썼을때)
SIGSTOP	프로세스의 일시중단 (Ctrl+ z)
SIGSUSR1	사용자를 위해 정의된 시그널

.

실습시 예제를 강제로 종료하기 위해 Ctrl-c 를 누른 순간 SIGINT 신호가 발생되는데 우리는 이 신호를 이용하여 정리를 해주고 나오면 된다. 예제로 살펴본다. **(예제: sigint.c)**

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

void sig_handler(int signo); // 마지막 종료 함수

unsigned char endflag = 1;
```

```
int main()
{
	int i = 0;
	signal(SIGINT, (void *)sig_handler);

	while(endflag)
	{
		printf("%d₩n", i);
		i++;
		sleep(1);
	}
	return 1;
}


void sig_handler(int signo)
{
	printf("process stop₩n");
    endflag = 0;
    exit(0);
}
```

실습을 진행하면서 GPIO 핀을 초기상태(low)로 초기화, 즉 액티브 되지 않은 상태로 만들어 주고 빠져나오게 하기 위해 위 코드를 이용하면 될것이다.

(1) 워터펌프

펌프란 사전적 의미로 전기적.기계적인 에너지를 받아 유체(기체, 액체)에 힘을 가해 압력, 속도 등을 높임으로써 낮은 곳에서 높은 곳으로의 이송, 관을 통한 수송, 압력용기 등에 가압하는 기계라고 정의하고 있으나 섬유, 모래 등의

고형물을 이송하는 특수펌프도 있어 넓은 의미에서 보면 유체뿐만 아니라 고체도 취급 대상이 된다. 수처리 플랜트를 중심으로 보면 펌프는 가장 중요한 역할을 하는 기계로 물을 이송하거나 가압하는 등의 목적에 사용되며 원심펌프(Centrifugal Pump)가 약 70% 이상을 차지하고 있다. 여기서 사용하는 워터펌프는 DWP2760 이며 자세한 사양은 아래를 참고한다.



그림 97 DWP-2760 워터펌프 외관

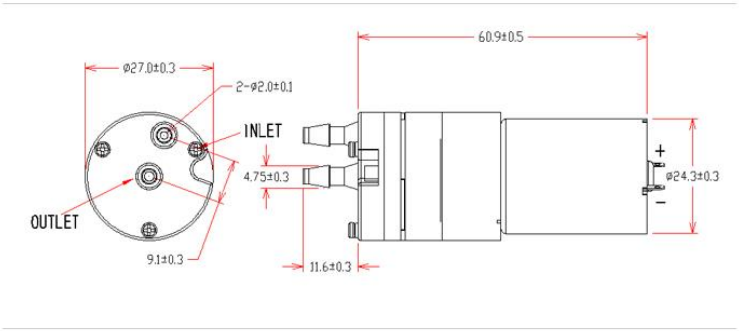


그림 98 DWP2760 워터펌프 기구도면

표 22 DWP2760 전기적 사양

정격전압	DC 12V

구동전압	DC 9V ~ DC15V
전류	무부하시 280mA 이하
최대입력	>14.5 PSI
Water Flow	≥400(±20%)mL/Min at 90℃
소음	65dB
용도	커피머신용, 90℃ 뜨거운 물 사용가능
수명	정격전류에서 50초 동작, 20초 멈춤이 1Cycle - 6,000번

아래는 워터펌프에 전원을 공급하는 회로도이며, GPIO5 번을 이용해 제어한다.

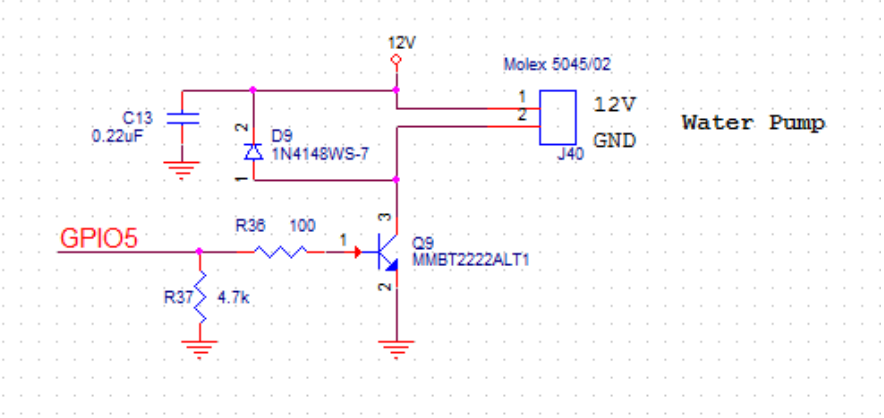


그림 99 워터펌프 구동 회로도

액츄에이터 실습시 안전하게 종료하기 위해 설명한 SIGINT 신호를 이용하여 구현하는 소스코드를 살펴보자. (pumpon.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일
```

```
#include <wiringPi.h>

#define PUMP           21 // BCM_GPIO 5

void sig_handler(int signo); // 마지막 종료 함수

int main (void)
{
    signal(SIGINT, (void *)sig_handler);  //시그널 핸들러 함수

    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            return 1 ;
    }

    pinMode (PUMP, OUTPUT) ;

    for (;;)
    {
            printf("here - pump on₩n");
            digitalWrite (PUMP, 1) ; // On

            delay (2000) ; // ms

            digitalWrite (PUMP, 0) ; // Off

            delay (2000) ;

    }
    return 0 ;
```

```
}

void sig_handler(int signo) //시스널 핸들러 함수 구현 //GPIO 핀 OFF 상태로 한다.
{
    printf("process stop₩n");
    digitalWrite (PUMP, 0) ; // Off

    exit(0);
}
```

위와 같이 소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래 명령어로 컴파일을 한다.

```
$gcc -o pumpon pumpon.c -lwiringPi
```

아래 명령어로 실행한다.

```
$sudo ./pumpon
```

워터펌프가 2 초 동작하고, 2 초 정지하는 것을 반복하는 것을 볼 수 있다. 언제나 ctrl-c 로 종료해도 워터펌프는 안전하게 종료할 것이다. 그러므로 소스코드의 pump off code 와 delay(2000) 코드들도 사실 필요가 없게 된다.

(2) Fan

FAN 은 보통 FAN, BLOWER, 환풍기, 송풍기, 냉각팬 등 그 지칭하는 이름이 다양하며,현재 국내에서는 그 이름이 혼동되어 사용중이다. 그러나 FAN 의 용도는 분명히 다르며 용도에 따라 구분 할 수가 있다. 보통 FAN 이라고 부르는 것은 정압이 약 100mmAq 미만의 소형 FAN 을 말한다. 일반적으로 적용되는 곳은 컴퓨터, PANEL 냉각용, SMPS(파워 서플라이 냉각), 사무용 OA 기기, 인터넷장비 RACK, 용유.공압기기용 그리고 모든 PANEL 에 적용할 수 있는 전기 장치로 사용된다.



그림 100 FAN KD1205 PFS1

본 실습에서 팬은 식물농장 내부의 공기 순환용 팬으로 SLEEVE BEARING SYSTEM 구조를 가지고 있다. 12V 전원을 공급하면 회전하므로 구동방법은 단순하다. 본 실습에서 사용되는 팬의 수명은 약 30,000 시간(MTBF 상온 25℃기준)이며 제품 특징은 아래와 같다.

표 4 KD1205PFS1 특징

Size (㎜)	50*50*10mm
정격전압(VDC)	12
전류 (A)	0.1
기동전압(V)	10.8~13.8
속도 (RPM)	5500
풍량(㎥/min)	0.29
풍압(Pa)	36
소음(db)	28
권장사용온도(℃)	10~60
무게(g)	20

팬 사용시는 전류와 회전수와 같은 물리적 특성에 주의해서 사용해야 하고, 바람의 흡 배기 방향에 맞도록 설치하여 사용해야 한다. 아래는 팬의 기구도면을 나타내었다. 바람의 방향과 팬의 사이즈를 나타내었다.

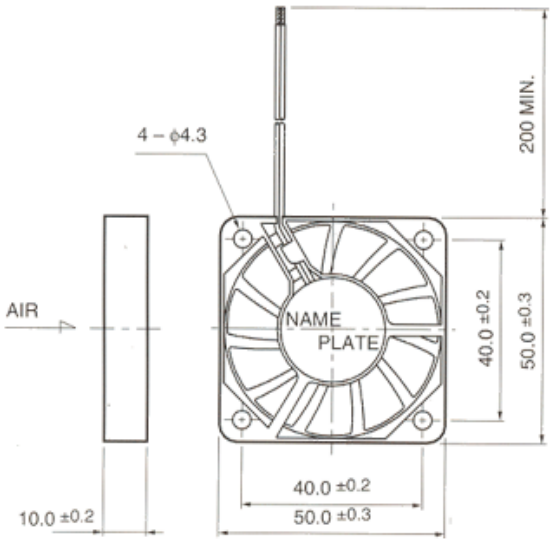


그림 101 팬의 기구도면

아래는 팬의 전원을 제어하는 회로도이며, GPIO 6 번을 사용하여 전원을 제어한다.

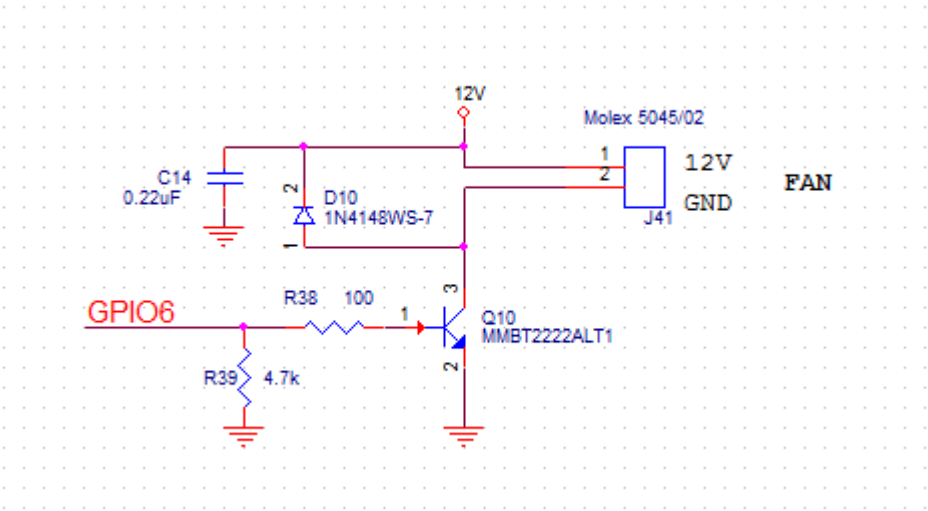


그림 102 팬 구동 회로도

아래는 디레이를 삭제하고 ctrl-c 로 종료할때를 보완한 소스코드이다. 워터펌프를 실습할때와 동일한 코드이다. (fanon.c)

```
#include <signal.h> //Signal 사용 헤더파일
```

```
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define FAN             22 // BCM_GPIO 6

void sig_handler(int signo); // 마지막 종료 함수

int main (void)
{
    signal(SIGINT, (void *)sig_handler);  //시그널 핸들러 함수

    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            return 1 ;
    }

    pinMode (FAN, OUTPUT) ;

    for (;;)
    {
            //printf("here - FAN on₩n");
            digitalWrite (FAN, 1) ; // On
    }
    return 0 ;
}
```

```
void sig_handler(int signo)
{
      printf("process stop₩n");
     digitalWrite (FAN, 0) ; // Off

     exit(0);
}
```

소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래 명령어로 컴파일을 한다.

```
$gcc -o fanon fanon.c -lwiringPi
```

아래 명령어로 실행하여 결과를 확인한다.

```
$sudo ./fanon
```

(3) DC Motor

DC 모터란, 고정자로 영구자석을 사용하고, 회전자(전기자)로 코일을 사용하여 구성한 것으로, 전기자에 흐르는 전류의 방향을 전환함으로써 자력의 반발, 흡인력으로 회전력을 생성시키는 모터이다. 모형 자동차, 무선조정용 장난감 등을 비롯하여 여러 방면에서 가장 널리 사용되고 있는 모터이다.

DC 모터는 다음과 같은 특징이 있다. (1) 기동 토크가 크다. (2) 인가전압에 대하여 회전특성이 직선적으로 비례한다. (3) 입력전류에 대하여 출력 토크가 직선적으로 비례하며, 또한 출력 효율이 양호하다. (4) 가격이 저렴하다.

DC 모터의 가장 큰 결점으로는 그 구조상 브러시(brush)와 정류자(commutator)에 의한 기계식 접점이 있다는 점이다. 이것에 의한 영향은

전류(轉流)시의 전기불꽃(spark), 회전 소음, 수명이라는 형태로 나타난다. 그리고, 마이크로컴퓨터 제어를 하려는 경우는 "노이즈"가 발생하게 된다. 따라서 이 노이즈 대책이 유일한 과제가 될 수 있다. 이 노이즈 대책을 위해서는 각 단자와 케이스 사이에 0.01μF~0.1μF 정도의 세라믹 콘덴서를 직접 부착한다. 이것으로 정류자에서 발생하는 전기불꽃을 흡수하여 노이즈를 억제할 수 있다. 본 실습에 사용되는 DC 모터는 비닐 하우스나 가림막등을 열고, 닫는 용도로 사용한다고 가정한다.

표 23 DC Motor Specification

품명	마이크로 DC 모터
Rated Voltage(V)	12
No load speed(rpm)	9,100
No load current(mA)	60
Rated speed(rpm)	7800
Rated torque(gf.ㅊ)	20
Rated current(mA)	350
Size	23.8Φ x 30.5mm
Noise level(유)	55

DC 모터를 선택할때에는 토크(회전력)와 회전수를 고려하여 사용에 필요한 용량의 1.5 배 정도 되는 모터를 사용해야 한다. 아래는 본 실습에 사용되는 DC 모터의 외관을 나타내었다.



그림 103 DC 모터 외관

아래는 본 실습에 사용되는 DC 모터의 기구 도면을 나타내었다. 기구물에 장착하거나 나사로 고정을 할 때 참고하기 위함이다.

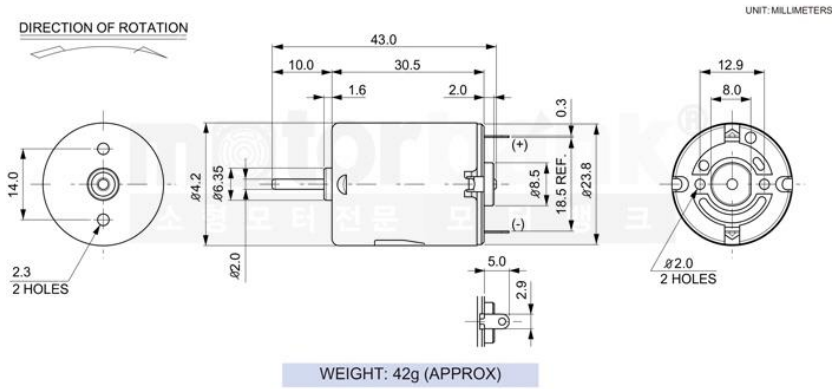


그림 104 DC motor 기구 도면

아래 [그림 35]에 DC 모터에 전원을 공급하는 회로도를 나타냈다. TR 을 이용하여 DC 모터에 공급되는 전원을 On/Off 한다. 보통 TR 을 사용할 경우 TR 의 전력 손실은 그대로 열로 발생되기 때문에 방열판이나 냉각팬등을 사용하는 방식으로 열 대책을 하여야 한다. 모터의 On/Off 제어는 여기서는 단방향이지만 회전의 방향을 바꾸고 싶을 때는 모터에 인가하는 전압의 플러스, 마이너스를 반대로 하면 모터는 역회전하게 된다. 그래서, 단일전원으로 모터에 가하는 전압의 방향을 바꿀 수 있는 회로로 고안된 것이 "H 브리지 회로"라고 한다.

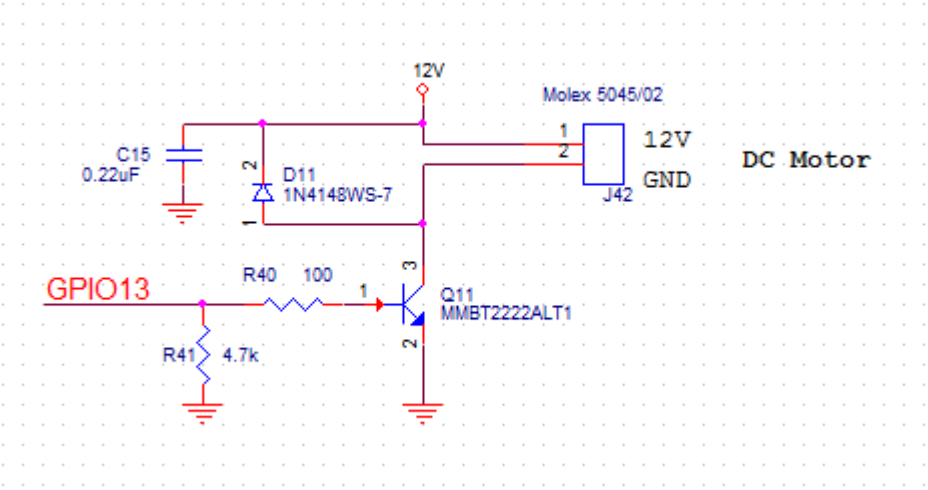


그림 105 DC motor 구동 회로도

아래 소스코드는 DC 모터를 제어하는 소스코드이다. 전원 On-Off 로 제어만 하면 회전 속도도 빠르고 소음이 많이 발생하므로 wiringPi 의 라이브러리를 이용하여 PWM 방식으로 제어하는 코드이다. (motorsoftpwm.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>
#include <softPwm.h>

#define MOTORCONTROL   23            //GPIO 13 - Motor

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
    if(wiringPicheck()) printf("Fail₩n");
```

```
    signal(SIGINT, (void *)sig_handler);

    pinMode (MOTORCONTROL, OUTPUT);

    softPwmCreate(MOTORCONTROL, 0, 100);

    int i;
    int delaytime = 100;
    while(1)
    {
            printf("here ₩n");

            softPwmWrite(MOTORCONTROL, 5);
            delay(delaytime);

    }
  return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            exit(1) ;
    }
}

void Bpluspinmodeset(void)
{
```

```
    pinMode (MOTORCONTROL, OUTPUT);
}


void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
     printf("process stop₩n");
    digitalWrite(MOTORCONTROL, 0);
    exit(0);
}
```

소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래의 명령을 사용하여 컴파일 한다.

```
$ gcc -o motorsoftpwm motorsoftpwm.c -lwiringPi
```

아래의 명령을 사용해 실행하면 낮은 속도로 회전하는 모터를 보게 된다.

```
$ sudo ./motorsoftpwm
```

(4) RGB Led

RGB led 모듈은 IWS-506-RGB-K3 led 3 개를 연결하여 제작되었다. 전원은 DC 12V 를 공급하고, GPIO 3 라인을 연결하여 다양한 색을 출력하도록 한다. RGB Led 가 내는 색상은 Red, Green, Blue 각각 256 가지 색을 내므로 조합하면 1,670 만 가지 색을 낼 수 있다. 여기서는 RGB 색을 시험하기로 한다. 아래는 RGB Led 의 이미지와 회로도를 나타낸다.

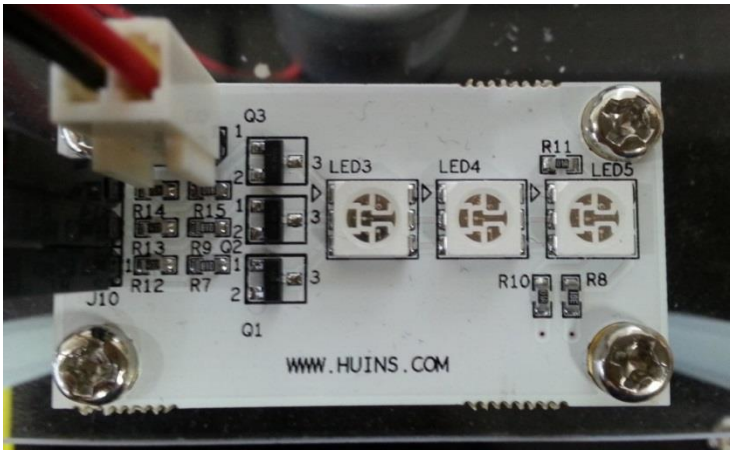


그림 106 RGB Led 모듈 외형

아래 [그림 36]은 GRB Led Moule 회로도를 나타낸다.

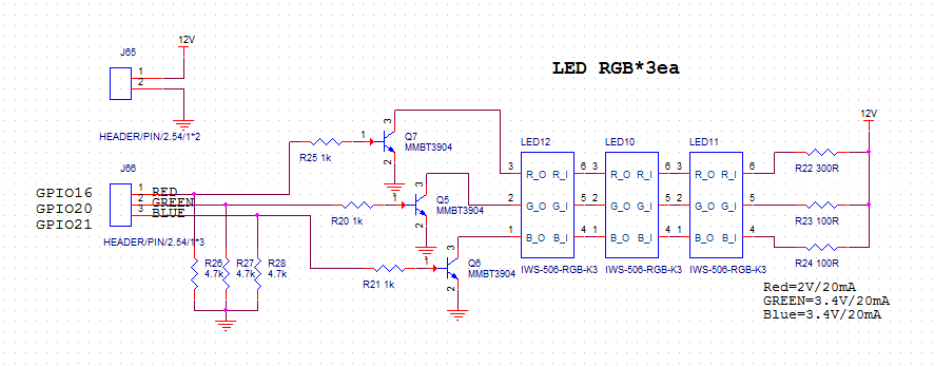


그림 107 RGB Led 모듈 회로도

아래는 [그림 37]은 GRB Led Moule 에 들어가는 전원을 제어하는 회로도이다.

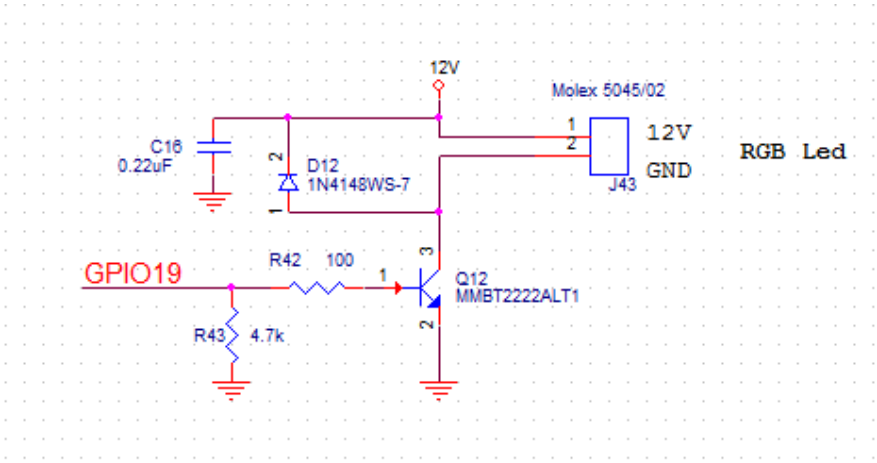


그림 108 RGB Led 모듈 전원 회로

RGB LED 를 테스트 하는 소스코드를 나타낸다. 마찬가지로 ctrl-c 를 눌러 종료할 때 정리하는 코드가 들어가 있다. GPIO 핀 번호를 정의하고, 핀 의 방향을 정해주고 1 초 마다 red, blue, green 색을 교대로 점등하는 간단한 프로그램이다. (rgbtest.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define RGBLEDPOWER   24 //BCM_GPIO 19

#define RED           27
#define GREEN         28
#define BLUE          29

void sig_handler(int signo); // SIGINT 사용 마지막 종료 함수
```

```
int main (void)
{
    signal(SIGINT, (void *)sig_handler);  //시그널 핸들러 함수

    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            return 1 ;
    }

    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE, OUTPUT);

    int i;

    for (i =0; i<10 ;i++)
    {

            digitalWrite(RGBLEDPOWER, 1);

            digitalWrite(RED, 1);
            digitalWrite(BLUE, 0);
            digitalWrite(GREEN, 0);

            delay(1000);

            digitalWrite(RED, 0);
            digitalWrite(BLUE, 1);
```

```
                digitalWrite(GREEN, 0);

                delay(1000);

                digitalWrite(RED, 0);
                digitalWrite(BLUE, 0);
                digitalWrite(GREEN, 1);

                delay(1000);
    }
  return 0 ;
}

void sig_handler(int signo)
{
     printf("process stop₩n");

    digitalWrite(RED, 0);
    digitalWrite(GREEN, 0);
    digitalWrite(BLUE, 0);
    digitalWrite(RGBLEDPOWER, 0); //Off

    exit(0);
}
```

소스코드의 입력을 완료하고 ctrl-x 늘 눌러 저장하려면 y 를 누르고 파일 이름을 그대로 저장하기 위해 enter 를 눌러 빠져 나온다. 아래의 명령을 사용하여 컴파일 한다.

```
$ gcc -o rgbtest rgbtest.c -lwiringPi
```

아래 명령어로 실행한다.

```
$ sudo ./rgbtest
```

RGB 컬러 전체 표시하기 실습

다음은 소프트웨어 PWMdm,f 이용하여 RGB 전체 색을 표시하는 실습을 할것이다. 실제로 1,600 만 가지 이상의 색을 표현하지만 우리가 구별하지 못할 뿐이다.

LED 로 여러가지 색을 나타내려면 기본적으로 LED 의 밝기를 조절해야하는데 LED 밝기 조절은 전압을 변경하여 할 수 있다. 하지만 GPIO 핀은 디지털인데, 즉, 끄고 켜는 두가지 외에는 방법이 없는데 어떻게 LED 의 밝기를 조절 할 수 있을까? 이를 위해서는 PWM 을 이해해야한다. PWM 은 Pulse Width Modulation 의 약자로 펄스 폭 변조라고 한다. PWM 은 디지털 기기에서 아날로그 결과를 얻기 위한 테크닉으로, 전원인 on 되는 시간을 펄스 폭이라고 하고, 펄스 폭을 조정함으로 on 과 off 사이의 전압을 흉내낼 수 있다.

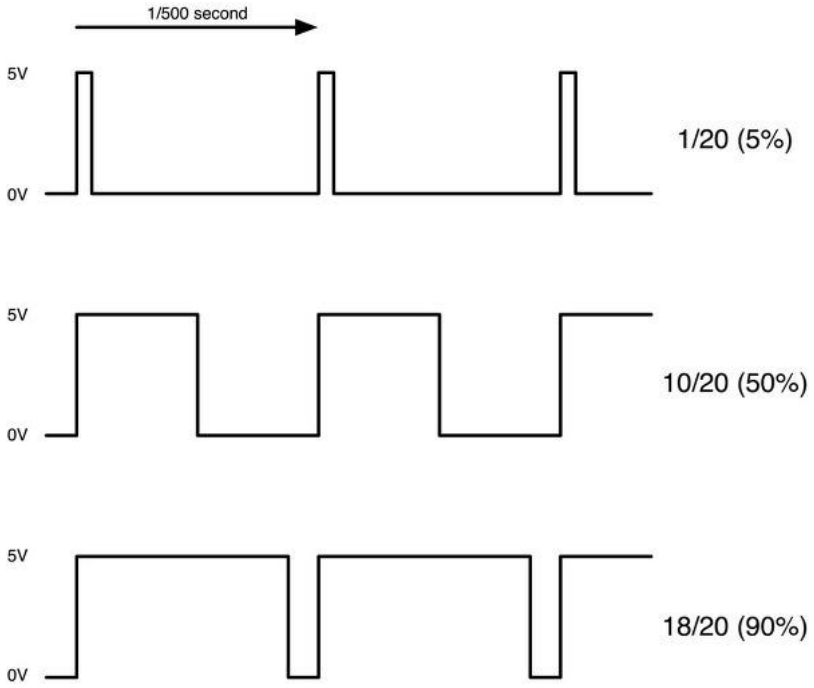


그림 109 PWM 신호 원리

이러한 on-off 패턴을 빠르게 반복하여 LED 에 출력하면 LED 의 밝기를 변경할 수 있게 되는 것이다. 예를 들어 위의 [그림 109]와 같이 on 이 지속 되는

시간, 즉, 펄스 폭을 조정하면, 위에서 아래로 갈 수록 밝은 LED 빛을 얻을 수 있다.

wiringPi 라이브러리에서 PWM 을 지원한다. 다음은 WiringPi 라이브러리의 PWM 함수를 이용해서 LED 의 색깔을 바꿔서 다양한 색을 표현하는 프로그램 코드이다. (allrgbcolor.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>
#include <softPwm.h>

#include <wiringPi.h>

#define RGBLEDPOWER   24   //BCM_GPIO 19

ㅣㄴ
#define RED           29        // BCM_GPIO 16 - OUT
#define GREEN         28        // BCM_GPIO 20 - OUT
#define BLUE          27                  // BCM_GPIO 21 - OUT

void Bpluspinmodeset(void);

void setRGB(int r, int g, int b);
void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
```

```
if(wiringPicheck()) printf("Fail");

Bpluspinmodeset();

signal(SIGINT, (void *)sig_handler);

int i;
digitalWrite(RGBLEDPOWER, 1);

while(1)
{

        int i = 0, j = 0, k=0;
        printf("RGB LED Various Color");

        softPwmCreate(RED, 0, 255);
        softPwmCreate(GREEN, 0, 255);
        softPwmCreate(BLUE, 0, 255);

        for (i = 0; i <= 250; i=i+ 10){
                for (j = 0; j <= 250; j=j+ 10){
                        for (k = 0; k <= 250; k=k+ 10){
                                setRGB(i,j,k);
                                delay(50);
                                printf("R:%d G:%d B:%d₩n", i, j, k);
                        }
                }
        }

        setRGB(0,0,0);//clear LED
        delay(50);
```

```
    }

  return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
                fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
                return 1 ;
    }
}

void Bpluspinmodeset(void)
{
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode (RED, OUTPUT);
    pinMode (GREEN, OUTPUT);
    pinMode (BLUE, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
     printf("process stop₩n");
    digitalWrite(RED, 0);
    digitalWrite(GREEN, 0);
    digitalWrite(BLUE, 0);
    digitalWrite(RGBLEDPOWER, 0); //Off
    exit(0);
```

```
}

void setRGB(int r, int g, int b){
   softPwmWrite(RED, 255-r);
   softPwmWrite(GREEN, 255-g);
   softPwmWrite(BLUE, 255-b);
}
```

소스입력을 다 했으면 ctrl-x 를 누르고 y 를 누르고 enter 를 입력하여 nano 에디터를 빠져 나온다. 위 코드를 조금 살펴보자. 우선 8 번째 줄을 보면 'softPwm.h' 헤더파일을 포함시키는데, 이는 WiringPi 에서 지원하는 헤더파일로 PWM 을 사용하기 위해서는 이 헤더파일이 필요하기 때문이다. 두번째로 36-38 번째 줄을 보면, softPwmCreate 함수를 사용하는데 이 함수는 소프트웨어로 제어되는 PWM 핀을 만든다. 이 함수는 핀번호, 기본값, PWM 값의 범위 이렇게 세가지를 파라미터로 갖는다. 따라서 36 번째 줄의 softPwmCreate(RED, 0, 255) 는 RED 핀을 0 과 255 사이의 범위를 갖고 기본값을 0 으로 하는 PWM 핀을 만든다. 마지막으로 void setRGB(int r, int g, int b) 함수는 LED 에 불을 켜는 함수로 softPwmWrite 를 이용해서 빨간색, 초록색, 파란색 불빛의 밝기를 조절하여 원하는 색깔의 빛을 만든다.

아래 명령어로 컴파일을 한다.

```
$ gcc -o allrgbcolor allrgbcolor.c -lwiringPi
```

아래 명령어로 실행을 시키면 천천이 색이 변하는 것을 볼 수 있다. For 루프의 증가분을 바꿔주면 더욱 빠르게 변하게 된다.

```
pi@raspberrypi ~/smartfarm $ sudo ./allrgbcolor
```

(5) 스마트 농장 통합 액츄에이터 실습

앞서 4 종의 액츄에이터에 대해 알아보았다. 다음 단계는 액츄에이터 4 종을 테스트하는 실습 과정이다. IO Adpater 보드의 J28, J29, J30, J31 에는 각각 워터펌프, 팬, DC 모터, RGB Led 가 연결되어 있다. 워터펌프, 팬, DC 모터는

TR 회로에 인가되는 GPIO 신호를 제어하여 전원을 On/Off 하여 동작시킨다. RGB Led 는 전원을 공급하고 Red, Blue, Green 신호선에 GPIO 신호를 공급하여 원하는 색을 출력한다.

다음 소스코드는 통합 액츄에이터 실습 프로그램으로 4 종의 액츄에이터를 일괄적으로 ON/OFF 한다. **(actutest .c 파일)**

```
#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>

#define PUMP          21        // BCM_GPIO 5
#define FAN           22                  // BCM_GPIO 6
#define DCMOTOR 23          // BCM_GPIO 13
#define RGBLEDPOWER   24 //BCM_GPIO 19

#define RED27
#define GREEN         28
#define BLUE          29

void Bplus_pinmodeset(void);
int testmode(void);

int main (void)
{
    if(wiringPicheck())
    {
                printf("Fail");
                return;
    }
```

```
    Bplus_pinmodeset();

    while(1)
    {
            int exit;
            exit = testmode();
            if(exit == 1) break;
    }
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            return 1 ;
    }
}

void Bplus_pinmodeset(void)
{
    pinMode (PUMP, OUTPUT);
    pinMode (FAN, OUTPUT);
    pinMode (DCMOTOR, OUTPUT);
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);
}

int testmode(void)
```

```
{
    int testnum;

   printf("Input Number for Smart Farm Actuator Test(1 ~ 4) and Enter !!! ₩n");
    printf("1. Water Pump Test₩n");
    printf("2. Fan Test₩n");
    printf("3. DC Motor Test₩n");
    printf("4. RGB Led Test₩n");
    printf("5. Exit₩n");

    scanf("%d", &testnum);

    if(testnum == 1)
    {
            //printf("1 input₩n");
            digitalWrite (PUMP, 1);   //on
            delay (2000) ; // ms
            digitalWrite (PUMP, 0) ; // Off
    }

    if(testnum == 2)
    {
            digitalWrite (FAN, 1) ; // On
            delay (2000) ; // ms
            digitalWrite (FAN, 0) ; // Off
    }

    if(testnum == 3)
    {
            digitalWrite(DCMOTOR, 1); //On
            delay (2000) ; // ms
```

```
                digitalWrite(DCMOTOR, 0); //OFF
        }

        if(testnum == 4)
        {
                digitalWrite(RGBLEDPOWER, 1); // power on
                digitalWrite(RED, 1);
                delay(500);
                digitalWrite(RED, 0);
                digitalWrite(GREEN, 1);
                delay(500);
                digitalWrite(GREEN, 0);
                digitalWrite(BLUE, 1);
                delay(500);
                digitalWrite(BLUE, 0);
                digitalWrite(RGBLEDPOWER, 1); // power off
        }

        if(testnum == 5)
        {
                return 1;
        }
        return 0;

}
```

소스입력을 다 했으면 ctrl-x 를 누르고 y 를 누르고 enter 를 입력하여 nano 에디터를 빠져 나온다. 컴파일은 다음과 같이 수행한다.

```
$gcc -o actutest actutest.c -lwiringPi
```

실행한 결과를 보여주는 화면이다.

```
$sudo ./actutest
```

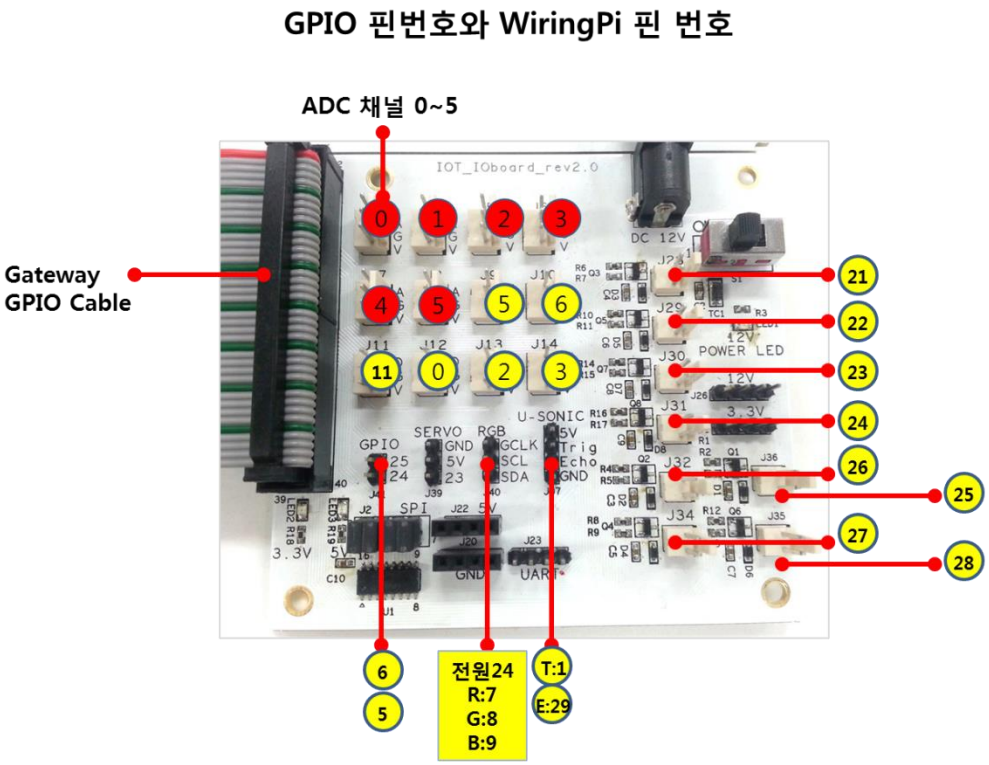
아래는 1 ~ 4 를 눌러 개별적인 액츄에이터를 실행하는 화면이다. 액츄에이터 실습을 종료하려면 숫자 5 를 입력하거나 ctrl-c 를 눌러 빠져나온다.

```
pi@raspberrypi ~/smartfarm $ sudo ./actutest
Input Number for Smart Farm Actuator Test(1 ~ 4) and Enter !!!
1. Water Pump Test
2. Fan Test
3. DC Motor Test
4. RGB Led Test
5. Exit
5
pi@raspberrypi ~/smartfarm $ ^C
```

(6) 다른 액츄에이터 테스트

다른 액츄에이터 - 서보모터, LED Bar, 부저 - 들도 위에서 설명한 액츄에이터 제어방시과 동일하게 테스트를 수행한다. 우리는 실습 편의성과 하드웨어 연결에 시간을 들이지 않기 위해 IO 어댑터 보드를 사용하였다. 중요한것은 액츄에이터 전원제어 신호에 해당되는 GPIO 핀 넘버가 몇 번인지 파악하는 것이다. 그런 상태에서라면 IO 어댑터 보드도 사용하지 않아도 되고, 프로그램도 거의 동일한 코드를 사용하여 시험할 수 있게 된다.

IO adapter 보드와 대응되는 GPIO 핀을 표시한 그림을 참고한다.



우선은 첨부 CD 의 사용자 매뉴얼 폴더에 smart iot-sensor-actu-all.pdf 파일을 참고한다. Smart IoT 시스템의 모든 센서와 액츄에이터를 테스트 할 수 있도록 설명한 문서이다.

10.5 스마트 농장관리 시스템 시나리오 실습

스마트 농장 관리 시스템의 전체 시나리오를 테스트 한다. 앞에서 설명한 전체 구조를 이해하고 센서와 액츄에이터 테스트를 잘 했다면 쉽게 따라 할 수 있을 것이다. 가급적이면 센서와 액츄에이터 연결을 바꾸지 말고 처음 개별적으로 테스트 하던 상태로 시험을 진행한다. 새로 연결을 하게 된다면 아래의 연결도를 참고한다. 극성에 주의하고, 센서 등은 전원 입출력을 바르게 연결해야 한다.

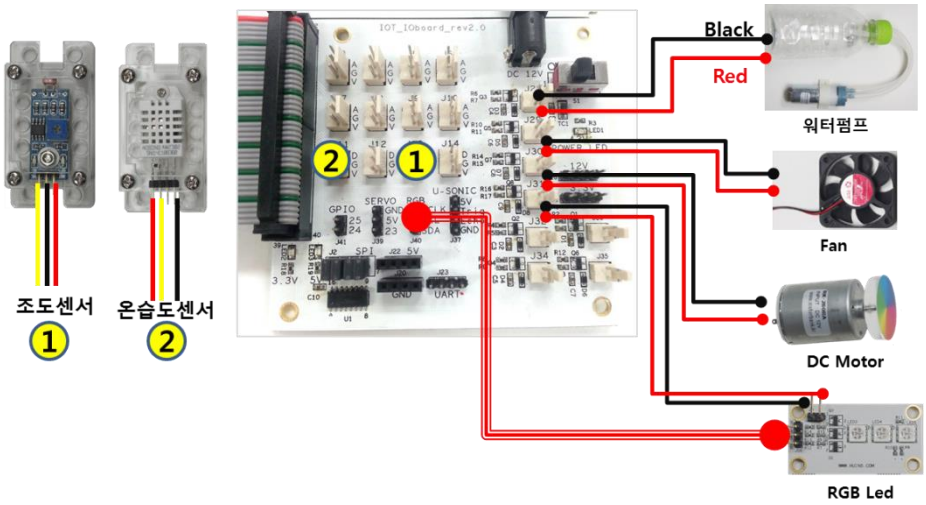


그림 110 센서와 액츄에이터 연결도

기준이 되는 조도값이나 온도, 습도의 값은 현재 시험하는 장소와 환경, 계절적인 영향에 따라 사용자가 센서 기준값을 임의로 정해주어도 상관이 없다. 여기서 중요한 것은 센서 데이터를 획득하여 사용자가 설정한 값보다 높거나 낮을 경우 원하는 액츄에이터를 동작시키는 것이다. 센서값이 기준 값에 미치지 못할 경우 더운 바람을 불어 넣거나 빛을 차단하는 등의 조치를 취해주어서 액츄에이터의 작동을 쉽게 해주는 바ㅣㅇ법을 사용해도 된다.

스마트 농장 관리 시스템의 시험을 간단하게 나마 전체적으로 테스트 할 수 있도록 시나리오를 그리고 프로그램도 이 그림에 따라 진행한다.

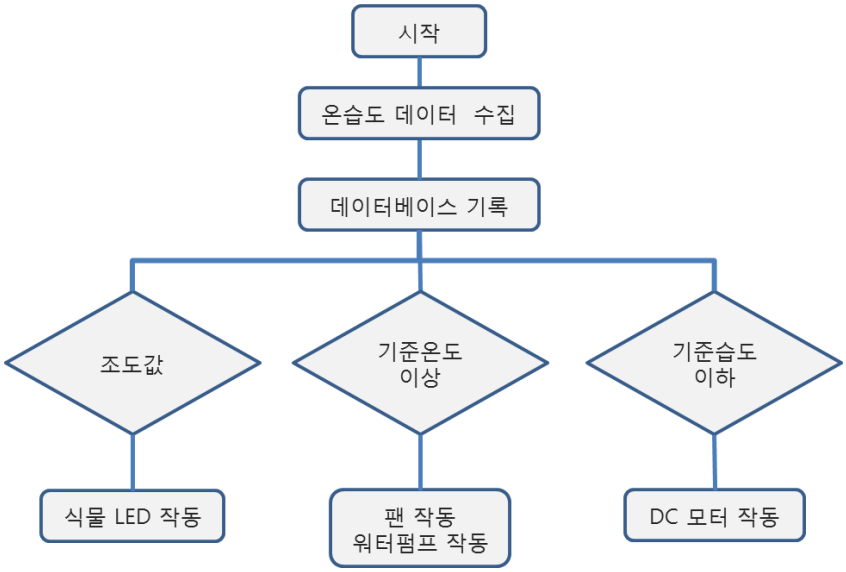


그림 111 스마트 농장 시나리오 테스트

위 시험에 대한 전체 소스를 첨부한다. 편의상 DC 모터는 가동하지 않기로 하고, 식물 Led 는 파랑색으로 On/Off 만을 수행한다. 항상 작업 디렉토리는 /home/pi/smartfarm 에서 수행해야 한다. 작업 디렉토리 확인 방법은 $pwd 명령으로 확인한다.

```
pi@raspberrypi ~/smartfam $ pwd
/home/pi/smartfam
pi@raspberrypi ~/smartfarm$
```

소스코드는 아래와 같이 입력한다. (farmstory.c)

```
$nano farmstory.c


#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
#include <string.h>
#include <errno.h>
```

```
#include <sys/types.h>
#include <unistd.h>
#include <wiringPi.h>

#define PUMP          21        // BCM_GPIO 5
#define FAN           22                  // BCM_GPIO 6
#define DCMOTOR 23            // BCM_GPIO 13
#define RGBLEDPOWER   24 //BCM_GPIO 19

#define RED27                 //GPIO 16
#define GREEN         28      //GPIO 20
#define BLUE          29      //GPIO 21

#define LIGHTSEN_OUT 2   //gpio27 - J13 connect

#define MAXTIMINGS 85

void Bpluspinmodeset(void);

int ret_humid, ret_temp;

static int DHTPIN = 11;
static int dht22_dat[5] = {0,0,0,0,0};

int ret_humid, ret_temp;

int main (void)
{
    if(wiringPicheck()) printf("Fail");

    Bpluspinmodeset();
```

```
int curtemp = 0, curhumid = 0;

while(1)
{

        if(get_light() == 0)
        {
                act_rgbled_off();
                printf("light ! ₩n");
        }
        else
        {
                act_rgbled_on();
                printf("dark ! ₩n");
        }

        curtemp = get_temperature_sensor();
        printf("%d ₩n", curtemp);

        if(curtemp > 26)digitalWrite (PUMP, 1) ; // On
        {

                act_waterpump_on();
                delay(2000);
                act_waterpump_off();
        }

        curhumid = get_humidity_sensor();
        if(curhumid < 50)
        {
```

```
                act_fan_on();
                delay(2000);
                act_fan_off();
            }
            delay(10000);
    }
    return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            return 1 ;
    }
}

void Bpluspinmodeset(void)
{
    pinMode(LIGHTSEN_OUT, INPUT);

    pinMode (PUMP, OUTPUT);
    pinMode (FAN, OUTPUT);
    pinMode (DCMOTOR, OUTPUT);
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);
}
```

```
int get_light()
{
    // sets up the wiringPi library
    if (wiringPiSetup () < 0)
    {
            fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
            return 1;
    }

    if(digitalRead(LIGHTSEN_OUT))       //day
            return 1;
    else //night
            return 0;


}

int get_temperature_sensor()
{
    int received_temp;

    DHTPIN = 11;

    if (wiringPiSetup() == -1)
            exit(EXIT_FAILURE) ;

    if (setuid(getuid()) < 0)
    {
            perror("Dropping privileges failed₩n");
            exit(EXIT_FAILURE);
```

```
    }
    while (read_dht22_dat_temp() == 0)
    {
            delay(500); // wait 1sec to refresh
    }
    received_temp = ret_temp ;
    printf("Temperature = %d₩n", received_temp);

    return received_temp;
}

static uint8_t sizecvt(const int read)
{
  /* digitalRead() and friends from wiringpi are defined as returning a value
  < 256. However, they are returned as int() types. This is a safety function */

  if (read > 255 || read < 0)
  {
    printf("Invalid data from wiringPi library₩n");
    exit(EXIT_FAILURE);
  }
  return (uint8_t)read;
}

int read_dht22_dat_temp()
{
  uint8_t laststate = HIGH;
  uint8_t counter = 0;
  uint8_t j = 0, i;
```

```
    dht22_dat[0] = dht22_dat[1] = dht22_dat[2] = dht22_dat[3] = dht22_dat[4] =
0;

    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepare to read the pin
    pinMode(DHTPIN, INPUT);

    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++) {
      counter = 0;
      while (sizecvt(digitalRead(DHTPIN)) == laststate) {
        counter++;
        delayMicroseconds(1);
        if (counter == 255) {
          break;
        }
      }
      laststate = sizecvt(digitalRead(DHTPIN));

      if (counter == 255) break;

      // ignore first 3 transitions
      if ((i >= 4) && (i%2 == 0)) {
```

```
        // shove each bit into the storage bytes
        dht22_dat[j/8] <<= 1;
        if (counter > 16)
          dht22_dat[j/8] |= 1;
        j++;
      }
    }

    // check we read 40 bits (8bit x 5 ) + verify checksum in the last byte
    // print it out if data is good
    if ((j >= 40) &&
        (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
          float t, h;

          h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
          h /= 10;
          t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
          t /= 10.0;
          if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

              ret_humid = (int)h;
              ret_temp = (int)t;
              printf("Humidity = %.2f %% Temperature = %.2f *C ₩n", h, t );
              printf("Humidity = %d Temperature = %d₩n", ret_humid, ret_temp);

      return ret_temp;
    }
    else
    {
      printf("Data not good, skip₩n");
```

```
      return 0;
   }
}



int get_humidity_sensor()
{
    int received_humid;

    DHTPIN = 11;

    if (wiringPiSetup() == -1)
                exit(EXIT_FAILURE) ;

    if (setuid(getuid()) < 0)
    {
                perror("Dropping privileges failed₩n");
                exit(EXIT_FAILURE);
    }
    while (read_dht22_dat_humid() == 0)
    {
                delay(500); // wait 1sec to refresh
    }
    received_humid = ret_humid;
    printf("Humidity = %d₩n", received_humid);

    return received_humid;
}

int read_dht22_dat_humid()
{
```

```
	uint8_t laststate = HIGH;
	uint8_t counter = 0;
	uint8_t j = 0, i;

	dht22_dat[0] = dht22_dat[1] = dht22_dat[2] = dht22_dat[3] = dht22_dat[4]
= 0;

	// pull pin down for 18 milliseconds
	pinMode(DHTPIN, OUTPUT);
	digitalWrite(DHTPIN, HIGH);
	delay(10);
	digitalWrite(DHTPIN, LOW);
	delay(18);
	// then pull it up for 40 microseconds
	digitalWrite(DHTPIN, HIGH);
	delayMicroseconds(40);
	// prepare to read the pin
	pinMode(DHTPIN, INPUT);

	// detect change and read data
	for ( i=0; i< MAXTIMINGS; i++ )
	{
		counter = 0;
		while (sizecvt(digitalRead(DHTPIN)) == laststate)
		{
			counter++;
			delayMicroseconds(1);
			if (counter == 255) {
			break;
		}
	 }
```

```
        laststate = sizecvt(digitalRead(DHTPIN));

        if (counter == 255) break;

        // ignore first 3 transitions
        if ((i >= 4) && (i%2 == 0))
       {
                // shove each bit into the storage bytes
                dht22_dat[j/8] <<= 1;
                if (counter > 16)
                        dht22_dat[j/8] |= 1;
                j++;
        }
   }

    // check we read 40 bits (8bit x 5 ) + verify checksum in the last byte
    // print it out if data is good
     if ((j >= 40) &&
        (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
            float t, h;

            h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
            h /= 10;
            t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
            t /= 10.0;
            if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

                ret_humid = (int)h;
                ret_temp = (int)t;
                printf("Humidity = %.2f %% Temperature = %.2f *C ₩n", h, t );
```

```
                printf("Humidity = %d Temperature = %d₩n", ret_humid, ret_temp);

     return ret_humid;
    }
    else
    {
                printf("Data not good, skip₩n");
                return 0;
    }
}

void act_waterpump_on()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (PUMP, OUTPUT);
    digitalWrite (PUMP, 1) ; // On

}
void act_waterpump_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (PUMP, OUTPUT);
    digitalWrite (PUMP, 0) ; // On

}
void act_fan_on()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (FAN, OUTPUT);
    digitalWrite (FAN, 1) ; // On
```

```
}
void act_fan_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (FAN, OUTPUT);
    digitalWrite (FAN, 0) ; // On

}
void act_dcmotor_on()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (DCMOTOR, OUTPUT);
    digitalWrite(DCMOTOR, 1); //On
}

void act_dcmotor_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (DCMOTOR, OUTPUT);
    digitalWrite(DCMOTOR, 0); //On
}

void act_rgbled_on()
{
    if(wiringPicheck()) printf("Fail");
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);

    digitalWrite(RGBLEDPOWER, 1); //PWR on
```

```
    digitalWrite(BLUE, 1); //PWR on
}
void act_rgbled_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);

    digitalWrite(RGBLEDPOWER, 0); //PWR on
    digitalWrite(BLUE, 0); //PWR on
}
```

아래 명령으로 컴파일을 한다.

```
$gcc -o farmstory farmstory.c -lwiringPi
```

에러 없이 컴파일이 잘 되었다면 실행을 한다.

```
$sudo ./farmstory
```

조도센서의 출력과 dark, light 메시지와 현재 온도와 습도를 5 분 간격으로 디스플레이 한다. 조도센서에서 감지한 빛이 Dark 일경우 식물농장의 Led 를 파란색으로 On 하고 밝아지면 Off 하는 동작을 볼 수 있다.

```
if(get_light() == 0)
{
    act_rgbled_off();
    printf("light ! ₩n");
}
Else
```

```
{

    act_rgbled_on();
    printf("dark ! ₩n");
}
```

또한 기준 온도(현재 26 도)로 설정한 온도값이 넘어가는 경우 워터 펌프가 2 초간 작동을 하고

```
if(curtemp > 26)digitalWrite (PUMP, 1) ; // On
    {

            act_waterpump_on();
            delay(2000);
            act_waterpump_off();
    }

    curhumid = get_humidity_sensor();
```

현재 습도가 50% 이하인 경우 마찬가지로 팬이 2 초간 작동함을 볼 수 있다.

```
            if(curhumid < 50)
            {

                    act_fan_on();
                    delay(2000);
                    act_fan_off();
            }
```

위의 조건에 따라 액츄에이터의 제어는 라즈베리 파이가 동작하는 동안 계속 된다. 여기에서는 워터 펌프의 동작시간, 식물 Led 동작시간을 적은 시간동안

동작시켜 결과를 확인하였다. 사용자는 센서값의 기준을 얼마든지 줄이거나 확장할 수 있고, 액츄에이터의 제어 또한 사용자의 설계에 따라 바꿀 수 있다. 각 사용자가 원하는 의도대로 제어하고 동작하는 시험을 수행하기 바란다.

10.7 웹(브라우져)에서 데이터베이스 모니터링

스마트폰이 광범위하게 대중화되면서 우리가 데스크 탑(PC)에서 보아왔던 웹은 새로운 형태로 변하기 시작했다. 대부분의 서비스와 회사들은 무선 모바일 서비스 영역으로 확대하며 모바일 개발센터를 구축하고 모바일 웹과 앱을 개발하는 데 집중하고 있다. 주위 환경과 사용자의 변화가 불러 온 모바일 웹과 앱. 과연 어떻게 다른지 알아보자. 모바일 웹과 앱 구축에 있어 구축 절차와 장단점, 그리고 구현 방법과 특징은 분명 차이가 있을 것이다.

10.7.1 모바일 웹과 모바일 앱의 구분

모바일 웹과 애플리케이션(이하 앱)은 모바일 환경인 스마트폰과 휴대용 단말 확산에 따라 해당 단말기의 해상도나 기기적 특성(CPU, 메모리), 서비스 네트워크 대역폭(3G, Wi-Fi), 모바일 브라우저의 특징을 고려해 최적화된 서비스 방식을 구현하는 것을 말한다. 최근 스마트폰 확산으로 모바일 웹과 앱 개발에 대한 기업의 니즈가 매우 증가하고 있고, 이에 따라 웹과 앱 구축 시 어떤 것이 더 효과적인지에 대한 고민 역시 깊어지고 있다.

10.7.2. 모바일 웹과 앱 차이점

모바일 웹과 앱은 구현, 설치, 배포, 동작 방식등에 따른 차이가 있다(아래 표 24 를 참조한다). 그러나, 잦은 변경이 필요한 기능의 경우, 모바일 웹과 앱을 혼합한 웹/앱 형식인 하이브리드(Hybride)방식을 활용하면 효과적이다. 이 방식은 웹으로 개발해 유지하고 운영할 수 있는 장점을 갖고 있다.

표 24 모바일 웹과 앱의 차이점

비교항목	모바일 웹	앱
구현방식	Script 언어, HTML을 이용해 특정 서버내에 구현	전용개발툴(Visual Studio, Eclipse, Android Studio)을 이용해 구현
설치방식	설치단계없음(웹브라우져 접속)	앱스토어를 통해 내려받아 설치(apk 파일)
배포방식	설치단계없음(웹브라우져 접속)	앱스토어 변경, 배포 방식에 따름.
동작방식	다양한 웹브라우저에서 기능수행, 부라우저 마다 동작방식 다름	모바일 단멀에 종속되어 동작(버전에 따른 기능 제약)

다음은 구축 절차에 따른 차이점을 살펴보자. 모바일 웹 구축 절차는 PC 웹과 동일하나 호환 단말 및 브라우저 범위 지정에 있어서 PC 에서 볼 수 있는 웹에 비해 엄격한 기준이 적용된다. 모바일 앱은 국내외의 다수 앱스토어 중 등록 범위 앱스토어를 지정하는 것이 필요하다. 각 단계에 따른 모바일 웹과 앱의 구축 절차는 다음과 같다.(아래 표 25 를 참조)

표 25 모바일 웹과 앱의 구축절차 비교

비교항목	모바일 웹	앱
호환 단말범위	제조사별 브라우저, 해상도, 크기에 따라 별도 지원	제조사별 브라우저, 해상도, 크기에 따라 별도 지원
호환 브라우저	웹표준, 신기술에 따라 적용 범위 상이	안드로이드, IOS, 윈도우 모바일에 따라 범위 지정 상이
기획	PC보다 좁은 화면공간에서 효과적인 네비게이션 설계가 중요	OS별 인터페이스 UI 특징, 동작 방식 인지
디자인	PC보다 느린 네트워크 속도에 따라 고품질 이미지 사용 제약 고려	OS별 인터페이스 UI 특징, 동작 방식 인지
개발	웹표준 엄격한 준수, 다수의 브라	적용 OS 개수 별 개발 진

	우저 호환 개발에 따라 개발 기간 및 스크립트 제약을 고려한 설계, 기간 산정 필요	행, OS 지원이 5개일 경우 5개의 앱을 제작해야 함
테스트	초기에 정의한 단말 및 브라우저별 엄격한 테스트 진행	적용하기로 한 단말기 별 테스트 및 동일 단말 내의 OS버전에 따른 호환성 테스트 진행
배포	사용자의 단말기(모바일 단말, PC 단말) 판단에 따라 모바일 웹과 PC 웹으로의 자동 이동	제작된 OS별 바이너리를 개발하여 앱스토어의 등록 절차에 따라 진행 앱스토어 마다 검수 절차 및 항목이 다르기 때문에 사전에 항목에 대한 자체 검수 후 등록을 해야 함

10.7.3. 모바일 웹과 앱의 장단점

모바일 웹

모바일 웹은 하나의 소스로 다양하게 활용할 수 있는 'One Source Multi Use'가 가능한 것이 가장 큰 장점이다. 또한, 배포 단계 없이 유지하고 운영할 수 있어 효과적이다. 하지만, 페이지 단위 로딩에 따라서 속도가 느려지고, 효과적인 인터페이스 개발이 어려운 단점이 있다. 온라인 환경에서만 이용할 수 있다는 제한점도 있다.

모바일 앱

모바일 앱은 빠른 반응 속도와 화려한 인터페이스를 가장 큰 장점으로 꼽을 수 있다. 또한 오프라인 상에서도 이용 가능한 특징을 갖고 있다. 하지만 장점을 수행하고 사용자에게 편리성을 제공하기 위해서는 OS 버전에 따른 패치가 필요하고, OS 종류에 맞춰 개별적으로 개발해야 하는 단점도 있다. 이를 위해 개발 전문인력을 양성하는 것이 앞으로의 과제다.

먼저 모바일 웹과 앱의 발전단계를 살펴보면, 초기에는 모바일 앱이 주목을 받으면서 앱스토어를 통한 수익 창출, 신규 비즈니스 영역 창출로 대중화된 것이 사실이다. 지금은 모바일 앱의 단점이 부각되면서 동시에 모바일 웹의 편리성이 대두, 모바일 웹에 대한 니즈가 증가하고 있다. 하지만, 앞으로는 모바일 웹과 앱의 특성을 결합한 모바일 웹/앱 형태인 하이브리드 방식과 모바일 단말의 다양화 및 산업군별 특징에 따라 모바일 웹과 앱의 선택적 사용이 증가할 것이다.

다음으로는 모바일이 발전하면서 각 산업에 미치는 다음과 같은 변화를 기대해 볼 수 있다. PC 산업에는 업무용 PC 와 가정용 단말기가 구분될 것이다. 즉, 기존 PC 는 업무용으로만 사용하게 되고, 가정에서는 미니 단말기(스마트폰, 아이패드 등)를 사용할 것이다.

서비스 산업에서는 모바일 단말기가 서비스 산업에 응용되면서 종이없는 서비스를 실현할 수 있다. 예를 들면, 레스토랑에서 주문할 때도 아이패드를 활용한 메뉴판을 통해 자동 주문 시스템으로 주문할 수 있고, 자동 결제 시스템도 대중화가 이뤄 질 것으로 보인다.

인터넷 산업은 웹 서비스와 온·오프라인 마케팅 산업 간의 경계가 모호해질 것이다. 지금까지는 기업이 웹 사이트를 구축하고 광고집행을 위해 웹 에이전시와 광고대행사를 모두 활용해야 했지만, 모바일 광고는 광고 툴을 사용해 개발단계에서 직접 제작 가능하기 때문에 별도 광고대행사를 통해 진행할 필요가 없다. 이러한 변화는 사업 간의 컨버전스로 인해 앞으로 새로운 비즈니스 영역을 생성할 것으로 전망된다.

10.7.4 실습

앞에서 데이터베이스 실습을 진행하였지만 여기서도 다시 데이터 베이스 실습을 한 번 더 진행한다. 데이터 베이스는 중요하고 숙달되도록 연습하는 것은 매우 유용하다.

우선 mysql 데이터 베이스에 로그인을 한다. 설치할때 사용한 아이디와 password 를 입력한다.

```
pi@raspberrypi ~ $ mysql -u root -p
Enter password: root    ---  보이지 않는다.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 52
Server version: 5.5.41-0+wheezy1 (Debian)

Copyright (c) 2000, 2014, Oracle and/or its affiliates. All rights
reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input
statement.
```

데이터 베이스를 확인한다.

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| iotfarm            |
| iotfire            |
| iotintruder        |
| iotpet             |
| iotsafe            |
| iotsmarthome       |
| iottoy             |
| mysql              |
| performance_schema |
| phpmyadmin         |
+--------------------+
11 rows in set (0.01 sec)
```

데모에 사용할 데이터 베이스를 demofarmdb 라는 이름으로 생성한다.

```
mysql> create database demofarmdb;
Query OK, 1 row affected (0.00 sec)
```

demofarmdb 데이터 베이스가 생성된것을 확인한다.

```
mysql> show databases;
```

```
12 rows in set (0.00 sec)
```

demofarmdb 를 사용하겠다는 명령을 사용해 사용 DB 를 설정한다.

```
mysql> use demofarmdb;
Database changed
mysql> show tables;
Empty set (0.00 sec)
```

테이블을 생성하는데 여기서는 시간, 온도, 습도, 조도데이터를 저장하는 테이블을 생성한다.

```
mysql> create table temphumilight( time char(19) primary key,
temperature int, humidity int, lightsensor int);
Query OK, 0 rows affected (0.03 sec)
```

테이블과 테이블에 생성된 항목을 확인한다.

```
mysql> show tables;
+----------------------+
| Tables_in_demofarmdb |
+----------------------+
| temphumilight        |
+----------------------+
1 row in set (0.00 sec)

mysql>


mysql> explain temphumilight;
+-------------+----------+------+-----+---------+-------+
| Field       | Type     | Null | Key | Default | Extra |
+-------------+----------+------+-----+---------+-------+
| time        | char(19) | NO   | PRI | NULL    |       |
| temperature | int(11)  | YES  |     | NULL    |       |
| humidity    | int(11)  | YES  |     | NULL    |       |
| lightsensor | int(11)  | YES  |     | NULL    |       |
+-------------+----------+------+-----+---------+-------+
4 rows in set (0.01 sec)
```

실습 진행시 테이블 이름이 너무 길어 불편하면 아래 명령을 이용해서 테이블 이름을 바꾼고 나서 확인한다.

```
mysql> rename table temphumilight to thl;
mysql> show tables; 확인
```

테이블에 수동으로 데이터를 입력하고 확인한다.

```
mysql> insert into thl values(now(), 1, 2, 3);
Query OK, 1 row affected (0.01 sec)

mysql> select * from thl;
+---------------------+-------------+----------+-------------+
| time                | temperature | humidity | lightsensor |
+---------------------+-------------+----------+-------------+
| 2015-07-02 17:51:05 |           1 |        2 |           3 |
+---------------------+-------------+----------+-------------+
1 row in set (0.01 sec)
```

데이터베이스의 테이블 내용을 지우고 확인한다..

```
mysql> delete from thl where time < now();
Query OK, 1 row affected (0.02 sec)

mysql> select * from thl;
Empty set (0.00 sec)
```

모든 데이터베이스와 테이블을 사용자가 접근할 수 있도록 변경하고, 설정을 반영하라는 명령어를 실행한다.

```
mysql> grant all privileges on *.* to root@localhost identified by
'root';
Query OK, 0 rows affected (0.00 sec)

mysql> flush privileges;
Query OK, 0 rows affected (0.00 sec)
```

아래 프로그램을 작성하여 데이터 베이스에
/home/pi 로 되어있는 Root 에서 폴더를 생성하고 이동한다.

```
$mkdir demofarmdb
$cd demofarmdb
```

nano 편집기를 이용하여 demodb.cpp 파일을 편집한다.

```
#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>
```

```
#include <wiringPiSPI.h>

#include <mysql/mysql.h>

#include <stdlib.h>
#include <stdint.h>
#include <sys/types.h>
#include <unistd.h>

#include <time.h>
#include <math.h>

#define CS_MCP3208  8       // BCM_GPIO_8

#define SPI_CHANNEL 0
#define SPI_SPEED   1000000   // 1MHz

#define VCC         4.8       // Supply Voltage

// define farm
#define MAXTIMINGS 85
#define RETRY 5

#define PUMP       21 // BCM_GPIO 5
#define FAN        22 // BCM_GPIO 6
#define DCMOTOR 23 // BCM_GPIO 13
#define RGBLEDPOWER  24 //BCM_GPIO 19
#define RED        27
#define GREEN      28
#define BLUE       29

#define LIGHTSEN_OUT 2  //gpio27 - J13 connect
int ret_humid, ret_temp;

static int DHTPIN = 11;
static int dht22_dat[5] = {0,0,0,0,0};

int read_dht22_dat_temp();
int read_dht22_dat_humid();
int get_temperature_sensor();
int get_humidity_sensor();
int get_light_sensor();

// here

#define DBHOST "192.168.1.239"
#define DBUSER "root"
#define DBPASS "root"
#define DBNAME "demofarmdb"

MYSQL *connector;
```

```
  MYSQL_RES *result;
  MYSQL_ROW row;

  int read_mcp3208_adc(unsigned char adcChannel)
  {
    unsigned char buff[3];
    int adcValue = 0;

    buff[0] = 0x06 | ((adcChannel & 0x07) >> 2);
    buff[1] = ((adcChannel & 0x07) << 6);
    buff[2] = 0x00;

    digitalWrite(CS_MCP3208, 0);  // Low : CS Active

    wiringPiSPIDataRW(SPI_CHANNEL, buff, 3);

    buff[1] = 0x0F & buff[1];
    adcValue = ( buff[1] << 8) | buff[2];

    digitalWrite(CS_MCP3208, 1);  // High : CS Inactive

    return adcValue;
  }


  int main (void)
  {
    int adcChannel  = 0;
    int adcValue[8] = {0};

    if(wiringPiSetupGpio() == -1)
    {
      fprintf    (stdout,    "Unable    to    start    wiringPi:    %s\n",
strerror(errno));
      return 1 ;
    }

    if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
    {
      fprintf (stdout, "wiringPiSPISetup Failed: %s\n", strerror(errno));
      return 1 ;
    }

    pinMode(CS_MCP3208, OUTPUT);

    // MySQL connection
    connector = mysql_init(NULL);
    if  (!mysql_real_connect(connector,  DBHOST,  DBUSER,  DBPASS,  DBNAME,
3306, NULL, 0))
    {
      fprintf(stderr, "%s\n", mysql_error(connector));
```

```
    return 0;
  }

  printf("MySQL(rpidb) opened.\n");

  while(1)
  {
    char query[1024];

    adcValue[0] = get_temperature_sensor(); // Temperature Sensor

    adcValue[1] = get_humidity_sensor(); // Humidity Sensor

    adcValue[2] = get_light_sensor(); // Illuminance Sensor

    adcValue[3] = read_mcp3208_adc(3); // Mic Sensor
    adcValue[4] = read_mcp3208_adc(4); // Flame Sensor
    adcValue[5] = read_mcp3208_adc(5); // Acceleration Sensor (Z-Axis)
    adcValue[6] = read_mcp3208_adc(6); // Gas Sensor
    adcValue[7] = read_mcp3208_adc(7); // Distace Sensor
    adcValue[7] = 27*pow((double)(adcValue[7]*VCC/4095), -1.10);

    sprintf(query,"insert    into    thl    values    (now(),%d,%d,%d)",
adcValue[0],adcValue[1],adcValue[2]);

    if(mysql_query(connector, query))
    {
      fprintf(stderr, "%s\n", mysql_error(connector));
      printf("Write DB error\n");
    }

    delay(1000); //1sec delay
  }

  mysql_close(connector);

  return 0;
}

int get_temperature_sensor()
{
   int received_temp;
   int _retry = RETRY;

   DHTPIN = 11;

   if (wiringPiSetup() == -1)
          exit(EXIT_FAILURE) ;

   if (setuid(getuid()) < 0)
   {
```

```
            perror("Dropping privileges failed\n");
            exit(EXIT_FAILURE);
    }
    while (read_dht22_dat_temp() == 0 && _retry--)
    {
            delay(500); // wait 1sec to refresh
    }
    received_temp = ret_temp ;
    printf("Temperature = %d\n", received_temp);

    return received_temp;
  }

  static uint8_t sizecvt(const int read)
  {
    /* digitalRead() and friends from wiringpi are defined as returning
a value
    < 256. However, they are returned as int() types. This is a safety
function */

    if (read > 255 || read < 0)
    {
      printf("Invalid data from wiringPi library\n");
      exit(EXIT_FAILURE);
    }
    return (uint8_t)read;
  }

  int read_dht22_dat_temp()
  {
    uint8_t laststate = HIGH;
    uint8_t counter = 0;
    uint8_t j = 0, i;

    dht22_dat[0]  =  dht22_dat[1]  =  dht22_dat[2]  =  dht22_dat[3]  =
dht22_dat[4] = 0;

    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepar

    pinMode(DHTPIN, INPUT);

    // detect change and read data
```

```
    for ( i=0; i< MAXTIMINGS; i++) {
      counter = 0;
      while (sizecvt(digitalRead(DHTPIN)) == laststate) {
        counter++;
        delayMicroseconds(1);
        if (counter == 255) {
          break;
        }
      }
      laststate = sizecvt(digitalRead(DHTPIN));

      if (counter == 255) break;

      // ignore first 3 transitions
      if ((i >= 4) && (i%2 == 0)) {
        // shove each bit into the storage bytes
        dht22_dat[j/8] <<= 1;
        if (counter > 16)
          dht22_dat[j/8] |= 1;
        j++;
      }
    }

    // check we read 40 bits (8bit x 5 ) + verify checksum in the last
byte
    // print it out if data is good
    if ((j >= 40) &&
        (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
          float t, h;

          h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
          h /= 10;
          t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
          t /= 10.0;
          if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

              ret_humid = (int)h;
              ret_temp = (int)t;
              printf("Humidity = %.2f %% Temperature = %.2f *C \n", h,
t );
              printf("Humidity  =  %d  Temperature  =  %d\n",  ret_humid,
ret_temp);

      return ret_temp;
    }
    else
    {
      printf("Data not good, skip\n");
      return 0;
    }
```

```
  }


  int get_humidity_sensor()
  {
     int received_humid;
     int _retry = RETRY;

     DHTPIN = 11;

     if (wiringPiSetup() == -1)
             exit(EXIT_FAILURE) ;

     if (setuid(getuid()) < 0)
     {
             perror("Dropping privileges failed\n");
             exit(EXIT_FAILURE);
     }

     while (read_dht22_dat_humid() == 0 && _retry--)
     {
             delay(500); // wait 1sec to refresh
     }

     received_humid = ret_humid;
     printf("Humidity = %d\n", received_humid);

     return received_humid;
  }

  int read_dht22_dat_humid()
  {
     uint8_t laststate = HIGH;
     uint8_t counter = 0;
     uint8_t j = 0, i;

     dht22_dat[0]  =  dht22_dat[1]  =  dht22_dat[2]  =  dht22_dat[3]  =
dht22_dat[4] = 0;

     // pull pin down for 18 milliseconds
     pinMode(DHTPIN, OUTPUT);
     digitalWrite(DHTPIN, HIGH);
     delay(10);
     digitalWrite(DHTPIN, LOW);
     delay(18);
     // then pull it up for 40 microseconds
     digitalWrite(DHTPIN, HIGH);
     delayMicroseconds(40);
     // prepare to read the pin
     pinMode(DHTPIN, INPUT);
```

```
    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++)
    {
            counter = 0;
            while (sizecvt(digitalRead(DHTPIN)) == laststate)
            {
                    counter++;
                    delayMicroseconds(1);
                    if (counter == 255) {
                    break;
            }
     }
     laststate = sizecvt(digitalRead(DHTPIN));

     if (counter == 255) break;

     // ignore first 3 transitions
     if ((i >= 4) && (i%2 == 0))
    {
            // shove each bit into the storage bytes
            dht22_dat[j/8] <<= 1;
            if (counter > 16)
                    dht22_dat[j/8] |= 1;
            j++;
     }
  }

   // check we read 40 bits (8bit x 5 ) + verify checksum in the last
byte
   // print it out if data is good
    if ((j >= 40) &&
       (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
         float t, h;

         h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
         h /= 10;
         t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
         t /= 10.0;
         if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

             ret_humid = (int)h;
             ret_temp = (int)t;
             printf("Humidity = %.2f %% Temperature = %.2f *C \n", h,
t );
             printf("Humidity  =  %d  Temperature  =  %d\n",  ret_humid,
ret_temp);

     return ret_humid;
    }
    else
```

```
    {
            printf("Data not good, skip\n");
            return 0;
    }
  }

  int wiringPicheck(void)
  {
    if (wiringPiSetup () == -1)
    {
            fprintf(stdout,   "Unable   to   start   wiringPi:   %s\n",
strerror(errno));
            return 1 ;
    }
  }

  int get_light_sensor(void)
  {
    // sets up the wiringPi library
    if (wiringPiSetup () < 0)
    {
            fprintf (stderr, "Unable to setup wiringPi: %s\n", strerror
(errno));
            return 1;
    }

    if(digitalRead(LIGHTSEN_OUT))   //day
            return 1;
    else //night
            return 0;

  }
```

소스코드에 대해서 간단하게 설명한다.

아래 명령어로 컴파일을 실행한다.

```
  $ g++ -o demodb demodb.cpp -lwiringPi -lm -lmysqlclient
```

컴파일이 에러없이 완료되었다면 아래와 같이 실행하고 결과를 확인한다.

```
  $ sudo ./demodb
  pi@raspberrypi ~/demofarmdb $ sudo ./demodb
  MySQL(rpidb) opened.
  Data not good, skip
  Humidity = 51.10 % Temperature = 26.80 *C
  Humidity = 51 Temperature = 26
  Temperature = 26
  Data not good, skip
```

```
Data not good, skip
Humidity = 51.10 % Temperature = 26.80 *C
Humidity = 51 Temperature = 26
Humidity = 51
Humidity = 51.10 % Temperature = 26.70 *C
Humidity = 51 Temperature = 26
Temperature = 26
Data not good, skip
Humidity = 51.10 % Temperature = 26.70 *C
Humidity = 51 Temperature = 26
Humidity = 51
```

2~3 초에 한 번씩 저장한 데이터를 확인한다.

```
pi@raspberrypi ~/demofarmdb $ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 48
Server version: 5.5.41-0+wheezy1 (Debian)

Copyright (c) 2000, 2014, Oracle and/or its affiliates. All rights
reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input
statement.

mysql>
```

사용하고자 하는 데이터 베이스를 선택한다.

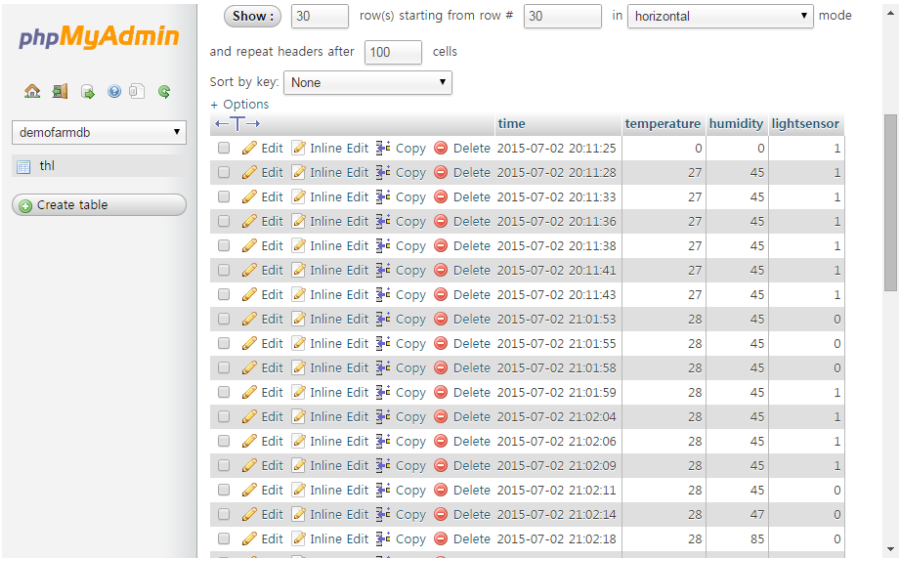
```
mysql> use demofarmdb;
Database changed
```

데이터 베이스 파일의 내용을 확인한다.

```
mysql> select * from thl;
+---------------------+-------------+----------+-------------+
| time                | temperature | humidity | lightsensor |
+---------------------+-------------+----------+-------------+
| 2015-07-02 20:11:25 |           0 |        0 |           1 |
| 2015-07-02 20:11:28 |          27 |       45 |           1 |
| 2015-07-02 20:11:33 |          27 |       45 |           1 |
| 2015-07-02 20:11:36 |          27 |       45 |           1 |
```

윈도우 웹브라우져를 열어 phpmyadmin 으로 접속하여 데이터를 확인한다.

브라우저에서 http://192.168.1.239/phpmyadmin/ 에 접속하여
id : root
password : root 접속하여 데이터를 확인한다. 아래는 브라우저에서 본 데이터베이스 테이블 thl 화면이다.



웹에서 실시간으로 변하는 데이터 베이스를 확인하기 위한 php 파일을 작성해야 한다. 아래 명령을 사용하여 /var/www/ 폴더로 이동하여 viewfarm.php 파일을 작성한다.

```
pi@raspberrypi ~ $ cd /var/www
pi@raspberrypi /var/www $
pi@raspberrypi /var/www $sudo nano viewfarm.php
```

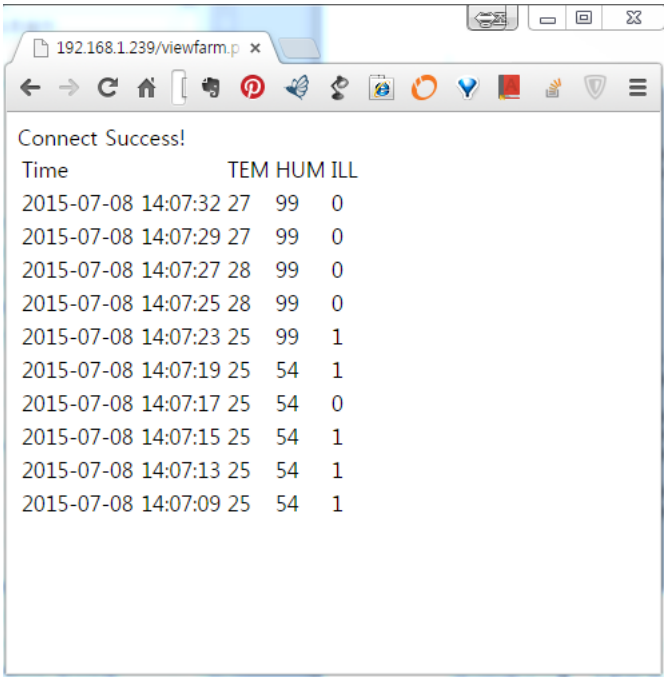
```
pi@raspberrypi /var/www $ sudo nano viewfarm.php
<script language='javascript'>
  window.setTimeout('window.location.reload()',1000); </script>
  <?php $connect = mysql_connect("localhost", "root", "root")
   or die("Connect Fail: " . mysql_error());
   echo "Connect Success!"; mysql_select_db("demofarmdb") or
die("Select DB Fail!"); $query = "select
* from thl order by time desc limit 10";
$result = mysql_query($query) or die("Query Fail: " . mysql_error());
```

```
echo "<table>\n";
echo "<tr><td>Time</td><td>TEM</td><td>HUM</td><td>ILL</td></tr>";
while ($line = mysql_fetch_array($result, MYSQL_ASSOC)) {
   echo "\t<tr>\n";
   foreach ($line as $col_value) {
       echo "\t\t<td>$col_value</td>\n";
   }
   echo "\t</tr>\n";
}
echo "</table>\n"; mysql_free_result($result); mysql_close($connect);
?>
```

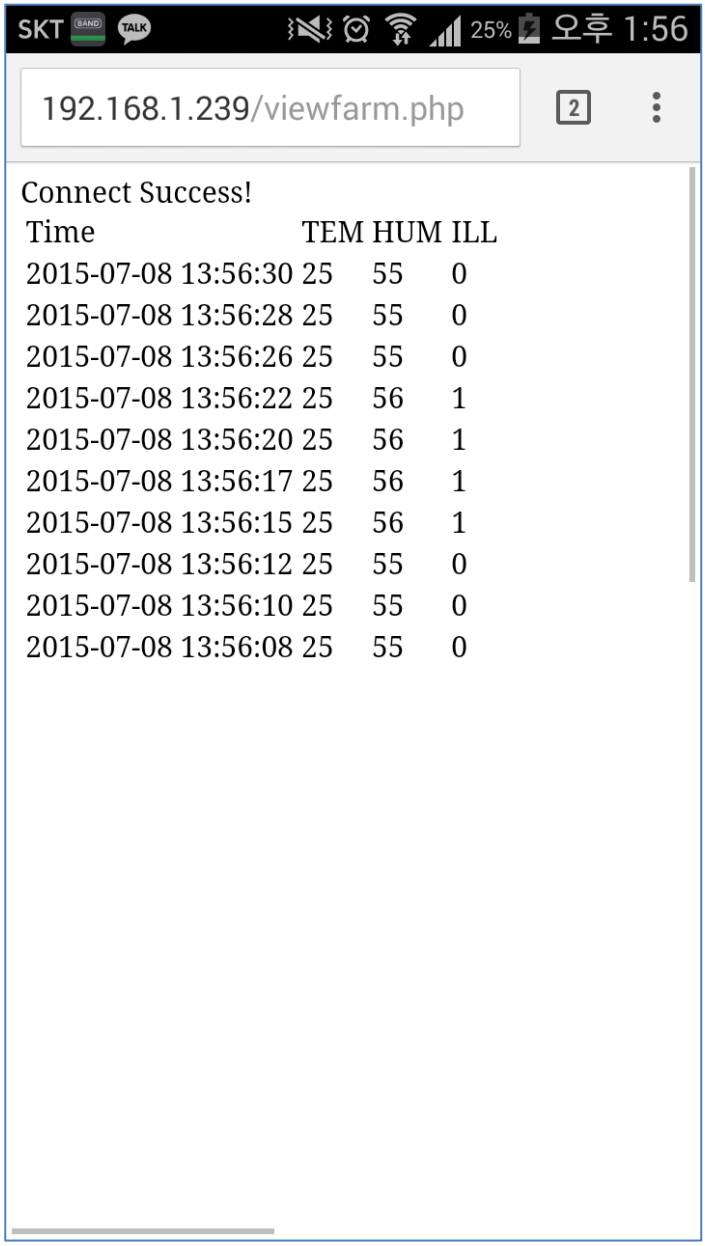
실시간으로 변하는 데이터 베이스 화면을 보기위하여 라즈베리 파이에서 작업 폴더로 이동 후 demnofarmdb 폴더로 이동하여 femodb 를 실행한다.

```
pi@raspberrypi ~ $ cd demofarmdb
pi@raspberrypi ~/demofarmdb $
pi@raspberrypi ~/demofarmdb $sudo ./demodb  실행
```

브라우져에서 http://192.168.1.239/viewfarm.php 에 접속하여 센서 데이터를 확인한다. 아래는 브라우저에서 본 데이터 화면이다.



마찬가지로 스마트 폰에서 같은 주소로 접속하여 확인한 화면을 아래에 나타낸다.



10.8 웹(브라우져)에서 스마트 농장 액츄에이터 제어

라즈베리 파이를 브라우저를 통해 원격제어 하는 방법중에 가장 간단한 방법을 실습해보자.

서버는 파이썬을 기반으로 한 SimpleHTTPServer 구현하고, 클라이언트는 부라우저의 메시지 전송방식중에 하나인 POST 방식을 이용하여 웹 서버를 경유하여 스마트 농장 액츄에이터를 제어하게 된다.

우선 작업 폴더를 생성하고 이동하여 작업한다.

```
$mkdir farmactubrowser
$cd farmactubrowser
pi@raspberrypi ~/farmactubrowser $ ls
controlServer.py  index.html
pi@raspberrypi ~/farmactubrowser $
```

vkdlTjsdmf 이요하여 HTTP 서버프로그램을 작성한다. 소스 코드를 아래에 나타내었다. controlServer.py

```
import cgi
import RPi.GPIO as gpio
from BaseHTTPServer import HTTPServer
from SimpleHTTPServer import SimpleHTTPRequestHandler

#Actuator define BCM GPIO Pin - is not wiringPi Pin number
PUMP = 5
FAN = 6
MOTOR = 13
RGB = 19

RED = 4
GREEN = 3
BLUE = 2

class Handler(SimpleHTTPRequestHandler):
    def do_POST(self):
        if self.path == '/rccar':
            form = cgi.FieldStorage(fp=self.rfile,
                                    headers=self.headers,
                                    environ={'REQUEST_METHOD':'POST'})
            cmd = form['cmd'].value
            print cmd

            if cmd == "PUMPON":
                gpio.output(PUMP, gpio.HIGH)
            elif cmd == "PUMPOFF":
```

```
                                gpio.output(PUMP, gpio.LOW)
            elif cmd == "FANON":
                                gpio.output(FAN, gpio.HIGH)
            elif cmd == "FANOFF":
                                gpio.output(FAN, gpio.LOW)
            elif cmd == "MOTORON":
                                gpio.output(MOTOR, gpio.HIGH)
            elif cmd == "MOTOROFF":
                                gpio.output(MOTOR, gpio.LOW)
            elif cmd == "RGBON":
                                gpio.output(RGB, gpio.HIGH)
                                gpio.output(RED, gpio.HIGH)
                                gpio.output(GREEN, gpio.HIGH)
                                gpio.output(BLUE, gpio.HIGH)
            elif cmd == "RGBOFF":
                                gpio.output(RGB, gpio.LOW)

            self.send_response(100)
            self.send_header('Content-type', 'text/html')

            return

        return self.do_GET()

gpio.setwarnings(False)
gpio.setmode( gpio.BCM )

#Pin Output Setup
gpio.setup(PUMP, gpio.OUT)
gpio.setup(FAN, gpio.OUT)
gpio.setup(MOTOR, gpio.OUT)
gpio.setup(RGB, gpio.OUT)

gpio.setup(RED, gpio.OUT)
gpio.setup(GREEN, gpio.OUT)
gpio.setup(BLUE, gpio.OUT)

#Pin Initialization
gpio.output(PUMP, gpio.LOW)
gpio.output(FAN, gpio.LOW)
gpio.output(MOTOR, gpio.LOW)
gpio.output(RGB, gpio.LOW)

gpio.output(RED, gpio.LOW)
gpio.output(GREEN, gpio.LOW)
gpio.output(BLUE, gpio.LOW)


server = HTTPServer(('', 8002), Handler).serve_forever()
```

다음으로 웹 브라우저를 통하여 HTTP 서버에 접속할 클라이언트 프로그램을 작성하자. HTML 페이지로 작성한다. Index.html

```
<html>
    <head>
        <meta charset="UTF-8">
        <title>Smart Farm actuator control</title>
        <meta   name="viewport"   content="width=200,   initial-scale=1,
maximum-scale=1">
    </head>

    <body>
    <p style="text-align:center; clear: none; float: none;">스마트 농장
액츄에이터 제어 실습</p>
            <div align="center" style="margin: 0 0 10px 10px">
            <ul >
                <input             type="button"             style=font-
size:10pt;width:70;height:60; value="pump on" onClick="cmd('PUMPON')">
                            <input       type="button"       style=font-
size:10pt;width:70;height:60; value="pump off" onClick="cmd('PUMPOFF')">
            </ul>
                    <ul >
                <input             type="button"             style=font-
size:10pt;width:70;height:60; value="fan on" onClick="cmd('FANON')">
                            <input       type="button"       style=font-
size:10pt;width:70;height:60; value="fan off" onClick="cmd('FANOFF')">
            </ul>

                    <ul >
                <input             type="button"             style=font-
size:10pt;width:70;height:60; value="motor on" onClick="cmd('MOTORON')">
                            <input       type="button"       style=font-
size:10pt;width:70;height:60;              value="motor             off"
onClick="cmd('MOTOROFF')">
            </ul>

                    <ul >
                <input             type="button"             style=font-
size:10pt;width:70;height:60; value="rgb led on" onClick="cmd('RGBON')">
                            <input       type="button"       style=font-
size:10pt;width:70;height:60;         value="rgb         led         off"
onClick="cmd('RGBOFF')">
            </ul>


        </div>

        <script type="text/javascript">
            function cmd(value) {
```

```
                if( window.XMLHttpRequest ) {
                    request = new XMLHttpRequest();
                }

                if( !request ) {
                    alert("XMLHttpRequest Error");
                    return false;
                }

                var send = 'cmd=' + value;
                request.open('POST','/rccar',true);

                request.setRequestHeader('Content-Type',
'application/x-www-form-urlencoded');
                request.setRequestHeader('Content-Length',  send.length);
                request.setRequestHeader('Connection', 'close');
                request.send(send);
            }
        </script>
    </body>
 </html>
```

위와 같이 소스를 작성하면 실습할 준비는 끝난것이다. 웹 에서 데이터베이스를 조회하거나 액츄에이터를 제어하는 프로그램을 작성할 때의 잇점은 단말기를 가리지 않고 어떤 환경에서든 적용할 수 있다는 점이다. 즉 운영체제나 단말기의 화면등에 제한없이 사용자에게 서비스를 제공할 수 있다는 것이다.

그럼 실습을 위해 파이썬 서버 프로그램을 실행하자. 명령은 $ sudo python controlServer.py 이다. 아래는 파이썬 서버 실행화면이다.

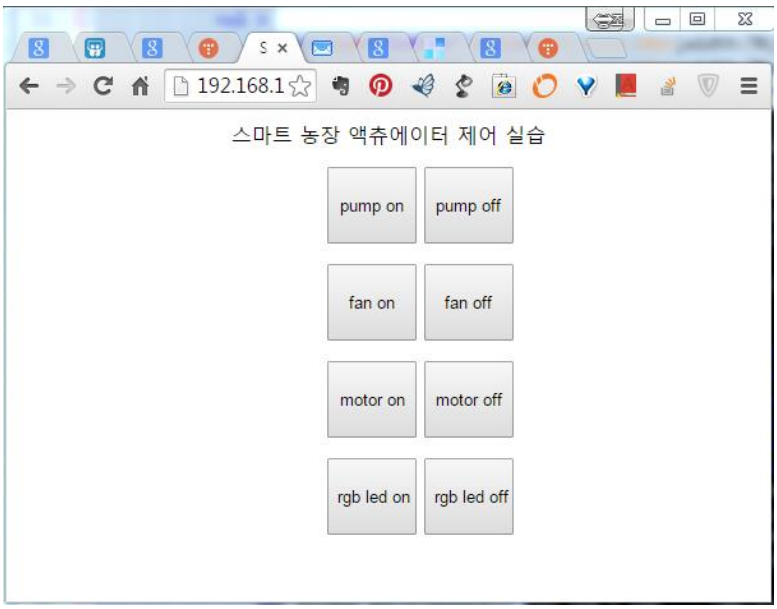
```
 pi@raspberrypi ~/farmactubrowser $ sudo python controlServer.py
 controlServer.py:62: RuntimeWarning: A physical pull up resistor is
fitted on this channel!
   gpio.setup(GREEN, gpio.OUT)
 controlServer.py:63: RuntimeWarning: A physical pull up resistor is
fitted on this channel!
   gpio.setup(BLUE, gpio.OUT)
 PUMPON
 192.168.1.199 - - [08/Jul/2015 11:39:29] "POST /rccar HTTP/1.1" 100 -
 PUMPOFF
 192.168.1.199 - - [08/Jul/2015 11:39:31] "POST /rccar HTTP/1.1" 100 -
 FANON
 192.168.1.199 - - [08/Jul/2015 11:39:32] "POST /rccar HTTP/1.1" 100 -
 MOTORON
 192.168.1.199 - - [08/Jul/2015 11:39:34] "POST /rccar HTTP/1.1" 100 -
 RGBON
```

```
192.168.1.199 - - [08/Jul/2015 11:39:35] "POST /rccar HTTP/1.1" 100 -
RGBOFF
192.168.1.199 - - [08/Jul/2015 11:39:35] "POST /rccar HTTP/1.1" 100 -
MOTOROFF
192.168.1.199 - - [08/Jul/2015 11:39:36] "POST /rccar HTTP/1.1" 100 -
FANOFF
192.168.1.199 - - [08/Jul/2015 11:39:36] "POST /rccar HTTP/1.1" 100 -
```

다음으로 PC 에서 브라우저를 띄워(크롬) 라즈베리 파이 주소에 접속하자. 브라우저 주소줄에 아래와 같이 입력한다.

http://192.168.1.239:8002

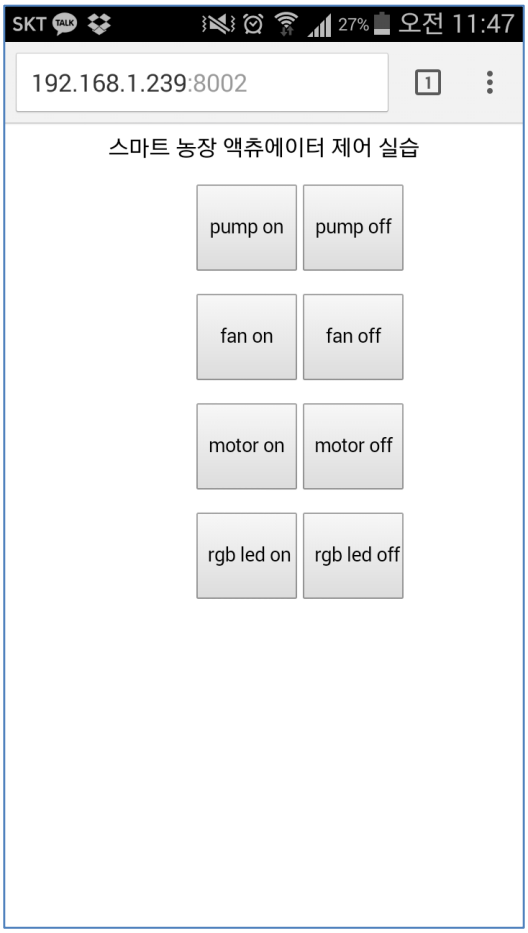
아래는 PC 의 크롬 브라우저에서 실행 화면이다.



버튼을 클릭하면 제대로 동작하는 것을 확인할 수 있다. 확인하였으면 다음은 스마트 폰에서 확인하기로 하자. 스마트폰의 무선 공유기를 사용하는 Wi-Fi 설정은 라즈베리 파이와 같은 네트워크에 있어야 한다. 스마트 폰의 크롬 브라우저를 띄워 주소창에 PC 에서 입력하였던 주소와 같은 주소를 입력한다.

http://192.168.1.239:8002

동일하게 스마트 폰 브라우저에서 작동함을 확인할 수 있다.



이상으로 브라우저에서 데이터베이스를 조회하고, 액츄에이터를 제어하는 실습을 마친다.

10.9 스마트 폰으로 농장 액츄에이터 제어하기

스마트 폰으로 농장 액츄에이터를 제어하느 방법을 알아보자. 물론 연결상태는 현재까지 테스트 하던 상태 그대로 진행하면 되겠다.

(1) 하드웨어 연결도

앞장에서 센서 테스트, 액츄에이터 테스트 환경과 동일한 연결과 하드웨어를 사용한다.

(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍

안드로이드 앱을 이요한 원격제어를 하기 위해서는 라즈베리 파이가 TCP/IP 의 서버가 되고, 안드로이드 단말기가 TCP/IP 의 클라이언트로서 프로그래밍 해야 한다. 일단 안드로이드 단말기는 클라이언트로서 서버인 라즈베리 파이에 접속한다. 그 후 안드로이드 단말기(클라이언트) 화면의 액츄에이터 제어 버튼을 클릭하면 정해진 문자열을 라즈베리파이(서버)로 전송하고, 라즈베리 파이는 그에 따른 처리를 수행하고 클라이언트에 결과를 전송하는 프로그램을 작성할 것이다.

즉, 서버 프로그래밍(라즈베리 파이), 클라이언트 프로그래밍(안드로이드 단말기)의 두 가지 작업이 필요하다. 안드로이드 단말기를 이용한 TCP/IP 프로그래밍 소스코드는 코드량이 많기 때문에 CD 의 스마트 폰 제어 프로그램 폴더의 코드를 참고하고 여기서는 대략적으로 코딩과정과 순서를 설명한다.

1. 프로젝트 생성
2. string.xml 에 각종 string 을 구성한다.
3. activit_main 을 편집하여 화면을 구성한다.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context=".actu_p2rcontrol" >
```

```
<ToggleButton
    android:id="@+ id/buttonRightTurn"
    android:layout_width="80dp"
    android:layout_height="wrap_content"
    android:layout_above="@+ id/buttonDown"
    android:layout_toRightOf="@+ id/buttonUp"
    android:text="@string/Right"
    android:textOff="Fan Off"
    android:textOn="Fan On" />

<ToggleButton
    android:id="@+ id/buttonLeftTurn"
    android:layout_width="80dp"
    android:layout_height="wrap_content"
    android:layout_below="@+ id/buttonUp"
    android:layout_toLeftOf="@+ id/buttonDown"
    android:text="@string/Left"
    android:textOff="Motor Off"
    android:textOn="Motor On" />


<ToggleButton
    android:id="@+ id/buttonUp"
    android:layout_width="80dp"
    android:layout_height="wrap_content"
    android:layout_below="@+ id/textViewStatus"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="37dp"
    android:text="@string/Up"
    android:textOff="펌프 Off"
    android:textOn="펌프 On" />
```

```
<ToggleButton
    android:id="@+ id/buttonDown"
    android:layout_width="80dp"
    android:layout_height="wrap_content"
    android:layout_below="@+ id/buttonLeftTurn"
    android:layout_centerHorizontal="true"
    android:text="@string/Down"
    android:textOff="RGB Off"
    android:textOn="RGB On" />

<Button
    android:id="@+ id/buttonConnect"
    android:layout_width="120dp"
    android:layout_height="wrap_content"
    android:layout_alignParentTop="true"
    android:layout_marginTop="66dp"
    android:layout_toLeftOf="@+ id/buttonUp"
    android:text="@string/Connect" />

<TextView
    android:id="@+ id/textViewStatus"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@+ id/buttonConnect"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="20dp"
    android:text="@string/DataView"
    android:textSize="20sp" />

<EditText
```

```
            android:id="@+ id/editTextIPAddress"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_above="@+ id/buttonClose"
            android:layout_centerHorizontal="true"
            android:layout_marginBottom="18dp"
            android:ems="10"
            android:gravity="center_vertical|center_horizontal"
            android:inputType="text"
            android:text="@string/IP" />

        <Button
            android:id="@+ id/buttonClose"
            android:layout_width="120dp"
            android:layout_height="wrap_content"
            android:layout_alignBaseline="@+ id/buttonConnect"
            android:layout_alignBottom="@+ id/buttonConnect"
            android:layout_toRightOf="@+ id/buttonUp"
            android:text="@string/Close" />

    </RelativeLayout>
```

클라이언트로 동작하는 안드로이드 프로그램은 3 개의 소스 파일로 구성되었다. 데이터를 보여주는 Logger 와 데이터를 파싱하여 메인 화면을 제어해주는 파일, 그리고 데이터 수신 쓰레드이다. Logger 와 수신 쓰레드는 변경사항이 없으므로 소스를 참고하고 여기서는 액츄에이터를 제어하는 소스를 보여준다.
파일들이 모두 있는 폴더는 farmactuphone 폴더이다.
아래 코드는 안드로이드 앱 소스이다. 다음 주소에서 다운 받아 사용한다.
http://smartiot.tistory.com/35

```
package huins.iot.actucontrol;
```

```
import android.os.Bundle;
import android.os.StrictMode;
import android.app.Activity;
import android.content.Context;
import android.graphics.Color;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.view.inputmethod.InputMethodManager;
import android.widget.TextView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ToggleButton;

import java.net.Socket;
import java.io.IOException;
import java.io.OutputStream;

public class actu_p2rcontrol extends Activity implements OnClickListener{

    private EditText editTextIPAddress;
    private TextView textViewStatus;
    private Button buttonConnect;
    private Button buttonClose;
    private ToggleButton buttonUp;
    private ToggleButton buttonLeftTurn;
    private ToggleButton buttonRightTurn;
    private ToggleButton buttonDown;



    private InputMethodManager imm;
```

```
    private String server = "192.168.1.238";
    private int port = 8888;
    //private int port = 8080;
    private Socket socket;

    private OutputStream outs;
    private Thread rcvThread;
    public logger logger;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);

            editTextIPAddress =
(EditText)this.findViewById(R.id.editTextIPAddress);
            editTextIPAddress.setText(server);
            imm =
(InputMethodManager)getSystemService(Context.INPUT_METHOD_SERVICE);

           textViewStatus = (TextView)this.findViewById(R.id.textViewStatus);
            textViewStatus.setText("스마트팜 액츄에이터 테스트");

            logger = new logger(textViewStatus);

            buttonConnect    = (Button)this.findViewById(R.id.buttonConnect);
            buttonClose      = (Button)this.findViewById(R.id.buttonClose);
            buttonUp         = (ToggleButton)this.findViewById(R.id.buttonUp);
            buttonLeftTurn =
(ToggleButton)this.findViewById(R.id.buttonLeftTurn);
```

```
                buttonRightTurn =
(ToggleButton)this.findViewById(R.id.buttonRightTurn);
                buttonDown =
(ToggleButton)this.findViewById(R.id.buttonDown);

                buttonConnect.setOnClickListener(this);
                buttonClose.setOnClickListener(this);
                buttonUp.setOnClickListener(this);
                buttonLeftTurn.setOnClickListener(this);
                buttonRightTurn.setOnClickListener(this);
                buttonDown.setOnClickListener(this);

        }

        @Override
        public boolean onCreateOptionsMenu(Menu menu) {
                getMenuInflater().inflate(R.menu.main, menu);
                return true;
        }
        @Override
        public void onClick(View arg0) {

                StrictMode.ThreadPolicy policy = new
StrictMode.ThreadPolicy.Builder()
                .detectAll()
                .penaltyLog()
                .build();
                StrictMode.setThreadPolicy(policy);

                if(arg0 == buttonConnect)
                {
```

```
    imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

                    try{
                        if(socket!=null)
                        {
                            socket.close();
                            socket = null;
                        }

                        server = editTextIPAddress.getText().toString();
                        socket = new Socket(server, port);
                        outs = socket.getOutputStream();

                        rcvThread = new Thread(new rcvthread(logger,
socket));
                    rcvThread.start();
                        logger.log("Connected");
                    } catch (IOException e){
                        logger.log("Fail to connect");
                        e.printStackTrace();
                    }
            }

            if(arg0 == buttonClose)
            {

    imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

                    if(socket!=null)
                    {
```

```
                    exitFromRunLoop();
                    try{
                        socket.close();
                        socket = null;
                        logger.log("Closed!");
                        rcvThread = null;
                    } catch (IOException e){
                        logger.log("Fail to close");
                        e.printStackTrace();
                    }
                }
            }


            if(arg0 == buttonUp)
            {
                String sndOpkey = "CMD";

                if(buttonUp.isChecked()){
                    buttonUp.setTextColor(Color.RED);
                    sndOpkey = "Pumpon";
                }

                else{
                    buttonUp.setTextColor(Color.BLACK);
                    sndOpkey = "Pumpoff";
                }
                try{
                    outs.write(sndOpkey.getBytes("UTF-8"));
                    outs.flush();
                } catch (IOException e){
```

```
                    logger.log("Fail to send");
                    e.printStackTrace();
                }
            }

            if(arg0 == buttonLeftTurn)
            {
                String sndOpkey = "CMD";

                if(buttonLeftTurn.isChecked()){
                    buttonLeftTurn.setTextColor(Color.RED);
                    sndOpkey = "Motoron";
                }

                else{
                    buttonLeftTurn.setTextColor(Color.BLACK);
                    sndOpkey = "Motoroff";
                }
                try{
                    outs.write(sndOpkey.getBytes("UTF-8"));
                    outs.flush();
                } catch (IOException e){
                    logger.log("Fail to send");
                    e.printStackTrace();
                }
            }

            if(arg0 == buttonRightTurn)
            {
                String sndOpkey = "CMD";
```

```
                    if(buttonRightTurn.isChecked()){
                            buttonRightTurn.setTextColor(Color.RED);
                            sndOpkey = "Fanon";
                    }

                    else{
                            buttonRightTurn.setTextColor(Color.BLACK);
                            sndOpkey = "Fanoff";
                    }
                    try{
                            outs.write(sndOpkey.getBytes("UTF-8"));
                            outs.flush();
                    } catch (IOException e){
                            logger.log("Fail to send");
                            e.printStackTrace();
                    }
            }

            if(arg0 == buttonDown)
            {
                    String sndOpkey = "CMD";

                    if(buttonDown.isChecked()){
                            buttonDown.setTextColor(Color.RED);
                            sndOpkey = "Rgbon";
                    }

                    else{
                            buttonDown.setTextColor(Color.BLACK);
                            sndOpkey = "Rgboff";
                    }
```

```
                    try{
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
                    } catch (IOException e){
                        logger.log("Fail to send");
                        e.printStackTrace();
                    }
                }

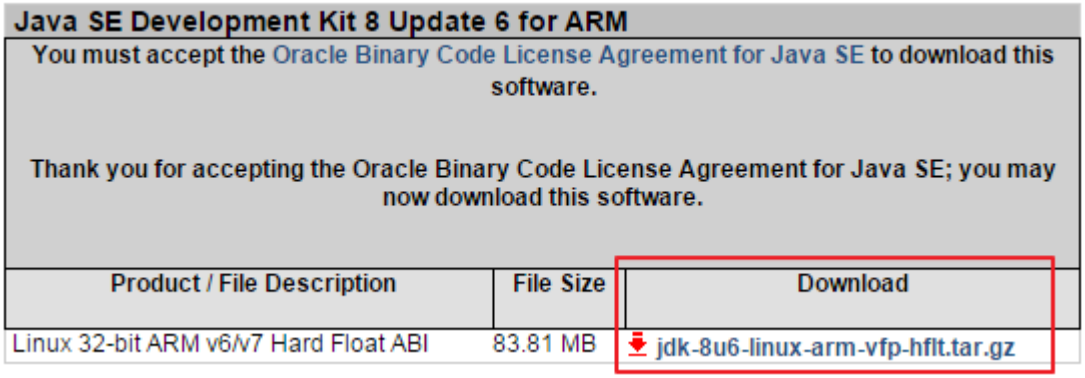
            }

             void exitFromRunLoop(){
                try {
                    String sndOpkey = "[close]";
                    outs.write(sndOpkey.getBytes("UTF-8"));
                    outs.flush();
                } catch (IOException e) {
                    logger.log("Fail to send");
                    e.printStackTrace();
                }
             }
        }
```

(3) 라즈베리 파이 TCP/IP 서버 프로그래밍

다 언어를 지원하는 TCP/IP 는 언어의 제한을 받지 않는다. 여기서는 참고 코드를 쉽게 구할 수 있고, 많은 예제들이 있는 자바를 사용해서 프로그래밍 하자. 라즈베리 파이에서 자바를 사용하기 위해서는 하드웨어 연산을 지원하는 JDK8 의 설치가 필요하다. JDK8 의 설치는 복잡하지만 어렵지 않으므로 순차적으로 실습해보자.

JDK8 설치에 관해 영문이지만 설명이 잘 되어있다.
http://yoo7577.tistory.com/181
JDK8 을 다운로드 하기 위해서이곳을 방문하고 아래 그림과 같은 파일을 저장한다.
http://www.oracle.com/technetwork/java/javase/downloads/jdk8-arm-downloads-2187472.html



Java SE Development Kit 8 Update 6 for ARM

You must accept the Oracle Binary Code License Agreement for Java SE to download this software.

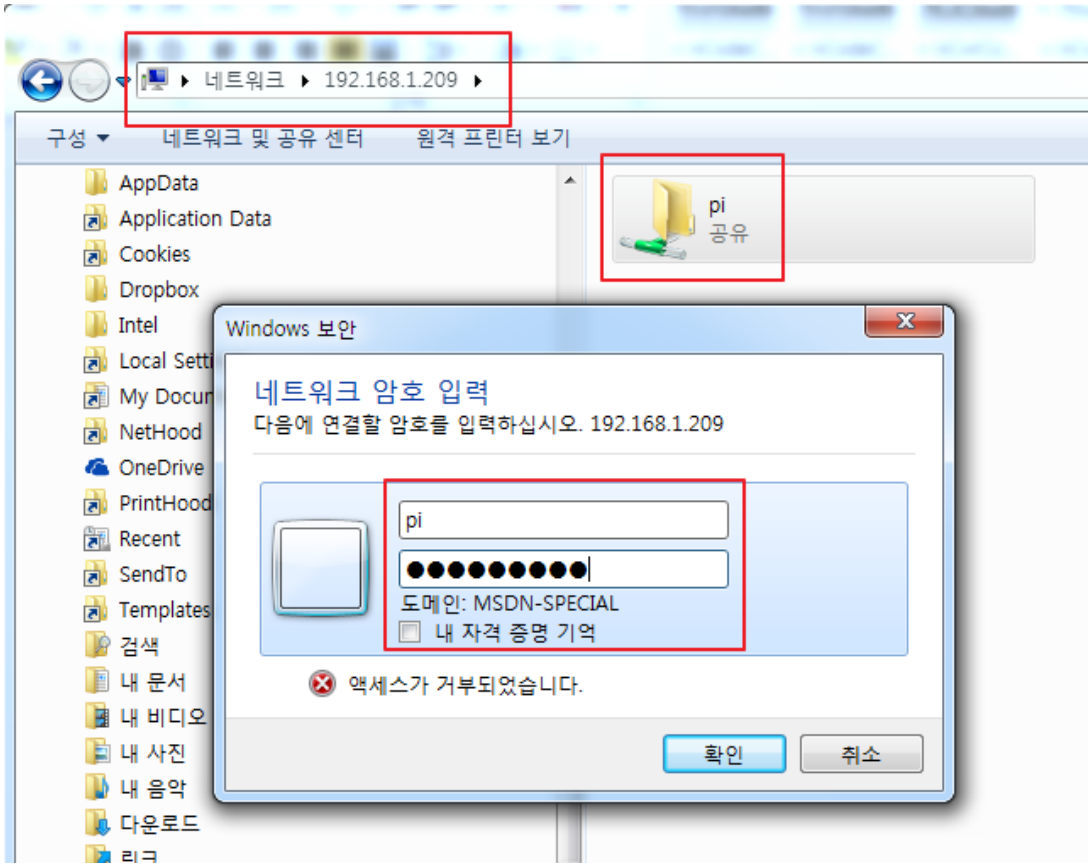
Thank you for accepting the Oracle Binary Code License Agreement for Java SE; you may now download this software.

Product / File Description	File Size	Download
Linux 32-bit ARM v6/v7 Hard Float ABI	83.81 MB	jdk-8u6-linux-arm-vfp-hflt.tar.gz

윈도우에서 다운 로드 한 위치를 기억한다.
아래 명령어로 라즈베리 파이의 주소를 확인하고 네트워크로 접속해준다.

```
pi@raspberrypi ~ $ ifconfig
eth0      Link encap:Ethernet  HWaddr b8:27:eb:0b:a7:4d
          inet addr:192.168.1.209  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:80528 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1942 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:5721058 (5.4 MiB)  TX bytes:216975 (211.8 KiB)
```

아이피 주소는 192.168.1.209 로 확인되었다.



네트워크 창을 열고 아이피 주소를 입력하고 /home/pi/ (작업 폴더임)에 다운 받은 파일을 옮기고 아래는 라즈베리 파이에서 작업한다.

아래와 같은 명령어로 /opt/java 라는 폴더를 생성하자.

$sudo mkdir -p -v /opt/java

현재 까지 하면 폴더구조는 아래와 같다.

```
pi@raspberrypi ~ $ cd ..
pi@raspberrypi /home $ cd ..
pi@raspberrypi / $ ls
bin boot dev etc home lib lost+found media mnt opt proc root
run sbin selinux srv sys tmp usr var
pi@raspberrypi / $
pi@raspberrypi / $ cd opt
```

```
pi@raspberrypi /opt $ ls
java  minecraft-pi  sonic-pi  vc  Wolfram
pi@raspberrypi /opt $ cd java
pi@raspberrypi /opt/java $ ls
pi@raspberrypi /opt/java $ cd ..
pi@raspberrypi /opt $ cd ..
pi@raspberrypi / $ cd home
pi@raspberrypi /home $ cd pi
pi@raspberrypi ~ $ pwd
/home/pi
pi@raspberrypi ~ $
```

다운로드한 폴더로 이동한다.

/opt/java 폴더에 다운로드한 파일을 압축해제 한다.

sudo tar xvzf ~/ jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java

```
pi@raspberrypi ~ $ sudo tar xvzf ~/jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java
jdk1.8.0_06/COPYRIGHT
jdk1.8.0_06/LICENSE
jdk1.8.0_06/README.html
jdk1.8.0_06/THIRDPARTYLICENSEREADME-JAVAFX.txt
jdk1.8.0_06/THIRDPARTYLICENSEREADME.txt
jdk1.8.0_06/bin/
jdk1.8.0_06/bin/javah
jdk1.8.0_06/bin/appletviewer
jdk1.8.0_06/bin/extcheck
jdk1.8.0_06/bin/jps
jdk1.8.0_06/bin/wsimport
jdk1.8.0_06/bin/unpack200
jdk1.8.0_06/bin/jarsigner
.
.
.
```

잠시 후 압축이 풀리고 Oracle JDK gz 파일들이 “/opt/java” 폴더의 아래에 “jdk1.8.0_06”폴더에 위치하게 된다.

다운로드 받은 파일을 더 이상 필요없기에 삭제한다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ rm ~/jdk-8u6-linux-arm-vfp-hflt.gz
pi@raspberrypi ~ $
```

시스템에게 새로운 JDK 가 인스톨 되었고, 위치를 알려주기 위해 아래와 같이 실행한다.

```
pi@raspberrypi ~ $ sudo update-alternatives --install "/usr/bin/java" "java" "/opt/java/jdk1.8.0_06/bin/java" 1
pi@raspberrypi ~ $
```

마지막으로 JDK 가 시스템의 디폴트 java 런타임이 될것이라고 알려준다.

```
pi@raspberrypi ~ $ sudo update-alternatives --set java /opt/java/jdk1.8.0_06/bin/java

pi@raspberrypi ~ $
```

이제 JDK8 이 설치되었다. 자바 버전명령을 사용해 아래와 같이 나온다면 잘 설치된 것이다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ java -version
java version "1.8.0_06"
Java(TM) SE Runtime Environment (build 1.8.0_06-b23)
Java HotSpot(TM) Client VM (build 25.6-b23, mixed mode)
pi@raspberrypi ~ $
```

진짜 마지막으로 JAVA_HOME 환경 변수들을 몇가지 더해야 한다. 우선 bashrc 선언문을 마지막에 추가해 준다.

```
sudo nano ~/.bashrc
```

```
export JAVA_HOME=/opt/jdk1.8.0
export PATH=$PATH:$JAVA_HOME/bin
```

를 마지막에 적어주고 ctrl-x 누르고 y 누르고 Enter 치고 빠져 나온다.

Reboot 를 수행한다. 부팅을 수행하고 나면 Java 샘플 프로그램 작성 및 동작 테스트를 실행하자.

소스코드 생성 및 작성 - nano 편집기를 이용하여 아래와 같은 샘플 코드를 작성하자.

```
$nano HelloWorld.java


public class HelloWorld
{
    public static void main(String args[])
    {
        java.lang.System.out.println("Hello World!!!");
    }
}
```

아래와 같이 컴파일을 수행한다.

```
$javac HelloWorld.java
```

실행한다. Classpath 지정을 잊지 말자.

```
$java -cp . HelloWorld


Hello World!!!
```

메시지가 보이면 설치와 테스트가 완료된 것이다.

여기까지 힘들게 실습하였다. 이제 라즈베리파이에서 JDK8 (with javaFX) 를 사용할 수 있게 되었다. 지금까지 라즈베리파이는 파이썬, C, C++, Java 등 정말 다양한 언어를 사용할 수 있게 되었는데, 이것은 단지 툴에 지나지 않다. 이를 이용하여 응용프로그램을 작성해 보도록 하자.

이제 부터는 쉬운 과정이다. 라즈베리파이의 GPIO 를 제어하기 위한 pi4j 라는 자바용 드라이브를 설치한다. pi4j 를 다운로드 하고 설치를 해준다. 아래는 명령어와 설치화면을 보여준다.

$curl -s get.pi4j.com | sudo bash

```
pi@raspberrypi ~ $ curl -s get.pi4j.com | sudo bash
=============================================================
INSTALLING Pi4J GPG PUBLIC KEY
=============================================================
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1761  100  1761    0     0   3134      0 --:--:-- --:--:-- --:--:--  5889
OK
=============================================================
ADDING Pi4J APT REPOSITORY
=============================================================
--2015-01-17 14:46:04--  http://get.pi4j.com/pi4j.list
Resolving get.pi4j.com (get.pi4j.com)... 54.231.17.212
Connecting to get.pi4j.com (get.pi4j.com)|54.231.17.212|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 42 [application/octet-stream]
Saving to: `/etc/apt/sources.list.d/pi4j.list'

100%[=======================================================>] 42          --.-K/s   in 0s
```

```
  2015-01-17 14:46:04 (547 KB/s) - `/etc/apt/sources.list.d/pi4j.list' saved
[42/42]

  ================================================
  UPDATING APT REPOSITORIES
  ================================================
  Hit http://repository.pi4j.com wheezy Release.gpg
  Hit http://repository.pi4j.com wheezy Release
  Hit http://repository.pi4j.com wheezy/rpi armhf Packages
  Ign http://repository.pi4j.com wheezy/rpi Translation-en_GB
  Ign http://repository.pi4j.com wheezy/rpi Translation-en
  Reading package lists... Done
  ================================================
  INSTALLING Pi4J
  ================================================
  Reading package lists... Done
  Building dependency tree
  Reading state information... Done
  The following NEW packages will be installed:
    pi4j
  0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
  Need to get 0 B/517 kB of archives.
  After this operation, 793 kB of additional disk space will be used.
  Selecting previously unselected package pi4j.
  (Reading database ... 87830 files and directories currently installed.)
  Unpacking pi4j (from .../pi4j_1.0~SNAPSHOT_all.deb) ...
  Setting up pi4j (1.0~SNAPSHOT) ...
  ================================================
  Pi4J INSTALLATION COMPLETE
  ================================================
```

```
The Pi4J JAR files are located at:
/opt/pi4j/lib

Example Java programs are located at:
/opt/pi4j/examples

You can compile the examples using this script:
sudo /opt/pi4j/examples/build

Please see http://www.pi4j.com for more information.

pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

위와 같이 설치하고 테스트를 진행할 때 에러나 나는 경우 언인스톨 명령어와 재설치를 아래에 나타낸다. 아래의 명령어로 언인스톨을 실행한다.

$ sudo apt-get remove pi4j

```
pi@raspberrypi ~ $ sudo apt-get remove pi4j
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  pi4j
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
1 not fully installed or removed.
After this operation, 793 kB disk space will be freed.
Do you want to continue [Y/n]? y
(Reading database ... 87881 files and directories currently installed.)
Removing pi4j ...
```

재설치명령어는 아래와 같다.
$ curl -s get.pi4j.com | sudo bash

안드로이드 스마트 폰(TCP/IP 클라이언트)과의 TCP/IP 통신을 위하여 쓰레드를 통해 데이터를 송수신 한다. 복수의 접속도 허용하고 있으며, 자원의 효율성도 고려하였다. 파일이 있는 위치는

```
pi@raspberrypi ~ $ cd farmactuphone
pi@raspberrypi ~/farmactuphone $ ls
Farmactu.class     HelloWorld.class     new     1.c           Racar.java
RcvThread.java     smartiotactu.java
Farmactu.java      HelloWorld.java      Racar.class     RcvThread.class
smartiotactu.class
pi@raspberrypi ~/farmactuphone $
```

다음의 코드를 참고한다. Farmactu.java

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Logger;

public class Farmactu {
    private static final int sizeBuf = 50;
    public static void main(String[] args) throws IOException {

        int port = 8888;
        ServerSocket serverSock = new ServerSocket(port);
        Logger logger = Logger.getLogger("Farmactu");

        while (true) {
            Socket clientSock = serverSock.accept();
```

```
                SocketAddress clientAddress = clientSock.getRemoteSocketAddress();
                Thread thread = new Thread(new RcvThread(clientSock, clientAddress, logger));
                thread.start();
                if(clientSock.isConnected())
                    System.out.println("Connected! Client IP : " + clientAddress);
            }
        }
```

TCP/IP 서버로 작동하는 소스코드다. 쓰레드로 동작하는 RcvThread.java 소스코드도 참고한다.

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Level;
import java.util.logging.Logger;

import com.pi4j.io.gpio.GpioController;
import com.pi4j.io.gpio.GpioFactory;
import com.pi4j.io.gpio.GpioPinDigitalOutput;
import com.pi4j.io.gpio.PinState;
import com.pi4j.io.gpio.RaspiPin;
import com.pi4j.io.gpio.impl.PinImpl;   //this!!!!

public class RcvThread implements Runnable{
    private static final int sizeBuf = 50;
    private Socket clientSock;
    private Logger logger;
```

```
        private SocketAddress clientAddress;

        final GpioController gpio = GpioFactory.getInstance();

        final GpioPinDigitalOutput pinlpwm =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_21, "lpwm", PinState.LOW);
        final GpioPinDigitalOutput pinlfwd =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_23, "lfwd", PinState.LOW);
        final GpioPinDigitalOutput pinlrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_22, "lrev", PinState.LOW);

        final GpioPinDigitalOutput rgbpwr =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_24, "rpwm", PinState.LOW);

        final GpioPinDigitalOutput rgbblue =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_27, "rfwd", PinState.LOW);

        final GpioPinDigitalOutput pinrrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_03, "rrev", PinState.LOW);

        public RcvThread(Socket clntSock, SocketAddress clientAddress, Logger
logger) {
            this.clientSock = clntSock;
            this.logger = logger;
            this.clientAddress = clientAddress;
        }

        public void run(){
            try {
                InputStream ins     = clientSock.getInputStream();
                OutputStream outs = clientSock.getOutputStream();
```

```
int rcvBufSize;
byte[] rcvBuf = new byte[sizeBuf];
while ((rcvBufSize = ins.read(rcvBuf)) != -1) {

    String rcvData = new String(rcvBuf, 0, rcvBufSize, "UTF-8");

                    if (rcvData.compareTo("Pumpon") == 0) {
        pinlpwm.high();
        System.out.println("Pumpon");
    }
                    if (rcvData.compareTo("Pumpoff") == 0) {
        pinlpwm.low();
        System.out.println("Pumpoff!");
    }


    if (rcvData.compareTo("Motoron") == 0) {
        pinlfwd.high();
                            System.out.println("Motoron");
                }
                    if (rcvData.compareTo("Motoroff") == 0) {
        pinlfwd.low();
        System.out.println("Motoroff");
    }

                    if (rcvData.compareTo("Fanon") == 0) {
        pinlrev.high();
                            System.out.println("Fanon");
                }
                    if (rcvData.compareTo("Fanoff") == 0) {
```

```
                    pinlrev.low();
                    System.out.println("Fanoff");
                }
                        if (rcvData.compareTo("Rgbon") == 0) {
                    rgbpwr.high();
                            rgbblue.high();
                            System.out.println("Rgbon");
                        }
                        if (rcvData.compareTo("Rgboff") == 0) {
                    rgbpwr.low();
                            rgbblue.low();
                    System.out.println("Rgboff");
                }




            logger.info("Received data : " + rcvData + " (" + clientAddress +
")");
                outs.write(rcvBuf, 0, rcvBufSize);
            }
            logger.info(clientSock.getRemoteSocketAddress() + " Closed");
        } catch (IOException ex) {
            logger.log(Level.WARNING, "Exception in RcvThread", ex);
        } finally {
            try {
                clientSock.close();
                System.out.println("Disconnected! Client IP : " + clientAddress);
            } catch (IOException e) {}
        }
    }
}
```

아래와 같은 명령어로 컴파일을 수행한다. 실행할 디렉토리는 farmactuphone 폴더이다.

```
$javac -classpath .:classes:/opt/pi4j/lib/'*' Farmactu.java
```

아래와 같은 명령어로 실행을 한다.

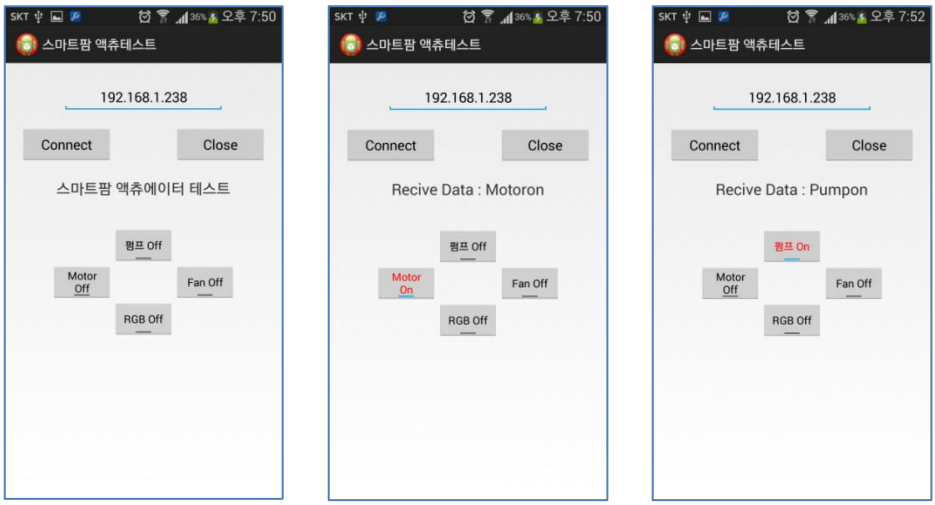
```
$sudo java -classpath .:classes:/opt/pi4j/lib/'*' Farmactu
```

라즈베리 파이 화면에서 보여지는 실행화면을 아래에 표시하였다

```
pi@raspberrypi ~/racar-test $ sudo java -classpath .:classes:/opt/pi4j/lib/'*'
Farmactu
Connected! Client IP : /192.168.1.98:37403
Pumpon
Jan 17, 2015 7:53:33 PM RcvThread run
INFO: Received data : Pumpon (/192.168.1.98:37403)
Pumpoff!
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Pumpoff (/192.168.1.98:37403)
Motoron
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Motoron (/192.168.1.98:37403)
Motoroff
Jan 17, 2015 7:53:36 PM RcvThread run
INFO: Received data : Motoroff (/192.168.1.98:37403)
Fanon
Jan 17, 2015 7:53:37 PM RcvThread run
INFO: Received data : Fanon (/192.168.1.98:37403)
Fanoff
Jan 17, 2015 7:53:39 PM RcvThread run
INFO: Received data : Fanoff (/192.168.1.98:37403)
Rgbon
Jan 17, 2015 7:53:39 PM RcvThread run
```

```
INFO: Received data : Rgbon (/192.168.1.98:37403)
Rgboff
Jan 17, 2015 7:53:40 PM RcvThread run
INFO: Received data : Rgboff (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: Received data : [close] (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: /192.168.1.98:37403 Closed
Disconnected! Client IP : /192.168.1.98:37403
```

아래는 안드로이드 스마트 폰으로 시험하는 화면을 보여준다.



10.10 스마트농장 시스템 서버 프로그램

스마트 농장 서버 프로그램은 Gateway 기능을 수행하는 라즈베리 파이에서 돌아가는 소프트웨어이다. 게이트웨이 서버 소프트웨어의 기능을 살펴보자.

○ 온습도 센서로부터 일정한 시간마다 센서 데이터 수집

○ 조도 센서로부터 일정한 시간마다 센서 데이터 수집

○ 수집된 센서 데이터는 시간마다 정보와 함께 데이터 베이스에 저장
○ 이미 설정된 환경 조건(물 부족, 야간)을 판단하여 액츄에이터 제어
○ 식물농장 상태를 카메라를 이용하여 동영상 스트리밍으로 전송
○ 원격지 사용자의 요청이 있을경우 액츄에이터 직접 제어

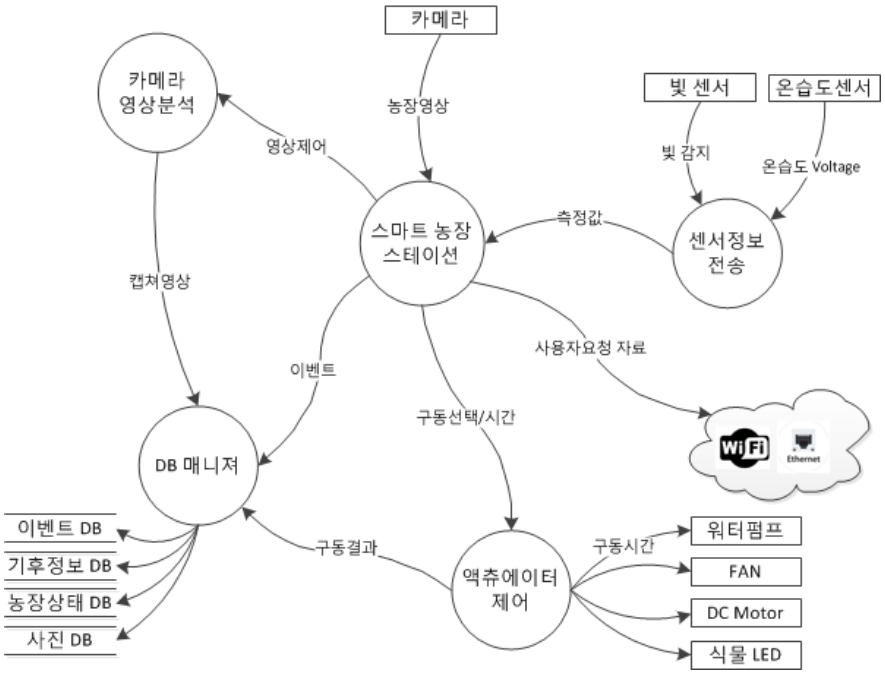


그림 112 서버 소프트웨어의 Data Flow Diagram

스마트 농장 서버 프로그램은 데이터를 파싱하고, 대기상태에서 사용자의 입력을 기다리게 된다. 사용자가 스마트 폰을 이용하여 현재 상태와 카메라 보기, 설정을 Setting 하는 기능을 통해 서버 프로그램은 해당 기능을 수행한다.

(1) 데이터 베이스 설계

일단 데이터 베이스에 대해 알아보자. 앞서 APM(Apache, PHP, MySQL)을 설치하였다. 그것을 바탕으로 데이터 베이스를 활용해보자. 여기서 사용하는 데이터 베이스는 스마트 농장의 모든 센서데이터와 이벤트 기록을 주기적으로, 아니면 발생시 데이터 베이스에 저장한다. 몇 개월이든 농장의 자료를 저장하는

것은 큰 의미가 있는 일이다. 지역의 기후와 농장의 계절 특성을 파악하는 것은 많은 도움이 될것이다. 다른 교재에서 데이터 베이스의 생성과 기초적인 부분들을 살펴보고 자세한 것은 Mysql 관련도서를 참조하자.

데이터 베이스의 설계

스마트 농장에서 데이터 베이스의 설계를 대략적으로 나타낸다. 데이터 베이스의 설계와 테이블 구성은 방대한 작업이므로 아래 실습에서 간단하게 실습하고, 실제 구현시에는 서버 코드를 참조하여 꼼꼼하게 구현한다.

DB Name : iotfarm

Table : tfarmsensorvalue 센서값 저장 테이블

1 분마다 센서값 저장한다.

날짜 day / 조도 int / 온도 float / 습도 float

Table : tfarmsensorcheck 센서 정상/불량 저장 테이블

센서 정상 불량 확인하여 저장 10 분마다.

날짜 day / 조도센서 (정상, 오류) / 온도센서 (정상, 오류) / 습도센서(정상, 오류)

Table : tfarmactuoperate 액츄에이터 동작상태(동작중/정지중)

액츄 동작시 데이터베이스 업데이트 하고, 동작 끝날 때 업데이트

날짜 day / pump(on,stop) / fan(on, stop) / dcmotor(on, stop) / rgbled(on, stop)

Table : tfarmactucheck 액츄에이터가 정상/불량 저장 데이블

액츄에이터 정상 불량 기록 – 현재는 모두 정상

pump(normal, fail), fan(normal, fail), dcmotor(normal, fail), rgbled(normal, fail)

Table : tfarmsetting 설정값 저장 테이블

Now(), 매일/2 일/3 일(0,1,2)

(2) 서버 실행과 카메라 영상

여기서는 스마트 농장 서버를 실행하고 CD 에 들어있는 smartfarm 앱을 실행시킨 상태에서 결과만을 확인하도록 한다.

```
pi@raspberrypi ~ $ cd smartfarm_server
pi@raspberrypi ~/smartfarm_server $
pi@raspberrypi ~/smartfarm_server $ sudo ./server
```

실행을 하고 스마트 폰에서 라즈베리 파이의 IP 주소를 연결한 후 확인하게 되면 다음과 같은 메시지를 보이면서 실행이 된다.

```
pi@raspberrypi ~/smartfarm_server $ sudo ./server
Handling client ip : 192.168.1.98
Handling client port : 56323
Handling client socket number : 5
connected user   : 1
[5Sock][MyUserNum:0]:[0|257X_▒X_]
message parsing start :0|257X_▒X_
 app type : 0x0000
 command : 0x0101
 data :
Data not good, skip
Data not good, skip
Humidity = 17.40 % Temperature = 24.40 *C
Humidity = 17 Temperature = 24
Temperature = 24
Data not good, skip
Humidity = 17.40 % Temperature = 24.40 *C
Humidity = 17 Temperature = 24
```

```
Humidity = 17
Smartfarm_server>server
```

카메라를 확인할 경우 서버에서 카메라 쉘 프로그램이 실행되고 있어야 한다. 따라서 현재 상태에서 Putty 프로그램을 하나 더 실행하여 루트에서 sh mjpg.sh 명령어를 실행한 상태로 실습을 진행한다.

```
pi@raspberrypi ~ $ sh mjpg.sh

MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
 i: camera parameters.....:
.
.
.
```

위와 같이 실행 한 상태에서 스마트 폰의 '농장모습'을 클릭하면 농장의 동영상을 볼 수 있다.

10.11 스마트 농장 안드로이드 앱 프로그램

이 장에서는 안드로이드 앱과 스마트 농장 관리 시스템의 연동에 대해 설명한다. 안드로이드 어플리케이션은 클라이언트 프로그램으로 동작한다. 라즈베리 파이에서는 앞 장에서 구현한 서버 프로그램과 동영상 스트림 쉘이 수행되는 상태에서 스마트 폰의 “스마트 농장” 어플리케이션을 실행시켜 동작 상태를 확인하고, 다양한 액츄에이터를 구동한다.

스마트 농장관리 시스템의 스마트 폰 앱은 안드로이드 OS 기반의 스마튼 폰에서 수행되는 Application 으로 아래와 같은 기능을 가지고 있다.

○ 화분이나 농장의 기후와 현재 기상상태를 조회.
○ 화분이나 농장의 현재 상태를 카메라로 영상 확인
○ 화분이나 농장의 과거 이벤트와 이력사항을 조회
○ 설정된 조건(습도, 온도, 조도, 시간)에 따라 정해진 액츄에이터 가동
○ 화분이나 농장에 설치된 Actuator 의 원격 제어

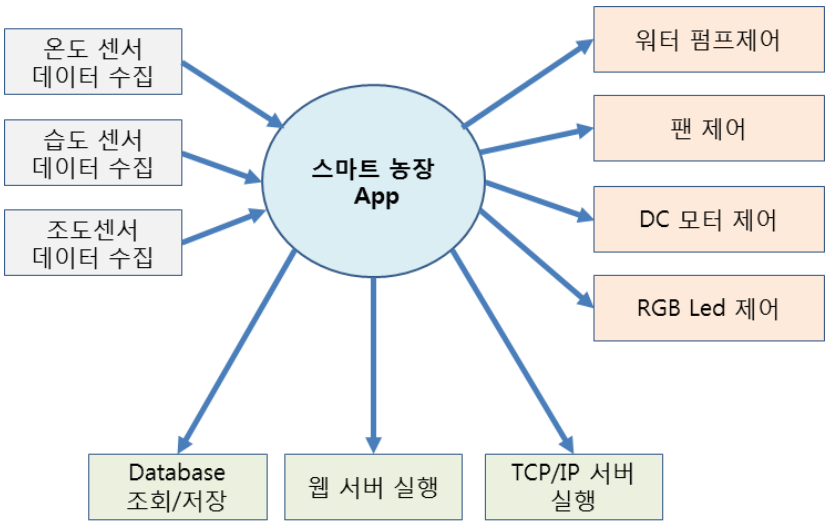


그림 113 스마트 농장관리 시스템 앱 소프트웨어 구조

여기서는 프로그램 구성을 설명하고 스마트 폰 앱을 실행하여 동영상을 확인하고 다양한 액츄에이터를 제어하는 실습을 진행한다. 소스코드의 구성은 사용자 스스로 분석하여 다양한 환경에 적용하도록 하자.

아래에 스마트 농장의 서버와 앱이 연동되는 테스트 과정을 설명한다. 이대로 시험하면 결과를 확인할 수 있고 나머지는 참고로 이용하길 바란다.

1. 라즈베리 파이 전원을 켜고, IO 어댑터 보드의 12V 전원을 켠다.
2. 로그인 아이디 : pi password : raspberry (화면에 나타나지 않음)
3. 라즈베리 파이 아이피 주소 : $ifconfig 명령어로 확인한다.
   예)192.168.1.219
4. 라즈베리 파이 스마트 팜 서버 실행

```
login as: pi
pi@192.168.1.219's password:
~~
pi@raspberrypi ~ $ cd iot_farmserver
pi@raspberrypi ~/iot_farmserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_farmserver $ cd output
pi@raspberrypi ~/iot_farmserver/output $ ls
server_farm
pi@raspberrypi ~/iot_farmserver/output $ sudo ./server_farm &
```

(백그라운드 실행) 엔터 입력하여 터미널 화면에서

5. 동영상 스트리밍 서버 실행 : $sh mjpg.sh
6. 스마트 폰 앱 실행 pi@raspberrypi ~ $ **sh mjpg.sh**

이 과정을 통해 연동 테스트를 수행한다.

라즈베리 파이의 서버 프로그램 위치는 iot_farmserver 폴더이다. 서버 실행파일의 위치를 알아보고 서버 실행파일을 만드는 과정을 알아보자.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ cd iot_farmserver
pi@raspberrypi ~/iot_farmserver $ ls
```

```
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_farmserver $ cd output
pi@raspberrypi ~/iot_farmserver/output $ ls
server_farm
pi@raspberrypi ~/iot_farmserver/output $
```

서버 프로그램을 컴파일하기 위해 $make clean 명령을 사용하여 이전 컴파일시 생성된 파일을 지우는 명령이다.

```
pi@raspberrypi ~/iot_farmserver $
pi@raspberrypi ~/iot_farmserver $ make clean
rm -rf /home/pi/iot_farmserver/output
Clearing in libs...
make[1]: Entering directory '/home/pi/iot_farmserver/libs'
rm -rf iot_db.o parser.o libiot.a core
make[1]: Leaving directory '/home/pi/iot_farmserver/libs'
Clearing in src...
make[1]: Entering directory '/home/pi/iot_farmserver/src'
Clearing in farm...
make[2]: Entering directory '/home/pi/iot_farmserver/src/farm'
rm -rf farm_main.o farm_sensor.o core
make[2]: Leaving directory '/home/pi/iot_farmserver/src/farm'
make[1]: Leaving directory '/home/pi/iot_farmserver/src'
pi@raspberrypi ~/iot_farmserver $
```

컴파일하기 위해 $make 명령을 실행한다.

```
pi@raspberrypi ~/iot_farmserver $
pi@raspberrypi ~/iot_farmserver $ make
make all in libs...
make[1]: Entering directory '/home/pi/iot_farmserver/libs'
gcc -c -I/home/pi/iot_farmserver/include -I/usr/include/mysql iot_db.c -o
iot_db.o
```

```
  gcc -c -I/home/pi/iot_farmserver/include -I/usr/include/mysql parser.c -o
parser.o
  ar rcs libiot.a -o iot_db.o parser.o
  make[1]: Leaving directory '/home/pi/iot_farmserver/libs'
  make all in src...
  make[1]: Entering directory '/home/pi/iot_farmserver/src'
  make all in farm...
  make[2]: Entering directory '/home/pi/iot_farmserver/src/farm'
  gcc -g -c -I/home/pi/iot_farmserver/include -c farm_main.c -o farm_main.o
  gcc -g -c -I/home/pi/iot_farmserver/include -c farm_sensor.c -o
farm_sensor.o
  gcc -o server_farm farm_main.o farm_sensor.o
/home/pi/iot_farmserver/libs/libiot.a -lmysqlclient -lpthread -lwiringPi
  make[2]: Leaving directory '/home/pi/iot_farmserver/src/farm'
  make[1]: Leaving directory '/home/pi/iot_farmserver/src'
  pi@raspberrypi ~/iot_farmserver $
```

에러 없이 실행되었다면 다음으로 실행파일을 출력 폴더에 생성하기 위해 make install 명령을 실행한다.

```
  pi@raspberrypi ~/iot_farmserver $
  pi@raspberrypi ~/iot_farmserver $ make install
  mkdir /home/pi/iot_farmserver/output
  Installing in libs...
  make[1]: Entering directory '/home/pi/iot_farmserver/libs'
  make[1]: *** No rule to make target 'install'.  Stop.
  make[1]: Leaving directory '/home/pi/iot_farmserver/libs'
  Installing in src...
  make[1]: Entering directory '/home/pi/iot_farmserver/src'
  Installing in farm...
  make[2]: Entering directory '/home/pi/iot_farmserver/src/farm'
  cp -a server_farm /home/pi/iot_farmserver/output/
```

```
make[2]: Leaving directory '/home/pi/iot_farmserver/src/farm'
make[1]: Leaving directory '/home/pi/iot_farmserver/src'
pi@raspberrypi ~/iot_farmserver $
```

여기까지 실행하면 output 폴더에 서버 프로그램이 생성된다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ cd iot_farmserver
pi@raspberrypi ~/iot_farmserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_farmserver $ cd output
pi@raspberrypi ~/iot_farmserver/output $ ls
server_farm
pi@raspberrypi ~/iot_farmserver/output $
```

서버프로그램을 실행한다.

```
$sudo ./server_farm
```

아래는 서버 프로그램을 실행시킨 화면이다. 서버는 온 습도, 조도 데이터를 수집하여 데이터 베이스에 저장하는 한편 클라이언트가 되는 스마트 폰에서 접속하기를 기다린다. 따라서 터미널 화면에서 서버를 수행시키면 다른 컴퓨터에서 putty 를 실행시켜 동영상 스트리밍을 서비스 하는 프로그램을 실행하는 방법으로 실습하여야 한다.

```
pi@raspberrypi ~/iot_homeserver $ cd output
pi@raspberrypi ~/iot_homeserver/output $ ls
server_smarthome
pi@raspberrypi ~/iot_homeserver/output $ sudo ./server_smarthome
CLOSE
validate database...
iot_send_query : query - CREATE TABLE IF NOT EXISTS thomesensorvalue
(id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, fire INT, gas
INT, sound INT, light INT, temperature FLOAT, humid FLOAT, human INT)
```

```
  iot_send_query : query - CREATE TABLE IF NOT EXISTS thomesensorcheck
(id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, fire INT, gas
INT, sound INT, light INT, temperature INT, humid INT, human INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS thomeactuoperate
(id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, lantern INT,
door INT, gas INT, light INT, water INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS thomeactucheck (id
INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, lantern INT,
door INT, gas INT, light INT, water INT)
  done
  starting server...
  get fire sensor
  get light sensor
  Data not good, skip
  Humidity = 9.40 % Temperature = 24.60 *C
  Humidity = 9 Temperature = 24
  Temperature = 24
```

이상태에서 putty 를 실행시키고, 라즈베리 파이에 접속한다. Putty 를 실행하는 이유는 위에 서버를 실행하고 다시 동영상 스트리밍 서버를 실행하기 위함인데 위에 설명한 $ sudo ./server_farm & 명령(뒤에 &를 붙여서)으로 백그라운드로 실행하게 하면 putty 프로그램을 실행할 필요는 없다.

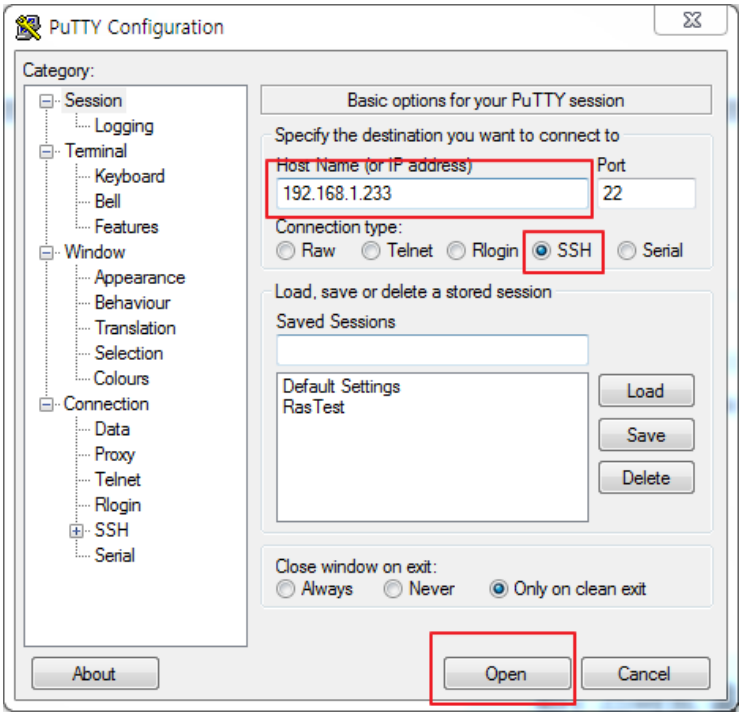


그림 114 동영상 스트림 서버를 실행시키기 위한 Putty 실행화면

아이디와 패스워드를 넣어 접속한 화면은 아래와 같다.

```
login as:
login as: pi
pi@192.168.1.219's password:
Linux raspberrypi 3.18.7+ #755 PREEMPT Thu Feb 12 17:14:31 GMT 2015
armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 09:47:01 2015
```

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

홈 디렉토리에서 mjpg.sh 파일을 확인하고 다음의 명령어로 mjpg 스트림 프로그램을 실행한다.

$sh mjpg.sh

```
pi@raspberrypi ~ $ sh mjpg.sh
MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
 i: camera parameters.....:
.
.
.
.
```

이 상태에서 모든 준비가 끝났다. 안드로이드 폰으로 다운 받은 스마트 농장 앱을 실행시키면서 결과를 확인한다. 아래는 앱 실행 메인 화면이다.

스마트 농장의 메인 화면은 농장상태, 농장모습, 물주기 설정, 디바이스 연결과 같은 네 가지 메뉴로 구성되어 있다. 간단하게 요약하면

농장상태 : 스마트 농장의 현재 상태 정보를 보여준다. 센서값과 액츄에이터의 이상 유무

농장모습 : 실시간으로 농장의 영상을 디스플레이

물주기 설정 : 물 주는 시간을 설정하고 액츄에이터 직접 제어

디바이스 연결 : 라즈베리 파이와 연결

아래 각 메뉴를 실행했을때의 메뉴와 설명을 나타낸다.



그림 115 스마트 농장 앱 메인화면 1



그림 116 스마트 농장 앱 메인화면 2

스마트 농장의 안드로이드 애플리케이션 메인에서 실행되는 메뉴는 농장 상태, 농장 모습, 물주기 설정, 디바이스 연결로 구성된다. 농장 상태에서는 현재 스마트 농장에서 사용하는 센서의 데이터와 액츄에이터의 상태를 볼 수 있다. 농장 모습은 현재 농장의 영상을 실시간으로 스트리밍하여 사용자 화면에 보여준다. 물주기 설정에서는 스마트 농장에서 사용되는 액츄에이터를 직접 제어할 수 있고, 물주는 시간을 직접 사용자가 설정하는 기능을 한다.



그림 117 스마트 농장 상태화면

스마트 농장 상태를 나타낸 화면이다. 현재 농장의 대기 온도와 습도를 나타내고, 조도센서의 센서값을 보여준다. 또한 센서 상태의 이상 유무와 농장 액츄에이터 동작상태의 이상 유무를 나타낸다.

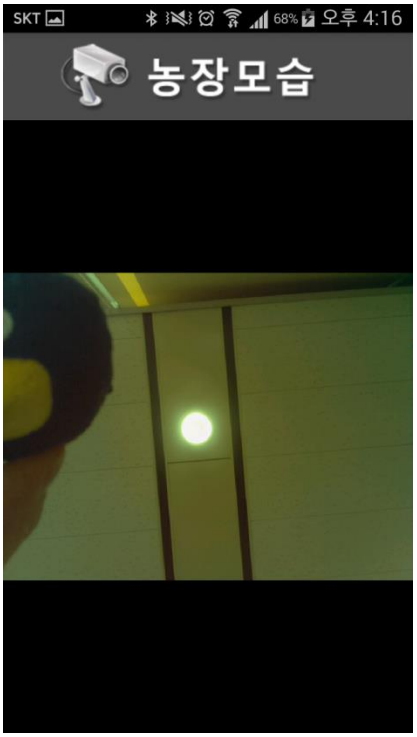


그림 118 스마트 농장 모습 화면

앱 화면은 스마트 농장 모습 보기에 해당하는 화면이다. mjpg 스트리밍을 사용하여 현재 농장의 모습을 보여준다. 서버에는 mjpg 스트리밍 쉘이 실행되는 상태에서 Wi-Fi 를 통하여 사용자 스마트 폰에 카메라 영상이 나타나게 된다.

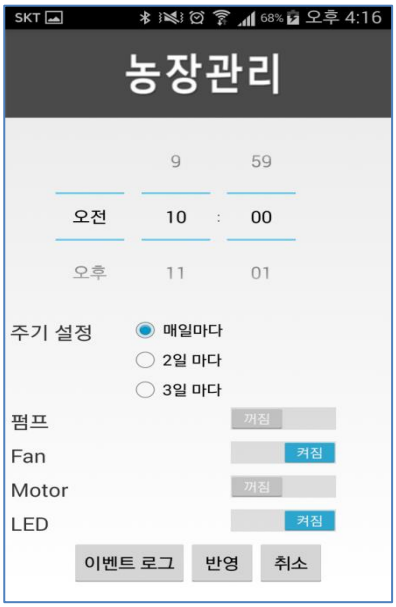


그림 119 물주기 설정 화면

물주기 설정 화면에서는 사용자가 임의의 시간 간격을 설정하여 그 시간에 워터 펌프가 작동하여 농장에 물을 주는 시간을 설정할 수 있다. 또한 이 화면에서 직접 스마트 농장에 설치된 액츄에이터(워터펌프, 팬, DC 모터, RGB Led)를 직접 제어할 수 있다.

IoT 설계 시스템의 스마트 농장 실습을 이것으로 마친다. 사용자의 창의적인 아이디어로 센서와 액츄에이터를 선정하여 다른 환경을 구성하여 실습하길 바란다.

Chapter 11 IoT 무인경비 시스템 실습

IoT 무인경비시스템은 방문자의 모니터링이 필요한 학교, 빌딩, 가정, 공장, 상가 등에서 사용할 수 있는 시스템으로 외부침입자가 발생시 실시간으로 센서 정보 및 영상정보를 수집하여 스마트폰으로 전송한다. 무인경비 시스템에는 카메라가 장착되어 있고, 센서로는 소리센서, 인체감지 센서, 초음파 센서가 있다. 센서로부터 데이터를 수집하여 인가받지 않은 침입자 발생시 Smart 폰으로 침입자 발생을 알려주고, 액츄에이터를 가동한다. 무인경비 시스템의 액추에이터는 싸이렌 부저, DC 모터, 조명 Led, 경광등 Led 등이 있으며 스마트폰을 통해 원격에서 직접 제어도 가능하도록 설계했다. 서버 역할을 하는 Gateway 시스템은 Database 관리, Web 서버, 액츄에이터 제어, 사용자 모바일 기기와 연동 하는 기능을 수행한다.

11.1 무인경비 시스템 시스템 개요

IoT 무인 경비시스템은 무인 경비시스템 Gateway 와 스마트 폰 모바일 프로그램으로 구성된다. 무인 경비시스템 Gateway 는 인체감지센서, 음향센서, 초음파 센서에서 수집한 데이터를 Database 에 저장하고, 센서 데이터를 판단하여 다양한 액츄에이터(부저, DCMotor, 전등, RGB Led)를 구동한다. 또한 안드로이드 폰과의 연동을 통해 원격지의 사용자 요청이 발생할 경우 Ethernet 이나 Wi-Fi 를 통해 센서데이터와 영상 정보를 송신한다.

IoT 무인 경비시스템의 주요기능은 아래와 같다.

- 스마트폰을 이용한 스마트 무인경비 시스템 Actuator 의 원격제어
- 음향센서와 인체감지센서를 통한 침입자 센싱 및 경광등(RGB LED), 싸이렌 경보 알림
- 스마트폰에 비상 알림 및 카메라 화상 전송 Display
- 원격영상을 통한 방문자 모니터링 및 Actuator 의 원격제어

아래는 IoT 통합 설계 시스템의 전체 구성도는 다른 장을 참조한다.

(1) 무인경비 시스템 하드웨어

Gateway 기능을 수행하는 하는 라즈베리 파이, 카메라, IO Adapter, 센서 3 종(인체감지 센서, 초음파 센서, 음향센서), 액츄에이터 4 종(부저 싸이렌, DC Motor, LED Bar, RGB Led 경광등)으로 구성되어 있다. 센서는 다른 센서 데이터가 필요할 경우 얼마든지 확장될 수 있고, 액츄에이터 또한 사용자의 설계 의도에 맞게 추가하여 구성할 수 있다. 여기에서는 인체감지와 음향센서에서 침입자 감지시 조명과 싸이렌을 울리고 문을 닫는 모터를 동작시키는 시험을 진행한다. 초음파 센서는 활용도가 많은 센서로 여기서는 약 2m 정도의 거리측정을 하는 기능으로 동작한다.

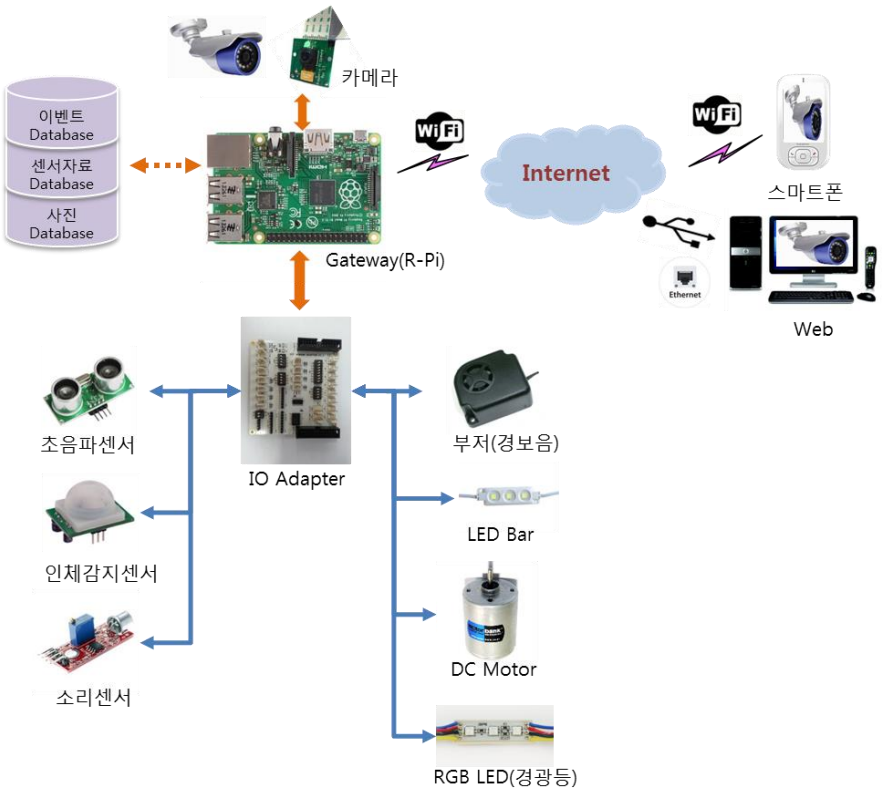


그림 120 무인 경비시스템의 하드웨어 구성도

IoT 무인경비 시스템을 구성하는 구성품과 사양 및 기능을 아래에 표로 정리하였다. 앞 장의 실습에서 설명한 라즈베리 파이, IO 어댑터 보드, 카메[라 모듈에 대한 설명은 동일하므로 생략하였다.

표 26 무인 경비시스템 하드웨어 구성 및 기능

구성품	사양	기능
센서	초음파센서	초음파로 물체와의 거리를 측정
	인체감지센서	인체감지 센서로 사람의 인기척이나 주위에 움직임을 감지
	소리센서	아주 작은 소리도 감지하는 음향 센서
Actuator	부저(경보음)	싸이렌 소리를 출력하는 부저
	LED bar	조명 역할을 하는 White Led
	Dc Motor	문을 닫는등의 역할을 하는 DC 모터
	RGB LED(경광등)	적색, 청색의 빛을 교대로 출력하여 경광등의 역할을 하는 Led
전원어댑터	DC 12V, 3A 전원 어댑터 DC5V, 1A 전원 어댑터	액츄에이터 전원 시스템 동작 전원

(2) 무인경비 시스템 소프트웨어

IoT 무인경비 시스템의 Software 는 크게 두부분으로 구성된다. 라즈베리 파이에서 수행되는 게이트웨이 소프트웨어는 센서로부터 데이터를 획득하고 저장하며 원격지 사용자 요청에 따라 액츄에이터를 동작시키며 카메라 영상을 전송하는 기능을 한다.

스마트 폰에서 동작하는 안드로이드 앱은 원격지의 영상을 실시간으로 확인하고, 경비 현장의 센서 데이터를 조회하며, 환경에 따라 액츄에이터를 직접 제어하는 기능을 한다. 카메라 구동 드라이버는 이미 게이트웨이에 포함되어 있으며 안드로이드 폰과 연동시 게이트 웨이의 서버 프로그램이 수행된다. Gateway 역할을 하는 라즈베리 파이는 리눅스 운영체제가 설치되어 있다. 센서로부터 수집된 데이터를 날짜 및 시간 데이터와 함께 데이터 베이스에 저장하고, 원격지의 사용자 요청에 따라 카메라 영상을 송신하고 설치된 액츄에이터를 제어하는 기능을 한다.

따라서 게이트 웨이 시스템과 스마트폰 Application 양 쪽에서 수행되는 Software 의 전체 구조를 살펴보면 아래와 같다.

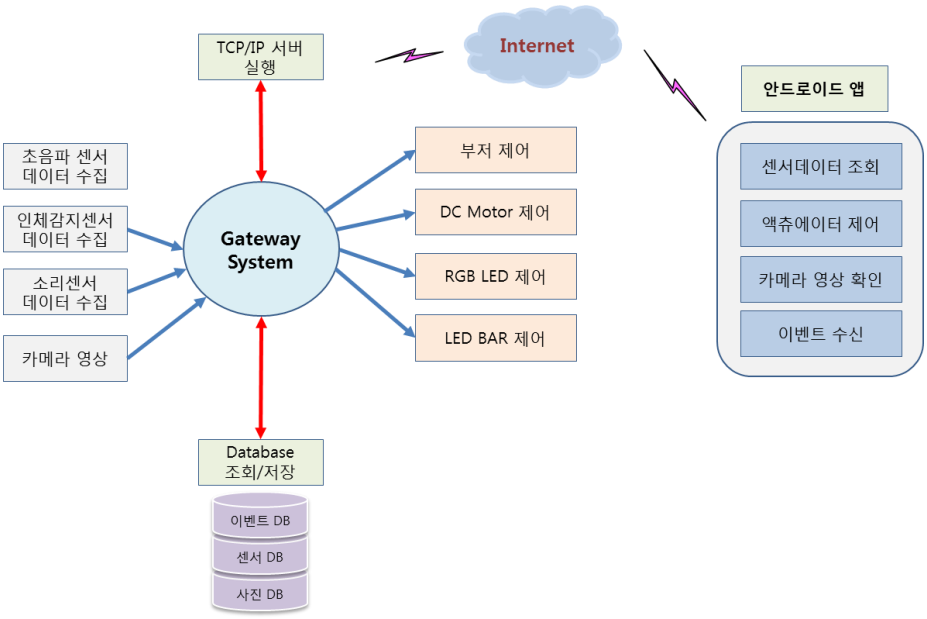


그림 121 스마트 무인경비 시스템의 소프트웨어 구성도

IoT 스마트 무인경비 관리시스템의 소프트웨어 구성은 앞 장에서 설명한 소프트웨어 구성과 크게 다르지 않다. 각 모듈에 대한 설명은 실습과정에 소개하기로 한다.

11.2 무인경비 시스템 연결 구성

IoT 무인경비 시스템의 연결은 아래와 같다. Gateway 기능의 라즈베리 파이에는 카메라와 와이파이 USB 무선 랜카드가 연결된다. IO Adapter 보드는 인체감지센서, 소리센서, 초음파 센서가 연결된다. IO Adapter 보드에 연결되는 액츄에이터는 부저, DC 모터, 조명 Led, 경광등 Led 가 있다. IO Adapter 보드는 센서와 액츄에이터를 Gateway 에 연결하는 역할을 한다. IO Adapter 보드에는 액츄에이터 구동을 위해 DC12V 전원이 공급되므로 각종 센서나 액츄에이터를 연결할 때 반드시 전원을 끈 상태에서 주의를 기울여 연결한다.

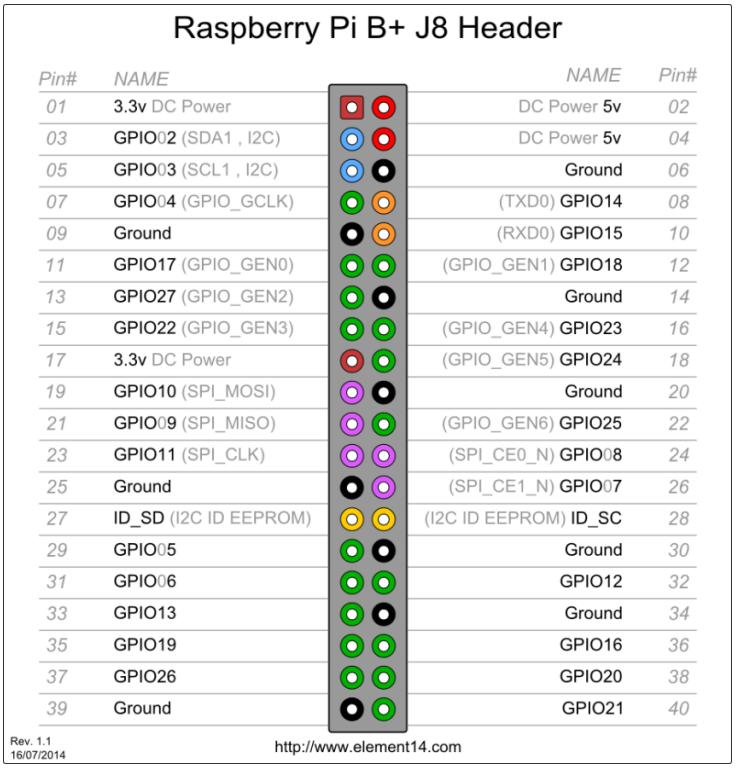


그림 122 라즈베리 파이 GPIO 핀 구성

게이트웨이 기능을 수행하는 라즈베리 파이 연결 커넥터는 40 핀 GPIO 케이블로 연결된다. 라즈베리 파이의 GPIO 포트는 사용자의 프로그램에서 입력으로 설정하거나 출력으로 설정을 한다. 또한 할당된 GPIO 핀 입력을 인터럽트로 설정하여 이벤트 리스너를 함수로 등록하여 사용하는것은 훌륭한 외부 인터럽트를 처리하는 방법이다. 라즈베리 파이에 아날로그 입력을 처리하는

방법은 MCP3208(ADC 컨버터)를 이용하여 처리한다. SPI 인터페이스를 사용하여 8 채널의 아날로그 신호를 처리한다. 아래는 무인경비 시스템의 센서와 액츄에이터 전체 연결도를 보여준다.

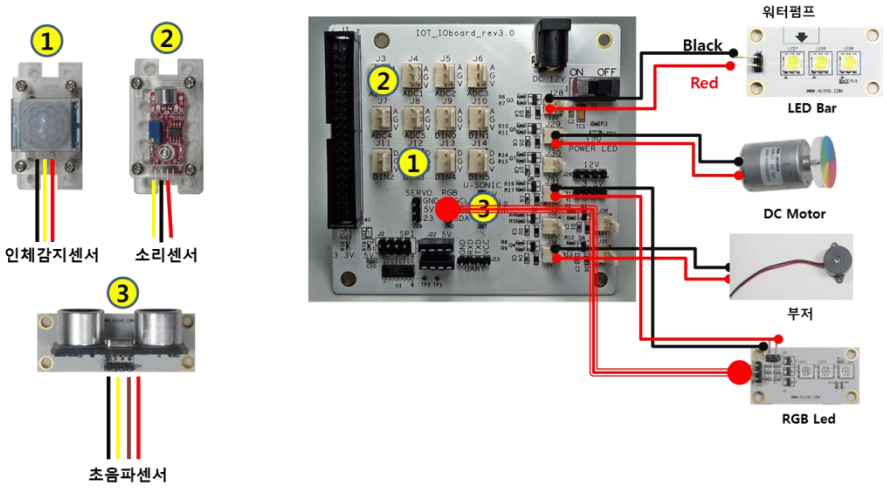


그림 123 무인경비 시스템 센서와 액츄에이터 연결도

IO Adapter 보드는 센서와 액츄에이터를 게이트웨이에 연결하는 역할을 한다. IO Adapter 보드에는 액츄에이터 구동을 위해 DC 12V 전원이 공급되므로 각종 센서나 액츄에이터를 연결할 때 반드시 전원을 끈 상태에서 주의를 기울여 연결한다.

주의

워터펌프, DC motor, RGB Led 등의 액츄에이터의 구동을 위해 IO Adapter 보드에는 DC 12V독전원이 연결되므로 센서나 액츄에이터를 연결할 때는 반드시 전원을 Off 하고 연결해야 하며, 쇼트, 단락에 주의해야 한다.

실습을 위해 센서를 액츄에이터를 연결하는 연결도를 확인하여 연결한다.

연결의 정확성

센서와 액츄에이터 보드의 연결도는 실습 후에 스마트 폰으로 입력을 확인하고 액츄에이터를 제어할 때까지 동일한 연결과 테스트를 수행해야 한다.

개별 센서와 액츄에이터 실습시에는 센서 데이터와 액츄에이터 제어가 올라보 되는지를 확인하고 실제 상황을 스마트 폰과 연동 실습할때에는 모든 연결이 정상적으로 정확하게 연결되어야 실습이 진행된다.

IO Adpater 보드는 직접 설계하여 제작한 보드인데, 다양한 센서와 여러종류의 액츄에이터를 연결하여 실습할 수 있도록 제작되었다. 실습에 필요한 GPIO 핀 번호와 연결을 확인하기 위해 센서부와 액츄에이터부 회로도를 참고한다.

11.3 무인경비 시스템 센서 실습

IoT 무인경비 시스템에서 사용되는 센서는 음향센서와 인체감지센서 그리고 초음파 센서이다. 사무실이나 집, 아니면 창고등에 무인경비를 설정했다고 가정하면 위의 3 가지 센서로부터 데이터를 수집하여 인기척이 있다든가, 소리가 들린다든가, 움직이는 물체가 가까이에서 검출된다면 우리는 누군가 인가받지 않은 사람이 침입하였다고 판단할 수 있다.

그때 붉은색과 파란색의 경광등을 울리고, 싸이렌 부저를 울리며 문을 열고 닫는 DC 모터를 작동한다고 생각해보면 이것이 바로 무인경비 스시템이라고 말할 수 있다. 여기서는 그러한 과정을 실습하고 응용할 수 있는 설계 방법을 실습한다.

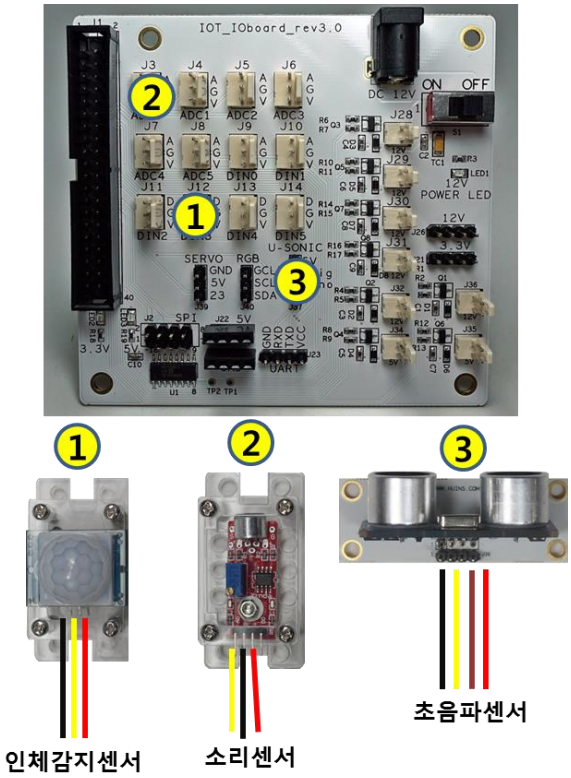


그림 124 센서 연결 Pinmap 과 연결 위치

(1) 사운드 센서(음향센서)

사운드 센서란 주변에서 흔히 볼 수 있는 마이크를 생각하면 된다. 소리가 발생할 경우 소리 센서는 소리의 크기와 지속시간에 따른 아날로그 신호를 출력한다. 소리로 전등을 켜고 끄거나 또는 주차 중인 차 안에 설치하여 주변에 소리가 발생할 경우 카메라가 동작하는 기능들은 사운드 센서를 사용해서 구현할 수 있는 기능들이다. 주변에서 여러가지 응용범위가 많은 사운드 센서를 실습하기로 한다. [그림 8]은 사운드 센서의 모양과 핀 맵을 보여준다.

표 27 사운드 센서 사양과 특징

사양	특징
크기	32mm*17mm*15mm(length*width*height)

Chip	LM393, 전기 콘덴서 마이크
작동 전압	DC 3-6v
출력	아날로그/디지털
LED	Power LED, Sensor Indicator LED

사운드 센서도 마찬가지로 출력은 4 핀으로 되어 있다. 전원과 GND 과 센서의 출력을 Analog 로 출력하는 Aout 과 HIGH, LOW 로 출력하는 Digital 출력핀이 그것이다. 아래는 사운드 센서의 외관과 핀맵을 나타낸다.



Pin1 – Analog Out
Pin2 – GND
Pin3 – VCC전원
Pin4 – Digital Out

그림 125 소리센서(음향센서의 외형과 핀 맵

여기서 사운드 센서 테스트는 아날로그 입력을 받아 아날로그 값을 출력하는 것을 테스트 한다. 스마트 홈 보드의 U4 인 MCP3208 IC 는 SPI 인터페이스를 사용한다. 이종 컴퓨터간의 데이타 통신을 위해 사용되는 것으로 센서, ADC, 다른 MCU 와의 통신에 많이 사용되며, 마스터 및 슬레이브로 나누어 상호간의 통신을 할 수 있다. 라즈베리파이와 8-Channel 12-Bit A/D Converters 인 MCP3208 을 SPI 기능을 통하여 통신하여, MCP3208 를 이용하여 센서 값을 받아오는 테스트를 해보도록 하겠다. 아래는 대표적인 SPI 인터페이스회로이다.

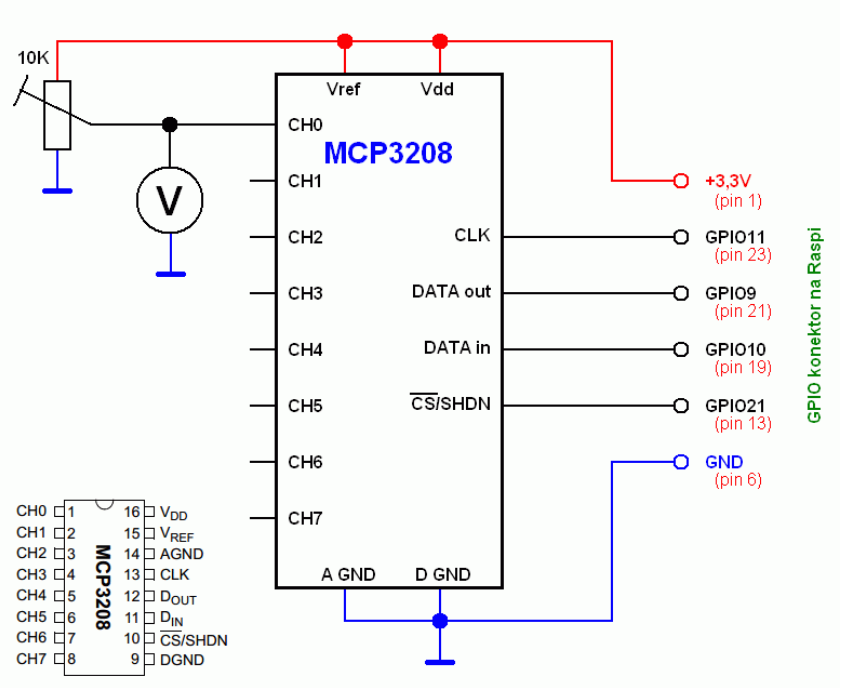


그림 126 아날로그 센서의 ADC 연결도

위와 같이 연결되어 SPI 인터페이스로 사운드 센서의 데이터를 획득할 수 있다. 아날로그 데이터를 얻기 위해 J15 에 연결된 모양을 아래 그림에 나타내었다.



그림 127 사운드 센서 연결도

시스템에서 IO Adapter 보드와 연결된 그림을 보자. 그림에는 디지털 출력과 아날로그 출력이 있는 4 핀 사운드 센서 연결도를 보여준다. 사운드 센서가 디지털 출력만을 지원하는 3 핀 센서라면 J11 ~ J14 커넥터에 연결하면 된다.

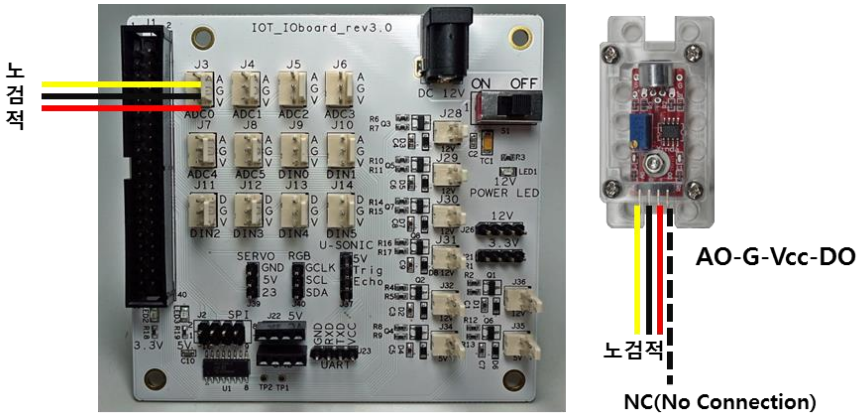


그림 128 사운드 센서와 IO 어댑터 보드의 연결

센서 캘리브레이션 (Calibration, 보정)

다른 센서와 마찬가지로 사운드 센서도 보정을 거친 후 사용해야 한다. 방법은 전원을 연결하면 센서모듈 중앙의 두 개의 LED 중 한 개 전원 LED 에 불이 들어온다. 이때 다른 LED 도 켜져 있다면 항상 신호가 들어오는 것이므로 가변저항을 돌려 약간만 시계, 반시계 방향으로 돌려도 신호 출력 LED 가 꺼지는 상태까지 보정해 주면 된다.

소스코드를 입력하자 (sounddetect.c) 소스를 편집하는 명령은 nano 명령어를 사용한다. $nano sounddetect.c 이다.

```
$nano home-sound.c


#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>
#include <wiringPiSPI.h>

#define CS_MCP3208 8 //GPIO 8
```

```
#define SPI_CHANNEL 0
#define SPI_SPEED 1000000 //1Mhz

// spi communication with Rpi and get sensor data

int read_mcp3208_adc(unsigned char adcChannel)
{
    unsigned char buff[3];
    int adcValue = 0;

    buff[0] = 0x06 | ((adcChannel & 0x07) >> 2);
    buff[1] = ((adcChannel & 0x07) << 6);
    buff[2] = 0x00;

    digitalWrite(CS_MCP3208, 0);
    wiringPiSPIDataRW(SPI_CHANNEL, buff, 3);

    buff[1] = 0x0f & buff[1];
    adcValue = (buff[1] << 8 ) | buff[2];

    digitalWrite(CS_MCP3208, 1);

    return adcValue;
}

int main(void) {

    unsigned char adcChannel_sound = 2;

    int adcValue_sound = 0;      //0번 채널 - 사운드 센서
```

```
    printf("start");

    if(wiringPiSetupGpio() == -1)
    {
            fprintf(stdout, "Unable to start wiringPi :%s₩n", strerror(errno));
            return 1;
    }

    if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
    {
            fprintf(stdout, "wiringPiSPISetup Failed :%s₩n", strerror(errno));
            return 1;
    }

    pinMode(CS_MCP3208, OUTPUT);

    while(1)
    {
            adcValue_sound = read_mcp3208_adc(adcChannel_sound);
            printf("sound = %u₩n", adcValue_sound);
            delay(100);
    }
    return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

$gcc -o sounddetect sounddetect.c -lwiringPi

아래 명령어로 실행하면 아날로그 사운드 데이터가 출력되는 것을 볼 수 있다.

$ sudo ./sounddetect

```
pi@raspberrypi ~/smartsafe$ sudo ./sounddetect


startsound = 2126
sound = 2125
sound = 2126
sound = 2128
sound = 2127
sound = 2126
sound = 2128
sound = 2127
sound = 2128
sound = 2128
sound = 2127
```

모든 센서들의 출력은 아날로그 출력을 기본적으로 가지고 있다. 정밀한 센서값과 필요할 때 언제든지 센서값을 읽어 처리가 가능하다는 면에서 아날로그 출력을 사용하는 장점이 있다. 사용상의 어려움으로 디지털 출력을 선호하고, 디지털 출력을 혼합하여 나오는 센서들이 대부분이다. 아날로그 출력을 얻기 위해서는 위의 방법을 사용한다.

디지털 출력을 사용하는 사운드 센서 실습 1 – 소리가 있나 없나 ?

사운드 센서의 디지털 출력을 이용하는 실습을 진행한다. 앞서 아날로그 출력을 이용하여 실습을 하였는데 아날로그 출력을 가지고 각종 소리의 유무, 소리 크기등을 판단하는 것이 약간은 어려울 수 있겠다고 생각되어 좀 더 쉬운 코드를 제공한다. 아래는 디지털 출력이 있는 사운드 센서 연결도를 나타내었다.

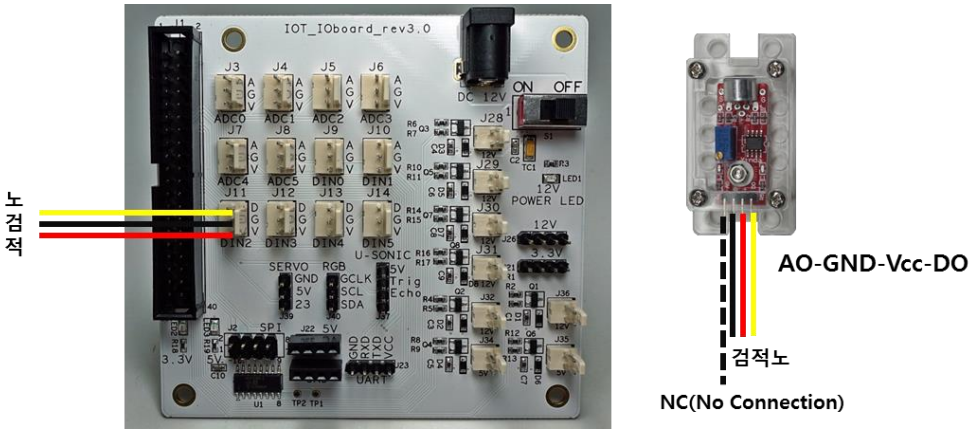


그림 129 디지털 출력이 있는 사운드 센서 연결

소스를 보면 인터럽트 방식으로 SOUND_IN 2 //gpio27 에 할당된 핀이 올라오는 순간에 인터럽트 처리 함수에서 soundFlag = 1; 로 하여 사운드가 있고 없고를 나타낸다. 소스코드를 입력한다. (soundtest.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>

// Use GPIO Pin 7, which is Pin 11 for wiringPi library

#define SOUND_IN 2   //gpio27

unsigned char soundFlag = 0;
// -------------------------------------------------------------
-------------
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
    soundFlag = 1;
```

```
}

// ------------------------------------------------------------
-------------
// main
int main(void) {
    // sets up the wiringPi library
    if (wiringPiSetup () < 0) {
            fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
            return 1;
    }

    // and attach myInterrupt() to the interrupt
    if ( wiringPiISR (SOUND_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
            fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
            return 1;
    }

    while(1)
    {
            if(soundFlag == 1){
                    printf( "SoundDetect ₩n");
                    soundFlag = 0;
            }
            else
            {
                    printf("No Sound ₩n");
                    soundFlag = 0;
            }
            delay(100);
    }
```

```
    return 0;
}
```

핀을 정의하는 부분

```
#define SOUND_IN 2   //gpio27
```

인터럽트 루틴

```
// myInterrupt:   called every time an event occurs
void myInterrupt(void) {
    soundFlag = 1;
}
```

인터럽트 함수 선언과 인터럽트를 받는 방식을 설정

```
if ( wiringPiISR (SOUND_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
                fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
                return 1;
    }
```

무한 루프문

```
while(1)
    {
}
```

에 주의하면서 코딩을 한다.

컴파일 한다. $ gcc -o soundtest soundtest.c -lwiringPi

실행한다. $ sudo ./soundtest

결과를 확인하면 사운드 유무를 100msec 마다 알려준다.

```
$ sudo ./soundtest
No Sound
No Sound
SoundDetect
SoundDetect
```

디지털 출력을 사용하는 사운드 센서 실습 2 - 소리가 큰가 작은가 ?

본 실습에서는 약 0.5 초 동안 펄스(사운드 발생시 증가)를 세어 카운트 숫자가 크면 큰소리가 장기간 발생되고, 펄스 가운터 숫자가 작으면 작은소리가 짧게 발생했다는 것을 알 수 있도록 프로그램 하는 실습을 한다. 연결도는 디지털 사운드 출력을 이용하는 것과 동일하게 연결한다. 소스코드를 나타내었다. 인터럽트 루틴에서 카운터 숫자를 증가시키는 것과 0.5 초마다 사운드 센서 카운터를 디스플레이 하는 것을 반복하는 코드로 구현된다. 소스코드를 입력한다. (soundtest2.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>



// Use GPIO Pin 7, which is Pin 11 for wiringPi library

#define SOUND_IN 2   //gpio27

// the event counter - not use
volatile int eventCounter = 0;
unsigned char soundFlag = 0;
// ------------------------------------------------------------
-------------
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
    eventCounter++;

}
```

```
// ------------------------------------------------------------
-------------
// main
int main(void) {
   // sets up the wiringPi library
   if (wiringPiSetup () < 0) {
        fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
        return 1;
   }

   // set Pin 17/0 generate an interrupt on high-to-low transitions
   // and attach myInterrupt() to the interrupt
   if ( wiringPiISR (SOUND_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
        fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
        return 1;
   }

   // display counter value every second.
   while ( 1 ) {
     printf( "%d₩n", eventCounter );
     eventCounter = 0;
     delay( 500 ); // wait 1 second
   }

   return 0;
}
```

소스코드를 입력하였다면 컴파일 한다.
$gcc -o soundtest2 soundtest2.c -lwiringPi
실행한다. $sudo ./soundtest2

결과를 보면 큰소리 일때와 작은소리 일때의 카운터 숫자가 다르게 나타나는 것을 확인할 수 있다. 소리의 유무와 소리 크기에 따라 제어를 하는 방법은 여러 분야에 응용할 수 있으므로 잘 알아두도록 한다.

```
$sudo ./soundtest2
0
142
215
146
201
0
206
0
105
141
199
268
129
0
0
0
0
126
150
```

(2) 인체 감지센서

인체감지센서는 Pyroelectric ("Passive") InfraRed sensors 라고도 하며 일정한 적외선을 띤 물체가 움직이는 것을 감지하는 센서이며 주로 인체감지에 쓰인다. 적외선을 띤 물체가 감지되어 일정한 양의 적외선을 감지하게 되면, 출력을 내 보내고 적외선을 띤 물체가 없을 때는 출력을 내보내지 않는다. 돔

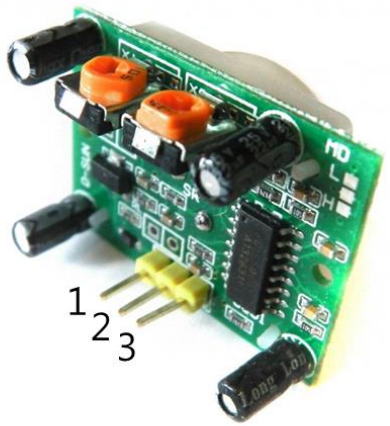
형태의 창(window)는 외부의 환경으로부터 센서 내부의 회로 및 감지부를 보호하고 원하는 파장대의 적외선만 통과시키는 광학 필터의 기능을 한다. 보통은 사람을 감지하여 일정시간 불을 켜는 용도로 사용한다.

표 28 인체감지 센서의 사양

사용 전압	3~5V
사용 전류	50μA
동작 온도 범위	0℃ ~ +70℃
센서 미 감지시 출력(HIGH)	4V
센서 감지시 출력(LOW)	0.4V
감지 각도	110 Degree
감지 거리	7M
Size	28mm×36mm
무게	25g

돔 형태의 창(window)는 외부의 환경으로부터 센서 내부의 회로 및 감지부를 보호하고 원하는 파장대의 적외선만 통과시키는 광학 필터의 기능을 한다. 보통은 사람을 감지하여 일정시간 불을 켜는 용도로 사용한다.

아래는 인체감지 센서의 외관과 Pin 기능을 보여준다.



Pin1 – VCC 전원
Pin2 – Digital Out
Pin3 – GND

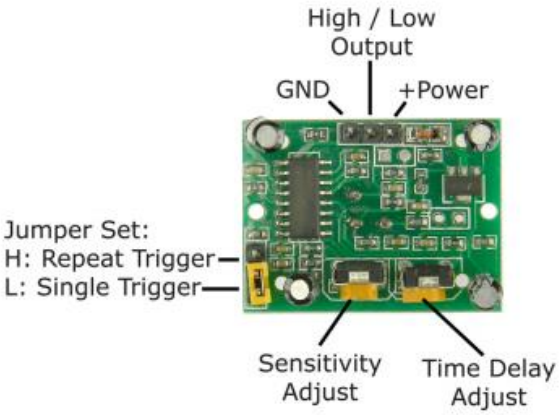
그림 130 인체감지 센서의 외형과 핀 맵

앞장에서의 사운드 센서와 거의 같은 방법으로 인체감지 센서를 테스트 한다. 전등에 사용되는 인체감지 센서는 감지가 된 후 약 5 초 정도 전등을 키고 OFF 하는 방식인데 여기에서 사용한 인체 감지 센서도 같은 방식으로 동작한다. 그 외 필요한 설정은 아래 설정을 보고 시험해야 한다.

Calibration- 센서보정

인체감지 센서를 사용하기 전에 두 가지 컬리브레이션을 진행해야 한다. 아래[그림 8]을 보면 Sensitivity Adjust 와 Time Delay Adjust 가 있다. Sensitivity Adjust 는 인체감지 센서의 민감도를 조절하는 가변저항이고, Time Delay Adjust 인체가 감지되어 출력핀이 하이가 유지되는 시간을 조절하는 가변저항이다.

Sensitivity Adjust 가변저항을 왼쪽 끝까지 돌리면 민감도가 가장 약하고, 오른쪽으로 돌리면 민감도가 증가한다. 실습시에는 **왼쪽 끝**으로 돌려서 민감도를 제일 약하게 하는 것이 편할 것이다. Time Delay Adjust 가변저항은 오른쪽으로 돌리면 하이가 출력되는 시간이 길어지고, 왼쪽으로 돌리면 약 4 초 후에 로우로 떨어지게 된다. 실습에서는 **가장 왼쪽으로** 돌려 일찍 로우상태(대기상태)로 되도록 하는 것이 결과를 확인하기 쉽다.



인체감지 센서 연결을 그림처럼 연결한다.

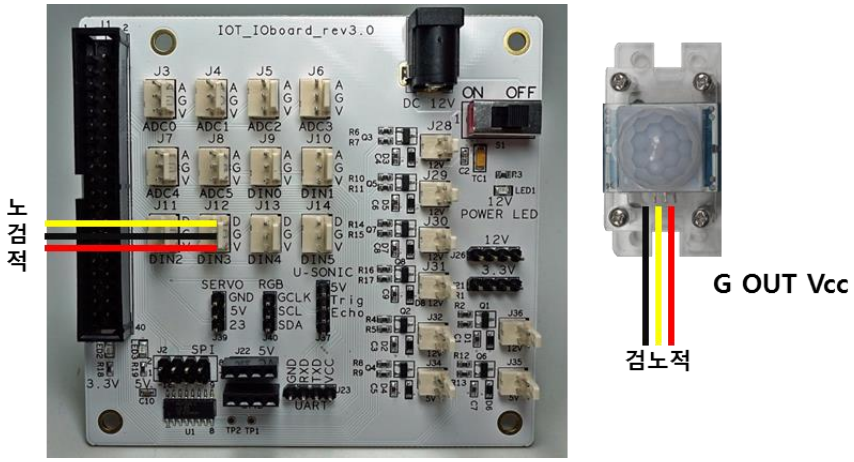


그림 131 인체 감지센서 연결도

센서의 자세한 스펙은 아래 링크에서 다운로드 한다.

(http://www.mpja.com/download/31227sc.pdf)에서 데이터 쉬트를 다운 받을 수 있다. 인체감지 센서를 시험하는 소스코드를 아래에 나타낸다. (motiondetect.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>


// Use GPIO Pin 6, which is Pin 22 for wiringPi library
#define MOTION_IN 0   //gpio17


// the event counter
volatile int eventCounter = 0;
unsigned char flag_detect;
// ------------------------------------------------------------
-------------
```

```
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
    eventCounter++;
    flag_detect = 0x01;
}



// ---------------------------------------------------------------------
// main
int main(void) {
  // sets up the wiringPi library
  if (wiringPiSetup () < 0) {
      fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
      return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  // and attach myInterrupt() to the interrupt
  if ( wiringPiISR (MOTION_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
      fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
      return 1;
  }

  // display counter value every second.
  while ( 1 )
  {
    if(flag_detect == 0x01)
   {
            printf("Motion Detect₩n");
            flag_detect = 0;
```

```
    }
     //eventCounter = 0;
     delay( 100 ); // wait 1 second
   }

   return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

$gcc -o motiondetect motiondetect.c -lwiringPi

컴파일하여 실행하면 Motion Detec 메시지를 볼 수 있다. 인체를 감지하면 약 3~4 초 동안 같은 상태를 유지하고 초기화 되는 식으로 동적하기 때문에 인지가 되어 있는 동안에는 다시 인지를 하지 못한다.

실행 명령어는 $ sudo ./motiondetect 이고, 디텍트가 되면 하이 펄스를 감지하여 보여주는 화면이다.

```
pi@raspberrypi ~/smartguard $ sudo ./motiondetect
 No detect
 No detect
 No detect
 No detect
Detect 1
high
high
high
high
```

(3) 초음파 센서

초음파 센서는 40KHz 의 펄스 신호를 출력하여 반사되어 돌아오는 펄스의 시간을 측정하여 거리를 계산한다. 여기서는 HC-SR04 초음파 센서를 사용한다. 초음파 센서의 자세한 스펙은 데이터 쉬트를 참고한다.

초음파 센서 데이터북 http://www.micropik.com/PDF/HCSR04.pdf

보통 초음파센서에는 4 개의 신호선이 존재하는데 각 각 VCC, Triger, Echo, GND 신호가 된다. VCC 와 GND 는 5v 라인에 연결하고, Triger 핀과 Echo 핀은 라즈베리파이의 GPIO 핀에 연결해준다.아래는 센서의 외관과 핀 맵을 보여준다.



Pin1 – VCC전원
Pin2 – Trigger 신호
Pin3 – Echo 입력신호
Pin4 - GND

그림 132 HC-SR04 초음파 센서의 외형과 핀 맵

HC-SR04 의 전기적 특성을 나타냈는데, 초음파 센서의 입력 전원으로 +5V 를 공급하는것에 주의한다. 출력은 펄스를 받아 계산하므로 센서의 Trigger 신호는 GPIO 핀으로 받게 된다.

표 29 HC-SR04 초음파 센서의 전기적 사양

Working Voltage	DC 5 V
Working Current	15mA
Working Frequency	40Hz
Max Range	4m
Min Range	2cm

MeasuringAngle	15 degree
Trigger Input Signal	적어도 10uS 이상의 TTL pulse
Echo Output Signal	Input TTL lever signal and the range in proportion
Dimension	45*20*15mm

초음파 센서의 타이밍도를 나타낸다.

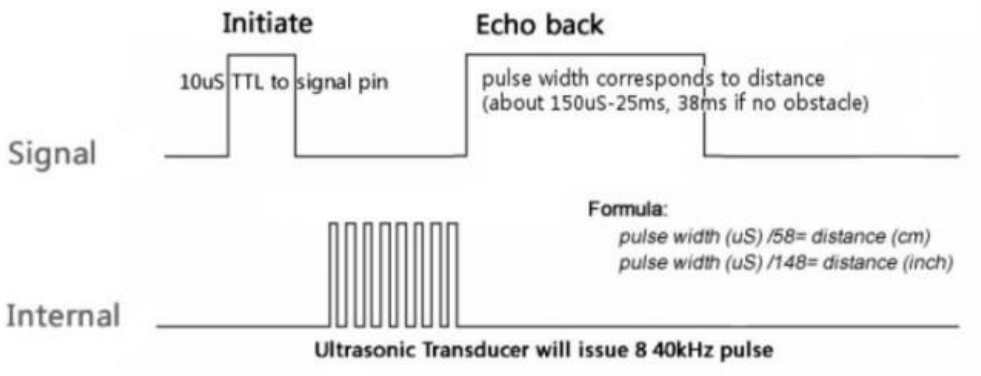


그림 133 초음파 센서의 타이밍도

위 그림은 센서의 작동원리를 설명해주는 그림이다. 우선 triger 핀에 10us 정도의 High 신호를 주면 초음파센서는 40khz 펄스를 자동적으로 8 번을 발생시키고 펄스를 발생시킨 직후에 echo 핀은 high 되고 반사된 초음파가 감지되었을때 echo 핀이 low 가 된다. 이 Echo 핀이 high 였다가 low 가 되는데 걸리는 시간을 측정하여서 그 시간을 초음파의 속도/2(즉 58)로 나누면 거리가 나온다.

초음파 센서를 실습하기 위해 IO adapter 보드의 J37 U-SONICzjsprxjdp 4 핀 케이블을 연결해야 한다.

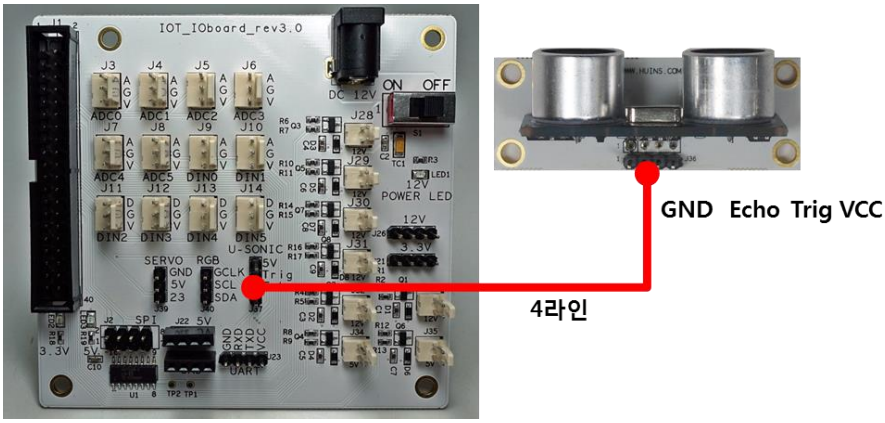


그림 134 초음파 센서와 IO 어댑터 보드 연결도

nano 편집기를 이용하여 소스,파일을 생성한다.

```
$nano ultrasonic.c
```

소스파일 편집기에서 아래 코드를 입력한다. (ultrasonic.c)

```
#include<stdio.h>
#include<wiringPi.h>

#define trigPin 25      //gpio 26, pin 5 of J25
#define echoPin 4       //gpio   J16 - pin3, GPIO 23

int main(void)
{
    int distance=0;
    int pulse = 0;

    long startTime;
    long travelTime;

    if(wiringPiSetup () == -1)
    {
            printf("Unable GPIO Setup");
```

```
        return 1;
    }

    pinMode (trigPin, OUTPUT);
    pinMode (echoPin, INPUT);

    for(;;)
    {
        digitalWrite (trigPin, LOW);
        usleep(2);
        digitalWrite (trigPin, HIGH);
        usleep(20);
        digitalWrite (trigPin, LOW);

        while(digitalRead(echoPin) == LOW);

        startTime = micros();

        while(digitalRead(echoPin) == HIGH);
        travelTime = micros() - startTime;

        distance = travelTime / 58;

        printf( "Distance: %dcm₩n", distance);
        delay(200);
    }
}
```

입력을 마치고 ctrl-x 를 입력하면 파일을 저장할것이냐고 묻는데 이때 y 를 입력한다.

Y 를 입력하고 파일이름을 묻어오면, 이름을 변경하지 않을 경우에는 그대로 Enter 를 치면 빠져 나오게 된다.

약 100usec 의 트리거 신호를 주고, 초음파 센서에서 측정한 거리값을 초 단위로 나타낸다. 이것을 0.2 초(200msec) 에 한번씩 출력하도록 프로그램 하였다.

입력을 다 했다면 컴파일을 수행한다.

```
$gcc –o ultrasonic ultrasonic.c –lwiringPi
```

아래명령으로 실행을 시키고 결과를 확인한다. 프로그램 실행을 중단할때는 ctrl – c 키를 누른다. 손이나 책을 센서 앞에 대면 거리가 변하는 것을 볼 수 있다.

```
$sudo ./ultrasonic
pi@raspberrypi ~/safeguard $ sudo ./ultrasonic
Distance: 268cm
Distance: 268cm
Distance: 266cm
Distance: 267cm
Distance: 269cm
Distance: 269cm
Distance: 268cm
Distance: 266cm
Distance: 266cm
Distance: 10cm
Distance: 7cm
Distance: 8cm
Distance: 8cm
Distance: 11cm
Distance: 293cm
Distance: 31cm
Distance: 266cm
Distance: 267cm
Distance: 268cm
```

11.4 무인경비 시스템 액츄에이터 실습

무인경비 시스템의 액츄에이터는 전등을 켜는 LED Bar, 문을 열고 닫는 DC Motor, 경보를 울리는 부저, 경광등 기능을 하는 RGB Led 로 구성된다. 액츄에이터 실습은 하나씩 개별적으로 수행한 다음 일괄적으로 테스트 하는 과정을 거친다. 실제로 환경에 적용할 때는 각 센서의 값에 따라 정해진 액츄에이터의 동작을 정의해야 한다. 여기서는 액츄에이터의 동작을 확인하는 정도로 테스트를 하고 센서값에 따른 실제적인 동작은 나중에 실습하기로 한다.

프로세스 종료시 주의할 점

액츄에이터 제어 실습시 보통 실험을 종료할 경우 ctrl-c 를 눌러 종료하게 되는데, 이 때 GPIO 신호는 그대로 실습때의 상태를 유지하게 된다. 즉, 전원이 켠 상태나 전등이 ON 상태로 프로그램을 종료해도 전원이 공급되고, 전등은 ON 상태로 남아있게 되는데 이렇게 종료되는 것을 방지하는 방법을 설명한다.

컴퓨터 시스템 상에서 발생할수 있는 상황도 여러가지가 있다. 이는 OS 에 따라 다르게 설정된다. OS 에서 어떠한 시그널을 지원하는 지는 kill 명령을 이용해서 알아볼수 있다. 아래는 라즈베리 파이에서 지원하는 시그널을 캡쳐한 화면이다.

```
pi@raspberrypi ~/safeguard $ kill -l
 1) SIGHUP       2) SIGINT       3) SIGQUIT      4) SIGILL       5) SIGTRAP
 6) SIGABRT      7) SIGBUS       8) SIGFPE       9) SIGKILL     10) SIGUSR1
11) SIGSEGV     12) SIGUSR2     13) SIGPIPE     14) SIGALRM     15) SIGTERM
16) SIGSTKFLT   17) SIGCHLD     18) SIGCONT     19) SIGSTOP     20) SIGTSTP
21) SIGTTIN     22) SIGTTOU     23) SIGURG      24) SIGXCPU     25) SIGXFSZ
26) SIGVTALRM   27) SIGPROF     28) SIGWINCH    29) SIGIO       30) SIGPWR
```

```
 31) SIGSYS       34) SIGRTMIN     35) SIGRTMIN+1   36) SIGRTMIN+2   37) SIGRTMIN+3
 38) SIGRTMIN+4   39) SIGRTMIN+5   40) SIGRTMIN+6   41) SIGRTMIN+7   42) SIGRTMIN+8
 43) SIGRTMIN+9   44) SIGRTMIN+10  45) SIGRTMIN+11  46) SIGRTMIN+12  47) SIGRTMIN+13
 48) SIGRTMIN+14  49) SIGRTMIN+15  50) SIGRTMAX-14  51) SIGRTMAX-13  52) SIGRTMAX-12
 53) SIGRTMAX-11  54) SIGRTMAX-10  55) SIGRTMAX-9   56) SIGRTMAX-8   57) SIGRTMAX-7
 58) SIGRTMAX-6   59) SIGRTMAX-5   60) SIGRTMAX-4   61) SIGRTMAX-3   62) SIGRTMAX-2
 63) SIGRTMAX-1   64) SIGRTMAX
 pi@raspberrypi ~/safeguard $ ^C
```

보통 32 번까지의 시그널은 대부분의 UNIX 계열에서 비슷한 값을 보여주며, 33 번부터 64 번까지는 아마도 OS 마다 약간식의 차이를 보여줄것이다.

각각의 시그널은 int 형의 숫자로 표시되는데, 시그널 번호에 따라서 다른 상황을 나타낸다.

Ctrl-c 를 누른 순간 SIGINT 신호가 발생되는데 우리는 이 신호를 이용하여 정리를 해주고 나오면 된다. 예제로 살펴본다. **(예제: sigint.c)**

```
#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

void sig_handler(int signo); // 마지막 종료 함수

unsigned char endflag = 1;
```

```
int main()
{
      int i = 0;
      signal(SIGINT, (void *)sig_handler);

      while(endflag)
      {
            printf("%d₩n", i);
            i++;
            sleep(1);
      }
      return 1;
}



void sig_handler(int signo)
{
      printf("process stop₩n");
     endflag = 0;
     exit(0);
}
```

실습을 진행하면서 GPIO 핀을 초기상태(low)로 초기화 시키고 빠져나오게 하기 위해 위 코드를 이용하면 될것이다.

무인경비 시스템의 액츄에이터는 부저와 DC 모터, RGB LED, LED Bar4 가지가 있다. 액츄에이터의 사용전원과 기능을 표로 나타내었다.

표 30 무인경비 시스템의 액츄에이터 전원 입력과 기능

액츄에이터	전원	기능
부저	+5V	경고음이나 사이렌 소리를 출력

DC 모터	+12V	모터 회전 – 셔터를 내리는 기능
RGB Led	+12V	적색과 청색의 경광등 효과
LED Bar	+12V	현장의 조명

분리되어 있는 액츄에이터 커넥터를 아래 그림과 같이 연결한다.

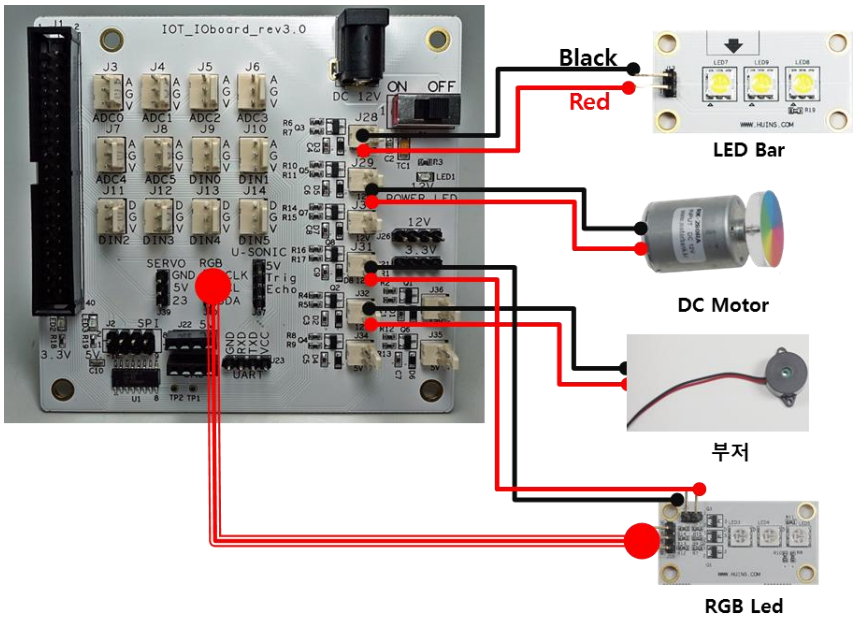


그림 135 무인경비 시스템 액츄에이터 연결도

액츄에이터의 실습은 액츄에이터 하나 하나 개별적으로 하거나 전체를 한번에 진행해도 된다. 액츄에이터 실습을 위한 프로그램 코딩과정을 살펴보자.

1. 핀 연결도와 회로도를 참조하여 IO Adapter 보드와 액츄에이터를 연결한다.
2. 제어하고자 하는 액츄에이터의 커넥터에 전원을 인가하는 GPIO 핀을 확인한다.
3. GPIO 핀에 high, low 로 파워 공급을 제어한다.

(1) LED Bar 실습

LED Bar 는 단순히 화이트 LED3 개를 연결하여 구성한 것이다. 흔히 조명으로 사용되는 LED 들이 대부분 Bar 로 구성되어 있다. Led 소자의 자세한 사양은 데이터북을 참조한다.

(http://www.snowdragonledhk.com/download/5050-smd-led/top-led-5050-smd-white-led-superbright.pdf)

표 31 Led 사양

MODEL N.O	SDS5050WTU-S-AP-A
ENG. N.O	120203002
Shape	8MM Round 5.0*5.4*1.6mm Surface Mount LEDs
Emitting Color	White
Lens Color	Yellow
Viewing Angle	117°
Chip Material	InGaN

LED bar 는 실내조명, 실외조명, 특수조명 등과 같은 많은 응용분야가 있다. 여기서는 실내 조명을 ON/OFF 하는 기능을 한다.



그림 136 LED bar 외관

LED bar 의 연결 회로도를 나타낸다.

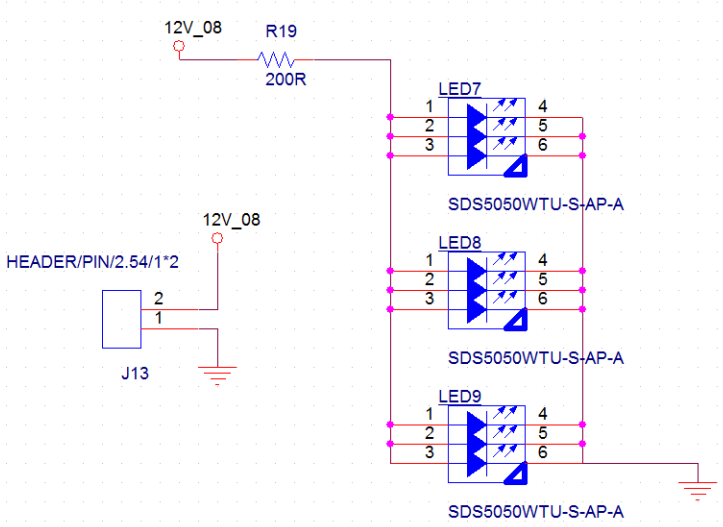


그림 137 LED bar 회로도와 전원공급 커넥터

조명 LED 를 시험 하기 위해 소스 코드를 입력한다. (lightcontrol.c) 이 코드에서는 앞에서 소개한 Signal 을 사용하는 방법을 포함하여 코딩하였다. 실습중 전등 LED 가 켜진채 ctrl-c 로 종료하여도 전등이 켜져있는 일은 없을 것이다. 물론 다른 액츄에이터 제어 프로그램에서도 적용시켜 프로그램 하면 실수로 액츄에이터를 장시간 작동하게 하는 실수를 줄일 수 있다.

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define LIGHTCONTROL   21  //GPIO 5

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
```

```
{
    if(wiringPicheck()) printf("Fail₩n");

    signal(SIGINT, (void *)sig_handler);

    int i;
    while(1)
    {
            printf("here ₩n");

            pinMode (LIGHTCONTROL, OUTPUT);

            digitalWrite(LIGHTCONTROL, 1);
            delay(1000);
            digitalWrite(LIGHTCONTROL, 0);
            delay(1000);



    }

  return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            return 1 ;
    }
}
```

```
void Bpluspinmodeset(void)
{
    pinMode (LIGHTCONTROL, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
     printf("process stop₩n");
    digitalWrite(LIGHTCONTROL, 0);
    exit(0);
}
```

소스를 입력하였으면 아래 명령어로 컴파일을 수행한다
$gcc -o lightcontrol lightcontrol.c -lwiringPi
컴파일 하고 실행하면 조명이 온 오프 되는 것을 확인 할 수 있다.
$sudo ./lightcontrol
실행중 ctrl-c 로 프로그램이 종료되어도 전등이 켜져 있는 일은 없을 것이다.

PWM 으로 LED bar 제어하기

간단한 LED 제어실습으로 softPwm 라이브러리를 사용하여 LED bar 를 제어하는 방법을 알아보자. PWM 은 디지털 기기에서 아날로그 결과를 얻기 위한 테크닉으로, 전원인 on 되는 시간을 펄스 폭이라고 하고, 펄스 폭을 조정함으로 on 과 off 사이의 전압을 흉내낼 수 있다. 이러한 on-off 패턴을 빠르게 반복하여 LED 에 출력하면 LED 의 밝기를 변경 할 수 있게 되는 것이다. softPwm 라이브러리나 헤더파일은 라즈베리 파이 실습시에 설치한 wiringPi 라이브러리가 설치되었다면 pi₩wiringPi₩wiringPi 아래에 있을것이다.

우선 소스코드를 살펴보면 아래와 같다. **(ledbarsoftpwm.c)**

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
#include <string.h>
```

```
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>
#include <softPwm.h>

//#define MOTORCONTROL   22          //GPIO 6 - Motor

#define LEDCONTROL   21// LED Bar
//const int LEDCONTROL 21 //

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
    if(wiringPicheck()) printf("Fail₩n");

    signal(SIGINT, (void *)sig_handler);

    pinMode (LEDCONTROL, OUTPUT);

    softPwmCreate(LEDCONTROL, 0, 100);

    int i;
    int delaytime = 100;
    while(1)
    {
            printf("here ₩n");

            softPwmWrite(LEDCONTROL, 10);
            delay(delaytime);
```

```
                softPwmWrite(LEDCONTROL, 20);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 30);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 40);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 50);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 60);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 70);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 80);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 90);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 100);
                delay(delaytime);
                softPwmWrite(LEDCONTROL, 0);
                delay(delaytime);
    }

  return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
                fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
                exit(1) ;
```

```
    }
}

void Bpluspinmodeset(void)
{
    pinMode (LEDCONTROL, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
     printf("process stop₩n");
    digitalWrite(LEDCONTROL, 0);
    exit(0);
}
```

wiringPi 의 소프트웨어 pwm 으로 사용하기 위해 아래의 두 파일을 include 한다.

```
#include <wiringPi.h>
#include <softPwm.h>
```

softPwm 은 최소 10usec 의 펄스 길이와 100hz 의 PWM frequency 를 제공하므로 모터와 LED 제어를 이해 사용하는데 적당하다.

softPwmCreate 함수를 사용하는데 이 함수는 소프트웨어로 제어되는 PWM 핀을 만든다. 이 함수는 핀번호, 기본값, PWM 값의 범위 이렇게 세가지를 파라미터로 갖는다.

softPwm(use pin, init value, range pwm);

실제로 정의된 핀에 PWM 신호를 출력하기 위해 핀 번호와 값을 인자로 갖는 아래 함수를 사용한다.

softPwmWrite(use pin, pwm value);

현재 pwm 출력값의 변화로 LED Bar 는 밝기가 밝아지고 꺼지고 하는 동작을 반복할 것이다.

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

소스코드를 이해하고 입력을 하였다면 컴파일을 수행한다. 아래와 같은 컴파일 에러가 발생하면 -lpthread 입력시 - 기호에 주의하기 바란다.

```
pi@raspberrypi ~/smartpet $ gcc -o ledbarsoftpwm ledbarsoftpwm.c -lwiringPi -pthread
gcc: error: -pthread: No such file or directory
```

정확히 입력하여 컴파일을 실행한다.

```
$ gcc -o ledbarsoftpwm ledbarsoftpwm.c -lwiringPi -pthread
```

실행을 하면 LED 조명의 밝기가 변화하는 것을 볼 수 있다.

```
$ sudo ./ledbarsoftpwm
```

이러한 PWM 제어는 모터, LED 조명의 밝기제어, RGB Color LED 의 색 제어 등에 많이 사용되므로 잘 알아두면 매우 유용할 것이다

(2) DC Motor 실습

한정된 지면 관계상 DC 모터를 On/Off 하는 예베 코드는 스마트 농장의 DC 모터 실습을 참고하고 여기서는 PWM 방식으로 모터의 회전 수ㅠ를 제어하는 예제를 실습한다.

PWM 으로 DC 모터 제어하기

간단한 제어실습으로 softPwm 라이브러리를 사용하여 DC 모터를 제어하는 방법을 알아보자. PWM 은 디지털 기기에서 아날로그 결과를 얻기 위한 테크닉으로, 전원이 on 되는 시간을 펄스 폭이라고 하고, 펄스 폭을 조정함으로 on 과 off 사이의 전압을 흉내낼 수 있다. 이러한 on-off 패턴을 빠르게 반복하여 DC 모터 전원 제어선에 공급하면 DC 모터의 속도를 여러가지로 변경할 수 있게 된다.

우선 소스코드를 살펴보면 아래와 같다. (motorsoftpwm.c)

```
#include <signal.h> //Signal 사용 헤더파일
#include <stdio.h>
```

```
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>
#include <softPwm.h>

#define MOTORCONTROL   22            //GPIO 6 - Motor

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
    if(wiringPicheck()) printf("Fail₩n");
    signal(SIGINT, (void *)sig_handler);
    pinMode (MOTORCONTROL, OUTPUT);
    softPwmCreate(MOTORCONTROL, 0, 100);

    int i;
    int delaytime = 100;
    while(1)
    {
            printf("here ₩n");

            softPwmWrite(MOTORCONTROL, 5);
            delay(delaytime);
    }

  return 0 ;
}
```

```
int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            exit(1) ;
    }
}

void Bpluspinmodeset(void)
{
    pinMode (MOTORCONTROL, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
     printf("process stop₩n");
    digitalWrite(MOTORCONTROL, 0);
    exit(0);
}
```

wiringPi 의 소프트웨어 pwm 방식으로 사용하기 위해 아래의 두 파일을 include 한다.

```
#include <wiringPi.h>
#include <softPwm.h>
```

softPwm 은 최소 10usec 의 펄스 길이와 100hz 의 PWM frequency 를 제공하므로 모터와 LED 제어를 이해 사용하는데 적당하다.

softPwmCreate 함수를 사용하는데 이 함수는 소프트웨어로 제어되는 PWM 핀을 만든다. 이 함수는 핀번호, 기본값, PWM 값의 범위 이렇게 세가지를 파라미터로 갖는다.

softPwm(use pin, init value, range pwm);

실제로 정으된 핀에 PWM 신호를 출력하기 위해 핀 번호와 값을 인자로 갖는 아래 함수를 사용한다.

softPwmWrite(use pin, pwm value);

현재 pwm 출력값은 5 로 되어있으니 천천이 회전할 것이다. 값이 범위인 100 에 가까울수록 모터의 회전수는 빨라질 것이다.

소스코드를 이해하고 입력을 하였다면 컴파일을 수행한다.

```
pi@raspberrypi ~/safeguard $ gcc -o motorsoftpwm motorsoftpwm.c -lwiringPi
-lpthread
```

실행을 하면 모터가 회전하는 것을 볼 수 있다.

```
pi@raspberrypi ~/safeguard $ sudo ./motorsoftpwm
```

이러한 PWM 제어는 모터, LED 조명의 밝기제어, RGB Color LED 의 색 제어 등에 많이 사용되므로 잘 알아두면 매우 유용할 것이다

(3) Buzzer(부저) 실습

무인 경비 시스템에 사용되는 부저는 침입자를 감지할 경우 싸이렌과 같은 큰 소리를 내는 부저이다. 실제 소리는 귀가 따가울 정도의 소리를 내어 학습에 지장을 초래하므로 여기서는 5V 에 동작하는 피에조 부저를 이용 하기로 한다.



그림 138 피에조 부저 외관

Product ID	KWR-G2910
Rated Voltage	12Vp-p Square Wave
Max.Allowable Voltage	30Vp-p Square Wave
Max.Rated Current	3mA at 1.3KHz/12Vp-p Square Wave
Min.Sound Pressure Level	90dB at 2.9KHz/12Vp-p Square Wave/30cm
Resonant Frequency	2.9 ±0.5KHz
Capacitance at 120Hz	30nF±30%
Operating Temperature	-20~+ 60°C
D(mm)	Ø29.5
H(mm)	24.5
weight	10.2g

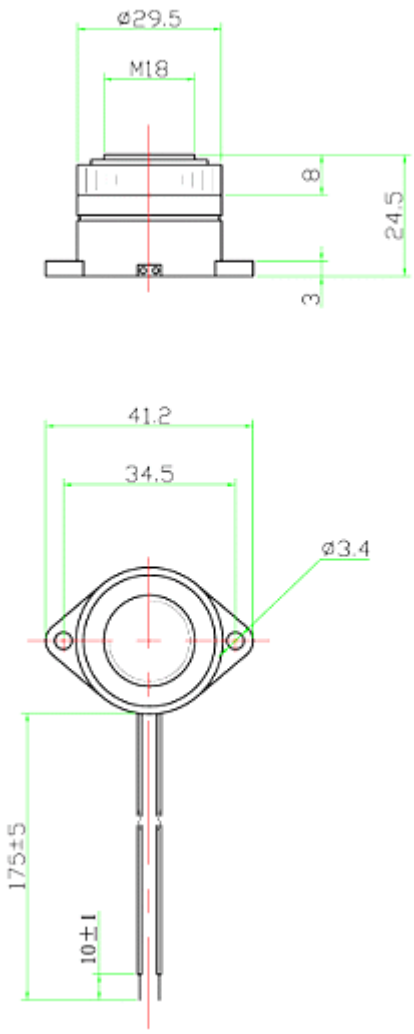


그림 139 피에조 부저 기구도면

부저는 내부의 PIEZO 를 이용하여 소리를 내는 제품이며, 영구자석보다 더 강한 자력에 노출되는 경우, 자력이 손실되어 제품 성능에 이상이 생길수 있다. 방수 제품이 아니므로, 플럭스나 솔벤트 등의 과다 사용으로 이물질이 내부에 침투시에는 소리에 이상이 있을수 있으니 주의하여야 한다.

아래와 같이 소스코드를 입력한다. (buzcontrol.c)

```
#include <signal.h> //Signal 사용 헤더파일
```

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

//#define BUZCONTROL   27   //GPIO 5
#define BUZCONTROL   26     //GPIO 12

void sig_handler(int signo); // SIGINT 핸들러 함수

int main (void)
{
    if(wiringPicheck()) printf("Fail₩n");

    signal(SIGINT, (void *)sig_handler);

    int i;
    while(1)
    {
            printf("here ₩n");

            pinMode (BUZCONTROL, OUTPUT);

            //delay(1000);

            digitalWrite(BUZCONTROL, 1);
            delay(1000);
            digitalWrite(BUZCONTROL, 0);
            delay(1000);
```

```
    }

  return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
                fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
                return 1 ;
    }
}

void Bpluspinmodeset(void)
{
    pinMode (BUZCONTROL, OUTPUT);

}

void sig_handler(int signo) // ctrl-c 로 종료시 실행되는 함수
{
     printf("process stop₩n");
    digitalWrite(BUZCONTROL, 0);
    exit(0);
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

```
$gcc -o buzcontrol buzcontrol.c -lwiringPi
```

실행을 시키면 삐 소리가 들리면 실습이 잘 된것이다.

```
$sudo ./buzcontrol
```

(4) RGB Led 실습

지면 관계상 스마트 농장의 RGB LED 실습을 참고한다. 실습을 위한 소스코드는 pi@raspberrypi ~/smartsafe $ 폴더에 있는데 rgbtest.c 파일은 rgb 색을 교대로 내는 소스코드이다.

컴파일은 $gcc -o rgbtest rgbtest.c -lwiringPi 로 하고

실행명령어는 $sudo ./rgbcolor 이다.

마찬가지로 PWM 방식을 사용하여 모든 색을 내는 소스코드는 pi@raspberrypi ~/smartsafe $ 폴더의 allrgbcolor.c 파일이며 컴파일은 $gcc -o allrgbcolor allrgbcolor.c -lwiringPi 명령어를 사용하고 실행은 $sudo ./allrgbcolor 를 사용한다.

(5) 액츄에이터 전체 실습

아래 코드는 무인경비 시스템의 액츄에이터를 일괄적으로 제어하는 소스이다. (safeactu.c)

```
$nano safeactu.c


#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>

#define WHITELIGHT          21          // BCM_GPIO 5
//#define DCMOTOR  22         // BCM_GPIO 6
```

```
#define DCMOTOR 23                    // BCM_GPIO 13
#define RGBLEDPOWER   24   //BCM_GPIO 19
#define BUZZER 25              //GPIO26

#define RED27                  //GPIO 16
#define GREEN 28     //GPIO 20
#define BLUE          29       //GPIO 21

void Bpluspinmodeset(void);

int main (void)
{
    if(wiringPicheck()) printf("Fail");

    Bpluspinmodeset();

    int i;

  for (i = 0; 1 < 100; i++)
  {
    digitalWrite (WHITELIGHT, 1) ; // On
    digitalWrite(DCMOTOR, 1); //On
    digitalWrite(BUZZER, 1);   //On
   digitalWrite(RGBLEDPOWER, 0);       //On

    delay (5000) ; // ms

    digitalWrite(RED, 1);
    delay(500);
    digitalWrite(RED, 0);
    digitalWrite(GREEN, 1);
```

```
      delay(500);
      digitalWrite(GREEN, 0);
      digitalWrite(BLUE, 1);
      delay(500);
      digitalWrite(BLUE, 0);

      digitalWrite (WHITELIGHT, 0) ; // Off
      digitalWrite(DCMOTOR, 0); //Off
     digitalWrite(BUZZER, 0); //Off
      digitalWrite(RGBLEDPOWER, 0); //Off



      delay (5000) ;
   }
   return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
                fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
                return 1 ;
    }
}

void Bpluspinmodeset(void)
{
    pinMode (BUZZER, OUTPUT);
    pinMode (DCMOTOR, OUTPUT);
```

```
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);


}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

```
$gcc -o safeactu safeactu.c -lwiringPi
```

실행하여 각각의 액츄에이터가 제대로 동작하는지 확인한다.

11.5 무인경비 시스템 시나리오 실습

IoT 무인경비 시스템의 전체 시나리오를 테스트 한다. 앞에서 설명한 전체 구조를 이해하고 센서와 액츄에이터 테스트를 잘 했다면 쉽게 따라 할 수 있을 것이다. 가급적이면 센서와 액츄에이터 연결을 바꾸지 말고 처음 개별적으로 테스트 하던 연결을 유지하고 시험을 진행한다.

외부인 침입의 기준이 되는 초음파 센서나 인체감지, 사운드 센서의 값은 현재 경비가 되고 있는 장소나 환경 환경에 따라 사용자가 센서 기준값을 임의로 정해주어도 상관이 없다. 일단 프로그램이 기동되어 동작을 하게 되면 센서 정보를 일정한 간격으로 수집하게 되며, 인체감지나 음향 센서는 소리나 사람의 움직임과 같은 센서 입력이 발생하면 인터럽트를 작동시켜 사용자에게 알려주거나 액츄에이터를 작동한다. 아래는 무인경비 시스템이 작동하는 전체 시나리오를 간략하게 그려보았다.

이러한 절차는 사용자의 설계에 따라 변형이 가능하고 센서나 액츄에이터의 확장 또한 사용자가 원하는 대로 할 수 있다. 프로그램은 이래 순서도에 따라 작성하여 시험해보자.

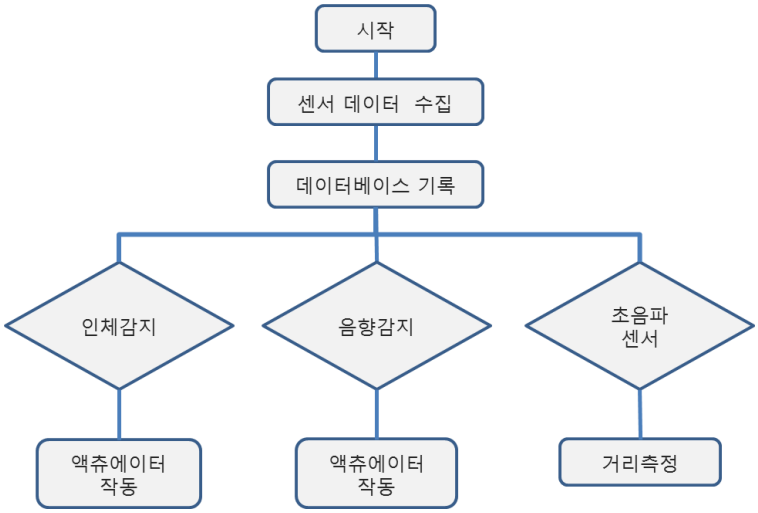


그림 140 무인경비 시스템 시나리오 순서도

11.6 스마트 폰으로 액츄에이터 제어하기

스마트 폰으로 농장 액츄에이터를 제어하느 방법을 알아보자. 물론 연결상태는 현재까지 테스트 하던 상태 그대로 진행하면 되겠다.

(1) 하드웨어 연결도

앞장에서 실습한 센서 테스트, 액츄에이터 테스트 환경과 동일한 연결과 하드웨어를 사용한다.

(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍

안드로이드 앱을 이요한 원격제어를 하기 위해서는 라즈베리 파이가 TCP/IP 의 서버가 되고, 안드로이드 단말기가 TCP/IP 의 클라이언트로서 프로그래밍 해야 한다. 일단 안드로이드 단말기는 클라이언트로서 서버인 라즈베리 파이에 접속한다. 그 후 안드로이드 단말기(클라이언트) 화면의 액츄에이터 제어 버튼을 클릭하면 정해진 문자열을 라즈베리파이(서버)로 전송하고, 라즈베리 파이는 그에 따른 처리를 수행하고 클라이언트에 결과를 전송하는 프로그램을 작성할 것이다.

즉, 서버 프로그래밍(라즈베리 파이), 클라이언트 프로그래밍(안드로이드 단말기)의 두 가지 작업이 필요하다. 안드로이드 단말기를 이용한 TCP/IP 프로그래밍 소스코드는 코드량이 많기 때문에 CD 의 스마트 폰 제어 프로그램 폴더의 코드를 참고하고 여기서는 대략적으로 코딩과정과 순서를 설명한다.

4. 프로젝트 생성
5. string.xml 에 각종 string 을 구성한다.
6. activit_main 을 편집하여 화면을 구성한다.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context=".actu_p2rcontrol" >

    <ToggleButton
        android:id="@+ id/buttonRightTurn"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_above="@+ id/buttonDown"
        android:layout_toRightOf="@+ id/buttonUp"
        android:text="@string/Right"
        android:textOff="Fan Off"
        android:textOn="Fan On" />

    <ToggleButton
        android:id="@+ id/buttonLeftTurn"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
```

```
        android:layout_below="@+ id/buttonUp"
        android:layout_toLeftOf="@+ id/buttonDown"
        android:text="@string/Left"
        android:textOff="Motor Off"
        android:textOn="Motor On" />


    <ToggleButton
        android:id="@+ id/buttonUp"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/textViewStatus"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="37dp"
        android:text="@string/Up"
        android:textOff="펌프 Off"
        android:textOn="펌프 On" />

    <ToggleButton
        android:id="@+ id/buttonDown"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonLeftTurn"
        android:layout_centerHorizontal="true"
        android:text="@string/Down"
        android:textOff="RGB Off"
        android:textOn="RGB On" />

    <Button
        android:id="@+ id/buttonConnect"
        android:layout_width="120dp"
```

```
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_marginTop="66dp"
        android:layout_toLeftOf="@+ id/buttonUp"
        android:text="@string/Connect" />

    <TextView
        android:id="@+ id/textViewStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonConnect"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="20dp"
        android:text="@string/DataView"
        android:textSize="20sp" />

    <EditText
        android:id="@+ id/editTextIPAddress"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+ id/buttonClose"
        android:layout_centerHorizontal="true"
        android:layout_marginBottom="18dp"
        android:ems="10"
        android:gravity="center_vertical|center_horizontal"
        android:inputType="text"
        android:text="@string/IP" />

    <Button
        android:id="@+ id/buttonClose"
        android:layout_width="120dp"
```

```
            android:layout_height="wrap_content"
            android:layout_alignBaseline="@+ id/buttonConnect"
            android:layout_alignBottom="@+ id/buttonConnect"
            android:layout_toRightOf="@+ id/buttonUp"
            android:text="@string/Close" />

</RelativeLayout>
```

클라이언트로 동작하는 안드로이드 프로그램은 3 개의 소스 파일로 구성되었다. 데이터를 보여주는 Logger 와 데이터를 파싱하여 메인 화면을 제어해주는 파일, 그리고 데이처 수신 쓰레드이다. Logger 와 수신 쓰레드는 변경사항이 없으므로 소스를 참고하고 여기서는 액츄에이터를 제어하는 소스를 보여준다.

```
package huins.iot.actucontrol;

import android.os.Bundle;
import android.os.StrictMode;
import android.app.Activity;
import android.content.Context;
import android.graphics.Color;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.view.inputmethod.InputMethodManager;
import android.widget.TextView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ToggleButton;

import java.net.Socket;
import java.io.IOException;
import java.io.OutputStream;
```

```
public class actu_p2rcontrol extends Activity implements OnClickListener{

    private EditText editTextIPAddress;
    private TextView textViewStatus;
    private Button buttonConnect;
    private Button buttonClose;
    private ToggleButton buttonUp;
    private ToggleButton buttonLeftTurn;
    private ToggleButton buttonRightTurn;
    private ToggleButton buttonDown;



    private InputMethodManager imm;
    private String server = "192.168.1.238";
    private int port = 8888;
    //private int port = 8080;
    private Socket socket;

    private OutputStream outs;
    private Thread rcvThread;
    public logger logger;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);

            editTextIPAddress                                    =
(EditText)this.findViewById(R.id.editTextIPAddress);
            editTextIPAddress.setText(server);
```

```
                imm =
(InputMethodManager)getSystemService(Context.INPUT_METHOD_SERVICE);

                textViewStatus = (TextView)this.findViewById(R.id.textViewStatus);
                textViewStatus.setText("스마트팜 액츄에이터 테스트");

                logger = new logger(textViewStatus);

                buttonConnect    = (Button)this.findViewById(R.id.buttonConnect);
                buttonClose      = (Button)this.findViewById(R.id.buttonClose);
                buttonUp         = (ToggleButton)this.findViewById(R.id.buttonUp);
                buttonLeftTurn =
(ToggleButton)this.findViewById(R.id.buttonLeftTurn);
                buttonRightTurn =
(ToggleButton)this.findViewById(R.id.buttonRightTurn);
                buttonDown =
(ToggleButton)this.findViewById(R.id.buttonDown);

                buttonConnect.setOnClickListener(this);
                buttonClose.setOnClickListener(this);
                buttonUp.setOnClickListener(this);
                buttonLeftTurn.setOnClickListener(this);
                buttonRightTurn.setOnClickListener(this);
                buttonDown.setOnClickListener(this);

        }

        @Override
        public boolean onCreateOptionsMenu(Menu menu) {
                getMenuInflater().inflate(R.menu.main, menu);
                return true;
```

```
    }
    @Override
    public void onClick(View arg0) {

            StrictMode.ThreadPolicy policy = new
StrictMode.ThreadPolicy.Builder()
            .detectAll()
            .penaltyLog()
            .build();
            StrictMode.setThreadPolicy(policy);

            if(arg0 == buttonConnect)
            {

    imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

                    try{
                            if(socket!=null)
                            {
                                    socket.close();
                                    socket = null;
                            }

                            server = editTextIPAddress.getText().toString();
                            socket = new Socket(server, port);
                            outs = socket.getOutputStream();

                            rcvThread = new Thread(new rcvthread(logger,
socket));
                    rcvThread.start();
                            logger.log("Connected");
```

```
                } catch (IOException e){
                        logger.log("Fail to connect");
                        e.printStackTrace();
                }
        }

        if(arg0 == buttonClose)
        {

imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

                if(socket!=null)
                {
                        exitFromRunLoop();
                        try{
                                socket.close();
                                socket = null;
                                logger.log("Closed!");
                                rcvThread = null;
                        } catch (IOException e){
                                logger.log("Fail to close");
                                e.printStackTrace();
                        }
                }
        }


        if(arg0 == buttonUp)
        {
                String sndOpkey = "CMD";
```

```
                    if(buttonUp.isChecked()){
                            buttonUp.setTextColor(Color.RED);
                            sndOpkey = "Pumpon";
                    }

                    else{
                            buttonUp.setTextColor(Color.BLACK);
                            sndOpkey = "Pumpoff";
                    }
                    try{
                            outs.write(sndOpkey.getBytes("UTF-8"));
                            outs.flush();
                    } catch (IOException e){
                            logger.log("Fail to send");
                            e.printStackTrace();
                    }
            }

            if(arg0 == buttonLeftTurn)
            {
                    String sndOpkey = "CMD";

                    if(buttonLeftTurn.isChecked()){
                            buttonLeftTurn.setTextColor(Color.RED);
                            sndOpkey = "Motoron";
                    }

                    else{
                            buttonLeftTurn.setTextColor(Color.BLACK);
                            sndOpkey = "Motoroff";
                    }
```

```
                try{
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
                } catch (IOException e){
                        logger.log("Fail to send");
                        e.printStackTrace();
                }
        }

        if(arg0 == buttonRightTurn)
        {
                String sndOpkey = "CMD";

                if(buttonRightTurn.isChecked()){
                        buttonRightTurn.setTextColor(Color.RED);
                        sndOpkey = "Fanon";
                }

                else{
                        buttonRightTurn.setTextColor(Color.BLACK);
                        sndOpkey = "Fanoff";
                }
                try{
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
                } catch (IOException e){
                        logger.log("Fail to send");
                        e.printStackTrace();
                }
        }
```

```
            if(arg0 == buttonDown)
            {
                        String sndOpkey = "CMD";

                        if(buttonDown.isChecked()){
                                    buttonDown.setTextColor(Color.RED);
                                    sndOpkey = "Rgbon";
                        }

                        else{
                                    buttonDown.setTextColor(Color.BLACK);
                                    sndOpkey = "Rgboff";
                        }
                        try{
                                    outs.write(sndOpkey.getBytes("UTF-8"));
                                    outs.flush();
                        } catch (IOException e){
                                    logger.log("Fail to send");
                                    e.printStackTrace();
                        }
            }
}

 void exitFromRunLoop(){
            try {
                        String sndOpkey = "[close]";
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
            } catch (IOException e) {
                        logger.log("Fail to send");
                        e.printStackTrace();
```

```
            }
        }
    }
```

(3) 라즈베리 파이 TCP/IP 서버 프로그래밍

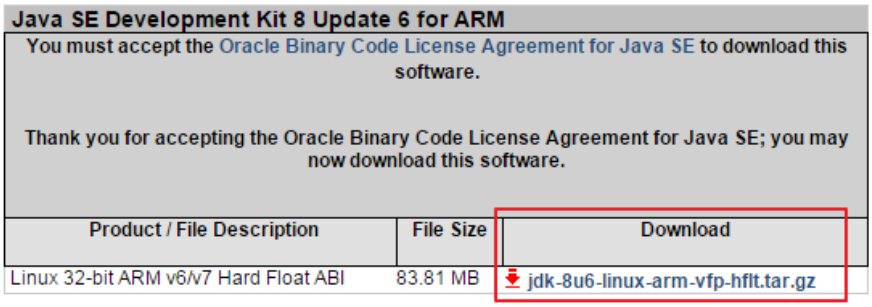
여러가지 언어를 지원하는 TCP/IP 는 언어의 제한을 받지 않는다. 여기서는 참고 코드를 쉽게 구할 수 있고, 많은 예제들이 있는 자바를 사용해서 프로그래밍 하자. 라즈베리 파이에서 자바를 사용하기 위해서는 하드웨어 연산을 지원하는 JDK8 의 설치가 필요하다. JDK8 의 설치는 복잡하지만 어렵지 않으므로 순차적으로 실습해보자.

JDK8 설치에 관해 영문이지만 설명이 잘 되어있다.

http://yoo7577.tistory.com/181

JDK8 을 다운로드 하기 위해서이곳을 방문하고 아래 그림과 같은 파일을 저장한다.

http://www.oracle.com/technetwork/java/javase/downloads/jdk8-arm-downloads-2187472.html



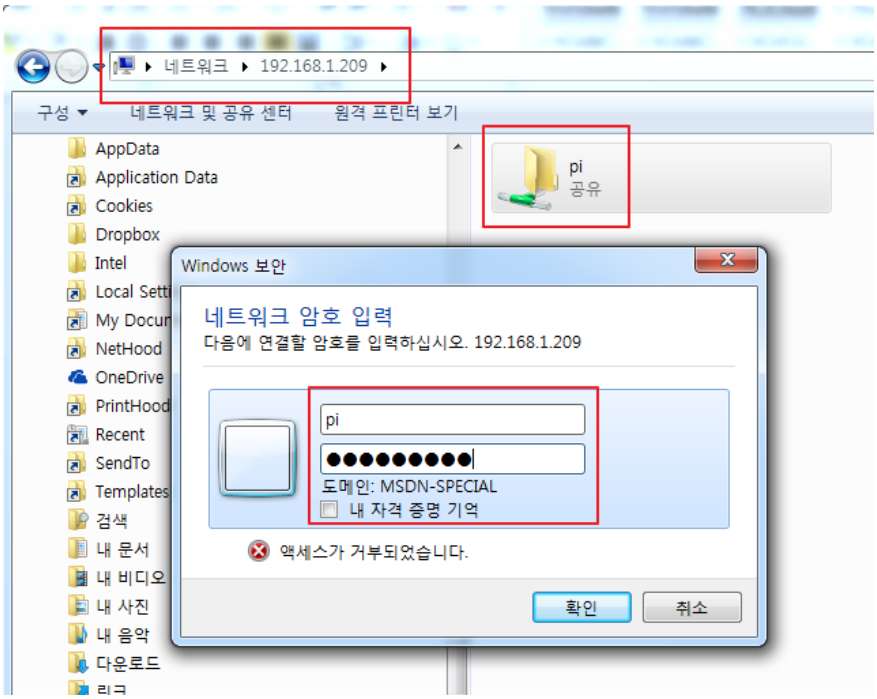
윈도우에서 다운 로드 한 위치를 기억한다.

$ifconfig 명령어로 라즈베리 파이의 IP 주소를 확인하고 네트워크 창을 열어 주소를 입력하여 네트워크로 접속한다.

```
pi@raspberrypi ~ $ ifconfig
eth0      Link encap:Ethernet  HWaddr b8:27:eb:0b:a7:4d
          inet addr:192.168.1.209  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
```

```
RX packets:80528 errors:0 dropped:0 overruns:0 frame:0
TX packets:1942 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:5721058 (5.4 MiB)  TX bytes:216975 (211.8 KiB)
```

아이피 주소는 192.168.1.209 로 확인되었다.



네트워크 창을 열고 아이피 주소를 입력하고 /home/pi/ (작업 폴더임)에 다운받은 파일을 옮기고 아래는 라즈베리 파이에서 작업한다.

아래와 같은 명령어로 /opt/java 라는 폴더를 생성하자.

$sudo mkdir -p -v /opt/java

현재 까지 하면 폴더구조는 아래와 같다.

```
pi@raspberrypi ~ $ cd ..
pi@raspberrypi /home $ cd ..
pi@raspberrypi / $ ls
bin  boot  dev  etc  home  lib  lost+found  media  mnt  opt  proc  root
run  sbin  selinux  srv  sys  tmp  usr  var
pi@raspberrypi / $
```

```
pi@raspberrypi / $ cd opt
pi@raspberrypi /opt $ ls
java  minecraft-pi  sonic-pi  vc  Wolfram
pi@raspberrypi /opt $ cd java
pi@raspberrypi /opt/java $ ls
pi@raspberrypi /opt/java $ cd ..
pi@raspberrypi /opt $ cd ..
pi@raspberrypi / $ cd home
pi@raspberrypi /home $ cd pi
pi@raspberrypi ~ $ pwd
/home/pi
pi@raspberrypi ~ $
```

다운로드한 폴더로 이동한다.

/opt/java 폴더에 다운로드한 파일을 압축해제 한다.

sudo tar xvzf ~/ jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java

```
pi@raspberrypi ~ $ sudo tar xvzf ~/jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java
jdk1.8.0_06/COPYRIGHT
jdk1.8.0_06/LICENSE
jdk1.8.0_06/README.html
jdk1.8.0_06/THIRDPARTYLICENSEREADME-JAVAFX.txt
jdk1.8.0_06/THIRDPARTYLICENSEREADME.txt
jdk1.8.0_06/bin/
jdk1.8.0_06/bin/javah
jdk1.8.0_06/bin/appletviewer
jdk1.8.0_06/bin/extcheck
jdk1.8.0_06/bin/jps
jdk1.8.0_06/bin/wsimport
jdk1.8.0_06/bin/unpack200
jdk1.8.0_06/bin/jarsigner
.
.
```

```
.
```

잠시 후 압축이 풀리고 Oracle JDK gz 파일들이 “/opt/java” 폴더의 아래에 “jdk1.8.0_06”폴더에 위치하게 된다.

다운로드 받은 파일을 더 이상 필요없기에 삭제한다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ rm ~/jdk-8u6-linux-arm-vfp-hflt.gz
pi@raspberrypi ~ $
```

시스템에게 새로운 JDK 가 인스톨 되었고, 위치를 알려주기 위해 아래와 같이 실행한다.

```
pi@raspberrypi ~ $ sudo update-alternatives --install "/usr/bin/java" "java" "/opt/java/jdk1.8.0_06/bin/java" 1
pi@raspberrypi ~ $
```

마지막으로 JDK 가 시스템의 디폴트 java 런타임이 될것이라고 알려준다.

```
pi@raspberrypi ~ $ sudo update-alternatives --set java /opt/java/jdk1.8.0_06/bin/java

pi@raspberrypi ~ $
```

이제 JDK8 이 설치되었다. 자바 버전명령을 사용해 아래와 같이 나온다면 잘 설치된 것이다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ java -version
java version "1.8.0_06"
Java(TM) SE Runtime Environment (build 1.8.0_06-b23)
Java HotSpot(TM) Client VM (build 25.6-b23, mixed mode)
pi@raspberrypi ~ $
```

진짜 마지막으로 JAVA_HOME 환경 변수들을 몇가지 더해야 한다. 우선 bashrc 선언문을 마지막에 추가해 준다.

```
sudo nano ~/.bashrc
```

```
export JAVA_HOME=/opt/jdk1.8.0
export PATH=$PATH:$JAVA_HOME/bin
```

를 마지막에 적어주고 ctrl-x 누르고 y 누르고 Enter 치고 빠져 나온다.

Reboot 를 수행한다. 부팅을 수행하고 나면 Java 샘플 프로그램 작성 및 동작 테스트를 실행하자.

소스코드 생성 및 작성 - nano 편집기를 이용하여 아래와 같은 샘플 코드를 작성하자.

```
$nano HelloWorld.java

public class HelloWorld
{
    public static void main(String args[])
    {
        java.lang.System.out.println("Hello World!!!");
    }
}
```

아래와 같이 컴파일을 수행한다.

```
$javac HelloWorld.java
```

실행한다. Classpath 지정을 잊지 말자.

```
$java -cp . HelloWorld

Hello World!!!
```

메시지가 보이면 설치와 테스트가 완료된 것이다.

여기까지 힘들게 실습하였다. 이제 라즈베리파이에서 JDK8 (with javaFX) 를 사용할 수 있게 되었다. 지금까지 라즈베리파이는 파이썬, C, C++, Java 등 정말

다양한 언어를 사용할 수 있게 되었는데~~ 이것은 단지 툴에 지나지 않다. 이를 이용하여 응용프로그램을 작성해 보도록 하자.

이제 부터는 쉬운 과정이다. 라즈베리파이의 GPIO 를 제어하기 위한 pi4j 라는 자바용 드라이브를 설치한다. pi4j 를 다운로드 하고 설치를 해준다. 아래는 명령어와 설치화면을 보여준다.

$curl -s get.pi4j.com | sudo bash

```
pi@raspberrypi ~ $ curl -s get.pi4j.com | sudo bash
=======================================================
INSTALLING Pi4J GPG PUBLIC KEY
=======================================================
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1761  100  1761    0     0   3134      0 --:--:-- --:--:-- --:--:--  5889
OK
=======================================================
ADDING Pi4J APT REPOSITORY
=======================================================
--2015-01-17 14:46:04--  http://get.pi4j.com/pi4j.list
Resolving get.pi4j.com (get.pi4j.com)... 54.231.17.212
Connecting to get.pi4j.com (get.pi4j.com)|54.231.17.212|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 42 [application/octet-stream]
Saving to: `/etc/apt/sources.list.d/pi4j.list'

100%[=======================================================>] 42          --.-K/s   in 0s
```

```
  2015-01-17 14:46:04 (547 KB/s) - `/etc/apt/sources.list.d/pi4j.list' saved
[42/42]

  ================================================
  UPDATING APT REPOSITORIES
  ================================================
  Hit http://repository.pi4j.com wheezy Release.gpg
  Hit http://repository.pi4j.com wheezy Release
  Hit http://repository.pi4j.com wheezy/rpi armhf Packages
  Ign http://repository.pi4j.com wheezy/rpi Translation-en_GB
  Ign http://repository.pi4j.com wheezy/rpi Translation-en
  Reading package lists... Done
  ================================================
  INSTALLING Pi4J
  ================================================
  Reading package lists... Done
  Building dependency tree
  Reading state information... Done
  The following NEW packages will be installed:
    pi4j
  0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
  Need to get 0 B/517 kB of archives.
  After this operation, 793 kB of additional disk space will be used.
  Selecting previously unselected package pi4j.
  (Reading database ... 87830 files and directories currently installed.)
  Unpacking pi4j (from .../pi4j_1.0~SNAPSHOT_all.deb) ...
  Setting up pi4j (1.0~SNAPSHOT) ...
  ================================================
  Pi4J INSTALLATION COMPLETE
  ================================================
```

```
The Pi4J JAR files are located at:
/opt/pi4j/lib

Example Java programs are located at:
/opt/pi4j/examples

You can compile the examples using this script:
sudo /opt/pi4j/examples/build

Please see http://www.pi4j.com for more information.

pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

위와 같이 설치하고 테스트를 진행할 때 에러나 나는 경우 언인스톨 명령어와 재설치를 아래에 나타낸다. 아래의 명령어로 언인스톨을 실행한다.

$ sudo apt-get remove pi4j

```
pi@raspberrypi ~ $ sudo apt-get remove pi4j
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  pi4j
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
1 not fully installed or removed.
After this operation, 793 kB disk space will be freed.
Do you want to continue [Y/n]? y
(Reading database ... 87881 files and directories currently installed.)
Removing pi4j ...
```

재설치

```
$ curl -s get.pi4j.com | sudo bash
```

안드로이드 스마트 폰(TCP/IP 클라이언트)과의 TCP/IP 통신을 위하여 쓰레드를 통해 데이터를 송수신 한다. 복수의 접속도 허용하고 있으며, 자원의 효율성도 고려하였다.

다음의 코드를 참고한다. farm_actu.java

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Logger;

public class Farmactu {
    private static final int sizeBuf = 50;
    public static void main(String[] args) throws IOException {

        int port = 8888;
        ServerSocket serverSock = new ServerSocket(port);
        Logger logger = Logger.getLogger("Farmactu");

        while (true) {
            Socket clientSock = serverSock.accept();
            SocketAddress clientAddress =
clientSock.getRemoteSocketAddress();
            Thread thread = new Thread(new RcvThread(clientSock,
clientAddress, logger));
            thread.start();
            if(clientSock.isConnected())
                System.out.println("Connected! Client IP : " + clientAddress);
        }
    }
```

TCP/IP 서버로 작동하는 소스코드다. 쓰레드로 동작하는 RcvThread.java 소스코드도 참고한다.

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Level;
import java.util.logging.Logger;

import com.pi4j.io.gpio.GpioController;
import com.pi4j.io.gpio.GpioFactory;
import com.pi4j.io.gpio.GpioPinDigitalOutput;
import com.pi4j.io.gpio.PinState;
import com.pi4j.io.gpio.RaspiPin;
import com.pi4j.io.gpio.impl.PinImpl;   //this!!!!

public class RcvThread implements Runnable{
        private static final int sizeBuf = 50;
        private Socket clientSock;
        private Logger logger;
        private SocketAddress clientAddress;

        final GpioController gpio = GpioFactory.getInstance();

        final                GpioPinDigitalOutput                pinlpwm                =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_21, "lpwm", PinState.LOW);
        final                GpioPinDigitalOutput                pinlfwd                =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_23, "lfwd", PinState.LOW);
```

```
        final GpioPinDigitalOutput pinlrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_22, "lrev", PinState.LOW);

        final GpioPinDigitalOutput rgbpwr =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_24, "rpwm", PinState.LOW);

        final GpioPinDigitalOutput rgbblue =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_27, "rfwd", PinState.LOW);

        final GpioPinDigitalOutput pinrrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_03, "rrev", PinState.LOW);

        public RcvThread(Socket clntSock, SocketAddress clientAddress, Logger
logger) {
            this.clientSock = clntSock;
            this.logger = logger;
            this.clientAddress = clientAddress;
        }

        public void run(){
            try {
                InputStream ins     = clientSock.getInputStream();
                OutputStream outs = clientSock.getOutputStream();

                int rcvBufSize;
                byte[] rcvBuf = new byte[sizeBuf];
                while ((rcvBufSize = ins.read(rcvBuf)) != -1) {

                    String rcvData = new String(rcvBuf, 0, rcvBufSize, "UTF-8");

                                    if (rcvData.compareTo("Pumpon") == 0) {
```

```
                pinlpwm.high();
                System.out.println("Pumpon");
            }
                        if (rcvData.compareTo("Pumpoff") == 0) {
                pinlpwm.low();
                System.out.println("Pumpoff!");
            }


            if (rcvData.compareTo("Motoron") == 0) {
                pinlfwd.high();
                                    System.out.println("Motoron");
                        }
                        if (rcvData.compareTo("Motoroff") == 0) {
                pinlfwd.low();
                System.out.println("Motoroff");
            }


                        if (rcvData.compareTo("Fanon") == 0) {
                pinlrev.high();
                                    System.out.println("Fanon");
                        }
                        if (rcvData.compareTo("Fanoff") == 0) {
                pinlrev.low();
                System.out.println("Fanoff");
            }
                        if (rcvData.compareTo("Rgbon") == 0) {
                rgbpwr.high();
                                    rgbblue.high();
                                    System.out.println("Rgbon");
                        }
```

```
                                    if (rcvData.compareTo("Rgboff") == 0) {
                        rgbpwr.low();
                                            rgbblue.low();
                        System.out.println("Rgboff");
                    }



                logger.info("Received data : " + rcvData + " (" + clientAddress +
")");
                    outs.write(rcvBuf, 0, rcvBufSize);
                }
                logger.info(clientSock.getRemoteSocketAddress() + " Closed");
            } catch (IOException ex) {
                logger.log(Level.WARNING, "Exception in RcvThread", ex);
            } finally {
                try {
                    clientSock.close();
                    System.out.println("Disconnected! Client IP : " + clientAddress);
                } catch (IOException e) {}
            }
        }
    }
```

아래와 같은 명령어로 컴파일을 수행한다.

$javac -classpath .:classes:/opt/pi4j/lib/'*' Farmactu.java

아래와 같은 명령어로 실행

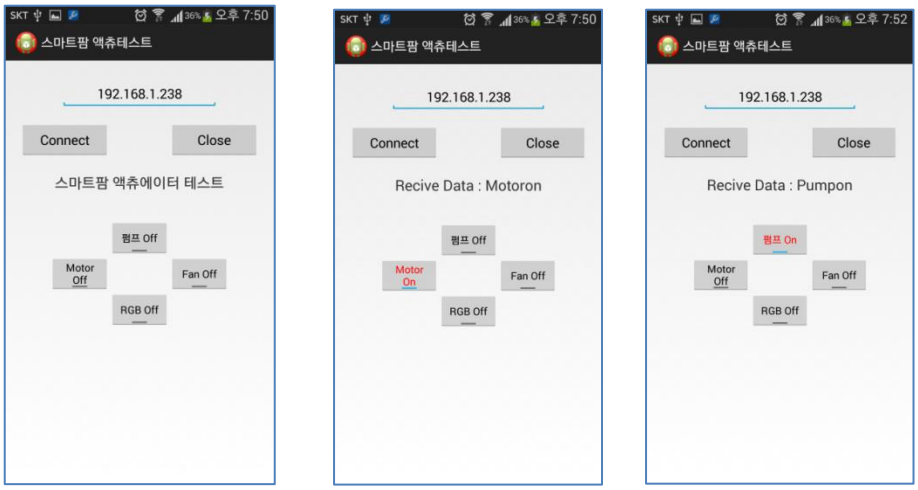
$sudo java -classpath .:classes:/opt/pi4j/lib/'*' Farmactu

라즈베리 파이 화면에서 보여지는 실행화면을 아래에 표시하였다

```
  pi@raspberrypi ~/racar-test $ sudo java -classpath .:classes:/opt/pi4j/lib/'*'
Farmactu
  Connected! Client IP : /192.168.1.98:37403
```

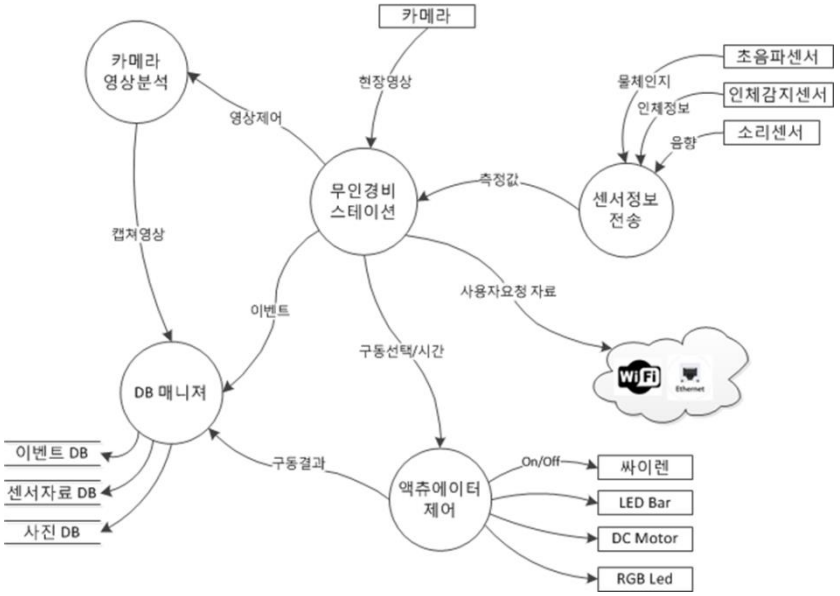
```
Pumpon
Jan 17, 2015 7:53:33 PM RcvThread run
INFO: Received data : Pumpon (/192.168.1.98:37403)
Pumpoff!
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Pumpoff (/192.168.1.98:37403)
Motoron
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Motoron (/192.168.1.98:37403)
Motoroff
Jan 17, 2015 7:53:36 PM RcvThread run
INFO: Received data : Motoroff (/192.168.1.98:37403)
Fanon
Jan 17, 2015 7:53:37 PM RcvThread run
INFO: Received data : Fanon (/192.168.1.98:37403)
Fanoff
Jan 17, 2015 7:53:39 PM RcvThread run
INFO: Received data : Fanoff (/192.168.1.98:37403)
Rgbon
Jan 17, 2015 7:53:39 PM RcvThread run
INFO: Received data : Rgbon (/192.168.1.98:37403)
Rgboff
Jan 17, 2015 7:53:40 PM RcvThread run
INFO: Received data : Rgboff (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: Received data : [close] (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: /192.168.1.98:37403 Closed
Disconnected! Client IP : /192.168.1.98:37403
```

아래는 안드로이드 스마트 폰으로 시험하는 화면을 보여준다.



11.7 무인경비 시스템 서버 프로그램

라즈베리 파이에서 동작하는 서버 프로그램은 무인경비 시스템에 연결된 센서로부터 센서값을 받아 데이터 베이스에 저장한다. 안드로이드 폰에서 동작하는 클라이언트의 요청을 받아 카메라 영상을 전송하고, 액츄에이터를 직접 제어하는 기능을 수행한다. 서버 프로그램의 기능을 DFD 로 나타내었다.



(1) 데이터베이스 설계

일단 데이터 베이스에 대해 알아보자. 앞서 APM(Apache, PHP, MySQL)을 설치하였다. 그것을 바탕으로 데이터 베이스를 활용해보자. 여기서 사용하는 데이터 베이스는 스마트 농장의 모든 센서데이터와 이벤트 기록을 주기적으로, 아니면 발생시 데이터 베이스에 저장한다. 몇 개월이든 농장의 자료를 저장하는 것은 큰 의미가 있는 일이다. 지역의 기후와 농장의 계절 특성을 파악하는 것은 많은 도움이 될것이다. 다른 교재에서 데이터 베이스의 생성과 기초적인 부분들을 살펴보고 자세한 것은 Mysql 관련도서를 참조하자.

데이터 베이스의 설계

스마트 농장에서 데이터 베이스의 설계를 대략적으로 나타낸다. 데이터 베이스의 설계와 테이블 구성은 방대한 작업이므로 아래 실습에서 간단하게 실습하고, 실제 구현시에는 서버 코드를 참조하여 꼼꼼하게 구현한다.

DB Name : iotsafe

Table : tsafesensorvalue 센서값 저장 테이블

1 분마다 센서값 저장한다.

날짜 day / 인체(motion) int / 음향(sound) int / 초음파(ultra) int

Table : tsafesensorcheck 센서 정상/불량 저장 테이블 - 모두 정상

센서 정상 불량 확인하여 저장 10 분마다.

날짜 day / 인체 (정상, 오류) / 음향 (정상, 오류) / 초음파(정상, 오류)

Table : tsafeactuoperate 액츄에이터 동작상태(동작중/정지중)

액츄 동작시 데이터베이스 업데이트 하고, 동작 끝날 때 업데이트

날짜 day / buzzer (on,stop) / dcmotor (on, stop) / rgbled (on, stop) / light (on, stop)

Table : tsafeactucheck 액츄에이터가 정상/불량 저장 데이블

액츄에이터 정상 불량 기록 - 현재는 모두 정상

buzzer(normal, fail), dcmotor(normal, fail), rgbled(normal, fail), light(normal, fail)

Table : tsafesetting 설정값 저장 테이블

Now(), 긴급연락처

여기서는 무인경비 시스템 서버를 실행하기 위해 D 에 들어있는 smartfarm 앱을 실행시킨 상태에서 결과만을 확인하도록 한다.

```
pi@raspberrypi ~ $ cd smartfarm_server
pi@raspberrypi ~/smartfarm_server $
pi@raspberrypi ~/smartfarm_server $ sudo ./server
```

실행을 하고 스마트 폰에서 라즈베리 파이의 IP 주소를 연결한 후 확인하게 되면 다음과 같은 메시지를 보이면서 실행이 된다.

```
pi@raspberrypi ~/smartfarm_server $ sudo ./server
Handling client ip : 192.168.1.98
Handling client port : 56323
Handling client socket number : 5
connected user   : 1
[5Sock][MyUserNum:0]:[0|257X_▒X_]
message parsing start :0|257X_▒X_
 app type : 0x0000
 command : 0x0101
 data :
Data not good, skip
Data not good, skip
Humidity = 17.40 % Temperature = 24.40 *C
Humidity = 17 Temperature = 24
Temperature = 24
Data not good, skip
Humidity = 17.40 % Temperature = 24.40 *C
Humidity = 17 Temperature = 24
Humidity = 17
```

```
Smartfarm_server>server
```

카메라를 확인할 경우 서버에서 카메라 쉘 프로그램이 실행되고 있어야 한다. 따라서 현재 상태에서 Putty 프로그램을 하나 더 실행하여 루트에서 sh mjpg.sh 명령어를 실행한 상태로 실습을 진행한다.

```
pi@raspberrypi ~ $ sh mjpg.sh

MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
 i: camera parameters.....:
.
.
.
```

위와 같이 실행 한 상태에서 스마트 폰의 '농장모습'을 클릭하면 농장의 동영상을 볼 수 있다.

(2) 서버실행과 카메라 영상

서버 역할을 하는 라즈베리 파이에서 실행되는 서버 프로그램은 안드로이드 폰과 TCP 로 통신을 수행하며 사용자 데이터를 분석하여 사용자 요청을 처리한다. 무인경비 시스템의 서버 프로그램은 여러 프로그램으로 구성되어 있다. 서버 프로그램은 프로토콜에 대한 이해와 데이터 베이스등에 대해 잘 알고 있어야 하므로 어려운 작업이다. CD 에 첨부된 서버 프로그램을 분석하고 적용해보는 것은 각자에게 맡기고 여기서는 폴더와 컴파일 실행에 관해 설명한다.

무인경비 스시템의 서버 프로그램 폴더로 이동한다.

```
pi@raspberrypi ~ $ cd iot_safeserver
pi@raspberrypi ~/iot_safeserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_safeserver $
```

아래 명령은 기존에 컴파일 되었던 파일들을 모두 지우는 명령이다. 이렇게 이전 컴파일 자료를 지워야 새로 수정된 프로그램이 컴파일된다.

```
pi@raspberrypi ~/iot_safeserver $ make clean
rm -rf /home/pi/iot_safeserver/output
Clearing in libs...
make[1]: Entering directory '/home/pi/iot_safeserver/libs'
rm -rf iot_db.o parser.o libiot.a core
make[1]: Leaving directory '/home/pi/iot_safeserver/libs'
Clearing in src...
make[1]: Entering directory '/home/pi/iot_safeserver/src'
Clearing in intruder...
make[2]: Entering directory '/home/pi/iot_safeserver/src/intruder'
rm -rf intruder_main.o core
make[2]: Leaving directory '/home/pi/iot_safeserver/src/intruder'
make[1]: Leaving directory '/home/pi/iot_safeserver/src'
```

컴파일이 에러없이 수행되면 다시 make 명령을 사용하여 모든 파일들에 대해 컴파일을 실행한다.

```
pi@raspberrypi ~/iot_safeserver $ make
make all in libs...
make[1]: Entering directory '/home/pi/iot_safeserver/libs'
gcc -c -I/home/pi/iot_safeserver/include -I/usr/include/mysql iot_db.c -o iot_db.o
gcc -c -I/home/pi/iot_safeserver/include -I/usr/include/mysql parser.c -o parser.o
ar rcs libiot.a -o iot_db.o parser.o
make[1]: Leaving directory '/home/pi/iot_safeserver/libs'
make all in src...
make[1]: Entering directory '/home/pi/iot_safeserver/src'
make all in intruder...
make[2]: Entering directory '/home/pi/iot_safeserver/src/intruder'
gcc -g -c -I/home/pi/iot_safeserver/include -c intruder_main.c -o intruder_main.o
gcc -o server_intruder intruder_main.o /home/pi/iot_safeserver/libs/libiot.a -lmysqlclient -lpthread
make[2]: Leaving directory '/home/pi/iot_safeserver/src/intruder'
make[1]: Leaving directory '/home/pi/iot_safeserver/src'
```

컴파일이 되었다면 마지막으로 실행파일을 만들어 준다. $make install 명령을 사용하여 실행파일이 output 폴더 아래에 생성되도록 한다.

```
pi@raspberrypi ~/iot_safeserver $ make install
mkdir /home/pi/iot_safeserver/output
Installing in libs...
make[1]: Entering directory '/home/pi/iot_safeserver/libs'
make[1]: *** No rule to make target 'install'.  Stop.
make[1]: Leaving directory '/home/pi/iot_safeserver/libs'
Installing in src...
```

```
make[1]: Entering directory '/home/pi/iot_safeserver/src'
Installing in intruder...
make[2]: Entering directory '/home/pi/iot_safeserver/src/intruder'
cp -a server_intruder /home/pi/iot_safeserver/output/
make[2]: Leaving directory '/home/pi/iot_safeserver/src/intruder'
make[1]: Leaving directory '/home/pi/iot_safeserver/src'
pi@raspberrypi ~/iot_safeserver $
```

실행파일이 있는 위치를 찾아가보도록 하자.

```
pi@raspberrypi ~/iot_safeserver $ ls
include libs Makefile output src
pi@raspberrypi ~/iot_safeserver $ cd output
pi@raspberrypi ~/iot_safeserver/output $ ls
server_intruder
pi@raspberrypi ~/iot_safeserver/output $
```

output 폴더 아래에 server_intruder 파일을 볼 수 있다. 라즈베리 파이 서버 프로그램을 실행시키면 서버동작의 메시지를 볼 수 있고, 안드로이드 폰과 연동되어 실행된다.

아래 명령으로 실행한다.

$sudo ./server_intruder

위와 같이 서버 프로그램을 실행하면 데이터 베이스를 생성하고 센서 데이터를 수집하는 것을 볼 수 있다.

```
pi@raspberrypi ~/iot_safeserver/output $ $sudo ./server_intruder
validate database...
iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafesensorvalue (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, motion INT, sound INT, ultra INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafesensorcheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, motion INT, sound INT, ultra INT)
```

```
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafeactuoperate (id
INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, buzzer INT,
dcmotor INT, rgbled INT, light INT, sensor INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafeactucheck (id
INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, buzzer INT,
dcmotor INT, rgbled INT, light INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafesetting (id INT
AUTO_INCREMENT PRIMARY KEY, emergency VARCHAR(20))
```

서버를 실행하고 영상을 실시간으로 확인하기 방법은 두가지가 있다.

1. 터미널 화면에서 서버구동을 백그라운드로 실행하는 방법이다. 즉 서버 프로그램 실행 명령에 & 를 추가해 실행을 하면 서버 프로그램이 백그라운드 프로그램으로 실행된다. $sudo ./server_intruder & 명령어이다.
2. 이 상태에서 enter 를 입력하면 $가 나오는데 여기서 동영상 스트리밍 프로그램을 동작시키는 것이다. $sh mjpg.sh 명령을 실행하면 스트리밍 서버가 작동되어 영상을 확인하게 된다.
3. 두 번 째 방법은 다른 컴퓨터에서 putty 를 실행하여 라즈베리 파이에 접속한 후에 $sh mjpg.sh 를 실행하는 방법이다.

11.8 무인경비 시스템 안드로이드 앱 프로그램

무인경비 시스템과 연동하는 안드로이드 앱 프로그램의 Data Flow Diagram 을 아래에 나타내었다. 안드로이드 앱은 Wi-Fi 를 통해 무인경비 시스템과 연결되고 사용자의 요구에 따라 실시간 영상을 스트리밍하거나 센서데이터를 제공하고 사용자는 직접 액츄에이터를 제어할 수 있다.

이 장에서는 안드로이드 앱과 스마트 농장 관리 시스템의 연동에 대해 설명한다. 안드로이드 어플리케이션은 클라이언트 프로그램으로 동작한다. 라즈베리 파이에서는 앞 장에서 구현한 서버 프로그램과 동영상 스트림 쉘이 수행되는 상태에서 스마트 폰의 “침입자감시” 어플리케이션을 실행시켜 동작 상태를 확인하고, 다양한 액츄에이터를 구동한다.

스마트 농장관리 시스템의 스마트 폰 앱은 안드로이드 OS 기반의 스마튼 폰에서 수행되는 Application 으로 아래와 같은 기능을 가지고 있다.

○ 센서 – 인체감지센서, 음향 센서, 초음파 센서 데이터 수집
○ 사람이 감지되면 즉시 사용자에게 침입자를 알림
○ 현장의 상태를 실시간으로 동영상 스트리밍으로 전송
○ 침입자 방생시 액츄에이터 – 부저, DC 모터, RGB LED, 전등을 가동
○ 사용자가 폰을 사용하여 원격으로 액츄에이터 제어

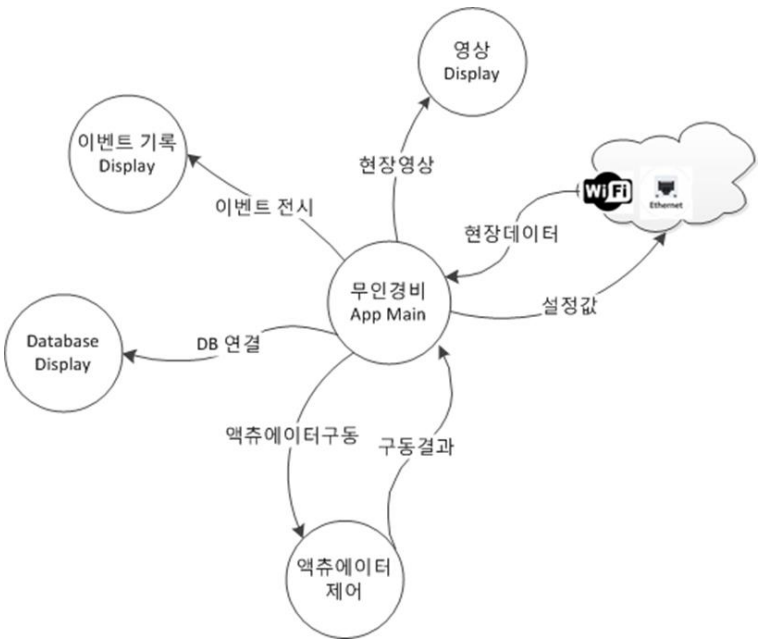


그림 141 스마트 무인경비 시스템 앱 프로그램 구조

여기서는 프로그램 구성을 설명하고 스마트 폰 앱을 실행하여 동영상을 확인하고 다양한 액츄에이터를 제어하는 실습을 진행한다. 소스코드의 구성은 사용자 스스로 분석하여 다양한 환경에 적용하도록 하자.

아래에 무인경비 시스템의 서버와 앱이 연동되는 테스트 과정을 설명한다. 이대로 시험하면 결과를 확인할 수 있고 나머지는 참고로 이용하길 바란다.

1. 라즈베리 파이 전원을 켜고, IO 어댑터 보드의 12V 전원을 켠다.
2. 로그인 아이디 : pi password : raspberry (화면에 나타나지 않음)
3. 라즈베리 파이 아이피 주소 : $ifconfig 명령어로 확인한다.

예)192.168.1.219

4. 라즈베리 파이 스마트 팜 서버 실행

```
pi@raspberrypi ~ $ cd iot_safeserver
pi@raspberrypi ~/iot_safeserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_safeserver $ cd output
pi@raspberrypi ~/iot_safeserver/output $ ls
mjpg.sh  server_intruder
pi@raspberrypi ~/iot_safeserver/output $ sudo ./server_intruder &
```

(백그라운드 실행) 엔터 입력하여 터미널 화면에서 실행

5. 동영상 스트리밍 서버 실행 : $sh mjpg.sh

6. 스마트 폰 앱 실행 pi@raspberrypi ~ $ **sh mjpg.sh**

이 과정을 통해 연동 테스트를 수행한다.

아래는 서버 프로그램을 실행시킨 화면이다. 서버는 인체감지 센서, 음향센서, 초음파 센서 데이터를 수집하여 데이터 베이스에 저장하는 한편 클라이언트가 되는 스마트 폰에서 접속하기를 기다린다. 따라서 터미널 화면에서 서버를 수행시키면 다른 컴퓨터에서 putty 를 실행시켜 동영상 스트리밍을 서비스 하는 프로그램을 실행하는 방법으로 실습하여야 한다.

```
pi@raspberrypi ~/iot_safeserver/output $ sudo ./server_intruder
validate database...
iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafesensorvalue (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, motion INT, sound INT, ultra INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafesensorcheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, motion INT, sound INT, ultra INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafeactuoperate (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, buzzer INT, dcmotor INT, rgbled INT, light INT, sensor INT)
```

```
iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafeactucheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, buzzer INT, dcmotor INT, rgbled INT, light INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tsafesetting (id INT AUTO_INCREMENT PRIMARY KEY, emergency VARCHAR(20))
done
iot_count_setting_data_from_db : query - SELECT COUNT(*) FROM tsafesetting
iot_count_setting_data_from_db data: 1
validate setting data... : 1
starting server...
SensorInterruptLoop : write to DB - INSERT INTO  tsafesensorvalue values (null ,now(), now(), 1, 1, 1)
iot_insert_data_from_db : query - INSERT INTO  tsafesensorvalue values (null ,now(), now(), 1, 1, 1)
SensorInterruptLoop : write to DB - INSERT into tsafesensorcheck values (null, now(), now(), 1, 1, 1)
iot_insert_data_from_db : query - INSERT into tsafesensorcheck values (null, now(), now(), 1, 1, 1)
SensorInterruptLoop : write to DB - INSERT into tsafeactuoperate values (null, now(), now(), 0, 0, 0, 0, 0)
iot_insert_data_from_db : query - INSERT into tsafeactuoperate values (null, now(), now(), 0, 0, 0, 0, 0)
```

이 상태에서 putty 를 실행시키고, 라즈베리 파이에 접속한다. Putty 를 실행하는 이유는 위에 서버를 실행하고 다시 동영상 스트리밍 서버를 실행하기 위함인데 위에 설명한 $ sudo ./server_farm & 명령(뒤에 &를 붙여서)으로 백그라운드로 실행하게 하면 터미널에서 엔터를 치면 명령줄이 나타나므로 putty 프로그램을 실행할 필요는 없다.

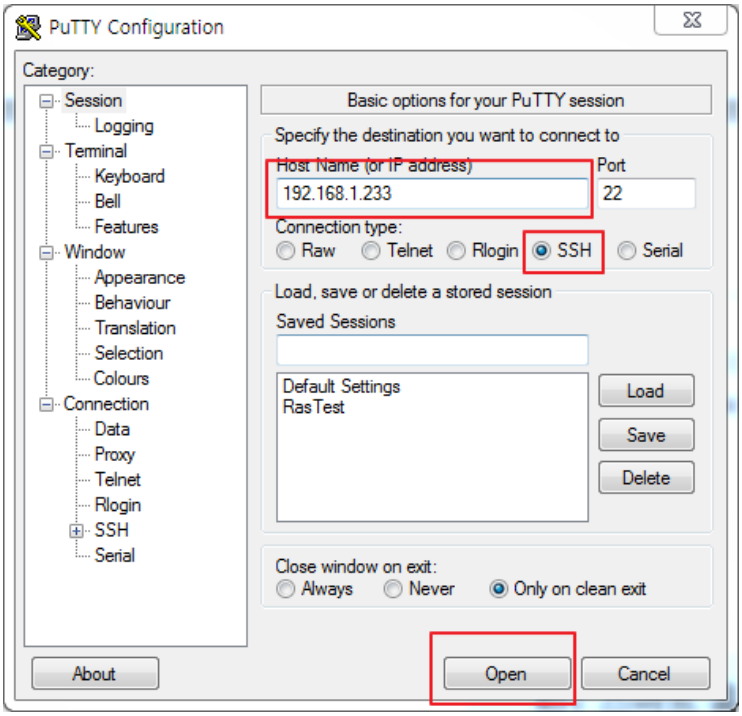


그림 142 동영상 스트림 서버를 실행시키기 위한 Putty 실행화면

아이디와 패스워드를 넣어 접속한 화면은 아래와 같다.

```
login as:
login as: pi
pi@192.168.1.219's password:
Linux raspberrypi 3.18.7+ #755 PREEMPT Thu Feb 12 17:14:31 GMT 2015
armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 09:47:01 2015
```

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

홈 디렉토리에서 mjpg.sh 파일을 확인하고 다음의 명령어로 mjpg 스트림 프로그램을 실행한다.

$sh mjpg.sh

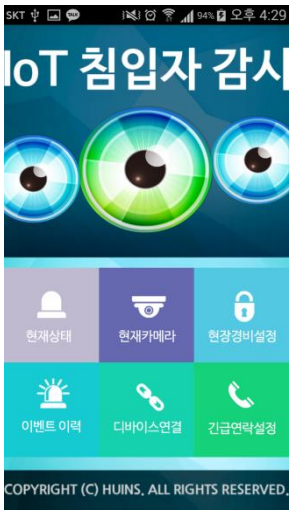
```
pi@raspberrypi ~ $ sh mjpg.sh
MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
 i: camera parameters.....:
.
.
.
.
```

안드로이드 앱이 실행되는 모습을 살펴보자. 아래는 무인경비 시스템 안드로이드 앱의 메인 화면이다. 각 메뉴의 기능을 간단하게 살펴보면 아래와 같다.

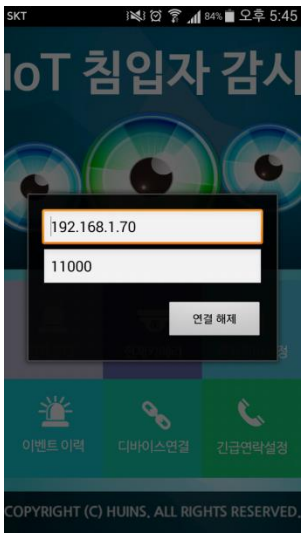
○ 현재상태 : 현재 무인경비 시스템의 정상 동작 상태

○ 현재 카메라 : 무인경비 시스템 카메라 영상을 전송
○ 현장경비 설정 : 경비 액츄에이터 동작 설정(근무시는 OFF)
○ 이벤트 이력 : 이벤트 발생, 동작 이력사항등 로그기록 전시
○ 디바이스 연결 : 처음 실행시 서버에 연결 기능
○ 긴급연락설정 : 침입자 발생시 긴급 연락 전화번호 설정 저장

앱이 실행하면서 보여주는 화면을 간단하게 설명한다.



침입자 감시 앱을 실행할때의 메인 화면



서버로 동작하는 라즈베리 파이에 연결하기 위하여 “디바이스 연결”을 선택하여야 한다.

같은 네트워크 내에 존재하는 서버의 IP 주소를 입력하여 연결을 수행한다.



현재 상태를 확인하는 화면이다. 무인경비 시스템의 센서 상태와 액츄에이터의 상태를 나타낸다. 센서와 액츄에이터의 상태가 이상이 있거나 고장이 발생할 경우 이상 있음을 나타낸다.



무인경비 시스템이 설치된 현장의 영상을 실시간 스트리밍으로 서비해 주는 화면이다. 서버에서는 mjpg 스트리밍 서버가 실행되고 있어야 화면이 정상적으로 재생된다. 약간의 시간지연이 발생하지만 화질은 선명한 편이다.



무인경비 시스템의 액츄에이터와 경비가 실행을 설정하는 화면이다. 각각의 액츄에이터를 직접 제어할 수 있고, 무인 경비 시스템의 동작을 OFF(근무중일 경우) 할 수 있다. 긴급연락 설정은 침입자가 감지될 경우 통화를 할 수 있는 연락처를 설정하게 된다.



침입자가 발생될 경우 긴급 경보를 나타내는 화면이다. 침입자 발생을 알리고 전화번호가 나오게 되어 긴급하게 연락을 취할 수 있게 해준다. 발생된 이벤트를 조회할 수 있도록 이벤트 이력 메뉴가 있다.

이상으로 무인경비 시스템의 실습을 마친다.

Chapter 12 IoT 화재경보 시스템 실습

12.1 화재경보 시스템 개요

IoT 화재경보 시스템은 화재 모니터링이 필요한 학교/빌딩/가정/공장/상가 등에서 사용할 수 있는 시스템으로 화재발생시 실시간으로 센서 정보 및 영상정보를 수집하여 스마트폰으로 전송한다. Smartphone 을 통해 현장을 확인하거나 시설물(Actuator)을 원격제어 및 관리할 수 있는 ARM11 Raspberry 기반의 IoT 실습용 Starter Kit 이다.

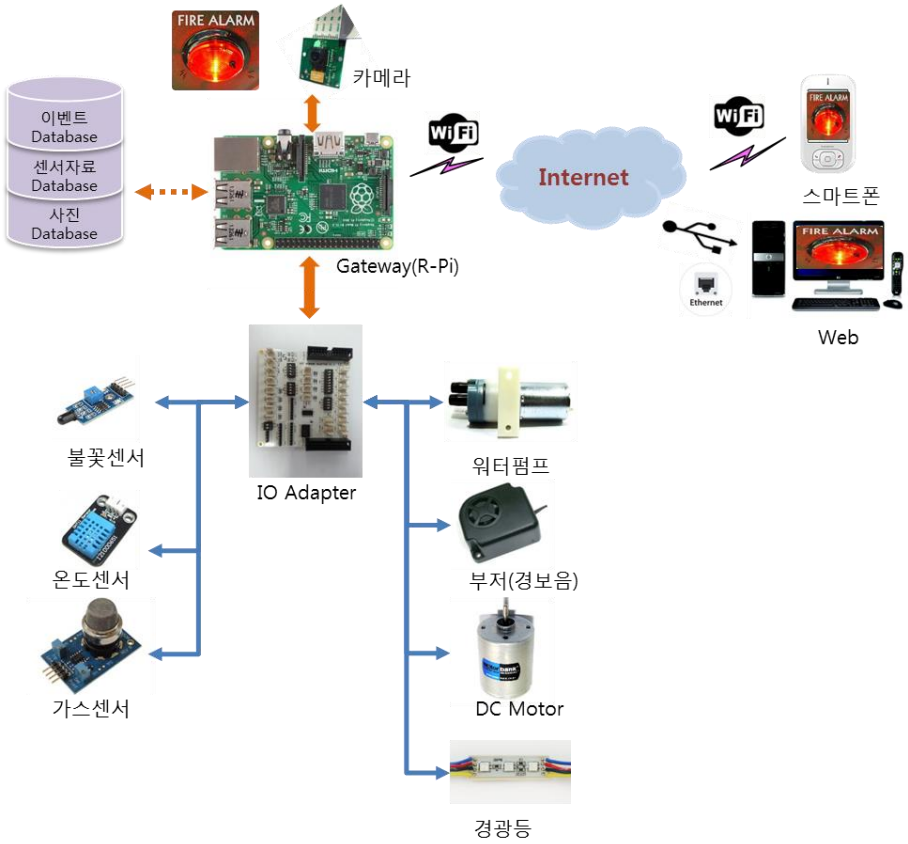


그림 143 화재경보 시스템 구성

화재경보 시스템의 주요기능은 다음과 같다.

○ 스마트폰을 이용한 스마트 화재경보시스템 Actuator 의 원격제어

○ 센서에 의한 화재발생 감지 시 가청(싸이렌) 경보 알림

○ 화재발생 감지 시 물펌프 동작
○ 센서에 의한 화재발생 감지 시 방화벽(모터) 동작
○ 온도, 불꽃, 가스 정보를 이용한 화재 위험도의 표시기능
○ 원격영상을 통한 실시간 모니터링 및 Actuator 의 원격제어

(1) 화재경보 시스템 하드웨어

Gateway 기능을 수행하는 하는 라즈베리 파이, 카메라, IO Adapter, 센서 3 종(온습도 센서, 불꽃센서, 가스센서), 액츄에이터 4 종(부저 싸이렌, DC Motor, 워터펌프, RGB 경광등)으로 구성되어 있다. 센서는 다른 센서데이터가 필요할 경우 얼마든지 확장될 수 있고, 액츄에이터 또한 사용자의 설계 의도에 맞게 추가하여 구성할 수 있다. 여기에서는 가스센서와 불꽃 센서의 데이터가 화재를 감지할 경우 싸이렌을 울리고 경광등을 켜고 워터펌프와 DC 모터가 동작하는것으로 설정한다.

앞 장의 실습 과정에서 설명한 라즈베리 파이와 IO 어댑터 보드, 카메라 부분에 대한 설명은 생략하기로 한다.

표 32 화재경보 시스템 하드웨어 구성표

구성품	사양	기능
센서	온습도센서	실내의 온도와 습도데이터
	불꽃센서	불이나 라이터에 반응하는 센서
	가스센서	부탄, LNG, LPG 가스 감지
Actuator	워터펌프	물을 뿜어내어 불을 끄는 기능
	싸이렌 부저	싸이렌을 울리는 부저기능
	Dc Motor	셔터 내리는 기능
	RGB LED	적색/파랑색 경광등의 기능
전원어댑터	DC 12V, 3A 전원 어댑터	DC 12V 전원공급
	DC5V, 1A 전원 어댑터	DC 5V 전원공급

(2) 화재경보 시스템 소프트웨어

IoT 화재경보 시스템의 Software 는 크게 두부분으로 구성된다. 라즈베리 파이에서 수행되는 게이트웨이 소프트웨어는 센서로부터 데이터를 획득하고 저장하며 원격지 사용자 요청에 따라 액츄에이터를 동작시키며 카메라 영상을 전송하는 기능을 한다. 게이트 웨이 시스템과 스마트폰 Application 양 쪽에서 수행되는 Software 의 전체 구조를 살펴보면 아래와 같다.

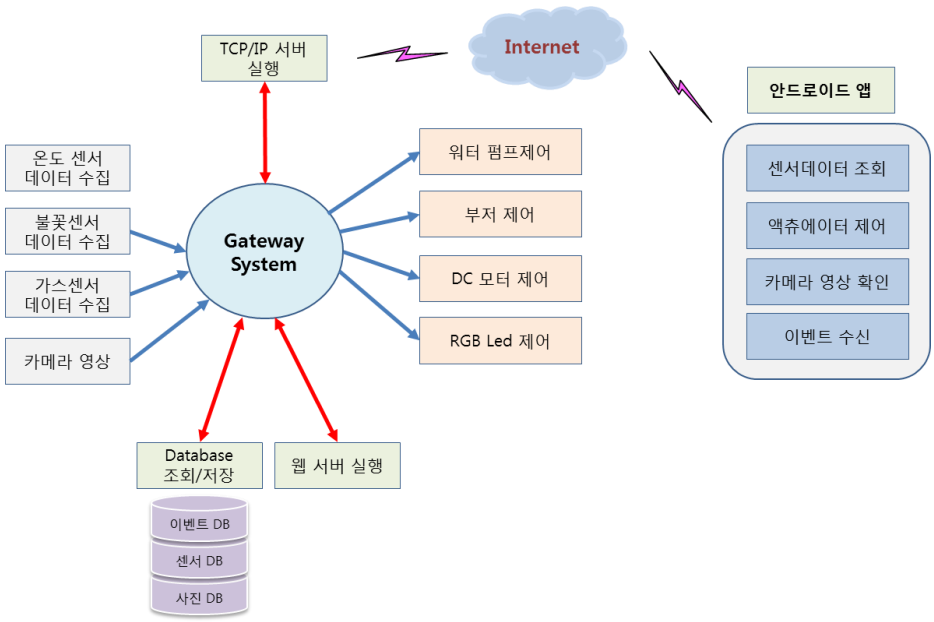


그림 144 화재경보 시스템 소프트웨어 구조

스마트 폰에서 동작하는 안드로이드 앱은 원격지의 영상을 실시간으로 확인하고, 화재 감시 현장의 센서데이터를 조회하며, 환경에 따라 액츄에이터를 직접 제어하는 기능을 한다. 카메라 구동 드라이버는 이미 게이트웨이에 포함되어 있으며 안드로이드 폰과 연동시 게이트 웨이의 서버 프로그램이 수행된다. Gateway 역할을 하는 라즈베리 파이는 리눅스 운영체제가 설치되어 있다. 센서로부터 수집된 데이터를 날짜 시간 데이터와 함께 데이터 베이스에 저장하고, 원격지의 사용자 요청에 따라 카메라 영상을 송신하고 설치된 액츄에이터를 제어하는 기능을 한다.

연결의 정확성

센서와 액츄에이터 보드의 연결도는 실습 후에 스마트 폰으로 입력을 확인하고 액츄에이터를 제어할 때까지 동일한 연결과 테스트를 수행해야 한다. 개별 센서와 액츄에이터 실습시에는 센서 데이터와 액츄에이터 제어가 올라보 되는지를 확인하고 실제 상황을 스마트 폰과 연동 실습할때에는 모든 연결이 정상적으로 정확하게 연결되어야 실습이 진행된다.

12.2 화재경보 시스템 연결

화재경보 시스템의 하드웨어 연결에 대해 설명한다. 화재경보 시스템의 센서는 불꽃 센서, 온습도 센서, 가스센서 3 종류가 사용된다. 사용자의 설계 개념에 따라 다른 센서를 확장하여 연결할 수 있다. 불꽃을 감지하고, 부탄 가스등의 가스를 감지하고, 현재 실내의 온도를 측정하는 기능을 한다. 하나의 센서값만 사용되지 않고 여러가지 센서의 결과값을 종합하여 판단하게 된다.

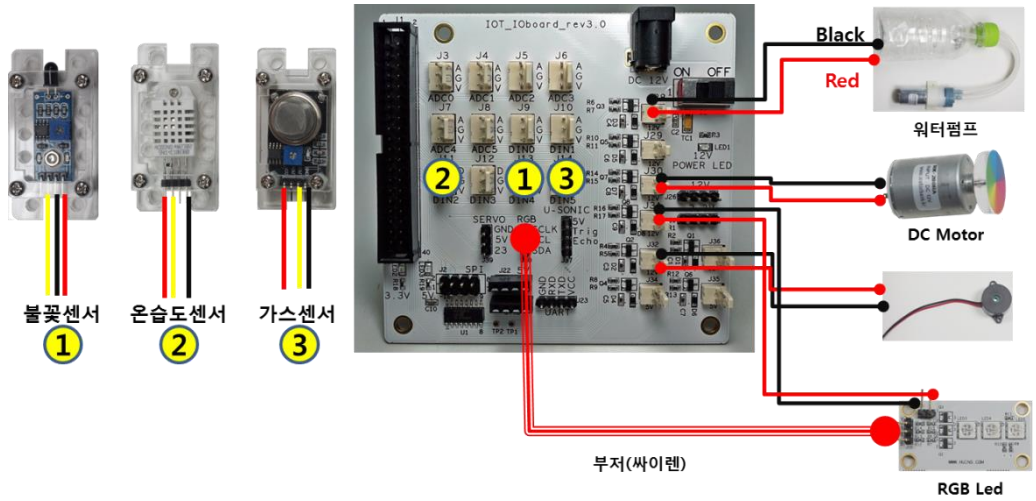


그림 145 화재경보 시스템 센서와 액츄에이터 연결도

화재 경보 시스템의 액츄에이터는 워터펌프, DC Motor, 부저(싸이렌), RGB Led 를 사용한다.

라즈베리 파이 카메라 모듈은 현장의 영상을 실시간으로 전송하게 된다. 센서에서 수집된 데이터는 서버의 데이터 베이스에 저장되고, 화재가 감지되면 사용자의 모바일 기기로 화재 발생을 즉시 전송한다.

[그림 144] 는 화재경보 시스템의 센서와 액츄에이터의 연결도를 나타낸다. 위와 같이 연결한 상태에서 센서와 액츄에이터 실습을 진행한다.

12.3 화재경보 시스템 센서 실습

해재경보 시스템의 센서 실습을 위해 아래 [그림 101]과 같이 연결한다. 아날로그 출력값을 사용하려면 다른 곳에 연결해야 하는데 여기서는 불꽃의 존재 유무와 가스 감지 순간을 인터럽트를 이용해서 감지하는 실습을 하게 된다.

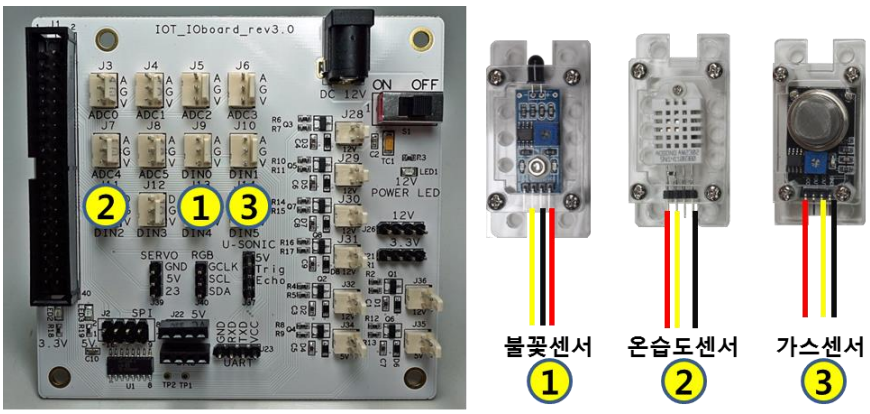


그림 146 화재경보시스템 센서 연결도

화재경보 시스템은 3 종류의 센서를 사용한다. 위와 같이 연결하고 실습을 진행한다.

(1) 불꽃 센서(Flame sensor)

화재, 가스 주의

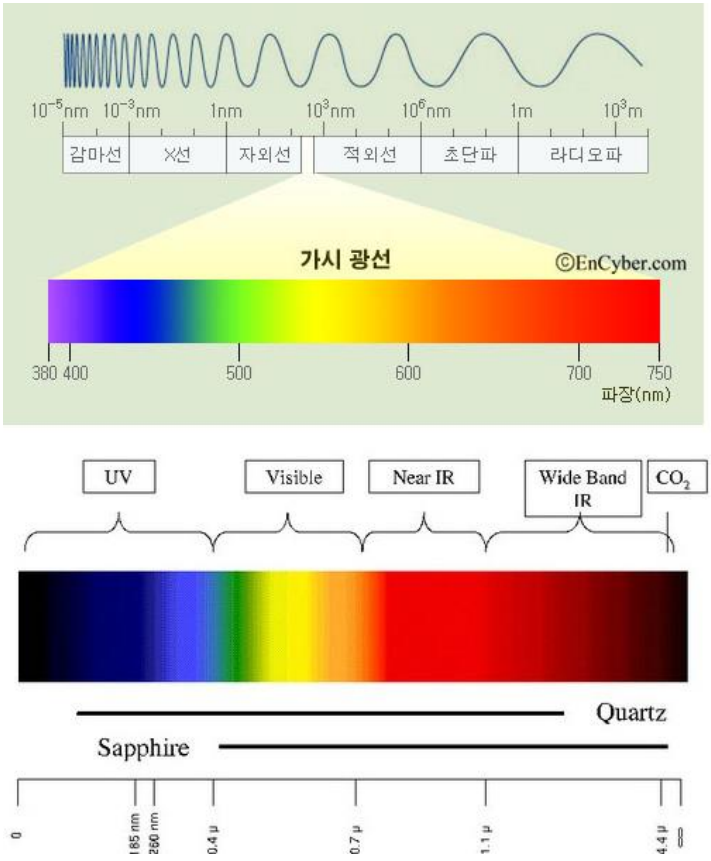
불꽃센서와 가스센서를 비롯하여 화재경보 시스템 시험시 간단하게 이용할 수 있는 장비로 라이터, 부탄가스, 가스레인지 등으로 화재나 신체상의 위험

을 주의해야 한다.

참고

불꽃 센서는 760 ~ 1100nm 사이의 파장 광원을 감지하여 출력하므로 야외나 햇빛이 직접 드는 곳에서는 오작동하거나 센싱 결과를 신뢰할 수 없다.

불꽃 감지 센서는 적외선 LED 를 통해 불꽃에서 감지되는 적외선 파장을 감지하여 이를 아날로그 혹은 디지털 신호로 변환해 주는 센서이다.



이미지출처 : 가시광선의 범위 – encyber.com

불꽃의 파장(주파수) 760nm ~ 1100nm 대역을 감지하는 센서로 소방활동 로봇 제작 및 화재 진압에 중요한 화재 경보기 제작에 활용이 가능하며

불꽃센서의 동작 환경은 25~85 도(섭씨)에서 작동 되며 불꽃을 감지하는 거리에 따라 아날로그 값이나 불꽃의 존재 유무를 알려주는 디지털 값으로 출력된다.

표 33 불꽃 센서의 제품 특징

동작전압	3.3V to 5V
감지 범위	20cm (4.8V) ~ 100cm (1V)
감지 대역(파장)	760nm ~ 1100nm
응답 시간 :	15usec
인터페이스 :	아날로그/디지탈
크기 :	22x30mm

불꽃 센서의 이미지와 출력 Pin Map 을 나타내었다.

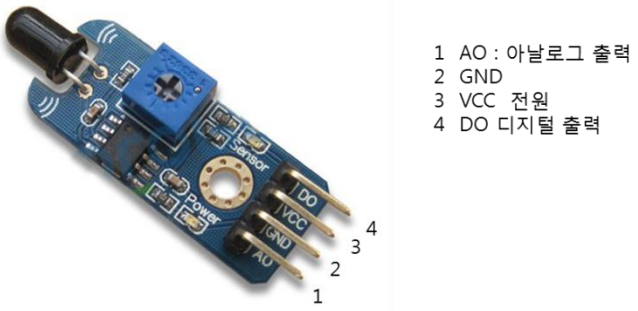


그림 147 불꽃 센서의 외형과 핀 맵

불꽃 센서의 컬리브레이션 방법을 설명한다. 전원을 연결하면 센서모듈 중앙의 두 개의 LED 중 빨강 LED 는 전원 LED 이고, 불꽃이 감지되면 켜지는 LED 가 녹색 LED 이다. 녹색 LED 가 항상 들어와 있다면 컬리브레이션이 되지 않은것이다. 중앙의 파란색 가변 저항을 돌려 녹색불이 켜지고 꺼지는 지점에서 꺼져 있도록 돌려놓은 지점을 찾는다. 그리고 라이터를 사용해 약 20cm 주변에서 라이터를 켰을때, 녹색불이 깜박이는 것이 확인된다면 컬리브레이션이 잘 된것이다.

불꽃 센서로부터 출력을 얻는 소스코드는 아래와 같다. 불꽃 센서의 디지털 출력으로부터 불꽃이 있을 때와 없을 때의 출력을 얻는데 불꽃이 없을 때 HIGH 신호에서 qnfRhcdmf 감지하면 LOW 신호로 내려가는 시점을 인지해 불꽃이 있음을 알려준다. 인터럽트 방식을 사용해 FALLING EDGE 에서 신호를 발생하여 등록한 이벤트 함수를 실행시킨다.

```
if ( wiringPiISR (SOUND_IN, INT_EDGE_FALLING, &myInterrupt) < 0 ) {
        fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
        return 1;
}
```

nano flaedetect.c 로 소스파일을 작성한다.

```
nano flamedetect.c



#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>



// Use GPIO Pin 7, which is Pin 11 for wiringPi library

#define SOUND_IN 11   //gpio7



// the event counter
volatile int eventCounter = 0;

// ---------------------------------------------------------------
-------------
```

```
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
    eventCounter++;

}



// ---------------------------------------------------------------------
// main
int main(void) {
  // sets up the wiringPi library
  if (wiringPiSetup () < 0) {
      fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
      return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  // and attach myInterrupt() to the interrupt
  if ( wiringPiISR (SOUND_IN, INT_EDGE_FALLING, &myInterrupt) < 0 ) {
      fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
      return 1;
  }

  // display counter value every second.
  while ( 1 ) {
    printf( "%d₩n", eventCounter );
    //eventCounter = 0;
    delay( 1000 ); // wait 1 second
  }
```

```
    return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

컴파일한다.

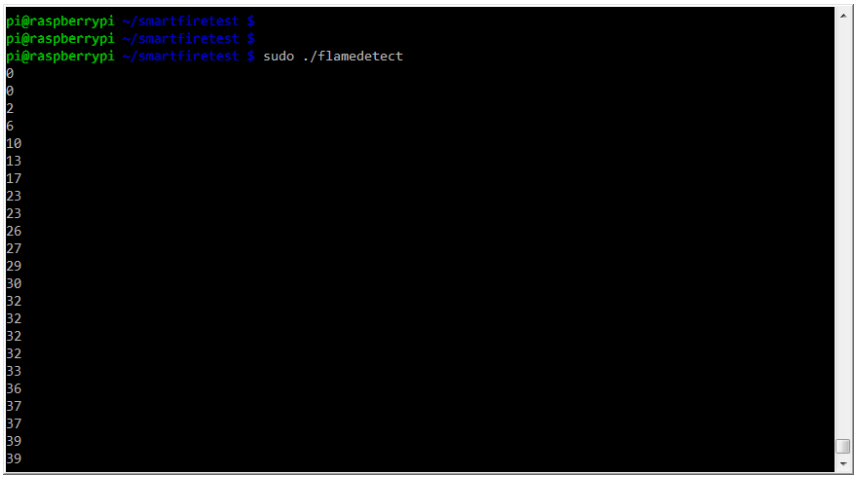
```
$gcc -o flamedetect flamedetect.c -lwiringPi
```

아래의 명령어로 실행한다.

```
$sudo ./flamedetect
```

거리에 따라 가스라이터의 반응을 조절하기 위해 센서 위 가변저항을 조절한다. 아날로그 출력을 입력으로 할 경우 불꽃의 유무뿐만 아니라 불꽃까지의 거리를 측정할 수 있지만 약 100cm 내의 거리에서 측정하므로 크게 의미는 없다. 아날로그 값을 읽어 출력을 센싱하는 방법은 현재 연결되어 있는 커넥터를 ADC 입력으로 연결해서 프로그램 하면 된다..

간단한 방법으로 가스라이터를 켜보면 켜보면 불꽃 센서가 반응하여 카운터 값이 증가되는 것을 볼 수 있다.



(2) 가스센서

주의

라이터, 부탄가스, 가스레인지 등으로 시험시 화재나 신체상의 위험을 주의해야 한다.

MQ 시리즈 가스 센서는 종류마다 다른 가스를 센싱하기도 한다. 시험에서는 MQ135 를 사용하여 LPG 가스나 부탄등을 검출하였는데 정상적으로 작동하였다 스펙에 적힌 용도의 센서를 쓰면 가장 좋지만 만약 없다면 다른 종류의 가스를 써서 시험을 진행한다.

MQ2 : Combustible Gas, Smoke
MQ5 : LPG, Natural Gas, Town Gas
MQ9 : Carbon Monoxide, Coal Gas, Liguefied Gas
MQ135 : Ammonia, Aromatics, Sulfer, Benzene vapor, other harmful

MQ135 는 내부에 포함된 히터와 센서로 맘모니아, 생활가스, 오염가스등을 검출해낼 수 있는 모듈이다. 내부에 포함된 히터가 적정온도로 올라가서 안정적인 결과를 얻기 위해서는 대기시간(burn-in time)이 필요하다. 때문에 MQ-2, MQ-3, MQ-4, MQ-5, MQ-6, MQ-7 모듈 종류별로 사용법이 약간씩 틀립니다. 센서가 보드에 결합되어 있는 경우 potentiometer 를 이용해서 히터에 걸리는 전압을 조절해서 센서의 감도를 조절한다.(센서만 있는 경우는 저항이나 PWM 핀으로 조절) 특정 가스에 대한 감도를 높이기 위해서는 다양하게 테스트 해 볼 필요가 있다. MQ135 의 경우에는 빠른 response time 을 갖고 있어서 거의 즉시 값을 추출할 수 있다. 아날로그 출력신호는 0~1023 사이의 값을 출력하므로 0~100%로 적절히 변환해서 사용할 수 있다.

반도체식 가스센서는 세라믹 반도체 표면에 가스가 접촉했을 때 일어나는 전기전도도의 변화를 이용하는 것이 많으며 대부분 대기 중에서 가열하여 사용되는 일이 많아 고온에서 안정한 금속산화물(세라믹스)이 주로 사용된다 금속 산화물은 반도체의 성질을 나타내는 것이 많고 이중 금속원자가 과잉(산소결핍)인 경우에는 n 형 반도체, 금속원자가 결핍인 경우에는 p 형 반도체가 사용된다. 이러한 세라믹반도체 중 전기전도도가 크고 융점이 높아서

사용온도 영역에서 열적으로 안정한 성질을 가진 반도체가 센서에 이용하고 있다.

반도체 가스센서는 1)대부분 유독가스 가연성가스에 어떤 응답을 나타내어 감지할 수 있는 가스의 종류가 많고, 2)센서제작이 용이하고 검출회로의 구성이 간단하다는 특징이 있다 그러나 감지하려는 가스만을 감지할 수 있는 선택성이 우수한 가스센서는 적고 아직도 연구개발 중에 있다. 반도체 가스센서의 모재료와 촉매를 여러가지로 바꾸거나 조합하고 센서 동작온도를 변경함으로써 선택성을 부여 하기도 한다. 많은 금속산화물 세라믹스를 가스센서의 연구 대상으로 하고 있으며, 가장 많이 연구되어 사용중인 것으로는 SnO2 , ZnO, Fe2O3 등이 있다.

아래의 테이블은 MQ 시리즈 센서의 여러 감지 기능을 요약한 것이다.

Semiconductor sensor for Flammable gas, Plastic or Metal cover		
Model	Target Gas	Datasheet
MQ-2	General combustible gas	
MQ-3	Alcohol	
MQ-4	Natural gas, Methane	
MQ-5	LPG, Natural gas, Coal gas	
MQ-6	LPG, Propane	
MQ-7	Carbon Monoxide	
MQ-8	Hydrogen	
MQ-9	CO and Combustible gas	
MQ306	LPG, Propane	
MQ307	Carbon Monoxide	
MQ303	Alcohol	
MQ131	Ozone	
MQ135	Air Quality Control	
Semiconductor sensor for Toxic gas		
MQ136	Sulfureted Hydrogen (H2S)	
MQ137	Ammonia (NH3)	
MQ138	VOC (Mellow, Benzene, Aldehyde, Ketone, Ester)	

자료출처 : 회사 홈페이지

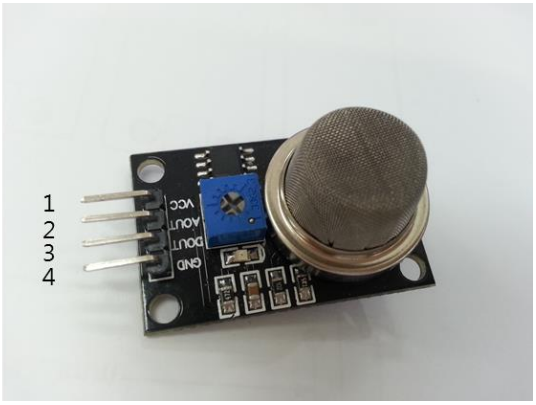
- MQ135 – 데이터 쉬트
- https://www.olimex.com/Products/Components/Sensors/SNS-MQ135/resources/SNS-MQ135.pdf

가스센서를 사용한 경보기 제작과 사용상의 주의점

가스는 공기보다 가벼운 가스의 그룹과 무거운 가스의 그룹으로 대별된다 따라서 공기보다 가벼운 가스를 감지할 때는 천장면에 근접하게, 공기보다 무거운 가스의 경우는 바닥면에서 30cm 정도에 설치한다. 또 도시가스가 실내에서 누설된 경우 폭발 농도까지 될 때까지는 상당한 시간을 요하며 가스가 상승 또는 강하하면서 수평방향으로 확산하는 성질이 있으므로 실내에서 가스의 흐름을 차단 하는 것이 없으면 가스경보기는 가스기구나 가스설비까지의 수평거리가 4m 이내인 곳에 설치한다.

아래에 가스센서의 모양과 PinMap 을 나타내었다.

1 VCC 전원
2 AO : 아날로그 출력
3 DO 디지털 출력
4 GND



가스 센서의 컬리브레이션 방법을 설명한다. 전원을 연결하면 센서모듈 중앙의 두 개의 LED 중 빨강 LED 는 전원 LED 이고, 불꽃이 감지되면 켜지는 LED 가 녹색 LED 이다. 녹색 LED 가 항상 들어와 있다면 컬리브레이션이 되지 않은것이다. 중앙의 파란색 가변 저항을 돌려 녹색불이 켜지고 꺼지는 지점에서 꺼져 있도록 돌려놓은 지점을 찾는다. 그리고 라이터를 사용해 약 20cm 주변에서 라이터를 켰을때, 녹색불이 깜박이는 것이 확인된다면 컬리브레이션이 잘 된것이다.

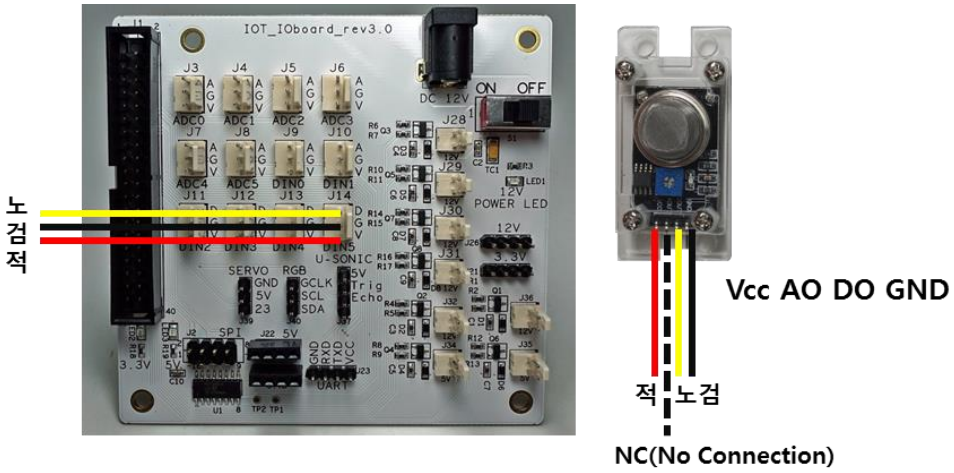


그림 148 가스센서 연결도

가스센서를 테스트 하는 소스코드를 나타내었다. 가스가 감지 되지 않은경우 디지털 출력은 HIGH 에 있다가 가스가 감지되면 LOW 로 하강하는 순간에 센싱을 한다. 가스에 주의하여 시험을 진행하도록 한다.

$nano gasdetect.c 명령어로 소스를 입력한다.

```
$nano gasdetect.c



#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>



// Use GPIO Pin 17, which is Pin 0 for wiringPi library


#define MOYION_IN 0   //gpio17
```

```
  // the event counter
  volatile int eventCounter = 0;

  // ------------------------------------------------------------
-------------
  // myInterrupt:  called every time an event occurs
  void myInterrupt(void) {
      eventCounter++;

  }



  // ------------------------------------------------------------
-------------
  // main
  int main(void) {
    // sets up the wiringPi library
    if (wiringPiSetup () < 0) {
        fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
        return 1;
    }

    // set Pin 17/0 generate an interrupt on high-to-low transitions
    // and attach myInterrupt() to the interrupt
    if ( wiringPiISR (MOYION_IN, INT_EDGE_FALLING, &myInterrupt) < 0 ) {
        fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
        return 1;
    }

    // display counter value every second.
    while ( 1 ) {
```

```
        printf( "%d₩n", eventCounter );
        //eventCounter = 0;
        delay( 1000 ); // wait 1 second
    }

    return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할것이냐 하고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

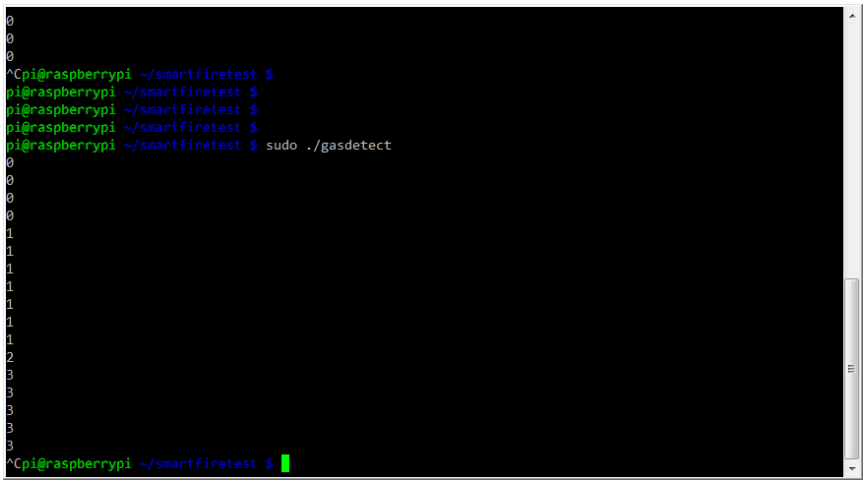
아래와 같이 컴파일 한다.

컴파일을 한다.

```
$gcc -o gasdetect gasdetect.c -lwiringPi
```

실행하여 결과를 확인한다.

```
$sudo ./gasdetect
```



가스를 흡착하여 출력을 발생시키므로 일단 가스 감지가 되면 가스가 공기중으로 날라가 안정화 될때까지 출력신호는 지연된다. 불꽃센서와 가스센서를 시험할때는 화재, 가스에 주의해야 한다.

(3) 온습도 센서

실습에서 사용되는 온습도 센서는 고정밀 디지털 온습도 센서 모듈 DHT22/AM2302 를 사용하기로 한다. 온도와 습도를 측정하고 이를 이용하여 제어하는 기술이 많은 분야에서 이용되고 있다. 아날로그 출력의 온습도 센서를 사용하여 마이크로컨트롤러에서 응용하기 위해서는 부가적인 회로나 AD 컨버터 회로가 별도로 필요한데 DHT22/AM2302 온습도 센서는 4.7KΩ~10KΩ 의 풀업 저항을 달아 1wire 통신을 이용해 센서 데이터를 획득할 수 있다.

요즘에는 신호 증폭, ADC, 보정, 통신 등의 회로를 내장하고 온도, 습도를 동시에 측정할 수 있는 센서들이 나오고 있다. DHT22/AM2302 은 DHT11 과 사용법이 비슷하나 DHT11 에 비해 정밀도가 높고 응답시간이 빠른 장점이 있다.

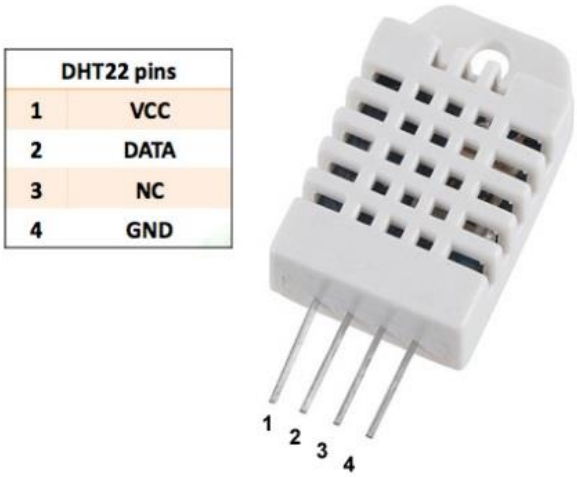


그림 149 DHT22/AM2302 온습도 센서

온습도 센서 연결도를 [그림 104]에 나타낸다. 한정된 지면으로 인하여 온습도 센서의 코딩 실습과 결과 확인은 스마트 농장 실습의 온습도 센서 설명을 참고한다.

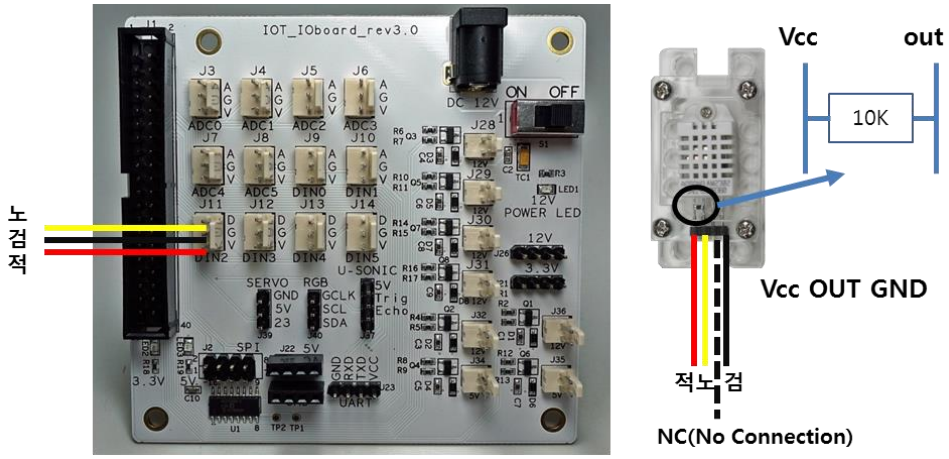


그림 150 온습도 센서 연결도

12.4 화재경보 시스템 액츄에이터 실습

Smart IoT 화재경보 시스템의 액츄에이터는 싸이렌 역할을 하는 부저와 물을 공급하는 워터펌프, 차단문을 닫는 기능을 하는 DC 모터, 경광등 효과를 내는 RGB led 로 구성된다. 4 종의 액츄에이터에 공급되는 전원 사양과 신호선을 테이블로 나타내었다.

액츄에이터	전원	제어신호
워터펌프	+12V	GPIO 제어
DC motor	+12V	GPIO 제어
사이렌 부저	+5	GPIO 제어
경광등	+12V	GPIO 제어 Red, Green, Blue 신호 GPIO

액츄에이터 연결도를 [그림 105]에 나타내었다.

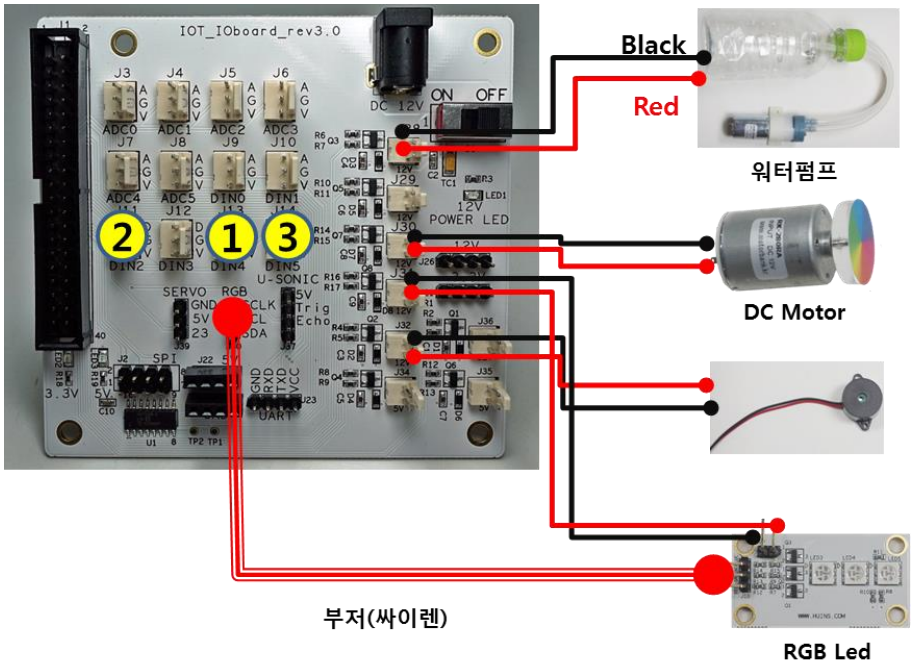


그림 151 화재경보 시스템 액츄에이터 연결도

(1) 워터펌프 실습

한정된 페이지의 지면 관계상 스마트 팜의 워터 펌프 실습편을 참조한다.

(2) DC Motor 실습

한정된 페이지의 지면 관계상 스마트 팜의 DC 모터 실습편을 참조한다.

(3) 부저 실습

부저는 기본적으로 소리를 내는 장치의 일종이다. 스피커, 확성기라고도 한다. 실습에서 사용되는 부저는 피에조 부저 (piezo buzzer)로 피에조 효과를 이용하여 소리를 내는 작은 스피커이다. 피에조 효과란 수정이나 세라믹 같은 결정체의 성질을 이용하는 것으로 압력을 주게되면 변형이 일어나면서 표면에

전압이 발생하고, 반대로 전압을 걸어주면 응축,신장을 하는 현상을 말하며 압전효과라고도 한다. 여기에 얇은 판을 붙여주면 미세한 떨림으로 소리가 나는 원리이다.

피에조 부저는 큰 소리를 내지 못 하는 단점이 있으나, 값이 싸고 단순하며, 주파수 2 만 Hz 이상의 초음파를 발생시킬 수 있기 때문에 초음파를 활용한 다양한 제품에서 사용되기도 한다.

(4) RGB Led 실습

한정된 페이지의 지면 관계상 스마트 팜의 워터 펌프 실습편을 참조한다.

(5) 액츄에이터 일괄 실습

액츄에이터가 연결된 커넥터로 일정한 시간 전원을 공급하기 위해 IO Adapter 보드로 GPIO 제어선에 HIGH 를 출력하면 액츄에이터는 동작할 것이다. 액츄에이터를 선택적으로 동작시키는 액츄에이터 테스트 프로그램 소스는 아래와 같다. fireactutest.c 파일

```
$nano fireactutest.c

#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>

#define PUMP          21        // BCM_GPIO 5
//#define FAN         22                  // BCM_GPIO 6
#define DCMOTOR 23          // BCM_GPIO 13
#define RGBLEDPOWER   24 //BCM_GPIO 19
#define RGBLEDPOWER   24 //BCM_GPIO 19
```

```
#define BUZZER 25                //GPIO26

#define RED  27
#define GREEN        28
#define BLUE         29

void Bplus_pinmodeset(void);
int testmode(void);

int main (void)
{
    if(wiringPicheck())
    {
            printf("Fail");
            return;
    }

    Bplus_pinmodeset();

    while(1)
    {
            int exit;
            exit = testmode();
            if(exit == 1) break;
    }
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
```

```
            fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
            return 1 ;
        }
    }

    void Bplus_pinmodeset(void)
    {
        pinMode (PUMP, OUTPUT);
        pinMode (BUZZER, OUTPUT);
        pinMode (DCMOTOR, OUTPUT);
        pinMode(RGBLEDPOWER, OUTPUT);
        pinMode(RED, OUTPUT);
        pinMode(GREEN, OUTPUT);
        pinMode(BLUE,OUTPUT);
    }

    int testmode(void)
    {
        int testnum;
        char stop;

       printf("Input Number for Smart Farm Actuator Test(1 ~ 4) and Enter !!! ₩n");
        printf("1. Water Pump Test₩n");
        printf("2. Buzzer Test₩n");
        printf("3. DC Motor Test₩n");
        printf("4. RGB Led Test₩n");
        printf("5. Exit₩n");
        printf("₩n₩nWater Pump Testing.... To Stop enter c or C ₩n");

        scanf("%d", &testnum);
```

```
if(testnum == 1)
{
        digitalWrite (PUMP, 1);       //on
        scanf("%c", &stop);
        if(stop == 'c') digitalWrite (PUMP, 0) ;

        //delay (2000) ; // ms
        //digitalWrite (PUMP, 0) ; // Off
}

if(testnum == 2)
{
        digitalWrite (BUZZER, 1) ; // On
        scanf("%c", &stop);
        if(stop == 'c') digitalWrite (BUZZER, 0) ;
        //delay (2000) ; // ms
        //digitalWrite (FAN, 0) ; // Off
}

if(testnum == 3)
{
        digitalWrite(DCMOTOR, 1); //On
        scanf("%c", &stop);
        if(stop == 'c') digitalWrite(DCMOTOR, 0); //OFF
}

if(testnum == 4)
{
        digitalWrite(RGBLEDPOWER, 1); // power on
        digitalWrite(RED, 1);
        delay(500);
```

```
                digitalWrite(RED, 0);
                digitalWrite(GREEN, 1);
                delay(500);
                digitalWrite(GREEN, 0);
                digitalWrite(BLUE, 1);
                delay(500);
                digitalWrite(BLUE, 0);
                digitalWrite(RGBLEDPOWER, 0); // power off
                scanf("%c", &stop);
                if(stop == 'c') digitalWrite(RGBLEDPOWER, 0);
        }

        if(testnum == 5)
        {
                return 1;
        }

        return 0;

}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

```
$gcc -o fireactutest fireactutest.c -lwiringPi
```

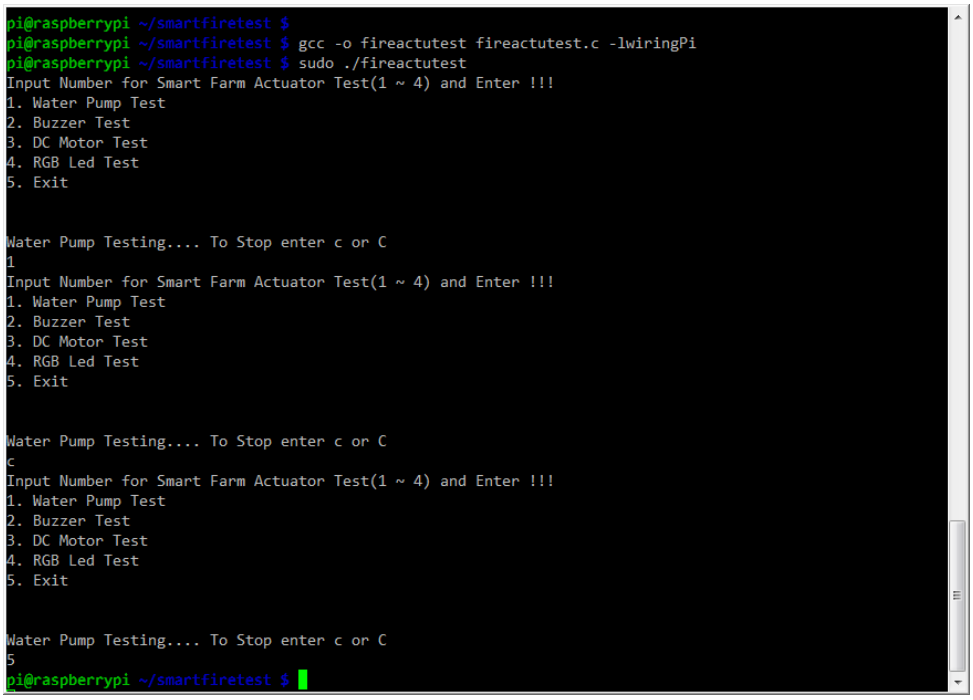
에러없이 완료되면 다음 명령어로 실행한다.

```
$sudo ./fireactutest
```

소스를 보면 테스트 화면의 처음에 테스트 할 액츄에이터 번호가 나온다. 번호를 누르고 Enter 를 치면 해당 액츄에이터의 테스트가 시작하고, 중단하기 위해 'c' 를 누르면 해당 액츄에이터 시험이 종료한다. '5' 를 누르면 프로그램이 종료된다. 동작화면을 보자.

```
Input Number for Smart Farm Actuator Test(1 ~ 4) and Enter !!!


1. Water Pump Test
2. Buzzer Test
3. DC Motor Test
4. RGB Led Test
5. Exit
Water Pump Testing.... To Stop enter c or C.
```



각 액츄에이터의 동작모습을 확인해 본다.

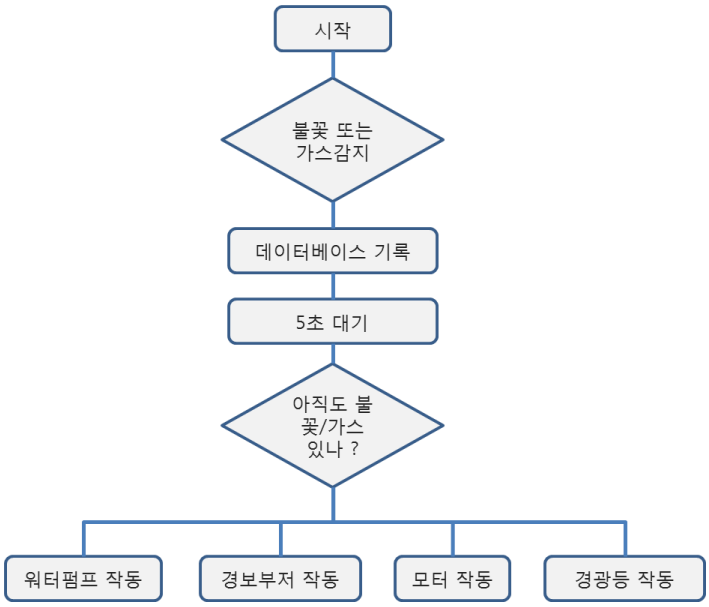
12.5 화재경보 시스템 시나리오 실습

화재 경보 시스템의 전체 시나리오를 테스트 한다. 지금까지 설명한 시스템의 전체 구조를 이해하고 센서와 액츄에이터 실습을 수행했다면 사용자가 원하는 시나리오대로 설정하고 제어하는 실습을 진행할 수 있을것이다. 앞서 실습한

환경 그대로 연결을 유지하거나, 사용자가 원하는 센서와 액츄에이터를 연결하여 확장할 수 도 있다.

화재감지의 기준이 되는 센서의 기준값을 정해야 한다. 가스 센서의 값을 감시하고 가스센서가 감지한 경우 경고음을 내고, 불꽃 센서에서 불꽃을 감지 할 경우 사용자에 메시지를 보내고 경보음과 경고등을 켜서 반응하여야 한다. 워터펌프는 물을 뿌려 화재를 진화하는 용도로 사용되었다.

아래는 화재경보 시스템이 작동하는 전체 시나리오를 간략하게 그려보았다. 이러한 절차는 사용자의 설계에 따라 변형이 가능하고 센서나 액츄에이터의 확장 또한 사용자가 원하는 대로 할 수 있다. 프로그램은 이래 순서도에 따라 작성하여 시험해보자.



위와 같이 설계하고 실습은 소스코드를 실행하여 결과를 확인한다. 가스와 불꽃을 위해 사용하는 라이터등의 취급에 주의해야 한다.

12.6 화재경보 시스템 서버 프로그램과 카메라 구동

(1) 데이터 베이스 설계

일단 데이터 베이스에 대해 알아보자. 앞서 APM(Apache, PHP, MySQL)을 설치하였다. 그것을 바탕으로 데이터 베이스를 활용해보자. 여기서 사용하는 데이터 베이스는 화재경보 시스템의 모든 센서데이터와 이벤트 기록을 데이터 베이스에 저장한다. 앞서 데이터 베이스의기본적인 내용을 바탕으로 아래 명세서를 따라 설계한다. 데이터 베이스에 대한 자세한 활용은 관련 도서를 참조하자.

데이터 베이스의 설계

화재경보 시스템에서 데이터 베이스의 설계를 대략적으로 나타낸다. 데이터 베이스의 설계와 테이블 구성은 방대한 작업이므로 아래 실습에서 간단하게 실습하고, 실제 구현시에는 서버 코드를 참조하여 꼼꼼하게 구현한다.

화재감시 Database 설계

DB Name : iotfire

○ Table : tfiresensorvalue 센서값 저장 테이블

1 분마다 센서값 저장한다.

날짜 day / 불꽃 int / 가스 int / 온도 float / 습도 float

○ Table : tfiresensorcheck 센서 정상/불량 저장 테이블

센서 정상 불량 확인하여 저장 10 분마다.

날짜 day / 불꽃센서 (정상, 오류) / 가스센서센서 (정상, 오류) / 온도센서(정상, 오류) / 습도센서(정상, 오류)

○ Table : tfireactuoperate 액츄에이터 동작상태(동작중/정지중)

액츄 동작시 데이터베이스 업데이트 하고, 동작 끝날 때 업데이트

날짜 day / pump(on,stop) / buzzer(on, stop) / dcmotor(on, stop) / rgbled(on, stop)

○ Table : tfireactucheck 액츄에이터가 정상/불량 저장 데이블

액츄에이터 정상 불량 기록

pump(normal, fail), buzzer(normal, fail), dcmotor(normal, fail), rgbled(normal, fail)

○ Table : tfiresetting 설정값 저장 테이블

Now(), 긴급연락처, 센서작동(on, off)

(2) 서버실행과 카메라 영상

IoT 화재 경보 시스템이 서버로 동작하는 환경을 구현하자. 위에서 테스트한 센서와 액츄에이터의 기능을 포함하여 동작 시나리오를 결정하여 코드를 구현하는 과정을 실습해본다.

화재 경보 시스템의 서버 프로그램의 구성을 DFD 로 나타내었다.

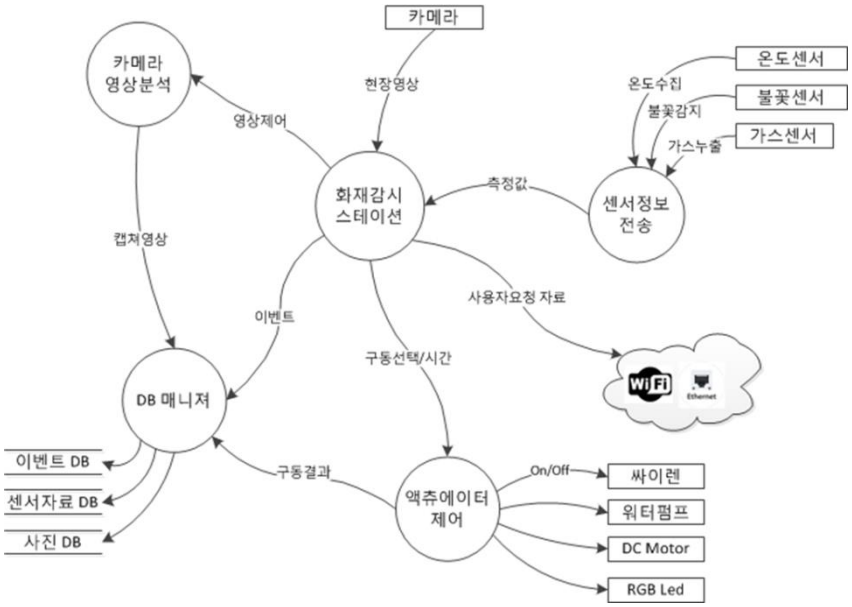


그림 152 화재경보 시스템 Data Flow Diagram

실내, 공장, 사무실 등의 환경에 적용하는 것으로 가정하고 시나리오를 설계한다. 여기서는 가장 단순한 기본 기능으로 실습하되 직접 개발하거나 실제 환경에 사용하기 위해서는 더욱 자세한 설계가 필요하다. 라즈베리 파이에서 동작하는 프로그램은 인터넷에 연결되어 스마트 폰의 사용자와 연동된다. 기본적으로 갖추어야 할 동작은 아래와 같다.

- ○ 액츄에이터 직접 제어
- ○ 카메라 영상 확인
- ○ 센서 데이터 조회

○ 비상 연락 설정 및 통화

○ 이벤트 이력 조회

서버 역할을 하는 라즈베리 파이에서 실행되는 서버 프로그램은 안드로이드 폰과 TCP 로 통신을 수행하며 사용자 데이터를 분석하여 사용자 요청을 처리한다. 무인경비 시스템의 서버 프로그램은 여러 프로그램으로 구성되어 있다. 서버 프로그램은 프로토콜에 대한 이해와 데이터 베이스등에 대해 잘 알고 있어야 하므로 어려운 작업이다. CD 에 첨부된 서버 프로그램을 분석하고 적용해보는 것은 각자에게 맡기고 여기서는 폴더와 컴파일 실행에 관해 설명한다. 서버프로그램을 컴파일 한 후 실행하는 방법과 카메라 구동방법으로 실습하기로 한다.

무인경비 스시템의 서버 프로그램 폴더로 이동한다.

```
pi@raspberrypi ~ $ cd iot_fireserver
pi@raspberrypi ~/iot_fireserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_fireserver $
```

아래 명령은 기존에 컴파일 되었던 파일들을 모두 지우는 명령이다. 이렇게 이전 컴파일 자료를 지워야 새로 수정된 프로그램이 컴파일된다.

```
pi@raspberrypi ~/iot_fireserver $ make clean
rm -rf /home/pi/iot_fireserver/output
Clearing in libs...
make[1]: Entering directory '/home/pi/iot_fireserver/libs'
rm -rf iot_db.o parser.o libiot.a core
make[1]: Leaving directory '/home/pi/iot_fireserver/libs'
Clearing in src...
make[1]: Entering directory '/home/pi/iot_fireserver/src'
Clearing in fire...
make[2]: Entering directory '/home/pi/iot_fireserver/src/fire'
rm -rf fire_main.o fire_sensor.o core
make[2]: Leaving directory '/home/pi/iot_fireserver/src/fire'
make[1]: Leaving directory '/home/pi/iot_fireserver/src'
```

```
pi@raspberrypi ~/iot_fireserver $
```

컴파일이 에러없이 수행되면 다시 make 명령을 사용하여 모든 파일들에 대해 컴파일을 실행한다.

```
pi@raspberrypi ~/iot_fireserver $ make
make all in libs...
make[1]: Entering directory '/home/pi/iot_fireserver/libs'
gcc -c -I/home/pi/iot_fireserver/include -I/usr/include/mysql iot_db.c -o iot_db.o
gcc -c -I/home/pi/iot_fireserver/include -I/usr/include/mysql parser.c -o parser.o
ar rcs libiot.a -o iot_db.o parser.o
make[1]: Leaving directory '/home/pi/iot_fireserver/libs'
make all in src...
make[1]: Entering directory '/home/pi/iot_fireserver/src'
make all in fire...
make[2]: Entering directory '/home/pi/iot_fireserver/src/fire'
gcc -g -c -I/home/pi/iot_fireserver/include -c fire_main.c -o fire_main.o
gcc -g -c -I/home/pi/iot_fireserver/include -c fire_sensor.c -o fire_sensor.o
gcc -o server_fire fire_main.o fire_sensor.o /home/pi/iot_fireserver/libs/libiot.a -lmysqlclient -lpthread -lwiringPi
make[2]: Leaving directory '/home/pi/iot_fireserver/src/fire'
make[1]: Leaving directory '/home/pi/iot_fireserver/src'
pi@raspberrypi ~/iot_fireserver $
```

컴파일이 되었다면 마지막으로 실행파일을 만들어 준다. $make install 명령을 사용하여 실행파일이 output 폴더 아래에 생성되도록 한다.

```
pi@raspberrypi ~/iot_fireserver $ make install
mkdir /home/pi/iot_fireserver/output
Installing in libs...
make[1]: Entering directory '/home/pi/iot_fireserver/libs'
```

```
make[1]: *** No rule to make target 'install'.  Stop.
make[1]: Leaving directory '/home/pi/iot_fireserver/libs'
Installing in src...
make[1]: Entering directory '/home/pi/iot_fireserver/src'
Installing in fire...
make[2]: Entering directory '/home/pi/iot_fireserver/src/fire'
cp -a server_fire /home/pi/iot_fireserver/output/
make[2]: Leaving directory '/home/pi/iot_fireserver/src/fire'
make[1]: Leaving directory '/home/pi/iot_fireserver/src'
pi@raspberrypi ~/iot_fireserver $
```

실행파일이 있는 위치를 찾아가보도록 하자.

```
pi@raspberrypi ~/iot_fireserver $ cd output
pi@raspberrypi ~/iot_fireserver/output $
pi@raspberrypi ~/iot_fireserver/output $ ls
server_fire
pi@raspberrypi ~/iot_fireserver/output $
```

output 폴더 아래에 server_intruder 파일을 볼 수 있다. 라즈베리 파이 서버 프로그램을 실행시키면 서버동작의 메시지를 볼 수 있고, 안드로이드 폰과 연동되어 실행된다.

아래 명령으로 실행한다.

$sudo ./server_fire

위와 같이 서버 프로그램을 실행하면 데이터 베이스를 생성하고 센서 데이터를 수집하는 것을 볼 수 있다.

```
pi@raspberrypi ~/iot_fireserver/output $ sudo ./server_fire
validate database...
iot_send_query : query - CREATE TABLE IF NOT EXISTS tfiresensorvalue (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, fire INT, gas INT, temperature FLOAT, humid FLOAT)
```

```
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tfiresensorcheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, fire INT, gas INT, temperature INT, humid)
  Mysql query error : You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ')' at line 1iot_send_query : query - CREATE TABLE IF NOT EXISTS tfireactuoperate (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, pump INT, buzzer INT, dcmotor INT, rgbled INT, sensor INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tfireactucheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, pump INT, buzzer INT, dcmotor INT, rgbled INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tfiresetting (id INT AUTO_INCREMENT PRIMARY KEY, emergency VARCHAR(20))
  done
  iot_count_setting_data_from_db : query - SELECT COUNT(*) FROM tfiresetting
  iot_count_setting_data_from_db data: 1
  validate setting data... : 1
  starting server...
  SensorInterruptLoop : write to DB - INSERT INTO  tfiresensorvalue values (null ,now(), now(), 1, 1, 1.000000, 1.000000)
  iot_insert_data_from_db : query - INSERT INTO  tfiresensorvalue values (null ,now(), now(), 1, 1, 1.000000, 1.000000)
  SensorInterruptLoop : write to DB - INSERT into tfiresensorcheck values (null, now(), now(), 1, 1, 0, 1072693248)
  iot_insert_data_from_db : query - INSERT into tfiresensorcheck values (null, now(), now(), 1, 1, 0, 1072693248)
```

서버를 실행하고 영상을 실시간으로 확인하기 방법은 두가지가 있다.

1. 터미널 화면에서 서버구동을 백그라운드로 실행하는 방법이다. 즉 서버 프로그램 실행 명령에 & 를 추가해 실행을 하면 서버 프로그램이

백그라운드 프로그램으로 실행된다. $sudo ./server_intruder & 명령어이다.

2. 이 상태에서 enter 를 입력하면 $가 나오는데 여기서 동영상 스트리밍 프로그램을 동작시키는 것이다. $sh mjpg.sh 명령을 실행하면 스트리밍 서버가 작동되어 영상을 확인하게 된다.
3. 두 번 째 방법은 다른 컴퓨터에서 putty 를 실행하여 라즈베리 파이에 접속한 후에 $sh mjpg.sh 를 실행하는 방법이다.

12.7 화재경보 시스템 안드로이드 앱 프로그램

화재 경보 시스템의 안드로이드 구동 프로그램은 클라이언트로 실행되는 프로그램이다. 사용자의 스마트폰에서 “화재감시” 앱을 구동하여 화재 경보 시스템이 설치된 현장의 영상을 실시간으로 확인하고 센서값을 조회하고, 설치된 액츄에이터를 직접 제어할 수 있다.

이 장에서는 안드로이드 앱을 구동하여 라즈베리 파이에서 돌아가는 서버 프로그램과 연동하는 실습을 할 것이다. 라즈베리 파이의 서버코드 분석과 안드로이드 클리이언트 프로그램은 용량이 크기에 지면에 옮기진 않는다.

안드로이드 환경에서 실행되는 화재경보 시스템의 소프트웨어 구성을 Data Flow Diagram 에 나타내었다. 그리고 안드로이드에서 실행되는 앱 화면에 대한 설명으로 화재경보 시스템 실습을 마치기로 한다.

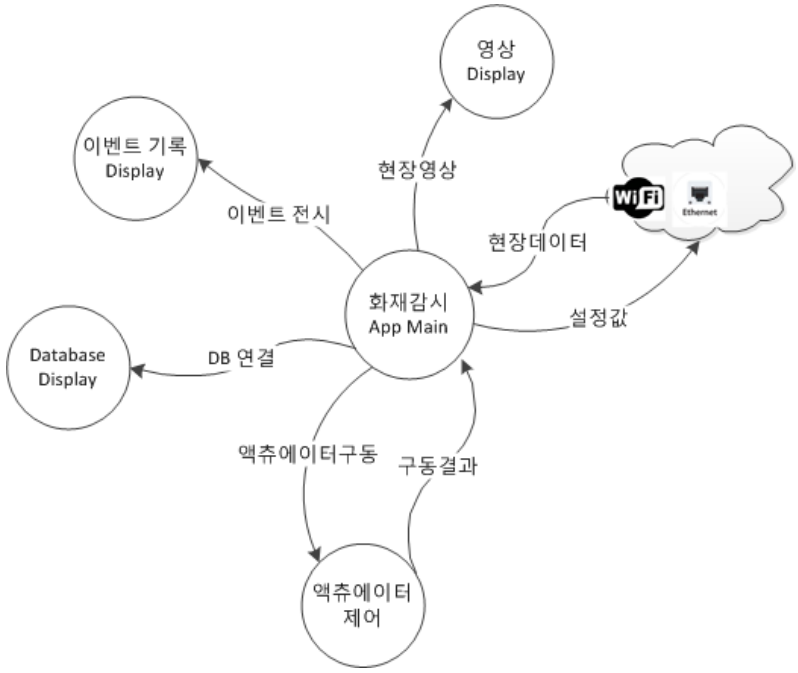


그림 153 안드로이드 앱 프로그램 Data Flow Diagram

여기서는 프로그램 구성을 설명하고 스마트 폰 앱을 실행하여 동영상을 확인하고 다양한 액츄에이터를 제어하는 실습을 진행한다.

소스코드의 구성은 사용자 스스로 분석하여 다양한 환경에 적용하도록 하자.

아래에 무인경비 시스템의 서버와 앱이 연동되는 테스트 과정을 설명한다. 이대로 시험하면 결과를 확인할 수 있고 나머지는 참고로 이용하길 바란다.

1. 라즈베리 파이 전원을 켜고, IO 어댑터 보드의 12V 전원을 켠다.
2. 로그인 아이디 : pi password : raspberry (화면에 나타나지 않음)
3. 라즈베리 파이 아이피 주소 : $ifconfig 명령어로 확인한다.
   예)192.168.1.219
4. 라즈베리 파이 화재경보 시스템 서버 실행

```
pi@raspberrypi ~ $ cd iot_fireserver
pi@raspberrypi ~/iot_fireserver $ cd output
pi@raspberrypi ~/iot_fireserver/output $
pi@raspberrypi ~/iot_fireserver/output $ ls
```

```
mjpg.sh   server_fire
pi@raspberrypi ~/iot_fireserver/output $
pi@raspberrypi ~/iot_fireserver/output $ sudo ./server_fire &
```

(백그라운드 실행) 엔터 입력하여 터미널 화면에서 실행

5. 동영상 스트리밍 서버 실행 : $sh mjpg.sh
6. 스마트 폰 앱 실행 pi@raspberrypi ~ $ **sh mjpg.sh**

이 과정을 통해 연동 테스트를 수행한다.

아래는 서버 프로그램을 실행시킨 화면이다. 서버는 인체감지 센서, 음향센서, 초음파 센서 데이터를 수집하여 데이터 베이스에 저장하는 한편 클라이언트가 되는 스마트 폰에서 접속하기를 기다린다. 따라서 터미널 화면에서 서버를 수행시키면 다른 컴퓨터에서 putty 를 실행시켜 동영상 스트리밍을 서비스 하는 프로그램을 실행하는 방법으로 실습하여야 한다.

```
pi@raspberrypi ~/iot_fireserver/output $ sudo ./server_fire
validate database...
iot_send_query : query - CREATE TABLE IF NOT EXISTS tfiresensorvalue (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, fire INT, gas INT, temperature FLOAT, humid FLOAT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tfiresensorcheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, fire INT, gas INT, temperature INT, humid)
Mysql query error : You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near ')' at line 1iot_send_query : query - CREATE TABLE IF NOT EXISTS tfireactuoperate (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, pump INT, buzzer INT, dcmotor INT, rgbled INT, sensor INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tfireactucheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, pump INT, buzzer INT, dcmotor INT, rgbled INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tfiresetting (id INT AUTO_INCREMENT PRIMARY KEY, emergency VARCHAR(20))
done
```

```
iot_count_setting_data_from_db : query - SELECT COUNT(*) FROM tfiresetting
iot_count_setting_data_from_db data: 1
validate setting data... : 1
starting server...
```

이 상태에서 putty 를 실행시키고, 라즈베리 파이에 접속한다. Putty 를 실행하는 이유는 위에 서버를 실행하고 다시 동영상 스트리밍 서버를 실행하기 위함인데 위에 설명한 $ sudo ./server_farm & 명령(뒤에 &를 붙여서)으로 백그라운드로 실행하게 하면 터미널에서 엔터를 치면 명령줄이 나타나므로 putty 프로그램을 실행할 필요는 없다.

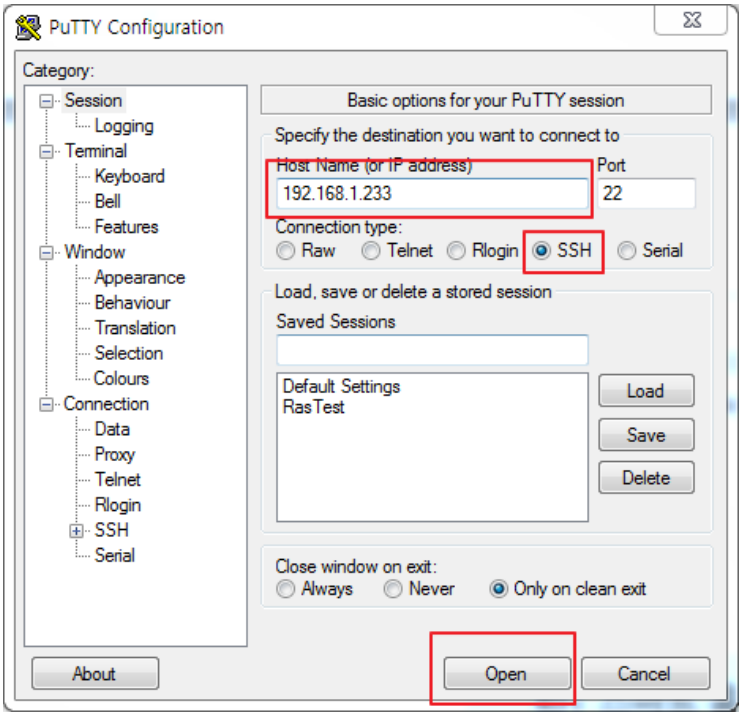


그림 154 동영상 스트림 서버를 실행시키기 위한 Putty 실행화면

아이디와 패스워드를 넣어 접속한 화면은 아래와 같다.

```
login as:
login as: pi
pi@192.168.1.219's password:
```

```
Linux raspberrypi 3.18.7+ #755 PREEMPT Thu Feb 12 17:14:31 GMT 2015
armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

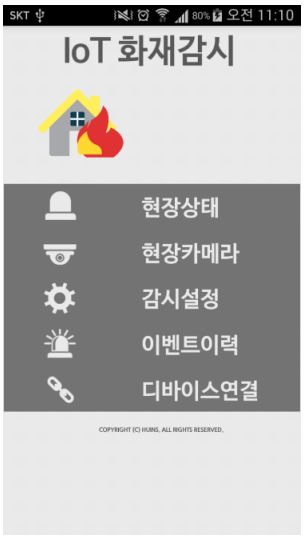
Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 09:47:01 2015
pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

홈 디렉토리에서 mjpg.sh 파일을 확인하고 다음의 명령어로 mjpg 스트림 프로그램을 실행한다.

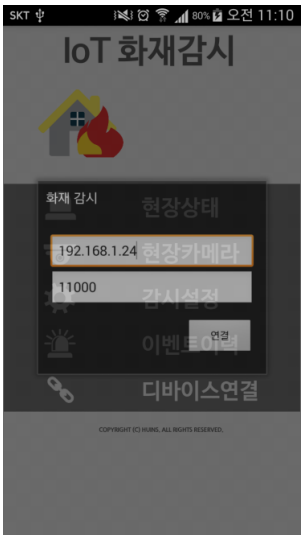
$sh mjpg.sh

```
pi@raspberrypi ~ $ sh mjpg.sh
MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116):
argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116):
argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116):
argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
```

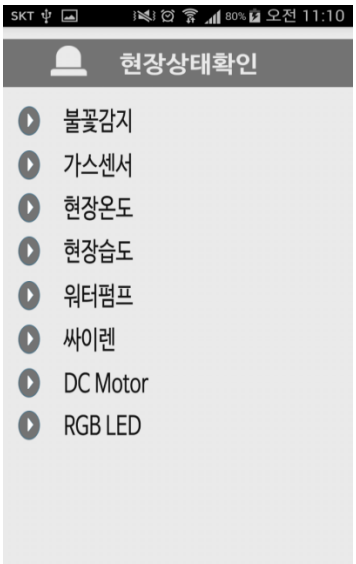
```
i: camera parameters.....:
.
.
.
.
```



IoT 화재 경보시스템 메인 화면을 나타낸다. 메뉴구성은 현장상태, 현장카메라, 감시설정, 이벤트 이력, 디바이스연결로 구성된다.



디바이스 연결 화면을 나타낸다. 디바이스 연결은 안드로이드 앱에서 가장 먼저 실행해야 하는 메뉴로 라즈베리 파이의 IP 주소를 입력하여 서버에 접속하게 된다.



현장상태 확인 화면을 나타낸다. 현장상태확인은 현장에 설치된 여러 센서의 상태와 센서 값들을 조회하고, 액츄에이터의 상태를 보여준다.



현장 카메라 확인 메뉴는 서버에서 실행중인 동영상 스트리밍 서버에 접속하여 영상 카메라의 실시간 영상을 보여준다. 약 1초 정도의 Delay 가 발생되지만 화질은 선명한 편이다.



현장 감시 설정 화면을 나타낸다. 액츄에이터 각각에 대해 작동을 계속할 것인지, 작동을 멈출것인지 설정한다. 현장에 사람이 상주하여 화재 감시 기능을 중지 시켜야 할 경우에는 센서 작동을 멈추도록 설정할 수 있다.



화재 발생이 감지되어 사용자 스마트폰으로 화재 발생 메시지를 보여주는 화면이다. 화재 발생이 감지되어 긴급 경보가 뜨면 사용자는 소방서에 신고를 한다든가, 긴급 연락처로 전화를 하는 조치를 취할 수 있도록 한다.

Chapter 13 IoT 애완동물 도우미 실습

13.1 애완동물 도우미 실습시스템 개요

IoT 애완동물 도우미는 집에 애완동물을 키우는 사용자에게 유용한 시스템이다. Wi-Fi 가 연결된 IoT 애완동물 도우미는 애완동물을 촬영하여 사용자에게 실시간 영상을 제공하고, 시간에 맞추어 음식을 제공한다. 애완동물 도우미 실습 시스템은 애완동물에 반응하여 애완 동물의 상태를 주인에게 전달하고, 애완동물의 영상을 확인하여 주인과 일체감을 느끼게 하는 시스템이다. 사용자는 카메라 제어와 영상 전송을 실습하고 시간에 맞추어 애완동물에게 밥을 주는 액츄에이터를 설계하고 실습한다.

IoT 애완동물 도우미의 시스템의 전체 구성을 보면 Database 와 카메라, 이더넷(Wi-Fi)이 지원되는 게이트웨이, 여러가지 센서와 액츄에이터를 제어하는 중간 연결부 인 IO Adapter 보드, 사용자 서버를 나타낸다.

IoT 애완동물 도우미의 주요기능을 정리하였다.

○ 스마트폰을 이용한 애완동물 도우미 원격 제어

○ 애완동물에게 음식을 직접 줄 수 있는 모터를 동작시킴

○ 애완동물의 상태를 영상을 이용해 직접 모니터링 가능

Smart IoT 설계 시스템의 통합버전에서 센서는 인체감지 센서와 소리 센서를 사용하고, 액츄에이터로는 서보모터, Led bar, DC Motor, RGB Led 만을 사용하여 애완동물 도우미를 구현한다.

IoT 애완동물 도우미 시스템은 Gateway 시스템과 모바일 프로그램으로 구성된다. 시스템에서 Gateway 는 인체감지 센서와 소리센서에서 수집한 데이터를 Database 에 저장하고, 센서 데이터를 판단하여 다양한 액츄에이터(서보모터, Led bar, DC 모터, RGB Led)를 구동한다. 또한 안드로이드 폰과의 연동을 통해 원격지의 사용자 요청이 발생할 경우 Ethernet 이나 Wi-Fi 를 통해 센서데이터나 영상 정보를 송신한다.

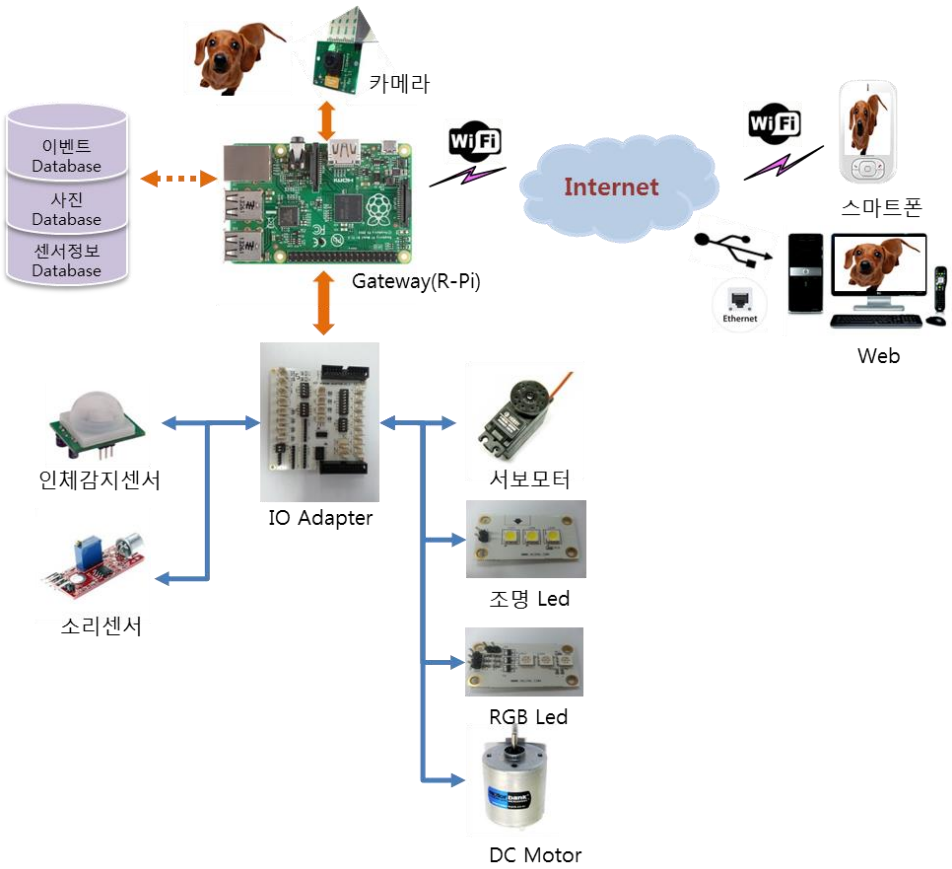


그림 155 애완동물 도우미 시스템 구성도

사용자가 실제 가정의 애완동물에 적용할 수 있도록 유사한 환경을 구현하여 실습할 수 있도록 설계되었다. Gateways 에는 라즈베리파이 기반으로 리눅스 운영체제를 최적화한 라즈비안이 설치되어 있다. 정해진 시간에 음식을 제공하는 용도로 서보모터를 사용하였고, 조명을 위한 Led bar 와 많은 색을 낼 수 있는 RGB LED, 기타 필요한 기구물을 작동하기 위해 DC 모터를 사용한다.

(1) 애완동물 도우미 시스템 하드웨어

IoT 애완동물 도우미의 하드웨어는 게이트 웨이 역할을 하는 라즈베리 파이와 카메라, IO Adapter 보드, 센서 2 종과 액츄에이터 1 개로 구성된다.

게이트웨이에는 애완동물 도우미를 관리하고, 영상을 전송하며, 액츄에이터를 제어하고, 스마트 폰과 통신하는 서버 프로그램들이 설치된다. 애완동물 도우미는 집에 주인이 외출 중일 때 애완동물의 상태를 스마트 폰으로 전송해주는 기능을 하며, 사용자는 언제든지 인터넷을 통하여 카메라에서 촬영한 애완동물 영상을 볼 수 있다. 정기적인 시간에 맞추어 애완동물 도우미는 애완동물에게 식사를 제공하는 기능을 한다. 현재 서보모터는 일정 각도만을 왕복하는 기능을 한다. 설정된 시간에 맞추어 밥이 담긴 용기를 꺼내고 들여보내는 역할을 한다.

아래는 IoT 애완농물 도우미의 H/W 구성품을 나타낸다. 앞장의 실습시에 설명한 라즈베리 파이와 IO 어댑터보드, 카메라에 설명은 생략하기로 한다.

표 34 IoT 애완동물 도우미의 주요 구성품

구성품	사양	기능
센서	인체감지센서	애완동물의 움직임 감지
	사운드센서	애완동물이 내는 소리 감지
Actuator	서보모터	음식물 통을 열고 닫음
전원어댑터	DC 12V, 3A 전원 어댑터	12V 전원 공급
	DC5V, 1A 전원 어댑터	5V전원공급

IoT 애완동물 도우미를 구성하는 구성하는 IO Adapter 보드는 여러 종류의 센서와 다양한 액츄에이터를 라즈베리 파이기반의 Gateway 와 연결해주는 역할을 한다. 무엇보다 IO Adapter 보드의 구조와 연결 커넥터에 대해 숙지해야 한다.

(2) 애완농물 도우미 시스템의 소프트웨어

IoT 애완동물 도우미를 이루는 소프트웨어 크게 게이트웨이(Gateway) 소프트웨어와 안드로이드 앱의 두 부분으로 구성된다. 라즈베리 파이의 펌웨어는 리눅스 기반에서 수행되는 소프트웨어로 직접 센서와 액츄에이터를 제어하고

센서 데이터를 수집한다. 카메라 구동 드라이버는 게이트웨이에 포함되어 있으며 안드로이드 폰과 연동시 게이트 웨이의 서버 프로그램이 수행된다.

따라서 게이트 웨이 시스템과 스마트폰 Application 양 쪽에서 수행되는 Software 의 전체 구조를 살펴보면 아래와 같다.

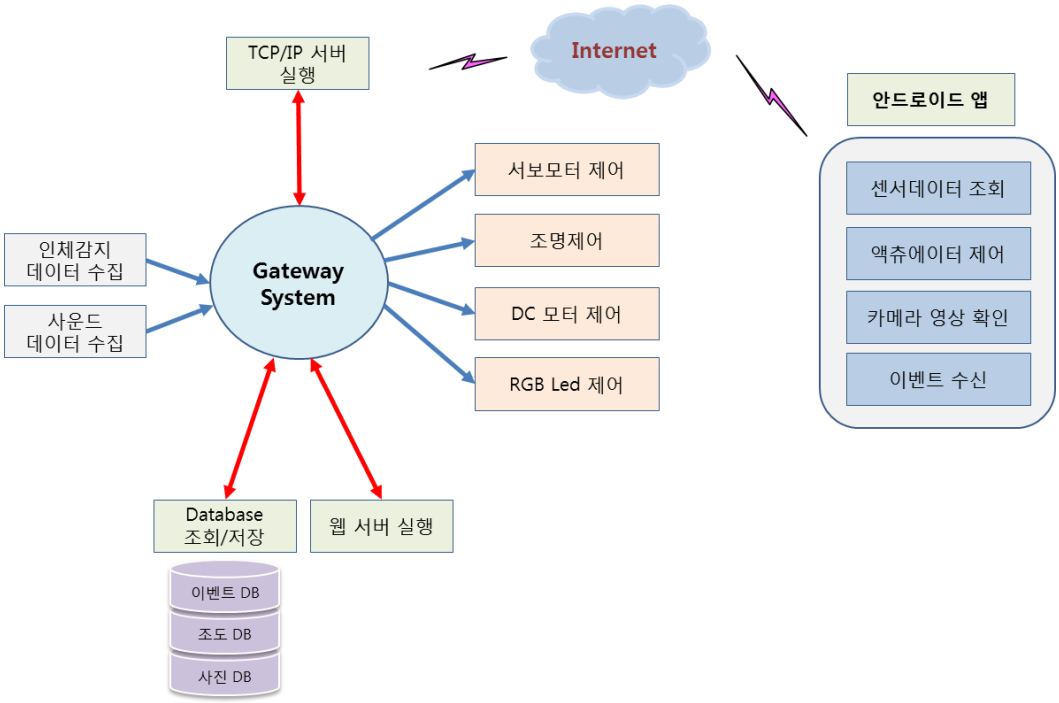


그림 156 애완동물 도우미 시스템 소프트웨어 구성도

스마트 폰에서 동작하는 안드로이드 앱은 원격지의 영상을 실시간으로 확인하고, 실내의 센서데이터를 조회하며, 환경에 따라 액츄에이터를 직접 제어하는 기능을 한다. 카메라 구동 드라이버는 이미 게이트웨이에 포함되어 있으며 안드로이드 폰과 연동시 게이트 웨이의 서버 프로그램이 수행된다. Gateway 역할을 하는 라즈베리 파이는 리눅스 운영체제가 설치되어 있다. 센서로부터 수집된 데이터를 날짜 시간 데이터와 함께 데이터 베이스에 저장하고, 원격지의 사용자 요청에 따라 카메라 영상을 송신하고 설치된 액츄에이터를 제어하는 기능을 한다.

13.2 애완동물 도우미 시스템 연결 구성

애완동물 도우미를 설계하기 위해 가정의 애완동물과 함께하는 상황에 대한 많은 분석이 필요하지만 여기서는 몇 가지 요구되는 기능 구현을 위해 단순화 할 필요가 있다. 즉 정해진 시간에 서보모터를 작동하여 음식을 제공하는 기능을 생각해 볼 수 있다. 또한 애완동물의 움직임이나 소리를 감지하여 적당한 조명과 여러가지 색의 RGB LED 를 작동하는것도 생각할 수 있다. 또한 원격지의 사용자는 언제든지 카메라에 접속하여 현재 애완동물의 상황을 볼 수 있다. 여러 가지 제약사항에도 불구하고 사용자는 실제 애완동물이 있다는것을 가정하고 사용자의 설계 의도대로 환경을 구성해야 한다.

게이트웨이 기능을 수행하는 라즈베리 파이 연결 커넥터는 40 핀 GPIO 케이블로 연결된다. 라즈베리 파이의 GPIO 포트는 사용자의 프로그램에서 입력으로 설정하거나 출력으로 설정을 한다. 또한 할당된 GPIO 핀 입력을 인터럽트로 설정하여 이벤트 리스너를 함수로 등록하여 사용하는것은 훌륭한 외부 인터럽트를 처리하는 방법이다.

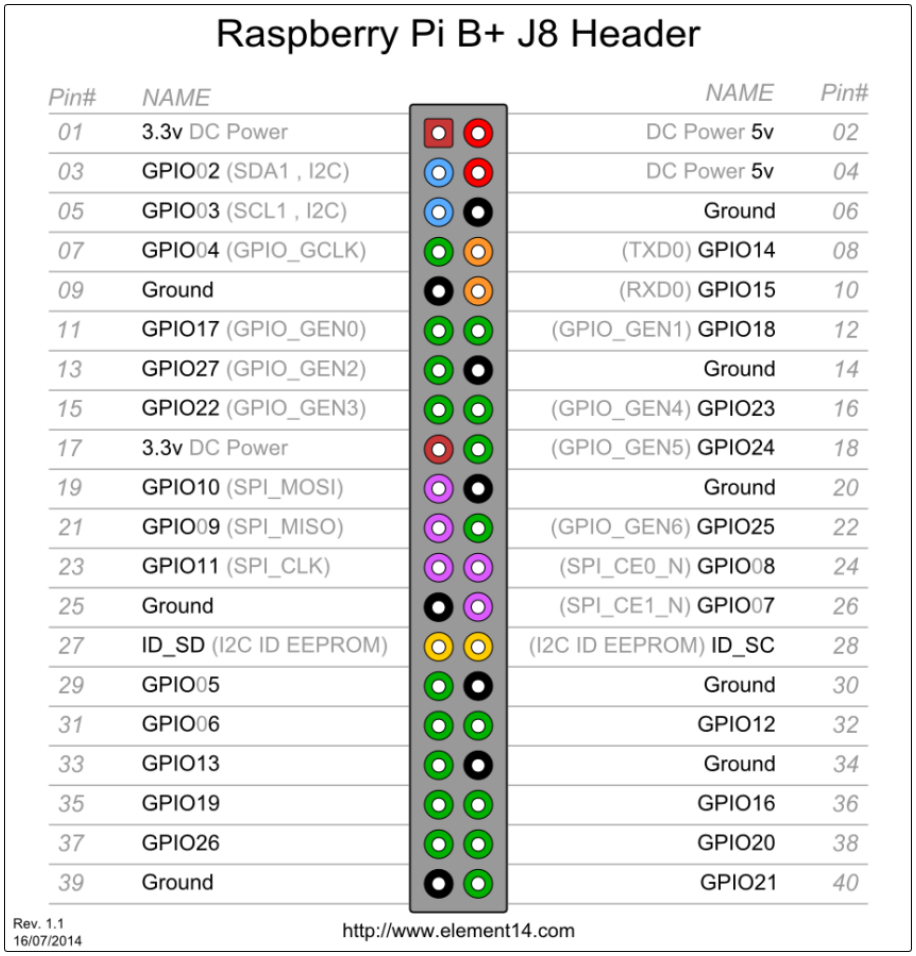


그림 157 라즈베리 파이 GPIO 핀 구성

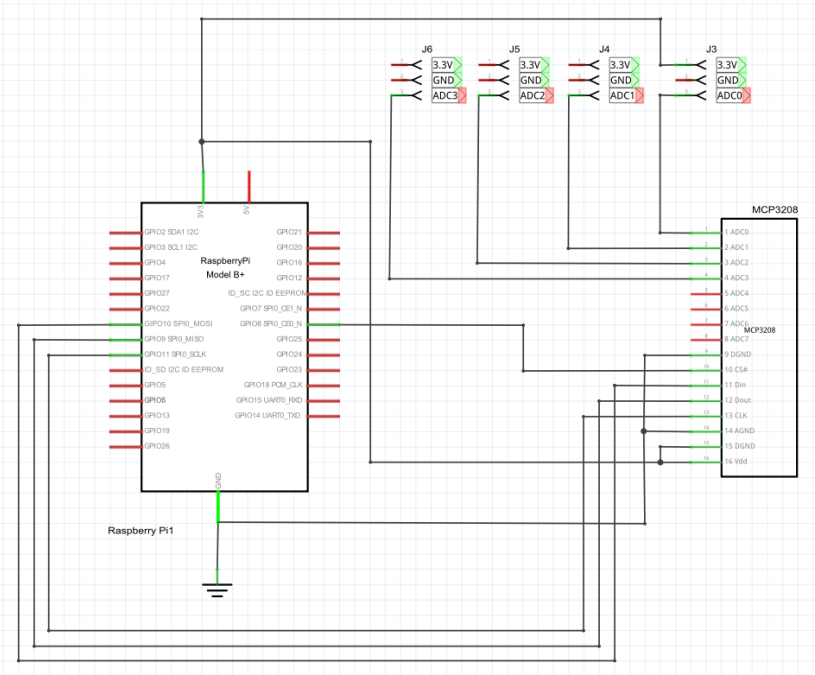


그림 158 라즈베리 파이 ADC 연결도

Gateway 기능의 라즈베리 파이에는 카메라와 Wi-Fi USB 무선 랜카드가 연결된다. IO Adapter 보드에 온습도 센서, 조도센서를 연결한다. IO Adapter 보드에 연결되는 액츄에이터는 워터펌프, FAN, DC Motor, RGB Led 이다.

IO Adapter 보드는 센서와 액츄에이터를 게이트웨이에 연결하는 역할을 한다. IO Adapter 보드에는 액츄에이터 구동을 위해 DC 12V 전원이 공급되므로 각종 센서나 액츄에이터를 연결할 때 반드시 전원을 끈 상태에서 주의를 기울여 연결한다.

주의

워터펌프, DC motor, RGB Led 등의 액츄에이터의 구동을 위해 IO Adapter 보드에는 DC 12V독전원이 연결되므로 센서나 액츄에이터를 연결할 때는 반드시 전원을 Off 하고 연결해야 하며, 쇼트, 단락에 주의해야 한다.

아래는 애완동물 도우미전체 연결도를 나타내었다.

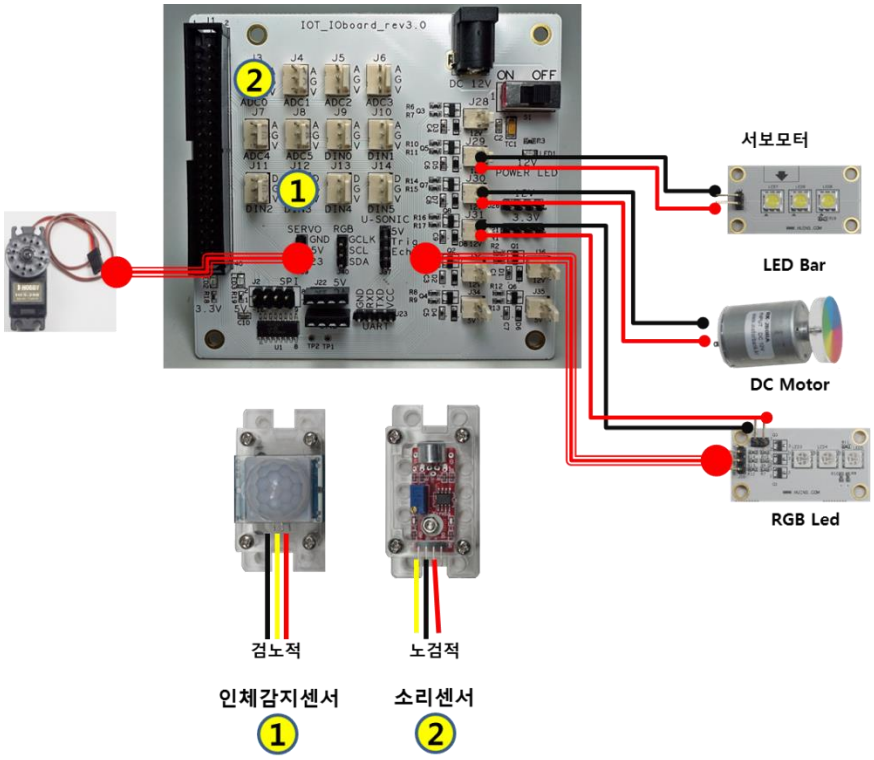


그림 159 애완동물 도우미 전체 연결도

연결의 정확성

센서와 액츄에이터 보드의 연결도는 실습 후에 스마트 폰으로 입력을 확인하고 액츄에이터를 제어할 때까지 동일한 연결과 테스트를 수행해야 한다. 개별 센서와 액츄에이터 실습시에는 센서 데이터와 액츄에이터 제어가 올라보 되는지를 확인하고 실제 상황을 스마트 폰과 연동 실습할때에는 모든 연결이 정상적으로 정확하게 연결되어야 실습이 진행된다.

13.3 애완농물 도우미 센서 실습

IoT 애완농물 도우미에서 사용하는 센서는 애완동물의 모션을 감지하는 인체감지 센서와 애완동물이 짖거나 주위에서 소리 발생을 감지하는 사운드

센서를 사용한다. 게이트 웨이의 GPIO 포트를 이용해 센서 데이터를 수신하고 터미날에서 센서데이터를 볼 수 있도록 실습한다.

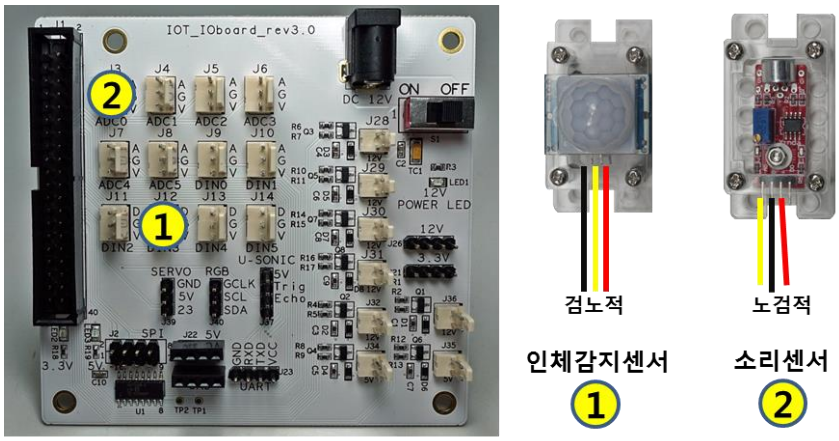


그림 160 애완동물 도우미의 센서구성과 연결 위치

(1) 인체감지 센서

인체감지 센서에 대해서는 한정된 지면으로 인하여 11 장의 무인경비 시스템 설명 부분의 인체감지 센서를 참조하기로 하고 여기서는 소프트웨어에 대해서만 설명하기로 한다. 특히 인체감지 센서의 연결도, GPIO 핀 할당 번호, 센서의 컬리브레이션에 대해 잘 알아두길 바란다. 센서의 자세한 스펙을 아래주소에서 다운 받을 수 있다.
(http://www.mpja.com/download/31227sc.pdf)

인체감지 센서를 시험하는 소스코드를 아래에 나타낸다. (motiondetect.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>
```

```
#define MOTION_IN 0   // wiringPi 할당 번호

// the event counter
volatile int eventCounter = 0;
unsigned char flag_detect;

// 이벤트(인터럽트)가 발생하면 호출되어 실행되는 myInterrupt 함수
void myInterrupt(void) {
    eventCounter++;
    flag_detect = 0x01;
}
//메인함수
int main(void) {
  // sets up the wiringPi library
  if (wiringPiSetup () < 0) {
        fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
        return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
  //인터럽트 감지 에지 상태와 호출 함수 등록
  if ( wiringPiISR (MOTION_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
        fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
        return 1;
  }

   while ( 1 ) // 인체감지(인터럽트) 루프
  {
    if(flag_detect == 0x01)
   {
                printf("Motion Detect₩n");
```

```
                flag_detect = 0;
    }
     //eventCounter = 0;
     delay( 100 ); // wait 1 second
  }

  return 0;
}
```

컴파일하여 실행하면 Motion Detect 메시지를 볼 수 있다. 인체를 감지하면 약 3~4 초 동안 같은 상태를 유지하고 초기화 되는 식으로 동적하기 때문에 인지가 되어 있는 동안에는 다시 인지 하지 못한다.

실행 명령어는 $ sudo ./motiondetect 이고, 디텍트가 되면 하이 펄스를 감지하여 보여주는 화면이다.

```
pi@raspberrypi ~/smartguard $ sudo ./motiondetect
 No detect
 No detect
 No detect
 No detect
Detect 1
high
high
high
high
```

(2) 사운드센서

사운드 센서에 대한 설명은 지면 관계상 11 장 무인경비 시스템의 사운드 센서를 참조하기로 하고 여기서는 소스코드를 테스트 하는 방법을 설명한다. 11 장에서 설명한 사운드 센서 코드와는 약간 다르게 실습을 진행할 것이다.

Smartpet 폴더에 포함된 soundadc0.c 소스는 채널 ADC0 에 연결된 사운드 세서의 아날로그 값을 디지털 값으로 출력하는 소스로 11 장에서 설명한 코드와 동일하다. 우리는 이 소스에서 쓰레쉬 홀드를 정하여 기준 되는 소리 크기 이상으로 입력이 될 때 출력을 보여줄 것이다. 즉, 입력되는 소음중에서 소리 크기가 어느정도 이상일 때 메시지를 출력하고 그때의 adc 값을 보여준다.

```
adcValue_sound = read_mcp3208_adc(adcChannel_sound);


if(adcValue_sound < 2000)
{
     printf("sound over thresh hold : %u\n", adcValue_sound);
}
```

while 루프에서 위 구문이 사운드 센서로부터 입력값이 2000 보다 작을 경우에만, 즉 소리가 큰 경우에만 메시지를 출력하는 코드이다. 전체 소스를 아래에 보여준다. (soundadc1.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>
#include <wiringPiSPI.h>

#define CS_MCP3208 8 //GPIO 8

#define SPI_CHANNEL 0
#define SPI_SPEED 1000000 //1Mhz

// spi communication with Rpi and get sensor data
// 사운드 센서 아날로그 출력 쓰레쉬 홀드 확인인

int read_mcp3208_adc(unsigned char adcChannel)
{
   unsigned char buff[3];
   int adcValue = 0;

   buff[0] = 0x06 | ((adcChannel & 0x07) >> 2);
   buff[1] = ((adcChannel & 0x07) << 6);
   buff[2] = 0x00;

   digitalWrite(CS_MCP3208, 0);
```

```
    wiringPiSPIDataRW(SPI_CHANNEL, buff, 3);

    buff[1] = 0x0f & buff[1];
    adcValue = (buff[1] << 8 ) | buff[2];

    digitalWrite(CS_MCP3208, 1);

    return adcValue;
  }

  int main(void) {

    unsigned char adcChannel_sound = 0;

    int adcValue_sound = 0;

    printf("start");

    if(wiringPiSetupGpio() == -1)
    {
            fprintf(stdout,   "Unable   to   start   wiringPi   :%s\n",
strerror(errno));
            return 1;
    }

    if(wiringPiSPISetup(SPI_CHANNEL, SPI_SPEED) == -1)
    {
            fprintf(stdout,     "wiringPiSPISetup     Failed     :%s\n",
strerror(errno));
            return 1;
    }

    pinMode(CS_MCP3208, OUTPUT);

    while(1)
    {
            adcValue_sound = read_mcp3208_adc(adcChannel_sound);

            //printf("sound = %u\n", adcValue_sound);

        if(adcValue_sound < 2000)
        {
            printf("sound over thresh hold : %u\n", adcValue_sound);
        }

            //delay(10);
    }
    return 0;
  }
```

컴파일은 아래 명령으로 실행한다.

```
pi@raspberrypi ~/smartpet $ gcc -o soundadc1 soundadc1.c -lwiringPi
```

아래 명령으로 실행하자.

```
pi@raspberrypi ~/smartpet $ sudo ./soundadc1
```

실행하면 아래와 같은 결과를 볼 수 있다.

```
sound over thresh hold : 1993
sound over thresh hold : 1993
sound over thresh hold : 1993
sound over thresh hold : 1993
sound over thresh hold : 1995
sound over thresh hold : 1997
sound over thresh hold : 1999
```

이상으로 사운드 센서 실습을 마친다.

13.4 애완동물 도우미 액츄에이터 실습

IoT 애완농물 도우미에서 사용하는 액츄에이터는 서보모터이다. 서보에 맞는 제어 신호(PWM)를 보내면 제어신호를 보내는 시간 동안 서보모터는 그 각도를 계속 유지한다. 서보모터(RC 서보모터)는 일반적인 스텝모터나 DC 모터와는 달리 펄스의 주기에 따라 정방향회전, 역방향 회전 정지 동작을 수행하는 모터이다.

RC (Remote Control) 서보(Servo) 모터는 주로 무선 자동차 및 무선 모형 비행기에 주로 사용된다. 액츄에이터 연결도를 아래 그림에 나타내었다. 반드시 아래 그림과 같이 연결하지 않아도 되지만 실습시에는 이러한 그림대로 연결을 하고 실습하고 실습이 완료된 후에는 사용자의 설계 의도에 따라 다른 액츄에이터, 다른 연결 커넥터를 사용할 수 있다.

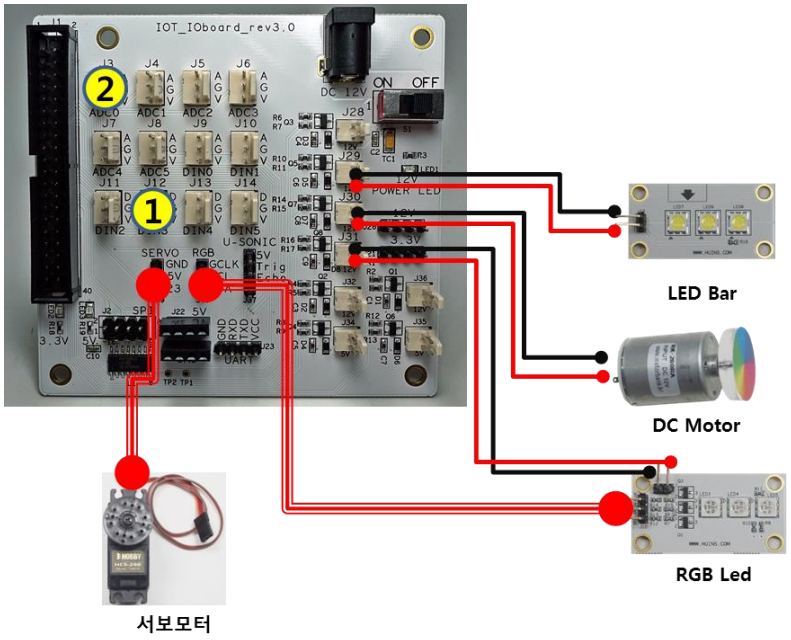


그림 161 애완동물 도우미 액츄에이터 연결도

(1) 서보모터 실습

서보모터는 별도의 부가회로 없이 간단한 로봇 콘트롤러에 접속해서 사용할수 있으며, 로봇 컨트롤러를 사용하면 -90°~+90° 의 범위내에서 원하는 각도로 제어가 가능하기 때문에 로봇의 관절 동작을 쉽게 제어 할수 있다.

표 35 서보모터 HES-288 사양 및 특징

서보종류	아날로그 서보모터
동작전압	4.8V ~ 6V
토크(Kg.cm)	2.5Kg / cm
스피드	0.2ch/60도
크기	
무게(g)	50g 이하
기어재질	프라스틱

특징	표준서보
회전각	180도(-90도 ~ +90도)



그림 162 서보모터의 외형

대게 RC 서보 모터의 선은 3 선으로 제어하는데 [검은선(갈색) : 그라운드], [적색선(빨강) : 전원(4.8~6V)], [노란선(주황) : 제어신호선(각도제어)] 이렇게 사용된다. 서보 모터의 3 개의 선은 신호 기능에 맞게 사용하여야 하며, 잘못 연결되면 서보 모터 내부의 회로가 손상을 입을수 있게 된다. 또 모터의 규정각도 이상으로 무리하게 움직이게 되면 내부의 기어가 망가지게 되므로 주의한다.

서보모터의 사용방법은 아래 그림과 같이 1.5msec 일 때는 중심각도(0)로 0.7msec 일 때는 오른쪽(+90), 2.3msec 일 때는왼쪽(-90)으로 동작한다. 서보모터로 현재의 위치를 계속 해서 유지하기 위해서는 위치 신호(PWM)를 10msec~ 20msec 정도 반복해서 보내 주어야 한다. 신호를 받지 못한 상태에서는 이 시간이 지나면, 서보 모터가 정지하게 되며, 따라서 정해진 위치를 벗어나게 되어도 이를 자동으로 복구하지 못하게 된다.

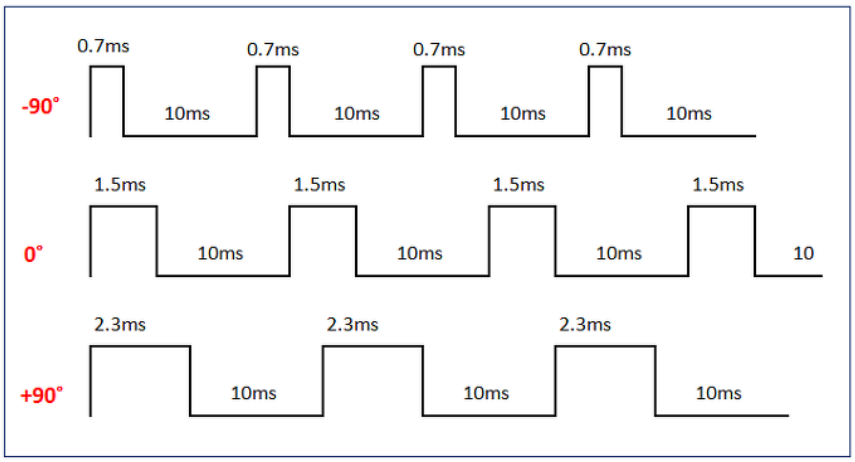


그림 163 서보모터의 PWM 신호

서보모터의 연결은 아래 그림과 같이 빨간색 선에 4~6V 전원을 공급 하고 주황색 선에는 제어신호(PWM)를 인가하고 갈색에는 GND 를 연결한다. 한가지 주의할 점은 노란색 선에 공급되는 제어신호 (PWM)의 신호의 펄스는 실제의 7~23%의 오차 범위에 있어야 정확한 제어를 할 수 있다.

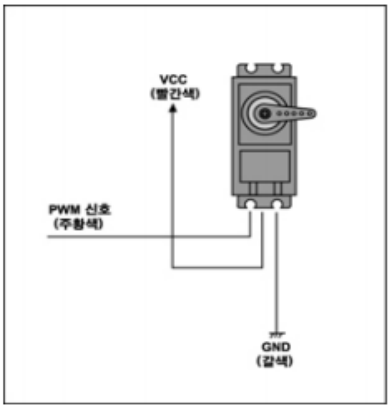


그림 164 서보모터 연결방법

RC 서보모터는 별도의 드라이버 칩을 사용하지 않는다. 다만, PWM 을 이용하기 위해서는 라즈베리 파이의 PWM 핀인 GPIO18 번을 이용하든지, 아니면 별도의 서프트웨어 방식의 PWM 을 사용하여야 한다. 여기서는 wiringPi 에서 제공하는 소프트웨어 PWM 방식을 사용하기로 한다.

작업 폴더를 생성하자.

```
$mkdir rcservotest
$cd rcservotest
```

우선 nano 편집기를 사용하여 소스코드를 작성한다. ($nano rcservo.c)

```
$nano rcservo.c

#include <stdio.h>
#include <stdlib.h>
#include <wiringPi.h>

#include "softServo.h"

#define RCSERVO   5//GPIO 5

int main(void)
{
  if (wiringPiSetupGpio() == -1)
    return 1 ;

  softServoSetup(RCSERVO,-1,-1,-1,-1,-1,-1,-1);

  printf("rc servo test ₩n");

  int i=0;

  for( i= 0; i<10 ; i++ ){
            printf("rc servo test ₩n");
            softServoWrite(RCSERVO, -250);
            delay(1000);
```

```
            softServoWrite(RCSERVO, 1250);
            delay(1000);

    };
}
```

위에서 호출하는 softServoWrite 함수가 softServo.c 에 있는 파일이다. 이 파일은 wiringPi 라이브러리 아래에 있다. softServo.c 와 softServo.h 를 복사하자.

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

현재 디렉토리를 나타내는 명령어는 $pwd 이다.

```
pi@raspberrypi ~/rcservotest $ pwd
/home/pi/rcservotest
pi@raspberrypi ~/rcservotest $ cp /home/pi/wiringPi/wiringPi/softServo.h .
pi@raspberrypi ~/rcservotest $ cp /home/pi/wiringPi/wiringPi/softServo.c .
pi@raspberrypi ~/rcservotest $
```

프로그램을 빌드하면

```
$gcc -o rcservo rcservo.c softServo.c -lwiringPi
```

프로그램을 실행하자.

```
$sudo ./rcservo
```

프로그램이 실행되면 정방향으로 1 초동안 회전하고 다시 역방향으로 1 초간 회전한다. 사용자의 설계대로 회전하고 규칙적으로 움직이도록 프로그램을 구성하면 된다.

(2) DC 모터 실습

한정된 지면 관계상 스마트 팜의 DC 모터 실습을 참고한다.

(3) LED Bar 실습

LED Bar 는 단순히 화이트 LED 를 연달아 붙여 놓은 것을 말한다. 흔히 조명으로 사용되는 LED 들이 대부분 Bar 로 구성되어 있다. Led 소자의 자세한 사양은 데이터북을 참조한다.
(http://www.snowdragonledhk.com/download/5050-smd-led/top-led-5050-smd-white-led-superbright.pdf)

표 36 Led 사양

MODEL N.O	SDS5050WTU-S-AP-A
ENG. N.O	120203002
Shape	8MM Round 5.0*5.4*1.6mm Surface Mount LEDs
Emitting Color	White
Lens Color	Yellow
Viewing Angle	117°
Chip Material	InGaN

LED bar 는 실내조명, 실외조명, 특수조명 등과 같은 많은 응용분야가 있다. 여기서는 실내 조명을 ON/OFF 하는 기능을 한다.



그림 165 LED bar 외관

LED bar 의 연결 회로도를 나타낸다.

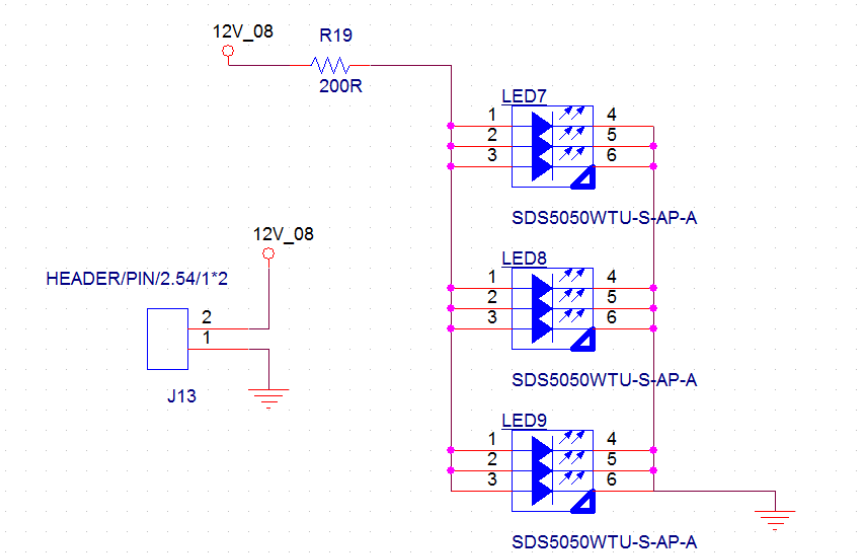


그림 166 LED bar 회로도와 전원공급 커넥터

LED Bar 는 GPIO 출력 포트 22 번, 12V 출력에 연결되어 있다. 아래와 같이 소스코드를 입력한다. (ledbar.c)

```
$nano ledbar.c  // 편집 명령


#include <signal.h> //Signal 사용 헤더파일
#include <unistd.h>
#include <stdio.h>
#include <string.h>
```

```
#include <errno.h>
#include <stdlib.h> //exit() 사용 헤더파일

#include <wiringPi.h>

#define LEDBAR     22 // BCM_GPIO 6

void sig_handler(int signo); // 마지막 종료 함수

int main (void)
{
   signal(SIGINT, (void *)sig_handler);    //시그널 핸들러 함수

   if (wiringPiSetup () == -1)
   {
            fprintf(stdout,   "Unable   to   start   wiringPi:   %s\n",
strerror(errno));
            return 1 ;
   }

   pinMode (LEDBAR, OUTPUT) ;

   for (;;)
   {
            printf("here - Led Bar on on\n");
            digitalWrite (LEDBAR, 1) ; // On

            delay (200) ; // ms

            printf("here - Led Bar on on\n");
            digitalWrite (LEDBAR, 0) ; // Off

            delay (200) ;

   }
   return 0 ;
}

void sig_handler(int signo)
{
    printf("process stop\n");
   digitalWrite (LEDBAR, 0) ; // Off

   exit(0);
}
```

아래 명령어로 컴파일을 실행한다.

```
$ gcc -o ledbar ledbar.c -lwiringPi
```

아래 명령어로 실행하게 되면 LED Bar 가 0.5 초 간격으로 점등하는 것을 볼 수 있다.

```
$sudo ./ledbar
```

다음은 11 장에서 실습한 LED Bar PWM 제어 실습을 실행한다. 소스코드는 11 장 무인경비 시스템의 ledbarsoftpwm.c 와 동일하고 GPIO 핀번호만 22 번으로 설정되었다.

컴파일하고 실행 명령어가 약간 복잡하므로 아래에 나타내었다.
캄파일 명령

```
$ gcc -o ledbarsoftpwm ledbarsoftpwm.c -lwiringPi -lpthread
```

아래와 같이 실행하면 밝기가 변하는 LED Bar 를 볼 수 있다.

```
$ sudo ./ledbarsoftpwm
here
here
here
```

또한 **ledbarfadeinout.c** 소스코드를 사용하여 LED Bar 의 밝기가 점점 밝아지면서 가장 밝게 되고, 이후 점점 어두워져서 꺼지게 되는 코드를 실습한다.

(4) RGB LED 실습

한정된 지면 관계상 스마트 농장의 RGB LED 실습을 참고한다. 다만 다양한 실습을 위하여 아래 소스 코드를 실습 코드에 추가 하였으니 실습해보기 바란다. Red, green, blue 색으로 변하면서 반족하는 소스코드 **rgbcircle.c**

(5) 통합 액츄에이터 실습

앞서 4 종의 액츄에이터에 대해 알아보았다. 다음 단계는 액츄에이터 4 종을 테스트하는 실습 과정이다. IO Adpater 보드의 J28, J29, J30 에는 각각 DC 모터,

LED bar, RGB LED 가 연결되어 있다. DC 모터, LED bar, RGB LED 는 TR 회로에 인가되는 GPIO 신호를 제어하여 전원을 On/Off 하여 동작시킨다. RGB Led 는 전원을 공급하고 Red, Blue, Green 신호선에 GPIO 신호를 공급하여 원하는 색을 출력한다.

다음 소스코드는 통합 액츄에이터 실습 프로그램으로 4 종의 액츄에이터를 일괄적으로 ON/OFF 한다. **(petactutest .c 파일)**

다음 소스코드는 통합 액츄에이터 실습 프로그램으로 4 종의 액츄에이터를 일괄적으로 ON/OFF 한다. **(actutest .c 파일)**

```
#include <stdio.h>
#include <string.h>
#include <errno.h>

#include <wiringPi.h>

#define PUMP          21        // BCM_GPIO 5
#define FAN           22                  // BCM_GPIO 6
#define DCMOTOR 23          // BCM_GPIO 13
#define RGBLEDPOWER   24 //BCM_GPIO 19

#define RED27
#define GREEN         28
#define BLUE          29

void Bplus_pinmodeset(void);
int testmode(void);

int main (void)
{
    if(wiringPicheck())
    {
```

```
                printf("Fail");
                return;
    }

    Bplus_pinmodeset();

    while(1)
    {
                int exit;
                exit = testmode();
                if(exit == 1) break;
    }
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
                fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
                return 1 ;
    }
}

void Bplus_pinmodeset(void)
{
    pinMode (PUMP, OUTPUT);
    pinMode (FAN, OUTPUT);
    pinMode (DCMOTOR, OUTPUT);
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
```

```
    pinMode(BLUE,OUTPUT);
}

int testmode(void)
{
    int testnum;

   printf("Input Number for Smart Farm Actuator Test(1 ~ 4) and Enter !!! ₩n");
    printf("1. Water Pump Test₩n");
    printf("2. Fan Test₩n");
    printf("3. DC Motor Test₩n");
    printf("4. RGB Led Test₩n");
    printf("5. Exit₩n");

    scanf("%d", &testnum);

    if(testnum == 1)
    {
            //printf("1 input₩n");
            digitalWrite (PUMP, 1);   //on
            delay (2000) ; // ms
            digitalWrite (PUMP, 0) ; // Off
    }

    if(testnum == 2)
    {
            digitalWrite (FAN, 1) ; // On
            delay (2000) ; // ms
            digitalWrite (FAN, 0) ; // Off
    }
```

```
    if(testnum == 3)
    {
            digitalWrite(DCMOTOR, 1); //On
            delay (2000) ; // ms
            digitalWrite(DCMOTOR, 0); //OFF
    }

    if(testnum == 4)
    {
            digitalWrite(RGBLEDPOWER, 1); // power on
            digitalWrite(RED, 1);
            delay(500);
            digitalWrite(RED, 0);
            digitalWrite(GREEN, 1);
            delay(500);
            digitalWrite(GREEN, 0);
            digitalWrite(BLUE, 1);
            delay(500);
            digitalWrite(BLUE, 0);
            digitalWrite(RGBLEDPOWER, 1); // power off
    }

    if(testnum == 5)
    {
            return 1;
    }
    return 0;

}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

```
$gcc -o actutest actutest.c -lwiringPi
```

실행한 결과를 보여주는 화면이다.

```
$sudo ./actutest
```

아래는 1 ~ 4 를 눌러 개별적인 액츄에이터를 실행하는 화면이다. 실험을 중단할때는 5 를 입력하여 빠져나온다.

```
pi@raspberrypi ~/smartfarm $ sudo ./actutest
Input Number for Smart Farm Actuator Test(1 ~ 4) and Enter !!!
1. Water Pump Test
2. Fan Test
3. DC Motor Test
4. RGB Led Test
5. Exit
5
pi@raspberrypi ~/smartfarm $ ^C
```

13.5 애완동물 도우미 시나리오 실습

애완동물 도우미의 전체 시나리오를 테스트 한다. 앞에서 설명한 전체 구조를 이해하고 센서와 액츄에이터 테스트를 잘 했다면 쉽게 따라 할 수 있을 것이다. 가급적이면 센서와 액츄에이터 연결을 바꾸지 말고 처음 개별적으로 테스트 하던 연결을 그대로 유지하고 실습을 진행한다. 새로 연결을 해야 한다면 아래 이미지를 참고한다.

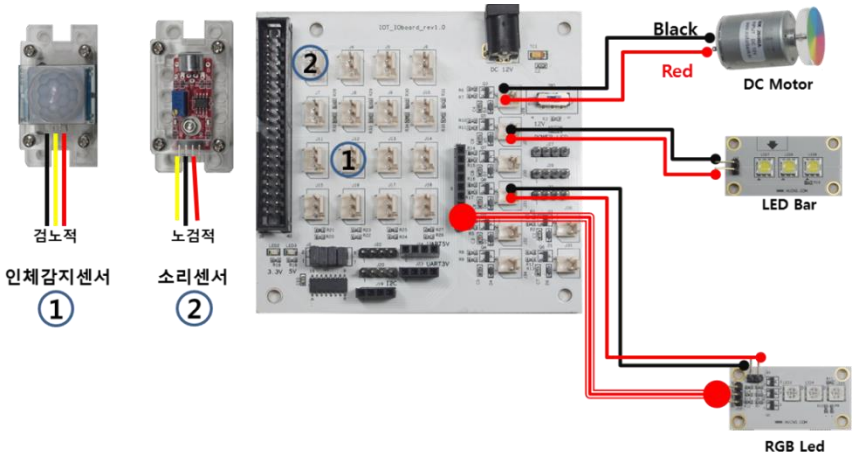


그림 167 센서 액츄에이터 연결도

기준이 되는 조도값이나 인체감지, 사운드 센서의 값은 현재 애완동물이 있는 장소의 환경에 따라 사용자가 센서 기준값을 임의로 정해주어도 상관이 없다. 식사 시간의 설정도 액츄에이터의 동작 확인을 위해 몇 분 뒤와 같은 임의 값을 주어도 된다.

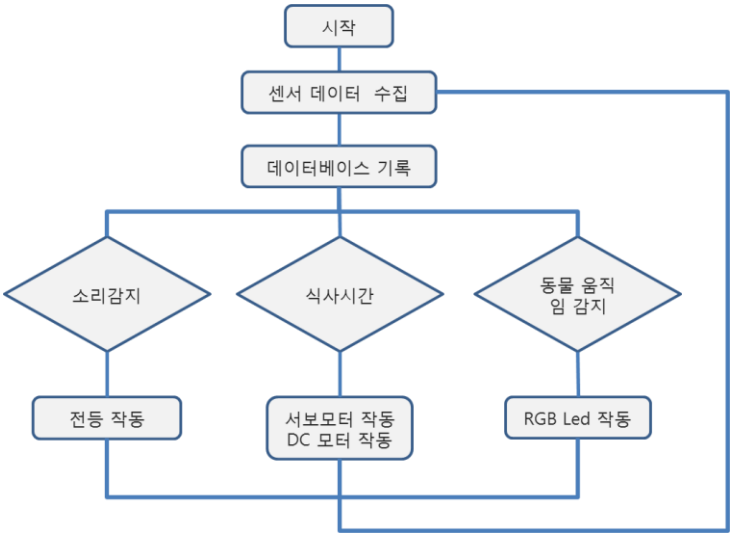


그림 168 애완동물 프로그램 시나리오

애완동물 도우미의 시험을 간단하게 나마 전체적으로 테스트 할 수 있도록 위와 같은 시나리오를 그리고 프로그램도 이 그림에 따라 진행한다.

여기서 중요한 것은 센서 데이터를 획득하여 사용자가 설정한 값보다 높거나 낮을 경우 원하는 액츄에이터를 동작시키는 것이다. 센서값이 기준 값에 미치지 못할 경우 손을 움직이거나 센서 근처에서 소리를 내는 등의 조치를 하여 액츄에이터의 작동을 쉽게 해주는 방법을 사용해도 된다.

13.6 스마트폰으로 액츄에이터 제어하기

(1) 하드웨어 연결도

앞장에서 센서 테스트, 액츄에이터 테스트 환경과 동일한 연결과 하드웨어를 사용한다.

(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍

안드로이드 앱을 이요한 원격제어를 하기 위해서는 라즈베리 파이가 TCP/IP 의 서버가 되고, 안드로이드 단말기가 TCP/IP 의 클라이언트로서 프로그래밍 해야 한다. 일단 안드로이드 단말기는 클라이언트로서 서버인 라즈베리 파이에 접속한다. 그 후 안드로이드 단말기(클라이언트) 화면의 액츄에이터 제어 버튼을 클릭하면 정해진 문자열을 라즈베리파이(서버)로 전송하고, 라즈베리 파이는 그에 따른 처리를 수행하고 클라이언트에 결과를 전송하는 프로그램을 작성할 것이다.

즉, 서버 프로그래밍(라즈베리 파이), 클라이언트 프로그래밍(안드로이드 단말기)의 두 가지 작업이 필요하다. 안드로이드 단말기를 이용한 TCP/IP 프로그래밍 소스코드는 코드량이 많기 때문에 CD 의 스마트 폰 제어 프로그램 폴더의 코드를 참고하고 여기서는 대략적으로 코딩과정과 순서를 설명한다.

1. 프로젝트 생성
2. string.xml 에 각종 string 을 구성한다.
3. activit_main 을 편집하여 화면을 구성한다.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context=".actu_p2rcontrol" >

    <ToggleButton
        android:id="@+ id/buttonRightTurn"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_above="@+ id/buttonDown"
        android:layout_toRightOf="@+ id/buttonUp"
        android:text="@string/Right"
        android:textOff="Fan Off"
        android:textOn="Fan On" />

    <ToggleButton
        android:id="@+ id/buttonLeftTurn"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonUp"
        android:layout_toLeftOf="@+ id/buttonDown"
        android:text="@string/Left"
        android:textOff="Motor Off"
        android:textOn="Motor On" />



    <ToggleButton
        android:id="@+ id/buttonUp"
```

```
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/textViewStatus"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="37dp"
        android:text="@string/Up"
        android:textOff="펌프 Off"
        android:textOn="펌프 On" />

    <ToggleButton
        android:id="@+ id/buttonDown"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonLeftTurn"
        android:layout_centerHorizontal="true"
        android:text="@string/Down"
        android:textOff="RGB Off"
        android:textOn="RGB On" />

    <Button
        android:id="@+ id/buttonConnect"
        android:layout_width="120dp"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
        android:layout_marginTop="66dp"
        android:layout_toLeftOf="@+ id/buttonUp"
        android:text="@string/Connect" />

    <TextView
        android:id="@+ id/textViewStatus"
        android:layout_width="wrap_content"
```

```
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonConnect"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="20dp"
        android:text="@string/DataView"
        android:textSize="20sp" />

    <EditText
        android:id="@+ id/editTextIPAddress"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+ id/buttonClose"
        android:layout_centerHorizontal="true"
        android:layout_marginBottom="18dp"
        android:ems="10"
        android:gravity="center_vertical|center_horizontal"
        android:inputType="text"
        android:text="@string/IP" />

    <Button
        android:id="@+ id/buttonClose"
        android:layout_width="120dp"
        android:layout_height="wrap_content"
        android:layout_alignBaseline="@+ id/buttonConnect"
        android:layout_alignBottom="@+ id/buttonConnect"
        android:layout_toRightOf="@+ id/buttonUp"
        android:text="@string/Close" />

</RelativeLayout>
```

클라이언트로 동작하는 안드로이드 프로그램은 3 개의 소스 파일로 구성되었다. 데이터를 보여주는 Logger 와 데이터를 파싱하여 메인 화면을 제어해주는 파일, 그리고 데이처 수신 쓰레드이다. Logger 와 수신 쓰레드는 변경사항이 없으므로 소스를 참고하고 여기서는 액츄에이터를 제어하는 소스를 보여준다.

```
package huins.iot.actucontrol;

import android.os.Bundle;
import android.os.StrictMode;
import android.app.Activity;
import android.content.Context;
import android.graphics.Color;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.view.inputmethod.InputMethodManager;
import android.widget.TextView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ToggleButton;

import java.net.Socket;
import java.io.IOException;
import java.io.OutputStream;

public class actu_p2rcontrol extends Activity implements OnClickListener{

    private EditText editTextIPAddress;
    private TextView textViewStatus;
    private Button buttonConnect;
    private Button buttonClose;
    private ToggleButton buttonUp;
```

```
    private ToggleButton buttonLeftTurn;
    private ToggleButton buttonRightTurn;
    private ToggleButton buttonDown;



    private InputMethodManager imm;
    private String server = "192.168.1.238";
    private int port = 8888;
    //private int port = 8080;
    private Socket socket;

    private OutputStream outs;
    private Thread rcvThread;
    public logger logger;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);

            editTextIPAddress =
(EditText)this.findViewById(R.id.editTextIPAddress);
            editTextIPAddress.setText(server);
            imm =
(InputMethodManager)getSystemService(Context.INPUT_METHOD_SERVICE);

           textViewStatus = (TextView)this.findViewById(R.id.textViewStatus);
            textViewStatus.setText("스마트폰 액츄에이터 테스트");

            logger = new logger(textViewStatus);
```

```
                buttonConnect    = (Button)this.findViewById(R.id.buttonConnect);
                buttonClose      = (Button)this.findViewById(R.id.buttonClose);
                buttonUp         = (ToggleButton)this.findViewById(R.id.buttonUp);
                buttonLeftTurn                                                  =
(ToggleButton)this.findViewById(R.id.buttonLeftTurn);
                buttonRightTurn                                                 =
(ToggleButton)this.findViewById(R.id.buttonRightTurn);
                buttonDown                                                      =
(ToggleButton)this.findViewById(R.id.buttonDown);

                buttonConnect.setOnClickListener(this);
                buttonClose.setOnClickListener(this);
                buttonUp.setOnClickListener(this);
                buttonLeftTurn.setOnClickListener(this);
                buttonRightTurn.setOnClickListener(this);
                buttonDown.setOnClickListener(this);

        }

        @Override
        public boolean onCreateOptionsMenu(Menu menu) {
                getMenuInflater().inflate(R.menu.main, menu);
                return true;
        }
        @Override
        public void onClick(View arg0) {

                StrictMode.ThreadPolicy          policy          =          new
StrictMode.ThreadPolicy.Builder()
                .detectAll()
                .penaltyLog()
```

```
                .build();
                StrictMode.setThreadPolicy(policy);

                if(arg0 == buttonConnect)
                {

    imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

                        try{
                                if(socket!=null)
                                {
                                        socket.close();
                                        socket = null;
                                }

                                server = editTextIPAddress.getText().toString();
                                socket = new Socket(server, port);
                                outs = socket.getOutputStream();

                                rcvThread = new Thread(new rcvthread(logger,
socket));
                            rcvThread.start();
                                logger.log("Connected");
                        } catch (IOException e){
                                logger.log("Fail to connect");
                                e.printStackTrace();
                        }
                }

                if(arg0 == buttonClose)
                {
```

```
imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

                if(socket!=null)
                {
                        exitFromRunLoop();
                        try{
                                socket.close();
                                socket = null;
                                logger.log("Closed!");
                                rcvThread = null;
                        } catch (IOException e){
                                logger.log("Fail to close");
                                e.printStackTrace();
                        }
                }
        }


        if(arg0 == buttonUp)
        {
                String sndOpkey = "CMD";

                if(buttonUp.isChecked()){
                        buttonUp.setTextColor(Color.RED);
                        sndOpkey = "Pumpon";
                }

                else{
                        buttonUp.setTextColor(Color.BLACK);
                        sndOpkey = "Pumpoff";
```

```
                }
                try{
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
                } catch (IOException e){
                        logger.log("Fail to send");
                        e.printStackTrace();
                }
        }

        if(arg0 == buttonLeftTurn)
        {
                String sndOpkey = "CMD";

                if(buttonLeftTurn.isChecked()){
                        buttonLeftTurn.setTextColor(Color.RED);
                        sndOpkey = "Motoron";
                }

                else{
                        buttonLeftTurn.setTextColor(Color.BLACK);
                        sndOpkey = "Motoroff";
                }
                try{
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
                } catch (IOException e){
                        logger.log("Fail to send");
                        e.printStackTrace();
                }
        }
```

```
if(arg0 == buttonRightTurn)
{
        String sndOpkey = "CMD";

        if(buttonRightTurn.isChecked()){
                buttonRightTurn.setTextColor(Color.RED);
                sndOpkey = "Fanon";
        }

        else{
                buttonRightTurn.setTextColor(Color.BLACK);
                sndOpkey = "Fanoff";
        }
        try{
                outs.write(sndOpkey.getBytes("UTF-8"));
                outs.flush();
        } catch (IOException e){
                logger.log("Fail to send");
                e.printStackTrace();
        }
}

if(arg0 == buttonDown)
{
        String sndOpkey = "CMD";

        if(buttonDown.isChecked()){
                buttonDown.setTextColor(Color.RED);
                sndOpkey = "Rgbon";
        }
```

```
                    else{
                        buttonDown.setTextColor(Color.BLACK);
                        sndOpkey = "Rgboff";
                    }
                    try{
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
                    } catch (IOException e){
                        logger.log("Fail to send");
                        e.printStackTrace();
                    }
                }

    }

     void exitFromRunLoop(){
            try {
                String sndOpkey = "[close]";
                outs.write(sndOpkey.getBytes("UTF-8"));
                outs.flush();
            } catch (IOException e) {
                logger.log("Fail to send");
                e.printStackTrace();
            }
     }
}
```

(3) 라즈베리 파이 TCP/IP 서버 프로그래밍

다 언어를 지원하는 TCP/IP 는 언어의 제한을 받지 않는다. 여기서는 참고 코드를 쉽게 구할 수 있고, 많은 예제들이 있는 자바를 사용해서 프로그래밍 하자. 라즈베리 파이에서 자바를 사용하기 위해서는 하드웨어 연산을 지원하는 JDK8 의 설치가 필요하다. JDK8 의 설치는 복잡하지만 어렵지 않으므로 순차적으로 실습해보자.

JDK8 설치에 관해 영문이지만 설명이 잘 되어있다.
http://yoo7577.tistory.com/181

JDK8 을 다운로드 하기 위해서이곳을 방문하고 아래 그림과 같은 파일을 저장한다.

http://www.oracle.com/technetwork/java/javase/downloads/jdk8-arm-downloads-2187472.html

Java SE Development Kit 8 Update 6 for ARM

You must accept the Oracle Binary Code License Agreement for Java SE to download this software.

Thank you for accepting the Oracle Binary Code License Agreement for Java SE; you may now download this software.

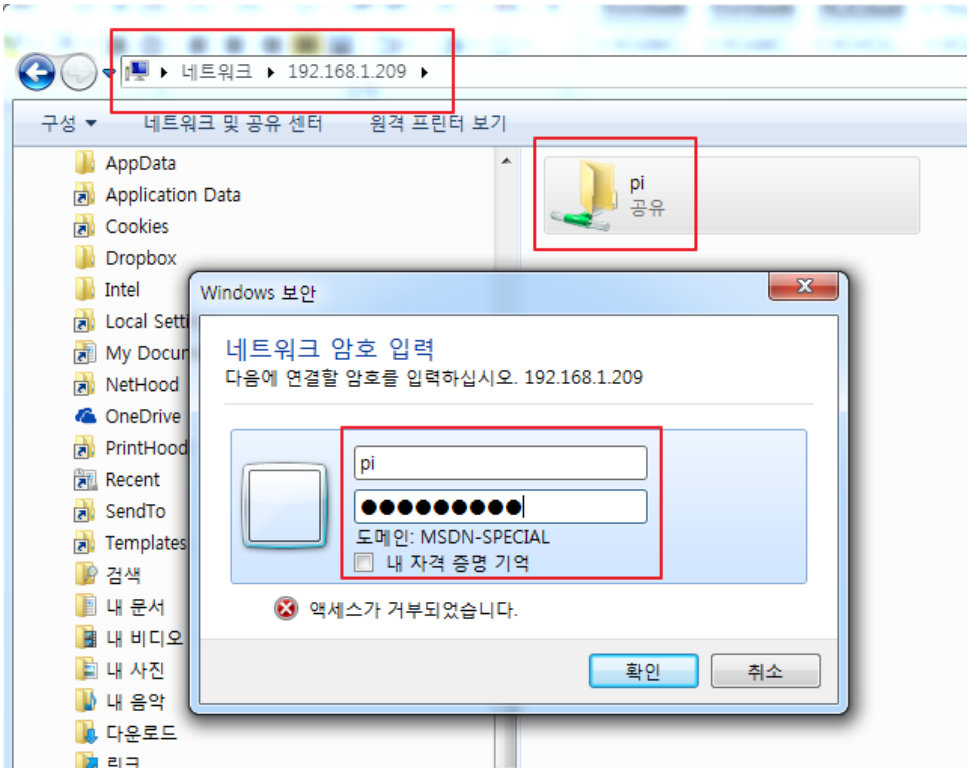
Product / File Description	File Size	Download
Linux 32-bit ARM v6/v7 Hard Float ABI	83.81 MB	jdk-8u6-linux-arm-vfp-hflt.tar.gz

윈도우에서 다운 로드 한 위치를 기억한다.

아래 명령어로 라즈베리 파이의 주소를 확인하고 네트워크로 접속해준다.

```
pi@raspberrypi ~ $ ifconfig
eth0      Link encap:Ethernet  HWaddr b8:27:eb:0b:a7:4d
          inet addr:192.168.1.209  Bcast:192.168.1.255  Mask:255.255.255.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:80528 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1942 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:5721058 (5.4 MiB)  TX bytes:216975 (211.8 KiB)
```

아이피 주소는 192.168.1.209 로 확인되었다.



네트워크 창을 열고 아이피 주소를 입력하고 /home/pi/ (작업 폴더임)에 다운 받은 파일을 옮기고 아래는 라즈베리 파이에서 작업한다.

아래와 같은 명령어로 /opt/java 라는 폴더를 생성하자.

$sudo mkdir -p -v /opt/java

현재 까지 하면 폴더구조는 아래와 같다.

```
pi@raspberrypi ~ $ cd ..
pi@raspberrypi /home $ cd ..
pi@raspberrypi / $ ls
bin boot dev etc home lib lost+found media mnt opt proc root
run sbin selinux srv sys tmp usr var
pi@raspberrypi / $
pi@raspberrypi / $ cd opt
pi@raspberrypi /opt $ ls
java minecraft-pi sonic-pi vc Wolfram
pi@raspberrypi /opt $ cd java
```

```
pi@raspberrypi /opt/java $ ls
pi@raspberrypi /opt/java $ cd ..
pi@raspberrypi /opt $ cd ..
pi@raspberrypi / $ cd home
pi@raspberrypi /home $ cd pi
pi@raspberrypi ~ $ pwd
/home/pi
pi@raspberrypi ~ $
```

다운로드한 폴더로 이동한다.

/opt/java 폴더에 다운로드한 파일을 압축해제 한다.

sudo tar xvzf ~/ jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java

```
pi@raspberrypi ~ $ sudo tar xvzf ~/jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java
jdk1.8.0_06/COPYRIGHT
jdk1.8.0_06/LICENSE
jdk1.8.0_06/README.html
jdk1.8.0_06/THIRDPARTYLICENSEREADME-JAVAFX.txt
jdk1.8.0_06/THIRDPARTYLICENSEREADME.txt
jdk1.8.0_06/bin/
jdk1.8.0_06/bin/javah
jdk1.8.0_06/bin/appletviewer
jdk1.8.0_06/bin/extcheck
jdk1.8.0_06/bin/jps
jdk1.8.0_06/bin/wsimport
jdk1.8.0_06/bin/unpack200
jdk1.8.0_06/bin/jarsigner
.
.
.
```

잠시 후 압축이 풀리고 Oracle JDK gz 파일들이 “/opt/java” 폴더의 아래에 “jdk1.8.0_06”폴더에 위치하게 된다.

다운로드 받은 파일을 더 이상 필요없기에 삭제한다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ rm ~/jdk-8u6-linux-arm-vfp-hflt.gz
pi@raspberrypi ~ $
```

시스템에게 새로운 JDK 가 인스톨 되었고, 위치를 알려주기 위해 아래와 같이 실행한다.

```
pi@raspberrypi ~ $ sudo update-alternatives --install "/usr/bin/java" "java" "/opt/java/jdk1.8.0_06/bin/java" 1
pi@raspberrypi ~ $
```

마지막으로 JDK 가 시스템의 디폴트 java 런타임이 될것이라고 알려준다.

```
pi@raspberrypi ~ $ sudo update-alternatives --set java /opt/java/jdk1.8.0_06/bin/java

pi@raspberrypi ~ $
```

이제 JDK8 이 설치되었다. 자바 버전명령을 사용해 아래와 같이 나온다면 잘 설치된 것이다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ java -version
java version "1.8.0_06"
Java(TM) SE Runtime Environment (build 1.8.0_06-b23)
Java HotSpot(TM) Client VM (build 25.6-b23, mixed mode)
pi@raspberrypi ~ $
```

진짜 마지막으로 JAVA_HOME 환경 변수들을 몇가지 더해야 한다. 우선 bashrc 선언문을 마지막에 추가해 준다.

```
sudo nano ~/.bashrc

export JAVA_HOME=/opt/jdk1.8.0
export PATH=$PATH:$JAVA_HOME/bin
```

를 마지막에 적어주고 ctrl-x 누르고 y 누르고 Enter 치고 빠져 나온다.

Reboot 를 수행한다. 부팅을 수행하고 나면 Java 샘플 프로그램 작성 및 동작 테스트를 실행하자.

소스코드 생성 및 작성 - nano 편집기를 이용하여 아래와 같은 샘플 코드를 작성하자.

```
$nano HelloWorld.java

public class HelloWorld
{
    public static void main(String args[])
    {
        java.lang.System.out.println("Hello World!!!");
    }
}
```

아래와 같이 컴파일을 수행한다.

```
$javac HelloWorld.java
```

실행한다. Classpath 지정을 잊지 말자.

```
$java -cp . HelloWorld

Hello World!!!
```

메시지가 보이면 설치와 테스트가 완료된 것이다.

여기까지 힘들게 실습하였다. 이제 라즈베리파이에서 JDK8 (with javaFX) 를 사용할 수 있게 되었다. 지금까지 라즈베리파이는 파이썬, C, C++, Java 등 정말 다양한 언어를 사용할 수 있게 되었는데~~ 이것은 단지 툴에 지나지 않다. 이를 이용하여 응용프로그램을 작성해 보도록 하자.

이제 부터는 쉬운 과정이다. 라즈베리파이의 GPIO 를 제어하기 위한 pi4j 라는 자바용 드라이브를 설치한다. pi4j 를 다운로드 하고 설치를 해준다. 아래는 명령어와 설치화면을 보여준다.

$curl -s get.pi4j.com | sudo bash

```
pi@raspberrypi ~ $ curl -s get.pi4j.com | sudo bash
=======================================================
INSTALLING Pi4J GPG PUBLIC KEY
=======================================================
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1761  100  1761    0     0   3134      0 --:--:-- --:--:-- --:--:--  5889
OK
=======================================================
ADDING Pi4J APT REPOSITORY
=======================================================
--2015-01-17 14:46:04--  http://get.pi4j.com/pi4j.list
Resolving get.pi4j.com (get.pi4j.com)... 54.231.17.212
Connecting to get.pi4j.com (get.pi4j.com)|54.231.17.212|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 42 [application/octet-stream]
Saving to: `/etc/apt/sources.list.d/pi4j.list'

100%[=======================================================>] 42          --.-K/s   in 0s

2015-01-17 14:46:04 (547 KB/s) - `/etc/apt/sources.list.d/pi4j.list' saved [42/42]
```

```
==============================================================
UPDATING APT REPOSITORIES
==============================================================
Hit http://repository.pi4j.com wheezy Release.gpg
Hit http://repository.pi4j.com wheezy Release
Hit http://repository.pi4j.com wheezy/rpi armhf Packages
Ign http://repository.pi4j.com wheezy/rpi Translation-en_GB
Ign http://repository.pi4j.com wheezy/rpi Translation-en
Reading package lists... Done
==============================================================
INSTALLING Pi4J
==============================================================
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following NEW packages will be installed:
  pi4j
0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
Need to get 0 B/517 kB of archives.
After this operation, 793 kB of additional disk space will be used.
Selecting previously unselected package pi4j.
(Reading database ... 87830 files and directories currently installed.)
Unpacking pi4j (from .../pi4j_1.0~SNAPSHOT_all.deb) ...
Setting up pi4j (1.0~SNAPSHOT) ...
==============================================================
Pi4J INSTALLATION COMPLETE
==============================================================


The Pi4J JAR files are located at:
/opt/pi4j/lib
```

```
Example Java programs are located at:
/opt/pi4j/examples

You can compile the examples using this script:
sudo /opt/pi4j/examples/build

Please see http://www.pi4j.com for more information.

pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

위와 같이 설치하고 테스트를 진행할 때 에러나 나는 경우 언인스톨 명령어와 재설치를 아래에 나타낸다. 아래의 명령어로 언인스톨을 실행한다.

$ sudo apt-get remove pi4j

```
pi@raspberrypi ~ $ sudo apt-get remove pi4j
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  pi4j
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
1 not fully installed or removed.
After this operation, 793 kB disk space will be freed.
Do you want to continue [Y/n]? y
(Reading database ... 87881 files and directories currently installed.)
Removing pi4j ...
```

재설치

$ curl -s get.pi4j.com | sudo bash

안드로이드 스마트 폰(TCP/IP 클라이언트)과의 TCP/IP 통신을 위하여 쓰레드를 통해 데이터를 송수신 한다. 복수의 접속도 허용하고 있으며, 자원의 효율성도 고려하였다.

다음의 코드를 참고한다. farm_actu.java

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Logger;

public class Farmactu {
    private static final int sizeBuf = 50;
    public static void main(String[] args) throws IOException {

        int port = 8888;
        ServerSocket serverSock = new ServerSocket(port);
        Logger logger = Logger.getLogger("Farmactu");

        while (true) {
            Socket clientSock = serverSock.accept();
            SocketAddress clientAddress =
clientSock.getRemoteSocketAddress();
            Thread thread = new Thread(new RcvThread(clientSock,
clientAddress, logger));
            thread.start();
            if(clientSock.isConnected())
                System.out.println("Connected! Client IP : " + clientAddress);
        }
    }
```

TCP/IP 서버로 작동하는 소스코드다. 쓰레드로 동작하는 RcvThread.java 소스코드도 참고한다.

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Level;
import java.util.logging.Logger;

import com.pi4j.io.gpio.GpioController;
import com.pi4j.io.gpio.GpioFactory;
import com.pi4j.io.gpio.GpioPinDigitalOutput;
import com.pi4j.io.gpio.PinState;
import com.pi4j.io.gpio.RaspiPin;
import com.pi4j.io.gpio.impl.PinImpl;   //this!!!!

public class RcvThread implements Runnable{
        private static final int sizeBuf = 50;
        private Socket clientSock;
        private Logger logger;
        private SocketAddress clientAddress;

        final GpioController gpio = GpioFactory.getInstance();

        final GpioPinDigitalOutput pinlpwm =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_21, "lpwm", PinState.LOW);
        final GpioPinDigitalOutput pinlfwd =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_23, "lfwd", PinState.LOW);
        final GpioPinDigitalOutput pinlrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_22, "lrev", PinState.LOW);
```

```
        final GpioPinDigitalOutput rgbpwr =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_24, "rpwm", PinState.LOW);

        final GpioPinDigitalOutput rgbblue =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_27, "rfwd", PinState.LOW);

        final GpioPinDigitalOutput pinrrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_03, "rrev", PinState.LOW);

        public RcvThread(Socket clntSock, SocketAddress clientAddress, Logger
logger) {
            this.clientSock = clntSock;
            this.logger = logger;
            this.clientAddress = clientAddress;
        }

        public void run(){
            try {
                InputStream ins     = clientSock.getInputStream();
                OutputStream outs = clientSock.getOutputStream();

                int rcvBufSize;
                byte[] rcvBuf = new byte[sizeBuf];
                while ((rcvBufSize = ins.read(rcvBuf)) != -1) {

                    String rcvData = new String(rcvBuf, 0, rcvBufSize, "UTF-8");

                                if (rcvData.compareTo("Pumpon") == 0) {
                        pinlpwm.high();
                        System.out.println("Pumpon");
```

```
}
                if (rcvData.compareTo("Pumpoff") == 0) {
    pinlpwm.low();
    System.out.println("Pumpoff!");
}



if (rcvData.compareTo("Motoron") == 0) {
    pinlfwd.high();
                        System.out.println("Motoron");
                }
                if (rcvData.compareTo("Motoroff") == 0) {
    pinlfwd.low();
    System.out.println("Motoroff");
}

                if (rcvData.compareTo("Fanon") == 0) {
    pinlrev.high();
                        System.out.println("Fanon");
                }
                if (rcvData.compareTo("Fanoff") == 0) {
    pinlrev.low();
    System.out.println("Fanoff");
}
                if (rcvData.compareTo("Rgbon") == 0) {
    rgbpwr.high();
                        rgbblue.high();
                        System.out.println("Rgbon");
                }
                if (rcvData.compareTo("Rgboff") == 0) {
    rgbpwr.low();
```

```
                                        rgbblue.low();
                        System.out.println("Rgboff");
                    }




                logger.info("Received data : " + rcvData + " (" + clientAddress +
")");
                    outs.write(rcvBuf, 0, rcvBufSize);
                }
                logger.info(clientSock.getRemoteSocketAddress() + " Closed");
            } catch (IOException ex) {
                logger.log(Level.WARNING, "Exception in RcvThread", ex);
            } finally {
                try {
                    clientSock.close();
                    System.out.println("Disconnected! Client IP : " + clientAddress);
                } catch (IOException e) {}
            }
        }
    }
```

아래와 같은 명령어로 컴파일을 수행한다.

$javac -classpath .:classes:/opt/pi4j/lib/'*' Farmactu.java

아래와 같은 명령어로 실행

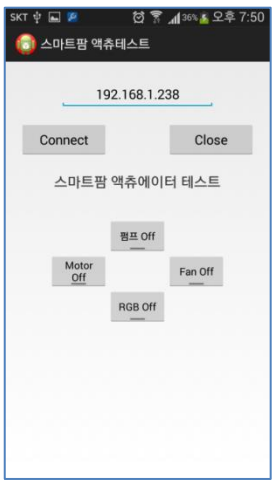
$sudo java -classpath .:classes:/opt/pi4j/lib/'*' Farmactu

라즈베리 파이 화면에서 보여지는 실행화면을 아래에 표시하였다

```
pi@raspberrypi ~/racar-test $ sudo java -classpath .:classes:/opt/pi4j/lib/'*'
Farmactu
Connected! Client IP : /192.168.1.98:37403
Pumpon
Jan 17, 2015 7:53:33 PM RcvThread run
```

```
INFO: Received data : Pumpon (/192.168.1.98:37403)
Pumpoff!
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Pumpoff (/192.168.1.98:37403)
Motoron
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Motoron (/192.168.1.98:37403)
Motoroff
Jan 17, 2015 7:53:36 PM RcvThread run
INFO: Received data : Motoroff (/192.168.1.98:37403)
Fanon
Jan 17, 2015 7:53:37 PM RcvThread run
INFO: Received data : Fanon (/192.168.1.98:37403)
Fanoff
Jan 17, 2015 7:53:39 PM RcvThread run
INFO: Received data : Fanoff (/192.168.1.98:37403)
Rgbon
Jan 17, 2015 7:53:39 PM RcvThread run
INFO: Received data : Rgbon (/192.168.1.98:37403)
Rgboff
Jan 17, 2015 7:53:40 PM RcvThread run
INFO: Received data : Rgboff (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: Received data : [close] (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: /192.168.1.98:37403 Closed
Disconnected! Client IP : /192.168.1.98:37403
```

아래는 안드로이드 스마트 폰으로 시험하는 화면을 보여준다.







13.7 안드로이드 앱 프로그램

이 장에서는 안드로이드 앱과 스마트 농장 관리 시스템의 연동에 대해 설명한다. 스마트 농장관리 시스템의 스마트 폰 앱은 안드로이드 OS 기반의 스마튼 폰에서 수행되는 Application 으로 아래와 같은 기능을 가지고 있다.

○ 애완동물이 있는 실내의 현재 상태를 조회.
○ 애완동물의 현재 상태를 카메라로 영상 확인
○ 애완동물의 존재 감지, 소리 감지하여 전송
○ 전해진 시간에 애완동물 밥주기 - 액츄에이터 가동
○ 애완동물 주변에 설치된 Actuator 의 원격 제어

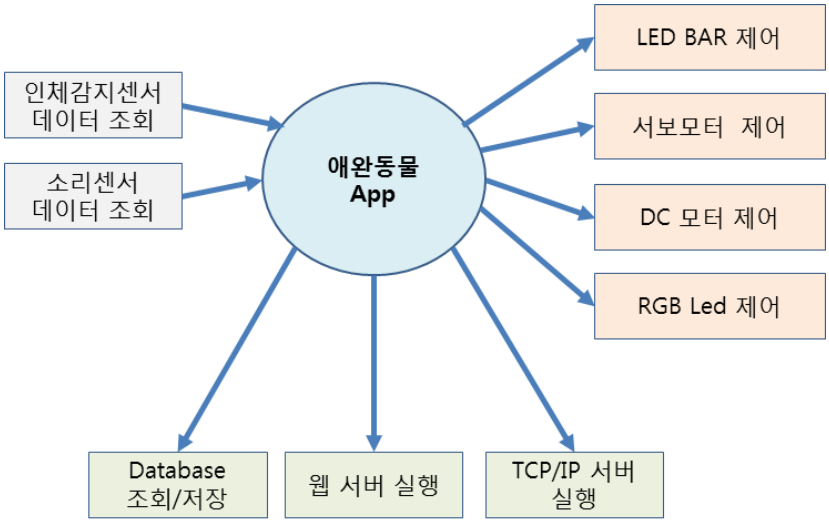


그림 169 애완동물 안드로이드 앱 S/W 구성

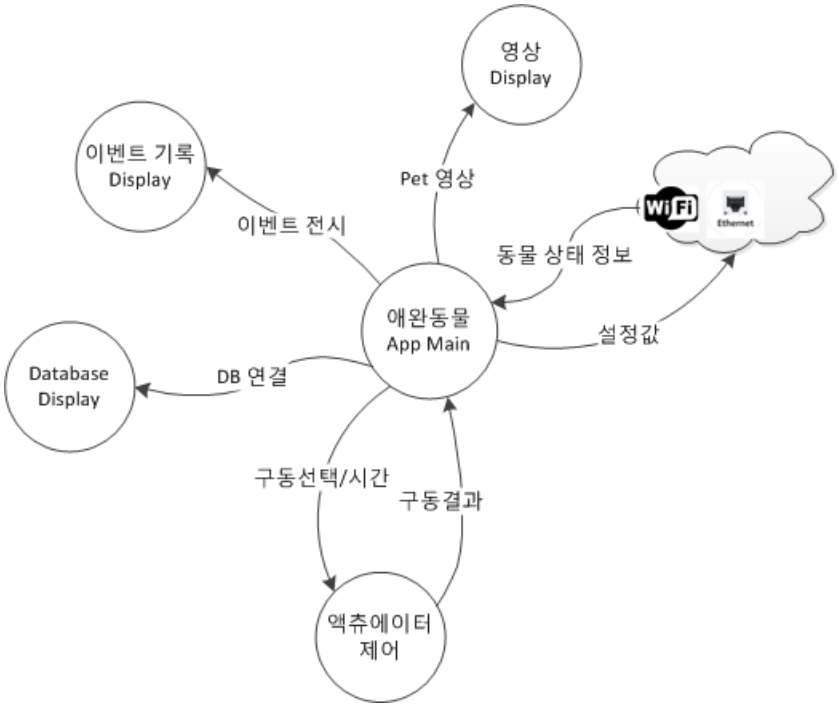
위에 설명된 기능에 더하여 사용자는 많은 조합된 기능을 추가하여 설계가 가능하다. 아래는 Application 으로 구현된 사용자 UI 화면을 나타낸다. 정밀하고 아름다운 UI 는 사용자의 환경과 의도에 맞게 구현해야 한다.

$pwd 명령은 현재 디렉토리를 알려주는 명령어이다.

라즈베리 파이의 smartfasrm_server 폴더에서 server 를 실행시키고 스마트 폰앱을 실행하면 서버와 스마트 폰 간의 통신이 시작되면서 온습도를 확인하고, 액츄에이터를 제어 할 수 있다.

```
pi@raspberrypi ~ $ cd iot_petserver
pi@raspberrypi ~/iot_petserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_petserver $ cd output
pi@raspberrypi ~/iot_petserver/output $ ls
mjpg.sh  server_pet
pi@raspberrypi ~/iot_petserver/output $
```

스마트폰 스마트 농장 시스템 소프트웨어의 DFD 를 나타낸다.



여기서는 프로그램 구성을 설명하고 스마트 폰 앱을 실행하여 동영상을 확인하고 다양한 액츄에이터를 제어하는 실습을 진행한다. 소스코드의 구성은 사용자 스스로 분석하여 다양한 환경에 적용하도록 하자.

아래에 스마트 농장의 서버와 앱이 연동되는 테스트 과정을 설명한다. 이대로 시험하면 결과를 확인할 수 있고 나머지는 참고로 이용하길 바란다.

1. 라즈베리 파이 전원을 켜고, IO 어댑터 보드의 12V 전원을 켠다.
2. 로그인 아이디 : pi password : raspberry (화면에 나타나지 않음)
3. 라즈베리 파이 아이피 주소 : $ifconfig 명령어로 확인한다.
   예)192.168.1.219
4. 라즈베리 파이 스마트 팜 서버 실행

```
pi@raspberrypi ~ $ cd iot_toyserver
pi@raspberrypi ~/iot_toyserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_toyserver $ cd output
pi@raspberrypi ~/iot_toyserver/output $
```

```
pi@raspberrypi ~/iot_toyserver/output $ ls
mjpg.sh   server_toy
pi@raspberrypi ~/iot_toyserver/output $
pi@raspberrypi ~/iot_toyserver/output $ sudo ./servertoy &
```

(백그라운드 실행) 엔터 입력하여 터미널 화면에서

5. 동영상 스트리밍 서버 실행 : $sh mjpg.sh
6. 스마트 폰 앱 실행 pi@raspberrypi ~ $ **sh mjpg.sh**

이 과정을 통해 연동 테스트를 수행한다.

라즈베리 파이의 서버 프로그램 위치는 iot_farmserver 폴더이다.

서버 프로그램을 컴파일하기 위해 $make clean 명령을 사용하여 이전 컴파일시 생성된 파일을 지우는 명령이다.

```
pi@raspberrypi ~/iot_petserver $ make clean
rm -rf /home/pi/iot_petserver/output
Clearing in libs...
make[1]: Entering directory '/home/pi/iot_petserver/libs'
rm -rf iot_db.o parser.o libiot.a core
make[1]: Leaving directory '/home/pi/iot_petserver/libs'
Clearing in src...
make[1]: Entering directory '/home/pi/iot_petserver/src'
Clearing in pet...
make[2]: Entering directory '/home/pi/iot_petserver/src/pet'
rm -rf pet_main.o core
make[2]: Leaving directory '/home/pi/iot_petserver/src/pet'
make[1]: Leaving directory '/home/pi/iot_petserver/src'
pi@raspberrypi ~/iot_petserver $
```

컴파일하기 위해 $make 명령을 실행한다.

```
pi@raspberrypi ~/iot_petserver $ make
make all in libs...
make[1]: Entering directory '/home/pi/iot_petserver/libs'
```

```
  gcc -c -I/home/pi/iot_petserver/include -I/usr/include/mysql iot_db.c -o
iot_db.o
  gcc -c -I/home/pi/iot_petserver/include -I/usr/include/mysql parser.c -o
parser.o
  ar rcs libiot.a -o iot_db.o parser.o
  make[1]: Leaving directory '/home/pi/iot_petserver/libs'
  make all in src...
  make[1]: Entering directory '/home/pi/iot_petserver/src'
  make all in pet...
  make[2]: Entering directory '/home/pi/iot_petserver/src/pet'
  gcc -g -c -I/home/pi/iot_petserver/include -c pet_main.c -o pet_main.o
  gcc -o server_pet pet_main.o /home/pi/iot_petserver/libs/libiot.a -lmysqlclient
-lpthread
  make[2]: Leaving directory '/home/pi/iot_petserver/src/pet'
  make[1]: Leaving directory '/home/pi/iot_petserver/src'
  pi@raspberrypi ~/iot_petserver $
```

에러 없이 실행되었다면 다음으로 실행파일을 출력 폴더에 생성하기 위해 make install 명령을 실행한다.

```
  pi@raspberrypi ~/iot_petserver $ make install
  mkdir /home/pi/iot_petserver/output
  Installing in libs...
  make[1]: Entering directory '/home/pi/iot_petserver/libs'
  make[1]: *** No rule to make target 'install'.  Stop.
  make[1]: Leaving directory '/home/pi/iot_petserver/libs'
  Installing in src...
  make[1]: Entering directory '/home/pi/iot_petserver/src'
  Installing in pet...
  make[2]: Entering directory '/home/pi/iot_petserver/src/pet'
  cp -a server_pet /home/pi/iot_petserver/output/
  make[2]: Leaving directory '/home/pi/iot_petserver/src/pet'
```

```
make[1]: Leaving directory '/home/pi/iot_petserver/src'
pi@raspberrypi ~/iot_petserver $
```

여기까지 실행하면 output 폴더에 서버 프로그램이 생성된다.

```
pi@raspberrypi ~ $ cd iot_toyserver
pi@raspberrypi ~/iot_toyserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_toyserver $ cd output
pi@raspberrypi ~/iot_toyserver/output $
pi@raspberrypi ~/iot_toyserver/output $ ls
mjpg.sh  server_toy
pi@raspberrypi ~/iot_toyserver/output $
pi@raspberrypi ~/iot_toyserver/output $ sudo ./server_pet
```

서버프로그램을 실행한다.

```
$sudo ./server_pet
```

아래는 서버 프로그램을 실행시킨 화면이다. 서버는 온 습도, 조도 데이터를 수집하여 데이터 베이스에 저장하는 한편 클라이언트가 되는 스마트 폰에서 접속하기를 기다린다. 따라서 터미널 화면에서 서버를 수행시키면 다른 컴퓨터에서 putty 를 실행시켜 동영상 스트리밍을 서비스 하는 프로그램을 실행하는 방법으로 실습하여야 한다.

```
pi@raspberrypi ~/iot_petserver/output $ sudo ./server_pet
validate database...
iot_send_query : query - CREATE TABLE IF NOT EXISTS tpetsensorvalue (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, human INT, sound INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS tpetsensorcheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, human INT, sound INT)
```

```
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tpetactuoperate (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, servomotor INT, ledbar INT, dcmotor INT, rgbled INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tpetactucheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, servomotor INT, ledbar INT, dcmotor INT, rgbled INT)
  iot_send_query : query - CREATE TABLE IF NOT EXISTS tpetsetting (id INT AUTO_INCREMENT PRIMARY KEY, time1 TIME, time2 TIME)
  done
  iot_count_setting_data_from_db : query - SELECT COUNT(*) FROM tpetsetting
  iot_count_setting_data_from_db data: 0
  validate setting data... : 0
  iot_insert_data_from_db : query - INSERT into tpetsetting values (null, now(), now())
  starting server...
```

이상태에서 putty 를 실행시키고, 라즈베리 파이에 접속한다. Putty 를 실행하는 이유는 위에 서버를 실행하고 다시 동영상 스트리밍 서버를 실행하기 위함인데 위에 설명한 $ sudo ./server_farm & 명령(뒤에 &를 붙여서)으로 백그라운드로 실행하게 하면 putty 프로그램을 실행할 필요는 없다.

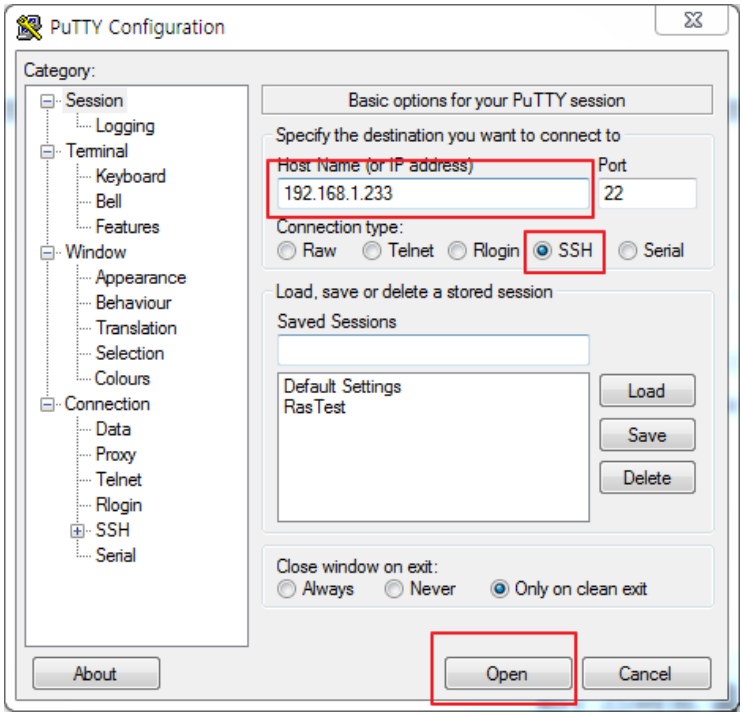


그림 170 동영상 스트림 서버를 실행시키기 위한 Putty 실행화면

아이디와 패스워드를 넣어 접속한 화면은 아래와 같다.

```
login as:
login as: pi
pi@192.168.1.219's password:
Linux raspberrypi 3.18.7+ #755 PREEMPT Thu Feb 12 17:14:31 GMT 2015
armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 09:47:01 2015
```

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

홈 디렉토리에서 mjpg.sh 파일을 확인하고 다음의 명령어로 mjpg 스트림 프로그램을 실행한다.

$sh mjpg.sh

```
pi@raspberrypi ~ $ sh mjpg.sh
MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116): argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
 i: camera parameters.....:
.
.
.
.
```

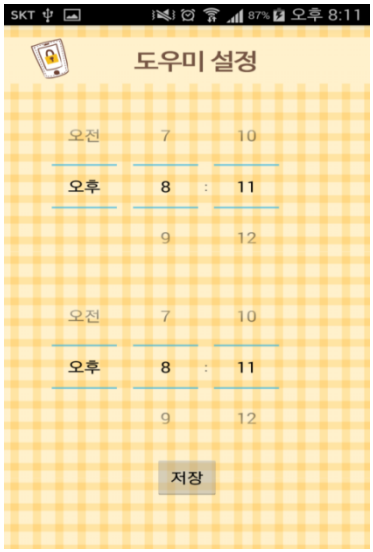
이 상태에서 모든 준비가 끝났다. 안드로이드 폰으로 다운 받은 인터넷 제어토이 앱을 실행시키면서 결과를 확인한다. 아래는 앱 실행 메인 화면이다.



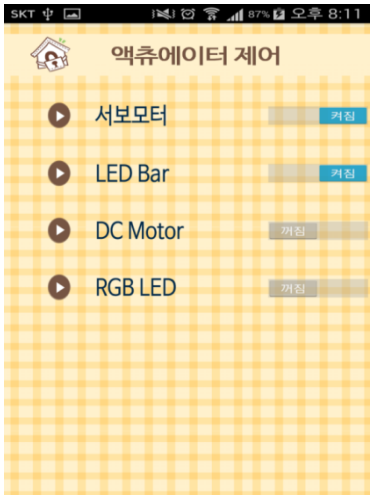
IoT 애완동물 도우미 메인 화면이다. 메뉴는 Pet 도우미 상태, Pet 영상, 도우미 설정, 액츄에이터 제어로 구성된다. 디바이스 연결은 앱이 실행되고 가장먼저 실행해야 한다. 서버로 동작하는 라즈베리 파이의 IP 주소를 입력하여 연결해야 한다.



애완동물 도우미의 상태를 확인하는 메뉴이다. 인체감지센서, 소리센서의 센서값을 확인하고, 액츄에이터의 상태를 보여준다. 센서와 액츄에이터의 이상 유무도 나타낸다.



애완동물 도우미의 밥주는 시간을 설정하는 화면이다. 밥그릇은 모터와 연결되어 있다고 가정하고 정해진 시간에 맞추어 밥그릇을 먹이를 주는 동작을 하도록 설정하는 화면이다.



애완동물 도우미의 액츄에이터를 직접 제어하는 화면이다. 각 액츄에이터를 사용자가 직접 애완동물을 유 | 해 제어할 수 있다.



Pet 영상 보기 화면이다. 실내에 있는 애완동물의 영상을 직접 확인하는 메뉴이다.

여기까지 인터넷 제어토이의 실습을 모두 마친다.

Chapter 14 IoT 인터넷제어 토이 실습

14.1 인터넷제어 Toy 개요

IoT 인터넷 제어 토이 시스템은 Gateway 시스템과 모바일 프로그램으로 구성된다. 인터넷 토이 Gateway 는 4 종류의 센서(인체감지센서, 초음파 센서, 근접센서, 소리센서)를 사용하여 주변 환경을 인지하여, 상황에 맞게 에츄에이터(서보모터, DC Motor, RGB Led, LED BAR)를 제어 하게 된다. 또한 안드로이드 폰과의 연동을 통해 원격지의 사용자 요청이 발생할 경우 Ethernet 이나 Wi-Fi 를 통해 센서데이터나 영상 정보를 송신한다.

사용자는 자유롭게 토이를 구성하여 인터넷에 연결된 상태로 센서 데이터를 확인하고, 실시간으로 영상을 확인하며, 액츄에이터를 원격으로 제어하는 실습을 진행하게 된다.

IoT 인터넷 제어 토이의 주요기능은 아래와 같다.

○ 스마트폰을 이용한 장난감 제어

○ 자동차에 카메라를 설치하여 이동 영상을 실시간으로 스마트폰에 전송

○ 자동차를 원격제어하여 보고 싶은 장소의 영상을 확인

○ 모션감지, 음향 센서를 사용하여 가정의 상황을 원격 모니터링 기능

○ 인터넷에 접속하여 원격 제어가 가능한 로봇과 장난감의 개발을 설계 실습할 수 있는 시스템

(1) 인터넷 제어 토이 하드웨어

인터넷 제어 토이는 Gateway 기능을 수행하는 하는 라즈베리 파이, 카메라, IO Adapter, 4 종류의 센서(인체감지센서, 초음파 센서, 근접센서, 소리센서)와 4 종류의 액츄에이터 서보모터, DC Motor, RGB Led, LED BAR)로 구성되어 있다. 센서는 다른 센서 데이터가 필요할 경우 얼마든지 확장될 수 있고, 액츄에이터 또한 사용자의 설계 의도에 맞게 추가하여 구성할 수 있다. 아래 [그림 126]은 인터넷 제어토이의 하드웨어 구성과 연결도를 나타낸 그림이다.

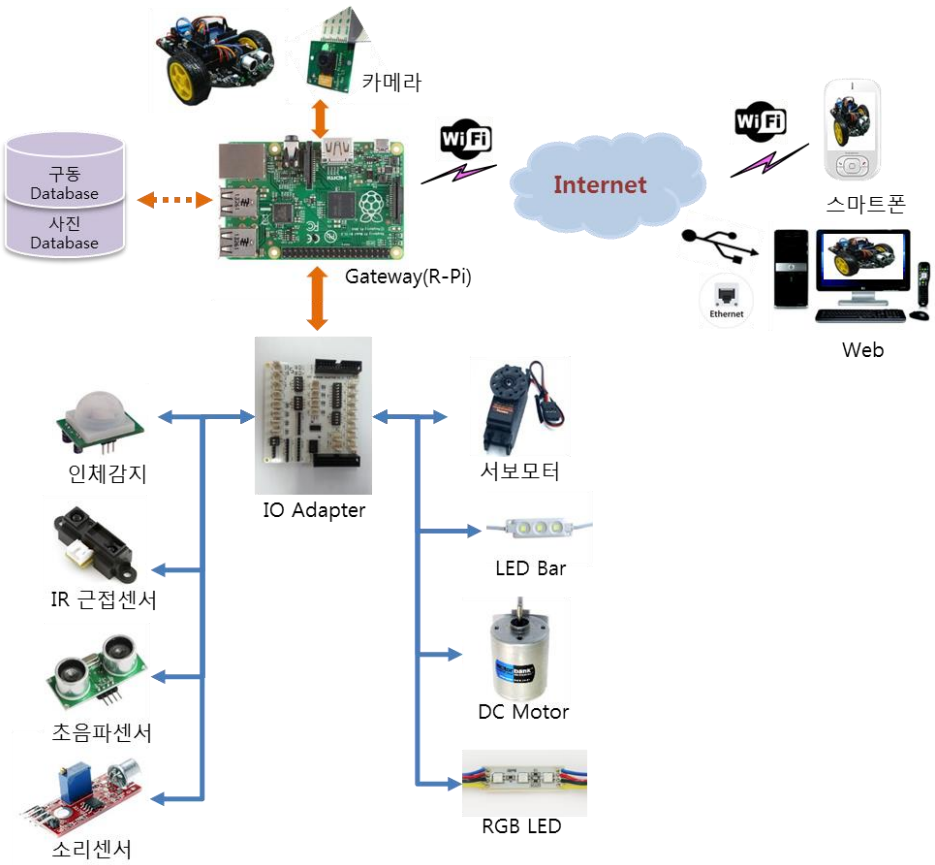


그림 171 IoT 인터넷 제어토이 하드웨어 구성도

스마트 농장 관리 시스템 구성품을 [표 38]에 나타내었다. 앞 장의 실습에서 설명한 라즈베리 파이, IO 어댑터 보드, 카메라에 대한 설명은 생략하기로 한다.

표 37 IoT 인터넷 제어토이 하드웨어 구성품

구성품	사양	기능
센서	인체감지센서	애완동물의 움직임 감지
	사운드센서	애완동물이 내는 소리 감지
Actuator	서보모터	음식물 통을 열고 닫음
전원어댑터	DC 12V, 3A 전원 어댑터	12V 전원 공급

	DC5V, 1A 전원 어댑터	5V전원공급

스마트 농장관리시스템을 구성하는 IO Adapter 보드는 여러 종류의 센서와 다양한 액츄에이터를 라즈베리 파이기반의 Gateway 와 연결해주는 역할을 한다. 무엇보다 IO Adapter 보드의 구조와 연결 커넥터에 대해 숙지해야 한다.

(2) 인터넷 제어토이 시스템 소프트웨어

인터넷 제어 토이의 Software 는 크게 게이트웨이(Gateway) 소프트웨어와 안드로이드 앱 소프트웨어 두 부분으로 구성된다. 라즈베리 파이에서 수행되는 게이트웨이 소프트웨어는 센서로부터 데이터를 획득하고 저장하며 원격지 사용자 요청에 따라 액츄에이터를 동작시키며 카메라 영상을 전송하는 기능을 한다.

스마트 폰에서 동작하는 안드로이드 앱은 원격지의 영상을 실시간으로 확인하고, 토이의 센서 데이터를 조회하며, 환경에 따라 액츄에이터를 직접 제어하는 기능을 한다. 카메라 구동 드라이버는 이미 게이트웨이에 포함되어 있으며 안드로이드 폰과 연동시 게이트 웨이의 서버 프로그램이 수행된다. 게이트웨이 역할을 하는 라즈베리 파이에는 리눅스 운영체제를 최적화한 라즈비안 OS 가 설치되어 있다. 센서로부터 수집된 데이터를 날짜 시간 데이터와 함께 데이터 베이스에 저장하고, 원격지의 사용자 요청에 따라 카메라 영상을 송신하고 설치된 액츄에이터를 제어하는 기능을 한다.

따라서 게이트웨이와 스마트폰 앱에서 수행되는 Software 의 전체 구조를 살펴보면 아래와 같다.

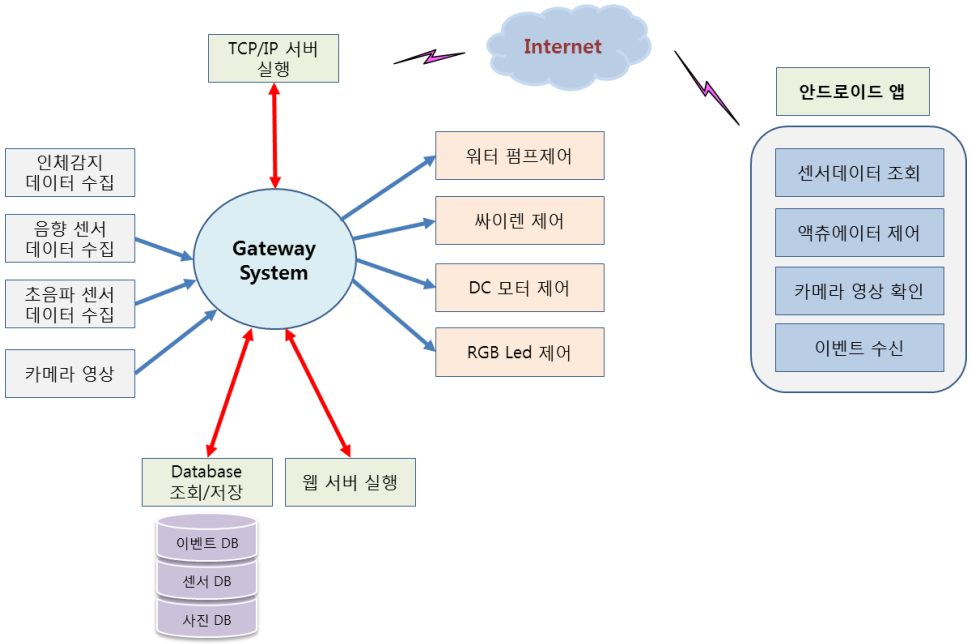


그림 172 인터넷 제어토이 시스템의 소프트웨어 구성도

14.2 인터넷 제어토이의 연결 구성

인터넷 제어토이는 용어 그대로 인터넷에 장난감이 연결되어 있다는 의미이다. Wi-Fi 동글로 인터넷에 연결된 토이는 영상을 송신하고, 센서 데이터를 저장하고, 상황에 맞는 액츄에이터를 동작시킨다. 실습에 이용되는 시스템은 이동할 수는 없지만 여러가지 센서를 이용하여 이동과 정지, 동영상 촬영, 데이터 송수신등의 기능을 수행한다면 충분이 인터넷 토이로서의 기능을 수행할 수 있다.

게이트웨이 기능을 수행하는 라즈베리 파이 연결 커넥터는 40 핀 GPIO 케이블로 연결된다. 라즈베리 파이의 GPIO 포트는 사용자의 프로그램에서 입력으로 설정하거나 출력으로 설정을 한다. 또한 할당된 GPIO 핀 입력을 인터럽트로 설정하여 이벤트 리스너를 함수로 등록하여 사용하는것은 훌륭한 외부 인터럽트를 처리하는 방법이다.

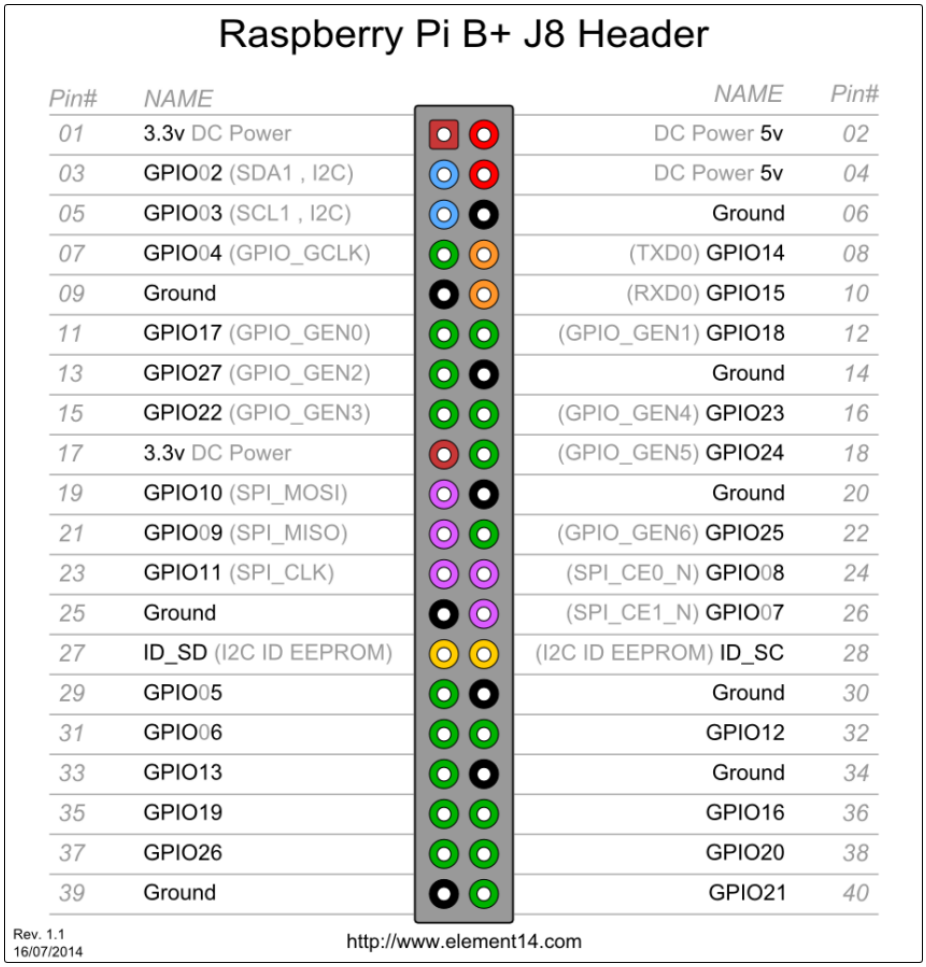


그림 173 라즈베리 파이 GPIO 핀 구성

라즈베리 파이에는 따로 AD 컨버터가 없기 때문에 외부에서 AD 컨버터(ADC)를 이용해서 아날로그 데이터를 처리할 수 있도록 해야한다. 설계 시스템에서는 ADC3208 을 사용하여 아날로그 출력 센서 데이터를 획득하도록 설계 되었다. ADC 컨버터를 이용한 아날로그 데이터의 수집은 센서 부분에서 상세히 설명한다.

Gateway 기능의 라즈베리 파이에는 카메라와 Wi-Fi USB 무선 랜카드가 연결된다. IO Adapter 보드에 인체감지센서, 근접센서, 초음파 센서, 소리센서를 연결하고, 액츄에이터로는 서보모터, LED bar, DC Motor, RGB Led 를 연결한다.

개별적인 센서 시험과 액츄에이터 시험을 진행하고 나면 전체 시나리로 시험과 스마트 폰에서 제어하는 시험을 진행하게 된다.

시험을 성공적으로 진행했다면 그대로 유지하는 것이 다음 시험을 위해 좋을 것이다. 센서와 액츄에이터 연결도를 숙지하여야 한다.

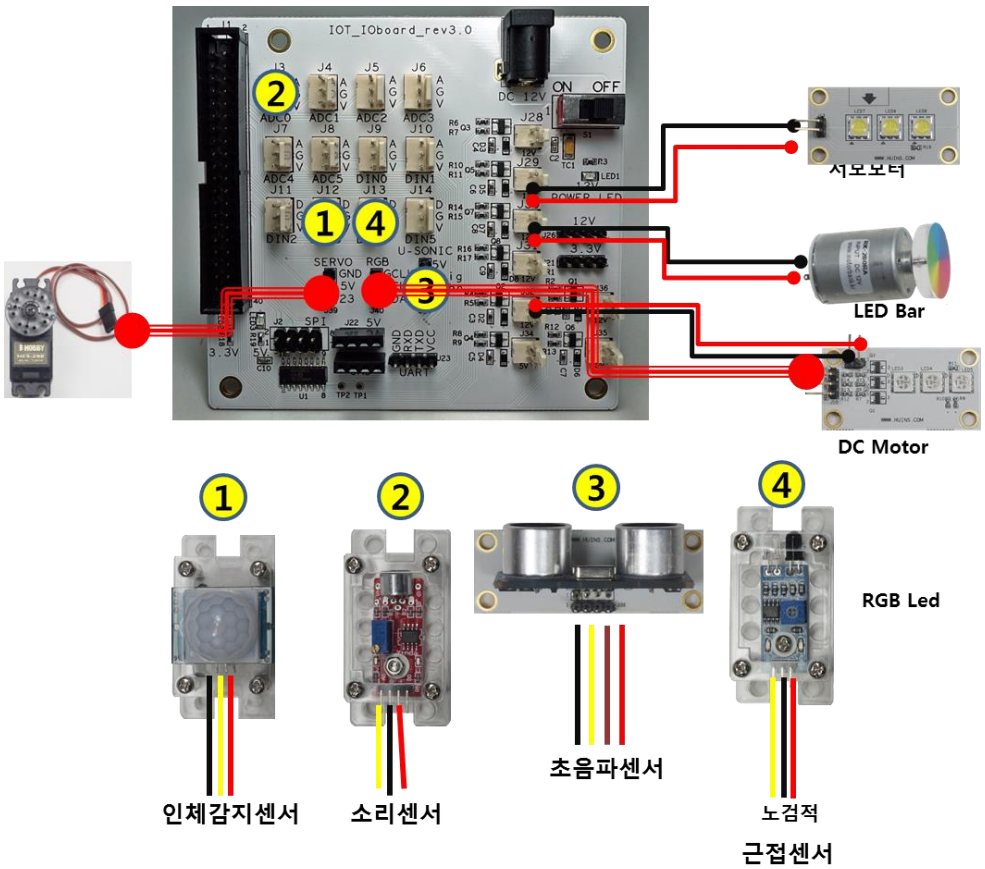


그림 174 센서 연결도

센서 연결을 나타낸 [그림 129]를 참조하여 센서를 연결한다. 4 종의 센서를 연결하고 센서 개별적인 실습을 진행할 것이다.

아래 [그림 173]은 액츄에이터 연결을 나타낸다. DC 12V 전원 어댑터가 연결되는 바로 아래에 12V 전원 곱급 온 오프 스위치가 있으니 실습할 때에는 반드시 확인하도록 하자.

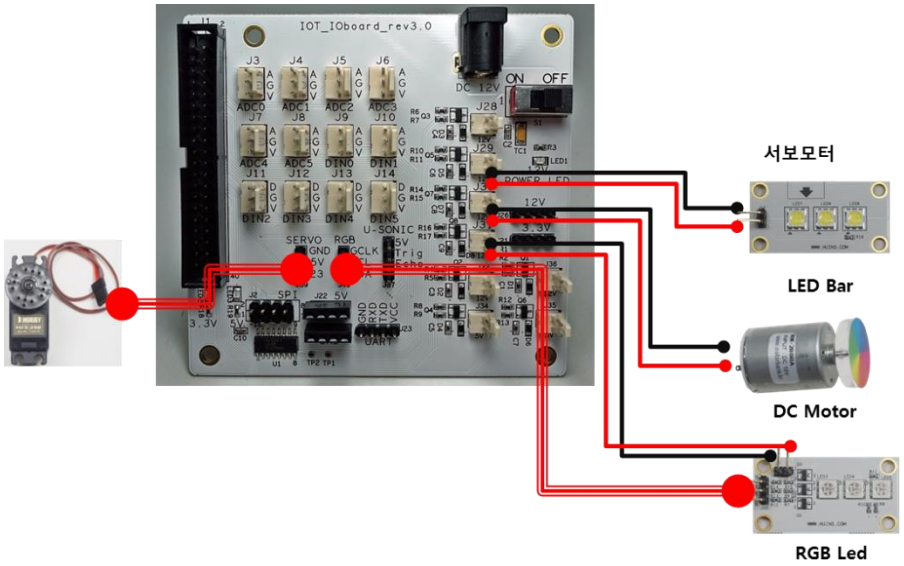


그림 175 액츄에이터 연결도

연결의 정확성

센서와 액츄에이터 보드의 연결도는 실습 후에 스마트 폰으로 입력을 확인하고 액츄에이터를 제어할 때까지 동일한 연결과 테스트를 수행해야 한다. 개별 센서와 액츄에이터 실습시에는 센서 데이터와 액츄에이터 제어가 올라보 되는지를 확인하고 실제 상황을 스마트 폰과 연동 실습할때에는 모든 연결이 정상적으로 정확하게 연결되어야 실습이 진행된다.

IO Adapter 보드는 센서와 액츄에이터를 게이트웨이에 연결하는 역할을 한다. IO Adapter 보드에는 액츄에이터 구동을 위해 DC 12V 전원이 공급되므로 각종 센서나 액츄에이터를 연결할 때 반드시 전원을 끈 상태에서 주의를 기울여 연결한다.

주의

워터펌프, DC motor, RGB Led 등의 액츄에이터의 구동을 위해 IO Adapter 보드에는 DC 12V독전원이 연결되므로 센서나 액츄에이터를 연결할 때는 반드시 전원을 Off 하고 연결해야 하며, 쇼트, 단락에 주의해야 한다.

14.3 인터넷 제어토이 센서 실습

IoT 인터넷 제어토이에 사용되는 센서를 알아보자. 인체감지 센서는 모션이나 사람의 움직임을 적외선으로 감지하는 센서다. 인체감지 센서의 출력은 디지털 출력을 사용하므로 인체감지가 되면 High 를 출력하고, 감지되지 않을때는 출력이 없는 상태가 된다. 소리센서는 주변의 소리에 따라 출력을 내는데 센서의 핀이 3 핀이면 디지털 값을 출력하는 센서고, 4 핀일 경우 디지털과 아날로그 출력을 내는 센서로 두 신호를 적당이 활용할 수 있다. 초음파 센서는 20cm ~ 2m 정도의 거리를 출력해주는 센서로 근거리 감지에 사용된다. 근접센서는 약 20cm~30cm 정도의 장애물이나 벽등을 감지 하는 센서로 막힌곳을 알아내는데 사용된다. [그림 172]는 IO Adapter 보드에 사용되는 센서들과 액츄에이터의 전체 연결 위치를 나타낸다.

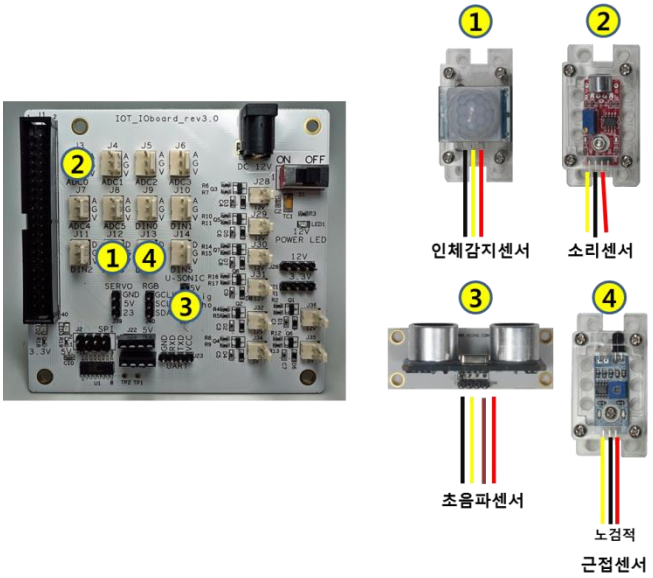


그림 176 스마트 농장관리시스템의 센서구성과 연결 위치

(1) 인체 감지센서

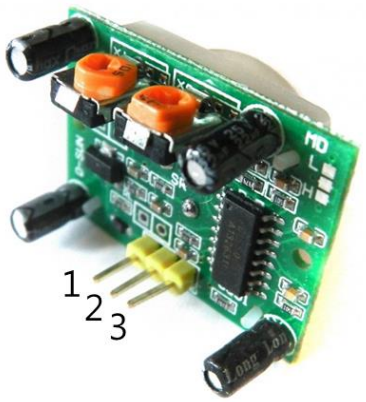
인체감지센서는 Pyroelectric ("Passive") InfraRed sensors 라고도 하며 일정한 적외선을 띤 물체가 움직이는 것을 감지하는 센서이며 주로 인체감지에 쓰인다. 적외선을 띤 물체가 감지되어 일정한 양의 적외선을 감지하게 되면,

출력을 내 보내고 적외선을 띤 물체가 없을 때는 출력을 내보내지 않는다. 돔 형태의 창(window)는 외부의 환경으로부터 센서 내부의 회로 및 감지부를 보호하고 원하는 파장대의 적외선만 통과시키는 광학 필터의 기능을 한다. 보통은 사람을 감지하여 일정시간 불을 켜는 용도로 사용한다.

표 38 인체감지센서 사양

사용 전압	3~5V
사용 전류	50μA
동작 온도 범위	0℃~+70℃
센서 미 감지시 출력(HIGH)	4V
센서 감지시 출력(LOW)	0.4V
감지 각도	110 Degree
감지 거리	7M
Size	28mm×36mm
무게	25g

아래는 인체감지 센서의 외관과 Pin 기능을 보여준다.



Pin1 – VCC 전원
Pin2 – Digital Out
Pin3 – GND

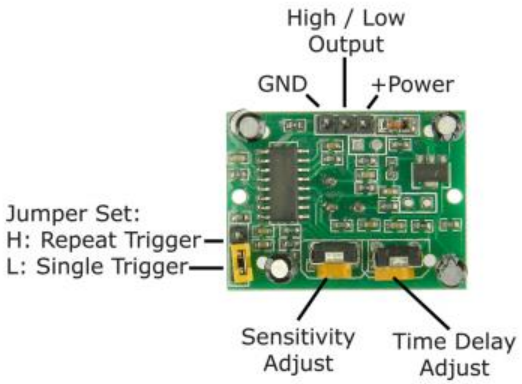
앞장에서의 사운드 센서와 거의 같은 방법으로 인체감지 센서를 테스트 한다. 전등에 사용되는 인체감지 센서는 감지가 된 후 약 5 초 정도 전등을 키고 OFF

하는 방식인데 여기에서 사용한 인체 감지 센서도 같은 방식으로 동작한다. 그 외 필요한 설정은 아래 설정을 보고 시험해야 한다.

Calibration- 센서보정

감지 센서를 사용하기 전에 두 가지 컬리브레이션을 진행해야 한다. 아래[그림 8]을 보면 Sensitivity Adjust 와 Time Delay Adjust 가 있다. Sensitivity Adjust 는 인체감지 센서의 민감도를 조절하는 가변저항이고, Time Delay Adjust 인체가 감지되어 출력핀이 하이가 유지되는 시간을 조절하는 가변저항이다.

Sensitivity Adjust 가변저항을 왼쪽 끝까지 돌리면 민감도가 가장 약하고, 오른쪽으로 돌리면 민감도가 증가한다. 실습시에는 왼쪽 끝으로 돌려서 민감도를 제일 약하게 하는 것이 편할 것이다. Time Delay Adjust 가변저항은 오른쪽으로 돌리면 하이가 계속 출력되는 시간이 길어지고, 왼쪽으로 돌리면 약 4 초 후에 로우로 떨어지게 된다. 실습에서는 가장 왼쪽으로 돌려 일찍 로우상태(대기상태)로 되도록 하는 것이 결과를 확인하기 쉽다.



센서의 자세한 스펙을 아래 링크에서 다운 받아 참고한다.
http://www.mpja.com/download/31227sc.pdf

인체감지 센서를 시험하는 소스코드를 아래에 나타낸다. (motiondetect.c)

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
```

```
#include <stdlib.h>
#include <wiringPi.h>



// Use GPIO Pin 6, which is Pin 22 for wiringPi library
#define MOTION_IN 0   //gpio17



// the event counter
volatile int eventCounter = 0;
unsigned char flag_detect;
// ------------------------------------------------------------------------
// myInterrupt:  called every time an event occurs
void myInterrupt(void) {
    eventCounter++;
    flag_detect = 0x01;
}



// ------------------------------------------------------------------------
// main
int main(void) {
  // sets up the wiringPi library
  if (wiringPiSetup () < 0) {
      fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
      return 1;
  }

  // set Pin 17/0 generate an interrupt on high-to-low transitions
```

```
    // and attach myInterrupt() to the interrupt
    if ( wiringPiISR (MOTION_IN, INT_EDGE_RISING, &myInterrupt) < 0 ) {
        fprintf (stderr, "Unable to setup ISR: %s₩n", strerror (errno));
        return 1;
    }

    // display counter value every second.
    while ( 1 )
    {
      if(flag_detect == 0x01)
     {
                printf("Motion Detect₩n");
                flag_detect = 0;
     }
      //eventCounter = 0;
      delay( 100 ); // wait 1 second
    }

    return 0;
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

$gcc -o motiondetect motiondetect.c -lwiringPi

컴파일하여 실행하면 Motion Detec 메시지를 볼 수 있다.

실행 명령어는 $sudo ./motiondetect 이다.

인체를 감지하면 약 3~4 초 동안 같은 상태를 유지하고 초기화 되는 식으로 동적하기 때문에 인지가 되어 있는 동안에는 다시 인지를 하지 못한다.

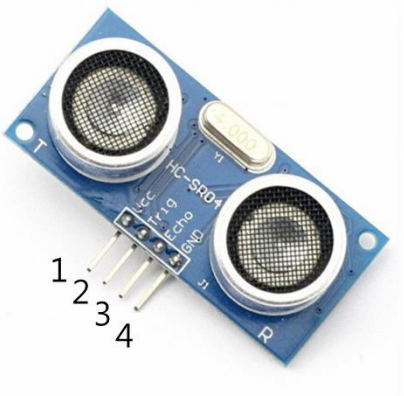
```
^Cpi@raspberrypi ~/smarthome $ sudo ./home-motion
Motion Detect
Motion Detect
Motion Detect
pi@raspberrypi ~/smarthome $
```

(2) 사운드 센서

한정된 지면 관계상 사운드센서의 실습은 무인경비 시스템의 사운드 센서 실습을 참고하기 바란다.

(3) 초음파 센서

초음파 센서는 40KHz 의 펄스 신호를 출력하여 반사되어 돌아오는 펄스의 시간을 측정하여 거리를 계산한다. 아래는 센서의 외관과 핀 맵을 보여준다.



Pin1 – VCC전원
Pin2 – Trigger 신호
Pin3 – Echo 입력신호
Pin4 - GND

그림 177 HC-SR04 초음파 센서의 외형과 핀 맵

여기서는 HC-SR04 초음파 센서를 사용한다. 초음파 센서의 자세한 스펙은 데이터 쉬트를 참고한다(http://www.micropik.com/PDF/HCSR04.pdf).

보통 초음파센서에는 4 개의 신호선이 존재하는데 각 각 VCC, Triger, Echo, GND 신호가 된다. VCC 와 GND 는 5v 라인에 연결하고, Triger 핀과 Echo 핀은 라즈베리파이의 GPIO 핀에 연결해준다.

HC-SR04 의 전기적 특성을 나타냈는데, 초음파 센서의 입력 전원으로 +5V 를 공급하는것에 주의한다. 출력은 펄스를 받아 계산하므로 센서의 Trigger 신호는 GPIO 핀으로 받게 된다.

표 39 HC-SR04 초음파 센서의 전기적 사양

Working Voltage	DC 5 V
Working Current	15mA
Working Frequency	40Hz
Max Range	4m
Min Range	2cm
MeasuringAngle	15 degree
Trigger Input Signal	적어도 10uS 이상의 TTL pulse
Echo Output Signal	Input TTL lever signal and the range in proportion
Dimension	45*20*15mm

초음파 센서의 타이밍도를 나타낸다.

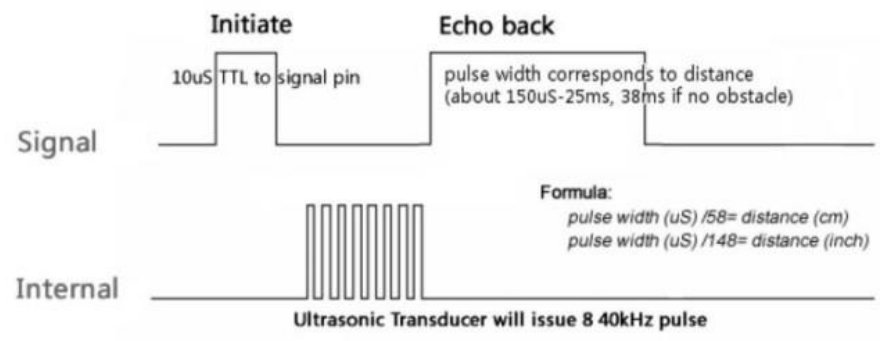


그림 178 초음파 센서의 타이밍도

위 그림은 센서의 작동원리를 설명해주는 그림이다. 우선 triger 핀에 10us 정도의 High 신호를 주면 초음파센서는 40khz 펄스를 자동적으로 8 번을 발생시키고 펄스를 발생시킨 직후에 echo 핀은 high 되고 반사된 초음파가 감지

되었을때 echo 핀이 low 가 된다. 이 Echo 핀이 high 였다가 low 가 되는데 걸리는 시간을 측정하여서 그 시간을 초음파의 속도/2(즉 58)로 나누면 거리가 나온다.

초음파 센서를 시험하기 위해 IO adapter 보드의 J16 에 VCC, GND, Echo 신호선을 연결하고 Trigger 신호는 J25 의 1 번 핀에 연결한다.

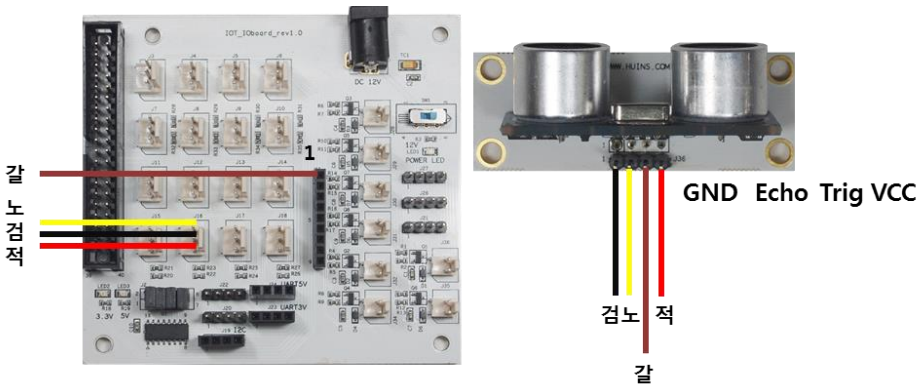


그림 179 초음파 센서 핀 연결도

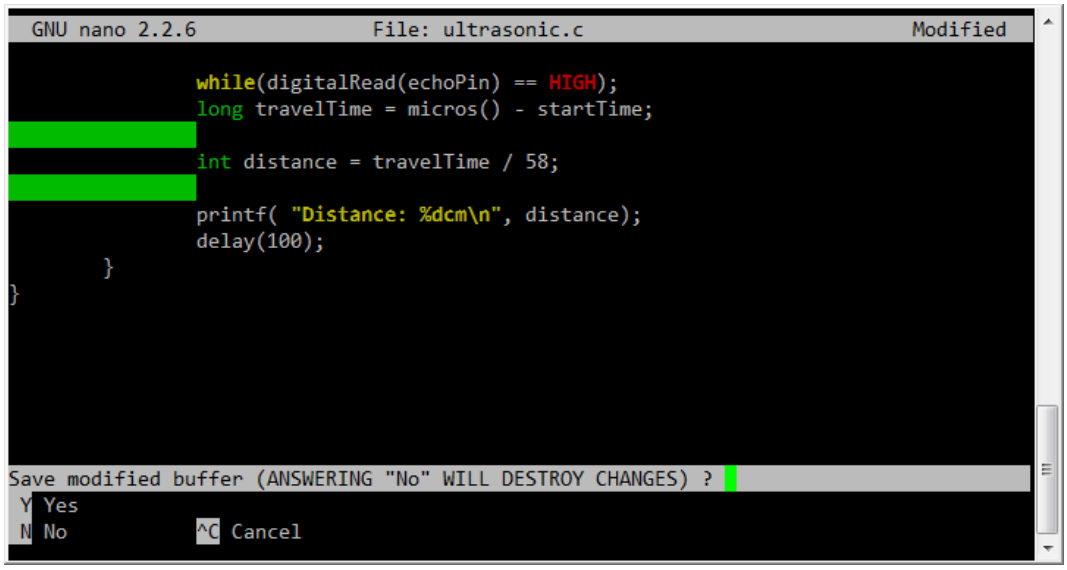
시험을 위하여 초음파센서 시험용 디렉토리를 만든다.

```
mkdir ultrasonic
```

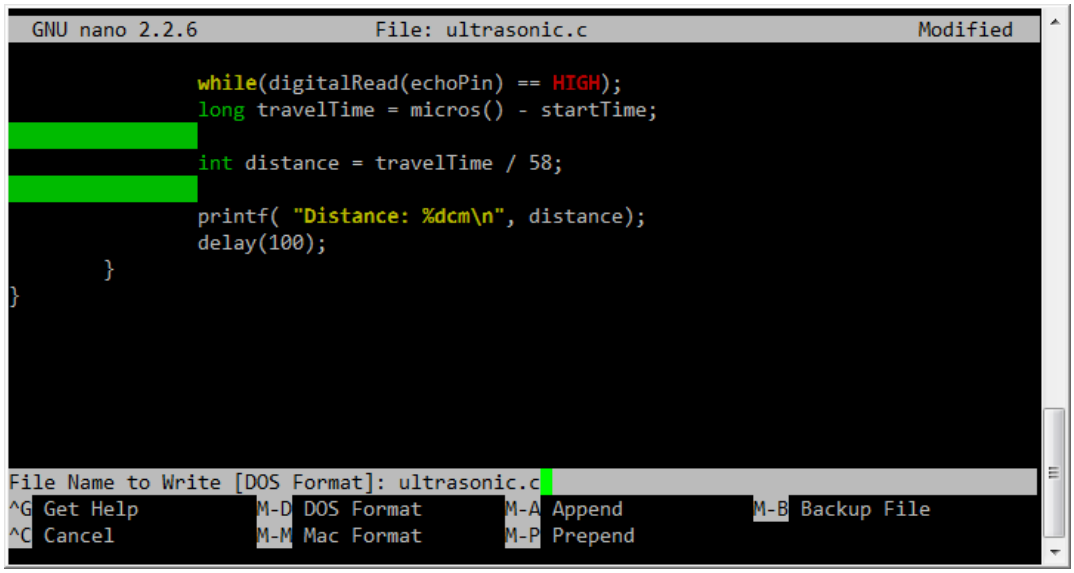
nano 편집기를 이용하여 소스,파일을 생성한다.

```
nano ultrasonic.c
```

소스파일 편집기에서 아래 코드를 입력한다. 입력을 마치고 ctrl-x 를 입력하면 파일을 저장할것이냐고 묻는데 이때 y 를 입력한다.



Y 를 입력하고 파일이름을 묻어오면, 이름을 변경하지 않을 경우에는 그대로 Enter 를 치면 빠져 나오게 된다.



아래에 초음파 센서 테스트 소스코드를 나타내었다. Dir 100usec 의 트리거 신호를 주고, 초음파 센서에서 측정한 거리값을 츠 단위로 나타낸다. 이것을 0.1 초(100msec) 에 한번씩 출력하도록 프로그램 하였다.

```
#include<stdio.h>
#include<wiringPi.h>
```

```
#define trigPin 21      //gpio 5
#define echoPin 4       //gpio   J16-pin3 GPIO 23

int main(void)
{
    int distance=0;
    int pulse = 0;

    long startTime;
    long travelTime;

    if(wiringPiSetup () == -1)
    {
            printf("Unable GPIO Setup");
            return 1;
    }

    pinMode (trigPin, OUTPUT);
    pinMode (echoPin, INPUT);

    for(;;)
    {
            digitalWrite (trigPin, LOW);
            usleep(2);
            digitalWrite (trigPin, HIGH);
            usleep(20);
            digitalWrite (trigPin, LOW);

            while(digitalRead(echoPin) == LOW);
```

```
            startTime = micros();


            while(digitalRead(echoPin) == HIGH);
            travelTime = micros() - startTime;


            int distance = travelTime / 58;


            printf( "Distance: %dcm₩n", distance);
            delay(100);
        }
    }
```

입력을 다 했다면 컴파일을 수행한다.

```
$gcc -o ultrasonic ultrasonic.c -lwiringPi
```

아래명령으로 실행을 시키고 결과를 확인한다. 프로그램 실행을 중단할때는 ctrl - c 키를 누른다.

```
$sudo ./ultrasonic
```

```
pi@raspberrypi ~/safeguardtest/ultrasonic $ nano ultrasonic.c
pi@raspberrypi ~/safeguardtest/ultrasonic $ nano ultrasonic.c
pi@raspberrypi ~/safeguardtest/ultrasonic $
pi@raspberrypi ~/safeguardtest/ultrasonic $
pi@raspberrypi ~/safeguardtest/ultrasonic $ gcc -o ultrasonic ultrasonic.c -lwiringPi
pi@raspberrypi ~/safeguardtest/ultrasonic $ sudo ./ultrasonic
Distance: 48cm
Distance: 48cm
Distance: 49cm
Distance: 49cm
Distance: 48cm
Distance: 48cm
Distance: 48cm
Distance: 47cm
Distance: 48cm
Distance: 48cm
Distance: 48cm
Distance: 49cm
Distance: 48cm
Distance: 54cm
Distance: 53cm
Distance: 30cm
Distance: 53cm
Distance: 17cm
Distance: 18cm
Distance: 19cm
^Cpi@raspberrypi ~/safeguardtest/ultrasonic $
```

그림 180 초음파 센서의 실행 결과 화면

(4) 근접센서

인터넷 제어 토이의 근접센서는 적외선 송/수광 일체형 전방감지 센서모듈로 물체 감지시 HIGH/LOW TTL 신호를 출력한다. 거리 20~30mm 이내에 물체가 감지되면 물체를 감지하면 검출 녹색 LED 가 켜지게 된다. 한쌍의 적외선 송신 수신부가 있어서 송신 후 반사되는 적외선을 검출하여 비교기를 통하여 출력된다. 물체가 검출이 되면 출력신호는 Low level 로 떨어지며 녹색의 LED 에 불이 들어오게 되고, 검출되지 않을경우 High 출력신호를 유지한다. 형광등 불빛과 같은 외광에 강한 적외선 필터를 내장한 수광 센서가 보다 안정된 동작을 유지하는 센서이다.

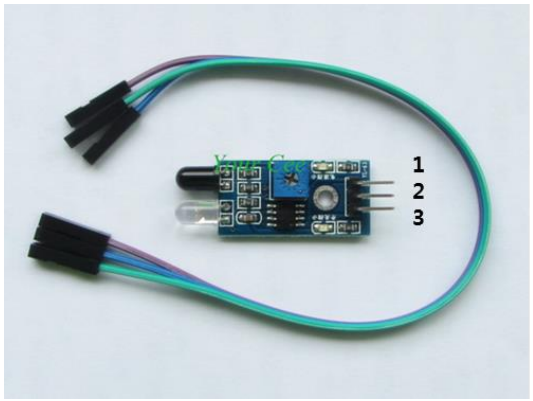


그림 181 근접센서의 모양과 핀 맵

면이나 물체를 감지하고, 충돌을 사전에 감지하여 방지하는 센서로 알려져 있으며 사용하기 쉬운 장점이 있다. 초음파 센서가 2m 이내의 거리를 측정한다면 근접센서는 2~3cm 의 가까운 물체를 감지하는데 사용한다. 근접센서를 사용하기 전에 다른 센서와 마찬가지로 컬리브레이션이 필요하다.

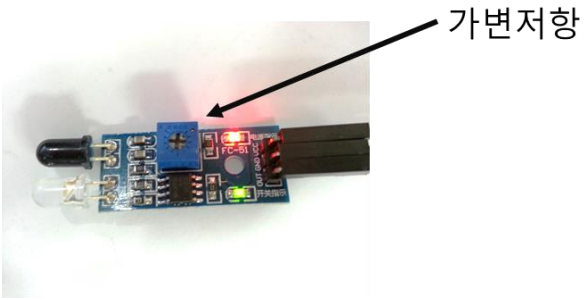


그림 182 근접센서의 컬리브레이션

근접 센서의 컬리브레이션을 설명한다. 근접센서를 전원과 연결하면 빨강 LED 가 켜진다. 녹색 LED 가 켜져 있다면, 근접센서의 중앙의 파란색 가변저항을 좌 우로 회전하여 녹색 LED 가 꺼지거나 켜지는 지점에서 꺼지는 지점에서 멈춘다. 이때 센서 3cm 가까이 손을 대면 감지가 발생할 경우 녹색 LED 가 점등되어 깜박이는 시점에 설정하면 컬리브레이션이 잘 된 것이다.

아래 [표 19]에 근접센서의 사양을 나타내었다.

표 40 근접센서의 사양

항 목	특징
센서 원리	적외선 감지
검출 거리	20 ~ 30mm
검출 각도	중앙에서 상하 35°
비교기	LM393
입력 전원	DC 3V ~ 5V
출력 형태	디지털 출력(0, 1)

실습을 위하여 근접센서의 연결도를 [그림 138]에 첨부하였다.

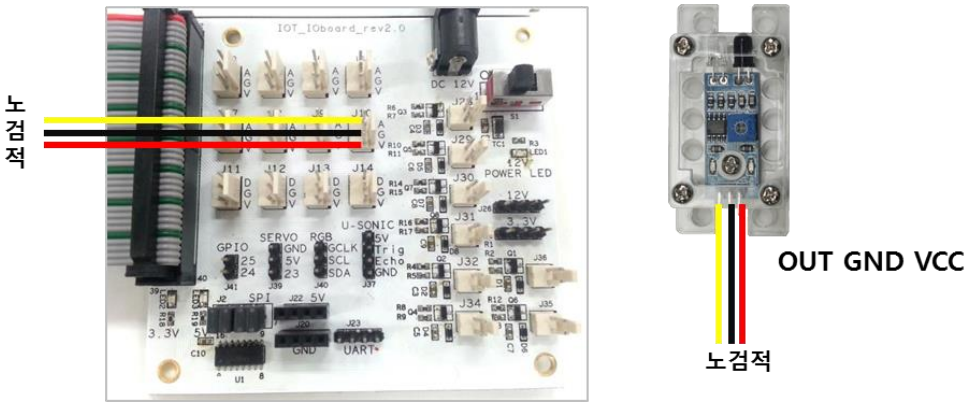


그림 183 근접센서의 연결도

근접센서를 실습하기 위해 nano 편집기를 이용하여 아래와 같이 소스코드를 입력한다.

$nano neardetect.c

```
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <stdlib.h>
#include <wiringPi.h>


// Use GPIO Pin 27, which is Pin 0 for wiringPi library

#define COLLISION 2   //gpio27 - J13 connect


// ------------------------------------------------------------
-------------
// main
int main(void)
{
```

```
    // sets up the wiringPi library
    if (wiringPiSetup () < 0)
    {
            fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
            return 1;
    }

    pinMode(COLLISION, INPUT);

    // display counter value every second.
    while ( 1 )
    {
            //printf( "%d₩n", eventCounter );
            //eventCounter = 0;

            if(digitalRead(COLLISION) == 0)
                    printf("Carefull~~~~ oops ₩n");
            if(digitalRead(COLLISION) == 1)
                    printf("Not Collioson... ₩n");

            delay( 200 ); // wait 1 second
    }

    return 0;
}
```

입력을 하였다면 ctrl-x 를 누르고 저장을 위해 y 를 입력하고 같은 파일이름으로 저장하기 위해 enter 를 누르고 nano 편집기를 종료한다.

컴파일을 실행한다.

```
$ gcc -o neardetect neardetect.c -l wiringPi
```

에러 없이 컴파일이 완료되었다면 실행시켜 결과를 확인한다.

```
$ sudo ./neardetect
```

아래처럼 근접센서 앞에 아무것도 없을경우와 손이나 물체를 앞에 두었을 경우에 출력되는 메시지를 확인할 수 있다.,

```
pi@raspberrypi ~/smarttoy $ sudo ./neardetect
Not Collioson...
Not Collioson...
Not Collioson...
Carefull~~~~ oops
Carefull~~~~ oops
Not Collioson...
Not Collioson...
Not Collioson...
Carefull~~~~ oops
Carefull~~~~ oops
```

이상으로 근접센서의 실습을 마치기로 하자.

14.4 제어토이 액츄에이터 실습

스마트 식물농장의 액츄에이터는 워터펌프, 팬, DC Motor, RGB Led 와 같이 4 가지를 사용한다. 토양이 말라 있을 때 물을 뿌려주고, 실내 농장의 온도에 따라 환기 팬을 작동하고, 야간이나 구름이 낀 날은 청색과 적색의 식물 Led 를 켜준다. 또한 차양을 올린다거나 문을 열기 위해 DC 모터를 사용하는것으로 가정한다.

4 종의 액츄에이터에 대해 외관, 사이즈와 제품특성에 대해 알아보고 전체 액츄에이터에 대한 테스트는 일괄적으로 수행하도록 프로그램을 제작하고 실행해 본다.

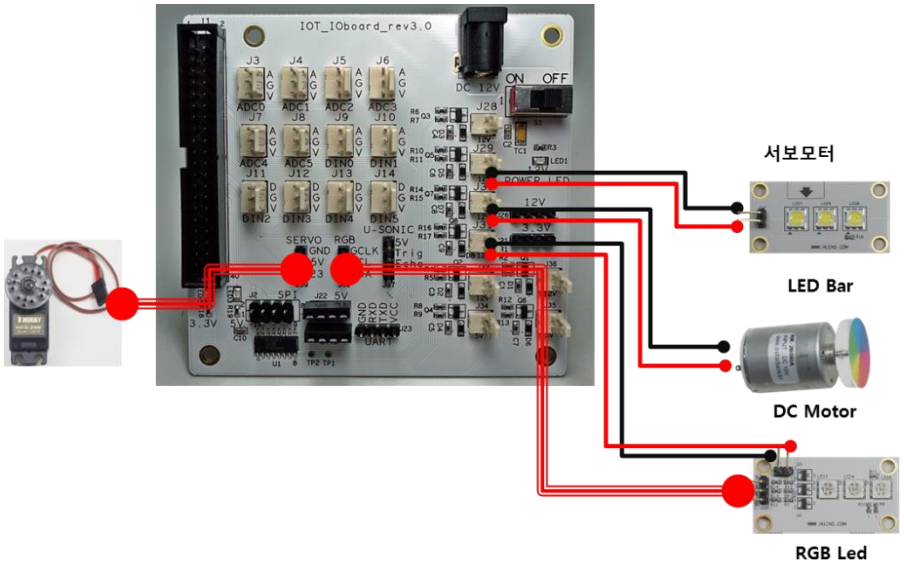


그림 184 스마트 농장관리시스템의 액츄에이터 구성과 연결위치

스마트 농장에 사용되는 액츄에이터를 제어하는 방법은 GPIO 핀을 통한 On/Off 제어가 전부이다. 각각의 액츄에이터는 DC 12V 와 DC 5V 전원을 사용하며 TR 과 저항등 부가적인 회로는 IO Adapter 보드에 실장되어 있다. 액츄에이터 부분의 상세한 회로는 자료를 참고한다. (IO Adapter 보드 회로도 참조)

(1) 워터펌프

한정된 지면 관계상 스마트 팜의 워터펌프 실습을 참고한다.

(2) Fan

한정된 지면 관계상 스마트 팜의 팬 실습을 참고한다.

(3) DC Motor

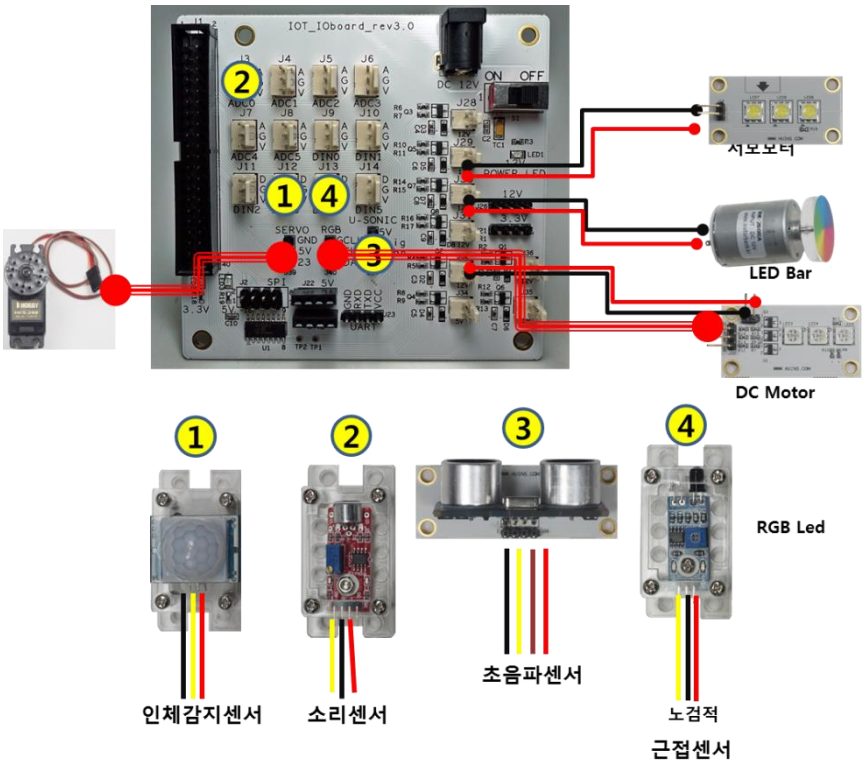
한정된 지면 관계상 스마트 팜의 DC 모터 실습을 참고한다.

(4) RGB Led

한정된 지면 관계상 스마트 팜의 LGB LED 실습을 참고한다.

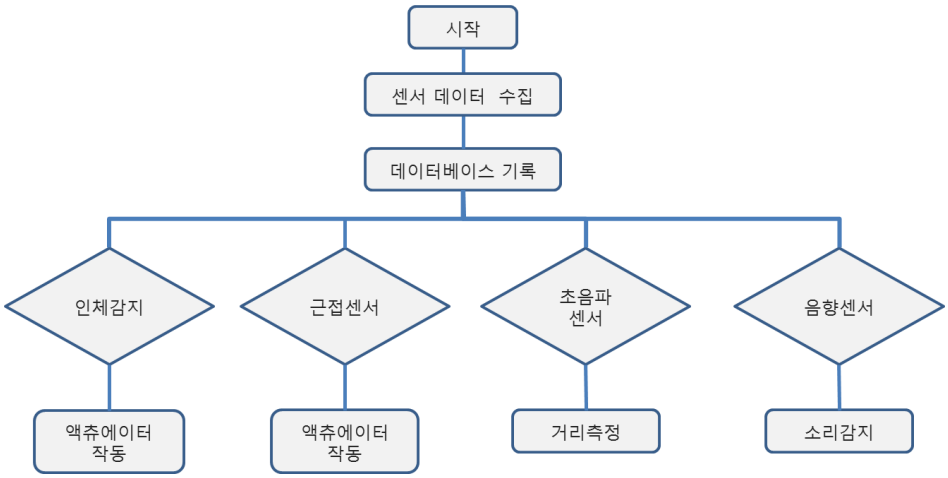
(5) 인터넷 제어토이 시나리오 실습

인터넷제어 Toy 시스템의 전체 시나리오를 테스트 한다. 앞에서 설명한 전체 구조를 이해하고 센서와 액츄에이터 테스트를 잘 했다면 쉽게 따라 할 수 있을 것이다. 가급적이면 센서와 액츄에이터 연결을 바꾸지 말고 처음 개별적으로 테스트 하던 상태로 시험을 진행한다.



기준이 되는 조도값이나 인체감지, 사운드 센서의 값은 현재 애완동물이 있는 장소의 환경에 따라 사용자가 센서 기준값을 임의로 정해주어도 상관이 없다.

식사 시간의 설정도 액츄에이터의 동작 확인을 위해 몇 분 뒤와 같은 임의 의 값을 주어도 된다.



여기서 중요한 것은 센서 데이터를 획득하여 사용자가 설정한 값보다 높거나 낮을 경우 원하는 액츄에이터를 동작시키는 것이다. 센서값이 기준 값에 미치지 못할 경우 손을 움직이거나 센서 근처에서 소리를 내는 등의 조치를 하여 액츄에이터의 작동을 쉽게 해주는 방법을 사용해도 된다.

인터넷 제어토이의 시험을 전체적으로 테스트 할 수 있도록 시나리오를 그리고 프로그램도 이 그림에 따라 진행한다.

항상 작업 디렉토리는 /home/pi 에서 수행해야 한다. 작업 디렉토리 확인 방법은 $pwd 명령으로 확인한다.

```
pi@raspberrypi ~ $ pwd
/home/pi
pi@raspberrypi ~ $
```

프로그램 : farmstory.c 참고

```
$nano farmstory.c
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>
```

```
#include <string.h>
#include <errno.h>
#include <sys/types.h>
#include <unistd.h>
#include <wiringPi.h>

#define PUMP          21        // BCM_GPIO 5
#define FAN           22                  // BCM_GPIO 6
#define DCMOTOR 23          // BCM_GPIO 13
#define RGBLEDPOWER   24 //BCM_GPIO 19

#define RED27                 //GPIO 16
#define GREEN         28      //GPIO 20
#define BLUE          29      //GPIO 21

#define LIGHTSEN_OUT 2   //gpio27 - J13 connect

#define MAXTIMINGS 85

void Bpluspinmodeset(void);



int ret_humid, ret_temp;

static int DHTPIN = 11;
static int dht22_dat[5] = {0,0,0,0,0};

int ret_humid, ret_temp;

int main (void)
{
```

```
if(wiringPicheck()) printf("Fail");

Bpluspinmodeset();

int curtemp = 0, curhumid = 0;

while(1)
{

        if(get_light() == 0)
        {
                act_rgbled_off();
                printf("light ! ₩n");
        }
        else
        {
                act_rgbled_on();
                printf("dark ! ₩n");
        }

        curtemp = get_temperature_sensor();
        printf("%d ₩n", curtemp);

        if(curtemp > 26)digitalWrite (PUMP, 1) ; // On
        {

                act_waterpump_on();
                delay(2000);
                act_waterpump_off();
        }
```

```
                curhumid = get_humidity_sensor();
                if(curhumid < 50)
                {

                        act_fan_on();
                        delay(2000);
                        act_fan_off();
                }
                delay(10000);
    }


    return 0 ;
}

int wiringPicheck(void)
{
    if (wiringPiSetup () == -1)
    {
                fprintf(stdout, "Unable to start wiringPi: %s₩n", strerror(errno));
                return 1 ;
    }
}

void Bpluspinmodeset(void)
{
    pinMode(LIGHTSEN_OUT, INPUT);

    pinMode (PUMP, OUTPUT);
    pinMode (FAN, OUTPUT);
    pinMode (DCMOTOR, OUTPUT);
```

```
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);
}

int get_light()
{
    // sets up the wiringPi library
    if (wiringPiSetup () < 0)
    {
            fprintf (stderr, "Unable to setup wiringPi: %s₩n", strerror (errno));
            return 1;
    }

    if(digitalRead(LIGHTSEN_OUT))       //day
            return 1;
    else //night
            return 0;



}

int get_temperature_sensor()
{
    int received_temp;

    DHTPIN = 11;

    if (wiringPiSetup() == -1)
            exit(EXIT_FAILURE) ;
```

```
    if (setuid(getuid()) < 0)
    {
            perror("Dropping privileges failed₩n");
            exit(EXIT_FAILURE);
    }
    while (read_dht22_dat_temp() == 0)
    {
            delay(500); // wait 1sec to refresh
    }
    received_temp = ret_temp ;
    printf("Temperature = %d₩n", received_temp);

    return received_temp;
}

static uint8_t sizecvt(const int read)
{
  /* digitalRead() and friends from wiringpi are defined as returning a value
  < 256. However, they are returned as int() types. This is a safety function */

  if (read > 255 || read < 0)
  {
    printf("Invalid data from wiringPi library₩n");
    exit(EXIT_FAILURE);
  }
  return (uint8_t)read;
}

int read_dht22_dat_temp()
{
```

```
    uint8_t laststate = HIGH;
    uint8_t counter = 0;
    uint8_t j = 0, i;

    dht22_dat[0] = dht22_dat[1] = dht22_dat[2] = dht22_dat[3] = dht22_dat[4] =
0;

    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepare to read the pin
    pinMode(DHTPIN, INPUT);

    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++ ) {
      counter = 0;
      while (sizecvt(digitalRead(DHTPIN)) == laststate) {
        counter++;
        delayMicroseconds(1);
        if (counter == 255) {
          break;
        }
      }
      laststate = sizecvt(digitalRead(DHTPIN));
```

```
        if (counter == 255) break;

        // ignore first 3 transitions
        if ((i >= 4) && (i%2 == 0)) {
          // shove each bit into the storage bytes
          dht22_dat[j/8] <<= 1;
          if (counter > 16)
            dht22_dat[j/8] |= 1;
          j++;
        }
      }

      // check we read 40 bits (8bit x 5 ) + verify checksum in the last byte
      // print it out if data is good
      if ((j >= 40) &&
          (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
            float t, h;

            h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
            h /= 10;
            t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
            t /= 10.0;
            if ((dht22_dat[2] & 0x80) != 0)  t *= -1;

                ret_humid = (int)h;
                ret_temp = (int)t;
                printf("Humidity = %.2f %% Temperature = %.2f *C ₩n", h, t );
                printf("Humidity = %d Temperature = %d₩n", ret_humid, ret_temp);

        return ret_temp;
```

```
    }
    else
    {
      printf("Data not good, skip₩n");
      return 0;
    }
}



int get_humidity_sensor()
{
    int received_humid;

    DHTPIN = 11;

    if (wiringPiSetup() == -1)
            exit(EXIT_FAILURE) ;

    if (setuid(getuid()) < 0)
    {
            perror("Dropping privileges failed₩n");
            exit(EXIT_FAILURE);
    }
    while (read_dht22_dat_humid() == 0)
    {
            delay(500); // wait 1sec to refresh
    }
    received_humid = ret_humid;
    printf("Humidity = %d₩n", received_humid);

    return received_humid;
```

```
}

int read_dht22_dat_humid()
{
    uint8_t laststate = HIGH;
    uint8_t counter = 0;
    uint8_t j = 0, i;

    dht22_dat[0] = dht22_dat[1] = dht22_dat[2] = dht22_dat[3] = dht22_dat[4]
= 0;

    // pull pin down for 18 milliseconds
    pinMode(DHTPIN, OUTPUT);
    digitalWrite(DHTPIN, HIGH);
    delay(10);
    digitalWrite(DHTPIN, LOW);
    delay(18);
    // then pull it up for 40 microseconds
    digitalWrite(DHTPIN, HIGH);
    delayMicroseconds(40);
    // prepare to read the pin
    pinMode(DHTPIN, INPUT);

    // detect change and read data
    for ( i=0; i< MAXTIMINGS; i++ )
    {
            counter = 0;
            while (sizecvt(digitalRead(DHTPIN)) == laststate)
            {
                    counter++;
                    delayMicroseconds(1);
```

```
                        if (counter == 255) {
                        break;
                }
        }
        laststate = sizecvt(digitalRead(DHTPIN));

        if (counter == 255) break;

        // ignore first 3 transitions
        if ((i >= 4) && (i%2 == 0))
       {
                // shove each bit into the storage bytes
                dht22_dat[j/8] <<= 1;
                if (counter > 16)
                        dht22_dat[j/8] |= 1;
                j++;
        }
   }

    // check we read 40 bits (8bit x 5 ) + verify checksum in the last byte
    // print it out if data is good
     if ((j >= 40) &&
        (dht22_dat[4] == ((dht22_dat[0] + dht22_dat[1] + dht22_dat[2] +
dht22_dat[3]) & 0xFF)) ) {
            float t, h;

            h = (float)dht22_dat[0] * 256 + (float)dht22_dat[1];
            h /= 10;
            t = (float)(dht22_dat[2] & 0x7F)* 256 + (float)dht22_dat[3];
            t /= 10.0;
            if ((dht22_dat[2] & 0x80) != 0)  t *= -1;
```

```
                ret_humid = (int)h;
                ret_temp = (int)t;
                printf("Humidity = %.2f %% Temperature = %.2f *C ₩n", h, t );
                printf("Humidity = %d Temperature = %d₩n", ret_humid, ret_temp);

     return ret_humid;
    }
    else
    {
                printf("Data not good, skip₩n");
                return 0;
    }
}

void act_waterpump_on()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (PUMP, OUTPUT);
    digitalWrite (PUMP, 1) ; // On

}
void act_waterpump_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (PUMP, OUTPUT);
    digitalWrite (PUMP, 0) ; // On

}
void act_fan_on()
{
```

```
    if(wiringPicheck()) printf("Fail");
    pinMode (FAN, OUTPUT);
    digitalWrite (FAN, 1) ; // On

}
void act_fan_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (FAN, OUTPUT);
    digitalWrite (FAN, 0) ; // On

}
void act_dcmotor_on()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (DCMOTOR, OUTPUT);
    digitalWrite(DCMOTOR, 1); //On
}

void act_dcmotor_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode (DCMOTOR, OUTPUT);
    digitalWrite(DCMOTOR, 0); //On
}

void act_rgbled_on()
{
    if(wiringPicheck()) printf("Fail");
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
```

```
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);

    digitalWrite(RGBLEDPOWER, 1); //PWR on
    digitalWrite(BLUE, 1); //PWR on
}
void act_rgbled_off()
{
    if(wiringPicheck()) printf("Fail");
    pinMode(RGBLEDPOWER, OUTPUT);
    pinMode(RED, OUTPUT);
    pinMode(GREEN, OUTPUT);
    pinMode(BLUE,OUTPUT);

    digitalWrite(RGBLEDPOWER, 0); //PWR on
    digitalWrite(BLUE, 0); //PWR on
}
```

입력을 마치면 ctrl-x 를 입력하면 저장할거냐고 묻는다. y 를 누르면 파일이름을 물어오는데 같은 파일이니 enter 를 입력하고 나온다.

아래와 같이 컴파일 한다.

```
$gcc -o farmstory farmstory.c -lwiringPi
```

실행을 하면

```
$sudo ./farmstory
```

조도센서의 출력과 dark ! , light ! 메시지와 현재 온도와 습도를 5 분 간격으로 디스플레이 한다. 조도센서에서 감지한 빛이 Dark ! 일경우 식물농장의 Led 를 파란색으로 On 하고 밝아지면 Off 하는 동작을 볼 수 있다.

```
if(get_light() == 0)
{
    act_rgbled_off();
    printf("light ! ₩n");
}
Else
{

    act_rgbled_on();
    printf("dark ! ₩n");
}
```

또한 기준 온도(현재 26 도)로 설정한 온도값이 넘어가는 경우 워터 펌프가 2 초간 작동을 하고

```
if(curtemp > 26)digitalWrite (PUMP, 1) ; // On
    {

            act_waterpump_on();
            delay(2000);
            act_waterpump_off();
    }

    curhumid = get_humidity_sensor();
```

현재 습도가 50% 이하인 경우 마찬가지로 팬이 2 초간 작동함을 볼 수 있다.

```
            if(curhumid < 50)
            {

                    act_fan_on();
```

```
                delay(2000);
                act_fan_off();
        }
```

위의 조건에 따라 액츄에이터의 제어는 라즈베리 파이가 동작하는 동안 계속 된다. 여기에서는 워터 펌프의 동작시간, 식물 Led 동작시간을 적은 시간동안 동작시켜 결과를 확인하였다. 사용자는 센서값의 기준을 얼마든지 줄이거나 확장할 수 있고, 액츄에이터의 제어 또한 사용자의 설계에 따라 바꿀 수 있다. 각 사용자가 원하는 의도대로 제어하고 동작하는 시험을 수행하기 바란다.

14.6 스마트 폰으로 인터넷 토이 액츄에이터 제어하기

스마트 폰으로 농장 액츄에이터를 제어하느 방법을 알아보자. 물론 연결상태는 현재까지 테스트 하던 상태 그대로 진행하면 되겠다.

(1) 하드웨어 연결도

앞장에서 센서 테스트, 액츄에이터 테스트 환경과 동일한 연결과 하드웨어를 사용한다.

(2) 안드로이드 단말기를 이용한 TCP/IP 프로그래밍

안드로이드 앱을 이요한 원격제어를 하기 위해서는 라즈베리 파이가 TCP/IP 의 서버가 되고, 안드로이드 단말기가 TCP/IP 의 클라이언트로서 프로그래밍 해야 한다. 일단 안드로이드 단말기는 클라이언트로서 서버인 라즈베리 파이에 접속한다. 그 후 안드로이드 단말기(클라이언트) 화면의 액츄에이터 제어 버튼을 클릭하면 정해진 문자열을 라즈베리파이(서버)로 전송하고, 라즈베리 파이는 그에 따른 처리를 수행하고 클라이언트에 결과를 전송하는 프로그램을 작성할 것이다.

즉, 서버 프로그래밍(라즈베리 파이), 클라이언트 프로그래밍(안드로이드 단말기)의 두 가지 작업이 필요하다. 안드로이드 단말기를 이용한 TCP/IP

프로그래밍 소스코드는 코드량이 많기 때문에 CD 의 스마트 폰 제어 프로그램 폴더의 코드를 참고하고 여기서는 대략적으로 코딩과정과 순서를 설명한다.

○ 프로젝트 생성

○ string.xml 에 각종 string 을 구성한다.

○ activit_main 을 편집하여 화면을 구성한다.

```
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:paddingBottom="@dimen/activity_vertical_margin"
    android:paddingLeft="@dimen/activity_horizontal_margin"
    android:paddingRight="@dimen/activity_horizontal_margin"
    android:paddingTop="@dimen/activity_vertical_margin"
    tools:context=".actu_p2rcontrol" >

    <ToggleButton
        android:id="@+ id/buttonRightTurn"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_above="@+ id/buttonDown"
        android:layout_toRightOf="@+ id/buttonUp"
        android:text="@string/Right"
        android:textOff="Fan Off"
        android:textOn="Fan On" />

    <ToggleButton
        android:id="@+ id/buttonLeftTurn"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonUp"
        android:layout_toLeftOf="@+ id/buttonDown"
```

```
        android:text="@string/Left"
        android:textOff="Motor Off"
        android:textOn="Motor On" />


    <ToggleButton
        android:id="@+ id/buttonUp"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/textViewStatus"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="37dp"
        android:text="@string/Up"
        android:textOff="펌프 Off"
        android:textOn="펌프 On" />

    <ToggleButton
        android:id="@+ id/buttonDown"
        android:layout_width="80dp"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonLeftTurn"
        android:layout_centerHorizontal="true"
        android:text="@string/Down"
        android:textOff="RGB Off"
        android:textOn="RGB On" />

    <Button
        android:id="@+ id/buttonConnect"
        android:layout_width="120dp"
        android:layout_height="wrap_content"
        android:layout_alignParentTop="true"
```

```
        android:layout_marginTop="66dp"
        android:layout_toLeftOf="@+ id/buttonUp"
        android:text="@string/Connect" />

    <TextView
        android:id="@+ id/textViewStatus"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_below="@+ id/buttonConnect"
        android:layout_centerHorizontal="true"
        android:layout_marginTop="20dp"
        android:text="@string/DataView"
        android:textSize="20sp" />

    <EditText
        android:id="@+ id/editTextIPAddress"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_above="@+ id/buttonClose"
        android:layout_centerHorizontal="true"
        android:layout_marginBottom="18dp"
        android:ems="10"
        android:gravity="center_vertical|center_horizontal"
        android:inputType="text"
        android:text="@string/IP" />

    <Button
        android:id="@+ id/buttonClose"
        android:layout_width="120dp"
        android:layout_height="wrap_content"
        android:layout_alignBaseline="@+ id/buttonConnect"
```

```
            android:layout_alignBottom="@+ id/buttonConnect"
            android:layout_toRightOf="@+ id/buttonUp"
            android:text="@string/Close" />

</RelativeLayout>
```

클라이언트로 동작하는 안드로이드 프로그램은 3 개의 소스 파일로 구성되었다. 데이터를 보여주는 Logger 와 데이터를 파싱하여 메인 화면을 제어해주는 파일, 그리고 데이처 수신 쓰레드이다. Logger 와 수신 쓰레드는 변경사항이 없으므로 소스를 참고하고 여기서는 액츄에이터를 제어하는 소스를 보여준다.

```
package huins.iot.actucontrol;

import android.os.Bundle;
import android.os.StrictMode;
import android.app.Activity;
import android.content.Context;
import android.graphics.Color;
import android.view.Menu;
import android.view.View;
import android.view.View.OnClickListener;
import android.view.inputmethod.InputMethodManager;
import android.widget.TextView;
import android.widget.Button;
import android.widget.EditText;
import android.widget.ToggleButton;

import java.net.Socket;
import java.io.IOException;
import java.io.OutputStream;

public class actu_p2rcontrol extends Activity implements OnClickListener{
```

```
    private EditText editTextIPAddress;
    private TextView textViewStatus;
    private Button buttonConnect;
    private Button buttonClose;
    private ToggleButton buttonUp;
    private ToggleButton buttonLeftTurn;
    private ToggleButton buttonRightTurn;
    private ToggleButton buttonDown;



    private InputMethodManager imm;
    private String server = "192.168.1.238";
    private int port = 8888;
    //private int port = 8080;
    private Socket socket;

    private OutputStream outs;
    private Thread rcvThread;
    public logger logger;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
            super.onCreate(savedInstanceState);
            setContentView(R.layout.activity_main);

            editTextIPAddress                                    =
(EditText)this.findViewById(R.id.editTextIPAddress);
            editTextIPAddress.setText(server);
            imm                                                  =
(InputMethodManager)getSystemService(Context.INPUT_METHOD_SERVICE);
```

```
                textViewStatus = (TextView)this.findViewById(R.id.textViewStatus);
                textViewStatus.setText("스마트팜 액츄에이터 테스트");

                logger = new logger(textViewStatus);

                buttonConnect    = (Button)this.findViewById(R.id.buttonConnect);
                buttonClose      = (Button)this.findViewById(R.id.buttonClose);
                buttonUp         = (ToggleButton)this.findViewById(R.id.buttonUp);
                buttonLeftTurn                                                    =
(ToggleButton)this.findViewById(R.id.buttonLeftTurn);
                buttonRightTurn                                                   =
(ToggleButton)this.findViewById(R.id.buttonRightTurn);
                buttonDown                                                        =
(ToggleButton)this.findViewById(R.id.buttonDown);

                buttonConnect.setOnClickListener(this);
                buttonClose.setOnClickListener(this);
                buttonUp.setOnClickListener(this);
                buttonLeftTurn.setOnClickListener(this);
                buttonRightTurn.setOnClickListener(this);
                buttonDown.setOnClickListener(this);

        }

        @Override
        public boolean onCreateOptionsMenu(Menu menu) {
                getMenuInflater().inflate(R.menu.main, menu);
                return true;
        }
        @Override
```

```
	public void onClick(View arg0) {

		StrictMode.ThreadPolicy policy = new StrictMode.ThreadPolicy.Builder()
		.detectAll()
		.penaltyLog()
		.build();
		StrictMode.setThreadPolicy(policy);

		if(arg0 == buttonConnect)
		{

	imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

				try{
					if(socket!=null)
					{
						socket.close();
						socket = null;
					}

					server = editTextIPAddress.getText().toString();
					socket = new Socket(server, port);
					outs = socket.getOutputStream();

					rcvThread = new Thread(new rcvthread(logger, socket));
				rcvThread.start();
					logger.log("Connected");
				} catch (IOException e){
					logger.log("Fail to connect");
```

```
                    e.printStackTrace();
                }
            }

            if(arg0 == buttonClose)
            {

imm.hideSoftInputFromWindow(editTextIPAddress.getWindowToken(), 0);

                if(socket!=null)
                {
                    exitFromRunLoop();
                    try{
                        socket.close();
                        socket = null;
                        logger.log("Closed!");
                        rcvThread = null;
                    } catch (IOException e){
                        logger.log("Fail to close");
                        e.printStackTrace();
                    }
                }
            }


            if(arg0 == buttonUp)
            {
                String sndOpkey = "CMD";

                if(buttonUp.isChecked()){
                    buttonUp.setTextColor(Color.RED);
```

```
                    sndOpkey = "Pumpon";
                }

                else{
                    buttonUp.setTextColor(Color.BLACK);
                    sndOpkey = "Pumpoff";
                }
                try{
                    outs.write(sndOpkey.getBytes("UTF-8"));
                    outs.flush();
                } catch (IOException e){
                    logger.log("Fail to send");
                    e.printStackTrace();
                }
            }

            if(arg0 == buttonLeftTurn)
            {
                String sndOpkey = "CMD";

                if(buttonLeftTurn.isChecked()){
                    buttonLeftTurn.setTextColor(Color.RED);
                    sndOpkey = "Motoron";
                }

                else{
                    buttonLeftTurn.setTextColor(Color.BLACK);
                    sndOpkey = "Motoroff";
                }
                try{
                    outs.write(sndOpkey.getBytes("UTF-8"));
```

```
                    outs.flush();
                } catch (IOException e){
                    logger.log("Fail to send");
                    e.printStackTrace();
                }
            }

            if(arg0 == buttonRightTurn)
            {
                String sndOpkey = "CMD";

                if(buttonRightTurn.isChecked()){
                    buttonRightTurn.setTextColor(Color.RED);
                    sndOpkey = "Fanon";
                }

                else{
                    buttonRightTurn.setTextColor(Color.BLACK);
                    sndOpkey = "Fanoff";
                }
                try{
                    outs.write(sndOpkey.getBytes("UTF-8"));
                    outs.flush();
                } catch (IOException e){
                    logger.log("Fail to send");
                    e.printStackTrace();
                }
            }

            if(arg0 == buttonDown)
            {
```

```
                String sndOpkey = "CMD";

                if(buttonDown.isChecked()){
                        buttonDown.setTextColor(Color.RED);
                        sndOpkey = "Rgbon";
                }

                else{
                        buttonDown.setTextColor(Color.BLACK);
                        sndOpkey = "Rgboff";
                }
                try{
                        outs.write(sndOpkey.getBytes("UTF-8"));
                        outs.flush();
                } catch (IOException e){
                        logger.log("Fail to send");
                        e.printStackTrace();
                }
        }

}

 void exitFromRunLoop(){
        try {
                String sndOpkey = "[close]";
                outs.write(sndOpkey.getBytes("UTF-8"));
                outs.flush();
        } catch (IOException e) {
                logger.log("Fail to send");
                e.printStackTrace();
        }
```

```
    }
}
```

(3) 라즈베리 파이 TCP/IP 서버 프로그래밍

다 언어를 지원하는 TCP/IP 는 언어의 제한을 받지 않는다. 여기서는 참고 코드를 쉽게 구할 수 있고, 많은 예제들이 있는 자바를 사용해서 프로그래밍 하자. 라즈베리 파이에서 자바를 사용하기 위해서는 하드웨어 연산을 지원하는 JDK8 의 설치가 필요하다. JDK8 의 설치는 복잡하지만 어렵지 않으므로 순차적으로 실습해보자.

JDK8 설치에 관해 영문이지만 설명이 잘 되어있다.

http://yoo7577.tistory.com/181

JDK8 을 다운로드 하기 위해서이곳을 방문하고 아래 그림과 같은 파일을 저장한다.

http://www.oracle.com/technetwork/java/javase/downloads/jdk8-arm-downloads-2187472.html

Java SE Development Kit 8 Update 6 for ARM

You must accept the Oracle Binary Code License Agreement for Java SE to download this software.

Thank you for accepting the Oracle Binary Code License Agreement for Java SE; you may now download this software.

Product / File Description	File Size	Download
Linux 32-bit ARM v6/v7 Hard Float ABI	83.81 MB	jdk-8u6-linux-arm-vfp-hflt.tar.gz

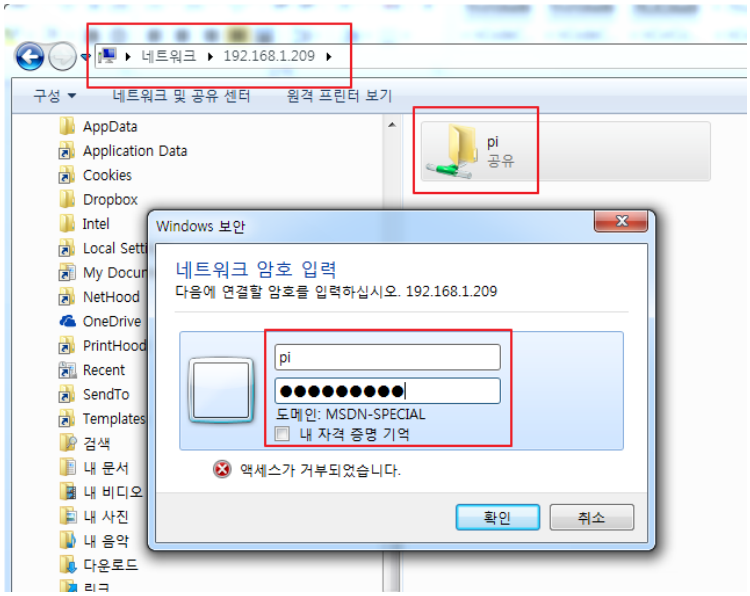
윈도우에서 다운 로드 한 위치를 기억한다.

아래 명령어로 라즈베리 파이의 주소를 확인하고 네트워크로 접속해준다.

```
pi@raspberrypi ~ $ ifconfig
eth0      Link encap:Ethernet  HWaddr b8:27:eb:0b:a7:4d
          inet addr:192.168.1.209  Bcast:192.168.1.255  Mask:255.255.255.0
```

```
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:80528 errors:0 dropped:0 overruns:0 frame:0
          TX packets:1942 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:5721058 (5.4 MiB)  TX bytes:216975 (211.8 KiB)
```

아이피 주소는 192.168.1.209 로 확인되었다.



네트워크 창을 열고 아이피 주소를 입력하고 /home/pi/ (작업 폴더임)에 다운 받은 파일을 옮기고 아래는 라즈베리 파이에서 작업한다.

아래와 같은 명령어로 /opt/java 라는 폴더를 생성하자.

$sudo mkdir -p -v /opt/java

현재 까지 하면 폴더구조는 아래와 같다.

```
pi@raspberrypi ~ $ cd ..
pi@raspberrypi /home $ cd ..
pi@raspberrypi / $ ls
bin  boot  dev  etc  home  lib  lost+found  media  mnt  opt  proc  root
run  sbin  selinux  srv  sys  tmp  usr  var
pi@raspberrypi / $
pi@raspberrypi / $ cd opt
```

```
pi@raspberrypi /opt $ ls
java  minecraft-pi  sonic-pi  vc  Wolfram
pi@raspberrypi /opt $ cd java
pi@raspberrypi /opt/java $ ls
pi@raspberrypi /opt/java $ cd ..
pi@raspberrypi /opt $ cd ..
pi@raspberrypi / $ cd home
pi@raspberrypi /home $ cd pi
pi@raspberrypi ~ $ pwd
/home/pi
pi@raspberrypi ~ $
```

다운로드한 폴더로 이동한다.

/opt/java 폴더에 다운로드한 파일을 압축해제 한다.

sudo tar xvzf ~/ jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java

```
pi@raspberrypi ~ $ sudo tar xvzf ~/jdk-8u6-linux-arm-vfp-hflt.gz -C /opt/java
jdk1.8.0_06/COPYRIGHT
jdk1.8.0_06/LICENSE
jdk1.8.0_06/README.html
jdk1.8.0_06/THIRDPARTYLICENSEREADME-JAVAFX.txt
jdk1.8.0_06/THIRDPARTYLICENSEREADME.txt
jdk1.8.0_06/bin/
jdk1.8.0_06/bin/javah
jdk1.8.0_06/bin/appletviewer
jdk1.8.0_06/bin/extcheck
jdk1.8.0_06/bin/jps
jdk1.8.0_06/bin/wsimport
jdk1.8.0_06/bin/unpack200
jdk1.8.0_06/bin/jarsigner
.
.
.
```

잠시 후 압축이 풀리고 Oracle JDK gz 파일들이 “/opt/java” 폴더의 아래에 “jdk1.8.0_06”폴더에 위치하게 된다.

다운로드 받은 파일을 더 이상 필요없기에 삭제한다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ rm ~/jdk-8u6-linux-arm-vfp-hflt.gz
pi@raspberrypi ~ $
```

시스템에게 새로운 JDK 가 인스톨 되었고, 위치를 알려주기 위해 아래와 같이 실행한다.

```
pi@raspberrypi ~ $ sudo update-alternatives --install "/usr/bin/java" "java" "/opt/java/jdk1.8.0_06/bin/java" 1
pi@raspberrypi ~ $
```

마지막으로 JDK 가 시스템의 디폴트 java 런타임이 될것이라고 알려준다.

```
pi@raspberrypi ~ $ sudo update-alternatives --set java /opt/java/jdk1.8.0_06/bin/java

pi@raspberrypi ~ $
```

이제 JDK8 이 설치되었다. 자바 버전명령을 사용해 아래와 같이 나온다면 잘 설치된 것이다.

```
pi@raspberrypi ~ $
pi@raspberrypi ~ $ java -version
java version "1.8.0_06"
Java(TM) SE Runtime Environment (build 1.8.0_06-b23)
Java HotSpot(TM) Client VM (build 25.6-b23, mixed mode)
pi@raspberrypi ~ $
```

진짜 마지막으로 JAVA_HOME 환경 변수들을 몇가지 더해야 한다. 우선 bashrc 선언문을 마지막에 추가해 준다.

```
sudo nano ~/.bashrc
```

```
export JAVA_HOME=/opt/jdk1.8.0
export PATH=$PATH:$JAVA_HOME/bin
```

를 마지막에 적어주고 ctrl-x 누르고 y 누르고 Enter 치고 빠져 나온다.

Reboot 를 수행한다. 부팅을 수행하고 나면 Java 샘플 프로그램 작성 및 동작 테스트를 실행하자.

소스코드 생성 및 작성 - nano 편집기를 이용하여 아래와 같은 샘플 코드를 작성하자.

```
$nano HelloWorld.java


public class HelloWorld
{
    public static void main(String args[])
    {
        java.lang.System.out.println("Hello World!!!");
    }
}
```

아래와 같이 컴파일을 수행한다.

```
$javac HelloWorld.java
```

실행한다. Classpath 지정을 잊지 말자.

```
$java -cp . HelloWorld


Hello World!!!
```

메시지가 보이면 설치와 테스트가 완료된 것이다.

여기까지 힘들게 실습하였다. 이제 라즈베리파이에서 JDK8 (with javaFX) 를 사용할 수 있게 되었다. 지금까지 라즈베리파이는 파이썬, C, C++, Java 등 정말

다양한 언어를 사용할 수 있게 되었는데~~ 이것은 단지 툴에 지나지 않다. 이를 이용하여 응용프로그램을 작성해 보도록 하자.

이제 부터는 쉬운 과정이다. 라즈베리파이의 GPIO 를 제어하기 위한 pi4j 라는 자바용 드라이브를 설치한다. pi4j 를 다운로드 하고 설치를 해준다. 아래는 명령어와 설치화면을 보여준다.

$curl -s get.pi4j.com | sudo bash

```
pi@raspberrypi ~ $ curl -s get.pi4j.com | sudo bash
=======================================================
INSTALLING Pi4J GPG PUBLIC KEY
=======================================================
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  1761  100  1761    0     0   3134      0 --:--:-- --:--:-- --:--:--  5889
OK
=======================================================
ADDING Pi4J APT REPOSITORY
=======================================================
--2015-01-17 14:46:04--  http://get.pi4j.com/pi4j.list
Resolving get.pi4j.com (get.pi4j.com)... 54.231.17.212
Connecting to get.pi4j.com (get.pi4j.com)|54.231.17.212|:80... connected.
HTTP request sent, awaiting response... 200 OK
Length: 42 [application/octet-stream]
Saving to: `/etc/apt/sources.list.d/pi4j.list'

100%[=======================================================>] 42          --.-K/s   in 0s
```

```
  2015-01-17  14:46:04  (547  KB/s)  -  `/etc/apt/sources.list.d/pi4j.list'  saved
[42/42]

  ================================================
  UPDATING APT REPOSITORIES
  ================================================
  Hit http://repository.pi4j.com wheezy Release.gpg
  Hit http://repository.pi4j.com wheezy Release
  Hit http://repository.pi4j.com wheezy/rpi armhf Packages
  Ign http://repository.pi4j.com wheezy/rpi Translation-en_GB
  Ign http://repository.pi4j.com wheezy/rpi Translation-en
  Reading package lists... Done
  ================================================
  INSTALLING Pi4J
  ================================================
  Reading package lists... Done
  Building dependency tree
  Reading state information... Done
  The following NEW packages will be installed:
    pi4j
  0 upgraded, 1 newly installed, 0 to remove and 0 not upgraded.
  Need to get 0 B/517 kB of archives.
  After this operation, 793 kB of additional disk space will be used.
  Selecting previously unselected package pi4j.
  (Reading database ... 87830 files and directories currently installed.)
  Unpacking pi4j (from .../pi4j_1.0~SNAPSHOT_all.deb) ...
  Setting up pi4j (1.0~SNAPSHOT) ...
  ================================================
  Pi4J INSTALLATION COMPLETE
  ================================================
```

```
The Pi4J JAR files are located at:
/opt/pi4j/lib

Example Java programs are located at:
/opt/pi4j/examples

You can compile the examples using this script:
sudo /opt/pi4j/examples/build

Please see http://www.pi4j.com for more information.

pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

위와 같이 설치하고 테스트를 진행할 때 에러나 나는 경우 언인스톨 명령어와 재설치를 아래에 나타낸다. 아래의 명령어로 언인스톨을 실행한다.

$ sudo apt-get remove pi4j

```
pi@raspberrypi ~ $ sudo apt-get remove pi4j
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be REMOVED:
  pi4j
0 upgraded, 0 newly installed, 1 to remove and 0 not upgraded.
1 not fully installed or removed.
After this operation, 793 kB disk space will be freed.
Do you want to continue [Y/n]? y
(Reading database ... 87881 files and directories currently installed.)
Removing pi4j ...
```

재설치

$ curl -s get.pi4j.com | sudo bash

안드로이드 스마트 폰(TCP/IP 클라이언트)과의 TCP/IP 통신을 위하여 쓰레드를 통해 데이터를 송수신 한다. 복수의 접속도 허용하고 있으며, 자원의 효율성도 고려하였다.

다음의 코드를 참고한다. farm_actu.java

```
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Logger;

public class Farmactu {
    private static final int sizeBuf = 50;
    public static void main(String[] args) throws IOException {

        int port = 8888;
        ServerSocket serverSock = new ServerSocket(port);
        Logger logger = Logger.getLogger("Farmactu");

        while (true) {
            Socket clientSock = serverSock.accept();
            SocketAddress clientAddress =
clientSock.getRemoteSocketAddress();
            Thread thread = new Thread(new RcvThread(clientSock,
clientAddress, logger));
            thread.start();
            if(clientSock.isConnected())
                System.out.println("Connected! Client IP : " + clientAddress);
        }
    }
```

TCP/IP 서버로 작동하는 소스코드다. 쓰레드로 동작하는 RcvThread.java 소스코드도 참고한다.

```
import java.io.IOException;
import java.io.InputStream;
import java.io.OutputStream;
import java.net.Socket;
import java.net.SocketAddress;
import java.util.logging.Level;
import java.util.logging.Logger;

import com.pi4j.io.gpio.GpioController;
import com.pi4j.io.gpio.GpioFactory;
import com.pi4j.io.gpio.GpioPinDigitalOutput;
import com.pi4j.io.gpio.PinState;
import com.pi4j.io.gpio.RaspiPin;
import com.pi4j.io.gpio.impl.PinImpl;   //this!!!!

public class RcvThread implements Runnable{
        private static final int sizeBuf = 50;
        private Socket clientSock;
        private Logger logger;
        private SocketAddress clientAddress;

        final GpioController gpio = GpioFactory.getInstance();

        final                GpioPinDigitalOutput                pinlpwm                =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_21, "lpwm", PinState.LOW);
        final                GpioPinDigitalOutput                pinlfwd                =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_23, "lfwd", PinState.LOW);
```

```
        final GpioPinDigitalOutput pinlrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_22, "lrev", PinState.LOW);

        final GpioPinDigitalOutput rgbpwr =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_24, "rpwm", PinState.LOW);

        final GpioPinDigitalOutput rgbblue =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_27, "rfwd", PinState.LOW);

        final GpioPinDigitalOutput pinrrev =
gpio.provisionDigitalOutputPin(RaspiPin.GPIO_03, "rrev", PinState.LOW);

        public RcvThread(Socket clntSock, SocketAddress clientAddress, Logger
logger) {
            this.clientSock = clntSock;
            this.logger = logger;
            this.clientAddress = clientAddress;
        }

        public void run(){
            try {
                InputStream ins     = clientSock.getInputStream();
                OutputStream outs = clientSock.getOutputStream();

                int rcvBufSize;
                byte[] rcvBuf = new byte[sizeBuf];
                while ((rcvBufSize = ins.read(rcvBuf)) != -1) {

                    String rcvData = new String(rcvBuf, 0, rcvBufSize, "UTF-8");

                                if (rcvData.compareTo("Pumpon") == 0) {
```

```
                pinlpwm.high();
                System.out.println("Pumpon");
        }
                        if (rcvData.compareTo("Pumpoff") == 0) {
                pinlpwm.low();
                System.out.println("Pumpoff!");
        }


        if (rcvData.compareTo("Motoron") == 0) {
                pinlfwd.high();
                                        System.out.println("Motoron");
                        }
                        if (rcvData.compareTo("Motoroff") == 0) {
                pinlfwd.low();
                System.out.println("Motoroff");
        }


                        if (rcvData.compareTo("Fanon") == 0) {
                pinlrev.high();
                                        System.out.println("Fanon");
                        }
                        if (rcvData.compareTo("Fanoff") == 0) {
                pinlrev.low();
                System.out.println("Fanoff");
        }
                        if (rcvData.compareTo("Rgbon") == 0) {
                rgbpwr.high();
                                        rgbblue.high();
                                        System.out.println("Rgbon");
                        }
```

```
                                    if (rcvData.compareTo("Rgboff") == 0) {
                        rgbpwr.low();
                                            rgbblue.low();
                        System.out.println("Rgboff");
                    }



                logger.info("Received data : " + rcvData + " (" + clientAddress +
")");
                    outs.write(rcvBuf, 0, rcvBufSize);
                }
                logger.info(clientSock.getRemoteSocketAddress() + " Closed");
            } catch (IOException ex) {
                logger.log(Level.WARNING, "Exception in RcvThread", ex);
            } finally {
                try {
                    clientSock.close();
                    System.out.println("Disconnected! Client IP : " + clientAddress);
                } catch (IOException e) {}
            }
        }
    }
```

아래와 같은 명령어로 컴파일을 수행한다.

```
$javac -classpath .:classes:/opt/pi4j/lib/'*' Farmactu.java
```

아래와 같은 명령어로 실행한다.

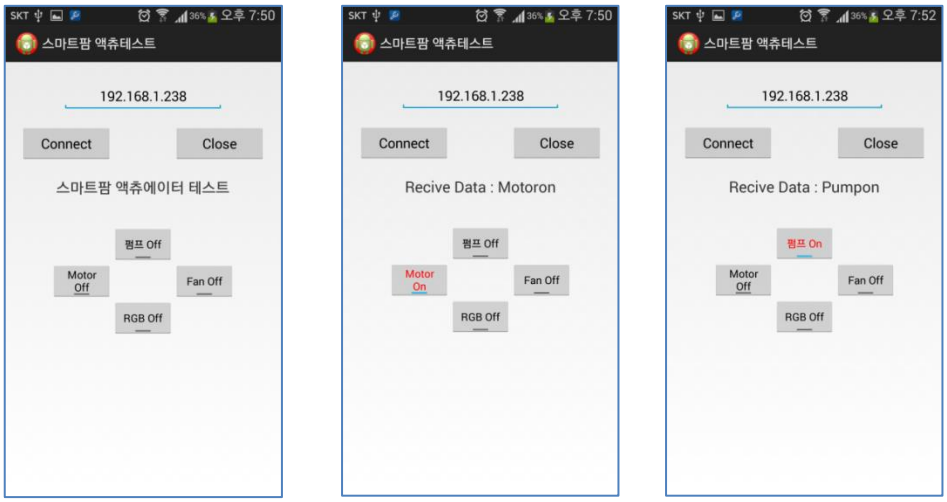
```
$sudo java -classpath .:classes:/opt/pi4j/lib/'*' Farmactu
```

라즈베리 파이 화면에서 보여지는 실행화면을 아래에 표시하였다

```
pi@raspberrypi ~/racar-test $ sudo java -classpath .:classes:/opt/pi4j/lib/'*'
Farmactu
```

```
Connected! Client IP : /192.168.1.98:37403
Pumpon
Jan 17, 2015 7:53:33 PM RcvThread run
INFO: Received data : Pumpon (/192.168.1.98:37403)
Pumpoff!
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Pumpoff (/192.168.1.98:37403)
Motoron
Jan 17, 2015 7:53:35 PM RcvThread run
INFO: Received data : Motoron (/192.168.1.98:37403)
Motoroff
Jan 17, 2015 7:53:36 PM RcvThread run
INFO: Received data : Motoroff (/192.168.1.98:37403)
Fanon
Jan 17, 2015 7:53:37 PM RcvThread run
INFO: Received data : Fanon (/192.168.1.98:37403)
Fanoff
Jan 17, 2015 7:53:39 PM RcvThread run
INFO: Received data : Fanoff (/192.168.1.98:37403)
Rgbon
Jan 17, 2015 7:53:39 PM RcvThread run
INFO: Received data : Rgbon (/192.168.1.98:37403)
Rgboff
Jan 17, 2015 7:53:40 PM RcvThread run
INFO: Received data : Rgboff (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: Received data : [close] (/192.168.1.98:37403)
Jan 17, 2015 7:53:51 PM RcvThread run
INFO: /192.168.1.98:37403 Closed
Disconnected! Client IP : /192.168.1.98:37403
```

아래는 안드로이드 스마트 폰으로 시험하는 화면을 보여준다.



14.7 인터넷 토이 서버 프로그램

스마트 농장 서버 프로그램은 Gateway 기능을 수행하는 라즈베리 파이에서 돌아가는 소프트웨어이다. 게이트웨이 서버 소프트웨어의 기능을 살펴보자.

- ○ 온습도 센서로부터 일정 시간마다 센서 데이터 수집
- ○ 조도 센서로부터 일정 시간 센서 데이터 수집
- ○ 수집된 센서 데이터는 시간 정보와 함께 데이터 베이스에 저장
- ○ 이미 설정된 환경 조건(물 부족, 야간)을 판단하여 액츄에이터 제어
- ○ 식물농장 상태를 카메라를 이용하여 동영상 스트리밍으로 전송
- ○ 원격지 사용자의 요청이 있을경우 액츄에이터 직접 제어

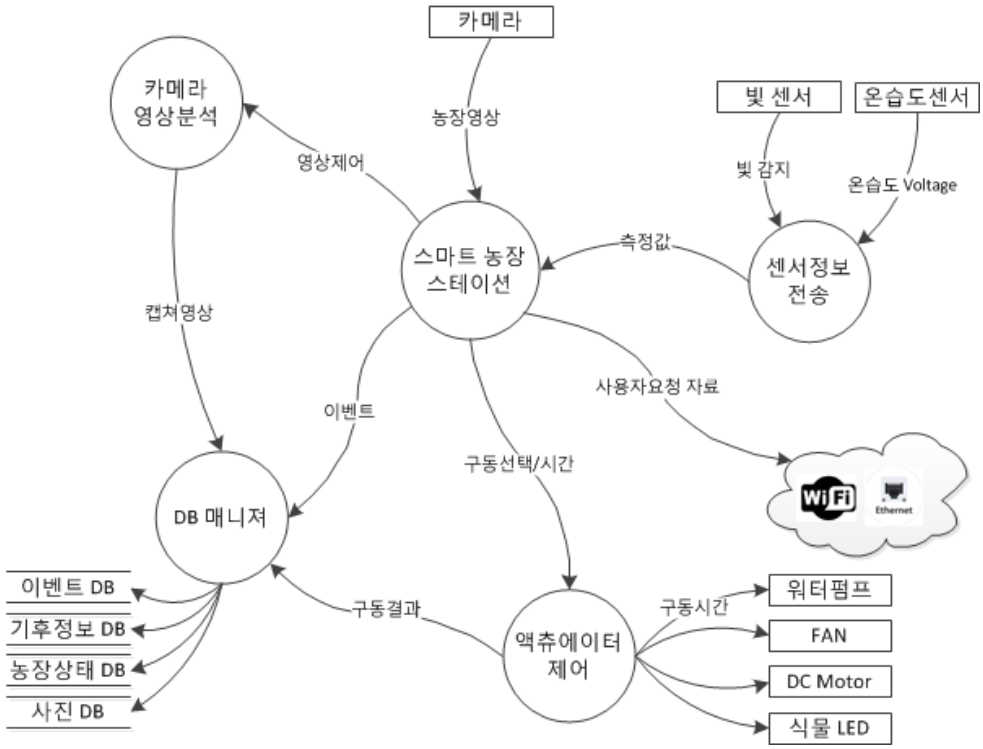


그림 185 서버 소프트웨어의 Data Flow Diagram

스마트 농장 서버 프로그램은 데이터를 파싱하고, 대기상태에서 사용자의 입력을 기다리게 된다. 사용자가 스마트 폰을 이용하여 현재 상태와 카메라 보기, 설정을 Setting 하는 기능을 통해 서버 프로그램은 해당 기능을 수행한다.

(1) 데이터 베이스 설계

일단 데이터 베이스에 대해 알아보자. 앞서 APM(Apache, PHP, MySQL)을 설치하였다. 그것을 바탕으로 데이터 베이스를 활요해보자. 여기서 사용하는 데이터 베이스는 스마트 농장의 모든 센서데이터와 이벤트 기록을 주기적으로, 아니면 발생시 데이터 베이스에 저장한다. 몇 개월이든 농장의 자료를 저장하는 것은 큰 의미가 있는 일이다. 지역의 기후와 농장의 계절 특성을 파악하는 것은 많은 도움이 될것이다. 데이터 베이스의 기초적인 부분들을 살펴보고 자세한 것은 Mysql 관련도서를 참조하자.

데이터 베이스의 설계

인터넷 토이 에서 데이터 베이스의 설계를 대략적으로 나타낸다. 데이터 베이스의 설계와 테이블 구성은 방대한 작업이므로 아래 실습에서 간단하게 실습하고, 실제 구현시에는 서버 코드를 참조하여 꼼꼼하게 구현한다.

DB Name : iottoy

Table : ttoysensorvalue 센서값 저장 테이블

1 분마다 센서값 저장한다.

날짜 day / 인체감지 int / 근접센서 int / 초음파 int / 소리 int

Table : ttoysensorcheck 센서 정상/불량 저장 테이블

센서 정상 불량 확인하여 저장 10 분마다.

날짜 day / 인체감지 (정상, 오류) / 근접센서 (정상, 오류) / 초음파 (정상, 오류) / 소리(정상, 오류)

Table : ttoyactuoperate 액츄에이터 동작상태(동작중/정지중)

(현재 이벤트 저장으로 사용중)

액츄 동작시 데이터베이스 업데이트 하고, 동작 끝날 때 업데이트

날짜 day / servomotor(on,stop) / ledbar(on, stop) / dcmotor(on, stop) / rgbled(on, stop)

Table : ttoyactucheck 액츄에이터가 정상/불량 저장 데이블

액츄에이터 정상 불량 기록 – 현재는 모두 정상

servomotor (normal, fail), ledbar (normal, fail), dcmotor (normal, fail), rgbled(normal, fail)

(2) 서버 실행과 카메라 영상

여기서는 스마트 농장 서버를 실행하고 CD 에 들어있는 smartfarm 앱을 실행시킨 상태에서 결과만을 확인하도록 한다.

```
pi@raspberrypi ~ $ cd smartfarm_server
pi@raspberrypi ~/smartfarm_server $
pi@raspberrypi ~/smartfarm_server $ sudo ./server
```

실행을 하고 스마트 폰에서 라즈베리 파이의 IP 주소를 연결한 후 확인하게 되면 다음과 같은 메시지를 보이면서 실행이 된다.

```
pi@raspberrypi ~/smartfarm_server $ sudo ./server
Handling client ip : 192.168.1.98
Handling client port : 56323
Handling client socket number : 5
connected user   : 1
[5Sock][MyUserNum:0]:[0|257X_▒X_]
message parsing start :0|257X_▒X_
 app type : 0x0000
 command : 0x0101
 data :
Data not good, skip
Data not good, skip
Humidity = 17.40 % Temperature = 24.40 *C
Humidity = 17 Temperature = 24
Temperature = 24
Data not good, skip
Humidity = 17.40 % Temperature = 24.40 *C
Humidity = 17 Temperature = 24
Humidity = 17
Smartfarm_server>server
```

카메라를 확인할 경우 서버에서 카메라 쉘 프로그램이 실행되고 있어야 한다. 따라서 현재 상태에서 Putty 프로그램을 하나 더 실행하여 루트에서 sh mjpg.sh 명령어를 실행한 상태로 실습을 진행한다.

```
pi@raspberrypi ~ $ sh mjpg.sh
```

```
  MJPG Streamer Version: svn rev:
   DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c,      input_init(),      116):
argv[0]=raspicam input plugin
   DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c,      input_init(),      116):
argv[1]=-d
   DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c,      input_init(),      116):
argv[2]=200
   DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
   i: delay.............: 200
   i: resolution........: 640 x 480
   i: camera parameters.............:
  .
  .
  .
```

위와 같이 실행 한 상태에서 스마트 폰의 '농장모습'을 클릭하면 농장의 동영상을 볼 수 있다.

14.8 인터넷 토이 안드로이드 앱 프로그램

여기서는 프로그램 구성을 설명하고 스마트 폰 앱을 실행하여 동영상을 확인하고 다양한 액츄에이터를 제어하는 실습을 진행한다. 소스코드의 구성은 사용자 스스로 분석하여 다양한 환경에 적용하도록 하자.

아래에 스마트 농장의 서버와 앱이 연동되는 테스트 과정을 설명한다. 이대로 시험하면 결과를 확인할 수 있고 나머지는 참고로 이용하길 바란다.

7. 라즈베리 파이 전원을 켜고, IO 어댑터 보드의 12V 전원을 켠다.
8. 로그인 아이디 : pi password : raspberry (화면에 나타나지 않음)
9. 라즈베리 파이 아이피 주소 : $ifconfig 명령어로 확인한다.
   예)192.168.1.219
10. 라즈베리 파이 스마트 팜 서버 실행

```
pi@raspberrypi ~ $ cd iot_toyserver
pi@raspberrypi ~/iot_toyserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_toyserver $ cd output
pi@raspberrypi ~/iot_toyserver/output $
pi@raspberrypi ~/iot_toyserver/output $ ls
mjpg.sh  server_toy
pi@raspberrypi ~/iot_toyserver/output $
pi@raspberrypi ~/iot_toyserver/output $ sudo ./servertoy &
```

(백그라운드 실행) 엔터 입력하여 터미널 화면에서

11. 동영상 스트리밍 서버 실행 : $sh mjpg.sh
12. 스마트 폰 앱 실행 pi@raspberrypi ~ $ **sh mjpg.sh**

이 과정을 통해 연동 테스트를 수행한다.

라즈베리 파이의 서버 프로그램 위치는 iot_farmserver 폴더이다.

서버 프로그램을 컴파일하기 위해 $make clean 명령을 사용하여 이전 컴파일시 생성된 파일을 지우는 명령이다.

```
pi@raspberrypi ~/iot_toyserver $ make clean
rm -rf /home/pi/iot_toyserver/output
Clearing in libs...
make[1]: Entering directory '/home/pi/iot_toyserver/libs'
rm -rf iot_db.o parser.o libiot.a core
make[1]: Leaving directory '/home/pi/iot_toyserver/libs'
Clearing in src...
make[1]: Entering directory '/home/pi/iot_toyserver/src'
Clearing in toy...
```

```
make[2]: Entering directory '/home/pi/iot_toyserver/src/toy'
rm -rf toy_main.o core
make[2]: Leaving directory '/home/pi/iot_toyserver/src/toy'
make[1]: Leaving directory '/home/pi/iot_toyserver/src'
pi@raspberrypi ~/iot_toyserver $
```

컴파일하기 위해 $make 명령을 실행한다.

```
pi@raspberrypi ~/iot_toyserver $ make
make all in libs...
make[1]: Entering directory '/home/pi/iot_toyserver/libs'
gcc -c -I/home/pi/iot_toyserver/include -I/usr/include/mysql iot_db.c -o iot_db.o
gcc -c -I/home/pi/iot_toyserver/include -I/usr/include/mysql parser.c -o parser.o
ar rcs libiot.a -o iot_db.o parser.o
make[1]: Leaving directory '/home/pi/iot_toyserver/libs'
make all in src...
make[1]: Entering directory '/home/pi/iot_toyserver/src'
make all in toy...
make[2]: Entering directory '/home/pi/iot_toyserver/src/toy'
gcc -g -c -I/home/pi/iot_toyserver/include -c toy_main.c -o toy_main.o
gcc -o server_toy toy_main.o /home/pi/iot_toyserver/libs/libiot.a -lmysqlclient -lpthread
make[2]: Leaving directory '/home/pi/iot_toyserver/src/toy'
make[1]: Leaving directory '/home/pi/iot_toyserver/src'
pi@raspberrypi ~/iot_toyserver $
```

에러 없이 실행되었다면 다음으로 실행파일을 출력 폴더에 생성하기 위해 make install 명령을 실행한다.

```
pi@raspberrypi ~/iot_toyserver $ make install
mkdir /home/pi/iot_toyserver/output
```

```
Installing in libs...
make[1]: Entering directory '/home/pi/iot_toyserver/libs'
make[1]: *** No rule to make target 'install'.  Stop.
make[1]: Leaving directory '/home/pi/iot_toyserver/libs'
Installing in src...
make[1]: Entering directory '/home/pi/iot_toyserver/src'
Installing in toy...
make[2]: Entering directory '/home/pi/iot_toyserver/src/toy'
cp -a server_toy /home/pi/iot_toyserver/output/
make[2]: Leaving directory '/home/pi/iot_toyserver/src/toy'
make[1]: Leaving directory '/home/pi/iot_toyserver/src'
pi@raspberrypi ~/iot_toyserver $
```

여기까지 실행하면 output 폴더에 서버 프로그램이 생성된다.

```
pi@raspberrypi ~ $ cd iot_toyserver
pi@raspberrypi ~/iot_toyserver $ ls
include  libs  Makefile  output  src
pi@raspberrypi ~/iot_toyserver $ cd output
pi@raspberrypi ~/iot_toyserver/output $
pi@raspberrypi ~/iot_toyserver/output $
pi@raspberrypi ~/iot_toyserver/output $ ls
mjpg.sh  server_toy
pi@raspberrypi ~/iot_toyserver/output $
```

서버프로그램을 실행한다.

```
$sudo ./server_toy
```

아래는 서버 프로그램을 실행시킨 화면이다. 서버는 온 습도, 조도 데이터를 수집하여 데이터 베이스에 저장하는 한편 클라이언트가 되는 스마트 폰에서 접속하기를 기다린다. 따라서 터미널 화면에서 서버를 수행시키면 다른

컴퓨터에서 putty 를 실행시켜 동영상 스트리밍을 서비스 하는 프로그램을 실행하는 방법으로 실습하여야 한다.

```
pi@raspberrypi ~/iot_toyserver/output $ sudo ./server_toy
validate database...
iot_send_query : query - CREATE TABLE IF NOT EXISTS ttoysensorvalue (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, human INT, proximity INT, ultra INT, sound INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS ttoysensorcheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, human INT, proximity INT, ultra INT, sound INT)
iot_send_query : query - CREATE TABLE IF NOT EXISTS ttoyactuoperate (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, servomotor INT, ledbar INT, dcmotor INT, rgbled INT)

iot_send_query : query - CREATE TABLE IF NOT EXISTS ttoyactucheck (id INT AUTO_INCREMENT PRIMARY KEY, date DATE, time TIME, servomotor INT, ledbar INT, dcmotor INT, rgbled INT)
done
starting server...
SensorInterruptLoop : write to DB - INSERT INTO  ttoysensorvalue values (null ,now(), now(), 1, 1, 1, 1)
iot_insert_data_from_db : query - INSERT INTO  ttoysensorvalue values (null ,now(), now(), 1, 1, 1, 1)
SensorInterruptLoop : write to DB - INSERT into ttoysensorcheck values (null, now(), now(), 1, 1, 1, 1)
iot_insert_data_from_db : query - INSERT into ttoysensorcheck values (null, now(), now(), 1, 1, 1, 1)
SensorInterruptLoop : write to DB - INSERT into ttoyactuoperate values (null, now(), now(), 0, 0, 0, 0)
iot_insert_data_from_db : query - INSERT into ttoyactuoperate values (null, now(), now(), 0, 0, 0, 0)
```

이상태에서 putty 를 실행시키고, 라즈베리 파이에 접속한다. Putty 를 실행하는 이유는 위에 서버를 실행하고 다시 동영상 스트리밍 서버를 실행하기 위함인데 위에 설명한 $ sudo ./server_farm & 명령(뒤에 &를 붙여서)으로 백그라운드로 실행하게 하면 putty 프로그램을 실행할 필요는 없다.

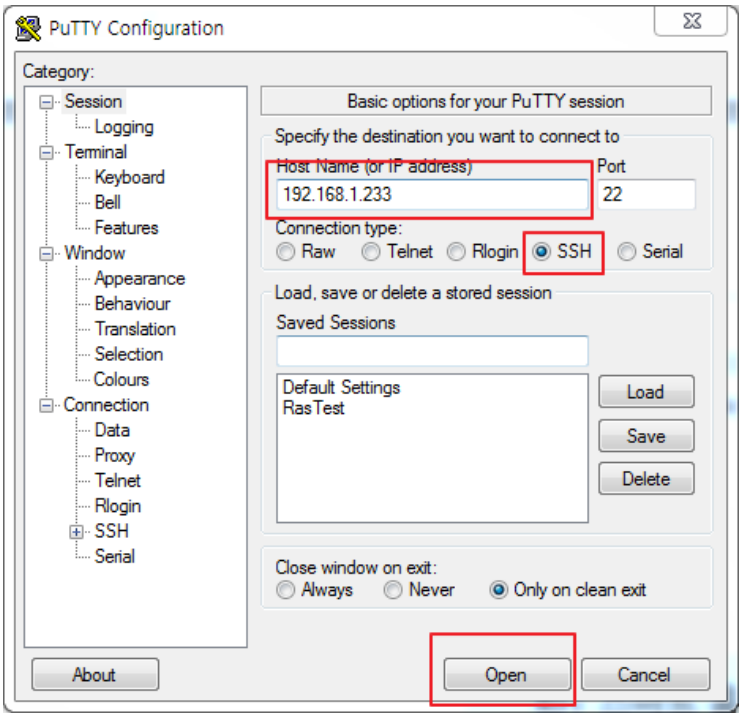


그림 186 동영상 스트림 서버를 실행시키기 위한 Putty 실행화면

아이디와 패스워드를 넣어 접속한 화면은 아래와 같다.

```
login as:
login as: pi
pi@192.168.1.219's password:
Linux raspberrypi 3.18.7+ #755 PREEMPT Thu Feb 12 17:14:31 GMT 2015
armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
```

```
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Feb 27 09:47:01 2015
pi@raspberrypi ~ $
pi@raspberrypi ~ $
```

홈 디렉토리에서 mjpg.sh 파일을 확인하고 다음의 명령어로 mjpg 스트림 프로그램을 실행한다.

$sh mjpg.sh

```
pi@raspberrypi ~ $ sh mjpg.sh
MJPG Streamer Version: svn rev:
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116):
argv[0]=raspicam input plugin
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116):
argv[1]=-d
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 116):
argv[2]=200
 DBG(/home/pi/mjpg-streamer/mjpg-streamer-
experimental/plugins/input_raspicam/input_raspicam.c, input_init(), 183): case 6, 7
 i: delay.....: 200
 i: resolution.....: 640 x 480
 i: camera parameters.....:
.
.
.
.
```

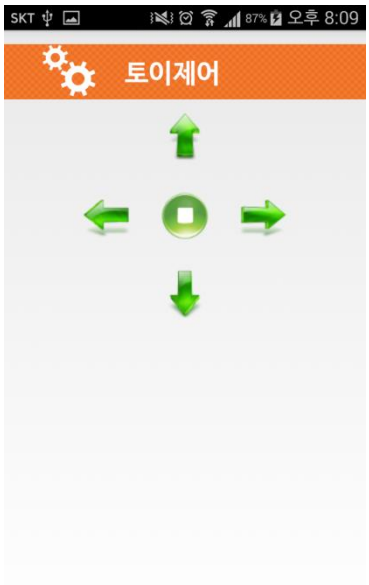
이 상태에서 모든 준비가 끝났다. 안드로이드 폰으로 다운 받은 인터넷 제어토이 앱을 실행시키면서 결과를 확인한다. 아래는 앱 실행 메인 화면이다.



인터넷 제어 토이의 메인화면 구성이다. 센서값 보기, 영상보기, 토이제어, 액츄에이터 제어로 메뉴가 구성된다. 디바이스 연결은 가장 먼저 실행되어야 할 메뉴로 라즈베리 파이 서버에 연결하는 기능을 한다.



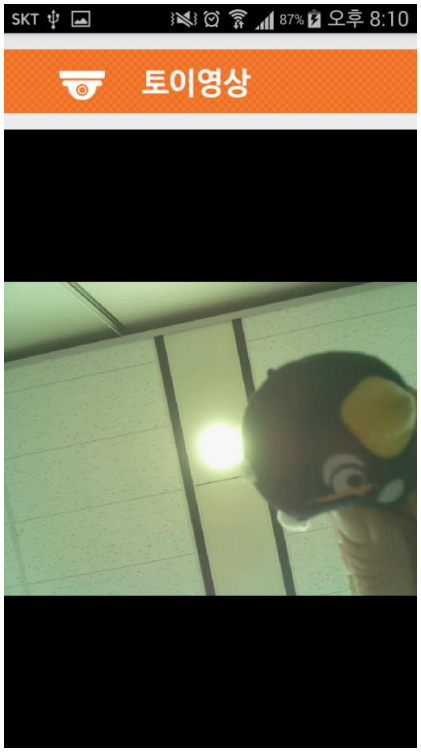
센서값 보기 메뉴화면이다. 인터넷 제어토이의 센서값을 확인하고, 액츄에이터 상태를 볼 수 있다. 센서와 액츄에이터에 이상 발생시 이상을 알려주는 기능도 가지고 있다.



토이제어 메뉴를 선택하면 보이는 화면이다. 토이제어 화면은 실제 토이라면 전 후진, 좌회전, 우회전의 기능을 수행하도록 설계 되었다. 여기서는 화면만을 보여주고 모터 회전에 이용할 수 있다. 실제 제어는 액츄에이터 제어에서 실행해야 한다.



액츄에이터 제어화면이다. 인터넷 제어토이의 액츄에이터를 ON, OFF 제어한다.



토이영상 메뉴화면이다. 현재 카메라가 설치된 토이에서 실시간으로 스트리밍 되는 영상을 보여준다.

Chapter 15 참고자료

Smart IoT 설계 시스템 사용자 설명서는 많은 자료와 인터넷 사이트를 참고하여 작성되었다. 그만큼 자료의 출처와 자료검토를 신중하게 숭행하였다. 소스코드는 검증하였고, 이미지나 글의 출처 또한 성실하게 표기하였다. 실수로 누락되었거나 빠뜨린 출처에 대해서는 꼭 보완하여 표기하도록 하겠다. IoT 관련분야나 연관 기술등에 대해 관심있는 사용자는 참고하길 바란다.

[1] (주)휴인스 : http://www.huins.com

[2] (주)휴인스 IoT Blog : smartiot.tistory.com

[3] 라즈베리 파이 활용백서 : 실전 프로젝트 20 - 이재상 표윤석 지음, 비제이 퍼블릭

[4] 라즈베리 파이 산딸기 마을 카뮤니티 http://www.rasplay.org/

[5] 리눅스와 함께하는 라즈베리 파이 - 피터 멤브리, 배장열 옮김, 제이펍

[6] ITU-T Internet Report “The Internet of Things – Executive Summary”, Nov. 2005.

[7] 민경식, Net Term, 인터넷 & 시큐리티 이슈, “사물 인터넷 (Internet of Things)”, 2012.

[8] 전자부품연구원(KETI), “IoT(Internet of Things) Presentation Design”, Nov. 2012.

[9] 이준섭, TTA 표준화 교육, 스마트 통신시장의 핵심열쇠 “M2M”, “M2M/IoT Standardization in ITU-T”, Oct. 23, 2012.

[10] ITU-T, ITU-T SG13, “Y.WoT, Framework of Web of Things”, Oct. 10, 2011.

[11] ITU-T, ITU-T SG13, “Y.2060, Overview of the Internet of Things”, July 04, 2012.

[12] IETF, Draft-lee-iot-problem-statement-05.txt

[13] JP Vasseur, IETF 77, Internet Area Meeting, “The Internet of Things”, March 2010.

[14] Internet of Things - Feb 6.2013, Yim Hyung-jun, Database Systems Lab. Dept. of Computer Engineering Chungnam National University

[15] 사물인터넷(IoT)분야의 주요 임베디드 SW 기술 - MDS 테크놀로지㈜ 장성균(2014/07/24)

[16] 사물인터넷 글로벌 생태계 구축 및 서비스 활성화를 위한 개방형 IoT 소프트웨어 플랫폼 기술 - 2013. 10. 25, 김재호 (jhkim@keti.re.kr), 전자부품연구원

[17] "라즈베리파이 공식 사이트", http://www.raspberrypi.org/

[18] "element14 의 레즈베리파이 커뮤니티", http://www.element14.com/community/groups/raspberry-pi

[19] "RASPBERRY PI INFORMATION AND VIDEOS", http://raspi.tv/

[20] "Raspberry Pi Spy", http://www.raspberrypi-spy.co.uk/

[21] "RaspiHub", http://www.raspihub.com/

[22] "Adafruit Blog", http://www.adafruit.com/blog/category/raspberry-pi/

[23] "Raspberry Pi Pod", http://www.recantha.co.uk/blog/

[24] "bcm2835", http://www.open.com.au/mikem/bcm2835/

[25] "Gordons Projects", https://projects.drogon.net/raspberry-pi/wiringpi/

[26]http://www.sainsmart.com/sainsmart-mifare-rc522-card-read-antenna-rf-rfid-reader-ic-card-proximity-module.html

[27]https://github.com/lthiery/SPI-Py

[28]https://github.com/mxgxw/MFRC522-python